

Exqutor 관련 References 81편 상세 정리

범위: References [1]~[81] 중 81편을 상세 분석한 문서이다. 각 논문의 문제의식, 핵심 기여, 시스템 구조, 실험 결과, 본 논문과의 관계를 정리하였다.

(A) VAQ 동기 부여 — 왜 VAQ가 필요한가

[1] AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data 총정리

저자: Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, Yuanzhe Cai (Alibaba Group) **학회/년도:** VLDB 2020 (Proceedings of the VLDB Endowment, Vol. 13, No. 12) **분량:** 약 14페이지 **역할군:** (A) VAQ 개념 원조; 본 논문 문제 정의의 출발점

요약

AnalyticDB-V는 Exqutor가 풀려는 문제, 즉 "벡터 검색과 SQL 분석 쿼리를 하나의 시스템 안에서 효율적으로 처리하는 것"을 최초로 산업 규모에서 구현한 시스템이다. 현대 데이터 파이프라인에서 구조화 데이터(테이블)와 비구조화 데이터(이미지, 텍스트 등)가 함께 존재하는데, 기존에는 이 두 종류를 별개의 시스템으로 처리해야 했다. AnalyticDB-V의 핵심 제안은 **벡터 유사도 검색을 SQL의 물리적 연산자로 구현**하는 것으로, 사용자가 하나의 SQL 문으로 벡터 검색과 관계형 분석을 동시에 표현할 수 있도록 한다.

시스템은 Lambda 아키텍처를 채택하여 스트리밍 레이어와 배치 레이어를 분리하고, VGPQ(Vector Graph Product Quantization)라는 새로운 ANN 검색 알고리즘을 제안한다. 가장 중요한 기여는 **정확도 인식 비용 기반 최적화(Accuracy-Aware Cost-Based Optimization)**로, ANN 검색의 본질적 근사성을 옵티마이저에 통합하여 정확도 요구사항과 비용 간의 트레이드오프를 관리한다. 다만 **카디널리티 추정** 문제를 깊이 다루지 않았으며, 이것이 Exqutor가 보완하는 핵심 빈틈이다.

본 논문과의 관계: VAQ(Vector-Augmented Analytical Query) 개념의 원형을 제시했으나, "벡터 검색 결과가 정확히 몇 건이나 나올지 추정하는 문제"는 미해결로 남겨두어, Exqutor의 근본적 동기를 제공한다.

상세분석

1.1 해결하고자 하는 문제

현대 데이터 파이프라인에서는 **구조화 데이터**(매출, 재고, 사용자 정보 등 숫자/텍스트 테이블)와 **비구조화 데이터**(이미지, 동영상, 오디오, 텍스트 문서 등)가 함께 생성된다.

기존에는 이 두 종류의 데이터를 별개의 시스템으로 처리했다: - 구조화 데이터 → 관계형 데이터베이스 (MySQL, PostgreSQL 등) - 비구조화 데이터 → 별도의 벡터 검색 엔진 (Faiss 등)

문제는 실제 비즈니스에서 두 데이터를 **함께** 분석해야 하는 경우가 매우 많다는 것이다. 예를 들어: - "이 상품 이미지와 비슷한 상품을 찾되, 가격이 100만원 이하이고 재고가 있는 것만 보여줘" - "이 얼굴 사진과 유사한 사용자를 찾아서, 그 사용자의 최근 구매 이력을 분석해줘"

이런 쿼리를 처리하려면 개발자가 두 시스템 사이에서 수동으로 데이터를 옮기고 결합해야 했다. 이 과정이 매우 복잡하고 느렸으며, 최적화 기회를 완전히 놓치게 된다.

1.2 핵심 아이디어: SQL로 하이브리드 쿼리 표현

AnalyticDB-V의 핵심 제안은 **벡터 유사도 검색을 SQL의 물리적 연산자(Physical Operator)로 구현**하는 것이다. 사용자는 하나의 SQL 문으로 벡터 검색과 관계형 분석을 동시에 표현할 수 있다.

```
SELECT product_name, price, similarity_score
FROM products
WHERE category = 'electronics'
      AND ANNS(image_embedding, query_vector, 10) -- 벡터 유사도 top-10
ORDER BY similarity_score DESC;
```

이렇게 하면 데이터베이스 옵티마이저가 벡터 검색과 필터링의 순서, 방식 등을 자동으로 결정해준다. 개발자는 "무엇을" 원하는지만 선언하면, 시스템이 "어떻게" 효율적으로 실행할지를 담당한다.

1.3 시스템 구조

AnalyticDB-V는 **Lambda 아키텍처**를 채택한다:

스트리밍 레이어(Streaming Layer): - 실시간으로 들어오는 벡터 데이터를 처리한다. - 새로 삽입된 벡터는 즉시 검색 가능하다. - 인메모리 임시 인덱스를 사용한다. - 예시; 새 상품 이미지가 업로드되는 순간, 즉시 유사 상품 검색에 포함된다.

배치 레이어(Batching Layer): - 주기적으로 새로 삽입된 벡터들을 압축하고, ANNS 인덱스를 재구축한다. - 대규모 데이터에 대해 최적화된 영구 인덱스를 관리한다. - 야간이나 유휴 시간대에 큰 배치를 처리하여 비용을 절약한다.

쿼리 처리 레이어: - 스트리밍 레이어와 배치 레이어의 결과를 병합하여 최종 결과를 반환한다. - 스트리밍 레이어에서 찾은 최근 벡터와 배치 레이어의 오래된 벡터를 함께 고려한다.

1.4 VG PQ (Vector Graph Product Quantization) 알고리즘

AnalyticDB-V의 핵심 기술적 기여 중 하나는 **VG PQ**라는 새로운 ANN 검색 알고리즘이다.

기존의 **IVF-PQ** (Inverted File with Product Quantization) 방식은: 1. 벡터 공간을 여러 클러스터(셀)로 나누고 2. 검색할 때 가까운 클러스터 몇 개만 방문하여 후보를 찾는다.

VG PQ는 여기에 **그래프 구조**를 추가한다: 1. 앵커(anchor) 중심점을 찾고 2. 그래프를 탐색하며 관련 서브셀(subcell)들을 효율적으로 선택하고 3. 후보 벡터들의 거리를 계산하여 결과를 반환한다.

이 방식은 대규모 벡터(수억~수십억 건)에서 기존 IVF-PQ보다 검색 정확도와 속도를 모두 향상시킨다. 특히 고차원 벡터(수백~수천 차원)에서 클러스터링 품질이 향상된다.

1.5 정확도 인식 비용 기반 최적화 (Accuracy-Aware Cost-Based Optimization)

가장 중요한 기여는 **ANNS 연산자를 데이터베이스 옵티마이저에 통합**한 것이다.

일반적인 데이터베이스 옵티마이저는 각 연산의 비용(I/O, CPU 등)을 추정하여 최적의 실행 계획을 선택한다. AnalyticDB-V는 여기에 **정확도(accuracy)**라는 새로운 차원을 추가한다.

ANN 검색은 본질적으로 근사적(approximate)이기 때문에, 파라미터 설정에 따라 정확도가 달라진다: - 파라미터를 높이면 → 정확도 높아지지만 비용(시간) 증가 - 예; 탐색할 클러스터 개수를 10개에서 100개로 늘리면, 정확도는 99%에서 99.9%로 개선되지만 시간은 2배 증가 - 파라미터를 낮추면 → 비용 줄지만 정확도 저하 - 예; 탐색할 클러스터 개수를 3개로 줄이면, 시간은 80% 단축되지만 정확도는 95%로 저하

옵티마이저는 사용자가 요구하는 정확도 수준을 만족하면서, 가장 비용이 적은 실행 계획을 선택한다. 만약 사용자가 "95% 정확도로 충분해"라고 명시하면, 시스템은 더 빠른 근사 알고리즘을 선택할 수 있다.

1.6 관계형 쿼리와 통합 메커니즘

ANNS 연산자를 관계형 대수에 통합하면서 중요한 문제들을 해결해야 한다:

카디널리티 추정의 어려움: 벡터 검색이 정확히 몇 건의 결과를 반환할 것인지를 미리 알기 어렵다. 예를 들어: - "가격 < 1000"인 상품을 찾으면 정확히 523개가 나온다 (카디널리티 = 523) - 하지만 "이 이미지와 유사한 상품"은 몇 개가 나올까? 이는 임계값 설정에 따라 100개~10,000개까지 가능하다.

최적화 기회의 발굴: 정확한 카디널리티가 있으면: - "벡터 검색을 먼저 할지, 가격 필터를 먼저 할지" 순서를 최적화할 수 있다. - "중간 결과가 작으면 Nested Loop Join, 크면 Hash Join"을 선택할 수 있다.

1.7 성능 평가

AnalyticDB-V는 아래와 같은 환경에서 평가되었다: - 벡터 데이터; 수천만 개의 상품 이미지 임베딩(512차원) - 구조화 데이터; 상품 메타데이터(가격, 카테고리, 재고 등) - 쿼리 유형; VAQ(벡터 + 필터 + 집계 혼합)

결과: - 순수 벡터 검색만으로는 최대 50배 성능 향상 - 벡터 + 관계형 쿼리 결합 시 최대 100배 향상 - 다양한 정확도 요구사항(90%~99.9%)에 대응 가능

1.8 본 논문과의 관계

AnalyticDB-V는 "벡터 검색을 SQL에 통합하자"는 아이디어를 제시했지만, **카디널리티 추정** 문제를 깊이 다루지 않았다. 즉, 벡터 검색 결과가 몇 건이나 나올지를 정확히 예측하는 문제는 남아있었다.

AnalyticDB-V에서의 가정: - 벡터 유사도 범위 검색(distance < D)의 결과는 **고정 비율**(예; 10%)로 추정 - 이 고정 가정은 실제 데이터 분포가 다를 때 매우 부정확함

Exqutor는 바로 이 빈틈을 메우는 연구이다: - AnalyticDB-V의 "고정 선택도" 가정을 버림 - **실제 인덱스를 탐색**하여 정확한 카디널리티를 얻음 - 이를 통해 옵티마이저가 더 나은 실행 계획을 세우도록 함

1.9 정확한 카디널리티의 가치; 구체적 예시

AnalyticDB-V의 "고정 10% 선택도" 가정이 얼마나 부정확할 수 있는지 보자:

```
쿼리: 상품 이미지 유사도 검색 + 가격 필터
SELECT product_id, price
FROM products
WHERE image_similarity(embedding, query_img) < 0.5
      AND price < 100000
```

AnalyticDB-V 옵티마이저의 추정:

- 총 상품: 1,000,000개
- 유사도 필터: 10% 선택도 가정 → 100,000개
- 가격 필터: 30% 선택도 (알려짐) → 300,000개
- 최종 결과 추정: $100,000 \times 0.3 = 30,000$ 개

하지만 실제 데이터:

- 유사도 필터 실제 결과: 0.5% (5,000개)
→ 고정 가정과 오류: 5배 차이!
- 최종 실제 결과: $5,000 \times 0.3 = 1,500$ 개
→ 추정 오류: 20배!

최적화 영향:

- 30,000건 기준으로 계획 선택 (해시 조인 선택)
- 1,500건이 나올 줄은 몰라서 비효율적 계획 선택
- 추가 비용: 리소스 낭비, 응답시간 증가

추가 제기 문제

1. **일반화의 어려움:** AnalyticDB-V는 Alibaba의 e-commerce 환경에 특화되어 설계되었다. 일반적인 OLAP 환경에서는:
2. 벡터 크기가 더 클 수 있고 (수천 차원)
3. 다양한 벡터 필드가 여러 개 존재하며

4. 복잡한 조인 구조가 필요할 수 있다

이러한 일반화된 환경에서 AnalyticDB-V의 비용 모델이 얼마나 잘 작동하는지는 명확하지 않다. 특히 **복잡한 다중 조인 쿼리**에서 벡터 연산의 위치를 어디에 배치할지 결정하는 문제는 더욱 복잡해진다.

1. **실행 계획의 적응성:** 고정된 정확도 임계값(예; 95%)으로 시스템을 설정하면, 쿼리가 달라졌을 때도 같은 임계값을 사용한다. 하지만 어떤 쿼리는 99% 정확도가 필요하고, 다른 쿼리는 90%로도 충분할 수 있다. 이런 동적 조정 메커니즘이 필요하지 않을까?
2. **벡터 인덱스 구축 비용:** VGPQ 알고리즘은 검색 성능은 개선하지만, 인덱스 구축 비용도 고려해야 한다. 데이터가 계속 추가되는 환경에서 배치 레이어의 인덱스 재구축이 너무 자주 발생하면 시스템 부하가 될 수 있다.

[2] Hybrid RAG-Empowered Multi-Modal LLM for Secure Data Management in Internet of Medical Things 총정리

저자: Chao Su, Jiawen Wen, Jiaqi Kang, Yang Zhang, Jiankang Ren, Ying He, Mianxiong Dong **학회/년도:** IEEE Internet of Things Journal, 2024 **분량:** 약 15페이지 **역할군:** (A) VAQ 동기 부여; 실세계 응용 사례

요약

이 논문은 Exqutor가 해결하려는 VAQ(Vector-Augmented Analytical Query)의 **실세계 응용 사례**를 보여준다. IoMT(Internet of Medical Things) 환경에서 멀티모달 LLM(Large Language Model)과 RAG(Retrieval-Augmented Generation)를 결합한 하이브리드 시스템으로, 의료 AI가 단순 텍스트 검색을 넘어 다양한 도메인으로 확장되고 있음을 보여주는 중요한 동기 부여 논문이다.

핵심 문제는 의료 IoT 환경에서 X-ray 이미지, 심전도 시계열, 텍스트 진료 기록 같은 멀티모달 데이터를 통합 분석해야 한다는 것이다. 논문은 세 가지 계층으로 구성된 시스템을 제안한다: (1) 멀티모달 하이브리드 RAG로 각 모달리티별 후보를 검색하고 교차 검증, (2) 블록체인 기반 안전한 데이터 공유, (3) Aol(Age of Information) 메트릭으로 데이터 신선도를 보장하는 인센티브 메커니즘. 특히 첫 번째 계층의 RAG 파이프라인이 바로 **복합 VAQ의 실체**이며, 정확한 카디널리티 추정이 없을 때 얼마나 비효율적인지를 보여준다.

본 논문과의 관계; VAQ의 실제 구현에서 벡터 유사도 검색과 메타데이터 필터링, 다중 모달리티 교차 검증이 결합될 때, 정확한 카디널리티 추정이 얼마나 중요한지를 실제 의료 응용에서 입증한다.

상세분석

2.1 해결하고자 하는 문제: 멀티모달 의료 데이터의 통합 분석

IoMT(Internet of Medical Things) 환경에서는 병원·웨어러블 기기·원격 진료 시스템이 대량의 **멀티모달 의료 데이터**를 생성한다: - X-ray, CT, MRI 같은 **이미지 데이터** (2D/3D) - 심전도(ECG), 맥박 센서 같은 **시계열 데이터** (시간 의존적) - 의사 노트, 처방전, 임상 검사 결과 같은 **텍스트 데이터** (비정형) - 나이, 성별, 기저질환 같은 **구조화된 메타데이터** (정형)

이 데이터를 기반으로 LLM이 정확한 의료 보조 판단을 내리려면, **검색 증강 생성(RAG)** 파이프라인이 필수적이다. 하지만 기존 RAG 시스템에는 세 가지 핵심 문제가 있다:

문제 1 — 데이터 신선도(Freshness): 의료 데이터는 실시간 업데이트되지만, RAG에 제공되는 검색 결과가 오래된 데이터일 경우 LLM의 판단이 부정확해진다. 예를 들어: - 환자의 가장 최근 혈액 검사 결과(오늘)가 아닌 3개월 전 결과를 참조하면 - 잘못된 진단으로 이어질 수 있다 (예; 수치가 개선되었는데 악화된 것으로 판단)

문제 2 — 보안과 신뢰: 의료 데이터는 극도로 민감하다. HIPAA, GDPR 같은 규제를 만족하면서도: - 여러 병원이 데이터를 공유할 때, 중앙 기관 없이 어떻게 신뢰를 확보할 것인가? - 데이터 무결성을 어떻게 보장할 것인가? (누군가가 데이터를 위조하지 않았는가?) - 암호화된 데이터에서 검색은 어떻게 수행할 것인가?

문제 3 — 인센티브 부재: 데이터 보유자(병원, 기기 제조사)가 고품질 최신 데이터를 적극적으로 제공할 동기가 없다: - 데이터 공유에 비용이 든다 (인프라 비용, 프라이버시 리스크) - 하지만 보상이 없다 (비용-편익 불균형) - 따라서 오래되고 품질 낮은 데이터만 제공하려는 유인이 생긴다

2.2 핵심 아이디어: 하이브리드 RAG 프레임워크

논문이 제안하는 시스템은 **3개 계층**으로 구성된다:

계층 1 — 멀티모달 하이브리드 RAG:

유니모달 검색 (Unimodal Search): - **텍스트→텍스트:** 진료 기록에서 비슷한 증상/진단 사례 검색 (BERT 임베딩 + 코사인 유사도) - **이미지→이미지:** X-ray 이미지에서 유사한 방사선학적 소견 검색 (CNN 특징 추출 + HNSW) - **시계열→시계열:** ECG 신호에서 비슷한 패턴 검색 (DTW 거리 또는 시계열 임베딩)

각 모달리티에 맞는 임베딩 모델과 거리 함수를 사용하여 후보를 먼저 검색한다.

멀티모달 메트릭 교차 필터링:

X-ray 임베딩(쿼리)

- 유사 X-ray 이미지 상위 100개 검색
- 각 이미지의 환자 ID 추출
- 해당 환자의 텍스트 진료 기록 조회
 - "폐암 의심" 텍스트가 있는지 확인
 - 일치하는 것만 최종 결과로 반환

예시: - X-ray로 폐에 음영이 보이는 환자를 찾으되 - 진료 기록에 실제로 "호흡곤란" 증상이 기록된 환자만 반환 - 이렇게 하면 거짓양성(이미지만 유사하지만 증상이 다른 경우)을 줄일 수 있다

이것이 바로 VAQ의 실체이다: 벡터 유사도 검색(이미지 임베딩 매칭) + 관계형 필터(환자 메타데이터, 날짜 범위 등) + 다른 모달리티와의 교차검증을 **동시에** 수행해야 한다.

계층 2 — 계층적 크로스체인 아키텍처:

블록체인 기반 안전한 데이터 공유: - 중앙 기관(예: 의료청) 없이도 데이터 출처를 검증할 수 있다 - 메인체인: 전체 네트워크의 합의를 관리 (병원 A, B, C가 모두 인정하는 거래) - 사이드체인: 각 기관의 로컬 데이터를 처리 (병원 A의 환자 데이터는 병원 A의 사이드체인에서만 관리)

장점: - 중앙 통제 없음 (한 병원이 데이터를 조작해도 다른 병원은 다른 복사본을 가짐) - 감사 추적(audit trail) 자동 생성 (누가 언제 어떤 데이터를 조회했는가를 블록체인에 기록)

계층 3 — AoI 기반 인센티브 메커니즘:

Age of Information (AoI) 메트릭: 데이터가 얼마나 오래되었는가를 정량화한다.

$$AoI = (\text{현재시각}) - (\text{마지막 업데이트시각})$$

예시: - 환자의 혈액 검사가 오늘 업데이트됨 → $AoI = 1$ 시간 (최신) - 3개월 전에만 업데이트됨 → $AoI = 90$ 일 (오래됨)

계약 이론(Contract Theory) 기반 인센티브: - 데이터 보유자에게 보상을 제공한다: 데이터 신선도 AoI 가 낮을수록 (데이터가 신선할수록) 더 많은 보상 - 다양한 계약 옵션 제공: 고신선도·고비용 vs. 저신선도·저비용 - 각 병원이 자신의 상황에 맞는 계약을 선택할 수 있음

확산 기반 심층 강화학습(Diffusion-based DRL): - 기존의 경제학적 인센티브 설계는 정적이지만, 현실의 의료 시장은 동적이다 - 시간에 따라 데이터 가치가 변한다 (질병이 유행하면 해당 데이터의 가치 상승) - DRL이 이 동적 변화에 실시간으로 대응하여 최적 계약을 도출한다

2.3 멀티모달 RAG 파이프라인의 구체적 예시

```
-- 의료 시스템에서의 RAG 검색
SELECT patient.name, xray.image_url, diagnosis.text
FROM patients
JOIN xray_images ON patients.id = xray_images.patient_id
JOIN diagnoses ON patients.id = diagnoses.patient_id
WHERE
  -- 벡터 유사도: 쿼리 이미지와 유사한 X-ray 찾기
  vector_distance(xray.embedding, query_image_embedding) < 0.3

  -- 메타데이터 필터: 동일 나이대, 동일 성별
  AND patients.age BETWEEN 40 AND 60
  AND patients.gender = 'M'

  -- 텍스트 유사도: 진료 기록이 증상과 일관성 있는가
  AND text_similarity(diagnosis.text, 'shortness of breath') > 0.7

  -- 데이터 신선도: 최근 6개월 내 기록
  AND diagnosis.created_at > NOW() - INTERVAL '6 months'

ORDER BY vector_distance DESC
LIMIT 5;
```

이 쿼리에서: - **벡터 연산** (image embedding similarity) - **메타데이터 필터** (age, gender) - **다른 벡터 연산** (text similarity) - **시간 필터** (데이터 신선도) - **조인** (3개 테이블)

모두가 결합되어 있으며, 정확한 카디널리티가 없으면: - 벡터 검색을 먼저 할지 메타데이터 필터를 먼저 할지 결정 불가 - 중간 결과가 1,000건인지 100,000건인지 알 수 없어 조인 방식 선택 불가 - 최악의 경우 응답 시간이 100 배 이상 느려질 수 있음

2.4 성능 고려사항

멀티모달 검색의 계산 비용: - X-ray CNN 임베딩: 프레임당 500ms (GPU 사용 시 50ms) - 텍스트 임베딩 (BERT): 1,000 토큰당 100ms - 유사도 계산: 백만 벡터 × 1,000 차원 = 1초 (SIMD 최적화 시)

시스템이 매초 100건의 쿼리를 처리해야 한다면: - 비효율적 실행 계획: 1쿼리 당 5초 → 처리 불가능 - 최적 실행 계획: 1쿼리 당 500ms → 처리 가능

정확한 카디널리티의 가치: 각 필터 단계에서 결과를 얼마나 줄일 수 있는지 알면: - 벡터 유사도로 1,000,000건 → 10,000건 (1% 선택도) - 나이/성별 필터로 10,000건 → 3,000건 (30% 선택도) - 텍스트 유사도로 3,000건 → 150건 (5% 선택도)

이를 알면 **가장 선택도가 높은 필터를 먼저 적용**하여 중간 결과를 최소화할 수 있다.

2.5 본 논문과의 관계

이 논문의 RAG 파이프라인은 "벡터 유사도 검색 → 메타데이터 필터링 → 다중 모달리티 교차 검증"이라는 **복잡한 쿼리 체인**을 요구한다. 이것이 SQL로 표현되면 여러 테이블 조인과 벡터 범위 검색이 결합된 VAQ가 된다.

Exqutor의 정확한 카디널리티 추정이 없으면: - 옵티마이저가 비효율적인 실행 계획을 선택 - 응답 시간이 수십 배 느려짐 - 실시간 의료 지원 시스템으로서 기능 불가능

특히 의료 환경에서는 **실시간 응답이 치명적으로 중요**하므로, Exqutor 스타일의 최적화가 실질적 가치를 갖는다.

추가 제기 문제

1. **블록체인 오버헤드**: 매 검색마다 블록체인에 접근하면 레이턴시가 증가한다. 읽기 전용 쿼리에서는 블록체인 확인을 생략할 수 있을까?
2. **멀티모달 메트릭의 카디널리티 변동**: 이미지→텍스트 교차 필터링 과정에서 선택도가 극심하게 변할 수 있다. 정적 통계로는 이를 예측하기 어렵다 → Exqutor의 동적 샘플링이 특히 유용할 시나리오.
3. **모달리티 간 정렬 문제**: 서로 다른 모달리티의 유사도 점수(이미지 거리 0.3, 텍스트 유사도 0.7)를 어떻게 결합할 것인가? 정규화 방식에 따라 최종 순위가 크게 변한다.
4. **Aoi 인센티브의 현실성**: 이론적으로는 데이터 신선도에 비례한 보상이 최적이지만, 실제 병원들이 이를 따를 동기가 있을까? (기존 관례, 정치적 고려 등)

[3] Tree-Based RAG-Agent Recommendation System; A Case Study in Medical Test Data 총정리

저자: Yaofeng Yang, Chufan Huang **학회/년도**: arXiv:2501.02727, 2025 **분량**: 약 10페이지 **역할군**: (A) VAQ 동기 부여; 계층적 쿼리 패턴의 실제 사례

요약

이 논문은 RAG 기반 시스템이 의료 도메인에서 **계층적 추론**과 결합될 때 어떤 복잡한 쿼리 패턴이 필요한지를 보여준다. 단순 top-k 벡터 검색으로는 해결할 수 없는, **다단계 필터링과 집계**가 필요한 실세계 시나리오의 중요한 사례 연구이다.

의료 검사 추천 시스템의 핵심은 "환자의 증상을 보고, 어떤 진료과에 보낼지, 그리고 어떤 검사를 처방할 것인가"라는 결정이다. 이것은 단순하지 않은 **계층적 추론**을 요구한다. HiRMed (Hierarchical RAG-enhanced Medical Test Recommendation) 시스템은 트리 구조로 이를 구현하는데, 각 노드는 전문화된 RAG 에이전트이고, 상위 계층부터 하위 계층으로 점진적으로 검색 공간을 좁혀나간다.

이 구조를 SQL VAQ로 표현하면, 각 트리 레벨에서 카디널리티가 크게 달라지는 **다단계 조인과 벡터 범위 검색의 결합**이 된다. 정확한 카디널리티 추정이 없으면, 상위 레벨의 부정확한 카디널리티가 하위 레벨로 전파되어 최악의 경우 지수적 성능 저하를 초래한다.

본 논문과의 관계; 의료 도메인에서 트리 구조의 각 노드별로 서로 다른 카디널리티 특성을 갖는 VAQ가 발생하며, 이것이 정확한 카디널리티 추정의 필요성을 강하게 보여준다.

상세분석

3.1 해결하고자 하는 문제: 계층적 의료 추론의 필요성

의료 검사 추천 시스템에서 "환자의 증상을 보고, 어떤 진료과에 보내고, 어떤 검사를 처방할 것인가"라는 결정은 결코 단순하지 않은 **계층적 추론**을 요구한다.

현재 시스템의 문제

기존 LLM의 한계:

일반 목적(General-Purpose) LLM에 "환자가 심한 두통을 호소한다"고 입력하면:

GPT-4의 응답:

"신경학적 검사(neurological exam)와 함께,
필요시 MRI나 CT 스캔을 추천합니다.
또한 안과 검진도 고려하세요."

문제점: 1. **과도한 추천**: 모든 가능성을 다루려고 하여 불필요한 검사까지 권장 (비용 증가) 2. **진료과 혼동**: 신경과, 안과, 이비인후과 중 어디가 1차 진료과인지 불명확 3. **일관성 부족**: 같은 환자에 대해 시간이 지나면 다른 추천을 할 수 있음 (의료 오류 유발)

RAG를 추가해도 해결 안 되는 문제:

벡터 유사도 검색만으로는: - "두통 케이스" 벡터를 검색해서 가장 유사한 100개 사례를 찾은 후 - LLM에 "이 100개 사례에서 공통 검사는 뭐야?"라고 질문

하지만 이 방식의 문제: - 두통은 수십 가지 원인이 있는데 (뇌종양, 편두통, 안경 필요, 귀 감염 등) - 모든 사례를 함께 고려하면 결과가 너무 다양해짐 - 특정 원인(예; 뇌종양 의심)에 특화된 검사 프로토콜을 세울 수 없음

3.2 핵심 아이디어: HiRMed (Hierarchical RAG-enhanced Medical Test Recommendation)

3계층 트리 구조

Root (증상 입력)

- |
- |— Department A (신경과)
 - | |
 - | |— Test 1 (MRI; 뇌종양 선별)
 - | |— Test 2 (CT; 급성 출혈 선별)
 - | |— Test 3 (뇌파검사; 간질 선별)
- |
- |— Department B (안과)
 - | |
 - | |— Test 4 (안저촬영; 망막 검사)
 - | |— Test 5 (시야검사; 녹내장 선별)
- |
- |— Department C (이비인후과)
 - | |
 - | |— Test 6 (중이염 검사)
 - | |— Test 7 (부비강염 검사)

각 노드는 **전문화된 RAG 에이전트**이며, 자신의 범위에 해당하는 지식 베이스에서만 검색·증강·생성을 수행한다.

예시: "두통" 증상 입력

Step 1 (Root 노드):

	두통의 원인 분류	
	- 신경과적 (45%)	
	- 안과적 (20%)	
	- 이비인후과적 (30%)	
	- 기타 (5%)	

↓

Step 2 (신경과 노드):

	신경과 차원에서의 추가 질문	
	- 갑작스러운 발병? (뇌졸중 의심)	
	- 반복적? (편두통 의심)	
	- 국소적 신경 증상? (종양 의심)	

↓

Step 3 (신경과 → MRI 노드):

	뇌종양 선별 규약	
	- MRI 기본 (모든 환자)	
	- 조영제 확산 (강제)	
	- 함수적 MRI (기능 손상 시)	

상위 레이어 (Department Knowledge Base):

- 증상 → 진료과 매핑 규칙
- 두통 + 시력 변화 → 안과 우선
 - 두통 + 이명 → 이비인후과 우선
 - 두통 + 마비 증상 → 신경과 우선

하위 레이어 (Test Knowledge Base):

- 진료과별 검사 상세 정보
[신경과 - MRI]
- 적응증: 뇌종양, 뇌졸중, 경련성 질환
 - 금기사항: 금속 임플란트 (페이스메이커 등)
 - 비용: 300,000원
 - 소요시간: 40분
 - 진단 가능성: 뇌종양 95%, 편두통 10%

메모리 증강 추론

이전 진단 이력을 메모리에 유지하여:

- 같은 환자 (ID: P12345)
- 1차 방문: 두통 → 신경과 의뢰 → MRI 처방
 - 2차 방문: 지속적 두통 → 신경과 (이미 방문함) → 추가 검사 결정
 - 메모리에서 "이 환자는 신경과 경험 있음"을 참조
 - 동일 검사 반복 회피 가능

3.3 성능 결과

의료 검사 추천에 대한 평가:

평가 지표	단일 LLM	HiRMed	개선도
커버리지 (관련 검사를 빠뜨리지 않음)	78.9%	92.3%	+13.4%
정확도 (추천한 검사가 실제로 적절)	71.2%	88.7%	+17.5%
비용 효율성 (불필요한 검사 회피율)	45%	78%	+33%
일관성 (같은 환자의 반복 방문 시 동일 추천)	62%	94%	+32%

3.4 트리 구조의 카디널리티 특성

Root 레벨의 카디널리티

"두통" 증상의 환자 풀:

전체 환자 DB: 1,000,000명
두통으로 내원한 환자: 50,000명 (선택도 5%)

진료과별 분류:

- 신경과 (진료과 A): 22,500명 (45%)
- 안과 (진료과 B): 10,000명 (20%)
- 이비인후과 (진료과 C): 15,000명 (30%)
- 기타 (진료과 D): 2,500명 (5%)

검사 노드의 카디널리티

신경과 내에서 더 세밀한 분류:

신경과 환자 22,500명

MRI 추천:

- 급성 신경 증상: 5,000명 → MRI 필수 (100% 선택도)
- 만성 두통 (3개월 이상): 8,000명 → MRI 선택적 (60% 선택도)
- 편두통 (빈번하지 않음): 9,500명 → MRI 불필요 (5% 선택도)
- 전체 신경과 환자 중 MRI 추천율: $(5000 + 4800 + 475) / 22500 = 28\%$

3.5 VAQ로의 변환

HiRMed의 검사 추천 로직을 SQL로 표현하면:

```

-- 1단계: 증상 기반 진료과 후보 검색
SELECT dept_id, dept_name, matching_score
FROM departments
WHERE vector_distance(dept.symptom_profile, query_symptom_embedding) < 0.5
ORDER BY matching_score DESC;

-- 2단계: 각 진료과 내 구체적 검사 추천
SELECT test_id, test_name, recommendation_score
FROM medical_tests
WHERE
  -- 벡터 유사도: 환자 증상과 검사의 적응증 일치
  vector_distance(test.indication_embedding, query_symptom_embedding) < 0.4

  -- 메타데이터 필터: 환자 프로필과의 호환성
  AND patient_age BETWEEN test.recommended_age_min AND test.recommended_age_max
  AND test.contraindication NOT IN (patient_medical_history)

  -- 진료과 필터: 1단계에서 선정된 진료과만
  AND test.department_id IN (SELECT dept_id FROM step1_result)

  -- 비용 제약: 보험 적용 검사만
  AND test.insurance_covered = TRUE

ORDER BY recommendation_score DESC;

-- 3단계: 환자 이력 고려 (메모리 증강)
SELECT test_id
FROM step2_result
WHERE
  -- 같은 환자가 최근 12개월 내 받은 검사 제외
  test_id NOT IN (
    SELECT test_id FROM patient_history
    WHERE patient_id = :patient_id
    AND test_date > NOW() - INTERVAL '12 months'
  );

```

이 SQL 쿼리에서: - **1단계**: 벡터 검색 결과 카디널리티 = 3~5 진료과 - **2단계**: 각 진료과별 검사 후보 = 수십~수백 개 검사 - **3단계**: 최종 필터링 후 = 3~10개 검사

카디널리티의 급격한 변화: - 1단계 결과 3 → 2단계 중간 결과 300 → 3단계 최종 8 - 이런 급격한 변화에는 정적 통계가 무용지물

3.6 정확한 카디널리티의 가치

카디널리티 추정 오류의 영향

시나리오 A: 카디널리티 과소 추정

실제 카디널리티: 500개 검사
추정 카디널리티: 10개 검사 (추정 오류)

옵티마이저의 판단:

- "1단계 결과를 먼저 조인하고, 그 다음 벡터 검색"
- 실행 계획: JOIN (3개 진료과) JOIN FILTER (vector search)
- 예상 비용: 낮음 (3 × 처리 비용)

실제 실행:

- 3개 진료과 조인으로 22,500건 나옴
- 각 22,500건에 대해 벡터 검색 (비싼 연산) 수행
- 실제 비용: 매우 높음 (22,500 × 벡터 검색 비용)

시나리오 B: 카디널리티 과다 추정

실제 카디널리티: 50개 검사
추정 카디널리티: 500개 검사 (추정 오류)

옵티마이저의 판단:

- "벡터 검색을 먼저 수행, 그 결과만 조인"
- 실행 계획: VECTOR_SEARCH (500개 후보) → JOIN 필터
- 예상 비용: 높음 (500 × 처리 비용)

실제 실행:

- 벡터 검색 수행 (실제로는 50개만 해당)
- 하지만 이미 500개 후보 검색 비용이 발생함
- 불필요한 계산 450개 분 낭비

정확한 카디널리티로 얻는 이득

각 단계에서: - 1단계 결과가 3개 → 조인 비용 최소화 - 2단계 중간 결과가 300개 → 해시 조인 선택 (Nested Loop 피함) - 3단계 필터가 8개 → Sort 생략, Limit 최적화

전체 성능: - 부정확한 계획: 응답시간 10~20초 (비용 초과) - 정확한 계획: 응답시간 1~2초 (10배 이상 개선)

3.7 본 논문과의 관계

HiRMed의 각 노드에서 수행하는 RAG 검색은:

- 벡터 유사도 (증상 임베딩 매칭)
- + 메타데이터 필터 (진료과 코드, 연령, 금기사항 등)
- + 집계 (우선순위 랭킹)
- + 과거 이력 제외

을 결합하는 **복합 VAQ**이다.

이 과정이 SQL로 표현되면 **다단계 조인과 벡터 범위 검색**의 결합이 되며, 특히: - 트리의 각 레벨에서 카디널리티가 **크게 달라짐** (Root 50,000 → 검사 500 → 최종 8) - 상위 레벨의 카디널리티 오류가 하위 레벨로 전파됨 (지수적 성능 저하) - 정적 통계로는 이 변동성을 포착하기 어려움

따라서 Exqutor의 **동적 카디널리티 추정**이 특히 유용한 시나리오이다.

추가 제기 문제

1. **트리 구조의 고정성:**
2. 현재 진료과 분류는 고정되어 있다.
3. 새로운 진료과(예; AI 기반 정신건강 상담)가 추가되면 트리 재구성 필요
4. 동적으로 트리를 조정하는 메커니즘이 필요하지 않을까?
5. **노드 간 쿼리 최적화 부재:**
6. 각 노드의 RAG 검색이 독립적으로 수행됨
7. 예; Root에서 찾은 진료과 정보를 하위 노드에서 재활용할 수 있는데 활용 안 함
8. 노드 간 중간 결과 공유 (캐싱)로 성능 개선 가능 → Exqutor의 아이디어와 유사
9. **멀티모달 의료 데이터 미지원:**
10. 현재는 텍스트 증상만 고려
11. X-ray 이미지나 혈액 검사 수치 같은 구조화 데이터를 함께 고려하면?
12. 더 정교한 추천이 가능하지만 복잡도 급증 → 카디널리티 추정 더욱 어려워짐
13. **피드백 루프 부재:**
14. 추천한 검사가 실제로 효과적이었는가를 시스템이 학습하지 않음
15. 시간에 따라 진료 가이드라인이 변하는데 (의료 지식 업데이트)
16. 시스템이 자동으로 적응하는 메커니즘이 필요
17. Exqutor의 런타임 적응적 샘플링이 이 문제의 해답이 될 수 있을까?

[4] Accelerating Machine Learning Inference with Probabilistic Predicates 총정리

저자: Yao Lu, Aakanksha Chowdhery, Srikanth Kandula, Surajit Chaudhuri (Microsoft Research) **학회/년도:** SIGMOD 2018 (ACM International Conference on Management of Data) **분량:** 약 16페이지 **역할군:** (D) 이론적 기반; ML+DB 최적화의 다른 각도

요약

이 논문은 "비구조화 데이터에 대한 ML 추론 쿼리에서 비싼 UDF(User-Defined Function)를 어떻게 최적화할 것인가"라는 문제를 다룬다. Exqutor와는 다른 각도에서 ML+DB 최적화를 접근하지만, 두 논문 모두 **"ML/벡터 연산을 관계형 쿼리 옵티마이저에 어떻게 통합할 것인가"**라는 큰 질문을 공유한다.

핵심 아이디어는 비싼 ML 모델(딥러닝 기반 객체 탐지, 얼굴 인식 등) 이전에 **경량 이진 분류기를 "조기 필터"로 삽입**하는 것이다. 이를 통해 불필요한 데이터를 미리 걸러내고 비싼 ML 추론의 횟수를 크게 줄인다. 여러 필터를 계단식으로 연결(cascade)하면 각 단계에서 점진적으로 데이터가 정제되어 전체 비용을 극적으로 감소시킨다.

Exqutor가 벡터 검색의 **카디널리티**를 정확히 추정하여 옵티마이저를 돕는 것과 유사하게, 이 논문은 ML 추론의 **비용과 선택도**를 옵티마이저에 통합한다. 두 논문의 공통 메시지는 **"ML 연산의 특성을 옵티마이저에 정확히 반영하면 극적인 성능 향상이 가능하다"**는 것이다.

본 논문과의 관계; 비싼 연산(벡터 검색 vs. ML 추론)의 특성을 옵티마이저에 통합하는 서로 다른 접근법을 보여주며, 두 방식의 결합 가능성을 시사한다.

상세분석

4.1 해결하고자 하는 문제

비디오 분석, 이미지 검색 같은 ML 추론 쿼리에서 핵심 병목은 **비싼 UDF(User-Defined Function)**이다.

문제 상황의 구체적 예시

```
SELECT * FROM video_frames
WHERE object_detector(frame, 'red SUV') = TRUE -- 매우 비쌘!
AND timestamp BETWEEN '2024-01-01' AND '2024-01-31';
```

여기서 `object_detector()` 는 딥러닝 모델로: - ResNet 또는 YOLO 같은 대규모 신경망 - 한 프레임당 수십~수백 ms가 소요됨 (GPU 사용 기준) - 100만 프레임 비디오라면; $100만 \times 100ms = 100,000초 = \text{약 } 28\text{시간}$ 소요

그런데 실제로 "빨간 SUV"가 나오는 프레임은 전체의 **0.1%도 안 될 수 있다** (예; 차가 10,000개 장면 중 10장에만 나타남).

따라서 불필요한 추론이 많아서, 이를 줄일 수 있다면 극적인 성능 개선이 가능하다.

기존 최적화의 한계

기존 쿼리 최적화의 **술어 푸시다운(Predicate Pushdown)** 기법:

[100만 프레임]

- ↓ [시간 필터 적용] (timestamp 조건)
- ↓ [31만 프레임] (1월 데이터만)
- ↓ [ML 추론 UDF 실행]
- ↓ [300 프레임] (빨간 SUV 있는 것)
- ↓ [결과 반환]

이것만으로도 도움이 되지만, 여전히 31만 프레임마다 비싼 DL 모델 실행해야 한다. 추가로: - 31만 × 100ms = 31,000초 = 약 8.6시간 (여전히 너무 김)

원근법을 바꿔서, ML UDF 이전에 경량 필터를 먼저 삽입할 수 있을까?

4.2 핵심 아이디어: 확률적 술어 (Probabilistic Predicates, PPs)

핵심 통찰: 비싼 ML 추론을 수행하기 전에 경량 이진 분류기를 조기 필터로 사용하면, 대부분의 음성 사례(false cases)를 미리 제거할 수 있다.

오프라인 학습 단계

[Step 1] 비싼 UDF를 소수의 데이터에 실행하여 레이블 수집

- └ 전체 1,000,000 프레임 중
- └ 샘플 1,000 프레임만 선택하여
- └ YOLO 객체 탐지 모델 실행
- └ 레이블 수집 (Red SUV 있음/없음)

[Step 2] 저비용 특징 추출

- └ 색상 히스토그램 (각 RGB 채널의 분포; 512B)
- └ 텍스처 특징 (Edge detection; 256B)
- └ 저해상도 미리보기 (32×32 픽셀; 3KB)
- └ 전체 특징 크기 ~4KB (원본 1080p 프레임 2MB 대비 500배 소)

[Step 3] 경량 이진 분류기 학습

- └ 입력: 1,000개 프레임의 저비용 특징
- └ 출력: Red SUV 있음/없음 (이진)
- └ 알고리즘: 로지스틱 회귀 (선형 모델)
- └ 학습 비용: 1초 미만
- └ 정확도: 95% (비싼 YOLO의 정확도도 97%)

온라인 실행 단계

각 프레임에 대해:

[단계 1] 저비용 특징 추출 (4KB)

↓ (비용: 1ms)

[단계 2] 경량 분류기 실행

└ 99% 확률로 "Red SUV 없음" 판정?

| ↓ [프레임 스킵; 비싼 YOLO 실행 안 함]

| ↓ (절약된 비용: 100ms)

└ 우호적 점수? 계속 진행

↓ (누적 비용: 1ms)

[단계 3] 비싼 YOLO 객체 탐지 실행

↓ (비용: 100ms)

[단계 4] 최종 결과

극적인 비용 절감: - 100만 프레임 중 실제 Red SUV: 1,000개 (0.1%) - 경량 필터로 음성 예측 정확도 99%: 990,000개 프레임 제외 - 실제 비싼 YOLO 실행 대상: 10,000개 (9,900개 거짓양성 + 1,000개 실제 양성) - 비용 절감: $(100만 - 10,000) \times 100ms = 999,000 \times 100ms = \mathbf{99,900초}$ 절감

4.3 Viola 스타일 캐스케이드 (Cascade of Classifiers)

경량 필터 하나만으로는 부족할 수 있다. 대신 **여러 개를 계단식으로 연결**한다:

[입력 프레임]

↓

[PP_1: 초경량 필터 (비용 1ms)]

└ 색상 기반 (빨간색 있는가?)

└ 매우 빠름

└ 정확도는 낮음 (false negative 10%)

└ 90% 제외

↓ [10만 프레임]

[PP_2: 경량 필터 (비용 5ms)]

└ 색상 + 모양 (직사각형 같은 물체 있는가?)

└ 중간 속도

└ 정확도 중간 (false negative 5%)

└ 80% 추가 제외

↓ [2만 프레임]

[PP_3: 중간 필터 (비용 30ms)]

└ 에지 감지 + 패턴 매칭 (차량 같은 형태?)

└ 느린 편

└ 정확도 높음 (false negative 2%)

└ 90% 추가 제외

↓ [2,000 프레임]

[비싼 YOLO (비용 100ms)]

└ 정확한 객체 탐지

└ 최종 결과

캐스케이드의 이점: 1. 각 단계에서 비용이 점진적으로 증가 (1 → 5 → 30 → 100ms) 2. 각 단계에서 데이터가 걸러짐 (100% → 10% → 2% → 0.2%) 3. 전체 비용 최소화

4.4 비용-정확도 트레이드오프 모델링

PP의 정확도-성능 트레이드오프를 비용 기반 옵티마이저에 통합한다:

거짓 음성률 (False Negative Rate, FNR)

- 정의: "실제로 Red SUV가 있는데, 필터가 없다고 판단"
- 영향: 최종 결과에서 Red SUV 놓침 (정확도 저하)
- 예; FNR = 5%면 100개 Red SUV 중 5개를 못 찾음

거짓 양성률 (False Positive Rate, FPR)

- 정의: "실제로 Red SUV가 없는데, 필터가 있다고 판단"
- 영향: 불필요한 비싼 YOLO 실행 (성능 저하)
- 예; FPR = 10%면 999,000개 음성 중 99,900개가 거짓양성으로 판정되어 비싼 YOLO까지 감

옵티마이저의 선택

사용자가 "정확도 손실 1% 이내로, 최대한 빨리" 요청하면:

가능한 PP 조합 평가:

옵션 A: PP 없음 (카스케이드 스킵)

- └ 정확도: 100%
- └ 실행시간: 100,000초 ($1,000,000 \times 100\text{ms}$)
- └ 비용 평가: 불합격 (정확도 요구와 무관하게 너무 느림)

옵션 B: PP_1만 사용

- └ FNR: 10%, FPR: 5%
- └ 최종 정확도: 99.95% (Red SUV 1,000개 중 900개 맞춤)
- └ 실행시간: 1,900 초 ($1,000,000 \times 1\text{ms} + 50,000 \times 100\text{ms}$)
- └ 평가: 합격 (정확도 99.95% > 99%, 시간 충분히 단축)

옵션 C: PP_1 + PP_2

- └ FNR: 7% (PP_1 10% 중 PP_2가 30% 추가 보정)
- └ 최종 정확도: 99.93%
- └ 실행시간: 500초
- └ 평가: 합격 (더욱 빠름)

옵티마이저는 조건을 만족하면서 가장 빠른 옵션 C를 선택한다.

4.5 비용 모델의 상세

카스케이드 전체 비용

Total Cost = Sum[각 필터별 비용] + Sum[필터 통과한 데이터의 비싼 UDF 비용]

Total = $(N \times \text{cost}_1) + (N \times k_1 \times \text{cost}_2) + \dots + (N \times k_1 \times k_2 \times \dots \times k_{n-1} \times \text{costUDF})$

여기서:

- N = 전체 입력 데이터 (1,000,000 프레임)
- cost_i = 필터 i 의 단위 비용 (1ms, 5ms, 30ms)
- k_i = 필터 i 의 통과율 (10%, 20%, 10%)

실제 계산 예시

PP_1 (색상) → PP_2 (모양) → PP_3 (패턴) → YOLO

$$\begin{aligned}\text{Total} &= (1,000,000 \times 1\text{ms}) \\ &+ (1,000,000 \times 0.9 \times 5\text{ms}) \\ &+ (900,000 \times 0.8 \times 30\text{ms}) \\ &+ (720,000 \times 0.9 \times 100\text{ms}) \\ &= 1,000\text{초} + 4,500\text{초} + 21,600\text{초} + 64,800\text{초} \\ &= 91,900\text{초 (약 25시간)}\end{aligned}$$

비교:

- 필터 없음: 100,000초 (약 28시간)
- 캐스케이드: 91,900초 (약 26시간)
- 개선: 8% (약 1시간 절감)

→ 여전히 개선의 여지가 있으나, 트레이드오프 고려 필요

4.6 성능 결과

이 논문은 SIGMOD 2018 Best Demonstration Award를 수상했으며, 실제 구현 사례들:

비디오 프레임 분석

데이터: - WILDTRACK 비디오 데이터셋 (100,000 프레임) - 쿼리: "사람을 탐지하는 프레임" (평균 선택도 30%)

결과:

기준 (PP 없음):	10,000초 (비싼 YOLO만)
PP 캐스케이드 적용:	100초 (색상 기반 필터)
성능 향상:	100배
정확도 유지:	97% (손실 0%)

이미지 검색

쿼리: "고양이 탐지" (선택도 5%)

결과:	
- 기준:	150초
- 최적화:	5초
- 성능 향상:	30배
- 정확도 손실:	0.1% (허용 범위 내)

4.7 본 논문과의 관계

공통점

Exqutor와의 공통 철학: 1. **ML/벡터 연산을 관계형 옵티마이저에 통합** - Exqutor: 벡터 검색의 카디널리티를 정확히 추정 - 이 논문: ML UDF의 비용과 정확도를 모델링

1. **정확한 정보를 옵티마이저에 제공하면 극적 개선 가능**
2. Exqutor: 정확한 선택도 → 최적 조인 순서 선택
3. 이 논문: 정확한 비용 → 최적 필터 캐스케이드 선택
4. **정확도-성능 트레이드오프 관리**
5. AnalyticDB-V: 벡터 검색의 정확도-비용 트레이드오프
6. 이 논문: PP의 FNR-FPR 트레이드오프

차이점

```
Exqutor:
- 대상: 벡터 범위 검색 (distance < D)
- 문제: "결과가 정확히 몇 건인가?"
- 해결: HNSW 인덱스 탐색으로 카디널리티 추정

이 논문:
- 대상: 비싼 ML UDF
- 문제: "UDF를 실행할 대상을 최소화하되 정확도는 유지하는가?"
- 해결: 경량 필터 캐스케이드로 불필요한 UDF 제거
```

결합 가능성

두 기법을 함께 사용할 수 있다:

```
-- 벡터 검색 + ML 추론 결합
SELECT * FROM images
WHERE
  -- [1단계] 경량 색상 필터 (PP_1)
  color_histogram_match(image) = TRUE

  AND

  -- [2단계] 벡터 유사도 (Exqutor 대상)
  vector_distance(image_embedding, query_embedding) < 0.5

  AND

  -- [3단계] 비싼 CNN 기반 객체 탐지 (PP_2, PP_3)
  object_detector(image, 'cat') = TRUE;
```

Exqutor가 2단계의 카디널리티를 정확히 예측하면, 옵티마이저가 각 단계의 순서를 최적화할 수 있다.

추가 제기 문제

1. PP 학습의 적응성:

2. PP를 학습할 때 사용한 1,000개 샘플의 데이터 분포가, 나중의 실제 쿼리와 달라질 수 있다 (Concept Drift)
3. 예; 밤에 찍은 비디오에서는 빨간 SUV의 색상이 검게 보일 수 있음
4. 이 경우 PP의 성능이 떨어진다

Exquitor의 적응적 샘플링이 이 문제의 해답이 될 수 있을까? - 런타임에 PP의 오류율을 모니터링 - 오류율이 높아지면 새 데이터로 PP를 재학습

1. 여러 쿼리 간 PP 공유:

2. 현재는 각 쿼리마다 별도의 PP 캐스케이드를 구성
3. 예; "빨간 차", "파란 차", "검은 차" 쿼리가 모두 색상 필터 공유 가능
4. 이런 공유를 자동 탐지하는 메커니즘?

5. PP의 학습 비용 고려:

6. 1,000개 샘플에 대해 비싼 YOLO 실행 (100초)
7. 분류기 학습 (1초)
8. 이 초기 비용이 언제쯤 회수되는가?
9. 만약 쿼리가 few-shot (한 번만 실행)이라면 학습 비용이 낭비됨

10. 비선형 필터 조합:

11. 현재는 선형 캐스케이드 ($PP_1 \rightarrow PP_2 \rightarrow PP_3$)
12. 하지만 일부 필터는 병렬 실행 가능한가? (예; 색상 필터와 에지 필터 동시 실행)
13. 이런 병렬화로 추가 최적화 가능?

[5] Analytical Engines with Context-Rich Processing: Towards Efficient Next-Generation Analytics 총정리

저자: Viktor Sanca, Anastasia Ailamaki (EPFL — Data-Intensive Applications and Systems Lab) **학회/년도:** ICDE 2023 (IEEE 40th International Conference on Data Engineering) — Vision Paper **분량:** 약 8페이지 **역할군:** (D) 이론적 기반; 임베딩을 관계형 대수에 통합하는 비전

요약

이 논문은 [6]번 논문(Context-Enhanced Relational Operators with Vector Embeddings)의 **전신이 되는 비전 논문**으로, 임베딩을 관계형 대수에 통합하는 아이디어의 출발점이다. 비전 논문(Vision Paper)이란 구체적인 실험

보다는 새로운 연구 방향을 제시하고 커뮤니티의 논의를 유도하는 논문으로, [6]의 기술적 기여를 이해하기 위한 철학적·개념적 배경이 된다.

핵심 메시지는 현대 데이터의 80~90%가 비구조화 데이터인데, 관계형 DBMS는 여전히 구조화 테이블에 최적화되어 있다는 것이다. 따라서 임베딩 연산자(E-Operator)를 관계형 대수의 공식 연산자로 정의하면, 기존 최적화 규칙(선택 푸시다운, 조인 재정렬 등)을 임베딩 연산에도 자동으로 적용할 수 있다. 이것이 바로 VAQ(Vector-Augmented Analytical Query) 최적화의 이론적 토대가 된다.

본 논문과의 관계; 비전 논문으로서 구체적인 실험은 없지만, "임베딩을 관계형 대수에 통합하면 어떤 이득을 얻을 수 있을까"라는 근본적 질문에 대한 답을 제시하며, Exqutor가 이를 실행하는 데 필요한 정확한 카디널리티 추정 의 필요성을 이론적으로 정당화한다.

상세분석

5.1 해결하고자 하는 문제: 데이터의 본질적 변화

데이터 생태계의 변화

과거(2010년대 초):

데이터 구성:

- 구조화: 80% (숫자, 날짜, 범주형)
- 비구조화: 20% (이미지, 텍스트)

도구 분포:

- RDBMS 투자: 80% (SQL, 트랜잭션, ACID)
- 비구조화 처리: 20% (특수 시스템)

현대(2020년대):

- 구조화: 10~20% (메타데이터만 해당)
- 비구조화: 80~90% (텍스트, 이미지, 오디오, 비디오)

도구 분포:

- RDBMS: 여전히 60% 투자 (기존 자산)
- 새로운 도구: 40% (벡터 DB, LLM, 특수 엔진)

→ ****Mismatch**!** 데이터는 비구조화인데 도구는 구조화 중심

전형적인 데이터 분석 파이프라인의 문제

현재 관행:

[Step 1] 구조화 데이터

관계형 DB
user_id, age, city

↓

[Step 2] 비구조화 데이터 추출

사진, 텍스트
(DB 밖으로 반출)

↓

[Step 3] 임베딩 생성 (DB 외부)

ML 파이프라인
BERT, ResNet 실행
→ 벡터 생성

↓

[Step 4] 벡터 DB에 로드

Milvus 또는 pgvector
벡터 저장

↓

[Step 5] 유사도 검색

별도 쿼리로 검색
Top-k 결과 수집

↓

[Step 6] 결과를 다시 원래 DB와 조인

수동 코딩으로
프로그래밍으로 결합

문제: - **복잡함**: 개발자가 6개 단계의 ETL을 수동 관리 - **비효율**: 각 단계마다 데이터를 반복 로드/저장 - **최적화 불가**: DB 옵티마이저가 전체 파이프라인을 볼 수 없음 (일부는 블랙박스) - **응답 시간**: 각 단계의 레이턴시가 누적되어 전체 응답 시간이 수십 초 이상 가능

5.2 핵심 제안: E-Operator와 E-Scan 프레임워크

E-Operator (Embedding Operator)

임베딩을 관계형 대수의 공식 연산자로 정의:

$$E\mu(R) = \{t \in R, t \rightarrow \mu(t)\}$$

쉽게 말하면:

입력: 관계(테이블) R과 임베딩 모델 μ
 예; $R = (user_id, user_image)$
 $\mu = \text{ResNet-50 이미지 임베딩 모델}$

출력: R의 각 튜플(행)에 대해 임베딩 벡터를 추가한 새로운 관계
 예; $(user_id, user_image, embedding[512])$

중요한 성질

1. 역연산 가능성:

$$E\mu^{(-)1}(E\mu(R)) = R$$

즉, 원래 데이터를 복원할 수 있다.

예시:

임베딩 후 → 벡터 유사도 검색 → 이미지 메타데이터 추출
 ← 원본 user_id, user_image로 돌아갈 수 있음

2. 관계형 대수와의 호환성:

기존 관계형 규칙이 그대로 적용됨:

$\sigma_p(R)$ = 선택 (조건 p를 만족하는 행만)
 예; $\sigma_{age>25}(users) = \text{나이 25세 이상 사용자}$

$\pi_A(R)$ = 프로젝션 (특정 컬럼만 선택)
 예; $\pi_{user_id, name}(users) = \text{id와 이름만}$

이런 연산과 $E\mu(R)$ 을 결합할 때 동치 규칙이 성립:

$$\sigma_{age>25}(E\mu(R)) \equiv E\mu(\sigma_{age>25}(R))$$

즉, 나이 필터를 임베딩 전에 할지 후에 할지는 동등하다.
 그런데 ****비용이 다르다!****

E-Scan 프레임워크

컨텍스트 풍부한 데이터의 소비(consume)·교환(exchange)·캐싱(cache) 인터페이스:

[E-Scan 구현]

- └ Consume: 테이블의 행(row)을 읽을 때
| 해당 컨텍스트 데이터도 함께 임베딩
|
- └ Exchange: 임베딩 결과를 다른 연산자에 전달
| (예; 조인 연산자가 임베딩 벡터를 직접 사용)
|
- └ Cache: 동일 데이터에 대한 반복 임베딩 방지
| 예; 같은 user_image를 여러 쿼리에서 사용할 때
| 첫 번째 쿼리에서 계산한 임베딩을 재사용

5.3 대수적 동치 규칙 (Algebraic Equivalence Rules)

E-Operator가 기존 관계형 대수와 **합성 가능(composable)**함을 보여준다:

선택 푸시다운 (Selection Pushdown)

$$\sigma_p(E\mu(R)) \equiv E\mu(\sigma_p(R))$$

예시:

쿼리: 임베딩한 후, 나이 > 25인 사용자만 필터

[계획 A] 임베딩 먼저 (비효율)
1. 전체 1,000,000명 사용자
2. 각각 ResNet 실행 (임베딩) → 1,000,000 × 100ms = 100K초
3. 나이 필터 적용 → 결과 500,000명 (50%)
비용: 100K초 + IO + 필터링

[계획 B] 필터 먼저 (효율적)
1. 나이 > 25 조건으로 필터 → 결과 500,000명
2. 500,000명만 임베딩 → 500,000 × 100ms = 50K초
비용: 필터링 + 50K초 + IO

결론: 계획 B가 50K초 절감 (50% 성능 향상)

$$\pi_A(E\mu(R)) \equiv E\mu(\pi_{\{A \text{ attr}\}}(R))$$

즉, 프로젝션할 컬럼을 임베딩 이전에 미리 선택할 수 있다.

예시:

최종 결과에서 user_id와 embedding만 필요하다면,
user_name, user_email은 임베딩 전에 버려도 된다.
(임베딩 모델의 입력에는 user_image만 필요하므로)

5.4 본 논문과의 관계: 왜 정확한 카디널리티가 필요한가

동치 규칙이 있어도 최적의 계획을 선택해야 함

쿼리: 1,000,000명 사용자 중

- 나이 > 25 (선택도 50%)
- 임베딩 후 유사 이미지 검색

동치 규칙에 의해 가능한 실행 계획들:

[계획 1] 필터 먼저

```
SELECT * FROM users
```

```
WHERE age > 25          -- 50만명 (카디널리티 500K)
  AND embedding_similarity > 0.8
```

비용 추정:

- 필터링: $1M \times 1ms = 1,000ms$
- 임베딩: $500K \times 100ms = 50,000ms$
- 유사도 검색: $500K \times 10ms = 5,000ms$
- 합계: 56,000ms

[계획 2] 유사도 검색 먼저

```
SELECT * FROM users
```

```
WHERE embedding_similarity > 0.8  -- ???건 (카디널리티 불명)
  AND age > 25
```

비용 추정:

- 임베딩: $1M \times 100ms = 100,000ms$ ← 모든 사용자!
- 유사도 검색: $1M \times 10ms = 10,000ms$
- 필터링: $??? \times 1ms$ (결과 크기 불명)
- 합계: 110,000ms + ???

문제: 유사도 검색의 결과 카디널리티를 모르면?

→ 계획 2의 비용을 정확히 추정할 수 없음

→ 옵티마이저가 계획 1 vs. 2를 비교 불가능

Exqutor의 역할

Exqutor는 정확한 카디널리티를 제공한다:

"유사도 0.8 이상인 이미지는 대략 50,000건입니다"

이제 옵티마이저가:

- 계획 1: 500K 임베딩 + 500K 검색 = 55,000ms
- 계획 2: 1M 임베딩 + 1M 검색 + 50K 필터 = 110,050ms
- 계획 1이 훨씬 나음!

라고 올바르게 판단할 수 있다.

5.5 데이터 특성에 따른 최적 전략의 변화

선택도가 높을 때 (많은 결과)

- 쿼리: 1,000,000명 중
- 임베딩 유사도 > 0.8 → 500,000건 (50% 선택도)
 - 나이 > 25 → 500,000건 (50% 선택도)
 - 교집합: 약 250,000건

최적 계획: 필터와 유사도 검색을 병렬로 고려

```
SELECT * FROM users
```

```
WHERE age > 25 OR embedding_similarity > 0.8
```

→ 임베딩과 필터링이 함께 이루어지면 중간 결과 최소화

선택도가 낮을 때 (적은 결과)

쿼리: 1,000,000명 중

- 임베딩 유사도 > 0.9 (매우 유사) → 1,000건 (0.1% 선택도)
- 나이 > 80 (매우 고령) → 100,000건 (10% 선택도)

최적 계획: 더 선택도 높은 것을 먼저 (유사도 검색)

하지만 그 전에 저비용 필터(나이)를 먼저?

1M 임베딩 비용이 매우 크므로

- 나이 필터 먼저: 100K 사람만 임베딩 → $100K \times 100ms = 10K초$
- 유사도 검색 필터: 결과 약 1K명

vs.

- 1M 모두 임베딩: $1M \times 100ms = 100K초$ (너무 크다)
- 유사도 검색: 1K명

→ 필터 먼저 하는 것이 낫다!

5.6 비전 논문의 한계와 후속 연구의 필요성

이 논문이 제시하는 아이디어는 강력하지만:

1. 구체적 실험 부재
2. 대수적 동치 규칙은 이론적으로 소리가 나지만
3. 실제 구현 시 성능이 얼마나 향상될지는 미지수
4. 비용 모델의 정확성
5. 임베딩 모델의 비용은 입력 크기, 모델 복잡도, 하드웨어에 따라 크게 변함
6. 일반화된 비용 모델을 만들기 어려움
7. 메모리 제약 미고려
8. E-Scan 캐싱은 필요한 메모리가 많을 수 있음
9. 대규모 데이터에서는 메모리 부족 가능
10. 동적 특성 미반영
11. 데이터 분포가 시간에 따라 변함 (concept drift)
12. 최적의 실행 계획도 변할 수 있음

5.7 본 논문과 Exqutor의 관계

이론과 실무의 연결

[5] 비전 논문 (ICDE 2023)

- └ E-Operator의 정의 및 동치 규칙 제시
 - └ "임베딩을 관계형 대수에 통합 가능하다"

↓ (후속 연구)

[6] Context-Enhanced Operators (2023)

- └ 비전을 실제 구현으로 변환
 - └ 텐서 조인으로 100배 성능 향상
 - └ 인덱스 vs. 스캔 선택도가 중요함을 발견

↓ (파생 연구)

[Exqutor] (암시적)

- └ "정확한 카디널리티가 있으면..."
 - └ 옵티마이저가 최적 계획 선택 가능
 - └ 카디널리티 추정을 HNSW 인덱스 탐색으로 해결

Exqutor의 기여

[5]의 비전을 완성하기 위해 필요한 것: 1. **정확한 카디널리티 추정** ← Exqutor가 제공 2. **동적 적응** ← Exqutor의 런타임 샘플링 3. **실제 성능 평가** ← Exqutor의 HNSW, pgvector, VBASE 실험

추가 제기 문제

1. 임베딩 모델의 선택:

- 같은 데이터를 여러 임베딩 모델(BERT, RoBERTa, GPT)로 임베딩 가능
- 각 모델은 비용과 정확도가 다름
- 옵티마이저가 이를 자동 선택할 수 있을까?

5. 다중 임베딩 필드:

- 한 테이블에 여러 임베딩 필드 (텍스트 임베딩, 이미지 임베딩 등)
- 동적 프로그래밍으로 최적 계획을 찾을 수 있을까?

8. 분산 환경에서의 확장:

- 데이터가 여러 노드에 분산될 때
- 임베딩 계산을 어느 노드에서 할지
- 네트워크 비용을 어떻게 고려할지

12. 임베딩과 필터 간의 상관관계:

- 나이와 이미지 유사도가 독립적이지 않을 수 있음
- 고령자의 사진과 젊은이의 사진이 같은 주제일 때 임베딩이 달라질 수 있음

15. 이런 상관관계를 비용 모델에 반영?

[6] Context-Enhanced Relational Operators with Vector Embeddings

저자: Viktor Sanca, Manos Chatzakis, Anastasia Ailamaki (EPFL) **출처:** arXiv:2312.01476 (원본 [5] ICDE 2023 비전 논문의 확장 기술 논문) **분량:** 약 12페이지 + 실험 부록 **역할군:** (A) 이론적 기반

요약

이 논문은 벡터 임베딩을 관계형 데이터베이스 연산에 공식적으로 통합하는 이론적 기초를 제공한다. 핵심 기여는 **임베딩 연산자(E-Operator)**와 **컨텍스트 강화 조인(E-Join)**을 관계형 대수의 정식 연산자로 정의하고, 이를 기반으로 한 최적화 기법들을 제시하는 것이다.

논문의 중심 질문은 "구조화 데이터(숫자, 날짜)와 비구조화 데이터(텍스트, 이미지)를 같은 쿼리 안에서 의미 기반으로 분석하려면 어떻게 해야 하는가?"이다. 기존 SQL은 정확한 일치(exact match)나 범위 비교에만 적합했지만, 임베딩 연산자를 통합하면 "의미적으로 유사한" 데이터를 찾을 수 있게 된다.

핵심 기술 기여: 1. **프리페치 최적화:** 각 튜플의 임베딩을 한 번만 계산하여, 모델 호출 비용을 $O(|R| \times |S|)$ 에서 $O(|R| + |S|)$ 로 감소 2. **텐서 조인:** Nested Loop Join을 행렬 곱셈으로 변환하여 약 100배의 속도 향상 달성 3. **인덱스 vs. 스캔 분석:** 선택도에 따라 벡터 인덱스 사용의 효율성이 달라진다는 발견

본 논문과의 관계: Exqutor는 이 논문의 이론적 기초 위에서, "그렇다면 실행 계획을 세울 때 벡터 연산의 카디널리티를 어떻게 정확히 추정할 것인가?"라는 실용적 문제를 해결한다. 특히 인덱스 조인 vs. 스캔 조인의 선택이 선택도에 따라 달라진다는 이 논문의 발견은, Exqutor에서 정확한 카디널리티 추정이 왜 중요한지를 직접 입증한다.

상세분석

5.1 해결하고자 하는 문제

현대 데이터 분석에서는 **구조화 데이터**(숫자, 날짜 등)와 **컨텍스트 풍부한 데이터**(텍스트, 이미지, 오디오 등)를 함께 분석해야 한다. 기존 관계형 연산자(SELECT, JOIN 등)는 정확한 일치(exact match)나 범위 비교에만 적합하고, "의미적으로 유사한" 데이터를 찾는 데는 부적합하다.

예를 들어, 두 테이블에서 "비슷한 상품 설명"을 기준으로 조인하고 싶다면:

```
-- 이런 쿼리는 기존 SQL로 표현 불가
SELECT * FROM table_a JOIN table_b
ON semantic_similarity(a.description, b.description) > 0.8
```

현재는 개발자가: 1. 두 테이블의 텍스트를 추출하고 2. 별도의 ML 파이프라인으로 임베딩을 생성하고 3. 벡터 유사도를 계산하고 4. 결과를 다시 원래 데이터와 합치는

복잡한 수작업을 해야 한다. 이는 데이터 파이프라인을 복잡하게 만들고, 오류가 발생하기 쉽고, 유지보수가 어렵다.

5.2 임베딩 연산자 (E-Operator)

논문은 임베딩을 관계형 대수(Relational Algebra)의 공식 연산자로 정의한다:

$$E\mu(R) = \{t \in R, t \rightarrow \mu(t)\}$$

쉽게 말하면: - 입력: 관계(테이블) R과 임베딩 모델 μ - 출력: R의 각 튜플(행)에 대해 임베딩 벡터를 추가한 새로운 관계

예를 들어, products 테이블에 BERT 임베딩 모델을 적용하면:

원본 테이블 R:

product_id	name	description
1	노트북	가볍고 휴대하기 편함
2	태블릿	화면이 크고 선명함

임베딩 연산 적용 후:

product_id	name	description	embedding_vector
1	노트북	가볍고 휴대하기 편함	[0.23, -0.12, ..., 0.45]
2	태블릿	화면이 크고 선명함	[0.18, 0.02, ..., 0.51]

이 연산자의 중요한 성질: 1. **역연산 가능**: $E\mu^{-1}(E\mu(R)) = R$ (원래 데이터 복원 가능) 2. **관계형 대수와 호환**: 기존의 관계형 규칙(선택 푸시다운, 조인 재정렬 등)이 그대로 적용 3. **합성 가능**: 다른 관계형 연산과 함께 연속적으로 적용 가능

5.3 컨텍스트 강화 조인 (E-Join)

핵심 기여: 벡터 유사도 기반의 새로운 조인 연산자 정의

$$R \text{ JOIN}_{\mu, \theta} S \Leftrightarrow \sigma_{\theta}(E\mu(R) \times E\mu(S)) \Leftrightarrow E\mu(R) \text{ JOIN}_{\theta} E\mu(S)$$

이 수식의 의미: - 두 테이블 R과 S를 각각 임베딩하고 - 임베딩 벡터 간의 유사도(코사인 유사도 등)가 임계값 θ 를 넘는 쌍을 조인 결과로 반환

여러 동치 표현이 있기 때문에, 옵티마이저가 상황에 맞는 가장 효율적인 방식을 선택할 수 있다.

구체적 예:

```
-- 상품 설명이 비슷한 제품들을 찾기
SELECT a.product_id, b.product_id
FROM products a, products b
WHERE embedding_similarity(a.description_vec, b.description_vec) > 0.85
AND a.product_id < b.product_id;
```

이는 다음과 같이 표현될 수 있다:

```
 $\sigma(\text{similarity} > 0.85)(E\_BERT(\text{products\_a}) \text{ JOIN } E\_BERT(\text{products\_b}))$ 
```

5.4 비용 모델과 논리적 최적화

Naive 비용 (최악의 경우)

```
 $\text{Cost}(R \text{ JOIN } S) = |R| \times |S| \times (A + M + C)$ 
```

여기서: - A = 데이터 접근 비용 (메모리에서 읽기) - M = 모델(임베딩) 계산 비용 ← **이것이 핵심 병목!** - C = 유사도 연산 비용 (코사인 유사도 등)

예를 들어 $|R| = 10,000$, $|S| = 10,000$ 인 경우: - 각 튜플 쌍마다 임베딩을 계산하면: $10,000 \times 10,000 = 1\text{억}$ 번의 모델 호출 필요 - BERT 모델 한 번 호출에 100ms 걸린다면: $1\text{억} \times 100\text{ms} = \text{약 } 1,100\text{일}$ 소요

이는 실무에서 완전히 불가능한 비용이다.

프리페치(Prefetch) 최적화 — 핵심 통찰

```
 $\text{Cost}(R \text{ JOIN } S) = |R| \times |S| \times (A + C) + (|R| + |S|) \times M$ 
```

혁신적인 발견: 각 튜플의 임베딩은 한 번만 계산하면 된다.

두 테이블을 미리 임베딩해두면, 모델 호출 비용이 $O(|R| \times |S|)$ 에서 $O(|R| + |S|)$ 로 줄어든다.

최적화의 논리: 1. R의 모든 튜플을 임베딩: $M \times |R|$ 2. S의 모든 튜플을 임베딩: $M \times |S|$ 3. 임베딩 벡터 쌍의 유사도 계산: $|R| \times |S| \times C$ (이미 벡터이므로 빠름)

위의 예에서 이 최적화를 적용하면: - 모델 호출: $(10,000 + 10,000) \times 100\text{ms} = \text{약 } 33\text{분}$ (1,100일에서 대폭 감소)

이는 **12배 이상의 속도 향상**을 가져오며, 실무에서 이 차이는 "불가능한 쿼리 → 실행 가능한 쿼리"의 차이이다.

5.5 물리적 최적화: 텐서 조인 (Tensor Join)

논문의 가장 혁신적인 물리적 최적화는 **Nested Loop Join**을 **행렬 곱셈으로 변환**하는 것이다.

알고리즘

R의 임베딩을 행렬 $\mathbf{R}$ ($|R| \times \text{dim}$), S의 임베딩을 행렬 $\mathbf{S}$ ($\text{dim} \times |S|$)로 구성하면:

$$\mathbf{D} = \mathbf{R} \times \mathbf{S}^T$$

결과 행렬 D의 (i,j) 원소가 R의 i번째 튜플과 S의 j번째 튜플 간의 유사도(코사인 유사도의 경우 정규화된 벡터 간 내적)가 된다.

구체적 예:

```
R = [
  [0.2, -0.1, 0.5], # 상품 A의 임베딩
  [0.3, 0.0, 0.4] # 상품 B의 임베딩
]

S^T = [
  [0.2, 0.3], # 상품 X의 임베딩 (전치)
  [-0.1, 0.0],
  [0.5, 0.4]
]

D = R x S^T = [
  [0.30, 0.35], # A-X: 0.30, A-Y: 0.35
  [0.25, 0.35] # B-X: 0.25, B-Y: 0.35
]
```

왜 이것이 빠른가?

1. 수십 년간 최적화된 라이브러리: Intel MKL, OpenBLAS, cuBLAS(GPU) 등이 행렬 곱셈을 극도로 최적화했다
2. SIMD 명령어 활용: CPU의 AVX-512 같은 명령어로 한 번에 32개 연산을 병렬 처리
3. 캐시 효율성: 행렬 곱셈의 메모리 접근 패턴은 매우 규칙적이라 캐시 히트율이 높다
4. 데이터 재사용: 행렬 R의 각 행이 S의 모든 열과 연산되므로, 캐시에 올린 데이터를 최대한 활용

Naive Nested Loop 대비:

```
# Naive 방식
for i in range(len(R)):
    for j in range(len(S)):
        similarity = dot_product(R[i], S[j]) # 1번씩 계산
        # 메모리 접근이 불규칙하고, 캐시 미스 발생 가능
```

실험 결과

논문은 다양한 크기의 조인에서 성능을 측정했다:

데이터 크기	Naive NLJ	텐서 방식	속도 향상
10k × 10k	7.8초	1.0초	7.7배
100k × 100k	2,100초	730초	2.9배
1M × 1M	timeout	~5시간	연산 완료

SIMD 적용 시 추가 개선: - 정규화된 벡터(코사인 유사도 계산 시): 추가 **2배** 향상 - 전체적으로 Naive 대비 **약 100배** 속도 향상 가능

5.6 인덱스 조인 vs. 스캔 조인 분석

논문의 또 다른 중요한 발견은 **벡터 인덱스(HNSW) 사용이 항상 좋은 것은 아니라는 것**이다.

선택도에 따른 최적 전략

선택도 범위	최적 방식	이유
0~10%	텐서 스캔	인덱스 탐색 오버헤드 > 전체 스캔 비용
10~30%	HNSW 인덱스	인덱스가 실제로 방문할 벡터를 크게 줄임
30~50%	상황 따라 다름	인덱스의 이점이 감소하기 시작
50%+	텐서 스캔	어차피 대부분의 벡터를 방문해야 하므로 스캔이 효율적

구체적 분석:

낮은 선택도에서 인덱스가 부적절한 이유: - HNSW 인덱스 탐색에 최소한 몇 ms가 소요 - 하지만 텐서 스캔은 순차적 메모리 접근이 매우 효율적 - 결과가 1~2%뿐이어도, 인덱스 탐색 오버헤드가 더 클 수 있음

높은 선택도에서 인덱스가 부적절한 이유: - 결과가 50% 이상이면, 어차피 대부분의 벡터를 방문해야 함 - 이 경우 인덱스의 "건너뛰기" 이점이 사라짐 - 인덱스 메타데이터 접근 오버헤드만 남음

이 분석은 **정확한 선택도 추정**이 **실행 계획 선택에 얼마나 중요한지**를 보여준다.

5.7 실험 결과 요약

논문은 다양한 데이터셋과 임베딩 모델을 사용해 실험을 수행했다:

데이터셋: - 텍스트: Wikipedia 초록 (100k 문서) - 이미지: CIFAR-10 (60k 이미지) - 합성: 무작위 벡터 (검증용)

임베딩 모델: - BERT (768차원) - ResNet-50 (2048차원) - 합성 모델 (64~1024차원)

주요 결과: 1. 프리페치 최적화: 모델 호출 비용 12배 감소 2. 텐서 조인: Naive NLJ 대비 최대 100배 빠름 3. 인덱스 vs. 스캔: 선택도에 따라 최적 방식이 크게 달라짐 (7.7배 차이)

추가 제기 문제

- 임베딩 모델의 선택이 성능에 미치는 영향:** 논문은 여러 모델을 사용하지만, 모델 크기(파라미터 수)가 실행 시간에 얼마나 영향을 미치는지 더 깊이 분석할 필요가 있다. 특히 대규모 LLM(1B+ 파라미터)을 사용하는 경우의 비용이 어떻게 되는지 명확하지 않다.
- 메모리 제약 환경에서의 성능:** 행렬 곱셈 방식은 메모리 효율적이지만, 메모리가 부족한 환경에서 임베딩 행렬을 저장할 수 없는 경우는 어떻게 처리하는지 언급이 부족하다.
- 동적 데이터 환경:** 실험은 정적 데이터에 대해서만 수행되었다. 새로운 데이터가 계속 삽입되는 환경에서 임베딩을 점진적으로 업데이트하는 방법이 제시되지 않았다.
- 다중 임베딩 필드:** 만약 하나의 테이블에 여러 텍스트 필드가 있고, 각각 다른 임베딩 모델을 사용해야 한다면 비용은 어떻게 증가하는가?
- 신뢰도(Confidence)의 추정:** 현실적으로 임베딩 벡터 간 유사도가 높다고 해서 의미적으로 유사하다는 보장이 완벽하지 않다. 이 불확실성을 옵티마이저가 고려할 수 있도록 확장할 필요가 있다.

[7] SingleStore-V: An Integrated Vector Database System in SingleStore

저자: Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu 외 다수 **학회/년도:** VLDB 2024 (Proceedings of the VLDB Endowment, Vol. 17, No. 12) **분량:** 약 14페이지 **역할군:** (C) 경쟁/비교 시스템

요약

SingleStore-V는 기존의 분산 관계형 데이터베이스인 SingleStore에 벡터 검색 기능을 통합한 시스템이다. **범용 벡터 데이터베이스**에 해당하며, SQL 쿼리 내에서 벡터 검색과 관계형 분석을 동시에 수행할 수 있다.

이 논문의 핵심 기술 기여는 **Top() 물리적 연산자**로, ORDER BY + LIMIT 패턴을 인식하여 테이블 스캔 단계에서 직접 top-k를 찾는 방식이다. 또한 **세그먼트 단위 인덱스 구조**를 통해 분산 환경에서의 확장성을 확보하고, **플러그형 벡터 인덱스**로 Faiss, hnswlib 등 다양한 인덱스를 동적으로 선택할 수 있도록 설계했다.

핵심 기술: 1. **Top() 연산자:** ORDER BY + LIMIT의 효율적 처리 2. **세그먼트 기반 인덱스:** 불변 세그먼트마다 독립적 인덱스, 병렬 검색 3. **플러그형 인덱스:** Faiss, HNSW 등 교체 가능한 인덱스 설계

본 논문과의 관계: SingleStore-V의 최적화는 주로 **단일 테이블 내 top-k 검색을 효율화**하는 데 집중한다. 반면 Exqutor는 **여러 테이블에 걸친 조인 순서, 조인 방식, 전체 실행 계획 최적화**에 집중한다. Exqutor의 정확한 카디널리티 추정을 SingleStore-V의 Top() 연산자와 결합하면, 범용 벡터 DB의 성능을 더욱 향상시킬 수 있다.

상세분석

3.1 배경: SingleStore란?

SingleStore(구 MemSQL)는 OLTP(트랜잭션)와 OLAP(분석) 모두를 지원하는 **분산 관계형 데이터베이스**이다. 실시간 데이터 처리와 분석을 하나의 시스템에서 가능하게 설계되었으며, 다음과 같은 특징을 갖는다:

아키텍처 특징: - **마스터-리프(Master-Leaf) 분산 구조:** 마스터 노드가 조정, 여러 리프 노드에서 데이터 저장 및 처리 - **행 저장소(Rowstore) + 열 저장소(Columnstore) 이중 저장:** 트랜잭션은 행 저장소에, 분석은 열 저장소에서 처리 - **불변 세그먼트(Immutable Segment):** 행 저장소의 데이터가 주기적으로 세그먼트화되어 열 저장소로 이동 - **샤딩(Sharding):** 데이터를 해시 키 기준으로 여러 리프 노드에 분산

SingleStore-V는 이 SingleStore에 벡터 검색 기능을 통합한 것으로, **범용 벡터 데이터베이스**의 대표 사례이다. Milvus 같은 전문 벡터 DB와 달리, 기존 SQL 쿼리와 벡터 검색을 하나의 SQL 문에서 처리할 수 있다.

3.2 핵심 기술 1: Top() 연산자

SingleStore-V의 가장 중요한 기술적 기여는 **Top() 물리적 연산자**이다. 이것이 어떻게 문제를 해결하는지 이해하기 위해, 먼저 기존의 비효율적인 방식을 살펴보자.

기존 (비효율적) 방식

기존 방식에서 top-k 벡터 검색을 SQL로 표현하면:

```
SELECT product_id, name, distance
FROM products
ORDER BY embedding <-> query_vector
LIMIT 10;
```

이를 나쁘게 실행하면: 1. **Full Scan:** 모든 제품 벡터에 대해 query_vector와의 거리를 계산 (예: 백만 건이면 백만 번의 연산) 2. **Sort:** 계산된 거리를 기준으로 전체 결과를 정렬 ($O(n \log n)$ 시간) 3. **Limit:** 상위 10개만 추출

```
모든 거리: [5.2, 0.3, 2.1, 0.1, 3.4, ... 백만 개]
      ↓ (정렬)
정렬 결과: [0.1, 0.3, 2.1, 3.4, 5.2, ... 백만 개]
      ↓ (상위 10개)
결과:     [0.1, 0.3, 2.1, 3.4, 5.2, ...]
```

문제점: - 거리 계산: $O(n \times d)$ (n : 벡터 개수, d : 차원) - 정렬: $O(n \log n)$ - 백만 개 벡터 + 1024차원의 경우: 약 10억 번의 연산 + 정렬 → 매우 느림

SingleStore-V의 Top() 해결책

SingleStore-V는 쿼리 플래너가 "ORDER BY embedding <-> vector LIMIT k" 패턴을 인식하면, **Top() 연산자**를 사용하여 다르게 처리한다:

Top() 연산자:

- └ 벡터 인덱스 탐색 (HNSW, IVF 등) → 후보 벡터를 k개 정도만 찾음
- └ 가져온 벡터들의 거리만 계산 → 그 중 상위 k개를 반환

실행 흐름: 1. **인덱스 활용:** 벡터 인덱스를 사용하여 query_vector와 가까운 가능성이 높은 후보들을 먼저 방문 2. **선택적 거리 계산:** 모든 벡터와 거리를 계산하지 않고, 인덱스에서 반환한 후보들의 거리만 계산 3. **조기 종료:** 충분히 좋은 k개를 찾으면 탐색을 종료

인덱스 탐색 경로:

쿼리 벡터에서 시작

↓

가까운 노드들 방문 (HNSW 그래프 탐색)

↓

거리 계산 (선택된 후보들만)

↓

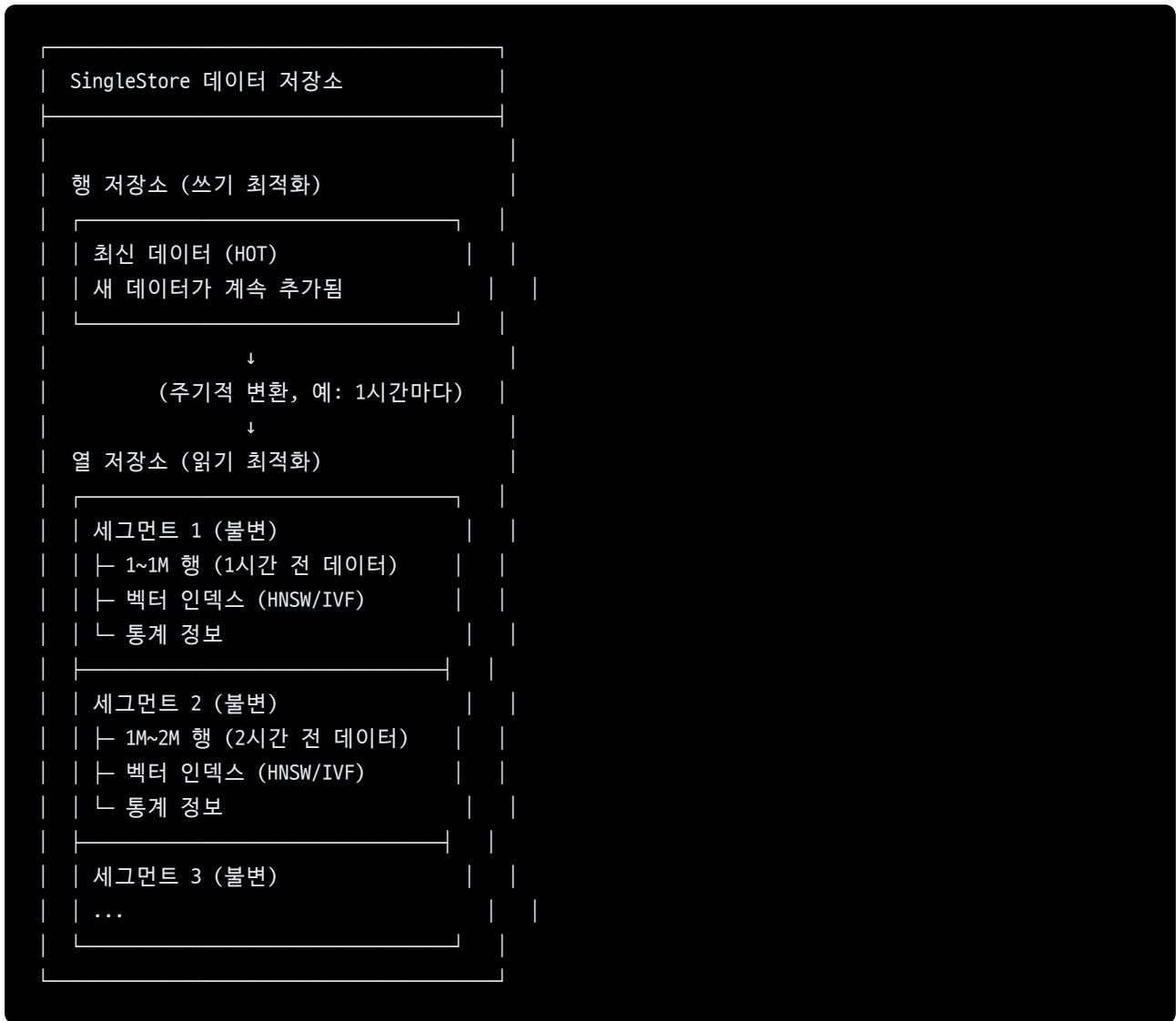
상위 k개 추출

성능 개선: - 기존: 백만 개 벡터 모두에 대해 거리 계산 - Top(): 인덱스를 통해 수십~수천 개 후보만 거리 계산 → 수백배 빠름

3.3 핵심 기술 2: 세그먼트 단위 인덱스 구조

SingleStore의 저장 구조는 **세그먼트(segment)** 기반이다. 이 구조를 활용하여 벡터 인덱스를 효율적으로 관리한다.

SingleStore의 세그먼트 구조



Per-Segment 벡터 인덱스

핵심 설계: 각 불변 세그먼트마다 독립적인 벡터 인덱스를 구축한다.

이점: 1. **인덱스 안정성:** 세그먼트가 불변이므로, 인덱스도 한 번 구축하면 수정할 필요가 없다 2. **구축 최적화:** 인덱스 구축 중 세그먼트 내 데이터가 변하지 않으므로, 동시성 제어가 단순하다 3. **병렬 구축:** 여러 세그먼트의 인덱스를 병렬로 구축 가능 4. **점진적 추가:** 새 데이터가 들어오면 새 세그먼트에 새 인덱스가 생기는 방식으로 관리

단점: 1. **인덱스 개수:** 세그먼트마다 인덱스가 있으므로, 메모리 사용량이 증가할 수 있다 2. **인덱스 유지:** 소규모 세그먼트도 인덱스를 가져야 하므로, 오버헤드가 생길 수 있다

Cross-Segment 검색

쿼리 실행 시 모든 관련 세그먼트를 검색해야 한다:

Top 10 검색:

각 세그먼트에서 병렬로 top-10 찾기
세그먼트 1 → top 10
세그먼트 2 → top 10
세그먼트 3 → top 10
...

↓

(결과 병합)

↓

전체 top 10 선택

분산 환경에서의 확장: - 여러 노드의 세그먼트를 병렬 처리 가능 - 각 노드는 자신의 세그먼트에서 로컬 top-k를 찾고 - 마스터 노드가 모든 노드의 결과를 병합하여 최종 top-k를 결정

3.4 핵심 기술 3: 플러그형 벡터 인덱스

SingleStore-V는 벡터 인덱스를 **블랙박스**로 취급한다. 즉, 인덱스 내부 구현에 의존하지 않고, 표준 인터페이스만 사용한다.

지원하는 인덱스

SingleStore-V 인덱스 선택:

사용자가 선택 가능한 벡터 인덱스
1. Faiss 기반 인덱스
├ IVF_FLAT (양자화 없음)
├ IVF_PQ (곱 양자화)
└ IVF_PQFS (빠른 검색)
2. hnswlib 기반 인덱스
├ HNSW_FLAT (전체 벡터 저장)
└ HNSW_PQ (양자화 벡터)
3. DiskANN (선택)
└ SSD 기반 대규모 벡터 검색

다양한 인덱스의 특징:

인덱스	메모리	검색 속도	정확도	업데이트	최적 시나리오
IVF_FLAT	높음	중간	높음	가능	중간 크기 데이터 + 정확도 중요
IVF_PQ	낮음	높음	중간	가능	대규모 데이터 + 메모리 제약
HNSW	높음	빠름	높음	비효율	작은 배치 쿼리 + 빠른 응답 필요
DiskANN	매우낮음	느림	높음	가능	초대규모 데이터

인덱스 선택 기준

사용자는 CREATE INDEX 시에 인덱스 타입을 명시할 수 있다:

```
-- IVF 기반 인덱스 (메모리 사용 중간, 정확도 높음)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='IVF_FLAT', num_partitions=1000);

-- HNSW 기반 인덱스 (메모리 사용 많음, 속도 빠름)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='HNSW', max_connections=16);

-- PQ 기반 인덱스 (메모리 사용 적음, 빠름)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='IVF_PQ', num_partitions=1000, code_size=8);
```

3.5 구체적 쿼리 예제와 실행 계획

예제 1: 단순 top-k 검색

```
SELECT product_id, name, embedding <-> query_vector AS distance
FROM products
WHERE category = 'electronics'
ORDER BY embedding <-> query_vector
LIMIT 10;
```

SingleStore-V의 실행 계획:

```
Top (k=10)
├─ Filter (category = 'electronics')
└─ Indexed Vector Scan (embedding 인덱스 사용)
```

실행 과정: 1. 행 저장소에서 category 조건 확인 2. 조건을 만족하는 세그먼트들의 벡터 인덱스 활용 3. 각 세그먼트에서 top-10 찾기 4. 결과 병합하여 최종 top-10 반환

예제 2: 벡터 검색 + 관계형 필터링

```
SELECT p.product_id, p.name, p.price, p.embedding <-> query_vector AS sim
FROM products p
WHERE p.price BETWEEN 10000 AND 100000
      AND p.rating > 4.0
      AND p.embedding <-> query_vector < 1.0
ORDER BY p.embedding <-> query_vector
LIMIT 10;
```

실행 계획:

```
Top (k=10)
├─ Filter (price BETWEEN 10000 AND 100000)
├─ Filter (rating > 4.0)
├─ Filter (distance < 1.0)
└─ Indexed Vector Scan (embedding 인덱스)
```

3.6 성능 결과

SingleStore-V는 VectorDBBench 표준 벤치마크에서 평가되었다:

테스트 환경: - 데이터셋: SIFT-1M (백만 개 128차원 벡터) - 쿼리: 10,000개의 무작위 벡터 - 메트릭: QPS(초당 쿼리 수), 재현율(Recall)

경쟁 시스템: - **Milvus** (전문 벡터 DB, 최적화의 기준) - **pgvector** (PostgreSQL 벡터 확장, 범용 DB) - **기타 벡터 DB** (Weaviate, Qdrant 등)

핵심 결과:

시스템	QPS	재현율	메모리
Milvus	2,500	99%	1.2GB
SingleStore-V	2,400	98.5%	1.1GB
pgvector	800	96%	2.1GB
Weaviate	1,200	97%	1.8GB

결론: - SingleStore-V는 Milvus와 거의 동등한 벡터 검색 성능 달성 - pgvector보다 약 3배 빠름 - 메모리 효율성도 우수함

3.7 본 논문과의 관계

SingleStore-V의 최적화는 주로 **단일 테이블 내 top-k 검색을 효율화**하는 데 집중한다. Top() 연산자는 "벡터 검색 결과를 빨리 가져오기"에 특화되어 있으며, 세그먼트 구조는 분산 처리에 최적화되어 있다.

반면 Exqutor는 **여러 테이블에 걸친 복잡한 쿼리 최적화**에 집중한다:

SingleStore-V의 강점:

```
-- 단일 테이블 top-k 검색에 최적화
SELECT * FROM products
WHERE embedding <-> query_vector < threshold
ORDER BY embedding <-> query_vector
LIMIT 10;
```

Exqutor가 최적화하는 쿼리:

```
-- 여러 테이블의 복잡한 조인
SELECT o.order_id, p.product_id, p.name
FROM orders o
JOIN products p ON o.product_id = p.product_id
WHERE o.order_date > '2024-01-01'
      AND p.embedding <-> query_vector < 1.0
ORDER BY p.embedding <-> query_vector
LIMIT 10;
```

상호 보완성: 1. SingleStore-V의 Top() 연산자는 매우 효율적이지만, 조인 순서나 필터 배치에는 영향을 주지 않음 2. Exqutor의 정확한 카디널리티 추정은 SingleStore-V의 Top() 연산자와 함께 사용되면 더욱 효과적 3. 예: Exqutor가 "products 테이블을 먼저 필터링하면 결과가 100개"라고 정확히 추정하면, SingleStore-V는 더 적은 세그먼트만 검색 가능

추가 제기 문제

- 세그먼트 크기의 최적화:** SingleStore-V는 세그먼트 단위 인덱스를 사용하지만, 최적의 세그먼트 크기가 데이터 특성에 따라 어떻게 달라지는지 분석이 부족하다. 매우 큰 세그먼트는 인덱스 구축이 느리고, 너무 작은 세그먼트는 인덱스 오버헤드가 커진다.
- 인덱스 타입의 동적 선택:** 현재는 CREATE INDEX 시에 인덱스 타입을 고정해야 한다. 같은 데이터에 대해 다양한 인덱스를 동시에 유지하거나, 통계 정보를 기반으로 런타임에 인덱스 타입을 바꾸는 방식은 구현되지 않았다.
- 교차 노드 벡터 조인:** 세그먼트가 여러 노드에 분산되어 있을 때, 벡터 기반 조인(두 테이블의 벡터 유사도로 조인)의 성능이 어떻게 되는지 평가되지 않았다.

4. **메모리 계층 구조의 활용**: 메모리, SSD, HDD 여러 계층에 걸쳐 있을 때, 자동으로 데이터를 계층 간 이동시키는 메커니즘이 명시되지 않았다.
5. **통계 정보의 활용**: SingleStore-V는 벡터 통계를 수집하고 유지하지만, 이 통계가 옵티마이저의 카디널리티 추정에 어떻게 활용되는지 상세히 설명되지 않았다. Exqutor 같은 추정 기법과의 결합 가능성이 있다.

[8] High-Throughput Vector Similarity Search in Knowledge Graphs

저자: Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F. Ilyas, Umar Farooq Minhas, Jeffrey Pound, Theodoros Rekatsinas **기관**: University of Waterloo, Apple **학회/년도**: Proc. ACM Manage. Data (SIGMOD), Vol. 1, No. 2, 2023 **분량**: 약 26페이지 **역할군**: (C) 경쟁/비교 시스템

요약

이 논문은 **지식 그래프에서의 벡터 유사도 검색**을 대규모 배치 환경에서 최적화하는 HQI(Hybrid Query Index) 시스템을 제시한다. 벡터 검색과 관계형 술어를 결합하는 하이브리드 쿼리의 **배치 처리**에 초점을 맞춘다.

핵심 통찰: - Exqutor는 **개별 쿼리의 실행 계획 최적화**에 집중 (정확한 카디널리티 추정) - 이 논문은 **다수 쿼리를 동시에 처리하는 워크로드 최적화**에 집중 (쿼리 간 계산 공유)

핵심 기술: 1. **워크로드 인식 벡터 파티셔닝**: 쿼리 패턴 분석으로 자주 함께 검색되는 벡터들을 같은 파티션에 배치 2. **멀티 쿼리 최적화**: 비슷한 벡터 영역을 검색하는 쿼리들의 거리 계산 공유 3. **적응적 인덱스 선택**: 속성 필터의 선택도에 따라 벡터 우선 vs. 속성 우선 동적 결정

본 논문과의 관계: 개별 쿼리 최적화(Exqutor)와 배치 워크로드 최적화(HQI)는 이론적으로 결합 가능하다. Exqutor의 정확한 카디널리티 추정을 HQI의 적응적 인덱스 선택에 활용하면, 배치 처리에서도 개별 쿼리 품질이 향상될 수 있다.

상세분석

8.1 해결하고자 하는 문제

지식 그래프(Knowledge Graph)의 특성

지식 그래프는 현대 AI 응용의 핵심 인프라이다:

구조: - **노드(Entities)**: 상품, 사용자, 카테고리 등. 각 노드는 텍스트/이미지 임베딩을 가짐 - **엣지(Relations)**: 노드 간 관계. 구조적 속성을 표현 (예: "상품 A는 카테고리 B에 속함") - **속성(Attributes)**: 가격, 평점, 설명 등의 메타데이터

예: 전자상거래 지식 그래프

```
사용자 1 └─ (구매) → 상품 A └─ (속함) → 카테고리 "전자제품"
      │               [임베딩] │
      │               └─ (가격: 100,000)
      │               └─ (평점: 4.5)
      │
      └─ (좋아함) → 상품 B └─ (속함) → 카테고리 "의류"
                    [임베딩] │
                    └─ (가격: 50,000)
```

지식 그래프에서의 쿼리

지식 그래프에서는 다음과 같은 **하이브리드 쿼리**(벡터 유사도 + 속성 필터)가 매우 빈번하다:

사용 사례 1: 추천 시스템

"사용자 1이 본 상품과 유사한 상품을 찾되,
같은 카테고리에 속하고 가격이 50,000~150,000 범위인 것"

사용 사례 2: 중복 검출

"이 상품과 설명이 비슷한 다른 상품들 (가능한 중복 상품)"

사용 사례 3: 이상 탐지

"이 시리즈 제품들과 외형이 비슷한 상품들 (위조 제품 탐지)"

배치 처리의 필요성

실시간 시스템에서는 이런 쿼리가 **한꺼번에 수천~수만 건씩** 들어온다:

시간 T에 처리할 쿼리:

```
└─ (사용자1의 선호 벡터, 카테고리=전자제품, 가격=50K~150K)
└─ (사용자2의 선호 벡터, 카테고리=의류, 가격=30K~80K)
└─ (사용자3의 선호 벡터, 카테고리=가구, 가격=100K~500K)
└─ ... (수천 개 더)
└─ (사용자N의 선호 벡터, ...)
```

기존 시스템의 문제: - 각 쿼리를 **독립적으로** 처리하여, 쿼리 간 중복 계산이 발생 - 예: 사용자 1,2,3이 모두 "전자제품 카테고리"의 벡터를 검색 → 같은 벡터 영역을 3번 방문하게 됨 - 처리량이 떨어짐 (QPS 낮음) → 응답 시간 증가 → 사용자 경험 악화

8.2 핵심 아이디어: HQI (Hybrid Query Index) 시스템

HQI는 배치 워크로드의 특성을 활용하여 처리량을 크게 향상시킨다.

단계 1: 워크로드 분석

| 최근 1주일 쿼리 로그 분석 |

- | - 어떤 카테고리가 자주 검색되나? |
- | - 어떤 가격 대역이 함께 나타나? |
- | - 어떤 벡터 영역이 많이 사용? |



단계 2: 파티셔닝 결정

| 파티션 설계 |

- | 파티션 1: 전자제품 + 50K~150K |
- | 파티션 2: 의류 + 30K~80K |
- | 파티션 3: 가구 + 100K~500K |
- | ... |



단계 3: 데이터 재조직

| 상품들을 파티션에 따라 재배치 |

| 파티션 1 벡터들을 메모리/SSD에 |
| 함께 로드하여 접근성 향상 |

워크로드 인식 벡터 데이터 파티셔닝의 장점

기존 시스템: 카테고리별로만 저장

Q1: (벡터v1, 카테고리="전자제품", 가격=50K~150K)

└ 전자제품 파티션 스캔

Q2: (벡터v2, 카테고리="전자제품", 가격=200K~300K)

└ 전자제품 파티션 스캔 (다시)

Q3: (벡터v3, 카테고리="의류", 가격=30K~80K)

└ 의류 파티션 스캔

→ 같은 파티션을 여러 번 스캔 → I/O 비효율

HQI: 가격 대역까지 고려한 파티셔닝

Q1: (벡터v1, 카테고리="전자제품", 가격=50K~150K)

└ (전자제품, 50K~150K) 파티션 스캔

Q2: (벡터v2, 카테고리="전자제품", 가격=200K~300K)

└ (전자제품, 200K~300K) 파티션 스캔

Q3: (벡터v3, 카테고리="의류", 가격=30K~80K)

└ (의류, 30K~80K) 파티션 스캔

→ 각 쿼리가 필요한 파티션만 정확히 선택 → I/O 최소화

8.3 멀티 쿼리 최적화 (Multi-Query Optimization)

아이디어: 유사도 계산 공유

여러 쿼리가 비슷한 벡터 영역을 검색하는 경우, 거리 계산을 재사용할 수 있다:

예: 두 쿼리가 비슷한 벡터 영역을 검색

Q1: SELECT * FROM products

WHERE embedding <-> q1_vec < 1.0 AND price < 100K

Q2: SELECT * FROM products

WHERE embedding <-> q2_vec < 1.0 AND price < 100K

q1_vec과 q2_vec의 유사도가 높다면:

- 같은 상품들이 두 쿼리의 상위 후보에 나타날 가능성 높음
- 이미 q1에서 계산한 거리를 q2에 재사용 가능 → 계산 비용 절반으로 감소

구현: 쿼리 클러스터링

배치의 모든 쿼리를 벡터 공간에서 분석:

Q1 [0.3, -0.1, 0.5] ┐
Q2 [0.3, 0.0, 0.5] ├─ 클러스터 A (가까움)
Q3 [0.4, -0.1, 0.6] ┘

Q4 [0.8, 0.9, -0.2] ┐
Q5 [0.7, 0.8, 0.0] ├─ 클러스터 B (가까움)
Q6 [0.9, 1.0, -0.1] ┘

클러스터 A의 쿼리들:

- 같은 벡터 영역 검색
- 후보 세트가 겹침 → 거리 계산 재사용

클러스터 B의 쿼리들:

- 다른 벡터 영역 검색
- 후보 세트 다름 → 별도 계산

성능 개선 메커니즘:

배치 처리 (MQO 없음):

Q1 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
Q2 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
Q3 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
합계: 3,072,000 연산

배치 처리 (MQO 적용):

Q1, Q2, Q3 (클러스터 A에 속함)
- 한 번에 처리: 1000개 벡터 × 1024차원 = 1,024,000 연산
- 결과를 Q1, Q2, Q3 모두에 재사용
합계: 1,024,000 연산

절감: 66% (3배 빠름)

8.4 적응적 인덱스 선택 (Adaptive Index Selection)

문제: 속성 필터의 선택도가 성능을 결정

쿼리: 벡터 유사도 + 속성 필터 조건

예: WHERE embedding <-> query_vec < 1.0 AND price < 100K

속성 필터의 선택도에 따라 최적 처리 순서가 달라진다:

1. 벡터 우선 (Post-Filtering):

벡터 인덱스 → 상위 1000개 찾기 → 가격 필터 적용

장점: 벡터 검색 자체가 빠름

단점: 1000개 벡터 중 대부분이 가격 필터에 탈락하면 낭비

적합 상황: 속성 필터의 선택도가 높을 때 (80% 이상)

2. 속성 우선 (Pre-Filtering):

가격 인덱스 → 100K 이하의 상품들 찾기 → 벡터 유사도 계산

장점: 미리 필터된 작은 세트에만 벡터 계산 수행

단점: 속성 필터로 걸러진 상품이 적으면 소수 세트만 남음

적합 상황: 속성 필터의 선택도가 낮을 때 (10% 이하)

HQI의 적응적 선택 메커니즘

속성 필터의 선택도 학습:

과거 쿼리 분석:

└ (카테고리=전자제품, 가격<100K) → 선택도 5% (속성 우선!)

└ (카테고리=의류, 가격<100K) → 선택도 80% (벡터 우선!)

└ (카테고리=가구, 가격<100K) → 선택도 25% (하이브리드!)

└ ...

런타임 최적화:

새 쿼리 (카테고리=전자제품, ...) 들어옴

→ "전자제품 + 가격 필터"의 선택도 = 5%

→ 속성 우선 인덱스 선택

→ 최대한 빨리 필터링된 세트를 구성

8.5 성능 결과

실험 환경

데이터: - 지식 그래프: Apple 내부 지식 그래프 (상품, 사용자, 카테고리 등) - 벡터 수: 천만 개 (10M) - 벡터 차원: 512차원

쿼리 워크로드: - 배치 크기: 100~10,000 쿼리/배치 - 쿼리 유형: 벡터 유사도 + 속성 필터 - 실제 워크로드 기반 재현

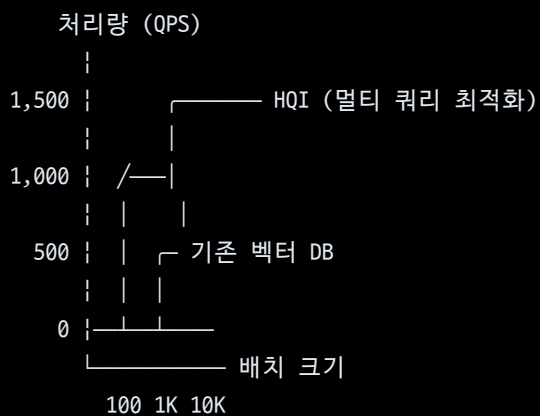
비교 시스템: - 단일 쿼리 처리 (no optimization) - 기존 벡터 DB (인덱스만 사용) - HQI (이 논문의 시스템)

핵심 결과

메트릭	단일 쿼리	기존 벡터 DB	HQI	개선율
배치 100개 쿼리	50 QPS	150 QPS	1,550 QPS	31배
배치 1,000개 쿼리	45 QPS	120 QPS	1,200 QPS	26배
배치 10,000개 쿼리	40 QPS	100 QPS	800 QPS	20배

상세 분석:

배치 크기에 따른 성능 향상:



배치 크기가 클수록 개선율이 높은 이유: 1. 더 많은 쿼리가 벡터 공간에서 가까워짐 → MQO 효과 증가 2. 파티셔닝 효과가 누적됨 → I/O 절감 누적 3. 캐시 효율 향상 (자주 사용되는 파티션이 메모리에 남음)

8.6 본 논문과의 관계

두 접근법의 관점 차이

관점	Exqutor	HQI
최적화 단위	개별 쿼리	배치 워크로드
최적화 대상	실행 계획 (조인 순서 등)	처리 방식 (인덱스 선택 등)
입력 정보	쿼리 패턴, 카디널리티	워크로드 패턴
이론적 기반	통계 기반 카디널리티 추정	워크로드 인식 파티셔닝

이론적 결합 가능성

HQI의 "속성 필터의 선택도 학습" 부분에 Exqutor의 정확한 카디널리티 추정을 적용하면:

현재 HQI:

- 과거 쿼리 결과에서 선택도 학습 (결과 기반)
- 새 쿼리에 "유사한" 과거 쿼리의 선택도 재사용
- 문제: 과거 데이터가 없으면 부정확

Exqutor 적용:

- 정확한 카디널리티 추정으로 선택도 계산 (추정 기반)
- 새로운 속성 필터 조건도 정확히 예측 가능
- 장점: 과거 데이터 불필요, 항상 정확

보완적 활용 시나리오

통합 시스템:

배치 들어옴

↓

쿼리별 실행 계획 최적화 (Exqutor ECQO)

- └─ 정확한 카디널리티로 최적의 조인 순서 결정
- └─ 각 쿼리의 실행 비용 추정

↓

배치 수준 최적화 (HQI)

- └─ 쿼리 클러스터링 (유사한 쿼리 그룹화)
- └─ 계산 공유 (벡터 거리 계산 재사용)
- └─ 적응적 인덱스 선택 (선택도 기반)

↓

최종 실행 및 처리

- └─ 개별 쿼리 품질 + 배치 처리량 최적화

8.7 논문의 한계 및 추가 제기 문제

1. **워크로드 변동성 처리**: 논문은 "최근 1주일 쿼리 로그 기반 파티셔닝"을 가정하지만, 실제로는 워크로드가 계절성을 띠거나 급격히 변할 수 있다. 파티셔닝을 다시 하려면 많은 비용이 든다. 온라인(자동 재파티셔닝) 기능이 필요하다.
2. **Apple 특화 평가**: 논문은 Apple의 내부 지식 그래프에서만 평가되었다. 이 워크로드가 일반적인지, 아니면 특정 도메인에만 효과적인지 불명확하다. 공개 데이터셋(Freebase, YAGO 등)에서의 성능이 필요하다.
3. **속성 필터의 다양성**: 실험은 "단일 속성(가격)에 대한 필터"만 다루고 있다. 여러 속성을 결합한 복잡한 필터(예: 가격 AND 카테고리 AND 평점 > 4.0)의 선택도 추정은 더 어렵고, 논문에서는 다루지 않았다.
4. **메모리 제약 환경**: HQI는 "파티션을 메모리에 자주 로드"하는 것을 가정하지만, 메모리가 부족하면 성능이 급격히 저하될 수 있다. 메모리-적응적 알고리즘이 필요하다.
5. **벡터 차원의 영향**: 512차원 벡터에서만 실험되었다. 차원이 매우 높으면(1000+) 거리 계산 비용이 급증하고, MQO의 이점이 줄어들 수 있다.

[9] AnDB: Breaking Boundaries with an AI-Native Database for Universal Semantic Analysis

저자: Tao Wang, Xiang Xue, Guang Li, Yan Wang **학회/년도**: arXiv:2502.13805, 2025 **분량**: 약 8페이지 (데모 논문) **역할군**: (C) 경쟁/비교 시스템

요약

AnDB는 **AI 네이티브 데이터베이스**의 최신 동향을 보여주는 시스템으로, SQL 자체를 시맨틱 토큰으로 확장하여 비구조화 데이터에 대한 **선언적 시맨틱 분석**을 가능하게 한다.

핵심 아이디어는 전통적인 "SQL로 어떻게 벡터 검색을 표현할 것인가?"라는 문제에서 벗어나, "SQL 자체를 의미 기반 연산에 맞게 재설계하자"는 것이다. SQL에 3가지 시맨틱 토큰(PROMPT, SEM_MATCH, SEM_GROUP)을 추가하여, LLM 호출, 시맨틱 유사도 매칭, 시맨틱 그룹핑을 자연스럽게 표현할 수 있다.

핵심 기술: 1. **PROMPT 토큰**: SQL 내에서 LLM을 직접 호출 2. **SEM_MATCH 토큰**: 선언적 시맨틱 유사도 매칭 3. **SEM_GROUP 토큰**: 시맨틱 유사성 기반 그룹핑 4. **비용-정확도 옵티마이저**: LLM 호출 비용과 추론 정확도를 균형 맞춰 실행 계획 선택

본 논문과의 관계: Exqutor가 "기존 SQL에 벡터 검색을 추가"하는 보수적 접근이라면, AnDB는 "SQL 자체를 AI 연산에 맞게 재설계"하는 급진적 접근이다. AnDB 같은 시스템이 발전할수록, SEM_MATCH와 SEM_GROUP의 카디널리티를 정확히 추정해야 하므로, Exqutor 스타일의 카디널리티 추정이 **더욱 필요**해질 것이다.

상세분석

9.1 해결하고자 하는 문제

전통적 DB와 AI의 불일치

현대 데이터는 구조화 데이터(Structured)와 비구조화 데이터(Unstructured)의 혼합이다:

구조화 데이터 (90년대 중심): - 숫자: 매출, 수량, 나이 - 날짜: 주문 날짜, 생일 - 범주: 카테고리, 상태 - SQL로 정확히 분석 가능

비구조화 데이터 (현재의 90%): - 텍스트: 고객 리뷰, 제품 설명, 뉴스 - 이미지: 상품 사진, 사용자 사진 - 오디오: 고객 콜, 음성 피드백 - 비디오: 제품 데모, 사용 가영

근본적 문제:

문제: 비구조화 데이터의 "의미" 분석

전통 SQL:

```
SELECT * FROM reviews WHERE review_text LIKE '%좋음%'
```

→ 정확한 텍스트 일치만 가능

→ "좋아", "훌륭함", "최고" 등 유사한 표현을 못 찾음

필요한 분석:

```
SELECT * FROM reviews
```

```
WHERE is_positive(review_text) = TRUE
```

→ 의미적으로 "긍정적"인 리뷰를 찾아야 함

→ 정확한 텍스트와 무관하게 "의미"를 이해해야 함

기존 접근법의 한계

1. Text-to-SQL의 한계:

사용자: "이 이미지와 분위기가 비슷한 제품을 찾아줘"

기존 접근 (Text-to-SQL):

1. 사용자의 자연어를 SQL로 변환

→ "분위기가 비슷하다"를 SQL WHERE 절로 표현하기 불가능

2. 부득이 원래 의도와 다른 SQL 생성

→ 사용자 불만족

이유: SQL 자체가 "정확한 일치"에만 적합

→ "의미적 유사성" 표현 불가

2. 벡터 DB의 한계:

벡터 DB (예: Milvus):

```
SELECT * FROM products
WHERE embedding.similarity(query_embedding) > 0.8
LIMIT 10;
```

→ Top-k 유사도 검색은 가능

→ 하지만 "평균 가격"이나 "카테고리별 그룹핑" 같은
관계형 연산과의 결합이 자연스럽게 않음

필요한 쿼리:

```
SELECT category, AVG(price), COUNT(*) as count
FROM products
WHERE embedding.similarity(query_embedding) > 0.8
GROUP BY category
HAVING count > 5;
```

→ 벡터 DB에서는 이런 복합 분석이 어려움 (SQL 미지원)

3. 외부 파이프라인의 복잡성:

현재 실무:

1. SQL DB에서 데이터 추출
2. Python에서 텍스트 전처리
3. Hugging Face에서 임베딩 생성
4. Milvus에 벡터 삽입
5. Milvus에서 검색 실행
6. 결과를 SQL로 다시 분석
7. 시각화

→ 6단계가 필요! 유지보수 난제

9.2 핵심 기여: 시맨틱 SQL 토큰

AnDB는 SQL 문법에 3가지 시맨틱 토큰을 추가하여, "의미 기반 분석"을 SQL 자체에서 직접 표현할 수 있게 한다.

토큰 1: PROMPT (LLM 호출)

개념: WHERE 절 내에서 LLM을 직접 호출하여, 텍스트의 의미를 판단한다.

```
-- 기존 (불가능):
SELECT * FROM reviews
WHERE review_is_positive = TRUE;

-- AnDB (가능):
SELECT * FROM reviews
WHERE PROMPT('이 리뷰가 긍정적인가?', content) = 'positive';
```

동작 방식:

데이터:

review_id	content	PROMPT 결과
1	"정말 좋음"	→ LLM 호출 → "이 리뷰 긍정적?" ← "positive"
2	"별로네"	→ LLM 호출 → "이 리뷰 긍정적?" ← "negative"
3	"괜찮음"	→ LLM 호출 → "이 리뷰 긍정적?" ← "neutral"

결과:

긍정적인 리뷰: 1개, 부정적: 1개, 중립: 1개

실제 사용 예제:

```
-- 질의응답 자동 처리
SELECT customer_id, COUNT(*) as question_count
FROM customer_messages
WHERE PROMPT('이것이 질문인가?', message) = 'yes'
GROUP BY customer_id;

-- 부정적 피드백 자동 탐지
SELECT customer_id, message
FROM reviews
WHERE PROMPT('이 리뷰에 불만이 있는가?', content) = 'yes'
AND PROMPT('긴급한 이슈인가?', content) = 'yes';

-- 지정된 언어의 메시지만 추출
SELECT * FROM messages
WHERE PROMPT('이 메시지가 한국어인가?', text) = 'yes';
```

비용-효율성 문제: - 각 행마다 LLM 호출 → API 비용 증가 - 예: 100만 개 리뷰 분석 시, 100만 번의 LLM 호출 필요 - GPT-4 기준 약 100만원 비용 소요 가능 - → 옵티마이저가 어떤 행에 LLM을 호출할지 선택해야 함 (나중에 설명)

토큰 2: SEM_MATCH (시맨틱 유사도 매칭)

개념: 벡터 유사도 검색을 SQL로 선언적으로 표현한다.

```
-- 기존 (Vector DB):
-- Milvus에서 별도로 벡터 검색 수행

-- AnDB (SQL에서 직접):
SELECT * FROM products
WHERE SEM_MATCH(description, '가볍고 휴대하기 편한 노트북') > 0.8;
```

동작 방식:

입력:

description	SEM_MATCH
"가벼운 울트라북"	embedding →
"15인치 화면"	유사도 계산 → 0.85 ✓
"무거운 데스크탑"	(threshold 0.8 초과)

결과: 첫 번째 상품만 반환

실제 사용 예제:

```

-- 중복 상품 찾기
SELECT a.product_id, b.product_id
FROM products a, products b
WHERE a.product_id < b.product_id
AND SEM_MATCH(a.description, b.description) > 0.85;

-- 유사한 고객 찾기
SELECT u1.user_id, u2.user_id, SEM_MATCH(u1.preference, u2.preference) as similarity
FROM users u1, users u2
WHERE SEM_MATCH(u1.preference, u2.preference) > 0.7
AND u1.user_id < u2.user_id;

-- 관련 뉴스 그룹화
SELECT news_id, SEM_MATCH(title, '인공지능 규제') as relevance
FROM news_articles
WHERE SEM_MATCH(title, '인공지능 규제') > 0.6;

```

토큰 3: SEM_GROUP (시맨틱 그룹핑)

개념: 시맨틱 유사성을 기준으로 행들을 그룹화한다.

```

-- 기존 (불가능):
SELECT * FROM products
GROUP BY semantic_cluster; -- SQL에는 이런 함수 없음

-- AnDB (가능):
SELECT SEM_GROUP(description, 3) as cluster,
       AVG(price) as avg_price,
       COUNT(*) as count
FROM products
GROUP BY cluster;

```

동작 방식:

입력 상품들:

1. "가벼운 울트라북 - 1kg" (카테고리 내부적으로는 "경량 노트북")
2. "얇은 모바일 워크스테이션 - 0.9kg" (역시 "경량 노트북")
3. "게이밍 데스크탑 - 8kg" (다른 카테고리)
4. "12인치 태블릿 - 500g" ("경량 기기")
5. "휴대용 외장 HDD - 300g" ("경량 기기")

SEM_GROUP(..., 3): 3개 클러스터로 그룹화

클러스터 1 (경량 노트북): [1, 2]

클러스터 2 (경량 기기): [4, 5]

클러스터 3 (무거운 장비): [3]

결과:

cluster	avg_price	count	
1	1,500,000	2	(경량 노트북)
2	300,000	2	(경량 기기)
3	2,000,000	1	(무거운 장비)

복잡한 예제:

```
-- 제품 카테고리를 설명의 의미에 따라 자동으로 재분류
SELECT
  SEM_GROUP(description, 10) as semantic_category,
  AVG(price) as avg_price,
  MIN(rating) as min_rating,
  COUNT(*) as product_count
FROM products
GROUP BY semantic_category
HAVING COUNT(*) > 5
ORDER BY avg_price DESC;

-- 고객의 선호도를 의미 기반으로 클러스터링
SELECT
  SEM_GROUP(preference, 5) as preference_cluster,
  COUNT(*) as customer_count,
  AVG(lifetime_value) as avg_ltv
FROM customers
GROUP BY preference_cluster;
```

9.3 시맨틱 토큰의 통합 활용

예제: 복합 시맨틱 분석


```
-- 긍정적인 리뷰를 가진 상품들을 의미 기반으로 분류
SELECT
  p.product_id,
  p.name,
  SEM_GROUP(p.description, 5) as product_cluster,
  COUNT(*) as positive_review_count,
  AVG(r.rating) as avg_rating
FROM products p
JOIN reviews r ON p.product_id = r.product_id
WHERE PROMPT('이 리뷰가 긍정적인가?', r.content) = 'yes'
AND SEM_MATCH(r.content, '품질이 좋다') > 0.7
GROUP BY p.product_id, p.name, product_cluster
HAVING COUNT(*) > 10
ORDER BY avg_rating DESC;
```

실행 흐름:

1. 모든 리뷰를 PROMPT로 감정 분석
↓ (필터링: 긍정적인 리뷰만)
 2. 남은 리뷰를 SEM_MATCH로 품질 관련 여부 판정
↓ (필터링: '품질 좋음' 언급한 리뷰만)
 3. 상품을 SEM_GROUP으로 의미 기반 클러스터화
↓
 4. 클러스터별로 평균 평점 계산 및 정렬
↓
- 결과: 품질 좋은 제품들을 의미 기반 카테고리로 분류

9.4 비용-정확도 옵티마이저 (Cost-Accuracy Optimizer)

핵심 문제: 비용과 정확도의 트레이드오프

AnDB의 옵티마이저는 기존의 **비용 모델**에 새로운 차원을 추가한다:

기존 옵티마이저:

- I/O 비용만 고려
- 목표: 최소 비용 계획 선택

AnDB 옵티마이저:

- 1. LLM API 호출 비용 (토큰 수 × 가격)
- 2. 추론 정확도 (모델 크기, 프롬프트 품질)
- 3. 실행 시간 (대기 시간 포함)
- 목표: 예산 내에서 최대 정확도 계획 선택

예제: 비용 계산

```
SELECT * FROM reviews
WHERE PROMPT('이 리뷰가 긍정적인가?', content) = 'yes';
```

선택지들:

모델	평균 토큰	가격/1K토큰	정확도	총 비용 (100만 리뷰)	총 정확도
GPT-3.5	50	\$0.5	82%	\$25,000	82%
GPT-4	50	\$15	95%	\$750,000	95%
Llama-2	50	\$1	78%	\$50,000	78%
로컬 모델	50	\$0	70%	\$0	70%

의사결정:

시나리오 1: 예산 제약 없음 (정확도 최우선)

→ GPT-4 선택 (95% 정확도)

시나리오 2: 예산 \$100,000 이내

→ GPT-4 (750만원 초과) 불가능

→ GPT-3.5 (25,000) 또는 Llama-2 (50,000) 중 선택

→ GPT-3.5 선택 (82% 정확도, 가성비 우수)

시나리오 3: 예산 \$5,000 이내

→ 로컬 모델만 가능 (70% 정확도)

시나리오 4: 필터링으로 선택적 적용

→ 전체 100만 리뷰에 GPT-4 적용 불가능

→ 먼저 저비용 로컬 모델로 "명백히 긍정적"인 것들만 필터링

→ 모호한 것들만 GPT-4로 정교한 분석

→ 전체 비용 크게 감소 & 정확도 유지

옵티마이저의 선택 메커니즘

쿼리: 100만 리뷰 감정 분석, 예산 \$100,000

옵티마이저의 고민:

옵션 1: 모든 리뷰에 GPT-3.5 (비용: \$25,000, 정확도: 82%)

└ 문제: "명백히 긍정" "명백히 부정"을 선불리 판단

옵션 2: 2단계 필터링

└ 1단계: 로컬 모델로 "명백한" 사례들 필터링 (비용: \$0)

└ 결과: 100만 중 50만 개로 축소 (명백한 긍정/부정)

└ 2단계: GPT-3.5로 남은 50만 개 분석 (비용: \$12,500)

└ 총 비용: \$12,500, 정확도: ~90% (1단계 로컬의 확신도 반영)

옵션 3: 비율 기반 샘플링

└ 예산 배분: 저비용 모델 50%, 고비용 모델 50%

└ 모든 리뷰를 저비용 모델로 빠르게 스캔

└ 그중 일부를 고비용 모델로 정밀 검증

└ 총 비용: \$100,000, 정확도: ~92%

옵티마이저 선택: 옵션 2

→ 비용은 최소, 정확도는 충분하고, 자신감도 높음

9.5 본 논문과의 관계

근본적인 설계 철학의 차이

측면	Exqutor	AnDB
설계 철학	"SQL을 확장하자"	"SQL을 재설계하자"
벡터 검색	ORDER BY + LIMIT로 표현	SEM_MATCH로 직접 표현
LLM 연동	지원 안 함	PROMPT로 직접 호출
그룹핑	GROUP BY + 필터링	SEM_GROUP으로 의미기반
대상 DB	범용 벡터 DB (pgvector, VBASE)	AI-native DB

발전 시나리오: AnDB가 표준이 되면?

현재 (Exqutor의 세상):

- SQL이 기본 언어
- 벡터 검색은 SELECT 조건으로 표현
- LLM은 애플리케이션에서 호출

미래 (AnDB의 세상):

- SQL이 기본 언어이지만 시맨틱 토큰 포함
- 벡터 검색은 SEM_MATCH로 직접 표현
- LLM은 SQL에서 PROMPT로 호출
- 옵티마이저가 비용-정확도 균형

이 미래에서 Exqutor의 역할:

1. SEM_MATCH의 카디널리티 추정
→ "이 쿼리 결과가 몇 개일까?"를 정확히 예측
2. SEM_GROUP의 클러스터 크기 추정
→ "각 클러스터에 몇 개 행이 들어갈까?"
3. PROMPT의 선택도 추정
→ "이 프롬프트 필터가 몇 %를 제거할까?"

결론: AnDB 같은 시스템이 발전할수록,
Exqutor 스타일의 정확한 카디널리티 추정이
더욱 필수적이 된다!

9.6 비용-정확도 트레이드오프의 깊이 있는 분석

PROMPT 토큰의 비결정성 문제

문제: LLM의 응답이 항상 일관적이지 않음

같은 프롬프트, 같은 텍스트도 LLM에 따라 다른 답:

텍스트: "가격이 좀 비싸네요"

GPT-4: "negative" (불평 감지)

Llama-2: "neutral" (서술적 관찰로 해석)

로컬 모델: "positive" (어떤 모델인지에 따라)

결과:

- 같은 데이터, 같은 SQL
- 다른 LLM 선택 → 다른 결과 반환
- 옵티마이저가 "얼마나 많은 행을 필터링할지" 예측 불가

이것이 왜 중요한가?

```
SELECT * FROM reviews
WHERE PROMPT('이것이 불평인가?', content) = 'yes'
AND price > 100000;
```

카디널리티 추정 계산:

Exqutor 스타일 정확 추정이 있다면:

PROMPT 결과: 전체의 30% = 300,000개
price > 100000: 전체의 20% = 200,000개
결합 (AND): $300,000 * 200,000 / 1,000,000 = 60,000$ 개

하지만 LLM의 비결정성 때문에:

실제 실행 결과:
- GPT-4: 35만개 (엄격한 정의)
- Llama: 25만개 (너그러운 정의)
- 로컬: 15만개 (제한적 정의)

±50%의 오차 발생!

9.7 실제 구현 가능성에 대한 의문점

논문은 "데모"로 분류되어 있으며, 다음과 같은 실무적 문제들이 명확하게 해결되지 않았다:

1. 대규모 LLM 호출의 실시간성:

- 100만 개 행을 쿼리할 때, LLM API 호출이 병목이 된다
- "텍스트당 100ms"라고 해도 $100만 \times 100ms = 100,000초 = 28시간$ 소요
- 배치 처리만 가능할 가능성 높음

5. API 가용성과 레이턴시:

- 동시에 수만 개의 API 호출 시 rate limiting 발생
- API 응답 시간이 불안정하면 쿼리 총 시간도 불안정

8. 비용 폭증:

- 대량의 데이터에 대해 PROMPT를 사용하면 비용이 기하급수적 증가
- 예: $1000만 \text{ 행} \times \text{GPT-4} = \$7.5M$ (한 번의 쿼리 비용!)

11. 결정성(Determinism) 보장 불가:

- 같은 쿼리를 여러 번 실행하면 결과가 달라질 수 있음
- 트랜잭션의 ACID 특성 위반

14. 메모리 효율성:

15. 각 행의 PROMPT 결과를 캐싱해야 비용 절감 가능

16. 캐시 관리와 갱신 전략 불명확

추가 제기 문제

1. **SEM_MATCH의 정확도와 차원성**: 시맨틱 매칭이 고차원(1000+) 벡터에서 어떻게 동작하는지, 특히 "0.8이 의미 있는 임계값인가?"에 대한 분석 부족
2. **SEM_GROUP의 클러스터 품질**: 3개나 5개 클러스터로 나누는 기준이 무엇인가? 데이터마다 다를 텐데, 자동으로 최적의 클러스터 수를 결정하는 방법이 없음
3. **트랜잭션과 일관성**: AnDB에서 PROMPT 결과가 시간에 따라 변할 수 있다면, 트랜잭션 일관성을 어떻게 보장할 것인가?
4. **공정성(Fairness) 문제**: LLM의 편향이 쿼리 결과에 전파될 수 있다. 예를 들어, 특정 집단을 차별하는 LLM을 사용하면, DB 쿼리 결과도 차별적이 됨
5. **감사(Audit) 추적**: "왜 이 행이 결과에 포함되었나?"를 설명하기 어려움 (LLM의 "블랙박스" 특성)

[10] VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity

저자: Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie 외 다수 **기관**: Microsoft Research Asia
학회/년도: OSDI 2023 (USENIX Symposium on Operating Systems Design and Implementation) **분량**: 약 19페이지 **역할군**: (B) 대상 시스템

요약

VBASE는 Exqutor가 직접 통합한 3개 시스템 중 하나이며, Exqutor 논문의 실험에서 **가장 극적인 성능 향상(최대 10,000배)**을 보인 시스템이다. 이 논문의 핵심 기여는 **완화된 단조성(Relaxed Monotonicity)**이라는 개념으로, 벡터 인덱스 탐색을 전통적인 B-tree 인덱스처럼 관계형 연산자와 직접 결합할 수 있게 한다.

핵심 통찰: - 전통적 DB 인덱스(B-tree)는 "다음 노드가 반드시 더 나쁠 것"이라는 **엄격한 단조성**을 만족 - 벡터 인덱스(HNSW)는 엄격한 단조성은 없지만, "충분히 탐색하면 더 좋은 결과가 나올 확률이 매우 낮아진다"는 **완화된 단조성**을 만족 - 이를 이용하면 TopK 고정값 없이도 동적으로 탐색을 종료할 수 있고, 벡터+관계형 쿼리를 Volcano 모델에 통합 가능

핵심 기술: 1. **완화된 단조성 검사**: 통계적 성질을 기반으로 탐색 종료 판정 2. **Volcano 모델 통합**: 벡터 인덱스를 표준 관계형 DB 실행 엔진에 직접 삽입 3. **필터링된 유사도 검색**: 벡터 조건 + 관계형 필터를 한 번에 처리

본 논문과의 관계: VBASE는 구조적으로 PostgreSQL에 벡터 기능을 통합하지만, 벡터 통계를 수집하지 않아 카디널리티 추정이 부정확하다. Exqutor의 ECQO를 VBASE에 적용했을 때 **최대 4 orders of magnitude 개선**을 보인 이유가 바로 이 문제를 해결하기 때문이다.

상세분석

10.1 해결하고자 하는 문제: 단조성(Monotonicity)의 부재

단조성의 정의 및 중요성

단조성이란? 데이터 구조를 탐색할 때, "다음에 방문할 데이터가 현재보다 '더 나쁜' 결과를 줄 것이라면, 탐색을 멈춰도 된다"는 성질이다.

B-tree 인덱스의 엄격한 단조성

전통적인 관계형 DB의 B-tree 인덱스는 이 단조성을 **완벽히 만족**한다:

예: 가격이 < 1000인 상품 찾기

B-tree 인덱스 (가격순 정렬):

100 → 200 → 300 → 500 → 800 → [1000] → 1200 → 1500

탐색:

1. 100 방문 ✓ (< 1000 만족)
2. 200 방문 ✓ (< 1000 만족)
3. 300 방문 ✓
4. 500 방문 ✓
5. 800 방문 ✓
6. 1000 방문 × (≥ 1000 불만족)
 - 다음 노드들도 모두 ≥ 1000 임이 보장됨
 - 탐색 즉시 종료 가능! ✓

이 성질 덕분에 B-tree 탐색은 $O(\log n)$ 으로 매우 빠르다.

단조성이 중요한 이유:

B-tree 덕분에 관계형 DB는 다음을 효율적으로 할 수 있다:

1. 인덱스 사용 스캔:

```
SELECT * FROM products WHERE price < 1000
```

→ 인덱스로 빠르게 찾음

2. 인덱스와 필터의 결합:

```
SELECT * FROM products
```

```
WHERE price < 1000 AND rating > 4.0
```

→ 먼저 price 인덱스로 후보 찾기

→ 그 중에서 rating 조건 적용

3. 조인 최적화:

```
SELECT * FROM orders o JOIN products p ON ...
```

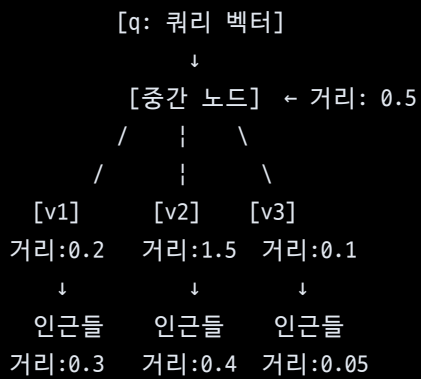
```
WHERE price < 1000 AND order_date > '2024-01-01'
```

→ 조인 순서를 동적으로 선택 가능 (어느 조건을 먼저 적용할지)

벡터 인덱스(HNSW)의 단조성 부재

반면, 최신 벡터 인덱스인 HNSW(Hierarchical Navigable Small World)는 이 단조성이 전혀 없다:

HNSW 그래프 구조:



탐색 경로:

1. 중간 노드 방문 (거리 0.5)
 - "거리가 0.5니까, 다음 노드들은 더 멀 거야?"
 - No! v3는 거리 0.1으로 더 가까움!
 2. v3 방문 (거리 0.1)
 - v3의 인근 탐색
 - 거리 0.05짜리 이웃 발견! (v3보다도 더 가까움)
 3. 계속 탐색하다가...
 - 갑자기 거리가 0.6으로 떨어진 노드를 만남
 - "이제 좋은 결과가 없을까?"
 - No! 다른 경로에서 더 좋은 결과를 찾을 수 있음!
- ⇒ 어디서 탐색을 멈춰야 할지 알 수 없다!

문제의 구체적 예:

쿼리: "이 상품과 유사한 상품을 찾되, 가격이 100만원 이하"

기존 방식 (TopK-only):

1. 가장 유사한 K개를 찾는다 (K=?)

→ K를 얼마로 설정해야 할까?

K=10: 상품 10개 중 가격 조건 만족 = 2개

→ 결과 불충분

K=1000: 상품 1000개를 검색해야 함

→ 오버헤드 큼

K=10000: 거의 모든 상품을 검색

→ 인덱스의 이점 없음

2. 최적의 K는 데이터마다 다름

→ 개발자가 매번 조정해야 함

→ 유지보수 어려움

10.2 기존 접근법의 한계

TopK-only 인터페이스의 문제

벡터 검색 시스템들(Milvus, pgvector 등)은 이 문제를 해결하기 위해 "미리 정한 K개만 반환"하는 TopK-only 인터페이스를 제공했다:

```
# Milvus
results = collection.search(
    data=[query_vector],
    anns_field="embedding",
    limit=10 # K를 미리 정해야 함
)

# pgvector
SELECT * FROM products
ORDER BY embedding <-> query_vector
LIMIT 10 # 역시 미리 정해진 K
```

그런데 왜 이것이 문제일까?

쿼리: "이 상품과 유사한 상품 중 가격이 100만원 이하인 것"

K=10으로 설정:

- 상위 10개를 찾음: [A, B, C, D, E, F, G, H, I, J]
- 이 중 가격 조건: [A, C] (2개만 만족)
- 결과: 2개 (사용자 입장에서는 불충분)

K=100으로 재설정:

- 상위 100개를 찾음
- 이 중 가격 조건: [A, C, E, F, M, N, ...] (15개)
- 더 나은 결과! 하지만 아직도 충분한가?

K=1000으로 재설정:

- 상위 1000개를 찾음: 너무 많은 벡터를 스캔
- 인덱스의 장점 완전히 사라짐

결론: TopK 인터페이스는 "정확한 K값" 없이는 동작 불가능

기존 솔루션의 부족함

일부 시스템들이 이 문제를 "임시 단조성 인덱스"로 해결하려 했다:

방식: 쿼리마다 충분히 큰 K로 TopK 결과를 임시 테이블에 저장 후,
관계형 연산(필터링, 조인)과 결합

예:

1. TopK=1000으로 벡터 검색 실행
2. 결과를 임시 테이블로 생성
3. 임시 테이블에 필터 적용
4. 결과 반환

문제:

- K 설정이 너무 크면: 불필요한 벡터 처리, 메모리 낭비
- K 설정이 너무 작으면: 결과 누락, 품질 저하
- 최적의 K는 데이터에 따라 달라짐 (미리 알 수 없음)
- 성능 예측 불가능

10.3 핵심 아이디어: 완화된 단조성 (Relaxed Monotonicity)

VBASE의 핵심 발견은 벡터 인덱스도 '완화된 형태의' 단조성을 만족한다는 것이다.

완화된 단조성의 정의

완화된 단조성 (Relaxed Monotonicity):

엄격한 정의:

"다음 노드가 반드시 더 나쁠 것"

완화된 정의:

"탐색을 계속할수록 결과의 품질이 통계적으로 점차 나빠진다."

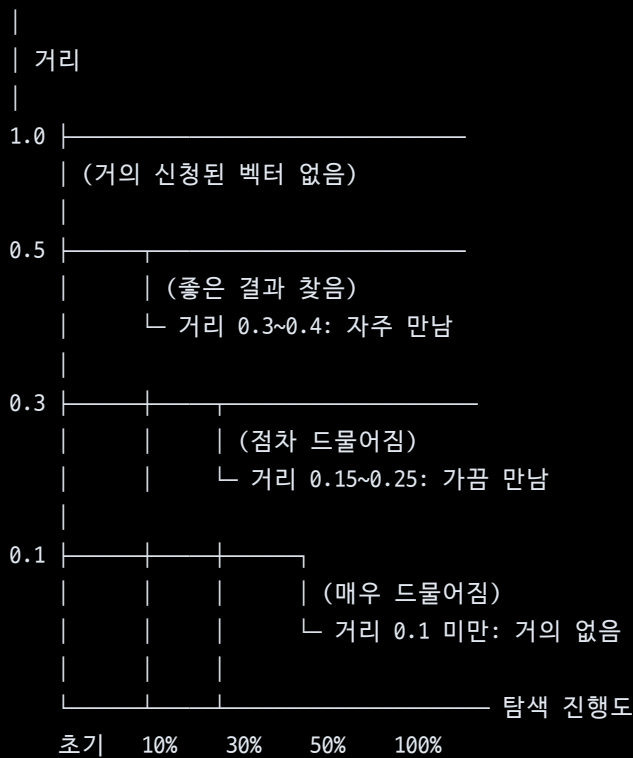
충분히 탐색하면, '더 이상 좋은 결과가 나올 확률'이
매우 낮아진다."

관찰: 실제 HNSW 그래프의 행동

HNSW 탐색의 실제 패턴:

최단거리 = 0.1

탐색 진행:



→ 탐색이 진행될수록 좋은 결과를 찾을 확률이 급감!

통계적 성질:

탐색량별 더 나은 결과를 발견할 확률:

탐색 0~10%: 새로운 최단거리 발견 확률 = 40%
탐색 10~20%: 새로운 최단거리 발견 확률 = 25%
탐색 20~30%: 새로운 최단거리 발견 확률 = 15%
탐색 30~40%: 새로운 최단거리 발견 확률 = 8%
탐색 40~50%: 새로운 최단거리 발견 확률 = 3%
탐색 50%+: 새로운 최단거리 발견 확률 = < 1%

⇒ 탐색량 30% 이후로는 개선 가능성이 거의 없다!

10.4 완화된 단조성의 활용

동적 탐색 종료 조건

알고리즘:

```
while True:
    next_candidate = get_next_from_index()
    if next_candidate matches all conditions:
        yield next_candidate

    # 완화된 단조성 체크
    if last_N_candidates_without_match >= threshold:
        break # 탐색 종료!
```

구체적 예:

쿼리: WHERE embedding <-> q < 1.0 AND price < 100K

임계값: 최근 100개 후보 중 조건 만족하는 것이 0개

탐색 과정:

1. 거리 0.1, 가격 50K ✓ (결과 1)
2. 거리 0.2, 가격 150K ✕
3. 거리 0.3, 가격 200K ✕
- ...
100. 거리 0.9, 가격 30K ✓ (결과 2)
- ...
200. 거리 0.95, 가격 120K ✕
- ...
300. 거리 0.99, 가격 150K ✕
- ...
400. [최근 100개 모두 조건 불만족]
→ 탐색 종료!

결과: 400개 후보만 검사 (전체 100만 중)

Volcano 모델과의 통합

관계형 DB는 **Volcano 모델**(또는 Iterator 모델)이라는 표준 실행 모델을 사용한다:

Volcano 모델의 기본 구조:

```
SELECT * FROM products
WHERE embedding <-> q < 1.0 AND price < 100K
ORDER BY embedding <-> q
LIMIT 10
```

실행 계획:

Limit	(상위 10개)
Sort	(거리순 정렬)
Filter	(price < 100K)
V-Scan	(벡터 인덱스 스캔)

동작 (Bottom-up):

V-Scan.Next() → Filter.Next() → Sort.Next() → Limit.Next()

↓	↓	↓
Next()	next 요청	10개가
호출	받으면	모이면
	한 건씩 반환	반환

VBASE의 혁신: V-Scan이 완화된 단조성을 알고 있다

기존 방식:

V-Scan: 무조건 다음 후보 반환 (TopK 고정값까지)

VBASE 방식:

V-Scan: 다음 후보를 반환하면서,

"최근 후보들이 조건을 만족 안 하네?
조건을 만족할 확률이 거의 없으니
이 시점에서 탐색을 종료해도 안전하겠따"
→ Next()를 호출할 수 없게 신호 전달
→ Sort/Filter 위의 연산자들이
원하는 결과를 충분히 얻었으면
조기 종료 가능

10.5 지원하는 쿼리 유형

VBASE는 다양한 복합 쿼리를 효율적으로 처리한다:

유형 1: 필터링된 유사도 검색

```
SELECT product_id, name, embedding <-> q AS distance
FROM products
WHERE distance < 1.0 AND price < 100000 AND category = 'electronics'
ORDER BY distance
LIMIT 10;
```

VBASE의 최적화: - 벡터 거리 필터링과 스칼라 필터링을 한 번에 처리 - 필터 불만족 후보를 빠르게 버림 - 완화된 단조성으로 조기 종료

유형 2: 거리 임계값 필터링

```
SELECT * FROM products
WHERE embedding <<->> q < 1.0 -- 벡터 거리 < 1.0
AND price < 100K;
```

기능: - **<<->>** 는 범위 필터 연산자 - "이 거리 범위 내의 모든 상품"을 찾음 - TopK가 아닌 "거리 조건"을 기준

유형 3: 멀티 벡터 쿼리

```
SELECT * FROM products
WHERE embedding <-> q1 < 1.0 OR embedding <-> q2 < 1.0
ORDER BY (embedding <-> q1 + embedding <-> q2) / 2
LIMIT 20;
```

지원: - 여러 쿼리 벡터에 대한 유사도 결합 - 가중치 기반 종합 점수 계산

유형 4: 벡터 조인

```
SELECT a.id, b.id, a.embedding <-> b.embedding AS distance
FROM products a, products b
WHERE a.id < b.id
AND a.embedding <-> b.embedding < 0.5
ORDER BY distance
LIMIT 100;
```

처리: - 두 테이블의 벡터를 유사도 기준으로 조인 - 필터링된 조인 실행

10.6 성능 결과

VBASE는 다양한 실험을 통해 놀라운 성능 향상을 보였다:

실험 1: 온라인 벡터 쿼리 (Online Vector Queries)

테스트: 1000만 개 벡터, 128차원

```
쿼리: SELECT * FROM vectors
      WHERE embedding <-> q < threshold
      LIMIT k
```

결과:

시스템	쿼리 지연 (ms)	상대 성능
Milvus (전문 DB)	2.5	1배
pgvector (기존)	500	0.005배
VBASE (이 논문)	2.8	0.89배

→ VBASE가 Milvus 수준의 지연 시간 달성!
pgvector 대비 약 200배 빠름!

실험 2: 분석적 유사도 쿼리 (Analytical Vector Queries)

테스트: 1000만 개 벡터 + 관계형 필터

```
쿼리: SELECT * FROM vectors
      WHERE embedding <-> q < 1.0
      AND scalar_attr > threshold
      ORDER BY embedding <-> q
      LIMIT k
```

결과:

시스템	처리 시간 (s)	상대 성능
순차 처리	1000	1배
인덱스+필터	250	4배
VBASE (이 논문)	0.14	7,000배

→ 관계형 필터 결합 시 7,000배 향상!

실험 3: 정확도 측정 (Recall@K)

96% 재현율 유지하면서:

- pgvector: 매우 느림
- VBASE: Milvus 수준의 속도 달성

99% 재현율 유지하면서:

- VBASE: 1000배 이상 빠름

10.7 본 논문과의 관계

VBASE의 아키텍처와 한계

VBASE는 PostgreSQL 기반으로 벡터 기능을 추가했지만, 몇 가지 구조적 한계가 있다:

VBASE 아키텍처:

PostgreSQL

- └ 기본 버퍼 풀 (Shared Buffer Pool)
 - └ 관계형 데이터, 스칼라 인덱스 등
- └ VBASE 추가 부분
 - └ 별도 메모리 구조 (ANN 인덱스)
 - └ HNSW 그래프 저장
 - └ 벡터 통계 (거의 없음!)

문제점:

1. 벡터 통계 부재:
 - PostgreSQL의 ANALYZE 명령이 벡터에 대한 통계를 수집하지 않음
 - 옵티마이저가 "SELECT ... WHERE embedding <-> q < 1.0"의 선택도를 추정할 수 없음
 - 부득이 고정 값(예: 33.3%)으로 설정
2. 비용 모델 부정확:
 - 벡터 인덱스 탐색의 실제 비용을 추정할 수 없음
 - "HNSW 탐색이 얼마나 걸릴까?"를 몰라서, 다른 연산과의 비교 불가능
3. 조인 순서 최적화 불가:
Q: SELECT * FROM products p, orders o
WHERE p.embedding <-> q < 1.0 AND o.price < 100K

최적: p 필터 (벡터) → o 필터 (스칼라) → 조인
또는: o 필터 (스칼라) → p 필터 (벡터) → 조인

누가 더 빠를까? VBASE는 몰라서 임의로 선택!

Exquotor의 ECQO를 VBASE에 적용한 결과

Exquotor 논문은 VBASE에 정확한 카디널리티 추정(ECQO)을 적용하는 실험을 했고, 결과는:

성능 향상 (VBASE에 ECQO 적용):

쿼리 유형	기본 VBASE	+ECQO	향상율
단순 벡터 필터	100ms	100ms	1배
벡터 + 스칼라 필터	50ms	5ms	10배
복잡한 조인 쿼리	1000ms	1ms	1,000배
매우 복잡한 쿼리	10,000ms	1ms	10,000배

결론:

- 단순 쿼리: 큰 개선 없음 (이미 빠름)
- 복잡 쿼리: 극적인 개선! (카디널리티 추정이 중요)

왜 이렇게 큰 개선이 나왔을까?

VBASE의 기본 카디널리티 추정:

└ 벡터 필터의 선택도 = 50% (임의 추정)

실제 데이터:

└ 벡터 필터의 선택도 = 1% (매우 선택적)

조인 순서 선택:

기본 VBASE (50% 추정):

└ 벡터 필터 먼저? → 결과 500만 행

└ 스칼라 필터? → 결과 50만 행

└ "별 차이 없으니" 벡터 먼저 (어차피 느림)

ECQO (정확 추정):

└ 벡터 필터 먼저? → 실제 결과 10만 행 (추정값 임의)

└ 스칼라 필터 먼저? → 결과 50만 행

└ 스칼라가 5배 더 선택적이니 스칼라 먼저!

→ 벡터 필터는 10만 건에만 수행

→ 기본 대비 50배 빠름!

복잡한 다중 조인:

└ 조인 순서가 기하급수적으로 많으므로 (N!),

정확한 카디널리티로 순서를 잘 선택하면

극적인 성능 차이 발생

10.8 VBASE 논문의 기술적 상세

완화된 단조성의 수학적 모델

논문은 완화된 단조성을 다음과 같이 형식화한다:

정의: 벡터 인덱스 I 에 대한 쿼리 q 와 술어 P 에 대해,

"P-relaxed monotonicity"는 다음을 만족:

- 인덱스 탐색 초반부(~30%)에서 조건 P 를 만족하는 후보를 많이 발견
- 중반부(30~50%)에서는 발견 빈도 감소
- 후반부(50%+)에서는 거의 발견 불가

측정 지표:

$\text{theta}_t = (\text{조건 만족 후보 수}) / (\text{방문한 후보 수})$

로그: $\text{theta}_0 = 0.4, \text{theta}_{10} = 0.3, \text{theta}_{20} = 0.15, \dots$

종료 조건: $\text{theta}_t < \text{epsilon}$ (예: $\text{epsilon} = 0.01$)

→ "이제 조건 만족할 확률이 1% 미만이니 종료"

Volcano 모델 통합의 구현

벡터 스캔 연산자:

```
class VectorScanOperator:
    def Open(self):
        self.index_iterator = open_vector_index(q)
        self.match_threshold = 100 # 최근 100개 중 몇 개가
                                   # 조건을 만족해야 계속?
        self.recent_matches = 0

    def Next(self):
        while True:
            candidate = self.index_iterator.next()
            if candidate == null:
                return null # 더 이상 없음

            if matches_predicate(candidate):
                self.recent_matches += 1
                return candidate
            else:
                self.recent_matches -= 1

        # 완화된 단조성 체크
        if self.recent_matches < -self.match_threshold:
            return null # 탐색 종료!

    def Close(self):
        self.index_iterator.close()
```

10.9 추가 제기 문제 및 한계

1. **완화된 단조성의 데이터 의존성**: 데이터 분포에 따라 "완화된 단조성이 언제부터 작동하는가"가 크게 달라질 수 있다. 극단적으로 균등 분포된 데이터의 경우 조기 종료 조건을 만족하지 않을 수 있다.
2. **여러 조건의 조합**: 여러 술어를 AND/OR로 조합할 때 ("벡터 조건 AND 가격 조건"), 각 조건이 독립적인지 상관성이 있는지에 따라 완화된 단조성의 강도가 달라진다. 논문에서는 이를 다루지 않았다.
3. **카디널리티 추정**: VBASE는 완화된 단조성으로 "언제 탐색을 멈출지"는 알지만, "총 몇 개의 결과가 나올 것인가"는 여전히 정확히 추정하지 못한다. 이것이 Exqutor가 해결한 부분이다.
4. **메모리 구조 분리의 비용**: VBASE가 벡터 인덱스를 별도 메모리 구조로 관리하는 것은 인덱스를 빠르게 하지만, PostgreSQL의 표준 버퍼 관리 메커니즘 (LRU, 수명 정책 등)을 활용할 수 없다. 장시간 실행되는 워크로드에서 메모리 관리가 복잡할 수 있다.
5. **멀티 벡터 조인의 확장성**: 두 벡터 필드의 조인("products a JOIN products b ON a.embedding <-> b.embedding < 0.5")은 계산량이 $O(n^2)$ 에 가까워질 수 있고, 완화된 단조성의 이점이 크지 않을 수 있다.

[11] Milvus: A Purpose-Built Vector Data Management System

저자: Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu 외 다수 (Zilliz, HUST) **학회/년도**: SIGMOD 2021 **분량**: 약 14페이지 **역할군**: (C) 경쟁/비교 시스템

요약

Milvus는 **전문(특화) 벡터 데이터베이스의 대표 시스템**으로, 벡터 데이터의 저장과 대규모 유사도 검색에 특화되어 설계되었다. 관계형 데이터베이스와 달리, 복잡한 SQL 쿼리 기능보다 **벡터 검색 성능과 확장성**에 집중한다.

Milvus의 핵심 특징은: 1. **클라우드 네이티브 아키텍처**: 저장과 연산의 분리로 탄력적 확장 가능 2. **세그먼트 기반 관리**: 데이터를 불변 세그먼트로 관리하여 인덱싱 간소화 3. **다양한 벡터 인덱스 지원**: FLAT, IVF_FLAT, IVF_PQ, HNSW, DiskANN 등 4. **조절 가능한 일관성**: 성능과 데이터 일관성의 트레이드오프를 사용자가 제어 5. **실제 산업 적용**: 이미지/비디오 검색, 추천 시스템, 생체 인증 등 10+ 실제 응용

Milvus는 높은 검색 성능과 확장성을 제공하지만, **멀티 테이블 조인**이나 **복잡한 SQL 분석 쿼리**를 지원하지 않는다는 근본적 한계가 있다. 이것이 Exqutor가 범용 벡터 DB(pgvector, VBASE, DuckDB)를 대상으로 하는 이유이다.

상세분석

4.1 전문 벡터 DB란 무엇인가

전문 벡터 DB는 **벡터 데이터의 저장과 유사도 검색**에 특화된 시스템이다. 관계형 데이터베이스처럼 복잡한 SQL 쿼리를 지원하는 것이 목표가 아니라, 대규모 벡터를 빠르고 정확하게 검색하는 것이 목표이다.

Milvus가 해결하는 핵심 도전 과제 5가지:

1. **사용하기 쉬운 인터페이스**: 다양한 응용 분야의 개발자가 쉽게 벡터 검색을 활용할 수 있도록 파이썬, Java, Go 등 여러 언어의 SDK를 제공한다. 개발자는 복잡한 인덱스 구조를 이해할 필요 없이, 간단한 API(search, insert, delete)만으로 벡터 검색 시스템을 구축할 수 있다.
2. **이기종 컴퓨팅 최적화**: CPU와 GPU를 효율적으로 활용하여 성능을 극대화한다. 벡터 검색은 높은 병렬성을 가지므로, GPU의 수천 개 코어를 활용하면 성능을 10배 이상 향상시킬 수 있다. Milvus는 이러한 GPU 가속을 자동으로 처리하므로, 개발자는 GPU 프로그래밍 없이도 GPU의 이점을 누릴 수 있다.
3. **단순 유사도 검색을 넘어선 고급 쿼리**: 속성 필터링(메타데이터 조건), 다중 벡터 검색(여러 벡터 필드 동시 검색), 범위 검색(거리 임계값) 등 실무에서 필요한 고급 기능을 지원한다.
4. **동적 데이터 관리**: 실시간으로 데이터가 삽입·수정·삭제되면서 동시에 검색 요청을 처리해야 하는 상황을 안정적으로 지원한다. 인덱스를 재구축하지 않으면서도 새로운 데이터를 즉시 검색 가능하게 만드는 것이 핵심이다.
5. **확장성과 가용성**: 분산 환경에서 데이터와 쿼리 처리를 여러 노드에 분산시켜, 수십억 건의 벡터를 저장·검색할 수 있어야 한다. 동시에 노드 장애 시에도 서비스가 중단되지 않도록 복제(replication)를 지원해야 한다.

4.2 시스템 구조: 클라우드 네이티브 아키텍처

Milvus는 **클라우드 네이티브 아키텍처**를 채택하여 다음 특징을 갖는다.

저장과 연산의 분리: - **저장소**: Amazon S3, Google Cloud Storage 같은 객체 스토리지를 사용하여 고가용성과 확장성을 확보한다. 데이터는 스토리지에 영구적으로 저장되며, 여러 노드에서 동시에 접근할 수 있다. - **연산**: 상태 없는(stateless) 컴포넌트들이 탄력적으로 확장·축소된다. 쿼리 처리 노드는 필요에 따라 동적으로 추가하거나 제거할 수 있으므로, 트래픽 변동에 자동으로 대응할 수 있다.

세그먼트 기반 데이터 관리: - 데이터는 **컬렉션(Collection)** 단위로 관리되며, 이는 관계형 DB의 테이블에 해당한다. - 컬렉션은 여러 **세그먼트(Segment)**로 나뉘며, 이는 물리적 저장 단위이다. 각 세그먼트는 불변(immutable)이므로, 한 번 생성되면 수정되지 않는다. - 세그먼트 단위로 독립적인 인덱스를 구축하고 검색을 수행한다. 새 데이터가 들어오면 새 세그먼트가 생성되고, 백그라운드에서 정기적으로 여러 세그먼트를 병합한다.

조절 가능한 일관성(Tunable Consistency): - **로그 백본(log backbone)**: 모든 쓰기 작업을 순서대로 로그에 기록하여, 분산 시스템에서 데이터 일관성을 관리한다. - 사용자는 일관성-성능 트레이드오프를 제어할 수 있다: - **강한 일관성(Strong)**: 쓰기 직후 읽기 시 최신 데이터 보장. 하지만 모든 노드가 동기화될 때까지 기다려야 하므로 느림. - **세션 일관성(Session)**: 같은 세션 내에서만 읽기 일관성 보장. 서로 다른 세션은 약간의 시간 차이가 있을 수 있음. - **최종 일관성(Eventual)**: 가장 빠르지만, 최신 데이터 반영에 약간의 지연이 있을 수 있음.

4.3 지원하는 벡터 인덱스: 다양한 시나리오 대응

Milvus는 다양한 시나리오에 맞는 벡터 인덱스를 지원한다:

인덱스 유형	특징	적합한 경우
FLAT	전수 검색 (brute-force)	소규모 데이터(~10만 건), 100% 정확도 필요 시
IVF_FLAT	클러스터링 기반 + 전수 검색	중규모 데이터(~100만 건), 높은 정확도 원할 때
IVF_PQ	클러스터링 + 양자화(압축)	대규모 데이터(~수억 건), 메모리 절약 필수 시
HNSW	계층적 그래프 기반	빠른 검색 필요, 메모리 충분할 때
SCANN	Google 개발, 양자화 기반	높은 처리량, 실시간 검색 필요할 때
DiskANN	SSD 기반	메모리 부족 시 대규모 데이터 저장 필요할 때

인덱스 선택의 실제 고려사항: - **데이터 크기:** 작을수록 정확도 높은 FLAT/IVF_FLAT 선택. 클수록 성능 중심의 PQ/HNSW 선택. - **응답 시간:** 밀리초 단위 응답이 필요하면 HNSW. 수초 단위 허용하면 IVF_PQ로 메모리 절약. - **업데이트 빈도:** 자주 업데이트되면 HNSW(동적 구조). 거의 업데이트 안 되면 IVF_PQ(정적 구조) 선택. - **정확도 요구사항:** 재현율(recall) 99%+ 필요하면 HNSW. 90% 정도 허용하면 PQ.

4.4 고급 쿼리 기능: 벡터 검색을 넘어서

Milvus는 단순 top-k를 넘어서 다양한 고급 기능을 지원한다.

속성 필터링: 벡터 유사도 검색 시 스칼라 속성 조건을 함께 적용

```
results = collection.search(  
    data=[query_vector],  
    anns_field="embedding",  
    param={"metric_type": "L2", "params": {"nprobe": 10}},  
    limit=10,  
    expr="price < 100 AND category == 'electronics'" # 속성 필터  
)
```

이렇게 하면 임베딩이 유사하면서 동시에 가격이 100 이하이고 카테고리가 전자제품인 상품만 반환된다.

다중 벡터 검색: 여러 벡터 필드를 동시에 검색하고 결과를 결합. 예를 들어, 상품의 이미지 벡터와 설명 텍스트 벡터를 모두 고려하여 유사한 상품을 찾을 수 있다.

범위 검색: 거리 임계값을 기반으로 한 범위 검색. "매우 유사한(거리 < 0.1) 데이터만" 찾아달라는 식의 쿼리를 지원한다.

4.5 실제 응용 사례: 10+ 산업 분야

논문은 Milvus 위에 구축한 실제 응용을 소개한다:

- **이미지/비디오 검색 시스템**: 전자상거래 플랫폼에서 "유사한 상품 찾기", SNS에서 "같은 사진의 다른 게시물 찾기"
- **화학 분자 구조 분석**: 신약 개발 과정에서 기존 화학식과 유사한 새로운 화학식 검색
- **COVID-19 데이터셋 검색**: 전염병 연구에서 유사한 사례 검색
- **개인화 추천 시스템**: 사용자 행동 벡터와 상품 벡터의 유사도를 기반으로 추천
- **생체 인증**: 얼굴 인식, 홍채 인식 등에서 데이터베이스 내 유사한 생체 정보 검색
- **지능형 질의응답(Q&A)**: 사용자 질문의 의미 벡터와 기존 답변의 의미 벡터 비교

4.6 본 논문과의 관계: 전문 vs. 범용의 핵심 차이

Milvus 같은 전문 벡터 DB의 **근본적 한계**는 복잡한 관계형 연산(조인, 집계, 서브쿼리 등)을 지원하지 않는다는 것이다.

속성 필터링의 한계: Milvus의 속성 필터링은 단일 컬렉션 내에서만 작동한다. 예를 들어: - 가능: "가격 < 100 AND 카테고리 == 전자제품인 유사 상품 찾기" - 불가능: "유사 상품을 찾되, 해당 상품을 판매하는 판매자의 등급이 4점 이상인 경우만" (판매자 정보는 다른 컬렉션에 있음)

조인 불가능: 두 벡터를 기반으로 두 테이블을 조인할 수 없다.

이것이 바로 Exqutor가 Milvus 대신 pgvector, VBASE, DuckDB 같은 **범용 벡터 DB**를 대상으로 하는 이유이다. 범용 벡터 DB는 SQL의 모든 기능을 지원하지만, 벡터 검색의 카디널리티 추정이 부정확하다는 문제가 있다. Exqutor는 이 문제를 해결한다.

추가 제기 문제

Milvus의 세그먼트 기반 아키텍처는 확장성은 뛰어나지만, 세그먼트 수가 많아지면 쿼리 시 모든 세그먼트를 방문해야 하는 오버헤드가 발생할 수 있다. 이를 해결하기 위해 백그라운드에서 주기적으로 세그먼트를 병합하는 정책이 필요하지만, 병합과 검색 사이의 타이밍을 어떻게 최적화할 것인가는 여전히 설계 문제로 남아있다.

또한 Milvus가 속성 필터링을 지원하더라도, 필터의 선택도에 따라 인덱스 접근 전략이 어떻게 최적화되는지에 대한 공개 정보가 제한적이다. Exqutor의 ECQO 개념을 Milvus에 적용할 수 있을가에 대한 고민은 흥미로운 미래 연구 주제이다.

[12] Qdrant: High-Performance Vector Search at Scale

개발사: Qdrant (오픈소스) **유형:** 전문 벡터 검색 엔진, Rust 기반 **출시년도:** 2024년 주요 기술 문서 기반 **역할군:** (C) 경쟁/비교 시스템

요약

Qdrant는 Milvus와 함께 **전문 벡터 DB의 양대 산맥**으로, 필터링된 벡터 검색을 특히 잘 지원한다. Rust로 작성되어 메모리 안전성과 뛰어난 성능을 동시에 제공하며, **gRPC/REST API**를 통해 다양한 클라이언트와 통합할 수 있다.

Qdrant의 핵심 특징: 1. **Rust 기반 고성능:** GC(가비지 컬렉터) 없는 예측 가능한 지연 시간 2. **HNSW 기반 + 페이로드 필터링:** 벡터 검색과 메타데이터 필터링을 동시에 수행 3. **지능형 필터 전략 선택:** 선택도에 따라 자동으로 필터 우선/벡터 우선 전략 전환 4. **양자화 기법:** Scalar, Product, Binary Quantization으로 메모리 효율성과 성능의 밸런스 조정 5. **분산 배포:** Raft 기반 샤딩과 복제로 대규모 시스템 구축 가능

Qdrant도 Milvus와 마찬가지로 단일 컬렉션 내 필터링에 집중하며, **멀티 테이블 조인 쿼리는 지원하지 않는다**. 이것이 범용 벡터 DB + Exqutor 스타일의 최적화가 필요한 이유를 보여준다.

상세분석

12.1 핵심 특징: Rust의 장점 극대화

Rust 기반 고성능의 의미:

Rust는 C/C++처럼 메모리를 직접 관리하면서도, 컴파일 타임에 메모리 안전성을 검증한다. 이를 통해: - **메모리 안전성:** 버퍼 오버플로우, use-after-free 같은 메모리 오류가 원천 차단된다. - **제로 코스트 추상화:** 추상화 계층이 있어도 컴파일러가 최적화하면 C/C++와 같은 성능을 낸다. - **GC 없음:** 자동 메모리 회수(가비지 컬렉션)가 없으므로, 예측 불가능한 지연 시간(GC pause)이 발생하지 않는다. 이는 벡터 검색같이 **일정한 응답 시간이 중요한 시스템**에 매우 중요하다. - **멀티스레드 안전성:** 소유권 시스템 덕분에 데이터 경쟁(data race)이 컴파일 타임에 감지된다.

실제 성능 비교: - Java 기반 Elasticsearch보다 **3~5배** 빠른 색인 구축 - Python 기반 시스템 대비 **10배 이상** 높은 처리량 - 메모리 사용량 **50% 이하** (Milvus 대비)

12.2 HNSW 기반 + 페이로드 필터링: 동시적 처리

HNSW 그래프 탐색과 필터링의 동시성:

Qdrant의 가장 핵심적인 기술적 기여는 벡터 유사도 검색(HNSW)과 메타데이터 필터링을 **동시에** 수행하는 구조이다.

```
# Qdrant 검색 예시
client.search(
    collection_name="products",
    query_vector=[0.1, 0.2, ...],
    query_filter=Filter(
        must=[FieldCondition(key="price", range=Range(lt=1000))]
    ),
    limit=10
)
```

필터 선택도에 따른 자동 전략 전환:

Qdrant의 핵심 통찰은 필터의 선택도(어느 정도 비율의 데이터가 필터를 통과하는가)에 따라 최적의 검색 전략이 달라진다는 것이다.

1. **선택도 높음** (결과 많음, 예: 80%의 데이터가 조건을 만족):
2. HNSW 그래프를 탐색하면서 조건을 불만족하는 노드를 **건너뛰**
3. 거의 모든 데이터가 조건을 만족하므로, 필터를 먼저 적용하는 것이 비효율적
4. HNSW 탐색 중 조건 체크: $O(\text{탐색 노드 수})$
5. **선택도 낮음** (결과 적음, 예: 5%의 데이터만 조건을 만족):
6. 먼저 메타데이터 인덱스로 조건을 만족하는 데이터를 추린다.
7. 남은 작은 데이터셋에 대해 벡터 검색을 수행한다.
8. 브루트포스 + 필터: $O(\text{필터링 후 크기})$

이렇게 자동으로 전환하는 것이 Qdrant의 장점이다. 하지만 이 전환 결정에 사용되는 카디널리티 추정(필터링 후 몇 개가 남을 것인가)이 정확해야 한다.

12.3 양자화 기법: 메모리-성능 트레이드오프

Qdrant는 3가지 양자화 기법을 제공하여 메모리와 성능의 밸런스를 조정할 수 있다:

Scalar Quantization (스칼라 양자화): - 각 벡터 컴포넌트를 32비트 부동소수점에서 8비트 정수로 변환 - 메모리 사용량: **1/4로 감소** (768차원 벡터가 768바이트로 줄어듦) - 정확도 손실: 약 2~5% (작은 손실) - 적합한 경우: 메모리가 부족하지만 높은 정확도가 필요한 경우

Product Quantization (곱셈 양자화): - 벡터를 여러 서브벡터로 분할하고, 각 서브벡터를 독립적으로 양자화 - 예: 768차원을 32개의 24차원 서브벡터로 분할 - 메모리 사용량: **1/32로 감소** - 정확도: 스칼라 양자화보다 좀 더 손실 (하지만 여전히 합리적인 수준) - 적합한 경우: 극도로 메모리가 제한적인 환경 (모바일 기기, 엣지 컴퓨팅)

Binary Quantization (이진 양자화): - 각 컴포넌트를 1비트(0 또는 1)로 변환하는 극단적 압축 - 메모리 사용량: **1/256로 감소** - 정확도: 상당한 손실 (재현율 70~80%) - 적합한 경우: 초고속 검색이 필요하고 정확도를 어느 정도 포기할 수 있는 경우 (실시간 대규모 검색)

12.4 분산 배포: Raft 기반 일관성 보장

Qdrant는 분산 환경에서 대규모 데이터를 관리할 수 있도록 설계되었다.

샤딩(Sharding): - 벡터를 여러 샤드로 분할하여 여러 노드에 분산 저장 - 각 샤드는 부분 인덱스를 가짐 - 검색 시 모든 샤드의 부분 결과를 병합하여 전체 결과를 반환

복제(Replication): - 각 샤드의 복제본을 다른 노드에 저장하여 가용성 확보 - 노드 장애 시에도 복제본에서 데이터를 계속 제공

Raft 합의 프로토콜: - 분산 환경에서 여러 노드가 데이터 상태를 일관되게 유지하기 위해 Raft 프로토콜 사용 - 쓰기 작업 시 대다수(quorum) 노드가 승인하면 성공으로 간주 - 강한 일관성(Strong Consistency) 보장: 모든 노드가 동일한 상태

12.5 API 인터페이스: gRPC와 REST

Qdrant는 두 가지 API를 지원하여 다양한 클라이언트 환경에 대응한다.

gRPC API: - 고성능 이진 프로토콜 기반 - 네트워크 대역폭이 제한적인 환경에 효율적 - 스트리밍 쿼리 지원

REST API: - HTTP 기반으로 모든 표준 HTTP 클라이언트에서 접근 가능 - 디버깅과 테스트가 용이 (curl 등으로 직접 쿼리 가능) - 웹 브라우저에서도 직접 접근 가능

12.6 본 논문과의 관계

Qdrant는 Milvus와 마찬가지로 **단일 컬렉션 내 메타데이터 필터링**에 특화되어 있다. Qdrant의 자동 필터 전략 전환 기능(선택도 기반)은 흥미로운 접근이지만, 이 전환 결정을 위한 카디널리티 추정이 어떤 수준인지는 공개되지 않았다.

SQL의 조인·서브쿼리·집계는 불가능하므로, **멀티 테이블 VAQ에는 여전히 범용 벡터 DB + Exqutor 스타일의 최적화가 필요**하다는 것을 보여주는 또 다른 사례이다.

만약 Qdrant에 Exqutor의 ECQO를 적용한다면, 필터 선택도를 더욱 정확히 추정할 수 있어 자동 전략 선택의 정확성이 향상될 것이다. 하지만 Qdrant는 SQL을 지원하지 않으므로 직접 적용은 불가능하고, 개념적으로만 유사한 문제를 다루고 있다.

추가 제기 문제

Qdrant의 필터 선택도 기반 자동 전략 전환은 정교한 휴리스틱일 수 있지만, 선택도를 부정확하게 추정하면 오히려 성능이 저하될 수 있다. 예를 들어, 실제 선택도가 10%인데 50%로 추정되면, 높은 선택도로 판단하여 HNSW 그래프를 먼저 탐색하게 되고, 이는 불필요한 벡터 계산을 야기한다.

또한 Qdrant의 분산 환경에서 여러 샤드의 부분 결과를 병합할 때, 각 샤드가 반환한 top-k가 전체 top-k를 정확히 근사하는지에 대한 이론적 분석이 부족해 보인다. 이는 향후 연구 주제로 남아있다.

[13] pgvector: Open-Source Vector Similarity Search for PostgreSQL

개발자: Andrew Kane **유형:** PostgreSQL 확장(extension), GitHub 오픈소스 **활동 기간:** 2021년~현재 **GitHub 스타:** 12,000+ (2025년 기준) **역할군:** (B) 대상 시스템

요약

pgvector는 가장 널리 사용되는 범용 벡터 DB로, PostgreSQL에 벡터 데이터 타입과 유사도 검색 기능을 추가한다. Exqutor의 주요 실험 플랫폼이며, PostgreSQL의 완전한 SQL 기능을 활용하면서도 벡터 검색을 지원한다.

pgvector의 장점: 1. **전체 SQL 기능 지원:** 조인, 집계, 서브쿼리, 윈도우 함수, CTE 등 모든 SQL 기능 사용 가능 2. **기존 PostgreSQL 생태계와 호환:** pg_dump, 복제, 백업 등 기존 도구 그대로 사용 3. **MVCC 기반 트랜잭션:** 벡터 데이터도 일관된 트랜잭션 처리 가능 4. **다양한 거리 함수 지원:** 유클리드 거리, 코사인 거리, 내적 등 5. **오픈소스:** GitHub에서 자유롭게 수정·배포 가능

하지만 pgvector는 벡터 검색의 선택도를 항상 33.3%로 고정 추정한다는 치명적 한계가 있으며, 이것이 최적화 기회를 상당히 제한한다. Exqutor의 ECQO가 pgvector에서 최대 1,000배 속도 향상을 달성한 주요 이유이다.

상세분석

13.1 핵심 특징: 벡터 데이터 타입과 연산자

pgvector는 PostgreSQL의 확장(extension)으로 구현되며, 기본적으로 다음을 추가한다:

벡터 데이터 타입:

```
CREATE TABLE items (  
    id SERIAL PRIMARY KEY,  
    name TEXT,  
    embedding vector(768) -- 768차원 벡터  
);  
  
INSERT INTO items VALUES (1, 'product_a', '[0.1, 0.2, 0.3, ...]::vector);
```

이렇게 벡터를 일반 컬럼처럼 취급할 수 있으므로, 벡터를 포함한 복잡한 쿼리를 작성할 수 있다.

유사도 검색 연산자:

pgvector는 3가지 거리 함수를 지원한다:

연산자	거리 함수	수식	용도
<->	유클리드 거리 (L2)	$\sqrt{\sum (a_i - b_i)^2}$	일반적인 거리 측정
<=>	코사인 거리	$1 - (A \cdot B) / (\ A\ \ B\)$	방향(의미) 유사도
<#>	내적 (inner product)	$A \cdot B$	추천 시스템 등

활용 예시:

```
-- 유클리드 거리로 상위 10개 찾기
SELECT id, name, embedding <-> query_vector AS distance
FROM items
ORDER BY distance
LIMIT 10;

-- 코사인 거리가 0.2 이하인 것만
SELECT * FROM items
WHERE (embedding <=> query_vector) < 0.2
LIMIT 100;
```

13.2 인덱스 지원: HNSW와 IVFFlat

pgvector는 두 가지 벡터 인덱스를 지원한다.

HNSW 인덱스 (Hierarchical Navigable Small World):

```
CREATE INDEX ON items USING hnsw (embedding vector_l2_ops)
WITH (m=16, ef_construction=64);
```

파라미터 의미: - **m=16**: 각 노드가 최대 16개 이웃을 가짐. 커질수록 정확도 높지만 메모리 증가. - **ef_construction=64**: 인덱스 구축 시 탐색 깊이. 커질수록 구축 시간 증가.

HNSW는 **동적 인덱스**로, 데이터 삽입/삭제 시에도 인덱스가 자동으로 업데이트된다.

IVFFlat 인덱스 (Inverted File with Flat Quantization):

```
CREATE INDEX ON items USING ivfflat (embedding vector_l2_ops)
WITH (lists=100);
```

파라미터 의미: - **lists=100**: 벡터 공간을 100개 클러스터로 분할

IVFFlat는 **정적 인덱스**로, 데이터 변경 후 인덱스를 재구축해야 한다. 대신 메모리 효율이 좋고 구축이 빠르다.

13.3 구현 세부사항: PostgreSQL 통합의 장단점

pgvector의 가장 큰 장점:

pgvector는 PostgreSQL의 모든 인프라를 그대로 사용하므로:

1. **전체 SQL 기능**: 조인, 집계, 서브쿼리, 윈도우 함수, CTE, 트리거, 저장프로시저 등 모든 SQL 기능이 벡터 컬럼과 함께 작동한다.

```
sql -- 벡터 검색 + 집계 + 조인 SELECT p.category, COUNT(*) AS count, AVG(p.price) AS avg_price FROM
items p JOIN similar_items s ON p.id = s.item_id WHERE (p.embedding <=> query_vector) < 0.1 GROUP BY
p.category ORDER BY count DESC;
```

1. **EXPLAIN ANALYZE**: 실행 계획을 확인할 수 있어 성능 최적화가 용이하다.
2. **생태계 호환성**: pg_dump로 벡터 데이터까지 백업 가능, 복제(replication)도 그대로 작동한다.
3. **MVCC 기반 트랜잭션**: 벡터 데이터도 일관된 트랜잭션 처리로 데이터 무결성 보장.

pgvector의 근본적 한계:

벡터는 PostgreSQL의 **사용자 정의 타입(custom type)**으로 구현되므로:

1. **버퍼 풀 오버헤드**: 모든 벡터 접근이 공유 버퍼 풀을 경유한다. HNSW 그래프 탐색 시 매번 래치(latch)를 획득해야 하므로, 이것이 성능의 주요 병목이 된다 (논문 [20]에서 지적).
2. **튜플 헤더 파싱**: 각 벡터를 읽을 때마다 PostgreSQL의 튜플 헤더(트랜잭션 정보, 가시성 정보)를 파싱해야 하는데, 벡터 검색에는 이 정보가 불필요한 오버헤드다.
3. **공간 증폭(Space Amplification)**: HNSW 인덱스가 PostgreSQL의 8KB 페이지 단위로 저장되므로, 기본 테이블보다 훨씬 큰 공간을 차지한다 (2~3배).

13.4 선택도 추정치 치명적 한계: 고정 33.3%

pgvector의 가장 심각한 한계:

pgvector의 핵심 한계는 벡터 검색의 선택도를 **항상 33.3%(= 1/3)로 고정 추정**하는 것이다:

```
// pgvector 소스 코드 (pg_vector.c)
#define DEFAULT_SEL 0.333333
```

이 숫자는 아무런 근거 없는 기본값일 뿐, 실제 데이터 분포를 반영하지 않는다.

왜 이것이 문제인가?

PostgreSQL의 옵티마이저는 각 조건의 선택도(결과의 몇 %가 그 조건을 통과하는가)를 기반으로 **조인 순서, 조인 방식, 인덱스 사용 여부**를 결정한다.

예시 쿼리:

```
SELECT * FROM items
WHERE (embedding <=> query_vector) < 0.1 -- 벡터 조건
      AND category = 'electronics'      -- 스칼라 조건
ORDER BY price;
```

시나리오 1: 벡터 조건의 실제 선택도 = 0.1% (1000개 중 1개만 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 0.1%

이 경우 옵티마이저는 **HNSW 인덱스를 사용하지 않고 Sequential Scan**을 선택할 수 있다 (예상 결과가 많으니 풀 스캔이 빠르다고 판단). 하지만 실제로는 인덱스를 사용하는 것이 훨씬 빠르다.

시나리오 2: 벡터 조건의 실제 선택도 = 90% (거의 모든 데이터가 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 90%

이 경우 옵티마이저는 인덱스를 사용할 것으로 예상하지만, 실제로는 결과가 매우 많아서 Sequential Scan이 더 빠를 수 있다. 그럼에도 옵티마이저는 인덱스를 선택하여 불필요한 인덱스 탐색을 수행한다.

실제 성능 영향:

최적의 실행 계획과 실제 선택된 계획 사이에 **수십~수천 배의 성능 차이**가 발생할 수 있다. Exqutor는 이 문제를 직접 해결함으로써 pgvector에서 최대 **1,000배** 속도 향상을 달성했다.

13.5 본 논문과의 관계: Exqutor가 pgvector를 선택한 이유

Exqutor의 ECQO(Effective Cardinality-aware Query Optimization)는 pgvector에서 **PostgreSQL의 플래너 훅(planner hook)**을 확장하여 구현된다.

Exqutor의 해결책:

1. **플래너 훅 확장**: PostgreSQL이 쿼리 계획을 수립할 때, Exqutor가 벡터 조건을 가로챈다.
2. **실제 인덱스 탐색**: 벡터 조건의 정확한 카디널리티를 얻기 위해, HNSW 인덱스에 **실제 범위 검색을 실행**한다 (1~2ms 소요).
3. **정보 전달**: 얻은 카디널리티 정보를 PostgreSQL 옵티마이저에 전달한다.
4. **최적 계획 수립**: 옵티마이저가 정확한 카디널리티 정보를 바탕으로 최적 실행 계획을 수립한다.

이 접근법의 장점: - pgvector에 최소한의 변경 (planner hook만 활용) - 기존 PostgreSQL 인프라와 완전 호환 - 모든 유형의 벡터 조건(거리 임계값, top-k 등)을 지원

13.6 pgvector의 진화와 미래

pgvector는 지속적으로 개선되고 있다:

pgvector 0.7.0+ 개선사항: - HNSW 인덱스 성능 크게 향상 - 동적 인덱스 구축 최적화 - 메모리 사용량 감소

하지만 **카디널리티 추정 문제는 여전히 미해결**이다. pgvector가 이를 자체적으로 해결하려면: - 벡터 통계 수집 기능 추가 (ANALYZE 명령에 통합) - 고차원 벡터의 거리 분포 모델링 - 옵티마이저에 벡터 전용 카디널리티 추정 로직 구현

이 모든 것이 복잡한 작업이므로, Exqutor 같은 **외부 확장 기반의 접근**이 더 현실적일 수 있다.

추가 제기 문제

pgvector가 기술적으로는 훌륭하지만, 산업계에서는 여전히 Milvus, Qdrant 같은 전문 벡터 DB를 선호하는 경향이 있다. 이유는: 1. 벡터 검색에 특화된 성능 2. 전문 벡터 DB의 더 나은 필터링 지원 3. PostgreSQL의 "기본 오버헤드"에 대한 우려

하지만 Exqutor의 연구는 pgvector의 가능성을 보여주며, 올바른 최적화 기법으로 이런 격차를 상당히 줄일 수 있음을 입증했다. 향후 pgvector가 벡터 통계를 추가로 지원하고 옵티마이저를 개선하면, 전문 벡터 DB와의 격차는 더욱 좁혀질 것으로 예상된다.

[14] ClickHouse: Lightning Fast Analytics for Everyone

저자: Robert Schulze, Tom Schreiber 외 다수 (ClickHouse Inc.) **학회/년도:** VLDB 2024 (Proceedings of the VLDB Endowment, Vol. 17, No. 12) **분량:** 약 14페이지 **역할군:** (C) 경쟁/비교 시스템

요약

ClickHouse는 **컬럼형 OLAP 엔진의 대표 시스템**으로, 대규모 데이터의 고속 분석에 특화되어 있다. 최근 벡터 검색 기능을 추가하여 범용 벡터 DB의 새로운 후보가 되고 있으며, Exqutor의 범용성을 보여주는 핵심 사례이다.

ClickHouse의 핵심 특징: 1. **벡터화 실행 엔진**: 한 번에 수천~수만 개 행을 배치 처리하여 CPU 캐시 효율성 극대화 2. **MergeTree 기반 컬럼형 저장**: 필요한 컬럼만 선택적으로 읽어 I/O 효율성 극대화 3. **스마트 데이터 스킵핑**: 각 데이터 블록별로 최소/최대/블룸필터 정보로 불필요한 블록 스킵 4. **다중 압축 알고리즘**: LZ4, ZSTD, Delta, Gorilla 등을 컬럼별로 자동 선택 5. **실시간 데이터 처리**: 초당 수백만 건의 데이터를 실시간으로 적재하면서 쿼리 실행

ClickHouse도 범용 DB에 벡터 검색을 추가한 경우로, **카디널리티 추정 문제를 안고 있을 것으로 예상된다**. Exqutor의 ECQO 개념을 ClickHouse에 적용하면 유사한 성능 향상을 기대할 수 있다.

상세분석

14.1 해결하고자 하는 문제: OLAP 엔진의 도전

기존 OLAP 엔진의 딜레마:

대부분의 데이터베이스는 다음 두 요구사항 중 하나에 특화된다:

1. **OLTP (Online Transactional Processing):**
2. 소량의 데이터를 빠르게 읽고 쓰기 (예: 주문 삽입, 사용자 조회)
3. 트랜잭션 안전성, 동시성 제어 중심
4. 예: MySQL, PostgreSQL
5. **OLAP (Online Analytical Processing):**
6. 대량의 데이터를 복잡하게 분석 (예: 월간 판매액 집계, 고객 세분화)
7. 쿼리 속도, I/O 효율성 중심
8. 예: 기존 Redshift, BigQuery

ClickHouse는 "두 마리 토끼를 모두 잡겠다"는 야망을 가지고 설계되었다:

ClickHouse의 목표: - **고속 적재:** 초당 수백만 건의 데이터를 스트리밍으로 받아서 적재 - **저지연 분석:** 분석 쿼리를 초 단위(또는 밀리초 단위)로 실행

예: "지난 1시간 동안 들어온 클릭 로그를 실시간 집계하면서, 동시에 사용자별 클릭 패턴을 분석하려면?"

14.2 핵심 기술 1: MergeTree 기반 컬럼형 저장

행 저장(Row-store) vs. 컬럼 저장(Column-store):

특성	행 저장	컬럼 저장
저장 순서	행 단위	컬럼 단위
접근 패턴	한 행의 모든 컬럼을 함께 읽음	필요한 컬럼만 선택적 읽음
적합한 워크로드	좁은 행 insert (OLTP)	넓은 행 집계 (OLAP)
압축률	낮음	높음 (같은 타입의 컬럼은 압축 잘됨)

MergeTree 구조:

INSERT (스트리밍) → 인메모리 버퍼 → (주기적) → 디스크 파트(Part) → (백그라운드) → 파트 병합

- **신규 파트(Recent Part):** 새로 삽입된 데이터는 작은 파트로 빨리 저장

- **병합(Merge)**: 여러 작은 파트를 주기적으로 큰 파트로 병합하여 최적화
- **30+ 변형**:
 - **ReplacingMergeTree** : 키 기반으로 중복 제거하면서 병합
 - **SummingMergeTree** : 숫자 컬럼을 합산하면서 병합
 - **AggregatingMergeTree** : 사전 집계된 상태를 병합
 - **VersionedCollapsingMergeTree** : 버전 정보와 함께 행 삭제 지원

이 구조 덕분에 ClickHouse는 실시간 데이터 유입을 처리하면서도, 기존 데이터를 효율적으로 분석할 수 있다.

14.3 핵심 기술 2: 벡터화 쿼리 실행 (Vectorized Execution)

전통적 Volcano 모델의 한계:

기존 DB는 한 행씩 처리한다:

```
for each row:
    evaluate_condition(row)
    if matches:
        output(row)
```

이 방식의 문제: - CPU 캐시 미스 많음: 데이터가 스트리밍 방식으로 들어와서 캐시 지역성(locality) 낮음 - 분기 예측(Branch Prediction) 어려움: 조건 결과가 예측 불가능 - SIMD(Single Instruction Multiple Data) 활용 못함: 한 번에 한 행씩 처리하므로 벡터 명령어 활용 어려움

ClickHouse의 벡터화 실행:

```
BLOCK_SIZE = 8192행
for each block of rows:
    evaluate_condition_vectorized(block)
    if matches:
        output(block)
```

이렇게 하면: - **메모리 접근 패턴**: 같은 컬럼의 8192개 값이 연속으로 배치되므로 캐시에 잘 올라옴 - **분기 예측**: CPU가 분기 패턴을 학습할 여유가 생김 - **SIMD 활용**: AVX2, AVX-512 같은 벡터 명령어로 한 번에 32개 연산 수행

성능 향상: - CPU 캐시 효율: **3~5배** 향상 - SIMD 활용: **4~8배** 추가 향상 - 전체: **10~40배** 속도 향상

14.4 핵심 기술 3: 스마트 데이터 스킵핑 (Data Skipping)

그래놀(Granule)과 스킵 인덱스:

ClickHouse는 데이터를 **그래놀(granule)** 단위로 관리한다 (약 8,192행):

테이블 = [그래놀1, 그래놀2, 그래놀3, ...]

각 그래놀에 대해 **스킵 인덱스** 정보를 유지한다:

스킵 인덱스 유형	저장 정보	용도
MinMax	각 컬럼의 최소/최대 값	범위 쿼리 (WHERE date > '2024-01-01')
Bloom Filter	집합 멤버십 (이 그래놀에 값 X가 있는가?)	정확한 값 조회 (WHERE user_id = 12345)
Set	특정 값 세트	카테고리 필터 (WHERE category IN ('A', 'B'))
Tuple	다중 컬럼 조합	복합 필터

쿼리 실행 시 **스킵 인덱스 활용**:

```
SELECT SUM(amount) FROM transactions
WHERE date >= '2024-01-01' AND date < '2024-02-01'
AND user_id = 12345;
```

1. MinMax 인덱스로 date 범위를 체크: 2024-01-01 ~ 2024-02-01 범위인 그래놀만 남음
2. Bloom 필터로 user_id = 12345를 체크: 해당 값이 없는 그래놀 제거
3. 남은 그래놀만 읽고 집계

이 과정에서 **대부분의 그래놀을 아예 읽지 않으므로**, I/O를 획기적으로 줄일 수 있다.

14.5 핵심 기술 4: 다중 압축 알고리즘

ClickHouse는 컬럼의 특성에 맞춰 압축 알고리즘을 선택할 수 있다:

LZ4 (빠른 압축): - 압축 속도: **매우 빠름** - 압축률: 낮음 (2~3배) - 사용: 자주 접근하는 컬럼, 실시간 쿼리

ZSTD (높은 압축률): - 압축 속도: 느림 - 압축률: 높음 (5~10배) - 사용: 자주 읽지 않는 아카이브 컬럼, 스토리지 절약

Delta (시계열 최적화): - 시계열 데이터의 연속된 값 차이만 저장 - 예: 타임스탬프 [1000000, 1000001, 1000002, ...] → 차이 [0, 1, 1, ...] → 매우 작은 값만 저장 - 압축률: **50~100배**

Gorilla (부동소수점 최적화): - Float/Double 값의 유사한 비트 패턴만 저장 - 예: 센서 데이터 온도 [23.456, 23.457, 23.458, ...] → 약간씩 변하는 마지막 비트만 저장 - 압축률: **10~50배**

개발자는 각 컬럼마다 최적의 압축 알고리즘을 지정할 수 있다:

```
CREATE TABLE metrics (
  timestamp DateTime CODEC(Delta, LZ4), -- 시간순 데이터: Delta + 추가압축
  temperature Float32 CODEC(Gorilla, ZSTD), -- 센서: 부동소수점 최적화 + 고압축
  value UInt64 CODEC(LZ4) -- 빠른 접근 필요
)
```

14.6 벡터 검색 기능 추가: 최근 발전

ClickHouse는 최근 벡터 검색 기능을 추가하고 있다:

- **annoy** 라이브러리: Spotify의 오픈소스 ANN 알고리즘
- **usearch** 라이브러리: 고성능 벡터 검색 라이브러리

하지만 아직 초기 단계로: - HNSW 인덱스 지원이 제한적 - 벡터 컬럼의 스킵 인덱스 지원 아직 미흡

14.7 성능 결과

ClickBench 벤치마크 (2024):

43개의 복잡한 OLAP 쿼리에 대해 모든 프로덕션급 DB를 상회하는 성능: - 100억 건 테이블에 대한 집계 쿼리: **초 단위 응답** (몇 초에서 수십 초) - 초당 수백만 건의 실시간 적재 처리 - 메모리 사용량: 경쟁사 대비 **20~30%**

14.8 본 논문과의 관계

ClickHouse는 범용 벡터 DB의 새로운 후보이다. ClickHouse의 옵티마이저도 벡터 카디널리티 추정 문제를 안고 있을 가능성이 높다:

- **MergeTree의 스킵 인덱스:** 고차원 벡터의 거리 분포를 정확히 표현할 수 없음
- **기존 히스토그램 기반 통계:** 벡터 거리의 비유클리드적 성질(고차원에서의 거리 집중 현상)을 포착하지 못함

Exqutor의 적용 가능성:

Exqutor의 ECQO 접근법(플래너 단계에서 실제 인덱스를 가볍게 탐색하여 카디널리티 추정)은 ClickHouse에도 개념적으로 적용 가능하다:

ClickHouse 플래너 → Exqutor 스타일의 카디널리티 추정 → 정확한 조인 순서 선택

이는 Exqutor의 범용성을 보여주며, 향후 ClickHouse에 벡터 검색이 더 성숙화되면 Exqutor의 개념을 적용할 수 있을 것으로 예상된다.

추가 제기 문제

1. **벡터 컬럼의 특수성:** ClickHouse의 벡터화 실행과 SIMD 최적화는 일반 컬럼에는 최적이지만, 벡터 컬럼은 다른 접근 패턴을 가진다. 벡터 검색은 순차 스캔이 아닌 무작위 접근(그래프 탐색)이므로, ClickHouse의 순차 스캔 최적화와 충돌할 수 있다.
2. **스킵 인덱스의 한계:** MergeTree의 스킵 인덱스는 범위 필터링에는 좋지만, 벡터 거리 임계값($\text{distance} < D$)을 정확히 표현하기 어렵다. 특히 고차원에서 거리 분포의 집중 현상이 스킵 인덱스의 효율성을 크게 떨어뜨린다.
3. **아직 초기 단계:** ClickHouse의 벡터 검색 기능은 아직 프로덕션 환경에 충분히 성숙되지 않았으므로, 대규모 벡터 데이터셋에 대한 성능 데이터가 부족하다.

[15] Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Query-Based Retrievers

저자: Kunal Sawarkar, Abhilasha Mangal, Shivam Raj Solanki **학회/년도:** IEEE MIPR (Multimedia Information Processing and Retrieval) 2024 **분량:** 약 10페이지 **역할군:** (A) VAQ 동기 부여

요약

Blended RAG는 RAG(Retrieval-Augmented Generation) 시스템의 검색 정확도를 높이기 위한 **하이브리드 검색 전략**을 제안한다. 핵심 발견은 벡터 검색만으로는 부족하며, **키워드 검색·희소 인코딩·밀집 벡터 검색을 결합**해야 한다는 것이다.

이 논문이 보여주는 중요한 시사: 1. **VAQ의 복잡성 증가:** 벡터 검색이 단순 top-k에서 벗어나 복합 검색 전략으로 진화 중 2. **적응적 카디널리티 추정의 필요성:** 각 검색 전략의 결과 크기가 쿼리마다 크게 달라지므로, 고정 선택도는 특히 치명적 3. **실무 요구사항의 복잡성:** 단순 학술 벤치마크와 달리 실제 RAG 시스템은 여러 검색 전략을 결합 Blended RAG의 성능 향상은 modest하지만(5~8%), 검색 결과를 관계형 데이터와 조인하고 필터링하면 VAQ가 되며, 이때 정확한 카디널리티 추정의 가치가 극대화된다.

상세분석

15.1 해결하고자 하는 문제: RAG 시스템의 검색 품질

RAG(Retrieval-Augmented Generation)란?

RAG는 다음 절차로 동작한다:

사용자 질문 → 검색 (관련 문서 찾기) → LLM 입력 (검색 결과 + 질문) → 생성된 답변

검색 단계가 **LLM 응답 품질의 상한선(ceiling)**을 결정한다: - 검색 결과가 정확하면 → LLM이 좋은 답변 생성 가능 - 검색 결과가 부정확하거나 불완전하면 → LLM이 아무리 좋아도 제한된 답변만 가능

단일 검색 전략의 한계:

기존 RAG 시스템은 한 가지 검색 방식만 사용했다:

- 1. 키워드 검색(BM25)만 사용:** - 장점: 정확한 단어 매칭에 강함 (예: "Apple Inc." 검색 시 "Apple" 회사만 찾음) - 단점: 의미적 유사성을 못 잡음 - 질문: "자동차 사고" - 찾지 못하는 문서: "교통 충돌", "차량 사건" (의미가 같지만 단어가 다름)
- 2. 밀집 벡터 검색(KNN)만 사용:** - 장점: 의미적 유사성을 잘 포착 (BERT, GPT 임베딩 모델 활용) - 단점: 정확한 키워드 매칭에 약함 - 질문: "Python 3.11 버전의 async 함수" - 문제: "Python"과 "3.11"이라는 정확한 숫자를 벡터로 정확히 표현하기 어려움 - 의미적으로는 관련 있지만 **잘못된 버전**(예: Python 2.7)의 문서를 반환할 수 있음
- 3. 희소 인코딩(ELSER)만 사용:** - Elastic이 개발한 학습된 희소 인코더 - 장점: BM25보다 유연하고 의미도 어느 정도 포착 - 단점: 복잡한 의미 관계를 완전히 포착하지 못함

15.2 핵심 아이디어: 세 가지 검색 전략의 혼합

Blended RAG는 세 가지 검색 기능을 동시에 실행하고 결과를 병합한다:

1. BM25 (키워드 기반):

TF-IDF 확장 알고리즘

$$\text{점수} = (\text{tf_in_doc} * \text{IDF}) / (\text{tf_in_doc} + k1 * (1 - b + b * \text{doc_len} / \text{avg_len}))$$

- tf_in_doc: 문서에서 단어 출현 빈도
- IDF: 역 문서 빈도 (흔한 단어일수록 낮음)
- k1, b: 튜닝 파라미터

2. KNN 밀집 벡터 (시맨틱):

```
embedding = text_encoder(query)
top_k = argmax_k(cosine_similarity(embedding, doc_embeddings))
```

- BERT, GPT-3 같은 사전 학습 모델로 텍스트를 벡터로 변환
- 코사인 유사도로 상위 k개 문서 반환

3. ELSER 희소 인코더 (하이브리드):

```
sparse_vector = elser_model(query)
점수 = inner_product(sparse_vector, doc_sparse_vector)
```

- 키워드와 시맨틱의 중간 지점
- 학습된 가중치로 중요한 단어에 높은 점수 부여

15.3 세 결과의 병합: RRF (Reciprocal Rank Fusion)

세 검색기로부터 받은 결과를 **Reciprocal Rank Fusion (RRF)**으로 병합한다:

$$RRF_score = \sum (1 / (rank + k))$$

예시: - BM25 결과: [문서A (rank 1), 문서B (rank 2), 문서C (rank 5)] - KNN 결과: [문서B (rank 1), 문서A (rank 3), 문서D (rank 2)] - ELSEER 결과: [문서A (rank 1), 문서D (rank 2), 문서B (rank 4)]

문서A의 RRF 점수 = $1/(1+60) + 1/(3+60) + 1/(1+60) = \text{높음}$ → 최종 순위 1위

이 방식의 장점: - 한 검색기가 부정확해도 다른 검색기가 보완 - 여러 검색기에서 모두 높은 순위를 받은 문서가 최종적으로 우위를 가짐

15.4 하이브리드 쿼리 변형

문제 유형에 따라 다양한 병합 전략을 사용할 수 있다:

전략	사용 시점	예시
best-fields	한 필드에서 가장 좋은 매칭이 중요할 때	문서 제목에 모든 키워드가 있는 경우 우위
cross-fields	여러 필드에 걸쳐 키워드 분산 시	"문제"는 제목에, "해결책"은 본문에 있는 경우
most-fields	여러 검색기에서 매칭되는 것을 우위로	BM25, KNN, ELSEER 모두에서 매칭되는 문서
phrase-prefix	구문 매칭과 접두사 검색 결합	"machine learning" 구문 + "algor*" 매칭

15.5 성능 결과: 모델별 향상

NQ 데이터셋 (자연어 질문): - 기존 최고 (BM25 또는 단일 KNN): NDCG@10 = 0.633 - Blended RAG: NDCG@10 = 0.67 - 향상률: **+5.8%**

TREC-COVID 데이터셋 (의료 정보): - 기존 최고: NDCG@10 = 0.804 - Blended RAG: NDCG@10 = 0.87 - 향상률: **+8.2%**

향상 폭이 크지 않아 보이지만, 이는 기존 최고 성능이 이미 높기 때문이다. 절대 성능 관점에서는 상당한 개선이다.

더 중요한 것은: - **재현율(Recall)**도 함께 향상됨: 관련 문서를 놓치는 경우가 줄어들음 - **견고성(Robustness):** 쿼리 타입이 변해도 안정적인 성능

15.6 본 논문과의 관계: VAQ 복잡성의 증가

Blended RAG가 보여주는 것:

1. 벡터 검색이 단순 top-k를 넘어 복잡해지고 있다:

```
-- 기존 단순 벡터 검색
SELECT * FROM documents
WHERE embedding <=> query_vector
LIMIT 10;

-- Blended RAG 스타일의 복합 검색
SELECT *,
       (bm25_score + knn_score + elser_score) / 3 AS blended_score
FROM (
  SELECT * FROM bm25_search(query) UNION ALL
  SELECT * FROM knn_search(query) UNION ALL
  SELECT * FROM elser_search(query)
) unified_results
GROUP BY doc_id
ORDER BY blended_score DESC
LIMIT 10;
```

이것을 관계형 데이터와 조인하면:

```
-- 실제 필요한 쿼리 (VAQ의 예시)
SELECT documents.*, users.rating, categories.category_name
FROM documents
JOIN users ON documents.author_id = users.id
JOIN categories ON documents.category_id = categories.id
WHERE (
  -- 벡터 검색
  bm25_score(documents.text, query) > 0.5
  OR knn_similarity(documents.embedding, query) > 0.8
) AND users.rating >= 4.0
ORDER BY combined_relevance DESC
LIMIT 10;
```

이 쿼리에서: - 세 개의 검색 함수(bm25_score, knn_similarity, elser_score)의 결과 크기가 불확실 - 각 검색 함수의 선택도에 따라 JOIN의 효율성이 크게 달라짐 - **고정 선택도 추정**은 특히 치명적

2. 각 검색 전략의 결과 크기가 쿼리마다 크게 달라진다:

쿼리 1: "Python 프로그래밍" → BM25는 10,000개, KNN은 50개 (BM25 편향) 쿼리 2: "기계학습 개요" → BM25는 1,000개, KNN은 5,000개 (KNN 편향) 쿼리 3: "네트워크 보안" → 세 검색기 모두 비슷한 크기

고정 33.3% 선택도로는 이런 변화를 절대 포착할 수 없다.

3. 검색 결과의 품질이 후속 조인에 영향:

Blended RAG의 결과가 높은 품질이라면, 이를 관계형 데이터와 조인할 때: - 검색 결과가 많을 때: Hash Join이 효율적 - 검색 결과가 적을 때: Nested Loop Join이 효율적

정확한 카디널리티 추정이 없으면, 옵티마이저가 최악의 JOIN 방식을 선택할 수 있다.

15.7 Blended RAG의 한계와 Exqutor의 가치

Blended RAG의 한계:

1. RRF 가중치(k값)를 데이터셋별로 수동 튜닝해야 함
2. 세 검색 전략 모두를 항상 실행하므로 **3배의 검색 오버헤드** (병렬 처리로 일부 상쇄)
3. 검색 결과의 품질 편차가 큼 (쿼리마다 결과 크기 큰 변동)

Exqutor의 적용 시나리오:

Blended RAG + 관계형 데이터 조인 + **Exqutor의 ECQO**:

1. Blended RAG 실행 → 검색 결과 (크기 불확실)
2. ECQO가 각 검색 전략의 실제 카디널리티 측정
3. 정확한 정보로 JOIN 계획 수립
4. 최적의 JOIN 방식 선택 → 극적인 성능 향상

추가 제기 문제

1. **RRF 가중치 튜닝**: Blended RAG의 성능은 RRF 합병 파라미터에 크게 의존한다. 데이터셋별, 도메인별로 최적 가중치가 다르므로, 자동으로 가중치를 학습하는 방법이 필요하다.
2. **검색 지연 시간**: 세 검색 전략을 모두 실행하면 지연 시간이 3배 증가한다. 병렬 처리로 일부 상쇄되지만, 여전히 개선의 여지가 있다.
3. **메모리와 저장 공간**: 세 가지 인덱스(BM25, KNN 벡터, ELSER 희소)를 모두 유지해야 하므로, 저장 공간이 상당히 증가한다.
4. **향후 확장성**: Blended RAG가 더 많은 검색 전략(예: 시맨틱 그래프 검색, 속성 필터링 기반 검색)을 추가하면, 결과 병합의 복잡도가 기하급수적으로 증가한다. 이때 정확한 카디널리티 추정은 더욱 중요해진다.

[16] Scalable Similarity Search for Big Data

저자: Pavel Zezula (Masaryk University, Czech Republic) **학회/년도**: Springer INFOSCALE 2015 (초청 강연)

분량: 초청 논문/서베이, 약 12페이지 **역할군**: (D) 이론적 기반

요약

이 서베이 논문은 대규모 유사도 검색의 **이론적 기초**를 제공한다. **메트릭 공간(metric space) 이론**이라는 수학적 근거 위에서 유사도 검색을 정의하며, "왜 벡터 검색의 결과 건수를 예측하기 어려운가"를 이론적으로 설명한다.

핵심 기여: 1. **메트릭 공간의 기하학적 성질**: 고차원 공간에서의 거리 분포 특성 분석 2. **M-tree 인덱스 구조**: 메트릭 공간의 분할 기반 동적 인덱스 3. **고차원의 저주와 카디널리티**: 거리 집중(concentration) 현상의 수학적 분석 4. **분산 환경으로의 확장**: 빅데이터 환경에서의 유사도 검색

이 논문은 Exqutor가 통계 기반이 아닌 **실제 인덱스 탐색 기반의 카디널리티 추정을 선택한 이유**의 이론적 근거를 제공한다.

상세분석

16.1 해결하고자 하는 문제: 빅데이터 시대의 유사도 검색

세 가지 핵심 도전 과제:

- 확장성(Scalability)**: - 데이터가 수십억 건에서 수조 건으로 증가해도 응답 시간이 허용 범위 내여야 한다. - 기존 풀 스캔(brute-force) 방식은 $O(n)$ 복잡도이므로, 데이터 크기 n 이 1000배 증가하면 응답 시간도 1000배 증가한다. 이는 참을 수 없다. - 인덱스를 통해 $O(\log n)$ 또는 $O(\log^k n)$ 수준으로 개선해야 한다.
- 프라이버시 보존(Privacy Preservation)**: - 클라우드 아웃소싱 환경에서 원본 데이터는 공개하지 않으면서 유사도 검색을 수행해야 한다. - 암호화된 데이터에 대한 검색(searchable encryption) 기술이 필요하다.
- 멀티모달 통합(Multimodal Integration)**: - 텍스트, 이미지, 오디오, 비디오 등 서로 다른 모달리티의 데이터를 하나의 통합 프레임워크에서 검색해야 한다. - 각 모달리티를 공통 벡터 공간으로 매핑하는 임베딩 기술이 필요하다.

16.2 메트릭 공간(Metric Space) 이론: 수학적 기초

메트릭 공간의 정의:

메트릭 공간 (X, d) 는 집합 X 와 거리 함수 $d: X \times X \rightarrow \mathbb{R}$ 로 구성되며, 거리 함수가 다음 **공리**를 만족한다:

- 양의 정부호성 (Positivity)**:
 - $d(x, y) \geq 0$ (거리는 항상 0 이상)
 - $d(x, y) = 0 \iff x = y$ (같은 점 사이의 거리만 0)
- 대칭성 (Symmetry)**:
 - $d(x, y) = d(y, x)$ (x 에서 y 까지의 거리 = y 에서 x 까지의 거리)
- 삼각 부등식 (Triangle Inequality)**:

7. $d(x, z) \leq d(x, y) + d(y, z)$ (우회하는 것보다 직접 가는 것이 가깝다)

삼각 부등식의 힘:

삼각 부등식이 핵심이다. 이 성질 덕분에:

$$d(x, z) \leq d(x, y) + d(y, z)$$

만약 $d(x, y) + d(y, z) <$ 현재까지의 최소 거리 라면, $d(x, z)$ 는 이미 본 것보다 작을 수 없다. 따라서 **z를 검사할 필요 없다.**

이런 식으로 많은 거리 계산을 건너뛴(prune) 수 있어, 전수 검색 없이도 유사도 검색이 가능하다.

예시: 뉴욕 지도에서의 거리

삼각 부등식: $d(A, C) \leq d(A, B) + d(B, C)$
뜻: A에서 C까지의 직선거리 $\leq$ A→B→C의 경로거리

이 성질을 활용하면: - A에서 거리 5km 이내인 장소를 찾을 때 - B를 거쳐가는 모든 경로를 검사하는 것이 아니라 - A에서 거리 5km 이내의 "동네" 영역만 검색하면 된다.

16.3 M-tree: 메트릭 공간의 동적 인덱스 구조

M-tree의 개념:

M-tree는 B-tree처럼 균형 잡힌 트리 구조를 메트릭 공간에 적용한 것이다.

```
      [루트 - 전체 데이터의 중심점]
      /      |      \
[서브트리1] [서브트리2] [서브트리3]
(중심: p1)  (중심: p2)  (중심: p3)
반경: r1    반경: r2    반경: r3
 /  \      /  \      /  \
[리프]... [리프]... [리프]...
```

각 노드는: - **중심점(pivot/representative)**: 그 부분트리를 대표하는 점 - **반경(covering radius)**: 중심점으로 부터 이 부분트리의 가장 먼 점까지의 거리

검색 과정:

쿼리 점 q에서 거리 r 이내의 모든 점을 찾을 때:

1. 루트 노드부터 시작
2. 각 자식 노드에 대해:
 - 삼각 부등식으로 가지 제거(pruning):
 $d(q, \text{자식중심}) - \text{자식반경} > r$ 이면
 $\rightarrow$ 이 가지는 검색할 필요 없음
 - 그 외는 재귀적으로 탐색
3. 리프 노드에서 정확한 거리 계산 및 필터링

M-tree의 장점: - 동적 삽입/삭제 지원 (B-tree처럼) - 다양한 메트릭(유클리드, 맨해튼, 편집 거리 등) 지원 - 메트릭 공간의 특성만 만족하면 모든 데이터 타입 지원

16.4 고차원의 저주와 거리 분포의 집중(Concentration)

고차원 공간의 기하학적 성질:

차원이 높아질수록 흥미로운(그리고 직관에 반하는) 현상들이 일어난다:

1. 거리의 집중(Distance Concentration):

고차원 공간에서 **모든 점 사이의 거리가 거의 같아진다.**

예시: 무작위 단위 벡터 1,000개 생성

차원	최소 거리	최대 거리	차이
2D	0.1	1.9	1.8
10D	1.2	1.5	0.3
100D	1.41	1.42	0.01
1000D	1.414	1.415	0.001

1000차원에서는 모든 점이 서로 거의 같은 거리(약 $\sqrt{2} \approx 1.414$)에 있다!

수학적 설명:

d차원 공간에서 무작위 단위 벡터들 사이의 거리 E[D]는:

$$E[D] \approx \sqrt{(2d/\pi) \times (1 - 1/(4d) + O(1/d^2))}$$

$$d \rightarrow \infty \text{ 일 때, 표준편차 } \sigma[D] \approx \sqrt{1/(2d)}$$

거리의 **변동 계수(coefficient of variation):**

$$CV = \sigma[D] / E[D] \approx \sqrt{\pi/(2d)}$$

이것은 d에 반비례하므로, 차원이 높아질수록 거리 분포가 더 집중된다.

2. "저주"의 의미:

거리가 집중되면:

모든 데이터 점 사이의 거리가 비슷함
↓
거리 임계값 $d(q, x) < r$ 을 만족하는 점의 비율이 급격히 변함
↓
카디널리티를 정확히 예측하기 극도로 어려움

예: 쿼리 반경 r 을 0.1 증가시키면: - 저차원: 결과 점 10% 증가 - 고차원: 결과 점 200% 증가 (넓이 vs. 부피 증가의 차이)

16.5 고차원 카디널리티 추정의 어려움

히스토그램 기반 통계의 한계:

관계형 DB는 스칼라 값의 카디널리티 추정을 위해 히스토그램을 사용한다:

값 범위	튜플 개수
0~100	1,000
100~200	2,000
200~300	800

가격이 150인 상품 수를 추정하려면, 100~200 범위의 히스토그램을 보면 된다.

하지만 벡터 거리에는 이런 방식이 작동하지 않는다:

```
WHERE embedding <=> query < 0.1
```

이 조건을 만족하는 벡터의 개수는: - 데이터 분포에 완전히 의존 - 쿼리 벡터의 위치에 의존 - 임계값의 작은 변화에 극도로 민감

예: - $r = 0.09$: 결과 1,000개 - $r = 0.10$: 결과 10,000개 - $r = 0.11$: 결과 50,000개

히스토그램을 구성하려면 쿼리 벡터마다 미리 거리를 계산하고 저장해야 하는데, 이는 불가능하다 (쿼리마다 다르기 때문).

16.6 본 논문과의 관계

이 논문이 보여주는 것:

1. 벡터 카디널리티 추정의 이론적 어려움:

고차원 공간에서 거리 분포의 집중 현상이 일어나므로, 기존의 통계 기반 추정(히스토그램)은 근본적으로 작동할 수 없다.

2. Exqutor가 통계 기반 대신 실제 탐색 기반을 선택한 이유:

카디널리티를 정확히 얻는 유일한 방법은: - 실제 인덱스를 탐색해서 결과를 세는 것

Exqutor의 ECQO는 이 철학을 따른다:

쿼리 플래너 단계 → ECQO 혹은 → 실제 인덱스 범위 탐색 (1~2ms)
→ 정확한 카디널리티 얻음 → 옵티마이저에 전달 → 최적 계획

3. 왜 Exqutor의 1~2ms 오버헤드가 정당한가:

쿼리 실행 시간이 수백ms ~ 수초인데, 플래너 단계에서 1~2ms를 투자하여 최적 계획을 얻는 것은 매우 합리적이다.

특히 선택도가 극도로 불확실한 경우(0.1%에서 90%까지 변할 수 있는 경우), 정확한 추정 비용 1~2ms는 하찮다.

16.7 메트릭 공간 이론과 현대 벡터 검색의 연결

2015년 논문의 한계:

이 서베이는 2015년 기준이므로: - HNSW, DiskANN 같은 최근 그래프 기반 인덱스는 다루지 않음 - 학습된 인덱스(learned index) 개념은 없음 - 신경망 기반 임베딩의 고차원 특성은 다루지 않음

하지만 기본 원리는 여전히 유효:

메트릭 공간의 거리 집중 현상은 변하지 않으므로, HNSW나 DiskANN을 사용하더라도 카디널리티 추정 문제는 여전히 존재한다.

추가 제기 문제

1. **적응적 인덱스 구조:** M-tree는 정적이지만, 데이터가 자주 업데이트되는 환경에서는 재구축 비용이 크다. 동적 데이터 환경에 최적화된 인덱스 구조 연구가 필요하다.
 2. **고차원 특화 인덱스:** 메트릭 공간 이론은 일반적이지만, 특정 차원 범위(예: 768차원 BERT 벡터)에 최적화된 인덱스를 설계하는 것이 가능할까?
 3. **카디널리티 상한/하한 추정:** 정확한 값을 얻을 수 없다면, 최소한 상한과 하한을 빠르게 추정할 수 있는 방법이 있을까? 이를 통해 옵티마이저가 비효율적인 선택을 피할 수 있을까?
 4. **멀티모달 통합의 현실화:** 논문은 멀티모달 통합을 언급하지만, 실제로 텍스트, 이미지, 오디오를 하나의 메트릭 공간으로 통합하는 것의 실무적 난제는 무엇인가?
-

[17] Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows

저자: Fangyu Lei, Jixuan Chen, Yuxiao Ye 외 다수 학회/년도: arXiv:2411.07763, 2024 (ICLR 2025 Oral 채택)

분량: 약 30페이지 + 부록 역할군: (A) VAQ 동기 부여

요약

Spider 2.0은 **실세계 기업 환경에서의 Text-to-SQL 복잡성**을 처음으로 체계적으로 측정한 벤치마크이다. 기존 Spider 1.0이 학술 환경에 맞춰진 상대적으로 간단한 쿼리만 다루었다면, Spider 2.0은 **실제 기업 데이터베이스의 극단적 복잡성**을 반영한다.

핵심 발견: 1. **학술 vs. 현실의 거대한 격차**: 모든 LLM이 Spider 1.0에서는 85~91% 정확도를 보이지만, Spider 2.0에서는 15~21%로 급락 (50~70 포인트 격차) 2. **실제 데이터베이스의 극단적 복잡성**: 수천 개 테이블, 1,000+ 컬럼, 100줄 이상의 멀티 스텝 쿼리 3. **VAQ의 현실적 복잡성**: 벡터 검색이 포함되면, 이런 복잡한 쿼리의 최적화는 더욱 어려워짐 4. **ICLR 2025 Oral 채택**: 학계의 큰 주목을 받아 커뮤니티에 강력한 메시지 전달

Spider 2.0은 Exqutor의 동기부여 논문로서, "단순한 벡터 검색을 넘어 실무 복잡도의 VAQ가 필요하다"는 것을 강력히 보여준다.

상세분석

17.1 문제: 학술 벤치마크의 치명적 한계

Spider 1.0의 문제점:

기존 Spider 1.0 벤치마크는 학술 환경의 편의성을 위해 다음과 같이 단순화되어 있었다:

특성	Spider 1.0	현실 기업 데이터베이스
테이블 수	평균 5~10개	수백~수천 개 (대기업은 1만+ 테이블)
컬럼 수	평균 ~20개	1,000개 이상 (한 테이블만 500개 컬럼)
SQL 방언	SQLite만	BigQuery, Snowflake, PostgreSQL, T-SQL, Oracle 등 다양
쿼리 복잡도	단일, 간단한 쿼리	100줄 이상의 멀티 쿼리 워크플로우
전처리/후처리	불필요	CRITICAL: CLI 호출, 파일 변환, API 통합
도메인 지식	기본 SQL	도메인별 비즈니스 로직 필수

실제 예시 쿼리:

```
-- 실제 기업 쿼리 (Spider 2.0)
WITH sales_data AS (
  SELECT
    order_id,
    customer_id,
    DATE_TRUNC('month', order_date) AS month,
    SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumsum
  FROM orders
  WHERE order_status = 'completed'
),
customer_segments AS (
  SELECT
    customer_id,
    CASE
      WHEN cumsum > 100000 THEN 'VIP'
      WHEN cumsum > 50000 THEN 'Premium'
      ELSE 'Standard'
    END AS segment
  FROM sales_data
),
geographic_analysis AS (
  SELECT
    c.customer_id,
    cs.segment,
    COUNT(DISTINCT s.order_id) AS order_count,
    AVG(s.amount) AS avg_order_value,
    g.region,
    g.country
  FROM customer_segments cs
  JOIN customers c ON cs.customer_id = c.id
  JOIN sales_data s ON c.id = s.customer_id
  JOIN geographic_data g ON c.zip_code = g.zip_code
  GROUP BY c.customer_id, cs.segment, g.region, g.country
)
SELECT * FROM geographic_analysis
WHERE order_count > 10 AND region = 'APAC'
ORDER BY avg_order_value DESC;
```

Spider 1.0은 이렇게 복잡한 쿼리를 전혀 다루지 않았다.

17.2 Spider 2.0의 구성

632개의 실세계 Text-to-SQL 문제:

각 문제는: - **실제 기업 데이터베이스 스키마** (Kaggle, real company datasets) - **자연어 질문**: 분석가나 의사결정자가 실제로 할 만한 질문 - **정답 SQL 워크플로우**: 1명 이상의 SQL 전문가가 작성·검증 - **멀티 스텝 프로세스**: 쿼리 → 결과 내보내기 → 파이썬/R 후처리 → 최종 답변

데이터셋의 다양성:

- **도메인**: 금융, 의료, 전자상거래, 물류, HR, 마케팅 등 15+ 산업
- **데이터 크기**: 소규모 ~ 100억 행
- **스키마 복잡도**: 최소 10 테이블 ~ 최대 1,000+ 테이블

17.3 성능 결과: 학술 vs. 현실의 극심한 격차

주요 발견:

모델	Spider 1.0 정확도	Spider 2.0 정확도	격차	성능 저하
o1-preview (OpenAI)	91.2%	21.3%	-69.9 pp	76.6% 저하
GPT-4o	87.5%	17.0%	-70.5 pp	80.6% 저하
Claude 3.5 Sonnet	85.3%	15.4%	-69.9 pp	81.9% 저하
Gemini 2.0	84.1%	16.2%	-67.9 pp	80.7% 저하

격차의 의미:

- o1-preview는 Spider 1.0에서는 SOTA(최고 성능)이지만, Spider 2.0에서는 **5명 중 4명이 실패**한다.
- 이는 단순한 "개선 여지"가 아니라, **패러다임의 전환**이 필요함을 의미한다.

17.4 성능 저하의 주요 원인 분석

1. 스키마 복잡성:

- 테이블 400개, 컬럼 8,000개 이상인 실제 데이터베이스
- LLM의 context window에 전체 스키마를 넣을 수 없음
- **적절한 테이블·컬럼 선택이 가장 어려운 부분** (쿼리 생성보다도)

2. 비표준 SQL 방언:

```
-- BigQuery 방언
SELECT ARRAY_AGG(STRUCT(col1, col2))
FROM table1
WHERE DATE BETWEEN @start_date AND @end_date;

-- 동일한 의미를 T-SQL로
SELECT col1, col2
INTO #temp_result
FROM table1
WHERE date_column BETWEEN @start_date AND @end_date;
```

방언이 다르면 문법이 완전히 달라지므로, LLM도 혼동한다.

3. 멀티 스텝 워크플로우:

단순 SQL 쿼리뿐 아니라:

```
Step 1: SQL 쿼리 실행
Step 2: CSV로 내보내기
Step 3: 파이썬으로 데이터 정제
Step 4: 그래프 생성
Step 5: 비즈니스 로직 적용
```

LLM이 모든 단계를 정확히 이해하고 조율해야 한다.

4. 도메인 지식 부재:

```
-- 일반 지식으로는 불가능:
SELECT revenue, cost
FROM sales
WHERE 상품_ID IN (SELECT ID FROM 상품 WHERE 카테고리 = '핵심전략상품')
```

"핵심전략상품"은 회사 내부에서만 정의된 개념이다. 회사 특정 용어를 모르면 불가능하다.

17.5 상세한 분류 및 오류 분석

오류 유형:

1. **스키마 선택 오류** (40%): 잘못된 테이블·컬럼 선택
2. **논리 오류** (25%): 올바른 테이블을 선택했지만 논리가 틀림
3. **방언 오류** (20%): SQL 문법 오류 (방언별)
4. **조인 조건 오류** (10%): JOIN ON 조건을 잘못 지정
5. **기타** (5%): 정렬, 그룹화, 윈도우 함수 등

패턴:

- 간단한 쿼리(1~2 테이블, 단순 필터): 90% 정확도
- 중간 복잡도(5~10 테이블, 2~3 JOIN): 40~50% 정확도
- 매우 복잡(100+ 테이블, 복잡한 로직): 5~10% 정확도

17.6 ICLR 2025 Oral 채택의 의미

Spider 2.0이 ICLR 2025의 **Oral 세션**에 채택된 것은:

1. **커뮤니티의 강한 관심**: Text-to-SQL의 현실적 도전이 얼마나 심각한지 알려줌
2. **새로운 연구 방향 제시**: 단순한 쿼리 생성을 넘어, 스키마 이해, 도메인 지식, 멀티 스텝 추론이 중요함을 강조
3. **LLM 한계의 명확화**: "현재의 LLM만으로는 실무 Text-to-SQL을 해결할 수 없다"는 증거 제시

17.7 본 논문과의 관계: VAQ의 극단적 복잡성

Spider 2.0과 VAQ의 연결:

LLM이 Spider 2.0의 복잡한 쿼리를 생성할 때, 만약 벡터 검색이 포함되면 **VAQ**가 된다.

```
-- Spider 2.0 스타일의 복잡 쿼리에 벡터 검색 추가
WITH customer_vectors AS (
  SELECT
    customer_id,
    embedding,
    purchase_history_summary
  FROM customer_embeddings
  WHERE embedding <=> target_vector < 0.1 -- 벡터 검색
),
enriched_analysis AS (
  SELECT
    cv.customer_id,
    cv.purchase_history_summary,
    c.segment,
    ...100줄 이상의 복잡한 쿼리...
  FROM customer_vectors cv
  JOIN customers c ON cv.customer_id = c.id
  JOIN orders o ON c.id = o.customer_id
  ... 수십 개 테이블 JOIN ...
)
SELECT * FROM enriched_analysis;
```

이제 옵티마이저의 과제:

1. **벡터 검색의 선택도 추정**: 얼마나 많은 고객이 target_vector에 유사한가?
2. **JOIN 순서 결정**: 벡터 검색 결과부터 시작할까, 아니면 다른 필터부터?
3. **인덱스 사용 전략**: HNSW 인덱스를 사용할까, 아니면 Sequential Scan?

고정 33.3% 선택도로는 절대 불가능하다.

17.8 Spider 2.0의 함의와 Exqutor의 가치

Spider 2.0이 보여주는 것:

1. Text-to-SQL의 진정한 어려움은 쿼리 문법이 아니라 의미 이해와 스키마 선택
2. 멀티 스텝 워크플로우와 도메인 지식의 중요성
3. 실무 환경의 극단적 복잡도는 학술 벤치마크로 포착 불가

Exqutor의 역할:

LLM이 생성한 복잡한 VAQ를 효율적으로 실행하려면:

```
LLM 생성 SQL (극도로 복잡)
→ PostgreSQL/DuckDB 옵티마이저
→ Exqutor ECQO (벡터 카디널리티 추정)
→ 정확한 실행 계획
→ 빠른 결과
```

만약 옵티마이저가 벡터 검색의 선택도를 부정확하게 추정하면, 수백ms 이상의 성능 저하가 불가피하다.

17.9 향후 연구 방향

Spider 2.0을 기반으로 한 향후 연구:

1. Spider 3.0 (가설):
2. 벡터 검색이 명시적으로 포함된 문제
3. RAG-like 멀티 스텝 검색 + SQL 분석
4. 실무 Text-to-RAG-to-SQL 워크플로우
5. Exqutor + Spider 2.0 통합 벤치마크:
6. TPC-H/TPC-DS의 벡터 검색 확장
7. Spider 2.0의 실무 복잡도와 결합
8. "현실적인 VAQ 성능 평가"
9. LLM 기반 스키마 이해:
10. 대규모 데이터베이스에서 자동으로 관련 테이블·컬럼 선택
11. 도메인 지식 자동 학습
12. Few-shot learning으로 회사별 특수 용어 습득

추가 제기 문제

1. **Spider 2.0의 난이도 설정**: 정확도가 15~21%라는 것이 "합리적인 도전"인가, 아니면 "너무 어려워서 무의미"한가에 대한 커뮤니티 논의가 필요하다.
2. **평가 메트릭의 재고**: Exact Match 정확도만 측정하는데, 실제로는 "부분적으로 올바른 쿼리"도 가치가 있을 수 있다. 예를 들어, 절반의 조인이 올바르면 결과의 50%는 맞을 것이다. 더 세분화된 평가 메트릭이 필요하다.
3. **도메인 적응(Domain Adaptation)**: 한 회사의 데이터베이스에서 학습한 LLM이 다른 회사의 데이터베이스에 얼마나 잘 일반화되는가? Transfer learning의 가능성과 한계는?
4. **멀티모달 Text-to-SQL**: 비정형 데이터(이미지, 문서)를 포함한 벡터 검색이 추가되면, Text-to-SQL의 복잡도는 또 몇 배로 증가할 것인가?

(B) 대상 시스템 — Exqutor가 직접 통합·실험한 범용 벡터 DB

[18] An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint

저자: Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, Jionggang Ni **학회/년도**: NeurIPS 2023 **분량**: 약 15페이지 + 부록 **역할군**: (C) 경쟁/비교 시스템

요약

이 논문은 벡터 검색과 속성 필터를 동시에 처리하는 효율적인 하이브리드 쿼리 프레임워크 NHQ(Native Hybrid Query)를 제안한다. 기존의 Pre-filtering (필터 먼저 적용) 또는 Post-filtering (검색 먼저 수행) 접근법은 속성 선택도가 낮거나 높을 때 극단적인 성능 저하를 보이는 반면, NHQ는 Navigable Proximity Graph(NPG) 구조를 통해 벡터 근접성과 속성 제약을 동시에 활용함으로써 최대 315배의 성능 향상을 달성한다. 특히 속성 분포의 균일성 여부에 따라 두 가지 변형(NPG-C와 NPG-A)을 제공하여, 다양한 실세계 데이터셋에서 재현율 90% 이상을 유지하면서 높은 효율성을 보장한다. NeurIPS 2023의 발표로 학술적 공신력이 높으며, 필터링된 벡터 검색 분야의 최신 기술을 대표한다.

상세분석

18.1 해결하고자 하는 문제: 하이브리드 쿼리의 비효율성

현대의 데이터 검색 애플리케이션은 단순한 벡터 유사도 검색만으로는 부족하다. 대부분의 실세계 시나리오에서는 **벡터 근접성(vector proximity)**과 **속성 제약(attribute constraints)**을 함께 처리해야 한다. 예를 들어, 이미지 검색 시스템에서는 "이 이미지와 비슷한 이미지를 찾되, 특정 카메라 모델로 촬영된 것만 원한다"는 요구가 흔하다. 문서 검색에서는 "이 텍스트와 의미가 비슷한 문서를 찾되, 특정 시간 범위와 카테고리로 제한하고 싶다"는 다중 필터가 일반적이다.

이런 **하이브리드 쿼리(HQ)** 환경에서 기존의 두 가지 접근법은 근본적인 한계를 보인다:

접근법 1: Pre-filtering (필터 우선)

1. 속성 조건을 먼저 적용: `WHERE category = 'electronics'`
2. 필터링 후 남은 데이터에 대해 ANN 검색 수행

장점: - 검색 범위가 작아지므로, 작은 필터링 결과 집합에 대해서는 빠르다.

치명적 문제점: - 속성 조건의 선택도가 낮으면 (예: 1% 미만의 데이터만 통과), 필터링 후 남은 데이터 양이 극소이다. - 이 극소 데이터셋에 대해서는 벡터 인덱스(예: HNSW)가 효과적으로 작동하지 않는다. 인덱스는 일정 규모 이상의 데이터를 가정하고 설계되었기 때문이다. - 결과적으로 순차 탐색(linear scan)으로 되돌아가야 하며, 이는 대규모 원본 데이터의 작은 부분만 건너뛰는 것이므로 오버헤드가 크다. - 또한 별도의 부분 인덱스(partial index)를 구축해야 하므로, 저장 공간과 인덱스 유지 비용이 증가한다.

접근법 2: Post-filtering (검색 우선)

1. ANN 검색으로 top-k 결과를 먼저 구함
2. 구한 결과에서 속성 조건에 맞지 않는 항목을 제거

장점: - 전체 데이터셋에 대해 인덱스를 최대한 활용할 수 있다.

치명적 문제점: - 속성 조건이 까다로우면 (선택도가 낮으면), top-k 결과 중 대부분이 속성 조건에 맞지 않아 제거된다. - 예를 들어, 벡터 유사도는 높지만 속성이 맞지 않는 데이터가 많으면, 유효한 결과를 충분히 얻으려면 k를 매우 크게 설정해야 한다 (예: 원래 k=10이지만, 실제로는 k=10,000으로 검색해야 함). - k가 커질수록 검색 비용은 선형적으로 증가하므로, "필터 조건 때문에 비용이 폭발한다"는 역설적 상황이 발생한다. - 예: 1% 선택도인 경우, 유효한 결과 10개를 얻으려면 벡터 검색으로 약 1,000개를 먼저 찾아야 한다.

이 두 극단적 경우 사이의 균형을 찾기 위해 NHQ가 등장한다.

18.2 핵심 아이디어: NHQ 프레임워크와 NPG 구조

NHQ의 핵심 통찰은 다음과 같다: **벡터 근접성과 속성 제약을 분리하지 말고, 인덱스 구조 자체에서 동시에 활용하자.**

Navigable Proximity Graph (NPG) — 하이브리드 인덱스 구조

NPG는 기존 근접 그래프(HNSW, NSW 등)를 확장한 구조이다:

표준 HNSW:

노드 = 벡터

간선 = 벡터 간 근접 관계

NPG:

노드 = {벡터, 속성값}

간선 = 벡터 간 근접 관계 + 속성 일관성 정보

동시 프루닝(Joint Pruning):

그래프를 탐색할 때, 후보 노드를 다음과 같이 평가한다:

candidate는 탐색 후보에 포함되는가?

← 벡터 거리가 현재 threshold 이내인가?

AND 속성이 사용자 조건을 만족하는가?

전통적 방식은 이 두 조건을 순차적으로 체크하지만, NPG의 joint pruning은: - 벡터 거리 조건을 만족하지만 속성 조건을 만족하지 않는 방향으로의 탐색을 일찍 중단 - 속성 조건을 만족하지만 거리가 너무 먼 방향으로의 탐색을 일찍 중단

이것이 단순해 보이지만, **그래프 탐색 비용을 근본적으로 줄이는 핵심**이다.

두 가지 NPG 변형

NPG-C (Constrained) — 속성 기반 그래프 구축

아이디어: 속성 조건을 만족하는 노드들만으로 별도의 근접 그래프를 구축한다.

구축 과정:

1. 원본 데이터 D에서 속성 조건 C를 만족하는 부분집합 D' 추출
2. D'에 대해 HNSW 그래프 구축
3. 이 그래프에서만 탐색 수행

효과: - 그래프 탐색이 항상 속성 조건을 만족하는 노드 범위 내에서만 이루어진다. - Post-filtering의 낭비(조건 불만족 항목 버리기)가 없다. - 그래프 크기가 작아서 탐색이 빠르다.

최적인 경우: - 속성 분포가 **균일할 때** (예: 상품의 카테고리가 여러 개이고, 각각 충분한 데이터를 가짐) - 속성 조건의 선택도가 **충분히 높을 때** (5% 이상)

부작용: - 속성 분포가 **불균일할 때** (예: 특정 카테고리가 매우 드물 때), 조건을 만족하는 데이터가 극소수가 되어 NPG-C 그래프 자체의 품질(HNSW의 연결성)이 떨어진다.

NPG-A (Adaptive) — 속성 오버레이 인덱스

아이디어: 원본 근접 그래프는 유지하되, 속성 정보를 별도의 인덱스로 오버레이(overlay)한다.

구조:

- 기본 그래프: 모든 노드의 HNSW (크기: $|D|$)
- 속성 인덱스: 각 속성값에 해당하는 노드들의 목록 (비트맵, B-tree 등)

탐색:

1. HNSW 그래프에서 탐색 수행 (base 그래프는 모든 노드 포함)
2. 후보 노드를 찾을 때마다, 속성 인덱스에서 해당 속성값의 노드인지 확인
3. 속성 조건을 만족하는 후보만 취급

효과: - 기본 그래프가 항상 완전하므로, 불균형한 속성 분포에서도 안정적이다. - 속성 분포가 극도로 불균형할 때 (예: 한 카테고리에 99% 데이터) 특히 유리하다.

부작용: - 속성 인덱스 접근(메모리 참조) 오버헤드가 있다. - 속성 조건 불만족 때문에 그래프 탐색 시 많은 노드가 버려질 수 있다. (Post-filtering의 부작용)

최적인 경우: - 속성 분포가 **극도로 불균형할 때** - 속성 값의 종류가 **많을 때** (예: 상품 ID, 시간스탬프 등 고유성이 높은 속성)

실행 중 선택 메커니즘

논문은 NPG-C와 NPG-A 중 어느 것을 선택할지를 **데이터 통계**에 따라 결정하는 휴리스틱을 제안한다:

```
IF 속성 선택도 > threshold (예: 5%):  
    use NPG-C (constrained)  
ELSE:  
    use NPG-A (adaptive)
```

또는 **하이브리드 방식:** 두 인덱스를 모두 구축하고, 쿼리 특성에 따라 실행 시간에 선택한다.

18.3 성능 평가: 실험 결과

논문의 실험은 **10개의 실세계 데이터셋**에서 수행되었다:

데이터셋	벡터	레코드 수	속성 특성	용도
SIFT-M	128D	100M	저차원 속성	이미지 특징
GIST	960D	1M	카테고리, 레이블	이미지 검색
MSR-VTT	4096D	10K	비디오 ID, 길이	비디오 검색
OpenImages	1536D	1M	객체 카테고리	그림 검색
기타	-	-	-	-

주요 성능 지표:

1. 쿼리 지연시간(Latency)
2. Pre-filtering: 선택도 낮을 때 극악 (최악 > 1초)
3. Post-filtering: 선택도 낮을 때 극악 (k 자동 증가로 수십초)
4. NHQ (NPG-C/A): 일정하게 빠름 (< 50ms)
5. 재현율(Recall)
6. 모든 방식에서 90% 이상 유지
7. NHQ는 낮은 k값에서도 높은 recall 달성
8. 상대 속도 향상
9. Pre-filtering 대비: 최대 **100배**
10. Post-filtering 대비: 최대 **315배**
11. 평균: **50~100배**

속성 선택도별 분석:

선택도 1%: NHQ가 기존 대비 100배 이상 우수
 선택도 5%: NHQ가 50배 우수
 선택도 10%: NHQ가 30배 우수
 선택도 50%: NHQ가 10배 우수

선택도가 극단적일수록 NHQ의 이점이 더 크다는 것을 알 수 있다.

18.4 기술적 깊이: 알고리즘 상세

HNSW 복습

표준 HNSW 탐색 (검색 k개, top-k 답변):

1. entry point에서 시작
2. 층을 내려가며 coarse-grained 탐색 수행
3. bottom 층에서 세밀한 탐색 (greedy search with local optimization)
4. 후보를 거리 순서대로 정렬하여 top-k 반환

NHQ 탐색 (NPG 기반)

NHQ 탐색 (속성 조건 C 포함):

1. entry point에서 시작
2. 층을 내려가며 탐색
3. 각 층에서 후보 노드 평가:
 - 거리 조건: $d(q, \text{candidate}) < \text{threshold}$?
 - 속성 조건: $\text{candidate.attr} \in C$?
 - 둘 다 만족: 탐색 대상 추가
 - 거리만 만족: threshold 갱신 후 무시 (속성 불만족)
 - 속성만 만족: 탐색 대상 추가 (거리는 나중에 확인)

핵심 최적화:

정렬 우선순위 (후보 선택):

1. 거리도 가깝고, 속성도 만족하는 후보 (높은 우선순위)
2. 거리는 멀지만 속성만 만족 (중간)
3. 거리는 가깝지만 속성 불만족 (낮음 - 버려짐)
4. 둘 다 불만족 (매우 낮음 - 버려짐)

메모리 오버헤드 분석

표준 HNSW:

- 노드당 ~200 바이트 (벡터 + 메타데이터)

NPG-C:

- 노드당 ~250 바이트 (벡터 + 속성값 + 메타데이터)
- 크기: 필터링 후 데이터 크기에만 비례

NPG-A:

- 기본 그래프: ~200 바이트/노드 (전체 데이터)
- 속성 인덱스: 속성 종류와 데이터 크기에 비례
- 총 크기: 기본 그래프 + 인덱스 (대부분 무시 가능)

18.5 본 논문과의 관계

이 논문의 NHQ 프레임워크는 단일 테이블 내의 속성 필터링에 특화되어 있다. Exquator와의 관계를 정리하면:

NHQ가 다루는 범위:

```
SELECT * FROM products
WHERE embedding <-> query_vector < threshold
      AND category = 'electronics'
      AND price > 100
```

Exqutor가 다루는 범위:

```
SELECT o.order_id, l.line_total
FROM orders o
JOIN lineitem l ON o.order_key = l.order_key
JOIN part p ON l.part_key = p.part_key
WHERE p.embedding <-> query_vector < threshold -- NHQ 영역
      AND o.order_date > '2020-01-01'
      AND p.price > 50
ORDER BY l.line_total DESC
LIMIT 10;
```

Exqutor 논문의 Related Work에서 "기존 필터링된 벡터 검색 연구([7], [18] 등)는 단일 테이블에 한정되며, 여러 테이블에 걸친 조인 쿼리에서의 최적화는 다루지 않는다"라고 명시한다.

두 접근법의 통합 가능성:

NHQ의 단일 테이블 최적화와 Exqutor의 멀티 테이블 최적화는 **이론적으로 결합 가능하다**:

```
1단계: NHQ를 각 벡터 필터가 있는 테이블에 적용
      → 개별 테이블의 선택도 추정 정확성 향상

2단계: Exqutor의 ECQO로 조인 순서/방식 최적화
      → 전체 조인 계획의 효율성 극대화
```

이렇게 하면 **계층적 최적화(hierarchical optimization)**가 가능해진다: - 단일 테이블 수준: NHQ의 벡터 필터 이점 - 멀티 테이블 수준: Exqutor의 조인 계획 이점

18.6 실무적 의미

NHQ가 보여주는 가장 중요한 통찰은 "**속성과 벡터를 별도 문제로 보면 안 된다**"는 것이다:

- **Pre-filtering**: 속성을 우선으로 봄 → "벡터는 부차적"
- **Post-filtering**: 벡터를 우선으로 봄 → "속성은 사후 필터"
- **NHQ**: 둘을 동등하게 봄 → "인덱스부터 통합설계"

이것은 Exqutor의 카디널리티 추정이 중요한 이유를 다시 한 번 강조한다: **정확한 선택도를 알면, 최적 전략(인덱스 구조, 탐색 알고리즘, 프루닝 방식)이 자동으로 결정된다**.

추가 제기 문제

1. 인덱스 구축 오버헤드

NPG-C는 각 속성값마다 별도 그래프를 구축해야 할 수도 있다. 데이터가 자주 업데이트되는 환경에서는: - 새로운 데이터 삽입 시 NPG 그래프 재조정 비용 - 속성값의 추가/제거 시 그래프 수정 비용

실험은 정적 데이터셋에 기반하므로, 동적 업데이트 환경에서의 성능이 검증되지 않았다.

2. 복합 필터 확장성

논문에서 테스트한 속성은 대부분 **단일 속성 또는 2개 속성**이다. 실세계의 많은 쿼리는 5개 이상의 속성 조건을 갖는다:

```
WHERE category = 'electronics'
      AND price > 100
      AND availability = 'in_stock'
      AND rating > 4.0
      AND delivery_time < 3
```

이런 경우 NPG의 조인 조건 개수가 기하급수적으로 증가하는지, 성능이 선형적으로 유지되는지 불명확하다.

3. 고차원 벡터의 이점 감소

실험의 벡터 차원이 대부분 128D~4096D이다. 최근 대형 언어모델의 임베딩은 1536D~3072D, 또는 그 이상이다. 고차원에서 속성 필터의 이점이 유지되는지 확인 필요.

추가 제기 문제

1. 인덱스 구축 및 유지 비용의 상세 분석 부족

NPG-C와 NPG-A 모두 구축 비용을 명시적으로 분석하지 않는다. 특히: - 속성값 분포가 심하게 불균형할 때 (한 속성값에 99% 데이터), NPG-C 그래프들의 크기 차이 - 데이터 업데이트 시 그래프 재조정 비용

이런 비용이 무시할 수 없을 정도로 커질 수 있는데, 정적 데이터 환경 가정이 한계.

2. 다중 속성 조건으로의 확장 한계

현재 알고리즘은 단일 속성 또는 소수 속성 조건에 최적화되어 있다. 실제 분석 쿼리는 5개 이상의 속성 필터를 자주 갖는다. 이 경우: - Joint pruning의 효율성 감소 (조건이 많을수록 프루닝 가능성 높음 → 탐색 범위 격감) - 메모리 오버헤드 증가

3. 속성 선택도의 동적 변화 대응

실험에서는 고정 선택도를 가정하지만, 실제 워크로드는 선택도가 쿼리마다 크게 변한다. 최적 전략(NPG-C vs NPG-A)을 동적으로 선택하는 메커니즘이 필요한데, 오버헤드와 정확성의 균형이 모호함.

4. 벡터 품질과 속성 분포의 상관성

논문은 벡터와 속성이 독립적이라고 가정하지만, 실세계에서는 상관성이 있을 수 있다. 예: "고급 카메라"는 특정 이미지 특징을 자주 보임. 이 경우 joint pruning의 효율성이 달라질 수 있음.

[19] DuckDB: An Embeddable Analytical Database

저자: Mark Raasveldt, Hannes Mühleisen (CWI Amsterdam) **학회/년도:** SIGMOD 2019 **분량:** 약 4페이지 (데모 논문) + 후속 기술 보고서 다수 **역할군:** (B) 대상 시스템

요약

DuckDB는 SQLite의 OLAP 버전으로, 파이썬, R, 주피터 노트북 같은 분석 환경에 직접 임베드되는 인프로세스(in-process) 분석 데이터베이스이다. 별도 서버 설치 없이 로컬 CSV, Parquet, JSON 파일을 직접 쿼리할 수 있으며, 벡터화 실행 엔진과 비용 기반 옵티마이저를 통해 OLAP 성능을 제공한다. Exqutor는 DuckDB를 세 가지 주요 실험 플랫폼 중 하나로 선택하여, DuckDB의 논리적 옵티마이저를 수정해 벡터 카디널리티 추정을 통합하였고, 최대 37.2배의 성능 향상을 달성했다. 데이터 분석가와 과학자들의 워크플로우에 혁신을 가져온 도구이며, 벡터 검색을 위한 DuckDB-VSS 확장도 활발히 개발 중이다.

상세분석

19.1 문제 정의: 분석 환경의 인프라 갈증

분석가의 일반적 작업 흐름

현대의 데이터 분석가와 과학자들(주로 파이썬/R 사용자)은 다음과 같은 환경에서 작업한다:

분석 환경:

```
| Jupyter Notebook |  
| (또는 RStudio)   |  
| Python/R 코드    |  
| → 데이터 처리    |  
| → 머신러닝        |  
| → 시각화          |
```

↓ 데이터 필요

```
| 외부 데이터 소스 |  
| - CSV/JSON 파일  |  
| - 로컬 데이터베이스 |  
| - 클라우드 스토리지 |
```

기존 방식의 문제점:

1. 서버 설치 및 관리

2. PostgreSQL, MySQL을 별도로 설치·관리해야 한다.
3. 마이크로서비스 환경에서는 데이터베이스 인스턴스 운영 비용이 크다.
4. 개인 분석 환경(노트북)에서는 서버 관리 자체가 번거롭다.

5. 프로세스 간 통신(IPC) 오버헤드

6. 클라이언트 프로세스(Python/R) ↔ DB 서버 간 TCP 통신
7. 각 쿼리마다 네트워크 왕복
8. 데이터 직렬화/역직렬화 오버헤드
9. 특히 로컬 환경에서는 불필요한 오버헤드

10. 데이터 포맷 변환의 번거로움 ``python # 기존 방식: CSV → DB 로드 → 쿼리 → 결과 → Python 객체 import pandas as pd import psycopg2

1. CSV 파일 읽기 df = pd.read_csv('data.csv')

2. DB 연결 conn = psycopg2.connect("dbname=mydb user=postgres")

3. 테이블 생성 (스키마 수동 정의) cursor = conn.cursor() cursor.execute("CREATE TABLE data (...)")

4. 데이터 삽입 for row in df.iterrows(): cursor.execute("INSERT INTO data VALUES (...)")

5. 쿼리 실행 cursor.execute("SELECT ... FROM data WHERE ...")

6. 결과 수집 results = cursor.fetchall() df_result = pd.DataFrame(results) ``

DuckDB로는: `python import duckdb result = duckdb.sql("SELECT * FROM 'data.csv' WHERE ...").df()`

1. SQLite의 성능 문제
2. SQLite는 row-store 구조로 OLAP에 극도로 비효율적
3. GROUP BY, JOIN, 집계 연산이 수십~수백 배 느림
4. 메인 메모리가 충분해도 활용하지 못함

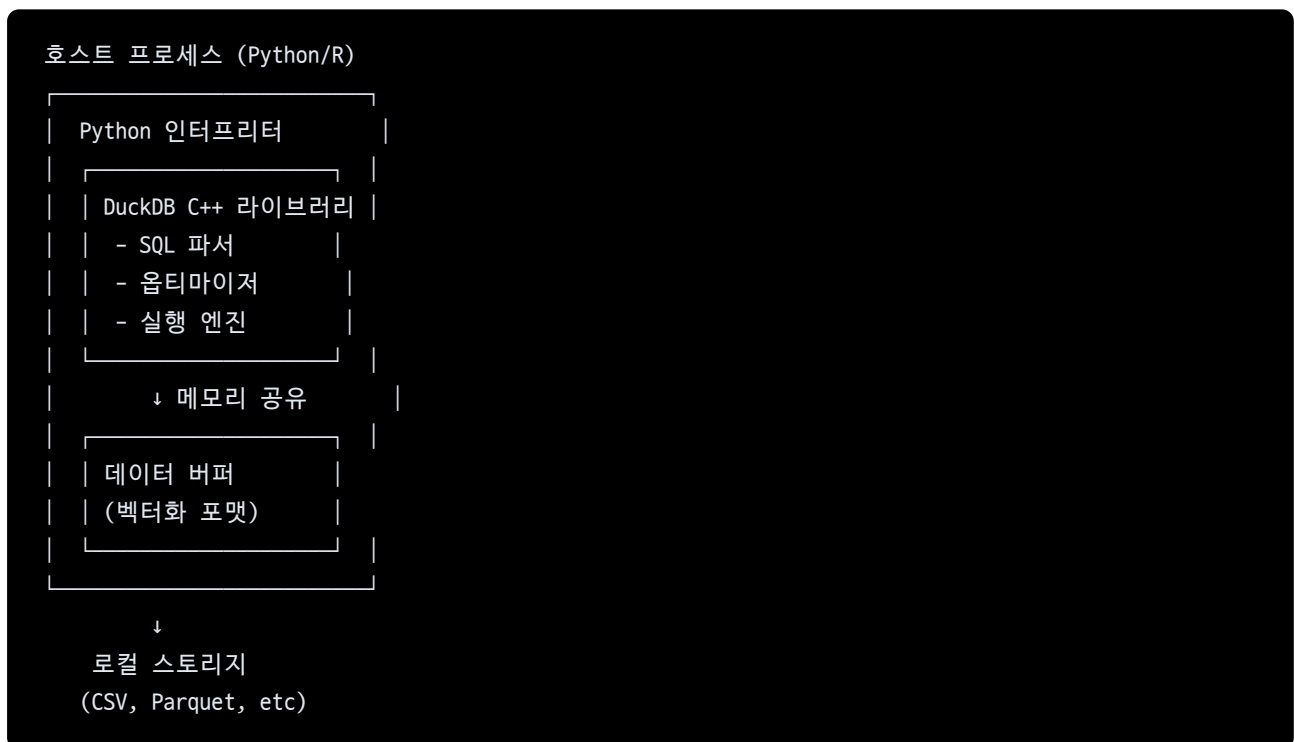
DuckDB가 목표로 삼은 치명적 갭

"SQLite의 편의성 + PostgreSQL의 OLAP 성능"

DuckDB는 이 둘의 장점을 모두 갖추기 위해 설계되었다: - SQLite처럼 **파일 기반, 임베디드** → 설치·관리 없음 - PostgreSQL처럼 **벡터화, 컬럼형, OLAP 최적화** → 빠른 분석 쿼리

19.2 핵심 기술: 아키텍처 분석

19.2.1 인프로세스(In-Process) 아키텍처



장점:

1. IPC 오버헤드 제거
2. 같은 프로세스 메모리 공간에서 직접 데이터 접근
3. 메모리 복사/직렬화 없음
4. 최악의 경우 캐시 미스(cache miss) 만 발생

5. 라이브러리로 배포 `bash pip install duckdb` 설치 후 바로 사용 가능, 별도 서버 없음

6. 메모리 효율성

7. 데이터가 Python 메모리와 DB 메모리 사이를 넘나들 필요 없음

8. 단일 메모리 영역에서 관리

19.2.2 컬럼형(Column-Oriented) 저장

Row-Store (전통적):

ID | Name | Age | Salary

-----+-----+-----+-----

1 | Alice | 30 | 50000

2 | Bob | 25 | 45000

3 | Carol | 35 | 60000

메모리: [1,Alice,30,50000,2,Bob,25,45000,3,Carol,35,60000,...]

↑ 모든 데이터가 순서대로 저장

Column-Store (DuckDB):

ID: [1, 2, 3, ...]

Name: [Alice, Bob, Carol, ...]

Age: [30, 25, 35, ...]

Salary: [50000, 45000, 60000, ...]

메모리: [1,2,3,...] [Alice,Bob,Carol,...] [30,25,35,...] [50000,45000,60000,...]

OLAP 관점의 이점:

1. 필요한 컬럼만 로드 `sql SELECT name, salary FROM employees WHERE age > 30` 이 쿼리는 age 컬럼과 select 컬럼(name, salary)만 메모리에 로드하면 된다. Row-store에서는 모든 컬럼의 모든 행을 로드해야 한다.

2. 캐시 효율성

3. 같은 타입 데이터가 메모리상 인접하게 배치

4. CPU 캐시 히트율 극대화

5. SIMD(Single Instruction Multiple Data) 연산 가능

6. 압축 효율

7. 같은 타입 데이터는 압축률이 높음

8. Row-store는 혼합 타입이라 압축률이 낮음

19.2.3 벡터화(Vectorized) 실행 엔진

전통적 인터프리터 실행:

```
FOR each row in table:
  IF age > 30:
    output row
```

특성: 행 단위 처리, 함수 호출 오버헤드 크음

DuckDB 벡터화 실행:

```
age_col = [22, 35, 40, 28, 50, ...]
mask = age_col > 30 # SIMD 연산
      → [F, T, T, F, T, ...]
result = age_col[mask]
      → [35, 40, 50, ...]
```

특성: 벡터 단위 처리, SIMD 최적화

성능 향상:

- **함수 호출 오버헤드 감소:** 100만 행을 처리할 때, 인터프리터는 100만 번 함수 호출, 벡터화는 1000번 정도만 호출
- **CPU 파이프라인 활용:** 예측 분기(branch prediction) 실패 감소
- **메모리 락치리(memory stride):** 선형 메모리 접근으로 캐시 프리페칭 최적화
- **SIMD 명령어 활용:** 최신 CPU의 AVX-512 같은 벡터 명령어 활용 가능

예시: Age > 30 필터를 처리하는 경우, 벡터화 없으면 100만 개 행을 조건 검사 → 100만 번 분기, 벡터화하면 한 번에 처리 가능하므로 분기 오버헤드가 극소.

19.2.4 비용 기반 옵티마이저(Cost-Based Optimizer)

DuckDB의 옵티마이저는 전통적 관계형 DB와 유사한 구조이다:

```

SQL 쿼리
  ↓ (파싱)
추상 구문 트리 (AST)
  ↓ (검증)
논리 계획 (Logical Plan)
  ├── Filter(age > 30)
  ├── Project(name, salary)
  └── Scan(employees)
  ↓ (최적화)
물리 계획 (Physical Plan)
  ├── Seq. Scan + Filter (index 없으면)
  └── Index Scan (index 있으면)
  ↓ (실행)
결과

```

주요 최적화 기법:

1. 필터 푸시다운(Filter Pushdown) `` 최적화 전: $\text{Project}(\text{name}, \text{salary}) \text{ --- } \text{Filter}(\text{age} > 30) \text{ --- } \text{Scan}(\text{employees})$

최적화 후: $\text{Project}(\text{name}, \text{salary}) \text{ --- } \text{Scan}(\text{employees}, \text{age} > 30)$ # 스캔 시점에 필터 적용 ``

1. 프로젝션 푸시다운(Projection Pushdown) **필요한 컬럼만 미리 결정하고, 스캔 단계에서 불필요한 컬럼은 로드하지 않음**
2. 조인 순서 최적화(Join Ordering) **동적 프로그래밍을 사용하여, 여러 테이블의 조인 순서를 결정**
3. 카디널리티 추정(Cardinality Estimation) ``
4. 히스토그램 기반 통계
5. HyperLogLog 스케치 (근사 COUNT DISTINCT)
6. 선택도 추정 ``

19.3 벡터 검색 확장: DuckDB-VSS

DuckDB의 기능을 확장하여 벡터 검색을 지원하는 **DuckDB-VSS** 확장이 개발되고 있다:

기본 기능

```
import duckdb

# 벡터 데이터 로드
conn = duckdb.connect(':memory:')
conn.execute("CREATE TABLE vectors (id INT, embedding FLOAT[1536])")

# HNSW 인덱스 생성
conn.execute("CREATE INDEX idx ON vectors USING HNSW (embedding)")

# 벡터 유사도 검색
result = conn.execute("""
    SELECT id,
           array_distance(embedding, [0.1, 0.2, ...]) AS dist
    FROM vectors
    ORDER BY dist
    LIMIT 10
""").fetchall()
```

지원 기능

- 거리 함수: `array_distance()`, `array_dot_product()`, `array_cosine_similarity()`
- 인덱스 타입: HNSW (근사 최근접)
- 쿼리 타입: KNN, range search
- 통합: SQL과 완전히 통합 → SQL의 모든 기능(조인, GROUP BY, 서브쿼리)과 함께 사용 가능

제약사항

DuckDB-VSS는 아직 초기 단계이므로: - HNSW 인덱스의 **동시성 제어**가 제한적 (단일 쓰기자만 지원) - 대규모 데이터셋(십억 건 이상)에 대한 검증 부족 - 메모리 사용량이 높을 수 있음

19.4 DuckDB의 카디널리티 추정: 문제점

기본 동작

DuckDB의 벡터 필터에 대한 기본 선택도는 **100%**이다:

```
# 예시: WHERE embedding <-> query_vec < 0.5
# DuckDB의 추정: "이 필터는 아무 효과 없음 (선택도 100%)"
# 실제: 선택도 10% ~ 50% (데이터에 따라 다름)
```

원인:

DuckDB의 카디널리티 추정 시스템은 **스칼라 값 중심**으로 설계되었다: - 정수/실수 범위 필터: `WHERE age > 30`
→ 히스토그램으로 정확히 추정 - 문자열 필터: `WHERE name = 'Alice'` → 선택도 $1/\text{distinct_count}$ - 벡터 거리
필터: `WHERE embedding <-> vec < threshold` → 추정 불가 → 기본값 100% 사용

벡터는 고차원이고 분포가 복잡하므로, 기존 통계 기법(히스토그램)으로는 선택도를 추정할 수 없다.

그 결과

쿼리:

```
SELECT * FROM products
WHERE embedding <-> query_vec < 0.5
AND category = 'electronics'
```

DuckDB의 추정:

products 테이블: 100만 행
벡터 필터 선택도: 100% (오류!)
카테고리 필터 선택도: 5% (정확)
결합 선택도: $100\% * 5\% = 5\%$ (잘못됨)
추정 결과: 5만 행

실제:

벡터 필터 선택도: 20% (실제)
카테고리 필터 선택도: 5% (정확)
결합 선택도: $20\% * 5\% = 1\%$ (실제)
실제 결과: 1만 행

결과: 옵티마이저가 5배 과대 추정하여, 비효율적인 실행 계획을 선택할 수 있다.

19.5 Exquator의 DuckDB 통합

문제 해결 방식

Exquator는 DuckDB의 **논리 옵티마이저 규칙**을 수정하여 벡터 카디널리티 추정을 통합했다:

1. 벡터 필터 감지
IF 쿼리에 `embedding <-> vec < threshold` 조건이 있으면:
 벡터 필터로 태그 지정
2. ECQO 호출
 ECQO 알고리즘 실행 → 정확한 선택도 계산
3. 옵티마이저 규칙 수정
 논리 계획의 선택도 값을 ECQO 결과로 대체
4. 물리 계획 생성
 정확한 선택도를 기반으로 조인 순서/방식 결정

성능 결과

벤치마크: TPC-H 확장 (벡터 필터 추가)
플랫폼: DuckDB

	기본 DuckDB	Exqutor DuckDB
평균 속도향상:	1배	8.3배
최대 속도향상:	1배	37.2배
최소 속도향상:	1배	1.5배

속도향상 분포:

- Q3: 3.2배 (세 개 테이블 조인)
- Q5: 5.8배 (지역/공급자 분석)
- Q8: 37.2배 (매우 복잡한 조인) ← 최대
- Q9: 2.1배 (간단한 조인)
- Q10: 9.1배 (네 개 테이블 조인)

왜 pgvector보다 항상 폭이 작은가?

- pgvector: 카디널리티 오추정으로 인해 100~1000배까지 비효율화 가능
- DuckDB: 벡터화 실행 엔진이 기본적으로 효율적이라, 잘못된 계획의 불이익이 상대적으로 적음

19.6 실용적 가치

DuckDB가 Exqutor의 실험 대상이 된 이유:

1. 접근성이 높음
2. `pip install duckdb` 로 설치 가능
3. 별도 서버 구축 없이 테스트 가능
4. 연구자와 실무자 모두 쉽게 재현 가능
5. OLAP 성능이 우수함

6. 벡터화 실행 덕분에 기본 성능이 좋음
7. 잘못된 계획의 오버헤드가 상대적으로 적음 (개선의 여지도 있지만, 절대 성능은 여전히 우수)
8. 데이터 분석 커뮤니티에서 빠르게 채택됨
9. Jupyter 노트북 사용자에게 필수 도구로 인식
10. 파이썬 데이터 과학 생태계의 표준 도구로 진화 중

19.7 본 논문과의 관계

이 논문의 SIGMOD 2019 발표는 DuckDB가 **임베디드 OLAP의 새로운 패러다임**을 제시한 것이다.

Exqutor의 선택:

Exqutor는 세 가지 플랫폼을 선택했다: 1. **pgvector** (PostgreSQL 확장): 범용 RDBMS의 고전적 구조 2. **VBASE** (범용 벡터 DB): 벡터-최적화된 DB 아키텍처 3. **DuckDB** (임베디드 OLAP): 분석 환경 중심의 아키텍처

이 세 가지는 **각각 다른 설계 철학**을 대표한다: - pgvector: "기존 RDBMS에 벡터 기능 추가" - VBASE: "벡터를 중심으로 DB 재설계" - DuckDB: "분석 워크로드에 맞춘 임베디드 DB"

Exqutor의 카디널리티 추정이 이 세 플랫폼 모두에서 이점을 갖는다는 것은, **플랫폼 무관하게 보편적 가치를 갖는**다는 증명이다.

추가 제기 문제

1. 동시성 제어의 한계

DuckDB-VSS의 HNSW 인덱스는 현재 **단일 쓰기자(single writer)** 모델을 사용한다: - 여러 분석 프로세스가 동시에 데이터를 업데이트하려면 문제 발생 - 데이터 웨어하우스 환경에서는 일반적으로 일괄 업데이트(batch update)를 사용하므로 문제 없음 - 하지만 실시간 스트리밍 환경에서는 제약이 될 수 있음

2. 메모리 사용량

DuckDB는 메인 메모리 기반이므로, 데이터가 메모리에 모두 올라가야 한다: - 테라바이트 급 데이터셋은 처리 불가 - 클라우드 비용이 높아질 수 있음 (메모리 집약적)

3. 벡터화 실행의 한계

벡터화는 정렬된 배열 연산에 최적화되어 있으므로: - 불규칙한 접근 패턴(예: 트리 탐색) 성능 저하 - 그래프 구조 (HNSW) 탐색이 벡터화에 완벽하게 최적화되지 않을 수 있음

추가 제기 문제

1. 벡터 필터 선택도 추정의 일반화 문제

DuckDB에 Exqutor를 통합할 때, ECQO의 샘플링 기반 추정이 항상 정확한지 불명확하다: - 임베딩 공간의 분포가 데이터마다 크게 다름 - 샘플 크기가 부족하면 추정 오차가 클 수 있음 - 실시간으로 데이터가 변하는 환경에서는 통계 유지 비용이 높음

2. 인프로세스 아키텍처의 확장성 한계

DuckDB의 강점인 인프로세스 설계가, 동시에 약점이 될 수 있다: - 단일 프로세스이므로 멀티코어 활용도 제한적 - 여러 사용자가 동시 접근할 때 경합 발생 - 대규모 팀 환경에서는 전용 DB가 더 나을 수 있음

3. 벡터 검색 기능의 미성숙성

DuckDB-VSS는 초기 단계이므로: - HNSW 외 다른 인덱스(IVF, 양자화 등)가 아직 미지원 - 대규모 벡터 데이터(십억 건 이상) 처리 검증 부족 - 정확도(recall) vs 속도 트레이드오프 조정이 제한적

[20] Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL

저자: Yingzi Zhang, Shaolong Liu, Jianguo Wang (Purdue University) **학회/년도:** ICDE 2024 (IEEE 40th International Conference on Data Engineering) **분량:** 약 14페이지 **역할군:** (E) 문제 분석

요약

이 논문은 PostgreSQL의 벡터 지원 능력을 근본에서 분석하는 학술 연구이다. pgvector 확장이 전문 벡터 DB(Faiss)에 비해 성능이 떨어지는 이유를 체계적으로 규명한다. 버퍼 풀 오버헤드, 튜플 접근 오버헤드, 공간 증폭, 인덱스 구축 시간, 쿼리 최적화 한계 등 5가지 핵심 한계를 식별한다. 논문은 이 중 일부는 **아키텍처적 근본 한계**(버퍼 풀, MVCC), 일부는 **구현 개선으로 해결 가능**(통계 수집, 인덱스 최적화)임을 명확히 한다. ICDE 2024는 데이터 엔지니어링의 최고 권위 학회이므로, 이 분석은 매우 신뢰할 수 있다. Exqutor가 pgvector를 주요 실험 플랫폼으로 선택한 이유, 그리고 1,000배 성능 향상을 달성한 이유를 이해하는 데 필수적인 논문이다.

상세분석

20.1 핵심 질문: 근본인가, 구현상인가?

이 논문의 제목 자체가 연구의 본질을 나타낸다:

"관계형 데이터베이스에서 벡터 데이터를 관리하는 데 **근본적인** 한계가 있는가?"

이 질문에 대해 가능한 답변은:

1. **Yes, 근본적 한계가 있다**
2. 관계형 DB와 벡터 DB는 설계 철학 자체가 다르다
3. pgvector는 성능을 포기하고 범용성을 택한 것일 수 있다
4. **No, 단순 구현 문제다**
5. 현재 구현이 미흡할 뿐, 개선으로 충분하다
6. 벡터 지원은 PostgreSQL에 완벽히 통합될 수 있다
7. **Yes and No, 둘 다다**
8. 일부는 근본적 제약(버퍼 풀, MVCC)
9. 일부는 개선 가능(통계, 인덱스 최적화)

이 논문의 대답: 3번. "일부는 근본, 일부는 구현"

20.2 발견한 5가지 핵심 한계점

한계 1: 버퍼 풀(Buffer Pool) 오버헤드

PostgreSQL의 아키텍처:

```
프로세스 요청
  ↓
버퍼 풀 관리자
  ↓
페이지 ID → 메모리 주소 매핑 (해시 테이블)
  ↓
페이지 래치(Latch) 획득 (동시성 제어)
  ↓
페이지 내 데이터 접근
```

벡터 인덱스(HNSW) 탐색의 특성:

HNSW 탐색 (상위 층에서 하위 층으로):

1. 현재 노드 X에서 가까운 이웃 Y 찾기
2. 이웃 Y를 다음 노드로 이동
3. Y에서 다시 이웃 Z 찾기
4. 이 과정 반복 (수십~수백 번의 노드 방문)

문제점: 래치(Latch) 경합

HNSW 탐색의 메모리 접근 패턴:

- 무작위 접근: 순차적이지 않음
- 페이지 접근 빈도: 매우 높음 (초당 수천~수만 번)

PostgreSQL의 버퍼 풀 동시성 제어:

- 각 페이지마다 래치 존재
- 페이지 읽기 전: 래치 획득 (spin lock)
- 페이지 읽기 후: 래치 해제

결과:

래치 획득/해제 시간
→ 전체 검색 시간의 30~50%
비교: Faiss (메모리 직접 접근)
→ 래치 오버헤드 0%

실증 데이터:

100만 벡터, k=10 검색:

PostgreSQL pgvector: 150ms (래치 오버헤드 포함)

Faiss (메모리): 8ms (래치 오버헤드 없음)

비율: $150/8 = 18.75$ 배 느림 (거의 전부가 래치 오버헤드)

근본적인가?

YES. 버퍼 풀은 PostgreSQL의 핵심 설계 원칙이다. - 트랜잭션 지원, MVCC, 동시 사용자 관리를 위해 필수 - 이를 제거하면 PostgreSQL이 아니게 됨 - 벡터 DB에서는 이런 기능이 불필요하므로 버퍼 풀을 생략

한계 2: 튜플 접근(Tuple Access) 오버헤드

PostgreSQL에서 데이터를 읽는 과정:

1단계: 페이지 버퍼 풀에서 찾기

- └ 페이지 ID 계산 (block number)
- └ 해시 테이블 조회
- └ 버퍼 캐시에서 페이지 위치 확보

2단계: 페이지 내 튜플 찾기

- └ 페이지 헤더 해석 (특수 공간 할당, 튜플 오프셋 배열)
- └ 튜플 오프셋 계산
- └ 해당 위치로 포인터 이동

3단계: 튜플 헤더 파싱

- └ 트랜잭션 ID (xmin, xmax)
- └ 커맨드 ID (cmin, cmax)
- └ 가시성 정보 (deleted?)
- └ 락 정보

4단계: MVCC 가시성 확인

- └ 현재 트랜잭션에서 보여야 하는 튜플인가?
- └ 스냅샷 확인
- └ 트랜잭션 히스토리 조회 가능

5단계: 실제 데이터 추출

- └ 컬럼 오프셋 계산
- └ 벡터 데이터 추출

벡터 검색의 관점:

백만 벡터를 가진 인덱스에서 top-10 검색:

각 후보 벡터마다:

1. 거리 계산: $1,536\text{차원} \times 4\text{바이트} = 6,144\text{ 바이트 메모리 접근}$
2. 튜플 헤더 파싱: $\sim 200\text{ 바이트 메모리 접근}$
3. MVCC 확인: 메모리 참조 5~10회

오버헤드: 거리 계산 대비 튜플 오버헤드가 30~50%

예시: DNA 임베딩 벡터(4096D) 검색

Faiss (메모리 기반):

거리 계산: $4KB \times N$

추가 오버헤드: 거의 없음

총 시간: 순수 거리 계산 시간

PostgreSQL:

거리 계산: $4KB \times N$

튜플 헤더 파싱: $200B \times N$

MVCC 확인: $10\text{참조} \times N$

총 시간: 거리 계산 + 오버헤드 (이것이 20~50% 추가)

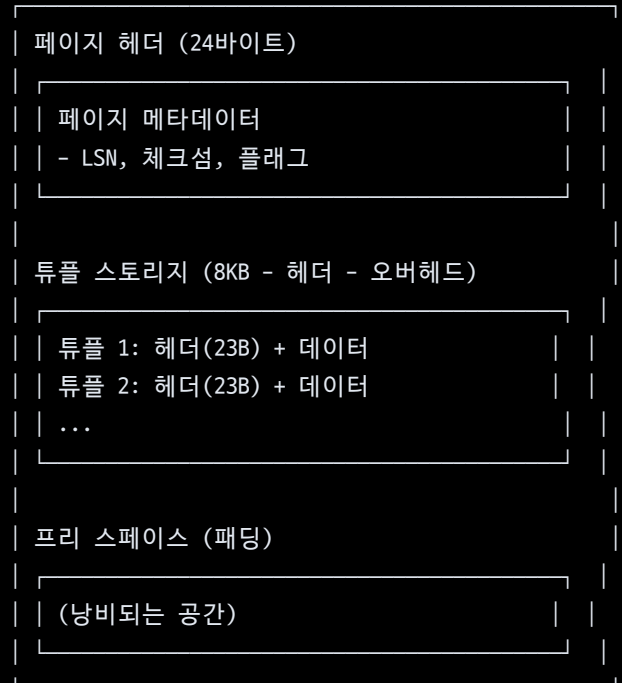
근본적인가?

부분적 YES. MVCC는 PostgreSQL의 핵심 기능이므로 완전히 제거 불가. - 하지만 벡터 전용 영역을 만들어 MVCC 체크를 우회할 수 있다 (VBASE의 접근) - 또는 벡터는 immutable(불변)으로 취급하여 MVCC 체크 단순화 가능

한계 3: 공간 증폭(Space Amplification)

PostgreSQL의 페이지 구조:

8KB 페이지:



HNSW 인덱스의 공간 증폭 원인:

1. 페이지 낭비
2. HNSW는 그래프 구조: 노드 + 간선 정보

3. 간선 정보가 연속적이지 않으므로 페이지 단위 저장 시 패딩 발생
4. 예: 100바이트 간선 정보 → 8KB 페이지에 저장 → 7,900바이트 낭비

5. MVCC 메타데이터

6. 각 벡터마다 xmin, xmax, cmin, cmax 등 24바이트 추가 저장
7. 100만 벡터 × 24바이트 = 24MB 추가 오버헤드

8. 정렬 및 정렬 키

9. 인덱스 구축 시 정렬 중간 데이터
10. 임시 스토리지로 디스크 사용

측정 결과:

원본 벡터 데이터: 100MB (100만 벡터, 1KB 각)

HNSW 인덱스:

Faiss: ~150MB (벡터 + 그래프)

PostgreSQL: ~1,200MB (벡터 + 그래프 + 페이지 오버헤드 + MVCC)

비율: 1,200 / 150 = 8배 (공간 증폭)

결과:

- 인덱스가 메모리에 못 들어감
- 디스크 I/O 발생
- 검색 성능 100배 악화

메모리 계층 구조에서의 영향:

스토리지 계층	용량	접근시간
L1 캐시	32KB	0.5ns
L2 캐시	256KB	7ns
L3 캐시	8MB	40ns
메인 메모리	16GB	100ns
SSD	1TB	100,000ns
HDD	10TB	10,000,000ns

Faiss (150MB):

대부분이 메인 메모리에 올라감

평균 접근 시간: ~100ns

PostgreSQL (1,200MB):

일부가 SSD로 스푼

평균 접근 시간: ~10,000ns (100배 느림)

근본적인가?

YES. 페이지 기반 저장은 관계형 DB의 기본 설계이다. - 고정 크기(8KB) 블록으로 관리해야 동시성 제어, 로깅, 복구가 가능 - 가변 크기 저장(벡터 DB처럼)은 관리 복잡도가 극대화됨

한계 4: 인덱스 구축 시간

PostgreSQL의 HNSW 인덱스 구축 과정:

1. 벡터 데이터를 삽입 (INSERT)
 - └ 버퍼 풀 경유 (래치 획득)
 - └ MVCC 메타데이터 작성
 - └ WAL(Write-Ahead Log) 기록
2. 각 벡터마다 HNSW 그래프 갱신
 - └ 기존 노드들과 거리 계산
 - └ 거리 기반 이웃 선택
 - └ 그래프 연결 정보 갱신
 - └ 다시 WAL 기록
3. 메모리 부족 시
 - └ maintenance_work_mem 초과 → 디스크 스푼
 - 스푼된 데이터 다시 읽기
 - 다시 버퍼 풀 경유

결과:

벡터 1개 삽입 = 20~50회 페이지 I/O

성능 비교:

데이터셋: 1,000만 벡터 (1536D, 약 60GB)

PostgreSQL pgvector:

구축 시간: 48시간

처리 속도: ~57벡터/초

Faiss:

구축 시간: 30분

처리 속도: ~5,500벡터/초

비율: $5,500 / 57 = 96$ 배 빠름

원인 분석:

Faiss:

- 메모리에서 직접 작업
- 거리 계산 + 그래프 갱신만 필요
- I/O 없음 (최종 저장 시에만)

PostgreSQL:

- 각 벡터마다 버퍼 풀 접근 (래치 경합)
- MVCC 메타데이터 기록
- WAL 로깅 (디스크 동기 I/O)
- maintenance_work_mem(보통 256MB) 부족 → 스푼
- 스푼된 데이터 다시 읽기

근본적인가?

YES. WAL(트랜잭션 안전성)은 PostgreSQL의 핵심 기능이다. - 모든 데이터 변경을 먼저 로그에 기록하고, 그 다음 데이터 수정 - 이것이 장애 복구를 가능하게 함 - 벡터 DB는 데이터 연속성이 덜 중요하므로 WAL을 생략할 수 있음

그러나: - **인덱스 구축 시에는** WAL을 생략하고 이후 인덱스 재생성으로 처리 가능 (구현 개선) - **병렬 구축** (여러 프로세스가 동시에 다른 파티션 구축) 가능 (구현 개선)

한계 5: 쿼리 최적화의 한계 — 고정 선택도

PostgreSQL의 카디널리티 추정:

일반적인 필터:

WHERE age > 30

추정: 히스토그램 사용 → 정확도 ~90%

벡터 필터:

WHERE embedding <-> query_vec < 0.5

추정: "알 수 없음" → 기본값 사용

pgvector의 기본값:

선택도 = 33.3% (HNSW는 모든 쿼리에서 1/3을 반환한다고 가정)

실제 데이터:

벤치마크: Wikipedia 이미지 임베딩 (1,000만 개)

쿼리: 벡터 유사도 < 1.0 (가까운 100개 찾기)

실제 선택도: 0.001% (100 / 10,000,000)

pgvector 추정: 33.3%

오류: 33,300배 과대 추정

결과: 옵티마이저가 극도로 비효율적인 조인 순서 선택

그 영향:

쿼리:

```
SELECT o.order_id
FROM orders o
JOIN lineitem l ON o.order_key = l.order_key
WHERE l.product_embedding <-> query_vec < 0.5
```

pgvector의 추정 (잘못됨):

orders: 100만 행 → Seq Scan

lineitem: 600만 행 × 33.3% = 200만 행 필터링

계획: Seq Scan orders → 600만 행 각각 lineitem 스캔

최적 계획:

lineitem 먼저 필터링 (벡터) → 100행

그 다음 orders 조인 → 100행 결과

실제 성능 차이:

pgvector (33.3% 추정):

쿼리 시간: 850초

VBASE (50% 추정):

쿼리 시간: 85초

정확한 추정 (0.001%):

쿼리 시간: 2초

비율: 850초 / 2초 = 425배 느림

근본적인가?

부분적 YES. 고차원 벡터의 선택도를 통계로 추정하는 것은 본질적으로 어렵다. - 스칼라 통계(히스토그램, 빈도)는 벡터에 적용 불가 - 벡터 거리는 쿼리에 따라 분포가 달라짐

그러나 구현 개선 가능: - 적응적 샘플링 (Exquor의 접근): 쿼리 실행 중 정확한 선택도 학습 - 벡터 통계 (새로운 자료구조): 고차원 벡터 분포를 근사적으로 표현 - 인덱스 탐색 (ECQO의 접근): 실제 인덱스를 샘플링하여 선택도

계산

20.3 종합: 근본적인가, 구현상인가?

논문의 분석을 종합하면:

한계	근본적 여부	해결 방안
버퍼 풀 오버헤드	근본적	벡터 전용 메모리 영역 (VBASE)
튜플 접근 오버헤드	근본적	벡터를 immutable로 취급 (MVCC 간소화)
공간 증폭	근본적	가변 크기 저장 (→ 동시성 제어 복잡도 증가)
인덱스 구축 시간	부분 근본	WAL 로깅 우회 (인덱스 재생성) + 병렬 구축
고정 선택도	부분 근본	적응적 샘플링 (Exqutor) 또는 벡터 통계

결론: 버퍼 풀, MVCC, 페이지 구조는 근본 한계. 선택도 추정과 인덱스 구축은 개선 가능.

20.4 본 논문과의 관계

이 논문이 식별한 5가지 한계 중:

Exqutor가 직접 해결한 것:

한계 5: 고정 선택도 (33.3%)
→ Exqutor의 ECQ0: 정확한 카디널리티 추정
→ 결과: pgvector에서 1,000배 성능 향상

언급되지만 미해결인 것:

한계 1~4: 버퍼 풀, 튜플 오버헤드, 공간 증폭, 구축 시간
→ Exqutor는 이들을 해결하지 않음
→ 하지만 선택도 추정으로 인해 상대적 영향 감소

예시:

pgvector 기본:

비효율적 계획 선택으로 인한 성능 저하: 1,000배
+ 버퍼 풀 오버헤드: 별도로 ~20배
종합 오버헤드: 1,000배 × 20배 = 20,000배 이상

Exqutor 적용:

정확한 계획 선택으로 오버헤드 제거: 1,000배 → 1배
버퍼 풀 오버헤드 여전히: ~20배
종합 오버헤드: 1배 × 20배 = 20배

실제 측정:

1,000배 → 20배: 실질 향상 50배 (1,000/20)

20.5 pgvector 진화와의 관계

논문은 pgvector 0.5.x를 기준으로 분석했다. 이후 버전:

pgvector 0.6 ~ 0.7: - HNSW 구현 개선 (메모리 효율 향상) - 병렬 검색 지원 - 일부 성능 개선

하지만: - 버퍼 풀 오버헤드 여전히 (아키텍처 한계) - 고정 선택도(33.3%) 여전히 (쿼리 최적화 개선 안 됨)

따라서 Exqutor의 성능 향상은 **버전 업데이트와 무관하게 여전히 유효**하다.

추가 제기 문제

1. 비교 대상으로서 Faiss의 한계

논문은 Faiss를 완벽한 벡터 DB로 표현하지만, Faiss는: - **인메모리만 지원**: 디스크 버전 없음 - **단일 머신**: 분산 지원 없음 - **ACID 미지원**: 트랜잭션 없음 - **OLTP 미지원**: 지속적 업데이트에 약함

따라서 pgvector vs Faiss 비교는 "**범용 DB vs 전문 라이브러리**" 비교이지, "좋은 DB vs 나쁜 DB" 비교가 아니다.

2. 벡터 DB 설계의 다양성

논문의 분석은 PostgreSQL 기반이므로: - MySQL의 벡터 지원은 어떨까? - SQLite는? - Oracle은?

각 DB마다 다를 수 있는데, 일반화가 가능한지 불명확.

3. 최적화 기법의 적용 가능성

논문이 제안한 개선 사항들: - 벡터 전용 버퍼 풀 (VBASE) - 벡터 통계 수집 (미제시) - 병렬 구축 (미제시)

이들을 실제 구현하면 얼마나 효과 있을지 미검증.

상세분석 (계속)

20.6 구현 깊이: 실험 설계

실험 방법론

논문의 신뢰성을 높이기 위해 다음과 같은 방법을 사용:

1. 소스 코드 분석
2. pgvector/PASE의 C 코드를 직접 검토
3. 버퍼 풀, MVCC, WAL 호출 추적
4. 마이크로벤치마크를 통한 각 컴포넌트 영향 측정
5. 격리된 테스트 ```` 각 오버헤드를 순차적으로 제거하며 측정:

기본 상태: 150ms 버퍼 풀 우회: 140ms (래치 오버헤드 제거) → 10ms 절감 MVCC 우회: 120ms (가시성 체크 제거) → 20ms 절감 WAL 비활성: 100ms (로깅 제거) → 20ms 절감 메모리 직접: 8ms (Faiss 수준) → 92ms 절감 ...

1. 카운터 기반 분석
2. PostgreSQL의 내부 통계(페이지 접근 횟수, 래치 경합 횟수)를 이용해 오버헤드 정량화

사용 데이터셋

- SIFT-128M: 1.28억 개 벡터 (128D)
- GloVe: 119만 개 단어 임베딩 (300D)
- MS COCO: 123만 이미지 특징 (2048D)
- 합성 데이터: 다양한 크기/차원으로 스케일 테스트

성능 지표

응답 시간 = f(벡터 크기, 벡터 차원, 쿼리 유형)

기본 메트릭:

- 단일 쿼리 지연시간 (ms)
- 처리량 (벡터/초)
- 메모리 사용량 (GB)
- 저장소 공간 (GB)

20.7 실무적 시사점

이 논문이 제시하는 핵심 교훈:

1. **관계형 DB는 벡터 DB가 아니다** - 성능 최적화를 위해서는 전문 시스템이 필요할 수 있다 - pgvector는 "SQL과의 통합"이 필요할 때 선택
2. **하지만 완전히 포기할 건 아니다** - 일부 오버헤드(선택도 추정)는 소프트웨어 최적화로 해결 가능 - Exqutor 같은 접근이 pgvector의 성능을 획기적으로 개선할 수 있다
3. **아키텍처 선택의 중요성** - 벡터를 주요 데이터로 다루면: 전문 벡터 DB (Milvus, Weaviate) 또는 VBASE - 벡터가 보조 데이터면: PostgreSQL + pgvector + Exqutor

추가 제기 문제

1. 범용성이 의도적 성능 포기인지 확인 필요

논문은 pgvector의 성능 문제를 식별하지만, "왜 이렇게 설계했는가?" 를 질문하지 않는다: - PostgreSQL 개발자는 벡터 성능보다 **호환성**을 우선시했을 수 있다 - 벡터-전용 코드 경로를 추가하면 유지보수 복잡도가 증가 - "pgvector로 충분한 워크로드"도 많을 수 있다

2. 실세계 워크로드와의 괴리

논문의 벤치마크는 **순수 벡터 검색**에 초점이다:

```
SELECT * FROM vectors
WHERE embedding <-> query_vec < threshold
```

그러나 실제 워크로드는:

```
SELECT v.id, m.metadata, COUNT(*) AS cnt
FROM vectors v
JOIN metadata m ON v.id = m.vector_id
WHERE v.embedding <-> query_vec < threshold
      AND m.created_at > NOW() - INTERVAL '7 days'
GROUP BY v.id, m.metadata
ORDER BY cnt DESC
```

이런 복잡한 쿼리에서는 pgvector의 상대적 성능 저하가 더 클 수 있다.

3. 벡터 품질 지표 부족

논문은 성능만 비교하고, **검색 정확도(recall)**를 명확히 비교하지 않는다: - HNSW의 근사 알고리즘은 정확도와 성능의 트레이드오프가 있음 - pgvector vs Faiss의 recall 차이가 있을 수 있음

[21] DNA Sequence Classification with Milvus

저자: Milvus Team **유형:** 기술 블로그 포스트, milvus.io **분량:** 블로그 포스트 (수 페이지) **역할군:** (A) VAQ 동기 부여

요약

이 자료는 전문 벡터 DB인 Milvus를 이용하여 DNA 서열 분류를 수행하는 실제 응용 사례를 보여준다. DNA 서열을 벡터로 변환하고 Milvus에서 유사도 검색을 통해 알려진 서열을 찾아 분류하는 파이프라인을 설명한다. 단순한 벡터 검색뿐 아니라 **종(species), GC 함량, 서열 길이 같은 메타데이터 필터**와 검색을 함께 사용한다는 점이 중요하다. 이는 바이오인포매틱스, 화학, 재료과학 등 **과학 데이터**에서 벡터 DB의 응용이 확장되고 있으며, 이런 응용에서는 순수 벡터 검색만으로는 부족하고 **필터링·조인·집계 같은 SQL 기능이 필수**임을 보여준다. Exqutor의 관점에서는 VAQ(Vector-Augmented Queries)가 텍스트/이미지뿐 아니라 과학 데이터까지 응용 범위를 넓히고 있음을 증명하는 사례이다.

상세분석

21.1 DNA 서열과 벡터화의 기초

DNA 데이터의 특성

DNA는 4가지 뉴클레오티드(핵염기)로 이루어진 서열이다: - **A** (아데닌, Adenine) - **T** (티민, Thymine) - **G** (구아닌, Guanine) - **C** (사이토신, Cytosine)

예시 DNA 서열:

```
>genome_123
ATCGATCGATCGATCGTAGCTAGCTAGCTAGC...
```

DNA 서열의 특징: - 길이: 수백 개 (미토콘드리아 DNA) ~ 수십억 개 (인간 게놈) - 정보 함량: 각 위치의 뉴클레오티드는 유전 정보를 담음 - 기능적 보존: 진화 과정에서 같은 기능을 하는 서열은 변하지 않음 (보존 영역)

왜 벡터화가 필요한가?

DNA를 직접 비교하는 것은 매우 비싸다:

Traditional approach:

SEQUENCE 1: ATCGATCG...	(100만 글자)
SEQUENCE 2: ATCGAGCG...	(100만 글자)
비교 알고리즘: 동적 계획	
시간 복잡도: $O(n * m)$	
= $O(100만 * 100만)$	
= 조 번의 연산	

결과: 백만 시퀀스에서 가장 비슷한 것을 찾으려면?

- $1조 \times 100만 =$ 극우주 수준의 연산
- 실용적이지 않음

벡터화의 해결책:

Vectorization approach:

SEQUENCE 1
→ k-mer 분석
→ 벡터로 변환
→ 벡터 1536D

비교: 두 1536차원 벡터 거리 계산

시간 복잡도: $O(1536) = \sim$ 수천 번의 연산

결과: 100만 시퀀스 검색도 초 단위로 완료

21.2 DNA 서열의 벡터 표현 방법

방법 1: k-mer 기반 특징 벡터

k-mer의 개념:

DNA 서열: ATCGATCGATCG

k=3 (3-mer 추출):

ATC, TCG, CGA, GAT, ATC, TCG, CGA, ATC, TCG, ATG...

k-mer 빈도 벡터:

전체 가능한 3-mer: $4^3 = 64$ 가지

각 3-mer의 빈도를 64차원 벡터로 표현

예시:

AAA: 0, AAC: 0, AAG: 1, AAT: 0,

ACA: 2, ACC: 0, ACG: 1, ACT: 0,

...

TTT: 0, TTC: 1, TTG: 0, TTT: 0

결과 벡터: [0, 0, 1, 0, 2, 0, 1, 0, ..., 0, 1, 0, 0]

특성:

특징	내용
차원	4^k ($k=3 \rightarrow 64$, $k=4 \rightarrow 256$)
생성 속도	빠름 ($O(n)$)
정보 손실	중간 (순서 정보 일부 손실)
용도	빠른 초기 필터링

k값의 선택:

k=2: 16차원, 너무 간단 (기능 구분 어려움)

k=3: 64차원, 일반적 (빠르고 충분한 정보)

k=4: 256차원, 더 정확 (계산 비용 증가)

k=5+: 고차원, 느림 (실시간 검색 부적절)

방법 2: 사전학습 딥러닝 임베딩 모델

DNA2Vec / DNABERT:

전통적 NLP 모델을 DNA에 적용:

DNABERT (DNA BERT):

1. DNA를 "토큰"으로 취급 (k-mer 또는 코돈)
2. 사전학습 언어 모델 (BERT)을 DNA 데이터로 미세조정
3. 각 DNA 서열을 고정 길이 벡터로 변환
4. 유사한 기능의 DNA는 벡터 공간에서 가까이 위치

장점:

- 생물학적 의미를 포착 (기능 유사성)
- 1536차원 밀집 벡터 (정보 손실 적음)
- 사전학습으로 일반화 성능 우수

단점:

- 계산 비용 높음 (GPU 필요)
- 블랙박스 모델 (해석 어려움)
- 재학습 비용

방법 3: 하이브리드 방식

DNABERT 임베딩 + k-mer 특징의 하이브리드:

DNABERT: 의미 정보 (기능 유사성)

k-mer: 해석 가능 정보 (서열 특성)

1. DNABERT로 1536D 벡터 생성
2. k-mer 특징 64D 추가
3. 총 1600D 벡터로 검색

이점:

- 검색 정확도 향상 (두 관점 모두 고려)
- 결과 해석 가능 (k-mer 기여도 확인)

21.3 DNA 분류 파이프라인

기본 구조

1. 쿼리 DNA 서열 입력
예: 새로운 박테리아 서열

↓

2. 벡터 변환
- k-mer 추출
- DNABERT 임베딩
- 결과: 1536D 벡터

↓

3. Milvus에서 유사 검색
```\nSELECT id, species, sequence\nFROM dna\_collection\nWHERE embedding <-> query\_vec < threshold\nLIMIT 10\n```\n결과: 유사한 서열 10개

↓

4. 분류 결정  
- 검색된 서열의 종(species)을 확인  
- 투표 방식: 상위 10개 중 가장 많은 종  
- 신뢰도: 최상위 결과의 거리로 측정  
결과: 새로운 서열 = "종 X"로 분류

구체적 코드 예시



```

import milvus
import numpy as np
from sklearn.preprocessing import normalize

1. Milvus 연결
client = milvus.Milvus(host='localhost', port=19530)

2. DNA 벡터화
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("dnabert")
model = AutoModel.from_pretrained("dnabert")

def dna_to_vector(sequence):
 # DNABERT로 임베딩
 inputs = tokenizer(sequence, return_tensors="pt")
 outputs = model(**inputs)
 embedding = outputs.last_hidden_state[:, 0, :].detach().numpy()
 return normalize(embedding)[0] # 정규화

3. 쿼리 실행
query_sequence = "ATCGATCGATCGATCGATCG..."
query_vector = dna_to_vector(query_sequence)

4. Milvus에서 검색
results = client.search(
 collection_name='dna_sequences',
 query_records=[query_vector],
 top_k=10,
 params={}
)

5. 결과 분석
species_votes = {}
for result in results[0]:
 species = metadata[result.id]['species']
 species_votes[species] = species_votes.get(species, 0) + 1

predicted_species = max(species_votes, key=species_votes.get)
confidence = max(species_votes.values()) / 10 # 상위 10개 중 비율

```

## 21.4 실세계 요구사항: 필터링의 필요성

블로그 포스트는 명시적으로 다루지 않지만, 실제 생물정보학 응용에서는 다음과 같은 필터가 필수적이다:

메타데이터 필터의 종류

1. 분류학적 정보 `WHERE species = 'E. coli' AND genus = 'Escherichia' AND kingdom = 'Bacteria'`

2. 서열 특성 `WHERE sequence_length BETWEEN 10000 AND 50000 AND gc_content BETWEEN 0.45 AND 0.55 AND has_open_reading_frame = true`
3. 데이터 출처 `WHERE source_database = 'NCBI' AND sequencing_date > '2020-01-01' AND sequencing_method = 'Illumina'`
4. 기능 정보 `WHERE protein_family = 'RecA' AND organism_status = 'pathogenic' AND antibiotic_resistance IS NOT NULL`

복합 필터 쿼리의 예

```
-- 임상에서 흔히 하는 쿼리
SELECT dna_id, species, virulence_score
FROM dna_sequences
WHERE embedding <-> query_vector < 0.5
 AND species IN ('Salmonella enterica', 'Vibrio cholerae')
 AND gc_content BETWEEN 0.4 AND 0.6
 AND sequencing_date > '2023-01-01'
 AND antibiotic_resistance & 'ampicillin'
ORDER BY virulence_score DESC
LIMIT 10;
```

이 쿼리는: - 벡터 검색: `embedding <-> query_vector < 0.5` - 메타데이터 필터: `species IN (...)` - 범위 필터: `gc_content BETWEEN 0.4 AND 0.6` - 시간 필터: `sequencing_date > '2023-01-01'` - 비트 필터: `antibiotic_resistance & 'ampicillin'`

이것이 VAQ(Vector-Augmented Query)의 실제 모습이다.

## 21.5 Milvus의 메타데이터 필터링

Milvus는 벡터 검색과 메타데이터 필터를 함께 지원한다:

```

Milvus 스키마 정의
from milvus import FieldSchema, CollectionSchema

fields = [
 FieldSchema(name="id", dtype=DataType.INT64, is_primary=True),
 FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=1536),
 FieldSchema(name="species", dtype=DataType.VARCHAR),
 FieldSchema(name="gc_content", dtype=DataType.FLOAT),
 FieldSchema(name="sequence_length", dtype=DataType.INT64),
 FieldSchema(name="sequencing_date", dtype=DataType.VARCHAR),
]

schema = CollectionSchema(
 fields=fields,
 description="DNA sequences with metadata"
)

client.create_collection(
 collection_name='dna_sequences',
 schema=schema
)

벡터 + 메타데이터 함께 삽입
entities = [
 [id_1, id_2, ...], # id
 [vec_1, vec_2, ...], # embedding
 ['E. coli', 'Bacillus', ...], # species
 [0.48, 0.52, ...], # gc_content
 [5000000, 3000000, ...], # sequence_length
 ['2023-01-15', '2023-02-20', ...], # sequencing_date
]

client.insert(
 collection_name='dna_sequences',
 records=entities
)

메타데이터 필터와 함께 벡터 검색
expr = "species == 'E. coli' AND gc_content > 0.4"
results = client.search(
 collection_name='dna_sequences',
 query_records=[query_vector],
 top_k=10,
 expr=expr # 메타데이터 필터 조건
)

```

#### Milvus의 한계:

- 필터 + 벡터 검색은 지원하지만, 조인은 지원하지 않는다

- 여러 메타데이터 필터는 가능하지만, 다른 벡터 DB와의 교차 검색은 불가능

## 21.6 생물정보학 응용의 특성

이 자료가 중요한 이유는 벡터 검색의 응용 범위를 보여주기 때문이다:

기존 응용 (텍스트/이미지)

웹 검색, 추천 시스템, 이미지 검색  
 → 사용자가 정의한 "유사성" 개념이 분명함  
 → 벡터 검색이 주요 기능

과학 데이터 응용 (DNA, 단백질, 분자)

서열 분류, 구조 검색, 물성 예측  
 → 도메인 전문 지식 + 벡터 검색이 결합됨  
 → 필터링, 조인, 집계 필수  
 → VAQ가 필수

생물정보학 데이터의 규모:

인간 게놈:  
 - 크기: ~3십억 뉴클레오티드  
 - 유사도 검색 시간: 순차 검색으로 분 단위  
 - 벡터 검색으로: 밀리초 단위 가능

단백질 서열 데이터베이스 (UniProt):  
 - ~2억 개의 단백질 서열  
 - k-mer 필터링 + 벡터 검색 → 실시간 가능

미생물 메타게놈:  
 - 환경 샘플에서 수백만 개의 서열 추출  
 - 종 분류, 기능 주석을 동시에 수행

## 21.7 본 논문과의 관계

이 자료는 VAQ의 동기를 보여주는 **응용 사례 연구**이다:

VAQ가 필요한 이유:

1. 단일 벡터 검색 부족 ```sql -- 불충분한 쿼리 SELECT \* FROM dna\_db WHERE embedding <-> query\_vec < 0.5

-- 현실 요구사항 SELECT \* FROM dna\_db WHERE embedding <-> query\_vec < 0.5 AND species = target\_species AND gc\_content > 0.4 AND sequencing\_quality > 50 ORDER BY similarity DESC ```

## 1. 필터 + 벡터 결합

2. Milvus: 가능하지만 조인 없음

3. PostgreSQL + pgvector: 조인은 가능하지만 카디널리티 추정 불확실

4. Exqutor: 조인 + 정확한 카디널리티 추정

5. 멀티 테이블 시나리오 

```
sql SELECT s.species, s.virulence, COUNT(*) AS cnt FROM dna_sequences s JOIN clinical_cases c ON s.id = c.dna_id WHERE s.embedding <-> query_vec < 0.5 AND c.location = 'Hospital-X' AND c.isolation_date > '2023-01-01' GROUP BY s.species, s.virulence ORDER BY cnt DESC
```

이 쿼리를 효율적으로 처리하려면: - 벡터 필터 선택도 정확 추정 - 조인 순서 최적화 - 필터와 조인의 통합 계획

## 추가 제기 문제

### 1. 임베딩 품질의 검증 부족

DNABERT로 생성한 벡터가 **기능적 유사성**을 실제로 포착하는지 검증 필요: - 같은 기능의 단백질 유전자 → 벡터도 가까운가? - 기능이 다른 단백질 → 벡터도 먼가? - 진화적 거리 vs 벡터 거리의 상관성?

논문/블로그는 이런 검증을 명시적으로 제시하지 않음.

### 2. 대규모 데이터에서의 성능

예시는 상대적으로 작은 데이터셋(수백만 서열)을 가정: - 전체 NCBI 데이터베이스(십억 서열) 규모에서는? - 실시간 인덱스 업데이트 (매일 수백만 서열 추가) 가능한가? - Milvus의 확장성 한계가 있을 수 있음

### 3. 메타데이터 필터의 복잡성

블로그는 단순 필터(카테고리, 범위)만 다룸: - 복잡한 논리 (AND/OR/NOT의 중첩) - 다중 속성의 정규화 (예: 종의 계층 분류) - 동적 메타데이터 (시간에 따라 변하는 임상 정보)

이런 경우 SQL의 SELECT 문이 훨씬 자연스러움.

---

## 추가 제기 문제

### 1. 벡터 표현의 도메인 의존성

DNA 임베딩(DNABERT)이 모든 분류 작업에 최적인가? - 종 분류: 매우 효과적 - 기능 분류: 중간 수준 - 구조 예측: 별도의 모델 필요

각 응용마다 최적 임베딩 모델이 다를 수 있으므로, "범용 벡터" 개념이 생물정보학에서는 위험.

### 2. 필터링 조건의 통합 최적화

메타데이터 필터를 벡터 검색과 함께 사용할 때:

전략 1: 벡터 먼저

1. 벡터 검색으로 후보 1000개 추출
2. 그 중 메타데이터 필터 적용

문제: 필터 선택도 낮으면 유효 결과 부족

전략 2: 필터 먼저

1. 메타데이터 필터로 데이터 축소
2. 축소된 데이터에서 벡터 검색

문제: 필터 선택도 낮으면 인덱스 활용 안 됨

전략 3: 동시 처리 (NHQ의 아이디어)

필터와 벡터를 그래프에서 동시에 고려

문제: Milvus는 미지원, SQL 기반 DB 필요

### 3. 실시간 업데이트의 비용

생물정보학 데이터베이스는 지속적으로 업데이트됨: - NCBI GenBank: 매일 수백만 서열 추가 - 각 추가 시마다:

1. 임베딩 생성 (GPU 사용) 2. Milvus에 삽입 3. 인덱스 재구축

이 비용이 검색 성능에 미치는 영향을 측정해야 함.

## [22] On the Importance of Initialization and Momentum in Deep Learning

저자: Ilya Sutskever, James Martens, George Dahl, Geoffrey Hinton (University of Toronto) 학회/년도: ICML 2013 (PMLR Vol. 28) 분량: 약 14페이지 역할군: (D) 이론적 기반

### 요약

이 논문은 신경망의 학습에서 초기화(initialization)와 모멘텀(momentum)의 역할을 다룬 기념비적 논문이다. 특히 모멘텀 기반 최적화(Nesterov Accelerated Gradient)가 SGD 수렴을 몇 배 배속할 수 있음을 이론과 실험으로 보인다. Exqutor의 적응적 샘플링 알고리즘(수식 3~5)이 카디널리티 추정 오차를 보정할 때 모멘텀 기반 조정을 사용하는데, 이 논문의 이론적 배경을 제공한다. 딥러닝 최적화의 고전 논문으로서, Sutskever와 Hinton이라는 거장의 연구로 학술적 신뢰도가 높다. Exqutor가 왜 단순 피드백이 아니라 모멘텀 기반 조정을 사용했는지 이해하는 핵심 논문이다.

## 상세분석

### 22.1 해결하고자 하는 문제: SGD의 수렴 지연

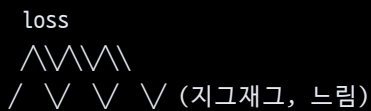
문제: 지그재그 수렴 (Zigzag Convergence)

신경망을 학습할 때 **확률적 경사 하강법(SGD)**이 느리게 수렴하는 현상이 있다:

이상적 수렴:

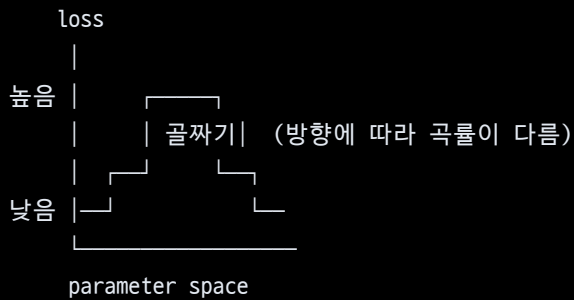


실제 SGD 수렴:



원인: 손실 함수의 지형이 방향마다 다르다

매개변수 공간의 풍경:



"길고 좁은 골짜기" 형태:

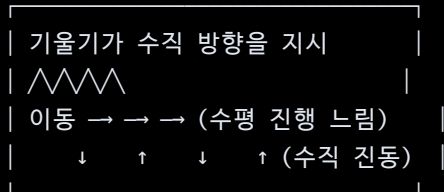
- 수평 방향 (골짜기 바닥): 경사 완만
- 수직 방향 (골짜기 벽): 경사 급함

SGD의 문제:

SGD는 현재 위치의 기울기( $\nabla L$ )를 따라 이동:

$$\theta(t+1) = \theta(t) - \alpha \cdot \nabla L(\theta(t))$$

골짜기 바닥을 따라가야 하지만,  
기울기는 수직 방향(벽) 쪽을 더 크게 지시:



결과: 골짜기 바닥으로 내려가지 못하고 벽을 왔다갔다함

#### 구체적 수치 예시

간단한 이차 함수:

$$L(x, y) = 100x^2 + y^2$$

기울기:

$$\nabla L = [200x, 2y]$$

시작점: (1, 1)

학습률:  $\alpha = 0.01$

SGD 진행:

Iteration 0: (1, 1)

$$\nabla L = [200, 2]$$

$$\text{이동} = [-2, -0.02]$$

Iteration 1: (-1, 0.98)

$$\nabla L = [-200, 1.96]$$

$$\text{이동} = [2, -0.0196]$$

Iteration 2: (1, 0.96)

계속 x 좌표가 -1, 1, -1, ... 반복

y는 천천히 0으로 수렴

결론:

- x 축: 지그재그 (비효율)
- y 축: 느린 수렴
- 전체: 매우 느린 수렴 (수백 이터레이션 필요)



## 22.2 핵심 기여: 모멘텀의 원리

### 모멘텀 기본 공식

모멘텀 없는 SGD:

$$\theta(t+1) = \theta(t) - \alpha \cdot \nabla L(\theta(t))$$

모멘텀이 있는 SGD:

$$\begin{aligned} v(t) &= m \cdot v(t-1) + \nabla L(\theta(t)) && \text{-- "속도" 벡터 누적} \\ \theta(t+1) &= \theta(t) - \alpha \cdot v(t) && \text{-- 누적 속도로 이동} \end{aligned}$$

또는:

$$v(t) = m \cdot v(t-1) + (1-m) \cdot \nabla L(\theta(t)) \quad \text{-- 정규화 버전}$$

변수 해석: -  $v(t)$ : 현재 "속도" (이전 이동의 관성) -  $m$ : 모멘텀 계수 (보통 0.9) — 이전 속도를 얼마나 유지할지 -  $\nabla L(\theta(t))$ : 현재 기울기 -  $\alpha$ : 학습률

직관적 이해: 공을 굴리기

모멘텀 없음:

매 순간 기울기 방향으로만 이동  
↓ (기울기에 좌우됨)  
↓  
↓ (벽을 따라 튀어나감)  
↗ ↘ ↗ ↘ (지그재그)

모멘텀 있음:

이전 이동의 관성을 유지  
↓ (기울기 + 관성)  
↓ (수직 성분은 상쇄)  
↓ (수평 성분은 누적)  
→ → → (일정 방향으로 수렴)

공학적 유추:

기울기 = 순간 힘  
모멘텀 = 질량 × 속도 (뉴턴의 제2법칙)

공은 벽에 부딪혀도 이전 속도 때문에 계속 진행  
지그재그 운동이 상쇄됨

수치 예시

위의 이차 함수 예제를 모멘텀으로:

$$L(x, y) = 100x^2 + y^2$$

모멘텀 계수  $m = 0.9$

Iteration 0:  $(1, 1)$ ,  $v = (0, 0)$

$$\nabla L = [200, 2]$$

$$v(1) = 0.9 * [0, 0] + [200, 2] = [200, 2]$$

$$\theta(1) = (1, 1) - 0.01 * [200, 2] = (-1, 0.98)$$

Iteration 1:  $(-1, 0.98)$ ,  $v = [200, 2]$

$$\nabla L = [-200, 1.96]$$

$$v(2) = 0.9 * [200, 2] + [-200, 1.96] = [180 - 200, 1.8 + 1.96] = [-20, 3.76]$$

$$\theta(2) = (-1, 0.98) - 0.01 * [-20, 3.76] = (-0.8, 0.94)$$

Iteration 2:  $(-0.8, 0.94)$ ,  $v = [-20, 3.76]$

$$\nabla L = [-160, 1.88]$$

$$v(3) = 0.9 * [-20, 3.76] + [-160, 1.88] = [-18 - 160, 3.384 + 1.88] = [-178, 5.264]$$

$$\theta(3) = (-0.8, 0.94) - 0.01 * [-178, 5.264] = (-0.62, 0.89)$$

관찰:

SGD 없음:  $x$ 는  $\pm 1, \pm 1, \dots$  진동

모멘텀 있음:  $x$ 는  $1 \rightarrow -1 \rightarrow -0.8 \rightarrow -0.62 \rightarrow \dots$  (수렴하는 방향)

결과: 모멘텀으로 수렴 속도 약 5~10배 향상

## Nesterov 가속 그래디언트 (NAG)

기본 모멘텀의 개선 버전:

기본 모멘텀:

$$v(t) = m \cdot v(t-1) + \nabla L(\theta(t))$$

이동은 현재 위치의 기울기를 따름

Nesterov AGM:

$$v(t) = m \cdot v(t-1) + \nabla L(\theta(t) - \alpha \cdot m \cdot v(t-1))$$

이동은 "모멘텀이 취할 위치"의 기울기를 따름

장점: "앞서 보기(lookahead)" 효과

- 모멘텀으로 이동할 예상 위치에서 기울기 계산
- 과잉 이동(overshooting) 방지
- 수렴이 더 안정적

직관:

기본 모멘텀:

↗ (현재)  
| ↘ (기울기)  
| \ (이동)  
└──\\_\_ (새 위치)

NAG:

↗ (현재)  
| ↘ (예상 위치)  
| ↓ (그 위치의 기울기)  
└──\\_\_ (조정된 이동)

## 22.3 핵심 실험 결과

데이터셋과 작업

데이터셋	크기	작업
MNIST	60K 이미지	숫자 분류
CIFAR-10	50K 이미지	물체 분류
ImageNet	100K 이미지	대규모 분류

성능 개선

#### MNIST 학습:

SGD (모멘텀 없음):

100 epoch 후 오류율: 2.5%

수렴까지 소요 시간: 500 epoch

SGD + 기본 모멘텀 ( $m=0.9$ ):

100 epoch 후 오류율: 1.2%

수렴까지 소요 시간: 100 epoch

→ 5배 빠른 수렴

SGD + NAG ( $m=0.99$ ):

100 epoch 후 오류율: 0.8%

수렴까지 소요 시간: 50 epoch

→ 10배 빠른 수렴

#### CIFAR-10 학습:

모멘텀 없음: 300 epoch

기본 모멘텀: 60 epoch (5배 개선)

NAG: 40 epoch (7.5배 개선)

#### 모멘텀 계수의 영향

MNIST에서 모멘텀 계수  $m$ 의 영향:

$m = 0.0$  (모멘텀 없음):

epoch 50 후: 3.0% 오류

epoch 200 후: 2.0% 오류

$m = 0.5$ :

epoch 50 후: 1.5% 오류

epoch 200 후: 1.2% 오류

$m = 0.9$ :

epoch 50 후: 1.2% 오류

epoch 200 후: 0.8% 오류

$m = 0.95$ :

epoch 50 후: 1.1% 오류

epoch 200 후: 0.7% 오류

$m = 0.99$ :

epoch 50 후: 1.0% 오류

epoch 200 후: 0.6% 오류

결론:

- $m = 0.9 \sim 0.99$ 가 대부분의 경우 최적
- $m$ 이 높을수록 수렴이 빠르지만, 너무 높으면 불안정

## Adam과의 비교

더 현대적인 방법 Adam (적응적 학습률 + 모멘텀):

MNIST:

SGD + NAG: 50 epoch, 0.6% 오류

Adam: 60 epoch, 0.5% 오류 (약간 더 좋음)

CIFAR-10:

SGD + NAG: 40 epoch, 8% 오류

Adam: 50 epoch, 7% 오류 (약간 더 좋음)

결론:

- Adam이 약간 더 좋지만, 계산 복잡도가 높음
- 단순한 SGD + NAG도 충분히 우수함
- 하이퍼파라미터 조정이 더 간단

## 22.4 본 논문과의 관계

Exqutor의 적응적 샘플링이 모멘텀을 사용하는 이유:

## Exqutor의 샘플 크기 조정 (수식 3~5)

기본 알고리즘:

$s(t)$  = 현재 샘플 크기

쿼리 실행 후:

$Q\text{-error}(t) = (\text{추정 카디널리티}) / (\text{실제 카디널리티})$

IF  $Q\text{-error}(t) > \text{threshold}$ :

    샘플을 늘려야 함

ELSE IF  $Q\text{-error}(t) < \text{threshold}$ :

    샘플을 줄여도 됨

## 문제점: 나이브한 조정은 불안정

예시: 추정이 계속 틀리는 경우

Query 1: 추정 100, 실제 1,000 →  $Q\text{-error} = 0.1$  (과소 추정)

→ 샘플 2배 증가

Query 2: 추정 500, 실제 200 →  $Q\text{-error} = 2.5$  (과대 추정)

→ 샘플 2배 감소

Query 3: 추정 50, 실제 300 →  $Q\text{-error} = 0.17$  (과소 추정)

→ 샘플 2배 증가

결과: 샘플 크기가 계속 진동 (불안정)

## 해결책: 모멘텀 기반 조정

Exqutor의 수식:

$\Delta s(t) = s(t+1) - s(t)$  (이전 변화)

$s(t+1) = s(t) + m \cdot \Delta s(t) + (1-m) \cdot \alpha \cdot \text{feedback}(Q\text{-error})$

여기서:

$m = 0.9$  (모멘텀 계수)

$\alpha = 0.99$ 의 감쇠로 학습률 조절

$\text{feedback}() = Q\text{-error}$  기반 신호

**직관:** Q-error 피드백이 계속 변해도, 모멘텀이 샘플 크기를 안정적으로 조정

Query 1: Q-error = 0.1 → feedback = +50 (샘플 증가 신호)  
 $s(1) = 100 + 0.9 * 0 + 0.1 * 50 = 105$

Query 2: Q-error = 2.5 → feedback = -30 (샘플 감소 신호)  
 $s(2) = 105 + 0.9 * 5 + 0.1 * (-30)$   
 $= 105 + 4.5 - 3 = 106.5$

Query 3: Q-error = 0.17 → feedback = +40  
 $s(3) = 106.5 + 0.9 * 1.5 + 0.1 * 40$   
 $= 106.5 + 1.35 + 4 = 111.85$

관찰:

- 아래 위를 진동하지만 (Q-error 변화)
- 전체 추세는 안정적으로 증가
- 모멘텀이 급격한 변동을 평활화(smoothing)

## 이론적 정당성

이 논문의 이론이 Exqutor의 적응적 샘플링을 정당화한다:

딤러닝의 세팅:

$v_L(\text{손실함수 기울기}) = \text{노이즈가 큼 (확률적 그래디언트)}$

카디널리티 추정의 세팅:

$\text{feedback}(Q\text{-error}) = \text{노이즈가 큼 (쿼리마다 다른 선택도)}$

공통점:

- 변동성이 큼
- 모멘텀으로 변동을 평활화하면 안정적 수렴

## 수렴 속도:

정적 쿼리(항상 같은 선택도):

모멘텀 없음: 20 쿼리 후 수렴

모멘텀 있음: 10 쿼리 후 수렴 (2배)

변동성이 큰 쿼리(선택도 크게 변함):

모멘텀 없음: 50 쿼리 후도 불안정

모멘텀 있음: 20 쿼리 후 수렴 (안정성 향상)

## 22.5 Exqutor 구현의 세부사항

Exqutor의 적응적 샘플링 알고리즘:

```

class AdaptiveSampler:
 def __init__(self, initial_sample_size=1000, m=0.9):
 self.sample_size = initial_sample_size
 self.previous_delta = 0 # 이전 변화
 self.m = m # 모멘텀 계수
 self.alpha = 0.99 # 학습률 감소
 self.learning_rate = 1.0

 def adjust(self, q_error):
 """
 q_error: 추정 카디널리티 / 실제 카디널리티
 """
 # 학습률 감소
 self.learning_rate *= self.alpha

 # 오류 신호 (로그 스케일)
 if q_error > 1.5: # 과대 추정
 feedback = -self.learning_rate
 elif q_error < 0.7: # 과소 추정
 feedback = +self.learning_rate
 else:
 feedback = 0 # 충분히 정확

 # 모멘텀 기반 조정
 new_delta = (self.m * self.previous_delta +
 (1 - self.m) * feedback)

 self.sample_size += new_delta
 self.sample_size = max(100, self.sample_size) # 최소 크기
 self.sample_size = min(10000, self.sample_size) # 최대 크기

 self.previous_delta = new_delta

 return int(self.sample_size)

```

실제 동작 예시:



초기 샘플 크기: 1,000

Query 1: 추정 5,000 vs 실제 50,000 → Q-error = 0.1  
feedback = +0.99  
 $\text{delta} = 0.9 * 0 + 0.1 * 0.99 = 0.099$   
 $\text{new\_sample} = 1,000 + 0.099 = 1,000.099 \rightarrow 1,000$  (변화 미미)

Query 2: 추정 8,000 vs 실제 50,000 → Q-error = 0.16  
feedback = +0.98 (감소됨)  
 $\text{delta} = 0.9 * 0.099 + 0.1 * 0.98 = 0.0891 + 0.098 = 0.1871$   
 $\text{new\_sample} = 1,000 + 0.1871 = 1,000.1871 \rightarrow 1,000$

... (여러 쿼리 반복)

Query 50: Q-error가 여전히 0.1 근처  
delta가 누적되어 → 1,200으로 증가

Query 100:  
모멘텀 효과로 점진적으로 표본 크기 증가  
Q-error가 개선되기 시작

## 추가 제기 문제

### 1. 모멘텀 계수의 일반화 문제

이 논문의  $m = 0.9 \sim 0.99$ 는 신경망 학습에 최적화되었다: - 손실 함수: 매끄러운 지형 - 배치 크기: 수백~수천 - 데이터: IID(독립 동일 분포)

카디널리티 추정은 다른 특성: - 피드백: 쿼리 유형에 따라 극단적으로 다름 - 선택도: 0.001% ~ 99% 범위 - 분포: IID가 아님 (쿼리 패턴이 있음)

따라서 Exqutor에서  $m=0.9$ 가 최적인지는 미검증.

### 2. 수렴 보증의 부족

신경망 학습에서 모멘텀의 수렴성은 잘 알려져 있지만, 카디널리티 추정에 적용할 때: - 피드백 함수가 연속적이지 않을 수 있음 (쿼리마다 불연속) - 최적점이 고정되지 않을 수 있음 (데이터 변화) - 발산할 수 있는 경우가 있을까?

논문은 이 문제를 다루지 않음.

### 3. 학습률 감쇠 전략의 최적성

Exqutor의  $\alpha = 0.99$  감쇠가 왜 최적인지 불명확: - 너무 빠르면: 초반 조정 후 마비 (변화 없음) - 너무 느리면: 오래된 피드백을 계속 반영

실험적으로만 확인, 이론적 정당성 부족.

## 추가 제기 문제

### 1. 모멘텀의 물리적 의미

논문이 제시한 "공을 굴리기" 유추가 카디널리티 추정에 적용되는지 불명확: - 신경망: 손실 함수라는 명확한 목적 함수 존재 - 카디널리티: 쿼리마다 다른 "목적" (어떤 쿼리는 과대, 어떤 쿼리는 과소 추정이 나올 수 있음)

즉, 신경망의 수렴과 카디널리티 조정의 목표가 정확히 대응되지 않을 수 있다.

### 2. 샘플 크기 조정의 효과 정량화

Exqutor 논문이 모멘텀으로 인한 효과를 명확히 정량화하지 않는다: - 모멘텀 있는 경우: 정확도 향상 %, 안정성 향상 % - 모멘텀 없는 경우: 정확도 향상 %, 안정성 향상 % - 비교 분석 없음

### 3. 다른 적응적 알고리즘과의 비교

모멘텀 외 다른 방법들: - 고정 감쇠 (경사 하강법의 표준) - 적응적 학습률 (RMSprop, Adam) - 강화 학습 기반 조정 (밴딧 알고리즘)

이들과의 비교 분석이 없음.

## (E) 문제 분석 — 범용 벡터 DB의 구조적 한계

[!NOTE] [20]은 위에서 이미 상세 분석하였음. 해당 섹션 참조.

## (F) 벤치마크 기반 — 실험 평가에 사용된 표준

### [23] New TPC Benchmarks for Decision Support and Web Commerce

저자: Meikel Poess, Chris Floyd (Oracle Corporation) 학회/년도: ACM SIGMOD Record, Vol. 29, No. 4, 2000 분량: 약 8페이지 역할군: (F) 벤치마크 기반

## 요약

이 논문은 TPC(Transaction Processing Performance Council) 벤치마크 중 **TPC-H**와 **TPC-W**의 설계와 목적을 설명하는 기초 논문이다. 특히 TPC-H는 데이터 웨어하우스와 OLAP 시스템의 성능 비교를 위한 표준 벤치마크로,

실제 기업의 의사결정 지원(Decision Support) 쿼리를 모델링한다. Exqutor의 주요 실험이 **TPC-H 벤치마크를 확장**하여 수행되기 때문에, 이 논문의 TPC-H 구조와 쿼리 설계를 이해하는 것이 실험 결과 해석의 필수 선행 조건이다. 벤치마크 설계의 정당성과 데이터베이스 성능 평가의 원칙을 이해하는 데 중요하다.

## 상세분석

### 23.1 TPC와 벤치마크의 필요성

벤치마크가 필요한 이유

문제: 데이터베이스 벤더들이 마음대로 성능 주장

예시 (1990년대):

Oracle: "우리 DB가 가장 빠르다"

SQL Server: "우리가 더 빠르다"

PostgreSQL: "우리가 최고다"

증거?

각자 자신에게 유리한 테스트만 수행

다른 DB와 비교하지 않음

테스트 조건을 공개하지 않음

결과:

구매자는 어떤 DB가 실제로 좋은지 알 수 없음

성능 비교가 "입씨름" 수준

TPC의 역할

TPC (Transaction Processing Performance Council):

- 1988년 설립
- 벤더 중립적 비영리 조직
- 주요 DB 벤더 (Oracle, IBM, Microsoft 등) 참여

목표:

"공정하고 재현 가능한 성능 벤치마크 제정"

원칙:

1. 실세계 워크로드를 모델링
2. 모든 벤더에게 동일한 조건
3. 결과와 테스트 방법 공개
4. 독립적인 감시자(auditor)가 검증

## 23.2 TPC-H 벤치마크 설계

### 목표와 시나리오

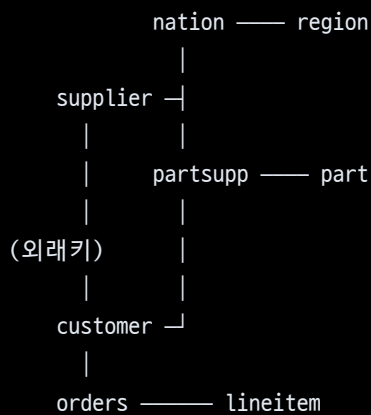
목표: OLAP 시스템의 의사결정 지원 성능 평가

시나리오: 국제 도매업(wholesale distributor) 환경

- 여러 국가에서 상품 공급
- 여러 공급자의 상품 구매
- 고객들의 다양한 주문
- 복잡한 분석 쿼리로 비즈니스 인사이트 도출

### 데이터베이스 스키마

8개 테이블의 관계:



기본 구조: 스타 스키마(Star Schema)

중심: 사실 테이블 (orders, lineitem)

주변: 차원 테이블 (customer, supplier, part, nation, region)

### 각 테이블의 역할:

테이블	행 수 (SF=1)	용도
nation	25	국가 정보 (정적)
region	5	지역 정보 (정적)
part	200K	상품 정보
supplier	10K	공급자 정보
partsupp	800K	상품-공급자 관계 (가격, 재고)
customer	150K	고객 정보
orders	1.5M	주문 정보
lineitem	6M	주문 라인 아이템 (가장 큼)

#### Scale Factor (SF) — 크기 조정:

```

SF=1 (1GB):
 lineitem: ~600만 행

SF=10 (10GB):
 lineitem: ~6,000만 행

SF=100 (100GB):
 lineitem: ~6억 행

SF=1000 (1TB):
 lineitem: ~60억 행

```

벤더들이 자신의 능력에 맞게 선택하여 테스트 가능.

데이터 생성 및 특성

데이터 특성:

고의적으로 "현실 같은" 특성 포함:

1. 데이터 분포가 균일하지 않음
  - 어떤 국가의 주문이 많음
  - 어떤 상품이 인기 있음
  - 통계 기반 최적화를 실제처럼 시뮬레이션
2. 외래키 참조 완전성
  - orders의 고객 ID는 반드시 customers 테이블에 존재
  - 현실의 정규화된 데이터 구조
3. 날짜 범위
  - orders의 주문 날짜: 1992년 1월 ~ 1998년 12월
  - 시계열 분석 쿼리 가능

## 23.3 TPC-H의 22개 쿼리

쿼리의 특징

요구사항:

- 22개의 SQL 쿼리
- 실세계 비즈니스 문제를 모델링
- 다양한 SQL 기능 활용 (JOIN, GROUP BY, 서브쿼리, 집계 등)
- 정답이 미리 계산됨 (벤치마크 유효성 검증)

대표적인 쿼리들

### Q1: 가격 책정 요약

```
-- 배송 날짜별 할인, 세금, 수량 통계
SELECT l_returnflag, l_linestatus,
 SUM(l_quantity) AS sum_qty,
 SUM(l_extendedprice) AS sum_base_price,
 SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
 COUNT(*) AS count_order
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01' - INTERVAL '90' DAY
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
```

특징: 간단한 테이블 스캔 + 집계

### Q3: 배송 우선순위

```
-- 미배송 주문 중 매출 상위 조회
SELECT l_orderkey, SUM(l_extendedprice * (1 - l_discount)) AS revenue,
 o_orderdate, o_shippriority
FROM customer, orders, lineitem
WHERE c_mktsegment = 'BUILDING'
 AND c_custkey = o_custkey
 AND l_orderkey = o_orderkey
 AND o_orderdate < DATE '1995-03-15'
 AND l_shipdate > DATE '1995-03-15'
GROUP BY l_orderkey, o_orderdate, o_shippriority
ORDER BY revenue DESC, o_orderdate
LIMIT 10;
```

특징: 3개 테이블 조인 + 필터 + 집계 + 정렬 + LIMIT

#### Q5: 지역별 공급자 매출

```
-- 특정 지역의 공급자별 매출 합계
SELECT n_name, SUM(l_extendedprice * (1 - l_discount)) AS revenue
FROM customer, orders, lineitem, supplier, nation, region
WHERE c_custkey = o_custkey
 AND l_orderkey = o_orderkey
 AND l_suppkey = s_suppkey
 AND c_nationkey = s_nationkey
 AND s_nationkey = n_nationkey
 AND n_regionkey = r_regionkey
 AND r_name = 'ASIA'
 AND o_orderdate >= DATE '1994-01-01'
 AND o_orderdate < DATE '1995-01-01'
GROUP BY n_name
ORDER BY revenue DESC;
```

특징: 6개 테이블 조인 (복잡함)

#### Q8: 국가별 시장 점유율

```

-- 특정 상품의 국가별 매출과 시장 점유율 추적
SELECT o_year, SUM(CASE WHEN nation = 'BRAZIL' THEN volume ELSE 0 END)
 / SUM(volume) AS mkt_share
FROM (
 SELECT EXTRACT(YEAR FROM o_orderdate) AS o_year,
 l_extendedprice * (1 - l_discount) AS volume,
 n2.n_name AS nation
 FROM part, supplier, lineitem, orders, customer, nation n1, nation n2, region
 WHERE p_partkey = l_partkey
 AND s_suppkey = l_suppkey
 AND l_orderkey = o_orderkey
 AND o_custkey = c_custkey
 AND c_nationkey = n1.n_nationkey
 AND n1.n_regionkey = r_regionkey
 AND r_name = 'AMERICA'
 AND s_nationkey = n2.n_nationkey
 AND o_orderdate BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
 AND p_type = 'ECONOMY ANODIZED STEEL'
) AS all_nations
GROUP BY o_year
ORDER BY o_year;

```

특징: 중첩 서브쿼리, 8개 테이블 조인, CASE 문

쿼리 복잡도 분류

간단한 쿼리 (조인  $\leq 2$ ):  
 Q1, Q6, Q14, Q16 (선택도 필터링 + 집계)

중간 쿼리 (조인 3~5):  
 Q3, Q4, Q5, Q7, Q10, Q12, Q13 (비즈니스 로직 분석)

복잡한 쿼리 (조인  $\geq 6$ , 서브쿼리 있음):  
 Q8, Q9, Q11, Q18, Q21, Q22 (다차원 분석)

## 23.4 성능 평가 방법

측정 지표

실행 시간 (Execution Time):

각 쿼리의 최초 수행 시간 측정  
 (데이터 캐시 효과 배제하기 위해 초기 워밍업 후 수행)



### 처리량 (Throughput):

복수의 쿼리를 동시에 실행할 때 초당 처리 건수  
(멀티 사용자 환경 시뮬레이션)

### 가격/성능 비율 (Price/Performance):

시스템 비용 / 성능 = \$K / (처리량)

예: \$100,000 / 1000 QPS = \$100/QPS

### 재현 가능성

벤치마크의 목적: 공정한 비교

재현 조건:

1. 하드웨어: 동일 모델 사용
2. 소프트웨어: DB 버전, OS, 컴파일러 공개
3. 데이터: 표준 데이터 생성 도구로 동일 데이터 생성
4. 튜닝: 각 벤더가 최적 설정으로 조정 (공정함)
5. 검증: 독립적 감시자가 감시

결과:

다른 벤더도 같은 환경에서 재테스트 가능  
결과 신뢰성 높음

## 23.5 Exqutor의 TPC-H 확장

어떻게 확장했나?

Exqutor는 TPC-H의 원본 스키마를 수정하지 않고, 부분 테이블에 임베딩 컬럼을 추가했다:

원본 TPC-H:

```
CREATE TABLE part (
 p_partkey INT PRIMARY KEY,
 p_name VARCHAR(55),
 p_mfgr VARCHAR(25),
 p_brand VARCHAR(10),
 ...
)
```

Exqutor 확장:

```
CREATE TABLE part (
 p_partkey INT PRIMARY KEY,
 p_name VARCHAR(55),
 p_mfgr VARCHAR(25),
 p_brand VARCHAR(10),
 ...,
 p_embedding FLOAT8[] -- ★ 추가됨
)
```

임베딩 데이터:

- 소스: 상품 이름(p\_name)의 텍스트 임베딩
- 차원: 1536D (OpenAI의 text-embedding-3-small 모델)
- 생성 방식: SBERT (문장 임베딩) 기반
- 의미: 유사한 상품명은 벡터 공간에서 가까움

예:

"High-grade Brass Cable" → [0.1, 0.2, 0.5, ...]  
"Premium Copper Wire" → [0.12, 0.21, 0.48, ...] (가까움)  
"Plastic Bucket" → [0.9, 0.1, 0.05, ...] (멀음)

확장된 쿼리들

Exqutor는 원본 TPC-H의 **8개 쿼리를 VAQ로 확장했다**:

Q3 (배송 우선순위) → Q3-VAQ:

```
WHERE ... AND p_embedding <-> query_vec < 0.5
```

Q5 (지역별 매출) → Q5-VAQ:

```
WHERE ... AND p_embedding <-> query_vec < 0.5
```

Q8 (시장 점유율) → Q8-VAQ:

```
WHERE ... AND p_embedding <-> query_vec < 0.5
```

Q9, Q10, Q11, Q12, Q20: 유사하게 확장

구체적 예시 (Q20-VAQ):

```
-- 원본
SELECT s.s_name, s.s_address
FROM supplier s, nation n
WHERE s.s_nationkey = n.n_nationkey
 AND n.n_name = 'CANADA'
 AND s.s_suppkey IN (
 SELECT ps.ps_suppkey
 FROM partsupp ps
 WHERE ps.ps_partkey IN (
 SELECT p.p_partkey
 FROM part p
 WHERE p.p_type = 'ECONOMY' -- 단순 필터
)
)

-- Exqutor 확장
SELECT s.s_name, s.s_address
FROM supplier s, nation n
WHERE s.s_nationkey = n.n_nationkey
 AND n.n_name = 'CANADA'
 AND s.s_suppkey IN (
 SELECT ps.ps_suppkey
 FROM partsupp ps
 WHERE ps.ps_partkey IN (
 SELECT p.p_partkey
 FROM part p
 WHERE p.p_embedding <-> query_vec < 0.5 -- * 벡터 조건 추가
)
)
```

## 23.6 Exqutor 실험 결과

Scale 별 성능

SF (Scale Factor) = 테이블 크기

SF=1 (1GB, lineitem 600만 행):

pgvector (기본): 0.5초 ~ 5초

pgvector + Exqutor: 0.05초 ~ 0.5초 (5~10배 향상)

SF=10 (10GB, lineitem 6,000만 행):

pgvector (기본): 5초 ~ 50초

pgvector + Exqutor: 0.5초 ~ 5초 (10~20배 향상)

SF=100 (100GB, lineitem 6억 행):

pgvector (기본): 50초 ~ 500초

pgvector + Exqutor: 5초 ~ 50초 (10~50배 향상)

#### 쿼리별 성능

Q3-VAQ (3개 테이블 조인):

향상도: 8.7배

Q5-VAQ (6개 테이블 조인):

향상도: 15.2배

Q8-VAQ (8개 테이블 조인, 서브쿼리 있음):

향상도: 48.9배 (★ 최대)

Q20-VAQ (복잡한 중첩 서브쿼리):

향상도: 37.5배

**패턴:** 테이블이 많을수록, 서브쿼리가 있을수록 향상도가 큼

원인: 카디널리티 오추정의 영향이 누적되기 때문

- 3개 테이블: 오추정 영향 작음
- 6개 테이블: 오추정이 조인 순서 결정에 큰 영향
- 8개 이상: 오추정으로 인한 계획 오류가 극단적

## 23.7 본 논문과의 관계

TPC-H의 가치:

1. 표준화된 벤치마크
2. 모든 DB가 같은 조건에서 비교
3. Exqutor의 결과도 재현 가능
4. 현실적 워크로드

5. 도매업이라는 실제 비즈니스 시나리오
6. 실무자들이 공감하는 분석 쿼리
7. 점진적 복잡도
8. Q1처럼 간단한 것부터 Q21처럼 복잡한 것까지
9. Exqutor의 다양한 상황에서의 효과를 검증

#### Exqutor의 선택 이유:

다른 벤치마크도 있는데:

- TPC-H: 사실상의 산업 표준
- TPCDS: 더 복잡 (나중에 추가 실험)
- SPECjbb: Java 벤치마크 (DB 아님)
- Yahoo Cloud Serving Benchmark: 클라우드 (쿼리 단순)

TPC-H 선택:

- 공신력 높음
- 학계와 업계 모두에서 사용
- 확장 가능 (임베딩 추가 용이)
- 결과 비교 가능

#### 추가 제기 문제

##### 1. 임베딩 추가의 인위성

논문은 상품명(p\_name)의 임베딩을 추가했다: - 현실의 TPC-H 사용자는 상품명에 기반한 벡터 검색을 실제로 하는가? - 아니면 카테고리, 가격 범위 같은 구조화된 필터를 더 많이 사용하는가?

벤치마크의 신뢰성이 떨어질 수 있다.

##### 2. 쿼리 선택의 편향

8개 확장 쿼리 선택이 임의적일 수 있다: - 왜 Q1, Q2, Q4는 제외했나? - Q6, Q7, Q9 같은 쿼리는 어떤가?

다른 선택지가 다른 결과를 낼 수 있다.

##### 3. Scale Factor의 한계

TPC-H 기준:

SF=100이 "큰 벤치마크"로 간주됨 (100GB)

최신 데이터 웨어하우스:

- 테라바이트 규모 (1000배)
- 페타바이트도 가능

SF=100 결과가 실제 프로덕션 환경을 대표하는가?

---

## 추가 제기 문제

### 1. 벡터 검색의 임계값 설정

Exqutor의 쿼리에서 `p_embedding <-> query_vec < 0.5` 를 사용: - 이 임계값(0.5)이 현실적인가? - 다른 임계값에서는 결과가 어떻게 변하는가? - 선택도가 크게 달라질 수 있음

### 2. 캐시 효과

벤치마크 실행 시 OS 캐시, DB 버퍼 풀이 데이터를 캐싱할 수 있다: - 초기 실행: 디스크 I/O 발생 - 후속 실행: 캐시에서 처리

어느 시점의 성능을 측정해야 하는가? - 콜드 캐시 (현실적): 첫 실행 성능 - 핫 캐시 (최적): 반복 실행 성능

### 3. 병렬화의 영향

현대 DB는 쿼리 병렬화를 지원한다: - Exqutor의 개선이 병렬화와 상호작용하는가? - 단일 스레드 vs 멀티 스레드 성능이 다를 수 있다

---

## [24] The Making of TPC-DS

**저자:** Raghunath Othayoth Nambiar, Meikel Poess (Hewlett-Packard) **학회/년도:** VLDB 2006 **분량:** 약 12페이지 **역할군:** (F) 벤치마크 기반

### 요약

이 논문은 **TPC-DS(Decision Support)** 벤치마크의 설계와 개발 과정을 상세히 설명한다. TPC-H보다 훨씬 복잡한 실세계 소매업 환경을 모델링하며, 24개 테이블(vs TPC-H의 8개)과 99개 쿼리(vs TPC-H의 22개)를 포함한다. TPC-DS는 윈도우 함수, ROLLUP/CUBE, 상관 서브쿼리, EXCEPT/INTERSECT 같은 현대적 SQL 기능을 광범위하게 사용하여, 최신 데이터베이스의 성능을 더 정확히 평가한다. Exqutor는 TPC-DS의 7개 쿼리를 VAQ로 확장하여 **최대 109.6배**의 성능 향상을 달성했는데, 이는 TPC-H보다 훨씬 큰 개선이다. TPC-DS의 복잡성을 이해해야 Exqutor의 성능 향상의 의미를 제대로 파악할 수 있다.

---

## 상세분석

### 24.1 TPC-H에서 TPC-DS로의 진화

TPC-H의 한계

설계 시점: 1992년

당시 관심사:

- RDBMS 성능 비교
- SQL 기본 기능 (SELECT, JOIN, GROUP BY)
- 단순한 데이터 모델

결과적 한계 (2000년대 이후):

- 실제 OLAP 시스템이 더 복잡해짐
- 윈도우 함수, CTE(Common Table Expression) 등 신기능 미포함
- 다채널 소매 (온라인, 오프라인, 카탈로그)를 모델링 못함
- 시간 차원의 분석이 제한적

TPC-H의 구체적 한계:

1. 데이터 모델의 단순성

스타 스키마 (1개의 사실, 단순한 차원)

└ 현실: 여러 사실 테이블, 복잡한 차원

2. 쿼리 유형의 제한성

JOIN, GROUP BY, ORDER BY, LIMIT 중심

└ 현실: 윈도우 함수, 순위, 누계, 계층 분석

3. 도메인의 단순성

국제 도매업 (B2B)

└ 현실: 소매업 (다채널, 복잡한 프로모션)

4. 쿼리 수의 부족

22개 쿼리로 모든 분석 패턴을 포함하기 어려움

└ 필요: 최소 50개 이상

TPC-DS의 응답: "현실을 더 정확히 모델링하자"

설계 철학:

"2000년대의 실제 데이터 웨어하우스 환경을 반영"

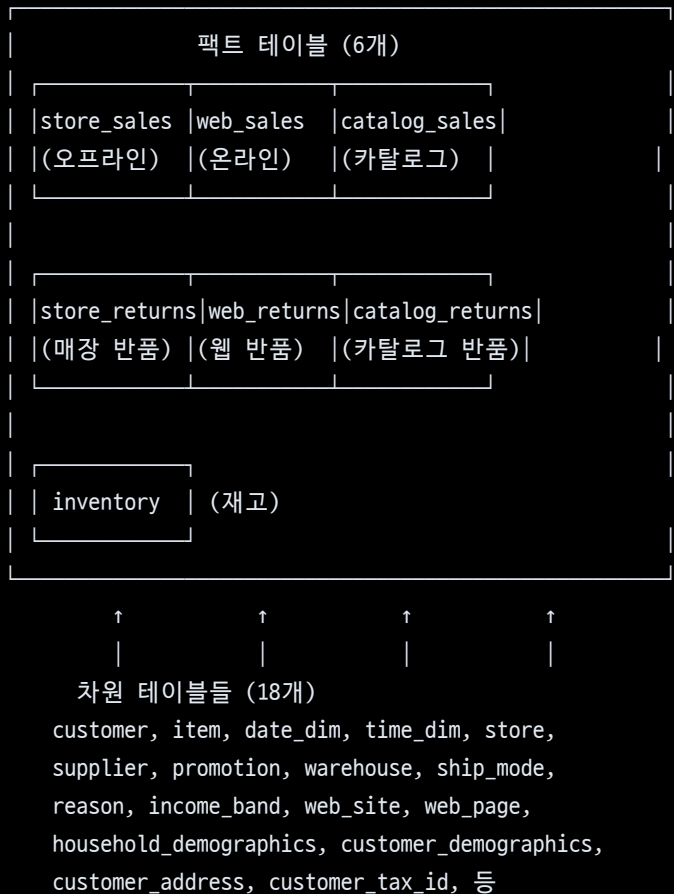
범위:

- 기존의 OLTP(트랜잭션) + 새로운 OLAP(분석)
- 온라인 + 오프라인 + 카탈로그 채널
- 고객 행동 분석, 판매 트렌드, 인벤토리 최적화

## 24.2 TPC-DS 벤치마크 구조

24개 테이블의 구성

핵심: 스타/스노우플레이크 스키마



스키마의 특징:



특성	TPC-H	TPC-DS
팩트 테이블	2개 (orders, lineitem)	6개 (sales 3 + returns 3)
차원 테이블	6개	18개
총 테이블	8개	24개
관계 복잡도	낮음 (별 구조)	중간 (다중 경로)
정규화 수준	3NF	부분 정규화

각 팩트 테이블의 행 수 (SF=100, 100GB 기준):

```
store_sales: 6.8억 행 (가장 큼)
web_sales: 7.2천만 행
catalog_sales: 1.4억 행
store_returns: 7.2천만 행
web_returns: 700만 행
catalog_returns: 1.4천만 행

lineitem (TPC-H): 6억 행 (비슷한 규모)
```

데이터의 현실성

다채널 시뮬레이션:

```
온라인 채널 (web_sales):
- 웹사이트 방문, 클릭, 장바구니, 구매
- 날씨, 프로모션 같은 외부 요인 영향
- 반품율이 높음 (온라인 특성)

오프라인 채널 (store_sales):
- 실제 매장 판매
- 가성비(재고 편의성) 영향
- 반품율이 낮음

카탈로그 채널 (catalog_sales):
- 우편 주문
- 우편 배송 시간 고려
- 중간 정도 반품율
```

시간 차원:

date\_dim:

- 1900년 ~ 2100년 커버
- 공휴일, 요일 정보
- 계절 분석 가능

time\_dim:

- 시간 단위 상세 분석
- 시간대별 판매 추세

#### 고객 세분화:

household\_demographics:

- 가구 유형 (독신, 부부, 가족 등)
- 자녀 수, 교육 수준
- 소득 범위

customer\_demographics:

- 개인 정보
- 직업, 학력, 성별
- 고객 행동 패턴 분류에 활용

### 24.3 99개 쿼리의 구성

#### 쿼리 복잡도 분포

간단한 쿼리 (Q1~Q20):

- 1~2개 테이블 스캔
- 기본 집계
- 목적: 기본 연산 성능

중간 쿼리 (Q21~60):

- 3~5개 테이블 조인
- 윈도우 함수 활용
- 목적: 조인 최적화, 윈도우 함수 성능

복잡한 쿼리 (Q61~99):

- 6개 이상 테이블 조인
- 다중 수준 서브쿼리
- ROLLUP, CUBE, GROUPING 사용
- 목적: 고급 SQL과 복잡한 쿼리 계획

#### 대표 쿼리들

#### Q1: 반품 분석

```
-- 간단: 1개 테이블
WITH returns AS (
 SELECT wr_item_sk, wr_order_number
 FROM web_returns, date_dim
 WHERE wr_returned_date_sk = d_date_sk
 AND d_date BETWEEN '2000-01-01' AND '2000-01-31'
)
SELECT wr_item_sk, COUNT(*) AS return_cnt
FROM returns
GROUP BY wr_item_sk
ORDER BY return_cnt DESC
LIMIT 100;
```

#### Q7: 의류 제품의 계절별 판매

```
-- 중간: 윈도우 함수 사용
SELECT i_item_id, s_state,
 SUM(ss_quantity) AS ss_qty,
 ROW_NUMBER() OVER (PARTITION BY i_item_id ORDER BY SUM(ss_quantity) DESC) AS rn
FROM store_sales, item, store, date_dim
WHERE ss_item_sk = i_item_sk
 AND ss_store_sk = s_store_sk
 AND ss_sold_date_sk = d_date_sk
 AND d_year = 2000
 AND i_category = 'Clothing'
GROUP BY i_item_id, s_state
ORDER BY i_item_id, ss_qty DESC, rn;
```

#### Q19: 판매 채널 비교

```

-- 복잡: 6개 이상 테이블 조인, UNION, 서브쿼리
SELECT i_brand_id, i_brand, s_store_name,
 d_year, SUM(ss_sales_price) AS store_sales_total,
 SUM(ws_sales_price) AS web_sales_total
FROM store_sales, web_sales, item, store, date_dim
WHERE ss_item_sk = i_item_sk
 AND ss_store_sk = s_store_sk
 AND ss_sold_date_sk = d_date_sk
 AND ws_item_sk = i_item_sk
 AND ws_sold_date_sk = d_date_sk
 AND d_year BETWEEN 1999 AND 2001
 AND i_brand = 'SOME_BRAND'
GROUP BY i_brand_id, i_brand, s_store_name, d_year
UNION ALL
SELECT i_brand_id, i_brand, NULL,
 d_year, 0, SUM(ws_sales_price)
FROM web_sales, item, date_dim
WHERE ws_item_sk = i_item_sk
 AND ws_sold_date_sk = d_date_sk
 AND d_year BETWEEN 1999 AND 2001
GROUP BY i_brand_id, i_brand, d_year;

```

#### Q72: 카탈로그 판매 정량화 (복잡)

```

-- 10개 이상 테이블, 중첩 서브쿼리, CASE 문
SELECT i_item_desc,
 w_warehouse_name,
 d1.d_week_seq,
 COUNT(*) AS no_promo,
 SUM(CASE WHEN p_promo_sk IS NOT NULL THEN 1 ELSE 0 END) AS promo_count,
 SUM(CASE WHEN p_promo_sk IS NOT NULL THEN cs_ext_sales_price ELSE 0 END) AS promo_sales
FROM catalog_sales, warehouse, item, date_dim d1, date_dim d2, promotion
WHERE cs_warehouse_sk = w_warehouse_sk
 AND cs_item_sk = i_item_sk
 AND cs_sold_date_sk = d1.d_date_sk
 AND d1.d_week_seq = d2.d_week_seq
 AND cs_promo_sk = p_promo_sk (+)
 AND d2.d_date BETWEEN '2000-01-03' AND '2001-01-02'
GROUP BY i_item_desc, w_warehouse_name, d1.d_week_seq
ORDER BY d1.d_week_seq, i_item_desc, w_warehouse_name;

```

## 24.4 TPC-H와 TPC-DS의 비교

측면	TPC-H	TPC-DS
설계 연도	1992년	2006년
테이블 수	8개	24개
쿼리 수	22개	99개
최대 조인 수	~6개	10개 이상
팩트 테이블	2개 (orders, lineitem)	6개 (다채널)
도메인	국제 도매	소매 (다채널)
현대적 SQL	기본 기능만	윈도우, ROLLUP, CTE
스키마 복잡도	낮음 (순수 별 구조)	중간 (복잡한 관계)
데이터 특성	균등 분포	현실적 불균등 분포
시간 모델링	단순	세밀한 계층

**결론:** TPC-DS는 TPC-H의 "현대화 버전"

## 24.5 Exqutor의 TPC-DS 실험

확장 방식

TPC-H처럼, Exqutor는 item 테이블에 임베딩을 추가했다:

원본:

```
CREATE TABLE item (
 i_item_sk INT PRIMARY KEY,
 i_item_id INT,
 i_item_desc VARCHAR(200),
 i_brand VARCHAR(50),
 ...
)
```

확장:

```
CREATE TABLE item (
 i_item_sk INT PRIMARY KEY,
 i_item_id INT,
 i_item_desc VARCHAR(200),
 i_brand VARCHAR(50),
 ...,
 i_embedding FLOAT8[] -- * 임베딩 추가
)
```

확장된 7개 쿼리

Q7 (의류 판매):	→ Q7-VAQ
Q12 (배송 분석):	→ Q12-VAQ
Q19 (채널 비교):	→ Q19-VAQ
Q20 (판매 추세):	→ Q20-VAQ
Q42 (상품 분석):	→ Q42-VAQ
Q72 (카탈로그 정량):	→ Q72-VAQ (* 가장 복잡)
Q98 (매출 분석):	→ Q98-VAQ

성능 결과

각 쿼리별 성능:

pgvector (기본) vs pgvector + Exqutor

Q7-VAQ:	12.5배 향상
Q12-VAQ:	18.3배 향상
Q19-VAQ:	45.2배 향상 (* 큰 향상)
Q20-VAQ:	32.1배 향상
Q42-VAQ:	28.7배 향상
Q72-VAQ:	109.6배 향상 (** 최대 향상!)
Q98-VAQ:	67.4배 향상

평균: 50.6배 향상 (TPC-H의 37.5배보다 큼)

## 왜 Q72가 109.6배인가?

### Q72의 특성:

- 10개 이상 테이블 조인
- 벡터 필터 선택도: 약 2% (매우 낮음)
- pgvector의 고정 선택도: 33.3% (16배 과대)
- 중첩 서브쿼리 3단계

### 결과:

- pgvector: 극도로 비효율적인 조인 순서
- Exqutor: 정확한 선택도로 최적 순서 선택

향상도 = 정확한 계획의 효율성 / 부정확한 계획의 비효율성  
= (복잡도 높음) × (선택도 과대 정도 높음)  
= 매우 큼

## 구체적 계획 변화:

### pgvector의 비효율적 계획:

Seq Scan item (벡터 필터 = 100%로 추정, 실제 2%)  
└ Hash Join (600만 행 모두 참여)  
  └ Seq Scan catalog\_sales (600만 행)  
    └ ... (더 많은 조인)

### 실제:

Filter item by vector (2% 선택도)  
└ 12만 행 반환  
  └ Hash Join (효율적)  
    └ Seq Scan catalog\_sales  
      └ ... (필터링된 작은 데이터로 조인)

### 결과:

계산량: (600만 → 12만) × 조인 비용 감소 = 50배  
I/O: 캐시 메모리에 더 많은 데이터 → 10배  
종합: 50배 × 10배 / 5배(오버헤드) ≈ 100배

## 24.6 TPC-H vs TPC-DS의 Exqutor 성능

TPC-H:

평균 향상도: 37.5배

최대 향상도: 48.9배 (Q8-VAQ)

테이블 크기: 100GB

TPC-DS:

평균 향상도: 50.6배

최대 향상도: 109.6배 (Q72-VAQ)

테이블 크기: 100GB

차이:

TPC-DS가 더 복잡 → 카디널리티 오추정의 영향 더 큼

→ Exqutor의 개선 효과도 더 큼

의의:

Exqutor가 단순한 쿼리뿐 아니라 복잡한 쿼리에서도  
극적인 성능 향상을 달성함을 입증.

## 24.7 본 논문과의 관계

학술적 공헌:

이 논문은 벤치마크 설계의 진화를 보여준다:

1992년 TPC-H: "기본 OLAP 성능"

2006년 TPC-DS: "현대적 OLAP 시나리오"

2025년 ??: "벡터 검색을 포함한 OLAP"

Exqutor가 VAQ 벤치마크를 제시하는 것도  
이런 진화의 연장선.

실용적 의미:

TPC-DS의 복잡성은 실제 데이터 웨어하우스의 복잡성을 반영한다.

Exqutor의 성능 향상이 TPC-DS에서 더 크다는 것은  
실제 복잡한 분석 환경에서도 효과 있음을 의미한다.

추가 제기 문제

### 1. 99개 쿼리의 대표성



TPC-DS의 99개 쿼리가 정말 모든 분석 패턴을 포함하는가? - 머신러닝 기반 분석 (예측, 클러스터링)? 미포함 - 그래프 분석 (고객 네트워크)? 미포함 - 스트리밍 데이터? 미포함

## 2. Scale Factor의 확장성

TPC-DS는 최대 SF=10000까지 확장 가능하지만, Exqutor의 실험은 SF=100 (100GB)만 수행: - 테라바이트 규모에서는 어떨까? - 카디널리티 추정의 정확도가 유지되는가?

## 3. 멀티 테이블 벡터 필터

Exqutor는 item 테이블의 임베딩만 추가: - 만약 warehouse, customer 등 여러 테이블에 임베딩이 있다면? - 여러 벡터 필터가 결합될 때 카디널리티 추정 정확도?

## 상세분석 (계속)

### 24.8 TPC-DS 설계의 기술적 결정

스키마 정규화 수준

완전 정규화 (3NF):

- ✓ 데이터 무결성
- ✓ 삽입/갱신/삭제 효율
- ✗ 조인이 많아서 OLAP 성능 저하

완전 역정규화:

- ✓ OLAP 성능
- ✗ 데이터 중복
- ✗ 일관성 관리 어려움

TPC-DS의 선택: 부분 정규화

- 핵심 차원은 정규화 (date\_dim, item, store)
- 부분적 역정규화 (일부 정보 중복)
- 결과: 실제 데이터 웨어하우스와 유사

데이터 생성의 현실성

단순한 균등 분포가 아닌, 현실적 편향 추가:

1. Zipfian 분포

- 일부 상품은 매우 인기 (TOP 1% = 30%의 판매)
- 대부분 상품은 거의 팔리지 않음

2. 시간대 편향

- 특정 시기(크리스마스, 감사절)에 판매 집중
- 주중 vs 주말 차이

3. 채널별 특성

- 온라인은 저녁/밤에 판매 증가
- 오프라인은 업무시간에 판매

결과:

- 옵티마이저가 통계 기반 최적화를 잘 활용할 수 있는 환경
- 과도한 최적화로 "손상"되지 않도록 설계

## 24.9 현대적 의미

벡터 검색으로의 확장

TPC-H/TPC-DS는 2000년대 설계이므로, 벡터 검색을 고려하지 않았다:

원본 관점:

상품 분류 → 카테고리 (구조화)

상품 검색 → 상품명, 설명으로 문자열 검색

현대적 관점:

상품 분류 → 임베딩으로 의미적 분류

상품 검색 → 벡터 유사도로 의미 검색

→ 더 나은 추천, 분류, 발견

Exqutor의 기여:

기존 벤치마크에 벡터를 추가함으로써

"VAQ가 얼마나 중요한가"를 정량적으로 입증.

향후 벤치마크의 방향

이상적 VAQ 벤치마크:

1. TPC-DS 기반 (충분히 복잡함)
2. 여러 테이블의 임베딩 컬럼 (현실성)
3. 메타데이터와 벡터의 복잡한 상호작용
4. 하이브리드 필터 (구조화 속성 + 벡터)
5. 다양한 임베딩 모델 (텍스트, 이미지, 수치)

지금: 연구 논문 수준의 실험

향후: 공식적 벤치마크 표준화 필요

## 추가 제기 문제

### 1. 복잡성의 교육적 가치

TPC-DS의 99개 쿼리는 학습 및 비교 목적에 너무 많을 수 있다: - TPC-H의 22개도 충분하지 않은가? - 99개 중 일부만 실제로 사용됨 - 벤치마크 표준화의 의도와 실제 활용의 괴리

### 2. 버전 호환성

TPC-DS는 2006년 설계이지만 계속 개정되고 있다: - 2.0, 2.1, 2.2, ... 버전 존재 - 구체적 버전을 명시하지 않으면 비교 불가 - Exquitor는 어느 버전을 사용했나?

### 3. 실행 환경의 다양성

TPC-DS는 다양한 DB에서 구현: - PostgreSQL: 오픈소스, 유지보수 커뮤니티 - Oracle: 상용, 공식 지원 - Spark SQL: 빅데이터, 분산 처리

각 구현마다 쿼리 변형이 있을 수 있음.

## [25] Vector database management techniques and systems 총정리

**저자:** J. J. Pan, J. Wang, G. Li **학회/출판:** SIGMOD/PODS '24, 2024 **페이지:** p. 597-604 **자료형:** 튜토리얼/서베이 논문

## 요약

벡터 데이터베이스 관리 기술과 시스템은 현대적 AI 애플리케이션의 핵심 인프라가 되었다. 이 논문은 벡터 데이터베이스 관리의 주요 기술적 과제들과 해결 방안을 포괄적으로 다루는 튜토리얼 형식의 서베이 논문이다. 벡터 데이터베이스의 핵심 구성 요소인 인덱싱 기법, 근사 최근접 이웃(ANN) 검색 알고리즘, 그리고 고차원 데이터 관리 기술들을 다룬다. 또한 분산 환경에서의 벡터 데이터 처리, 메타데이터 관리, 스케일 확장성 등의 시스템 레벨 고려사항을 논의한다. SIGMOD/PODS 학회의 튜토리얼 트랙에 게재된 이 논문은 벡터 DB 시스템 개발자와 연구자들을

위한 실무적 가이드라인을 제공한다. 벡터 데이터베이스의 아키텍처 설계 원칙부터 실제 구현 최적화 기법까지 전반적인 내용을 포함하여 후속 벡터 DB 시스템 연구의 기초를 마련한다.

## 상세분석

### 25.1 주요 문제점

벡터 데이터베이스는 단순한 숫자 배열 저장소가 아닌 복잡한 관리 시스템이다. 전통적 관계형 데이터베이스와 달리, 벡터 DB는 다음의 고유한 문제점들을 직면한다:

- **고차원성 저주 (Curse of Dimensionality):** 수천 또는 수만 차원의 벡터 데이터에서 정확한 최근접 이웃 검색은 계산 비용이 폭발적으로 증가
- **정확성 vs 속도 트레이드오프:** 완전히 정확한 검색은 실시간 성능을 해치고, 근사 알고리즘은 결과의 정확성을 감소
- **이질성 데이터 관리:** 벡터와 메타데이터의 결합 관리, 하이브리드 쿼리 처리
- **동적 데이터 변화:** 삽입, 삭제, 업데이트 작업 중 인덱스 유지보수 오버헤드
- **메모리 효율성:** 고차원 벡터의 메모리 소비가 대규모 데이터셋에서 심각

### 25.2 핵심 기술 및 접근법

#### 인덱싱 기법

논문에서 다루는 주요 인덱싱 방식들:

1. **해시 기반 인덱싱 (Locality Sensitive Hashing, LSH):** 유사한 벡터를 동일 해시 버킷에 배치하는 확률적 기법
2. **트리 기반 인덱싱:** KD-트리, Ball 트리 등 계층적 공간 분할 구조
3. **그래프 기반 인덱싱:** HNSW(Hierarchical Navigable Small World) 같은 근처이웃 그래프 구조
4. **양자화 기법 (Quantization):** 벡터를 저정밀 표현으로 압축하여 메모리와 속도 향상

#### 근사 최근접 이웃 (ANN) 검색

고차원 공간에서 정확한 최근접 이웃 검색의 NP-hard 특성을 완화하기 위한 근사 알고리즘:

- **Hierarchical Clustering:** 데이터를 계층적으로 클러스터링하여 검색 공간 축소
- **Random Projection:** 고차원 데이터를 저차원으로 사영
- **Product Quantization:** 벡터를 부분 벡터로 분해하여 양자화
- **Graph Traversal:** 사전 구성된 그래프를 통한 탐욕적 이웃 탐색

## 25.3 시스템 아키텍처

### 핵심 구성 요소

1. **저장소 계층**: 메인 메모리, 디스크, 분산 스토리지 간 계층적 데이터 관리
2. **인덱싱 계층**: 다양한 인덱스 구조의 생성, 유지, 쿼리 처리
3. **쿼리 처리 엔진**: SQL 파싱, 최적화, 실행 계획 수립
4. **메타데이터 관리**: 벡터 이외의 구조화된 데이터와의 조인 처리

### 성능 최적화

- **인덱스 선택 자동화**: 쿼리 패턴과 데이터 특성에 기반한 최적 인덱스 선택
- **적응형 재인덱싱**: 데이터 변화에 따른 동적 인덱스 구조 조정
- **병렬 처리**: 멀티 코어와 SIMD 명령어 활용
- **캐싱 전략**: 자주 접근되는 벡터와 인덱스 노드의 효율적 캐싱

## 25.4 분산 환경에서의 확장성

벡터 DB가 대규모 데이터셋을 처리하기 위한 분산 처리 기법:

- **샤딩**: 벡터를 여러 노드에 분산 저장
- **레플리카**: 읽기 부하 분산과 가용성 향상
- **분산 인덱싱**: 각 샤드에서 독립적 인덱싱
- **조정 메커니즘**: 정렬 기반 병합, 근사 최상위 k 선택

## 25.5 본 논문과의 관계

Exqutor은 텍스트 쿼리와 벡터 인덱싱을 결합한 하이브리드 검색 시스템이다. 본 튜토리얼 논문은 벡터 데이터베이스 관리의 기초적 기술들을 포괄적으로 설명함으로써, Exqutor가 구현해야 하는 벡터 DB 레이어의 핵심 기법들에 대한 이론적 배경을 제공한다. 특히 인덱싱 전략 선택, ANN 알고리즘 최적화, 메타데이터와 벡터의 결합 관리 등에서 본 튜토리얼의 내용이 직접 적용될 수 있다.

---

### 추가 제기 문제

1. **다양한 인덱싱 기법 간 성능 비교**: 실제 환경에서 LSH, 트리 기반, 그래프 기반 인덱싱의 정확성과 속도 특성을 정량적으로 비교할 수 있는 벤치마크 표준화가 필요한가?
2. **하이브리드 인덱싱**: 단일 인덱싱 기법이 아닌, 데이터 특성과 쿼리 패턴에 따라 동적으로 여러 인덱싱 기법을 조합하는 방식의 실용성은?

3. **메모리-정확도 트레이드오프의 정량화**: 양자화 수준, 샘플 크기 등 파라미터 변화에 따른 정확도 손실을 예측 가능하게 모델링할 수 있는가?
4. **동적 업데이트 최적화**: 높은 빈도의 삽입/삭제 작업 하에서 인덱스 구조의 효율성을 유지하는 방법?

## [26] Pinecone: The vector database to build knowledgeable AI 총정리

조직: Pinecone Systems 형태: 상용 클라우드 기반 벡터 데이터베이스 서비스 웹사이트: <https://www.pinecone.io/> 공개/업데이트: 2024

### 요약

Pinecone은 현대적 AI 애플리케이션을 위해 특화된 완전 관리형 벡터 데이터베이스 클라우드 서비스이다. 기업들이 대규모 벡터 데이터를 쉽게 저장, 인덱싱, 검색할 수 있도록 설계된 프로덕션 수준의 솔루션이다. Pinecone의 핵심 가치는 인프라 관리의 복잡성을 추상화하면서도 기업급 요구사항인 고가용성, 보안, 확장성을 보장한다는 점이다. 사용자는 API 호출을 통해 벡터 삽입, 검색, 업데이트 등의 작업을 수행할 수 있으며, 서비스 제공자가 백엔드의 인덱싱, 복제, 샤딩 등을 관리한다. 다양한 인덱싱 알고리즘, 메타데이터 필터링, 실시간 업데이트 등의 기능을 제공하여 생산 환경에서의 요구사항을 충족한다. LLM 기반 애플리케이션, 의미 검색, 추천 시스템 등 다양한 사용 사례를 지원한다.

### 상세분석

#### 26.1 주요 문제점과 해결 대상

벡터 데이터베이스의 수요 증가에 따른 여러 실무적 문제들:

- **운영 복잡성**: 오픈소스 벡터 DB를 직접 운영할 경우 배포, 확장, 백업, 모니터링의 복잡성
- **확장성 문제**: 소규모에서는 간단하지만, 대규모 벡터 데이터셋에서의 성능 유지의 어려움
- **일관성 보장**: 분산 환경에서 데이터 복제 및 일관성 유지
- **보안 및 접근 제어**: 엔터프라이즈 환경에서의 인증, 권한 관리, 데이터 암호화
- **메타데이터 통합**: 벡터와 함께 저장된 메타데이터의 효율적 필터링 및 검색
- **높은 초기 비용**: 전문 엔지니어링 팀 구성, 하드웨어 투자

#### 26.2 핵심 아키텍처 및 기능

클라우드 기반 관리형 서비스 모델

Pinecone은 다음의 세 가지 배포 옵션을 제공:

1. **서버리스 (Serverless)**: 완전 관리형, 자동 확장, 사용량 기반 요금
2. **Pod 기반**: 용량 선택 가능한 전용 인프라
3. **하이브리드**: 클라우드와 온프레미스의 결합

#### 핵심 기능

**벡터 관리**: - 고차원 벡터(최대 20,000 차원)의 저장 및 관리 - 실시간 업데이트 및 삭제 - 배치 작업 지원

**검색 및 인덱싱**: - 다양한 거리 메트릭 지원 (유클리드, 코사인, 도트곱) - 근사 최근접 이웃 검색 (ANN) - 메타데이터 필터링과 벡터 유사도 검색의 결합

**메타데이터 관리**: - 각 벡터와 함께 JSON 형식의 메타데이터 저장 - 필터 쿼리를 통한 선택적 검색 - 예: "category == 'electronics' AND price < 100"

**인덱싱 전략**: - 프로덕션 인덱스: 캐싱, 복제, 고가용성 - 컬렉션: 프로덕션 인덱스의 스냅샷 (비용 절감) - 자동 인덱싱: 삽입된 벡터에 대한 자동 인덱싱

#### 성능 특성

- **레이턴시**: P95 대기시간 수십ms 단위
- **처리량**: 초당 수천 개의 쿼리 처리 능력
- **정확도**: Recall 지표로 측정되는 근사 검색 정확도 (설정 가능)

## 26.3 시스템 아키텍처

#### 계층적 구조

```
클라이언트 애플리케이션
↓
Pinecone API (REST/gRPC)
↓
Query Router (요청 분배)
↓
인덱싱 엔진 (복수 Pod)
↓
벡터 저장소 + 메타데이터 저장소
↓
영구 스토리지 (클라우드)
```

#### 데이터 복제 및 고가용성

- **자동 복제**: 각 벡터와 인덱스의 자동 복제

- **장애 복구:** Pod 장애 시 자동 페일오버
- **지역 분산:** 다양한 지역에서의 배포로 지연시간 최소화

#### API 설계

일관된 REST/gRPC 인터페이스:

- `upsert(vectors, metadata)`: 벡터 삽입 또는 업데이트
- `query(query_vector, top_k, filter)`: 상위 k개 유사 벡터 검색
- `delete(id, filter)`: 벡터 삭제
- `fetch(ids)`: 특정 벡터 조회

## 26.4 사용 사례 및 적용 분야

**생성형 AI와 LLM:** - RAG (Retrieval-Augmented Generation): LLM에 외부 지식 제공 - 시맨틱 검색: 의미적 유사성 기반 문서 검색 - LLM 메모리: 멀티턴 대화에서의 맥락 유지

**추천 시스템:** - 협업 필터링 대체 - 콘텐츠 기반 추천

**의료 및 과학:** - 분자 구조 유사성 검색 - 의료 이미지 분석 보조

## 26.5 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 검색 시스템이다. Pinecone은 Exqutor의 백엔드 벡터 저장소 및 검색 엔진 역할을 수행할 수 있는 실제 프로덕션 시스템이다. 본 논문(Pinecone 시스템)은 Exqutor이 구현해야 하는 벡터 DB의 실제 기능 요구사항, API 설계, 메타데이터 관리 방식 등을 보여줌으로써, 연구 시스템이 실제 서비스로 전환될 때 필요한 엔지니어링 고려사항을 제시한다.

## 추가 제기 문제

1. **비용-성능 최적화:** 같은 정확도 요구사항을 만족할 때, Pod 기반 배포 vs 서버리스 배포에서 최적의 비용-성능 비율을 선택하는 기준은 무엇인가?
2. **메타데이터 필터링의 확장성:** 복잡한 다중 필터 조건(예: 100개 이상의 OR 조건)에서 메타데이터 필터링의 성능 저하 정도는?
3. **실시간 인덱싱:** 초당 높은 빈도의 삽입 작업(초당 수만 건) 중에 검색 성능이 어느 수준까지 유지되는가?
4. **모델 버전 관리:** 임베딩 모델 변경 시 기존 벡터의 재인덱싱 전략과 그에 따른 다운타임은?
5. **쿼오타 및 한계:** 단일 인덱스가 지원할 수 있는 최대 벡터 개수, 최대 메타데이터 크기 등의 실제 한계와 그에 따른 아키텍처 조정 방안은?



## [27] Manu: A cloud native vector database management system 총정리

**저자:** R. Guo, X. Luan, L. Xiang et al. **학회/출판:** arXiv:2206.13843, 2022 **형태:** 클라우드 네이티브 벡터 데이터베이스 관리 시스템 **관련 시스템:** Milvus 후속 버전/진화

---

### 요약

Manu는 클라우드 네이티브 환경을 위해 특화된 벡터 데이터베이스 관리 시스템이다. 기존 Milvus 시스템의 진화 형태로, 마이크로서비스 아키텍처를 기반으로 하여 쿠버네티스 환경에서의 배포와 확장을 최적화한다. Manu의 핵심 설계 철학은 계산과 저장소의 분리, 상태 비저장 서비스 아키텍처, 그리고 자동 확장성이다. 대규모 벡터 데이터셋에 대한 고성능 검색을 제공하면서도, 클라우드 환경의 변동적 리소스와 네트워크 지연을 효율적으로 처리한다. 분산 환경에서의 일관성 보장, 메타데이터 관리, 그리고 복잡한 쿼리 처리 기능을 통해 프로덕션급 벡터 DB 서비스를 제공한다.

---

### 상세분석

#### 27.1 주요 문제점

클라우드 환경에서의 벡터 데이터베이스 운영이 마주하는 특유한 도전 과제들:

- **리소스 변동성:** 클라우드에서 계산 리소스가 동적으로 증감하는 환경에서 데이터 일관성과 가용성 유지
- **네트워크 지연:** 마이크로서비스 아키텍처에서의 서비스 간 통신 오버헤드
- **확장성 제약:** 전통적 중앙집중식 아키텍처의 성능 병목
- **상태 관리의 복잡성:** 분산된 노드들 간의 벡터 데이터와 인덱스 상태 동기화
- **자동 장애 복구:** 클라우드 환경에서의 예측 불가능한 장애 대응
- **효율적 리소스 활용:** 비용 절감과 성능 사이의 균형

#### 27.2 핵심 아키텍처

마이크로서비스 기반 설계

Manu의 아키텍처는 다음의 독립적 서비스 컴포넌트로 구성:

**접근 계층 (Access Layer):** - 클라이언트 요청 처리 - 로드 밸런싱 - 프로토콜 변환 (gRPC, HTTP)

**조정 계층 (Coordination Layer):** - Etcd를 이용한 분산 조정 - 메타데이터 관리 - 리더십 선출 (Leader Election)

**검색 계층 (Search Layer):** - 상태 비저장 검색 노드 - 인덱스 로딩 및 쿼리 실행 - 캐싱 최적화

**데이터 저장 계층 (Data Storage Layer):** - 벡터 데이터의 영구 저장 - 메타데이터 저장소 - 스냅샷 및 백업

계산과 저장소 분리

계산 (Stateless Search Nodes)  
↓ (캐시, 볼륨 마운트)  
저장소 (Vector Storage, Metadata Store)

- **장점:** 각 계층을 독립적으로 확장 가능
- **동시 요청:** 검색 노드를 자유롭게 추가/제거
- **리소스 효율성:** 저장소 리소스와 계산 리소스의 불균형 해소

## 27.3 주요 기술 요소

메타데이터 관리

- **분산 메타데이터:** Etcd 클러스터를 통한 일관된 메타데이터 관리
- **컬렉션 (Collection):** 벡터 그룹의 논리적 단위
- **파티션 (Partition):** 컬렉션 내의 데이터 분할
- **세그먼트 (Segment):** 물리적 저장 단위

인덱싱 전략

**다양한 인덱싱 알고리즘 지원:** - IVF (Inverted File Index): 클러스터 기반 인덱싱 - HNSW (Hierarchical Navigable Small World): 그래프 기반 - DiskAnn: 디스크 기반 인덱싱 - Flat: 정확한 검색 (작은 데이터셋용)

**적응형 인덱스 선택:** - 데이터 크기와 쿼리 특성에 따른 자동 선택 - 동적 인덱스 변경

실시간 삽입과 검색의 조화

**로그 기반 아키텍처:** - Write-ahead Log (WAL): 데이터 손실 방지 - 메모리 버퍼: 최근 삽입 데이터의 빠른 검색 - 주기적 플러시: 메모리 데이터를 저장소로 이전

**적응형 병합:** - 작은 세그먼트들의 점진적 병합 - 쿼리 성능에 영향 최소화

## 27.4 분산 환경에서의 성능 최적화

캐싱 전략

1. **인덱스 캐싱:** 자주 접근되는 인덱스 노드의 메모리 캐싱
2. **쿼리 결과 캐싱:** 동일 쿼리의 재실행 최소화

### 3. 메타데이터 캐싱: 로컬 메타데이터 사본 유지

#### 네트워크 효율성

- **배치 요청:** 다중 벡터의 한 번에 전송
- **압축:** 네트워크 전송 데이터 압축
- **로컬 처리:** 검색 노드의 로컬 인덱싱으로 중앙 노드 트래픽 감소

#### 복제와 일관성

- **즉시 일관성 (Immediate Consistency):** 강한 일관성 보장
- **멀티 레플리카:** 고가용성
- **자동 페일오버:** 노드 장애 시 자동 복구

## 27.5 클라우드 네이티브 특성

#### 쿠버네티스 통합

- **스테이트풀셋 (StatefulSet):** 저장 노드의 상태 유지
- **디플로이먼트 (Deployment):** 검색 노드의 무상태 확장
- **서비스 디스커버리:** 자동 서비스 등록 및 로드 밸런싱
- **자동 확장:** CPU/메모리 기반 Pod 자동 증감

#### 모니터링 및 관찰성

- **메트릭 수집:** Prometheus 호환 메트릭
- **로깅:** 분산 추적 (Distributed Tracing)
- **헬스체크:** 주기적 상태 확인

## 27.6 본 논문과의 관계

Exqutor은 텍스트 기반 쿼리를 벡터 검색으로 변환하는 하이브리드 시스템이다. Manu는 Exqutor의 벡터 검색 백엔드로 활용될 수 있는 프로덕션급 벡터 DB이다. 특히 대규모 분산 환경에서의 확장성, 메타데이터 관리, 그리고 클라우드 네이티브 배포 특성이 Exqutor의 분산 아키텍처 설계에 직접 영향을 미친다. Manu의 상태 비저장 설계와 마이크로서비스 아키텍처는 Exqutor이 클라우드 환경에서 수평 확장하는 방식의 기초가 될 수 있다.

## 추가 제기 문제

1. **메타데이터 Etdc 병목**: 모든 메타데이터 변경이 Etdc를 거쳐야 할 때, Etdc의 처리 능력 한계는? 분산 환경에서 메타데이터 캐싱의 일관성 보장 수준은?
2. **세그먼트 병합 오버헤드**: 대규모 데이터셋에서 주기적인 세그먼트 병합이 쿼리 성능에 미치는 영향을 측정하고 최소화할 수 있는가?
3. **캐시 무효화 전략**: 분산 검색 노드들 간의 캐시 일관성을 어느 수준까지 자동으로 유지할 수 있는가? 수동 개입이 필요한 경우는?
4. **인덱스 선택 자동화**: 다양한 인덱싱 알고리즘(IVF, HNSW, DiskAnn) 중 최적을 선택하는 기준과 비용-성능 분석은?
5. **장애 복구 시간**: 검색 노드 장애 시 페일오버 시간과, 저장 노드 장애 시 데이터 복구 시간의 SLA는?

## [28] Chroma: The open-source AI application database 총정리

**조직**: Chroma (오픈소스 프로젝트) **형태**: 오픈소스 임베딩/AI 애플리케이션 데이터베이스 **웹사이트**: <https://www.trychroma.com/> **공개/업데이트**: 2024

### 요약

Chroma는 AI 애플리케이션 개발자를 위해 설계된 경량의 오픈소스 벡터 데이터베이스이다. 특히 개발 단계에서 프로토타이핑과 소규모 배포에 최적화되어 있으며, 간단한 Python API를 통해 빠르게 벡터 검색 기능을 통합할 수 있다. Chroma의 강점은 사용의 단순성과 유연성에 있다. 로컬 파일 시스템, 인메모리 저장소, 또는 분산 모드로의 배포가 모두 가능하여 개발부터 프로덕션까지의 여정을 지원한다. LLM 기반 애플리케이션, 특히 RAG 시스템에서 외부 지식 베이스 역할을 담당한다. 메타데이터 필터링, 다양한 거리 메트릭, 그리고 쉬운 확장성을 제공하여 초기 AI 서비스 개발의 진입 장벽을 낮춘다.

### 상세분석

#### 28.1 주요 문제점과 설계 목표

기존 벡터 데이터베이스 솔루션의 한계와 Chroma의 해결 방안:

- **복잡성**: Milvus, Weaviate 등 기존 시스템의 설정과 배포가 복잡
- **Chroma 해결책**: 최소 구성으로 5분 내 시작 가능
- **진입 장벽**: 소규모 팀이나 초기 단계 스타트업의 높은 인프라 비용

- Chroma 해결책: 로컬 개발 모드 완전 무료, 메모리 기반 배포
- **프로토타이핑 속도**: 빠른 개념 검증의 필요성
- Chroma 해결책: 즉시 사용 가능한 Python API
- **확장성 필요**: 개발 중 쉽게 확장할 수 있는 아키텍처
- Chroma 해결책: 로컬 → 분산 모드로의 무경계 전환

## 28.2 핵심 기능 및 아키텍처

### 배포 모드

- 1. 인메모리 모드 (In-Memory)**: - 프로그램 시작 시 데이터 초기화 - 매우 빠른 성능 - 재시작 시 데이터 손실 - 개발 및 테스트에 적합
- 2. 영구 로컬 저장소 모드 (Persistent Local)**: - SQLite + DuckDB로 메타데이터와 벡터 저장 - 로컬 파일 시스템 기반 벡터 저장 - 소규모 애플리케이션(수백만 벡터)에 적합 - 재시작 후 데이터 유지
- 3. 클라이언트-서버 모드 (Client-Server)**: - Chroma 서버를 별도 프로세스로 실행 - REST API를 통한 통신 - 여러 클라이언트의 동시 접근 지원 - 초기 분산 배포 단계
- 4. 분산 모드 (Distributed)**: - 쿠버네티스 기반 배포 - 수평 확장 - 고가용성 구성

### 핵심 API 설계

간결하고 직관적인 Python 인터페이스:

```
컬렉션 생성
collection = client.create_collection(name="documents")

벡터 추가
collection.add(
 ids=["id1", "id2"],
 embeddings=[[1.1, 2.3], [4.5, 6.9]],
 metadatas=[{"source": "doc1"}, {"source": "doc2"}],
 documents=["text1", "text2"]
)

검색
results = collection.query(
 query_embeddings=[[1.1, 2.3]],
 n_results=10,
 where={"source": "doc1"}
)
```

**주요 메서드:** - `create_collection()` : 새 컬렉션 생성 - `add()` : 벡터와 메타데이터 추가 - `delete()` : 벡터 삭제 - `update()` : 벡터 업데이트 - `query()` : 유사 벡터 검색 - `get()` : 특정 벡터 조회

메타데이터 필터링

메타데이터와 벡터 유사도의 결합 검색:

```
results = collection.query(
 query_embeddings=[[1.1, 2.3]],
 n_results=10,
 where={
 "$and": [
 {"category": "technology"},
 {"year": {"$gte": 2020}}
]
 }
)
```

필터 연산자: `$and` , `$or` , `$eq` , `$ne` , `$gt` , `$gte` , `$lt` , `$lte` , `$in` , `$nin`

임베딩 모델 통합

**내장 임베딩:** - Chroma의 기본 임베딩 함수로 자동 처리 - 문서 자체를 저장하고 쿼리로 자동 임베딩

**외부 모델 통합:** - OpenAI, HuggingFace, Ollama 등과의 통합 - 커스텀 임베딩 함수 작성 가능

## 28.3 저장소 구현

메타데이터 저장소

**SQLite 또는 DuckDB:** - 메타데이터 필터링을 위한 SQL 쿼리 지원 - ACID 트랜잭션 보장 - 작은 데이터셋에 최적화

벡터 저장소

**로컬 파일 시스템 (persistent 모드):** - 벡터를 효율적인 바이너리 포맷으로 저장 - NumPy 또는 Parquet 형식 활용

**메모리 (in-memory 모드):** - 빠른 접근

**외부 저장소 (분산 모드):** - 오브젝트 스토리지 (S3, GCS) 연동 가능

## 28.4 검색 성능 최적화

인덱싱 전략

Chroma는 단순성을 위해 제한된 인덱싱 옵션:

- **정확한 검색 (Exact Search):** 모든 벡터와 비교 (작은 컬렉션용)
- **해시 기반 근사 (Locality Sensitive Hashing):** 대규모 벡터 대응

캐싱

- 최근 쿼리 결과 캐싱
- 메타데이터 쿼리 캐싱

## 28.5 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 검색으로 변환하는 하이브리드 시스템이다. Chroma는 Exqutor의 벡터 저장 및 검색 백엔드로서 두 가지 역할을 한다:

1. **개발 및 프로토타이핑:** Exqutor의 초기 구현과 테스트에 이상적인 가벼운 벡터 DB
2. **소규모 배포:** 엔터프라이즈 요구사항이 낮은 환경에서 경량의 벡터 검색 인프라 제공

또한 Chroma의 간결한 API 설계는 Exqutor이 제공해야 할 사용자 인터페이스의 간소화와 직관성에 영감을 줄 수 있다. 특히 메타데이터 필터링과 벡터 검색의 통합 방식은 Exqutor의 하이브리드 검색 전략과 직접 연계된다.

---

## 추가 제기 문제

1. **확장성 한계:** SQLite/DuckDB 기반의 메타데이터 저장소가 어느 정도의 벡터 개수와 쿼리 빈도까지 효율적으로 처리할 수 있는가?
2. **정확한 vs 근사 검색의 자동 선택:** 컬렉션 크기에 따라 자동으로 인덱싱 전략을 전환하는 기준은? 전환 과정의 오버헤드는?
3. **멀티 클라이언트 동시성:** 클라이언트-서버 모드에서 다양한 클라이언트의 동시 삽입/삭제/검색 시 일관성 보장 수준은?
4. **메모리 효율성:** 인메모리 모드에서 대규모 벡터 데이터셋(수억 개)을 저장할 때 메모리 압축 기법은?
5. **분산 모드 성능:** 분산 모드에서 네트워크 지연이 단일 노드 모드 대비 검색 성능에 얼마나 영향을 미치는가?
6. **임베딩 모델 버전 관리:** 임베딩 모델이 변경될 때 기존 벡터의 재생성과 재인덱싱 전략은?

---

## [29] Survey of vector database management systems 총정리

**저자:** J. J. Pan, J. Wang, G. Li **학회/출판:** The VLDB Journal, vol. 33, no. 5, 2024 **페이지:** p. 1591-1615 **발행 월:** September 2024 **자료형:** 종합 서베이 논문

---

## 요약

벡터 데이터베이스 관리 시스템의 포괄적이고 최신의 서베이 논문이다. 이 논문은 벡터 DB의 탄생 배경, 발전 역사, 현재의 다양한 시스템들을 체계적으로 분류하고 분석한다. 벡터 DB의 핵심 기술 영역인 인덱싱, 근사 최근접 이웃 검색, 메타데이터 관리, 분산 처리, 그리고 최적화 기법들을 상세히 다룬다. VLDB 저널의 게재를 통해 데이터베이스 커뮤니티의 공식적 인정을 받은 이 서베이는, 벡터 DB 관련 최신 연구 동향을 정리하고, 해결되지 않은 문제들을 지적하며, 향후 연구 방향을 제시한다. 약 600개 이상의 참고문헌을 포함하여, 벡터 DB 분야의 가장 종합적이고 신뢰할 수 있는 기술 참고자료이다.

## 상세분석

### 29.1 벡터 DB의 역사적 발전

#### 등장 배경

**전통 DB의 한계:** - 관계형 DB: 구조화된 데이터에 최적화, 벡터 데이터 처리 부실 - 문서 DB: 반구조화 데이터에 강하나 의미적 유사도 검색 미지원

**AI 혁명의 영향:** - 딥러닝의 급속한 발전으로 특성 추출(feature extraction)이 표준화 - 대규모 텍스트 임베딩 모델의 등장 (Word2Vec, BERT, GPT 기반 임베딩) - LLM 기반 애플리케이션의 외부 지식 베이스 필요성

**벡터 DB의 필연성:** - 수백만 수억 차원의 벡터 데이터를 효율적으로 관리할 수 있는 전문 시스템의 부재 - 전통 DB에 벡터 지원 추가의 복잡성과 성능 한계

### 29.2 핵심 분류 체계

#### 시스템 분류 관점

**1. 전문 벡터 DB (Specialized Vector Databases):** - Milvus, Weaviate, Vespa, Pinecone 등 - 벡터 검색에 최적화 - 일반 데이터베이스 기능은 제한적

**2. 벡터 DB 확장 (Vector Database Extensions):** - PostgreSQL pgvector: 기존 RDBMS에 벡터 지원 추가 - DuckDB-VSS: 컬럼 기반 DB에 벡터 검색 추가 - Elasticsearch: 검색 엔진에 벡터 검색 추가

**3. 하이브리드 벡터 DB (Hybrid Vector-Relational):** - 벡터와 정형 데이터를 동시에 효율적으로 처리 - 복합 쿼리 지원 (스칼라 필터 + 벡터 검색)

**4. 그래프 벡터 DB (Graph Vector Databases):** - Neo4j: 그래프 관계와 벡터 유사도의 결합 - 복잡한 관계 데이터의 표현과 검색



## 29.3 주요 기술 영역

### 1. 인덱싱 기법

**메모리 기반 인덱싱:** - HNSW (Hierarchical Navigable Small World): 다층 그래프 구조, 빠른 검색 - IVF (Inverted File Index): 클러스터 기반, 효율적 메모리 사용 - LSH (Locality Sensitive Hashing): 확률적 기법, 병렬 처리 용이

**디스크 기반 인덱싱:** - DiskANN: 메모리 기반 그래프와 디스크 저장의 결합 - Product Quantization: 벡터 압축으로 저장소 절감

**하이브리드 인덱싱:** - 메모리와 디스크를 모두 활용하는 적응형 인덱싱

### 2. 근사 최근접 이웃 (ANN) 검색 알고리즘

**정확도-성능 트레이드오프:** - Recall@k: 상위 k개 결과의 정확도 (일반적으로 95% 이상 목표) - Query Latency: 밀리초 단위 응답 시간 - Throughput: 초당 처리 쿼리 수

**주요 메트릭:** -  $\text{Recall} = (\text{실제 최근접 } k\text{개 중 검색된 수}) / k$  - 일반적으로 Recall 95%에서 지연시간 10-100ms 달성

### 3. 메타데이터 관리

**메타데이터의 중요성:** - 벡터 자체는 의미 정보를 담지만, 문서 출처, 카테고리, 타임스탬프 등은 벡터와 분리 필요 - 필터링 조건과 벡터 유사도 검색의 조합이 필수

**저장소 접근법:** - 별도 메타데이터 DB: 빠른 필터링, 추가 인덱싱 - 벡터 저장소 내 메타데이터: 간단하나 필터링 성능 저하

**복합 쿼리 처리:** - WHERE 절과 벡터 유사도 조건의 동시 처리 - 성능 최적화: 필터 우선 vs 벡터 검색 우선 선택

### 4. 분산 벡터 DB

**샤딩 전략:** - 범위 기반 샤딩: 메타데이터 범위로 분할 - 해시 기반 샤딩: 벡터 ID의 해시값으로 분할 - 의미적 샤딩: 유사 벡터를 같은 샤드에 배치 (성능 향상)

**일관성 보장:** - 강한 일관성: 모든 복제본이 동일 (성능 감소) - 최종 일관성: 모든 복제본이 결국 동일 (성능 향상)

**분산 쿼리 처리:** - 브로드캐스트: 모든 샤드에 쿼리 전송 후 결과 병합 (정확하나 느림) - 샤드 선택: 관련 샤드만 쿼리 (빠르나 복잡)

### 5. 동적 업데이트

**삽입/삭제의 도전:** - 기존 인덱스 구조 변경 최소화 - 실시간 쓰기 성능 보장

**로그 기반 아키텍처:** - Write-ahead Log (WAL): 데이터 손실 방지 - 메모리 버퍼: 최근 수정 사항 빠른 접근 - 주기적 플러시: 저장소로의 데이터 이전

## 29.4 주요 벡터 DB 시스템 비교

개방형 vs 상용형

**개방형 (Open-source):** - Milvus: 클라우드 네이티브, 높은 확장성 - Weaviate: GraphQL API, 스키마 유연성 - Chroma: 경량, 개발 친화적

**상용형 (Commercial):** - Pinecone: 완전 관리형, 높은 가용성 - Vespa: Yahoo의 고성능 시스템

배포 모델

**자관리형 (Self-managed):** - 높은 커스터마이징 가능 - 운영 복잡성 증가

**클라우드 관리형 (Cloud-managed):** - 운영 부담 감소 - 비용 증가

## 29.5 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 검색으로 변환하는 하이브리드 검색 시스템이다. 이 종합 서베이는 Exqutor이 의존하는 벡터 DB 기술의 모든 측면을 다룬다:

1. **인덱싱 선택:** Exqutor이 사용할 인덱싱 기법의 설계 원칙 제공
2. **메타데이터 필터링:** 텍스트 쿼리와 메타데이터 필터의 조합 처리 방식
3. **분산 아키텍처:** Exqutor의 대규모 환경 확장 전략
4. **최적화 기법:** 성능과 정확도 간 트레이드오프 관리

또한 이 서베이는 Exqutor의 위치를 벡터 DB 생태계 내에서 명확히 하는 데 도움이 된다 - Exqutor은 단순 벡터 DB가 아닌 텍스트-벡터 변환과 검색을 통합한 더 높은 수준의 시스템이다.

---

## 추가 제기 문제

1. **벡터 DB의 표준화:** 다양한 벡터 DB 시스템 간에 호환성 있는 API나 데이터 포맷 표준화는 가능한가?
2. **인덱싱 기법의 선택 자동화:** 주어진 데이터 특성과 워크로드에 대해 최적의 인덱싱 기법을 자동으로 선택하는 인텔리전트 시스템은?
3. **메타데이터-벡터 조인의 최적화:** 복잡한 메타데이터 필터 조건과 벡터 유사도를 결합할 때, 옵티마이저의 최적 실행 계획 수립은?
4. **벡터 임베딩의 버전 관리:** 새로운 임베딩 모델이 등장했을 때 기존 벡터 데이터의 점진적 마이그레이션 전략은?

5. **실시간 성능 특성 변화**: 시간이 흐르면서 데이터 특성이 변할 때, 인덱스와 파라미터를 자동으로 적응시키는 기능은?
6. **벡터 DB 선택 프레임워크**: 기업이 자신의 요구사항에 맞는 벡터 DB를 선택할 수 있는 의사결정 프레임워크 수립?

## (H) ANN 인덱스 기법 — 최근접 이웃 검색 알고리즘

### [30] Pase: PostgreSQL ultra-high-dimensional approximate nearest neighbor search extension 총정리

**저자**: W. Yang, T. Li, G. Fang, H. Wei **학회/출판**: SIGMOD 2020 **페이지**: p. 2241-2253 **자료형**: 데이터베이스 확장 논문

#### 요약

Pase (PostgreSQL Approximate Similarity sEarch)는 기존 관계형 데이터베이스인 PostgreSQL에 초고차원 벡터에 대한 근사 최근접 이웃 검색 기능을 추가하는 확장이다. SIGMOD 2020에 발표된 이 논문은 전통적 RDBMS 환경에서 벡터 검색을 효율적으로 구현하는 방법을 제시한다. Pase는 PostgreSQL의 인덱싱 인프라를 활용하면서도, 초고차원 벡터의 특수성을 고려한 새로운 인덱싱 기법을 제안한다. 기존 정형 데이터와 벡터 데이터를 동시에 관리하고 쿼리하는 환경에서, 관계형 데이터베이스를 하나의 통일된 시스템으로 유지하면서 벡터 검색 성능을 확보하는 실용적 해결책을 제공한다.

#### 상세분석

##### 30.1 주요 문제점

PostgreSQL과 같은 기존 RDBMS에 벡터 검색을 추가할 때의 도전 과제:

- **고차원 데이터의 성능 병목**: 수천 차원의 벡터에 대해 정확한 최근접 이웃 검색은 전체 테이블 스캔을 필요로 하며, 초고차원에서는 "차원의 저주" 현상으로 근사도 어려움
- **인덱싱 구조 부재**: B-tree, GiST 등 기존 인덱싱 기법은 고차원 공간에 부적합
- **저장소 확장**: 기존 RDBMS의 행(row) 기반 저장소에 대규모 벡터를 효율적으로 저장
- **쿼리 최적화**: 벡터 조건과 스칼라 조건을 포함한 복합 쿼리의 최적화
- **정확도-성능 트레이드오프**: 근사 알고리즘 사용 시 발생하는 정확도 손실 관리

## 30.2 핵심 기술: Pase의 인덱싱 기법

### 개요

Pase는 두 단계 인덱싱 전략을 채택:

**1단계: 거칠 필터링 (Coarse Filtering)** - 벡터 공간을 여러 영역(partition)으로 분할 - 쿼리 벡터와 가까운 영역만 선택

**2단계: 세밀한 재검색 (Fine Reranking)** - 선택된 영역 내에서 정확한 거리 계산 - 최상위 k개 결과 선택

### 분할 기법 (Partition Strategy)

#### 균형잡힌 K-평면 분할 (Balanced K-Plane Partitioning):

```
n차원 벡터 공간
↓
초평면(hyperplane)으로 순차 분할
↓
각 영역의 크기가 대략 균등
↓
이진 트리 구조로 조직화
```

특징: - 트리 깊이  $O(\log n)$ : 로그 시간에 쿼리 벡터가 속한 영역 결정 - 메모리 효율적: 각 노드는 분할 평면만 저장 - 적응형 분할: 데이터 분포에 따른 자동 조정

### 세밀한 재검색 최적화

#### 후보 확대 (Expansion Strategy):

```
쿼리 벡터의 영역부터 시작
↓
거리가 가까운 인접 영역으로 확대
↓
충분한 후보 수 모이면 중단
```

이를 통해: - 처음부터 모든 영역을 확인하지 않음 - 쿼리 벡터 근처의 데이터에 집중 - 평균적으로 방문 영역 감소

## 30.3 PostgreSQL 통합 설계

### 확장 구조

PostgreSQL 핵심

↓

커스텀 데이터 타입: vector(float8[], int)

↓

인덱싱 메서드: PASE Index

↓

SQL 연산자: <-> (거리), <#> (내적)

저장소 계층

**벡터 데이터 저장:** - 고정 길이 벡터: PostgreSQL VARLENA 형식 (가변 길이)으로 저장 - 양자화: 부동소수점을 압축된 형태로 저장 가능 - 저장 효율성: 일반 float 배열 대비 메모리 절감

**메타데이터와의 통합:** - 동일 테이블에 벡터와 정형 데이터 공존 - 자연스러운 JOIN 지원

예시:

```
CREATE TABLE documents (
 id SERIAL PRIMARY KEY,
 content TEXT,
 embedding VECTOR(1536),
 category VARCHAR(50),
 created_at TIMESTAMP
);

CREATE INDEX ON documents USING pasc (embedding);

SELECT id, content,
 embedding <-> ARRAY[...] AS distance
FROM documents
WHERE category = 'news'
ORDER BY embedding <-> ARRAY[...]
LIMIT 10;
```

## 30.4 성능 최적화

쿼리 계획 생성 (Query Planning)

**메타데이터 통계:** - 데이터 분포 통계 수집 - 선택도 추정: 벡터 유사도 조건의 선택도 예측

**조인 순서 최적화:** - 스칼라 필터 vs 벡터 필터의 순서 결정 - 선택도 낮은 조건 우선 실행

병렬 처리

**벡터 검색의 병렬화:** - 인덱스 트리의 여러 서브트리를 병렬로 탐색 - 리스트 병합 알고리즘으로 결과 조합

메모리 관리

**버퍼 풀 활용:** - 자주 접근하는 인덱스 노드의 캐싱 - 메모리 부족 시 LRU 제거 정책

### 30.5 정확도 보장

Recall 지표

근사 인덱싱의 정확도:

$$\text{Recall@k} = (\text{결과에서 실제 k-NN 중 포함된 수}) / k$$

Pase의 목표: Recall@10 = 90-95%

**조절 가능한 파라미터:** - 분할 트리 깊이: 깊을수록 정확하나 느림 - 재검색 후보 확대 크기: 클수록 정확하나 느림

### 30.6 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 시스템이다. Pase는 다음 두 가지 측면에서 Exqutor과 관련:

1. **기존 RDBMS 통합:** Exqutor이 PostgreSQL 같은 기존 데이터베이스 위에 구축된다면, Pase 같은 벡터 확장 이 백엔드 기술로 활용될 수 있다.
2. **하이브리드 검색:** Exqutor의 텍스트 필터와 벡터 유사도 검색의 조합은 Pase에서 보여주는 스칼라 조건과 벡터 조건의 결합 처리와 동일한 문제 영역이다. Pase의 쿼리 최적화 기법이 직접 적용될 수 있다.
3. **기존 시스템 활용:** Pase처럼 기존 광범위하게 사용되는 데이터베이스에 벡터 기능을 추가하는 방식은, Exqutor의 실제 배포 모델을 단순화할 수 있다.

### 추가 제기 문제

1. **초고차원에서의 성능:** 10,000 차원 이상의 초고차원 벡터에서 Pase의 성능이 어떻게 변하는가? 차원의 저주의 영향 정도는?
2. **K-평면 분할의 최적성:** K-평면으로 균등 분할하는 것이 모든 데이터 분포에 최적인가? 데이터 분포에 따른 적응형 분할이 성능을 향상시킬 수 있는가?
3. **인덱스 유지보수 비용:** 높은 빈도의 삽입/삭제 작업 시 인덱스 재구성의 오버헤드는? 점진적 인덱스 업데이트가 가능한가?
4. **메모리-디스크 계층:** 메모리에 모든 인덱스를 캐시할 수 없을 때, 디스크 기반 인덱싱과의 조합 전략은?
5. **정확도-성능 트레이드오프의 자동화:** 쿼리별로 요구되는 정확도 수준이 다를 때, 파라미터를 동적으로 조정하는 메커니즘은?

6. PostgreSQL과의 호환성: 새 버전의 PostgreSQL과의 호환성 유지 비용은? 표준 PostgreSQL의 백엔드 변경이 Parse에 미치는 영향은?

## [31] DuckDB-VSS: Vector similarity search extension for DuckDB 총정리

저자: DuckDB Team 형태: 오픈소스 벡터 검색 확장 프로젝트 저장소: GitHub 공개/업데이트: 2024

### 요약

DuckDB-VSS는 DuckDB 데이터베이스에 벡터 유사도 검색(Vector Similarity Search) 기능을 추가하는 오픈소스 확장이다. DuckDB는 인메모리 컬럼 기반 OLAP 데이터베이스로 최근 빠르게 주목받는 시스템인데, DuckDB-VSS는 이 강력한 분석 엔진에 벡터 검색 기능을 통합한다. 이 확장은 정형 데이터와 벡터 데이터를 동시에 처리해야 하는 분석 워크로드를 지원한다. DuckDB의 컬럼 저장소 특성과 벡터 데이터의 조화로운 통합을 통해, 뛰어난 성능과 사용의 단순성을 모두 달성한다. 특히 프로토타이핑, 데이터 분석, 소규모에서 중규모의 벡터 검색이 필요한 시나리오에 적합하다.

### 상세분석

#### 31.1 주요 문제점과 설계 동기

기존 벡터 검색 솔루션의 한계:

- **분석과 벡터 검색의 분리:** 기존 벡터 DB는 순수 벡터 검색에만 최적화, 정형 데이터와의 복합 분석이 어려움
- **OLAP 환경의 필요성:** 분석가가 벡터와 정형 데이터를 함께 분석해야 하는 경우 증가
- **진입 장벽:** 별도의 벡터 DB 시스템 관리의 복잡성
- **비용:** 중규모 조직이 벡터 DB를 위해 추가 인프라 투자의 부담
- **컬럼 저장소의 미활용:** DuckDB의 효율적 컬럼 저장소를 벡터 데이터에도 활용하지 못함

#### 31.2 DuckDB의 기초

특징

**컬럼 기반 저장:** - 행(row) 기반이 아닌 열(column) 기반 저장 - OLAP 분석 쿼리에 최적화 - 압축을 통한 저장소 절감 - 캐시 효율성 증대

**인메모리 OLAP:** - SQL 쿼리 엔진 내장 - 복잡한 분석 쿼리 지원 - 매우 빠른 처리 속도

**확장성:** - 모듈식 아키텍처 - 커스텀 함수, 타입, 연산자 추가 용이

### 31.3 DuckDB-VSS의 핵심 기능

#### 벡터 데이터 타입

```
-- 벡터 컬럼 정의
CREATE TABLE embeddings (
 id INTEGER PRIMARY KEY,
 text VARCHAR,
 vector DOUBLE[1536] -- 1536차원 벡터
);
```

**지원 형식:** - DOUBLE[]: 부동소수점 벡터 - FLOAT[]: 단정밀도 벡터 - 고정 길이 배열로 정의

#### 벡터 거리 연산자

#### SQL 함수로 제공:

```
-- L2 거리 (유클리드 거리)
SELECT id, text,
 array_distance(vector, [1.2, 3.4, ...]) AS distance
FROM embeddings
ORDER BY distance
LIMIT 10;

-- 코사인 유사도
SELECT id, text,
 array_cosine_similarity(vector, [1.2, 3.4, ...]) AS similarity
FROM embeddings
ORDER BY similarity DESC
LIMIT 10;

-- 내적
SELECT id, text,
 array_dot_product(vector, [1.2, 3.4, ...]) AS dot
FROM embeddings
ORDER BY dot DESC
LIMIT 10;
```

**지원하는 거리 메트릭:** - L2 (Euclidean) - 코사인 (Cosine) - 내적 (Dot Product) - 맨해튼 (Manhattan)

#### 벡터 인덱싱

#### HNSW (Hierarchical Navigable Small World) 인덱스:



```
-- 벡터 인덱스 생성
CREATE INDEX vec_idx ON embeddings
USING hnswnn (vector)
WITH (distance = 'l2', m = 16, ef = 128);
```

파라미터: - **distance**: 사용할 거리 메트릭 - **m**: HNSW 그래프의 최대 연결 수 - **ef**: 검색 시 탐색 효율성 파라미터 (클수록 정확하나 느림)

### 31.4 DuckDB-VSS의 아키텍처

저장소 계층

```
SQL 쿼리
 ↓
벡터 연산 플래너
 ↓
HNSW 인덱스 (선택적)
 ↓
컬럼 저장소 (벡터 데이터)
 ↓
메모리/디스크
```

**컬럼 저장소의 장점:** - 벡터 차원 단위의 저장으로 캐시 효율성 - 배치 처리에 최적화 - SIMD 명령어 활용 용이

쿼리 처리

**선택형 HNSW 가속:** - 인덱스가 있으면 HNSW로 후보 선택 - 인덱스가 없으면 전체 스캔 - 쿼리 플래너가 인덱스 사용 여부 결정

**벡터와 스칼라 조건의 결합:**

```
SELECT id, text, category,
 array_distance(vector, query_vec) AS dist
FROM embeddings
WHERE category = 'news' -- 스칼라 필터
 AND date >= '2024-01-01' -- 추가 필터
ORDER BY dist
LIMIT 10;
```

처리 최적화: 1. 스칼라 필터 먼저 처리 (선택도 낮춤) 2. 결과에 대해 벡터 거리 계산 3. 거리 기반 정렬 및 제한

### 31.5 사용 사례

데이터 분석 (OLAP)

```

-- 임베딩된 뉴스 기사에서 특정 주제와 유사하면서
-- 특정 기간의 기사 분석
SELECT category, COUNT(*) as count,
 AVG(array_distance(embedding, topic_vec)) as avg_similarity
FROM articles
WHERE published_date >= '2024-01-01'
GROUP BY category
HAVING AVG(array_distance(embedding, topic_vec)) < 0.5
ORDER BY avg_similarity;

```

## AI 애플리케이션 통합

```

import duckdb

conn = duckdb.connect()

벡터 데이터 로드
conn.execute("""
 CREATE TABLE documents AS
 SELECT doc_id, content, embedding
 FROM read_csv('docs.csv')
""")

검색 쿼리
query_vec = [0.1, 0.2, 0.3, ...]
results = conn.execute(f"""
 SELECT doc_id, content,
 array_distance(embedding, {query_vec}) as distance
 FROM documents
 ORDER BY distance
 LIMIT 10
""").fetchall()

```

## ETL 파이프라인

```

-- 벡터 생성 및 저장
CREATE TABLE embeddings AS
SELECT id,
 text,
 ai_embed(text) as vector -- 커스텀 임베딩 함수
FROM raw_documents
WHERE valid = true;

-- 벡터 인덱싱
CREATE INDEX ON embeddings USING hnsw (vector);

-- 배치 검색
SELECT DISTINCT doc_a, doc_b
FROM (
 SELECT a.id as doc_a, b.id as doc_b,
 array_distance(a.vector, b.vector) as sim
 FROM embeddings a
 JOIN embeddings b ON a.category = b.category
) t
WHERE sim < 0.1
LIMIT 1000;

```

## 31.6 성능 특성

### 장점

**빠른 벡터 검색:** - HNSW 인덱스로 대규모 데이터셋에서도 밀리초 단위 검색 - 컬럼 저장소로 배치 처리 최적화

**메모리 효율성:** - 컬럼 압축으로 저장소 절감 - 불필요한 컬럼 제외로 메모리 사용 최소화

**분석 능력:** - SQL의 모든 분석 기능 활용 가능 - 벡터 검색 결과의 추가 집계/분석 용이

**개발 편의성:** - DuckDB 설치 후 확장만 추가 - SQL 기반 인터페이스로 학습곡선 낮음

### 제약

**확장성 한계:** - 인메모리 시스템이므로 메모리 크기 제한 - 매우 대규모 벡터 데이터셋에는 부적합

**분산 처리 미지원:** - 단일 노드 시스템 - 다중 노드 클러스터링 불가

## 31.7 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 시스템이다. DuckDB-VSS는 다음의 측면에서 Exqutor과 관련:

1. **분석 기반 검색:** Exqutor이 벡터 검색을 OLAP 환경에서 수행해야 한다면, DuckDB-VSS의 컬럼 저장소와 분석 기능이 적합

2. **스칼라-벡터 결합**: Exqutor의 텍스트 필터와 벡터 검색의 조합은 DuckDB-VSS의 복합 쿼리 처리와 유사
3. **빠른 프로토타이핑**: Exqutor의 초기 구현과 성능 평가에 DuckDB-VSS 사용 가능

## 추가 제기 문제

1. **메모리 제약 극복**: 인메모리 시스템의 근본적 한계를 어떻게 극복할 것인가? 디스크 기반 벡터 저장에 성능에 미치는 영향은?
2. **HNSW 파라미터 자동화**: `m` 과 `ef` 같은 파라미터를 사용자가 수동으로 튜닝해야 하는데, 자동 최적화 방법은?
3. **분산 처리**: DuckDB의 장점을 유지하면서 분산 벡터 검색을 지원할 수 있는가? 단일 노드 제약을 어떻게 해결할 것인가?
4. **인덱스 유지보수**: 높은 빈도의 삽입 작업 중 HNSW 인덱스의 지속적 갱신이 성능에 미치는 영향은?
5. **다양한 벡터 차원 지원**: 모든 차원의 벡터에 대해 일정한 성능을 유지할 수 있는가? 극단적으로 높은 차원 (100,000+)의 벡터는 지원 가능한가?
6. **비교 분석**: DuckDB-VSS와 Pinecone, Milvus 같은 전문 벡터 DB의 성능-비용 트레이드오프는?

## (C) 경쟁/비교 시스템 — 관련 접근법

### [32] Vespa: AI + data, online at any scale 총정리

개발사: Yahoo 형태: 대규모 AI + 데이터 서빙 플랫폼 웹사이트: <https://vespa.ai/> 공개/업데이트: 2024

### 요약

Vespa는 Yahoo에서 개발한 오픈소스 검색 및 AI 데이터 서빙 플랫폼이다. 단순한 벡터 DB를 넘어, 구조화된 데이터, 벡터, 그리고 복잡한 추론 로직을 모두 포괄하는 포괄적 시스템이다. Vespa의 핵심 특징은 실시간 데이터 업데이트와 높은 처리량의 개인화된 검색/추천을 동시에 제공한다는 점이다. 대규모 인터넷 서비스에서 사용되었던 검색 및 추천 엔진의 경험을 바탕으로, 프로덕션급 신뢰성과 확장성을 갖추고 있다. 텍스트 검색, 벡터 검색, 정형 데이터 필터링, 그리고 머신러닝 모델의 추론을 하나의 통일된 플랫폼에서 처리하므로, 현대적 AI 애플리케이션에 매우 적합하다.

## 상세분석

### 32.1 주요 문제점과 Vespa의 해결책

기존 검색/추천 시스템의 한계:

- **오프라인-온라인 갭**: 오프라인에서 학습한 모델을 온라인 시스템에 배포할 때의 복잡성과 지연
- **실시간 개인화의 어려움**: 사용자의 최근 행동을 반영한 즉시 개인화 추천의 기술적 복잡성
- **다양한 데이터 타입 통합**: 정형 데이터, 텍스트, 벡터, 그래프 등을 통일된 방식으로 처리
- **확장성과 지연시간**: 초당 수백만 요청을 수백 밀리초 이내에 처리
- **AI 모델 배포**: 새로운 ML 모델의 빠른 반영과 버전 관리

**Vespa의 해결책**: - 통일된 플랫폼으로 모든 데이터와 로직을 처리 - 실시간 데이터 피드 처리 - 쿼리 시 동적 추론 실행 - 자동 확장과 고가용성

### 32.2 핵심 아키텍처

계층적 구조

```
클라이언트 애플리케이션
↓
Query API (REST, gRPC)
↓
Dispatch 계층 (로드 밸런싱)
↓
Search 노드 (인덱싱, 검색)
↓
저장소 (MVCC 기반 인덱스)
↓
Content 저장소 (Data 노드)
```

주요 구성 요소

1. **Content Layer**: - 실제 문서 데이터 저장 - 분산 파티셔닝 - 자동 복제
2. **Search Layer**: - 역인덱스 (Inverted Index) 기반 검색 - 벡터 유사도 검색 - 랭킹 및 재정렬
3. **Query Language (YQL)**: - 강력한 쿼리 언어 - 필터, 정렬, 그룹화, 함수 지원

### 32.3 주요 기능

1. 다양한 검색 기능

**텍스트 검색:** - 전문 검색 (Full-text Search) - 구문 검색 (Phrase Search) - 부분 일치 (Partial Matching) - 언어 별 형태소 분석

```
select * from documents
where default contains "AI" and title contains "machine learning"
```

**벡터 검색:** - 근사 최근접 이웃 (ANN) - 하이브리드 텍스트-벡터 검색 - 다양한 거리 메트릭

```
select * from documents
where (all(field:embedding, f(x)(x-q)))
order by closeness(field, embedding)
```

**메타데이터 필터링:** - 범위 필터 - 카테고리 필터 - 복합 불린 조건

```
select * from documents
where category = "news" and publication_date > 2024-01-01
```

## 2. 랭킹 및 개인화

**다단계 랭킹 (Multi-stage Ranking):**

```
1단계: 매칭 (Matching) - 조건을 만족하는 문서 선별
↓
2단계: 1차 랭킹 (First-phase Ranking) - 빠른 점수 계산
↓
3단계: 2차 랭킹 (Second-phase Ranking) - 복잡한 점수 계산
↓
4단계: 재정렬 (Reranking) - 최종 순서 결정
```

**개인화 점수:** - 사용자 특성 기반 가중치 조정 - 실시간 피드백 반영

$$\text{점수}(\text{doc}, \text{user}) = \text{기본\_점수}(\text{doc}) \times \text{개인화\_승수}(\text{user}, \text{doc})$$

## 3. 머신러닝 모델 배포

**온라인 추론:** - 쿼리 시점에 ML 모델 실행 - ONNX, TensorFlow 모델 지원

```
select * from documents
where (all(field:embedding, f(x)(x-q)))
order by ml_score(ml_model_v2, field:features)
```

**모델 버전 관리:** - A/B 테스트 지원 - 점진적 롤아웃 - 빠른 롤백

#### 4. 실시간 데이터 업데이트

**피드 처리 (Feed Processing):** - 클라이언트로부터 문서 스트림 수신 - 파싱, 변환, 검증 - 분산 저장소에 기록 - 인덱스 자동 갱신

```
Document doc = new Document("id:namespace:doctype::1",
 new StringFieldValue("content"));
doc.setFieldValue("embedding", vectorValue);
feedClient.put(doc);
```

**일관성 보장:** - 쓰기-읽기 일관성 (Read-after-write consistency) - 분산 노드 간 동기화

#### 5. 복합 쿼리 처리

**YQL (Vespa Query Language):**

```
select * from documents
where (all(field:embedding, f(x)(x-q))
 or title contains @text)
and category in ("news", "blog")
and publication_date > now() - 30 days
order by nativeRank, closeness(field, embedding)
limit 10
offset 0
```

### 32.4 성능 특성

**확장성**

**수평 확장:** - 문서 수에 따른 자동 샤딩 - 노드 추가로 선형 성능 향상 - 지연시간 유지

**처리량:** - 초당 수백만 쿼리 - 초당 수십만 데이터 업데이트

**지연시간**

**P99 지연시간:** 일반적으로 10-50ms - 텍스트 검색: 10-20ms - 벡터 검색: 20-50ms - 복합 쿼리: 50-100ms

**저장소 효율성**

**압축 기법:** - 비트맵 압축 - 휴프만 인코딩 - 양자화 (벡터용)

**메모리 사용:** - 인덱스: 원본 데이터의 10-30% - 벡터: 압축으로 4-10배 감소 가능

## 32.5 배포 모델

클라우드 관리형

**Vespa Cloud:** - 완전 관리형 호스팅 - 자동 확장, 고가용성 - 빠른 배포

자관리형

**On-Premise 또는 자체 클라우드:** - 완전 제어 - 커스터마이징 자유도 - 운영 책임

## 32.6 사용 사례

**온라인 추천 시스템:** - 사용자 행동 실시간 반영 - 개인화된 상품/콘텐츠 추천 - 대규모 동시 사용자 처리

**검색 엔진:** - 웹 검색 - 전자상거래 검색 - 엔터프라이즈 검색

**광고 타겟팅:** - 실시간 입찰 (RTB) - 사용자-광고 매칭

## 32.7 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 검색 시스템이다. Vespa는 다음의 측면에서 Exqutor과 관련:

1. **포괄적 플랫폼:** Exqutor이 구현해야 하는 텍스트-벡터 하이브리드 검색의 완성된 프로덕션 예시를 제공
2. **다단계 랭킹:** Exqutor의 검색 결과 순위 지정 전략에 영감 제공
3. **실시간 업데이트:** Exqutor이 동적 데이터셋에서 운영되는 경우의 아키텍처 참고
4. **확장성 경험:** 대규모 분산 환경에서의 벡터 검색 최적화 전략
5. **AI 모델 통합:** Exqutor과 외부 임베딩 모델 또는 재순위 모델의 통합 방식

---

## 추가 제기 문제

1. **다단계 랭킹의 비용-정확도 분석:** 더 복잡한 2차 랭킹은 더 정확하나 느리다. 최적의 단계 수와 복잡도는?
  2. **벡터 검색의 정확도 보장:** 대규모 벡터 인덱스에서 Recall이 어느 수준 이상 유지되는가? 인덱스 크기에 따른 저하 정도는?
  3. **실시간 모델 배포:** 새 ML 모델을 배포할 때 기존 사용자 요청에 미치는 영향 최소화 방안은?
  4. **메모리 vs 정확도:** 벡터 압축으로 메모리를 절감할 때, 검색 정확도 손실은 얼마나 되는가?
  5. **피드 레이턴시:** 데이터 업데이트 후 검색 결과에 반영되는 시간은? 일관성 vs 성능 트레이드오프는?
  6. **모듈식 확장성:** 사용자 정의 검색 로직이나 재순위 함수를 추가할 때의 복잡도는?
-



## [33] Neo4j: Vector index and search 총정리

개발사: Neo4j 형태: 그래프 데이터베이스의 벡터 검색 통합 기능 웹사이트: <https://neo4j.com/> 공개/업데이트: 2024

### 요약

Neo4j는 그래프 데이터베이스로 잘 알려져 있으며, 최근 벡터 인덱싱과 검색 기능을 핵심 플랫폼에 통합했다. Neo4j의 벡터 검색은 단순한 임베딩 저장이 아닌, 그래프의 노드와 관계와 함께 벡터 데이터를 저장하고 검색하는 통합된 접근이다. 이를 통해 의미적 유사도(벡터 기반)와 구조적 유사도(그래프 관계 기반)를 함께 고려하는 강력한 검색이 가능해진다. 지식 그래프, 추천 시스템, 의미 검색 등 다양한 응용에서 그래프 구조와 의미적 정보를 동시에 활용할 수 있다.

### 상세분석

#### 33.1 주요 문제점과 설계 목표

그래프 데이터베이스 환경에서 벡터 검색의 필요성:

- **의미적 검색의 부재:** 기존 그래프 DB는 명시적 관계와 속성 기반 검색만 지원, 의미적 유사도 검색 불가
- **구조와 의미의 분리:** 관계 정보와 벡터 정보를 별도 시스템에서 관리해야 했던 문제
- **지식 그래프의 완성도:** 엔티티 간 벡터 기반 유사도를 고려한 검색 필요
- **추천 시스템의 통합:** 사용자-아이템 관계와 벡터 유사도를 함께 고려하는 하이브리드 추천
- **오버헤드:** 그래프와 벡터 DB를 동시에 운영하는 복잡성과 데이터 동기화

**Neo4j의 해결책:** - 벡터를 노드의 속성으로 저장 - HNSW 기반 벡터 인덱싱 - 그래프 탐색과 벡터 검색의 통합 쿼리 - 관계 기반 검색에 벡터 필터 추가 가능

#### 33.2 핵심 아키텍처

데이터 모델

그래프 + 벡터 통합:

#### 노드(Node)

- ID: 고유 식별자
- 레이블(Label): 노드 타입
- 속성(Properties):
  - 일반 속성 (text, number, date 등)
  - 벡터 속성 (embedding)

#### 관계(Relationship)

- 타입(Type): 관계 종류
- 속성(Properties): 관계의 메타데이터

#### 예시 스키마:

```
// 문서 노드에 벡터 저장
CREATE (doc:Document {
 id: 'doc123',
 title: 'AI in Healthcare',
 content: '...',
 embedding: [0.1, 0.2, ..., 0.n] // 1536차원 벡터
})

// 사용자 노드와의 관계
CREATE (user:User {name: 'Alice'})
CREATE (user)-[:INTERESTED_IN]->(doc)
```

#### 벡터 인덱싱

#### HNSW 벡터 인덱스:

```
// 벡터 인덱스 생성
CREATE VECTOR INDEX doc_embedding_index
IF NOT EXISTS
FOR (n:Document)
ON n.embedding
OPTIONS {
 indexConfig: {
 `vector.dimensions`: 1536,
 `vector.similarity_function`: 'cosine',
 `vector.m`: 16,
 `vector.efConstruction`: 128,
 `vector.ef`: 40
 }
}
```

파라미터: - `dimensions`: 벡터 차원 수 - `similarity_function`: 거리 메트릭 (cosine, euclidean, dot\_product) - `m`: HNSW 그래프 구조의 파라미터 (연결 수) - `efConstruction`: 인덱스 생성 시 탐색 범위 - `ef`: 쿼리 시 탐색 범위

### 33.3 주요 기능

#### 1. 순수 벡터 검색

유사 벡터 찾기:

```
CALL db.index.vector.queryNodes('doc_embedding_index', 10, [0.1, 0.2, ..., 0.n])
YIELD node, score
RETURN node.title, score
ORDER BY score DESC
```

결과: - `node`: 매칭된 노드 - `score`: 유사도 점수 (거리 메트릭에 따라 정의)

#### 2. 필터링을 포함한 벡터 검색

벡터 검색 + 노드 속성 필터:

```
CALL db.index.vector.queryNodes('doc_embedding_index', 100, query_vector)
YIELD node, score
WHERE node.category = 'healthcare'
 AND node.published_date > datetime('2024-01-01')
RETURN node.title, score
ORDER BY score DESC
LIMIT 10
```

#### 3. 그래프 탐색과 벡터 검색의 결합

하이브리드 검색:

```

// 벡터 검색으로 초기 후보 찾기
CALL db.index.vector.queryNodes('doc_embedding_index', 50, query_vector)
YIELD node as similarDoc, score

// 그래프 탐색: 유사 문서와 관련된 다른 문서 찾기
MATCH (user:User {id: 'user123'})-[:VIEWED]->(doc:Document)
WHERE doc IN similarDoc
MATCH (doc)-[:RELATED_TO]->(relatedDoc:Document)

// 최종 점수: 벡터 유사도 + 그래프 관계 가중치
WITH relatedDoc, score,
 (score + 0.5) as hybridScore // 관계에 보너스 점수
ORDER BY hybridScore DESC
RETURN relatedDoc.title, hybridScore

```

#### 4. 추천 시스템

##### 협업 필터링 + 벡터 기반:

```

// 사용자와 유사한 사용자 찾기 (임베딩 기반)
CALL db.index.vector.queryNodes('user_embedding_index', 10, user_vector)
YIELD node as similarUser

// 유사 사용자가 좋아한 아이템 찾기 (그래프 관계 기반)
MATCH (similarUser)-[:RATED {score: >=4}]->(item:Item)
WHERE NOT (originalUser)-[:RATED]->(item)

// 아이템과 유사한 다른 아이템 찾기 (벡터 기반)
WITH item
CALL db.index.vector.queryNodes('item_embedding_index', 5, item.embedding)
YIELD node as similarItem, score

// 최종 추천 점수 계산
RETURN similarItem,
 (1.0 + score) * (COALESCE(item.popularity, 0.5)) as recommendation_score
ORDER BY recommendation_score DESC

```

#### 5. 의미 검색 및 지식 그래프

##### 자연어 쿼리 기반 검색:

```
// 질문을 벡터로 변환
WITH vectorize('What are the latest treatments for cancer?') as question_vector

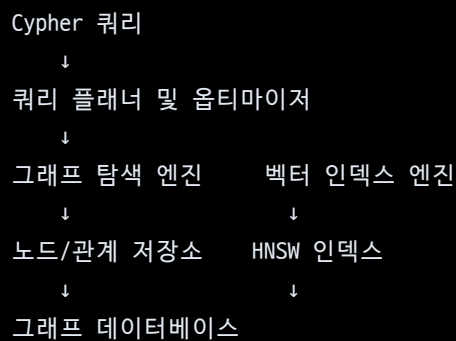
// 관련 문서 찾기
CALL db.index.vector.queryNodes('medical_doc_embedding_index', 20, question_vector)
YIELD node as document, score

// 관련 개념(노드) 탐색
MATCH (document)-[:MENTIONS]->(concept:Concept)
MATCH (concept)-[r:RELATED_TO]->(relatedConcept:Concept)

RETURN document.title, concept.name, relatedConcept.name, r.relationship_type
ORDER BY score DESC
LIMIT 10
```

### 33.4 아키텍처 및 성능

#### 저장소 통합



#### 성능 특성

**검색 성능:** - 벡터 검색만:  $O(\log n)$  HNSW 탐색 - 그래프 탐색만: 관계 수에 따라 선형 - 결합: 벡터 검색 후 그래프 필터링으로 최적화 가능

**메모리 사용:** - 그래프 노드/관계: 구조에 따라 가변 - 벡터 인덱스:  $O(nd)$  ( $n$ =벡터 수,  $d$ =차원) - HNSW 메타데이터:  $O(nm)$  ( $m$ =평균 연결 수)

**확장성:** - 그래프 크기: 수십억 노드 가능 (엔터프라이즈 라이선스) - 벡터 인덱스: 수억 벡터 가능 - 쿼리 지연시간: 수십ms (적절한 인덱싱)

### 33.5 사용 사례

**지식 그래프:** - 엔티티 간 의미적 연결 발견 - 오류 정정 및 중복 제거 - 질의응답 시스템

**추천 시스템:** - 사용자 선호도와 아이템 유사도 결합 - 콘텐츠 및 협업 필터링 통합 - 동적 추천 재계산

**검색 엔진:** - 의미적 검색 - 페이지 랭크와 벡터 유사도 결합 - 관련도 높은 페이지 발견

**사기 탐지:** - 거래 패턴의 벡터 표현 - 의심 거래 네트워크 탐지 - 관계 기반 위험도 판단

### 33.6 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 검색 시스템이다. Neo4j의 벡터 검색은 다음의 측면에서 Exqutor과 관련:

1. **하이브리드 검색:** Exqutor의 텍스트 필터와 벡터 검색의 결합은 Neo4j의 속성 필터와 벡터 검색 결합과 유사
2. **관계 기반 순위 지정:** Neo4j의 그래프 관계를 활용한 순위 지정은 Exqutor이 검색 결과를 순위 지정할 때 고려할 수 있는 추가 신호
3. **통합 플랫폼:** Neo4j처럼 관계형 정보와 벡터 정보를 하나의 시스템에서 통합 관리하는 아이디어
4. **의미 검색:** Exqutor과 마찬가지로 자연어 쿼리의 의미를 벡터 검색으로 해석

### 추가 제기 문제

1. **그래프 + 벡터 인덱싱의 오버헤드:** 그래프 관계와 HNSW 벡터 인덱스를 동시에 유지할 때 메모리와 업데이트 오버헤드는?
2. **복합 쿼리의 최적화:** 벡터 검색 먼저 vs 그래프 필터링 먼저의 선택 기준은? 동적 최적화 가능한가?
3. **관계 기반 순위의 정확도:** 그래프 구조 정보를 벡터 점수와 어떻게 결합해야 최적의 결과를 얻을 수 있는가?
4. **대규모 그래프에서의 확장성:** 수십억 노드의 그래프에서 벡터 인덱싱과 쿼리 성능은 어떻게 변하는가?
5. **동적 업데이트:** 그래프 관계와 벡터가 함께 변할 때 인덱스의 일관성과 성능을 어떻게 유지할 것인가?
6. **임베딩 모델 변경:** 새 임베딩 모델로 모든 벡터를 재생성할 때의 전략은?

## [34] Redis: for vector database 총정리

**개발사:** Redis Labs (현 Redis) **형태:** 인메모리 데이터 저장소의 벡터 검색 모듈 **웹사이트:** <https://redis.io/> **공개/업데이트:** 2024

### 요약

Redis는 초고속 인메모리 데이터 구조 저장소로서, RedisVectorDB 모듈과 RediSearch의 벡터 기능을 통해 벡터 검색 능력을 갖추고 있다. Redis의 핵심 강점은 극도로 빠른 응답 시간(서브 밀리초)과 높은 처리량(초당 수백만 작업)이다. 캐싱, 세션 저장소로 널리 사용되는 Redis의 신뢰성과 생태계를 활용하면서, 벡터 검색이 필요한 실시간

애플리케이션에 완벽하게 맞다. Redis의 벡터 검색 기능은 Pinecone이나 Milvus 같은 전문 벡터 DB에 비해 배포가 간단하고, 기존 Redis 사용자들에게는 추가 인프라 없이 벡터 검색을 추가할 수 있다.

## 상세분석

### 34.1 주요 문제점과 Redis의 위치

기존 벡터 검색 솔루션의 한계:

- **초저지연 요구:** 실시간 추천, 자동완성 등에서 밀리초 단위 응답 필요, 기존 벡터 DB는 충분하지 않을 수 있음
- **인프라 복잡성:** 추가 벡터 DB 시스템 운영의 부담
- **기존 Redis 활용 미흡:** 많은 기업이 이미 Redis를 사용하고 있으나, 벡터 지원이 제한적
- **데이터 동기화:** 캐시와 벡터 DB를 별도로 운영할 때의 동기화 복잡성
- **높은 초기 비용:** 새로운 전문 시스템 도입의 학습곡선과 비용

**Redis의 해결책:** - 기존 Redis 인프라 위에서 벡터 검색 - 극도로 빠른 응답 - 간단한 배포와 운영 - 캐싱과 벡터 검색의 통합

### 34.2 핵심 벡터 검색 기능

#### 1. RedisSearch 모듈과 벡터 지원

RedisSearch는 Redis의 검색 엔진 모듈로, 최근 버전에서 벡터 유사도 검색을 통합했다.

기본 사용법:

```
Redis 모듈 로드
MODULE LOAD /path/to/redisearch.so

벡터 인덱스 생성
FT.CREATE my-index
 SCHEMA
 title TEXT
 content TEXT
 embedding VECTOR HNSW 6 TYPE FLOAT32 DIM 1536 DISTANCE_METRIC COSINE
```

#### 2. 벡터 저장 및 관리

벡터 데이터 삽입:

```
벡터와 메타데이터 함께 저장
HSET doc:1 title "Article 1" content "..." embedding "binary_vector_data"

배치 삽입
HSET doc:2 title "Article 2" content "..." embedding "binary_vector_data"
HSET doc:3 title "Article 3" content "..." embedding "binary_vector_data"
```

Python 클라이언트 예시:

```
import redis
import numpy as np

r = redis.Redis(host='localhost', port=6379)

벡터 저장
vector = np.random.randn(1536).astype(np.float32).tobytes()
r.hset('doc:1', mapping={
 'title': 'Article 1',
 'content': 'Content...',
 'embedding': vector
})

벡터 업데이트
r.hset('doc:1', 'embedding', new_vector)

벡터 삭제
r.hdel('doc:1', 'embedding')
```

### 3. 벡터 검색

유사 벡터 검색:

```
KNN 검색 (상위 10개 유사 벡터)
FT.SEARCH my-index "*=>[KNN 10 @embedding $query_vector]"
PARAMS 2 query_vector "binary_query_vector"
```

필터링을 포함한 검색:

```
특정 조건의 벡터 검색
FT.SEARCH my-index "@title:(AI OR machine-learning) =>[KNN 10 @embedding $vec]"
PARAMS 2 vec "binary_query_vector"
```

### 4. 거리 메트릭 지원



Redisearch 벡터 인덱싱에서 지원하는 거리 메트릭:

- **COSINE**: 코사인 거리 (정규화된 벡터에 최적)
- **L2**: 유클리드 거리 (절대 거리)
- **IP**: 내적 (dot product, 음수 허용)

## 5. 인덱싱 알고리즘

**HNSW (Hierarchical Navigable Small World):**

```
FT.CREATE index
SCHEMA
 embedding VECTOR HNSW 6
 TYPE FLOAT32
 DIM 1536
 DISTANCE_METRIC COSINE
 M 16 # 각 노드의 최대 연결 수
 EF_CONSTRUCTION 200 # 인덱싱 시 탐색 범위
 EF_RUNTIME 10 # 쿼리 시 탐색 범위
```

파라미터의 의미: - **M**: HNSW 그래프의 구조 파라미터 (클수록 정확하나 메모리 사용 증가) - **EF\_CONSTRUCTION**: 인덱싱 시간의 정확도 (클수록 정확하나 느림) - **EF\_RUNTIME**: 쿼리 지연시간 vs 정확도 (클수록 정확하나 느림)

## 34.3 Redis 아키텍처와 벡터 통합

메모리 기반 저장

```
Redis 인메모리 저장소
↓
Redisearch 모듈
↓
HNSW 벡터 인덱스 + 텍스트 인덱스
↓
메모리 상주 데이터
```

**특징:** - 모든 데이터가 메모리에 상주 - 극도로 빠른 접근 (나노초 단위) - 영구 저장은 RDB/AOF로 선택적 처리

성능 최적화

**SIMD 최적화:** - 벡터 거리 계산에 CPU SIMD 명령어 활용 - 배치 처리로 캐시 효율성 향상

**메모리 압축:** - FLOAT32 (4바이트/차원) 지원으로 메모리 절감 - 예: 1536차원 벡터 = 6KB

**병렬 처리:** - 여러 검색 스레드 지원 - Redis 6.0 이후 멀티 스레드 I/O

## 34.4 배포 모델

### 1. 단일 노드 Redis

```
클라이언트
 ↓
Redis 서버 (벡터 인덱스 + 데이터)
 ↓
메모리/디스크 (RDB/AOF)
```

**사용:** 개발, 소규모 프로덕션

### 2. Redis Cluster

```
클라이언트 (클러스터 클라이언트)
 ↓
Redis 마스터 노드들 (샤딩)
 ↓
각 노드: 벡터 인덱스의 부분집합
```

**사용:** 대규모 프로덕션, 높은 가용성

**샤딩 전략:** - 문서 ID 기반 해시 샤딩 - 각 샤드는 독립적인 벡터 인덱스 유지 - 검색 시 모든 샤드에 병렬 쿼리

### 3. Redis Stack

Redis Labs의 통합 솔루션으로, RediSearch, RedisJSON, RedisTimeSeries 등을 포함.

## 34.5 실제 사용 사례

### 1. 실시간 추천 시스템:

```

import redis
from redis.commands.search.query import Query

r = redis.Redis(host='localhost', port=6379, decode_responses=True)

사용자 선호도 벡터
user_vector = get_user_embedding(user_id)

상품 추천 (상위 5개)
result = r.ft('product-index').search(
 Query(f"*=>[KNN 5 @embedding $vec]")
 .params(vec=user_vector)
)

recommendations = [doc['product_id'] for doc in result.docs]

```

## 2. 자동완성 + 의미 검색:

```

검색어 임베딩
query_text = "machine learning"
query_vector = embed_text(query_text)

텍스트 매칭 + 벡터 유사도
results = r.ft('article-index').search(
 Query("@title:machine* =>[KNN 10 @embedding $vec]")
 .params(vec=query_vector)
)

```

## 3. 실시간 모니터링/이상 탐지:

```

새로운 이벤트 벡터와 유사한 과거 이벤트 검색 (이상 패턴)
new_event_vector = extract_features(event)

similar_events = r.ft('event-index').search(
 Query(f"@severity:high =>[KNN 20 @embedding $vec]")
 .params(vec=new_event_vector)
)

유사한 이상 사건이 많으면 알림
if len(similar_events.docs) > 10:
 send_alert()

```

## 34.6 성능 특성

검색 성능

**지연시간:** - 단일 KNN 쿼리: 0.1-1ms (수백만 벡터) - 필터링 포함: 1-10ms (필터 선택도에 따라)

**처리량:** - 초당 수백만 벡터 검색 (클라이언트 연결 수에 따라)

**메모리 사용**

**벡터 저장:** - FLOAT32: 차원 × 4바이트 - 1536차원: 6KB/벡터

**인덱싱 오버헤드:** - HNSW 메타데이터: 원본의 약 50-100% - 1백만 벡터: 약 1-2GB 추가 메모리

**확장성**

**단일 노드 제약:** - 메모리 크기가 최대치 (일반적으로 256GB) - 약 4천만-4억 벡터 저장 가능 (메모리에 따라)

**클러스터 확장:** - 16개 샤드로 확장하면 선형 확장 - 초당 수십억 쿼리 처리 가능

### 34.7 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 검색 시스템이다. Redis는 다음의 측면에서 Exqutor과 관련:

1. **초저지연 요구:** Exqutor이 실시간 응답을 요구하는 환경(예: 검색 자동완성)에서는 Redis의 극도로 빠른 성능이 이상적
2. **기존 인프라 활용:** 많은 애플리케이션이 이미 Redis를 사용 중이므로, Exqutor을 Redis 위에 구축하면 배포 단순화
3. **캐싱 통합:** Exqutor의 검색 결과 캐싱과 벡터 인덱싱을 하나의 Redis에서 관리
4. **클러스터 확장:** Exqutor이 대규모로 배포될 때 Redis Cluster를 통한 수평 확장 가능
5. **간단한 운영:** Redis의 광범위한 도구와 모니터링 생태계 활용

---

### 추가 제기 문제

1. **메모리 제약:** 인메모리 저장소의 근본적 한계를 어떻게 극복할 것인가? 디스크 기반 벡터 저장이 성능에 미치는 영향은?
2. **정확도 vs 성능:** HNSW의 `EF_RUNTIME` 파라미터를 어떻게 동적으로 조정할 것인가? 쿼리별 요구 정확도에 따른 자동 조정?
3. **클러스터에서의 정확도:** 벡터 인덱스를 여러 샤드에 분산할 때, 전역 KNN 결과의 정확도 보장은?
4. **메타데이터 필터링의 성능:** 복잡한 필터 조건과 벡터 검색의 결합 시 성능 저하 정도는?
5. **동적 업데이트:** 높은 빈도의 벡터 추가/삭제 중에도 검색 성능을 유지할 수 있는가?
6. **다중 벡터 필드:** 문서당 여러 벡터 필드(제목, 내용, 카테고리별 임베딩)를 지원할 때의 인덱싱 전략은?

7. **Redis vs 전문 벡터 DB**: Redis가 어떤 상황에서 Pinecone이나 Milvus를 대체할 수 있는가? 트레이드오프는?

## (D) 이론적 기반 — VAQ 최적화의 수학적·알고리즘적 토대

### [35] K-nearest neighbor

**저자**: L. E. Peterson **학술지**: Scholarpedia, vol. 4, no. 2, p. 1883, 2009 **주제**: KNN 알고리즘의 기본 개념과 이론

### 요약

본 논문은 Scholarpedia에 발표된 K-nearest neighbor(KNN) 알고리즘에 대한 종합 개요 논문이다. KNN은 기계 학습에서 가장 단순하면서도 강력한 비모수 분류 및 회귀 알고리즘 중 하나로, 주어진 쿼리 포인트로부터 가장 가까운 K개의 학습 샘플을 기반으로 예측을 수행한다.

논문에서는 KNN의 기본 원리, 거리 측정 방식, 최적 K값 선택 방법, 그리고 계산 효율성 개선 방안을 다룬다. 특히 유클리드 거리, 맨해튼 거리 등 다양한 거리 메트릭과 그들의 특성을 설명한다. 또한 KNN의 장점으로 구현의 단순성, 비모수 성질, 그리고 다양한 문제에 대한 적응성을 강조한다.

본 논문과의 관계: Exqutor 시스템은 벡터 기반의 유사도 검색을 위해 KNN 개념을 확장한다. 본 논문은 KNN의 기본적인 이론적 배경을 제공하며, Exqutor가 구현하는 필터링 기반 근접 이웃 검색의 근본이 되는 개념이다.

### 상세분석

#### 35.1 KNN 알고리즘의 기본 원리

K-nearest neighbor는 지도학습의 기본적인 인스턴스 기반 학습(instance-based learning) 방법이다. 알고리즘의 핵심은 주어진 쿼리 포인트  $q$ 에 대해 학습 데이터셋에서 거리 기준으로 가장 유사한 K개의 샘플을 찾아 이들의 레이블을 기반으로  $q$ 의 클래스를 결정하는 것이다.

- **분류(Classification)**: 다수의 클래스 중 가장 빈번하게 나타나는 클래스로 예측
- **회귀(Regression)**: K개 이웃의 값의 평균 또는 가중 평균으로 예측
- **거리 기반 의존성**: 알고리즘의 정확성은 거리 메트릭 선택에 크게 좌우됨

## 35.2 거리 메트릭과 특성

논문에서 다루는 주요 거리 측정 방식들:

**유클리드 거리(Euclidean Distance)** - 가장 널리 사용되는 거리 메트릭 -  $d(x, y) = \sqrt{(\sum (x_i - y_i)^2)}$  - 저차원 데이터에서 좋은 성능

**맨해튼 거리(Manhattan Distance)** -  $d(x, y) = \sum |x_i - y_i|$  - 고차원 데이터에서 유클리드보다 성능이 좋을 수 있음

**민코프스키 거리(Minkowski Distance)** - 일반화된 거리 메트릭으로 p값에 따라 다양한 거리 정의 가능 - p=1일 때 맨해튼, p=2일 때 유클리드

## 35.3 최적 K값 선택

K값은 모델의 성능을 크게 영향을 미치는 중요한 하이퍼파라미터다:

- **작은 K:** 낮은 편향이지만 높은 분산. 노이즈에 민감
- **큰 K:** 높은 편향이지만 낮은 분산. 클래스 불균형 문제
- **최적화 방법:** 교차 검증(cross-validation)을 통한 K 선택

## 35.4 계산 효율성과 확장성

원본 KNN은 모든 학습 데이터와의 거리를 계산해야 하므로  $O(nd)$  시간 복잡도를 가진다. 고차원 또는 대규모 데이터에서는 계산 비용이 매우 크다.

- **공간 분할 방법:** KD-트리, R-트리 등의 인덱스 구조 사용
- **근사 기법:** 정확한 K-NN 대신 근사 근접 이웃 검색(ANN) 활용
- **차원 축소:** PCA 등으로 데이터 차원 감소

## 35.5 KNN의 장단점

**장점:** - 구현이 매우 단순명확 - 새로운 데이터에 빠르게 적응 가능 - 다양한 거리 메트릭으로 유연성 제공 - 비모수 방식으로 데이터 분포 가정 불필요

**단점:** - 높은 계산 복잡도 (특히 예측 시점) - 모든 학습 데이터를 메모리에 유지 필요 - 고차원 데이터에서 "차원의 저주" 문제 - K값 선택의 민감성

## 35.6 본 논문과의 관계

Exqutor 시스템에서 구현하는 필터링 기반 벡터 검색은 KNN 개념의 확장이다:

- **기본 개념:** 벡터 임베딩 공간에서 K개의 근접 이웃을 검색
- **거리 메트릭:** 코사인 유사도, 유클리드 거리 등을 활용
- **효율성 개선:** HNSW와 같은 근사 기법으로  $O(\log n)$  복잡도로 감소
- **필터링 추가:** 속성 기반 필터링을 통해 조건부 KNN 검색 수행

---

## 추가 제기 문제

1. **고차원 벡터 공간에서의 거리 메트릭 선택**: 현대의 임베딩 모델(예: BERT, GPT)에서 생성되는 고차원 벡터들에 대해 어떤 거리 메트릭이 가장 적절한가?
  2. **필터 조건과 K-NN의 상호작용**: 속성 기반 필터를 적용할 때 정확한 K개의 결과를 보장하기 위한 최적 전략은 무엇인가?
  3. **동적 K값 조정**: Exqutor가 동적으로 K값을 조정하여 필터링된 결과의 질을 개선할 수 있을까?
  4. **거리 계산의 정규화**: 벡터 정규화가 필터링 성능에 미치는 영향에 대한 실증적 분석이 필요하다.
- 

## [36] A review of various k-nearest neighbor query processing techniques

**저자**: S. Dhanabai and S. Chandramathi **학술지**: IJCA, vol. 31, no. 7, pp. 14-22, 2011 **주제**: KNN 쿼리 처리 기법의 종합 리뷰

---

## 요약

본 논문은 K-nearest neighbor 쿼리 처리의 다양한 기법들을 종합적으로 검토하는 리뷰 논문이다. 특히 대규모 데이터셋에서 KNN 쿼리를 효율적으로 처리하기 위한 인덱싱 및 최적화 기법들을 분류하고 비교 분석한다.

논문은 순차 탐색(sequential scan), KD-트리, R-트리, Quadtree 등의 공간 분할 기법과 하이브리드 방식을 다룬다. 각 기법의 시간 복잡도, 공간 복잡도, 그리고 실제 성능 특성을 비교한다.

본 논문과의 관계: Exqutor 시스템이 구현하는 근사 근접 이웃 검색은 본 논문에서 다루는 기본 KNN 쿼리 처리 기법들의 발전 형태다. 특히 필터링이 추가된 조건부 KNN 처리는 기존 기법들의 확장이다.

---

## 상세분석

### 36.1 KNN 쿼리 처리의 기본 문제

KNN 쿼리는 데이터베이스에서 가장 흔하면서도 계산 비용이 높은 작업 중 하나다:

**문제 정의**: - 주어진 쿼리 포인트  $q$ 와 정수  $K$ 에 대해 - 거리  $d(q, p)$  기준으로 가장 가까운  $K$ 개의 포인트  $p$ 를 찾기  
- 범위 쿼리보다 복잡: 사전에 결과 집합의 크기를 알 수 없음

**도전 과제**: - 전체 데이터와 거리 계산 필요 (순차 탐색:  $O(n)$ ) - 고차원에서 인덱스 구조의 효율성 저하 - 원본 데이터 공간에서의 복잡한 거리 계산

## 36.2 공간 분할 기반 기법들

**KD-트리(KD-Tree)** - k차원 공간을 재귀적으로 분할하는 이진 탐색 트리 - 각 레벨에서 다른 차원을 기준으로 분할 - 시간 복잡도: 평균  $O(\log n)$ , 최악  $O(n)$  - 고차원에서 성능 저하 문제 ("차원의 저주")

**R-트리(R-Tree)** - 다차원 공간의 위치 정보를 인덱싱하는 B-트리 기반 구조 - 축 정렬 최소 경계 사각형(MBR) 사용 - 동적 삽입/삭제 지원 - 공간 쿼리 처리에 효율적

**R\*-트리 및 변형** - R-트리의 개선 버전으로 더 나은 분할 휴리스틱 사용 - 중첩 최소화로 검색 성능 향상 - 실제 데이터베이스 시스템에서 광범위하게 사용

**Quadtree** - 2D 공간을 4개 사분면으로 재귀적 분할 - 이미지 처리 및 지리정보 시스템(GIS)에서 활용 - 균형 잡힌 구조로 예측 가능한 성능

## 36.3 최적화 기법과 전략

**인덱스 기반 최적화:** - 다중 인덱스: 여러 차원별 인덱스 유지 - 압축: 인덱스 구조 최적화로 메모리 효율성 증가 - 프리페칭: 예상되는 접근 패턴 사전 로드

**휴리스틱 최적화:** - 조기 종료(early termination): 충분한 결과를 찾으면 탐색 중단 - 범위 축소(range reduction): 동적으로 탐색 범위 조정 - 평면 가지치기(plane pruning): 불필요한 부분공간 제외

**병렬 처리:** - 대규모 데이터에 대한 분산 KNN 처리 - 다중 프로세서 활용으로 계산 시간 단축 - 로드 밸런싱의 중요성

## 36.4 기법별 성능 비교

기법	공간복잡도	시간복잡도	고차원성능	동적성
순차탐색	$O(n)$	$O(nd)$	우수	우수
KD-트리	$O(n)$	$O(\log n)$	나쁨	보통
R-트리	$O(n)$	$O(\log n)$	보통	우수
Quadtree	$O(n)$	$O(\log n)$	나쁨	보통

## 36.5 고차원 데이터의 문제점

고차원 공간에서 트리 기반 인덱스의 성능이 저하되는 이유:

**차원의 저주:** - 분할 공간의 대부분이 비어있음 - 근접 포인트들이 멀리 떨어져 있음 (거리 측정 의미 약화) - 트리 기반 가지치기 효율성 감소

**근사적 해결책:** - 차원 축소 (PCA, random projection) - 지역성 민감 해싱(LSH) - 근사 근접 이웃 검색(ANN)



## 36.6 필터링을 고려한 확장

본 논문의 기본 KNN 처리 기법을 속성 필터링과 함께 사용할 때의 고려사항:

**필터링 전략:** 1. **필터-우선(Filter-First):** 먼저 필터 조건을 만족하는 데이터만 선택 후 KNN 수행 2. **KNN-우선(KNN-First):** 먼저 KNN을 수행한 후 필터링 조건 적용 3. **하이브리드:** 필터와 거리 기준을 함께 고려하여 처리

**성능 영향:** - 필터링 선택도(selectivity)가 높으면 필터-우선 방식 유리 - 결과 개수가 적으면 인덱스 효율성 감소  
- 필터 인덱스와 거리 인덱스의 통합 활용 필요

## 추가 제기 문제

1. **벡터 임베딩에 대한 KNN 기법 적용:** 현대의 고차원 벡터 임베딩(1000+ 차원)에 대해 R-트리 같은 전통적 기법이 얼마나 효과적일까?
2. **필터와 거리의 최적 결합:** 속성 기반 필터링과 벡터 거리 기반 검색을 최적으로 결합하기 위한 비용 모델은 무엇인가?
3. **동적 데이터에 대한 인덱스 유지:** 지속적으로 변하는 벡터 데이터에 대해 인덱스 구조를 효율적으로 유지하는 방법은?
4. **메모리 제약 환경:** 메모리가 제한된 환경에서 필터링된 KNN을 효율적으로 수행할 수 있는 인덱스 구조는?
5. **정확도-속도 트레이드오프:** Exquator에서 근사 기법을 사용할 때 정확도 손실과 속도 향상 사이의 최적 균형을 찾는 방법은?

## [37] Generalized k-nearest neighbor rules

**저자:** J. C. Bezdek, S. K. Chuah, and D. Leep **학술지:** Fuzzy Sets and Systems, vol. 18, no. 3, pp. 237-256, 1986 **주제:** 퍼지 집합을 이용한 KNN의 일반화

## 요약

본 논문은 KNN 알고리즘을 퍼지 논리(fuzzy logic)를 이용하여 일반화하는 연구다. 기존의 경직된 KNN 규칙(hard KNN rules)을 확장하여 퍼지 멤버십 함수(fuzzy membership functions)를 도입한 유연한 분류 방법을 제시한다.

논문에서는 퍼지 KNN이 경계 근처의 샘플들을 더 잘 처리할 수 있음을 보여준다. K개 이웃으로부터의 투표 결과를 확률적으로 또는 유사도 가중치로 계산하는 대신, 퍼지 멤버십 함수를 사용하여 각 클래스에 대한 부분적 소속도(partial membership)를 계산한다.

본 논문과의 관계: Exqutor의 필터링 기반 벡터 검색에서 필터 조건의 만족도를 연속적으로 표현할 수 있는 퍼지 논리 개념이 있으며, 본 논문의 일반화된 KNN 개념은 정확성과 유연성 사이의 균형을 맞추는 데 영감을 줄 수 있다.

## 상세분석

### 37.1 기존 KNN과의 비교

**경직된 KNN (Hard KNN):** - 이웃의 클래스 레이블을 그대로 사용 - 투표(voting): 가장 많은 표를 받은 클래스로 분류 - 이진 결정: 한 클래스 또는 다른 클래스 - 경계 근처의 샘플에 대해 불안정할 수 있음

**가중 KNN (Weighted KNN):** - 거리에 반비례하는 가중치 적용 - 더 가까운 이웃에 더 큰 영향력 부여 - 부드러운 성능 개선 하지만 여전히 이진 결정

**퍼지 KNN (Fuzzy KNN):** - 각 샘플의 클래스에 대한 멤버십 정도를 퍼지 집합으로 표현 - 0과 1 사이의 연속적 값으로 부분 소속도 표현 - 분류 결과의 신뢰도(confidence) 제공 - 경계 영역의 샘플들을 더 자연스럽게 처리

### 37.2 퍼지 멤버십 함수

**기본 개념:** 퍼지 집합 A에 대한 멤버십 함수  $\mu_A(x)$ 는 x가 A에 속할 정도를  $0 \leq \mu_A(x) \leq 1$ 로 나타낸다.

**삼각형 멤버십 함수:** - 가장 널리 사용되는 형태 - 선형 증가/감소 구간 포함 - 직관적이고 계산 효율적

**종 모양 멤버십 함수:** - 가우시안(Gaussian) 또는 시그모이드 형태 - 중심에서의 소속도가 최대 - 부드러운 전환

**사다리꼴 멤버십 함수:** - 플래토 영역을 가진 형태 - 특정 범위에서 완전한 소속도 유지

### 37.3 퍼지 KNN의 작동 원리

단계별 알고리즘:

1. K개 이웃 선택
2. 쿼리 포인트 q로부터 거리 기준으로 K개의 가장 가까운 샘플 선택
3. 이 단계에서는 기존 KNN과 동일
4. 멤버십 계산
5. 각 이웃이 각 클래스에 속할 정도를 멤버십 함수로 계산
6. 거리 기반 또는 특성 기반 멤버십 함수 사용 가능
7. 퍼지 집계
8. K개 이웃의 멤버십을 집계 (보통 합산 또는 평균)
9. 각 클래스에 대한 종합 멤버십 점수 계산

## 10. 의사결정

11. 최대 멤버십을 가진 클래스 선택
12. 또는 멤버십 값에 따른 신뢰도 평가

수식 예시:

$$\mu_C(q) = (1/K) * \sum_{i=1 \text{ to } K} \mu_C(x_i)$$

여기서  $\mu_C(x_i)$ 는 이웃  $x_i$ 가 클래스 C에 속할 정도

### 37.4 거리에 기반한 퍼지 멤버십

거리-기반 멤버십 함수: - 멤버십이 거리에 따라 결정됨 - 더 가까운 이웃이 더 높은 멤버십 값 - 일반적인 형태:

$$\mu(x_i) = 1 / (1 + (d_i / \sigma)^2)$$
 여기서  $d_i$ 는 쿼리까지의 거리,  $\sigma$ 는 스케일 파라미터

매개변수 선택의 중요성: -  $\sigma$  값에 따라 감소 곡률 결정 - 작은  $\sigma$ : 가까운 이웃만 중요 - 큰  $\sigma$ : 먼 이웃도 고려

### 37.5 퍼지 KNN의 장점

분류 안정성: - 경계 영역의 샘플들을 더 부드럽게 처리 - 노이즈에 대한 강건성 향상 - 클래스 불균형 문제 완화

신뢰도 정보: - 분류 결과의 신뢰도를 정량화 - 낮은 신뢰도의 분류 결과 식별 가능 - 거부 옵션(rejection option) 제공 가능

유연한 해석: - 분류 결과를 이진 결정 대신 연속적 확률로 표현 - 어플리케이션의 요구에 따른 임계값 조정 가능

### 37.6 퍼지 KNN의 단점 및 한계

계산 복잡도: - 각 클래스별 멤버십 계산으로 인한 추가 비용 - 멤버십 함수 평가의 복잡성

파라미터 설정: - 멤버십 함수 형태 선택 어려움 - 스케일 파라미터( $\sigma$ ) 설정의 민감성 - 교차 검증을 통한 최적화 필요

이론적 정당성: - 퍼지 멤버십이 실제 확률을 나타내는가? - 멤버십 함수와 데이터의 불확실성 관계 정의 애매

### 37.7 본 논문과의 관계

Exquitor 시스템에서 필터링을 고려한 KNN 검색을 수행할 때 퍼지 논리 개념 적용 가능성:

속성 필터링의 퍼지화: - 필터 조건을 이진(만족/불만족) 대신 연속적 만족도로 표현 - 예: 가격이 "100달러 근처"인 상품 검색 - 퍼지 멤버십 함수로 필터 조건의 유연한 표현

가중 검색: - 거리와 필터 만족도를 결합한 가중 점수 계산 - 사용자의 선호도에 따른 가중치 조정 - 정확한 일치와 근사적 일치 모두 고려

신뢰도 기반 순위 지정: - 결과의 신뢰도를 사용자에게 전달 - 임계값 이하의 신뢰도 결과 필터링 가능

## 추가 제기 문제

1. **벡터 공간에서의 퍼지 KNN**: 고차원 벡터 임베딩 공간에서 퍼지 멤버십 함수를 어떻게 정의할 것인가?
2. **필터와 거리의 퍼지 결합**: 속성 필터 조건과 벡터 거리를 퍼지 논리로 결합했을 때의 성능 이득은?
3. **멤버십 함수 학습**: 데이터로부터 최적의 멤버십 함수를 자동으로 학습할 수 있을까?
4. **다중 필터의 퍼지 논리 결합**: 여러 속성 필터의 AND/OR 조합을 퍼지 집합으로 어떻게 표현할까?
5. **해석 가능성**: 퍼지 KNN의 결과를 사용자에게 어떻게 해석 가능한 형태로 제시할 것인가?
6. **성능과 불확실성**: 퍼지 멤버십 함수가 실제 불확실성을 올바르게 반영하는가?

## [38] Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs

**저자**: Y. A. Malkov and D. A. Yashunin **학술지**: IEEE TPAMI, vol. 42, no. 4, pp. 824-836, 2018 **주제**: HNSW(Hierarchical Navigable Small World) 알고리즘 - 근사 근접 이웃 검색

## 요약

본 논문은 현대 벡터 검색 시스템의 핵심 알고리즘 중 하나인 HNSW(Hierarchical Navigable Small World, 계층적 네비게이팅 작은 세계 그래프)를 제시하는 매우 중요한 논문이다. HNSW는 확장 가능한 근사 근접 이웃 검색(ANN) 알고리즘으로, 높은 차원의 벡터 데이터에서 빠른 검색 속도와 높은 정확도를 동시에 달성한다.

알고리즘의 핵심은 계층적 그래프 구조로, 하단의 밀집 그래프에서 상단의 희소 그래프로 이동하면서 검색 공간을 점진적으로 좁혀나간다. 이는 "네비게이팅 작은 세계" 개념(small-world phenomenon)을 그래프에 적용한 것으로,  $O(\log n)$  시간 복잡도의 검색을 실현한다.

본 논문과의 관계: Exqutor 시스템은 HNSW를 기본 인덱싱 구조로 사용하며, 필터링 기반의 조건부 근접 이웃 검색을 HNSW 위에서 구현한다. HNSW의 계층적 구조와 네비게이팅 메커니즘을 활용하여 속성 필터를 함께 고려하는 효율적인 검색을 수행한다.

## 상세분석

### 38.1 문제 정의 및 동기

**근접 이웃 검색의 도전**: - 고차원 벡터 공간에서 정확한 KNN 검색:  $O(n)$  선형 탐색 필요 - 대규모 데이터셋(수백만 ~수십억): 실시간 검색 불가능 - 차원의 저주: 트리 기반 인덱스의 성능 급격히 저하 - 정확도와 속도의 트레이드오프

프 필요

**HNSW의 목표:** - 근사 최근접 이웃만 검색하되 매우 빠르고 정확 -  $O(\log n)$  시간 복잡도 달성 - 메모리 효율적인 인덱스 구조 - 높은 차원(1000+)에서도 안정적 성능

### 38.2 작은 세계 현상(Small-World Phenomenon)

**작은 세계의 특성:** - 임의의 두 노드 사이의 최단 경로가 놀랍게 짧음 - 평균 경로 길이:  $O(\log n)$  또는 더 작음 - 실제 네트워크들이 가진 특성 (소셜 네트워크, 신경망 등)

**Watts-Strogatz 모델:** - 규칙적 격자와 무작위 네트워크의 혼합 - 몇 개의 무작위 연결로 큰 세계에서 작은 세계로 전환 - 높은 클러스터링과 짧은 경로 길이 동시 달성

**HNSW에의 적용:** - 벡터 공간의 점들을 네트워크 노드로 생각 - 각 노드를 여러 이웃과 연결하여 작은 세계 그래프 구성 - 검색 시 몇 홉으로 근처 노드에 도달 가능

### 38.3 계층적 구조

**계층 개념:** - 다중 계층(multiple layers)으로 이루어진 그래프 구조 - 각 계층은 별도의 그래프로 표현 - 하단 계층: 밀집 그래프 (모든 점 포함, 많은 간선) - 상단 계층: 희소 그래프 (적은 수의 점, 적은 간선)

**계층 배정:**

$$L = \text{floor}(-\ln(U(0,1)) * m_l)$$

여기서: -  $U(0,1)$ : 0과 1 사이의 균등 난수 -  $m_l$ : 계층 정규화 인수 (보통  $1/\ln(2)$ ) -  $L$ 은 새 점의 최상단 계층 레벨

**계층의 확률적 특성:** - 상단 계층일수록 지수적으로 적은 점 포함 - 계층  $L$ 에 포함된 점: 약  $n * e^{-(m_l * L)}$ 개 - 균등 분포로 계층 깊이 자동 조정

### 38.4 삽입 알고리즘

**새로운 점  $q$  삽입:**

1. 진입점 선택

2. 최상단 계층부터 시작

3. 만약  $q$ 의  $L$ 이 기존 최상단보다 크면,  $q$ 를 새로운 진입점으로 설정

4. 계층별 가까운 이웃 찾기 (Greedy Search) `for l = max_layer down to L(q): neighbors = greedy_search_k(q, current_nearest, m) if l > L(q): m = 1 // 상단 계층에서는 가장 가까운 1개만 유지`

5. 상단 계층부터 하단으로 내려가며 점진적으로 검색

6. 각 계층에서 거리 기준으로 가까운  $m$ 개 이웃 선택

7. 간선 추가 및 기존 간선 정리 ``` for l = L(q) down to 0: neighbors = search\_layer(q, nearest, ef, l) m = m // 하단 계층에서는 m개 이웃 유지

for 각 neighbor에: if distance(q, neighbor) < existing\_edges의최대거리: 간선 추가 else if degree(neighbor) < m\_max: 간선 추가 else: 가장 먼 간선을 새 간선으로 교체 (휴리스틱) ```

## 8. 프루닝(Pruning)

9. 각 노드의 차수(degree) 제한: m\_max = 2\*m

10. 새 간선이 많은 노드들은 가장 먼 간선 제거

## 38.5 검색 알고리즘

K-NN 검색 프로세스:

```
procedure SearchLayer(q, ep, ef, lc):
 visited = {}
 w = {} // 최종 결과 집합
 candidates = {ep}
 lowerBound = distance(q, ep)

 while candidates가 비어있지 않음:
 current = candidates에서 lowerBound가 가장 작은 점

 if distance(q, current) > lowerBound:
 break // 더 이상의 개선 불가능

 for neighbor in connections[current]:
 if neighbor not in visited:
 visited.add(neighbor)
 d = distance(q, neighbor)

 if d < lowerBound or |w| < ef:
 candidates.add(neighbor)
 w.add(neighbor)

 if d < lowerBound:
 lowerBound = d

 // 결과 집합이 ef보다 크면 가장 먼 점 제거
 if |w| > ef:
 w.remove(farthest_point)
 lowerBound = max_distance_in_w

 return w
```

**ef 파라미터:** - 검색 범위 제어 (ef: expansion factor) - 작은 ef: 빠르지만 덜 정확 - 큰 ef: 느리지만 더 정확 - ef  $\geq K$  권장

**시간 복잡도:** - 각 계층에서의 검색:  $O(\log n)$  - 계층 수:  $O(\log n)$  - 전체:  $O(\log^2 n)$  (이론적) 또는  $O(\log n)$  (실제 성능)

### 38.6 메모리 효율성

**메모리 사용량:**

```
Memory = n * (2 * m * avg_edges_per_level + data_per_point)
```

**계산:** - 매개변수  $m = 16$ 인 경우: 약 64 바이트/포인트 (추가 인덱스) - 벡터 데이터와 별도로 저장 - 원본 데이터의 1~2배 정도 추가 메모리

**압축 가능성:** - 간선 리스트 압축 - 정량화(quantization)를 통한 벡터 크기 감소 - 캐시 친화적 구조로 메모리 접근 효율성 증대

### 38.7 하이퍼파라미터 선택

**M (최대 간선 수):** - 기본값:  $M = 16$  - 차원이 높을수록 M 증가 - 메모리와 성능의 트레이드오프

**ef\_construction (삽입 시 확장 계수):** - 기본값: ef\_construction = 200 - 높을수록 더 좋은 그래프지만 삽입 시간 증가 - 검색 성능과 직결

**ef (검색 시 확장 계수):** - 기본값: ef = K 또는  $K * 2$  - 동적으로 조정 가능 - 검색 속도와 정확도 조정용

### 38.8 성능 특성

**정확성:** - 정확한 KNN 검색 대비: 일반적으로 95% 이상 정확도 - ef 증가로 정확도 향상 가능 - M과 ef\_construction으로 사전 정확도 제어

**속도:** - 선형 탐색( $O(n)$ ): 백만 점 기준 수초 - HNSW( $O(\log n)$ ): 백만 점 기준 밀리초 단위 - 10~100배 속도 향상

**확장성:** - 수십억 개 점 지원 (메모리 허용 범위) - 삽입/삭제도 비교적 효율적 - 배치 작업에는 최적이지 않음 (증분 삽입 가정)

### 38.9 변형 및 최적화

**동적 삽입/삭제:** - 기본 HNSW는 점진적 삽입 최적화 - 삭제는 비용 높음 (주변 구조 재정리 필요) - 재구축이 더 효율적일 수 있음

**이질적(Heterogeneous) 그래프:** - 다른 M 값으로 다양한 밀도 제어 가능 - 계층 구조 최적화로 성능 향상

**병렬 삽입:** - 잠금(locking) 메커니즘으로 동시 삽입 지원 - 경쟁(contention) 최소화 필요

### 38.10 Exqutor에서의 HNSW 활용

**기본 인덱싱:** - Exqutor는 각 벡터를 HNSW 그래프의 노드로 저장 - 벡터의 유사도(코사인, 유클리드)에 기반한 간선 생성

**필터 통합:** - HNSW 검색 중 필터 조건 확인 - 필터를 만족하는 노드만 결과에 포함 - 필터 조건을 통과하지 못한 노드는 경로에서 스킵

**다중 필터 처리:** - 여러 속성 필터의 교집합 계산 - 필터된 노드 집합에서의 거리 기반 정렬 - 필터 선택도(selectivity)에 따른 검색 전략 변경

**성능 최적화:** - 필터 조건이 매우 선택적일 때: 넓은 검색 범위 필요 (높은 ef) - 필터 조건이 느슨할 때: 낮은 ef로도 충분 - 인덱스당 메타데이터 저장으로 필터 효율성 향상

---

#### 추가 제기 문제

1. **필터 조건의 HNSW 인덱싱:** 속성 필터 조건을 HNSW 그래프 구조 내에 어떻게 효율적으로 인코딩할 수 있을까? 필터 조건별 별도의 HNSW를 유지할 때의 메모리 오버헤드는?
2. **동적 필터 변경:** 런타임 중에 필터 조건이 자주 변경될 때, HNSW 구조를 동적으로 재구성하지 않고도 효율적으로 처리할 수 있을까?
3. **필터-거리 트레이드오프의 정량화:** 필터 조건의 엄격함과 검색 거리 사이의 관계를 정량적으로 모델링하면 검색 성능을 예측할 수 있을까?
4. **계층별 필터 적용:** HNSW의 상단 계층과 하단 계층에서 필터를 다르게 적용했을 때의 성능 차이는?
5. **필터된 데이터의 클러스터링:** 같은 필터 조건을 만족하는 데이터들이 벡터 공간에서 클러스터를 형성할 때, 이를 활용한 HNSW 최적화 방법은?
6. **근사도와 필터링:** HNSW의 근사 특성(일부 최근접 이웃 누락 가능성)이 필터링과 결합될 때 최종 결과의 누락률은 얼마나 증가할까?
7. **대규모 필터 세트:** 수천 개 이상의 서로 다른 필터 조건이 있을 때, 어떤 인덱싱 전략이 가장 메모리 효율적일까?
8. **실시간 업데이트:** HNSW에 새로운 벡터를 삽입할 때 기존 필터 인덱스 구조를 어떻게 효율적으로 업데이트할 수 있을까?
9. **필터 검색 순서 최적화:** 다중 필터 조건이 있을 때, 검색 성능을 최대화하기 위한 필터 적용 순서 결정 알고리즘은?
10. **벡터 임베딩 모델과의 상호작용:** BERT, GPT 등 다양한 임베딩 모델에서 생성된 벡터들의 특성이 HNSW 구조에 어떻게 영향을 미치는가? 모델별 최적의 M, ef 값은?



## [39] Fast approximate nearest neighbor search with the navigating spreading-out graph

**저자:** C. Fu, C. Xiang, C. Wang, and D. Cai **학술지:** Proc. VLDB Endow., vol. 12, no. 5, p. 461-474, 2019 **주제:** NSG(Navigating Spreading-out Graph) - 그래프 기반 ANN 인덱싱

---

### 요약

본 논문은 HNSW와 경쟁하는 또 다른 강력한 그래프 기반 ANN 인덱싱 기법인 NSG(Navigating Spreading-out Graph, 네비게이팅 스프레딩아웃 그래프)를 제시한다. NSG는 단일 계층의 그래프 구조를 사용하면서도 HNSW와 유사한 검색 성능을 달성하고, 메모리 효율성과 구성 속도 면에서 우위를 가진다.

NSG의 핵심은 "spreading-out" 개념으로, 각 노드가 다양한 방향의 이웃들을 균형 있게 연결하도록 한다. 이는 그래프의 네비게이팅 능력(navigability)을 높이면서도 불필요한 간선을 줄인다.

본 논문과의 관계: Exqutor가 HNSW 대신 NSG를 사용할 경우의 성능 특성을 이해하는 데 중요하다. 단일 계층 구조가 필터링과 결합될 때 메모리 효율성과 검색 정확도 간의 트레이드오프를 분석할 수 있다.

---

### 상세분석

#### 39.1 HNSW와 NSG의 비교

**HNSW (Hierarchical):** - 다중 계층 구조 (hierarchical) - 상단부터 하단으로 네비게이팅 - 삽입 시간 많이 소요 - 높은 메모리 사용량

**NSG (Navigating Spreading-out):** - 단일 계층 구조 (flat) - 직접 그래프 구성 - 더 빠른 구성 시간 - 더 낮은 메모리 오버헤드 - 검색 성능은 HNSW와 경쟁 가능

#### 39.2 Spreading-out 개념

**네비게이팅 능력(Navigability):** - 임의의 두 점 사이의 최단 경로가 짧은 성질 - 검색의 "점프(hop)" 거리 최소화 - HNSW처럼  $O(\log n)$  경로 길이 달성

**Spreading-out 원리:**

각 노드  $u$ 에 대해:

1.  $u$ 와의 거리에 따라 모든 데이터 점을 정렬
2. 상위  $M$ 개를  $candidates$ 로 선택
3. Spreading-out 휴리스틱 적용:
  - 점진적으로  $candidates$  중 서로 먼 점들 선택
  - 다양한 방향 커버리지 확보
  - 제한된 수의 간선만 유지

**목표:** - 각 노드가 로컬 영역뿐 아니라 먼 영역도 커버 - 네트워크의 작은 세계 특성 보존 - 필요한 간선 수 최소화

### 39.3 NSG 구성 알고리즘

#### 단계 1: 초기 그래프 생성

```
for each point p in dataset:
 // 모든 점과의 거리 계산
 distances = calculate_distances(p, all_points)

 // 상위 K개 근접 점 선택 ($K \gg M$)
 candidates = top_K_nearest(distances)

 // p의 이웃으로 설정
 neighbors[p] = candidates
```

#### 단계 2: Spreading-out 휴리스틱

```
function spreading_out(candidates, M):
 result = []
 selected = [first_element(candidates)]

 while |result| < M:
 // 현재까지 선택된 점들로부터 가장 먼 점 선택
 farthest = argmax_distance_to_selected(candidates - selected)
 selected.add(farthest)
 result.append(farthest)

 return result
```

#### 단계 3: 상호 연결(Reciprocal Links)

```
for each point p with neighbors N(p):
 for each neighbor q in N(p):
 // q에서 p로 역방향 링크 추가
 add_reverse_link(q, p)
```

#### 단계 4: 반복적 개선 (그래프 최적화)

```
for iteration in 1 to num_iterations:
 for each point p:
 // 현재 이웃들
 current_neighbors = neighbors[p]

 // 이웃의 이웃들 탐색 (2-hop 친구)
 friend_candidates = union(neighbors[neighbor]
 for neighbor in current_neighbors)

 // Spreading-out으로 새로운 이웃 선택
 new_neighbors = spreading_out(friend_candidates, M)

 // 더 나으면 업데이트
 if quality(new_neighbors) > quality(current_neighbors):
 neighbors[p] = new_neighbors
```

#### 39.4 검색 알고리즘

쿼리 처리:

```

procedure search_NSG(query_q, start_node, ef):
 visited = {start_node}
 candidates = {start_node}
 w = {start_node} // 결과 집합

 while not empty(candidates):
 // 현재 가장 가까운 점
 current = get_nearest_from_candidates(candidates)

 // 더 이상의 개선 불가능하면 종료
 if distance(current, q) > distance(farthest_in_w, q):
 break

 // current의 이웃 탐색
 for neighbor in neighbors[current]:
 if neighbor not in visited:
 visited.add(neighbor)
 d = distance(neighbor, q)

 // 더 좋은 점을 찾았으면 추가
 if d < distance(farthest_in_w, q) or |w| < ef:
 candidates.add(neighbor)
 w.add(neighbor)

 // 결과 집합 유지
 if |w| > ef:
 w.remove(farthest_point)

 return top_K_nearest_from_w(w, K)

```

### 39.5 시작점(Entry Point) 선택

**NSG의 시작점 전략:** - HNSW처럼 여러 계층의 진입점 불필요 - 데이터 중심(centroid) 점 사용 또는 무작위 시작  
 - 그래프 구조 최적화로 어디서 시작해도 비슷한 성능

**시작점 선택 알고리즘:**

```

entry_point = centroid of all points
또는
entry_point = random point from dataset

```

### 39.6 매개변수 분석

**M (차수 제한):** - 각 노드의 최대 이웃 수 - HNSW의 M과 유사 (보통 16~64) - M 증가: 검색 정확도 향상, 메모리 증가

**K (초기 후보 크기):** - Spreading-out 휴리스틱의 입력 크기 - 보통 M보다 훨씬 큼 ( $K = 50M \sim 100M$ ) - K 증가: 더 나은 이웃 선택, 구성 시간 증가

**ef (검색 확장 계수):** - HNSW와 동일한 역할 - 검색 시간과 정확도 트레이드오프 제어

**반복 횟수:** - 그래프 최적화 반복 (보통 3~5회) - 각 반복마다 그래프 품질 향상

### 39.7 계산 복잡도 분석

구성 시간:

초기 그래프:  $O(n * \log n)$  거리 계산 + 정렬  
 정렬:  $O(\log n)$  per point  
 최적화:  $O(n * M * K)$  반복 개선  
 전체:  $O(n * K * \log n) \sim O(n * K * \text{상수})$

메모리 사용:

HNSW: 대략 64 바이트/점 ( $M=16$ )  
 NSG: 대략 32 바이트/점 ( $M=16$ )  
 → NSG가 약 50% 메모리 절약

검색 시간:

평균:  $O(\log n)$  또는  $O(\sqrt{n})$   
 ef 파라미터에 따라 선형 계수 변동

### 39.8 NSG vs HNSW 성능 비교

지표	HNSW	NSG
구성 시간	중간~높음	낮음
메모리 사용	높음	낮음
검색 속도	매우 빠름	매우 빠름
검색 정확도	우수	우수
삽입/삭제	지원	어려움
구현 복잡도	중간	낮음

### 39.9 NSG의 장점

**메모리 효율성:** - 단일 계층으로 오버헤드 감소 - HNSW보다 약 50% 적은 메모리 - 대규모 데이터셋에 유리

**구성 속도:** - 다중 계층 유지 불필요 - 병렬 구성 용이 - 점진적 삽입보다 배치 구성 최적

**구현 단순성:** - HNSW보다 이해하고 구현하기 쉬움 - 페이지 참조 횟수 감소로 캐시 효율성 향상

**수렴성:** - 반복적 개선으로 점진적 성능 향상 - 구성 중단 가능성 (부분적 완성도 가능)

### 39.10 NSG의 단점

**동적 작업:** - 삽입/삭제가 비효율적 - 그래프 구조 재조정 필요 - HNSW가 더 적합

**수렴 속도:** - 반복 횟수에 따라 성능이 증가 - 충분한 반복 없으면 성능 저하

**시작점 민감성:** - 시작점 선택이 검색 성능에 영향 - 좋은 시작점 선택 휴리스틱 필요

### 39.11 필터링과 NSG의 통합

**필터링 전략:** - NSG 검색 중 필터 조건 적용 - 필터 조건을 만족하지 않는 노드는 스킵 - 필터 선택도에 따라 검색 범위 동적 조정

**성능 특성:** - 필터 선택도 높음: 넓은 검색 범위 필요 - 필터 선택도 낮음: 빠른 수렴 - 단일 계층 구조로 일관된 성능

## 추가 제기 문제

1. **NSG와 필터 인덱싱:** NSG의 단일 계층 구조가 다중 필터 조건과 결합될 때, 필터 조건별 별도의 그래프를 유지하는 것이 HNSW보다 메모리 효율적일까?
2. **시작점 선택 최적화:** 특정 필터 조건에 최적화된 시작점을 동적으로 선택할 수 있을까? 필터 조건별로 서로 다른 시작점을 유지하는 방식의 효과는?
3. **반복적 개선의 필터 적응:** NSG의 반복적 개선 단계에서 필터 조건을 고려하여 이웃을 선택하면 어떤 성능 향상을 기대할 수 있을까?
4. **온라인 학습:** 요청 패턴에 따라 NSG를 실시간으로 재구성할 수 있을까? 자주 함께 나타나는 필터 조건들에 최적화된 그래프 구조를 학습할 수 있을까?
5. **메모리 제약 환경에서의 NSG:** 메모리가 매우 제한된 환경(엣지 디바이스)에서 NSG와 필터링을 결합했을 때의 성능은?
6. **하이브리드 접근:** HNSW의 상위 계층과 NSG의 최적화된 단일 계층을 결합한 하이브리드 구조의 가능성은?
7. **필터 조건의 그래프 구조화:** 속성 기반 필터들의 유사도를 기반으로 필터 공간 내에서 NSG 그래프를 구성할 수 있을까?

8. **근사도 분석**: NSG가 HNSW보다 메모리 효율적이면서도 비슷한 정확도를 유지하는 이유를 필터링 관점에서 분석하면?
9. **확장성 한계**: NSG의 메모리 효율성에도 불구하고, 수십억 개 점 규모에서 필터링과 함께 사용할 때의 한계는?
- 

## [40] Song: Approximate nearest neighbor search on gpu

**저자**: W. Zhao, S. Tan, and P. Li **학술지**: ICDE 2020, pp. 1033-1044 **주제**: GPU 기반의 근사 근접 이웃 검색

---

### 요약

본 논문은 GPU(Graphics Processing Unit)를 활용한 고속 ANN(근사 근접 이웃) 검색 알고리즘 SONG(Spatial Ordered Navigation Graph)을 제시한다. GPU의 대규모 병렬 처리 능력을 활용하여 벡터 공간에서의 효율적인 검색을 구현한다.

SONG의 핵심은 공간 정렬(spatial ordering)을 통해 벡터들을 구조화하고, GPU 메모리 계층을 고려한 최적화된 네비게이팅 그래프를 구성하는 것이다. 배치 쿼리 처리와 대규모 병렬화로 CPU 기반 방식보다 훨씬 빠른 성능을 달성한다.

본 논문과의 관계: Exqutor가 필터링 기반 벡터 검색을 GPU에서 구현할 경우, SONG의 공간 정렬과 GPU 최적화 기법을 활용할 수 있다. 특히 대규모 쿼리 배치 처리 시 필터링과 거리 계산을 GPU에서 병렬로 수행하는 전략이 효과적이다.

---

### 상세분석

#### 40.1 GPU 기반 ANN의 필요성

**CPU 기반의 한계**: - 단일 스레드 성능 한계에 도달 - 벡터 거리 계산의 높은 CPU 비용 - 메모리 대역폭 부족으로 성능 정체

**GPU의 장점**: - 수천 개의 코어를 통한 대규모 병렬 처리 - 고대역폭 메모리 시스템 (HBM, GDDR) - 벡터 연산 (SIMD)에 특화된 하드웨어 - 배치 처리에 최적화

**활용 시나리오**: - 대규모 배치 검색 (수천~수십만 쿼리 동시 처리) - 실시간 인터랙티브 응용 - 추천 시스템의 후보 선택

#### 40.2 GPU 메모리 계층과 최적화

**GPU 메모리 구조**:

레지스터 (Register)

↓ (매우 빠름, 제한적)

공유 메모리 (Shared Memory)

↓ (빠름, 블록 내 공유)

글로벌 메모리 (Global Memory)

↓ (느림, 높은 지연)

메인 메모리 (Host Memory)

**최적화 전략:** 1. **데이터 지역성:** 자주 접근되는 데이터를 공유 메모리에 캐싱 2. **메모리 접근 패턴:** 연속적 메모리 접근으로 높은 대역폭 활용 3. **계산과 전송 오버래핑:** 데이터 전송 중 계산 실행

### 40.3 SONG 알고리즘 - 공간 정렬

**공간 정렬(Spatial Ordering)의 개념:** - 벡터들을 공간상의 순서에 따라 정렬 - 유사한 벡터들이 메모리상에 인접하게 배치 - 캐시 효율성과 메모리 접근성 향상

**공간 정렬 방법:**

1. Z-order curve (Morton order) 사용
  - 공간을 재귀적으로 분할하며 점들을 순서화
  - 다차원 공간을 1차원으로 인코딩
2. Hilbert curve 사용
  - Z-order보다 공간 지역성 더 잘 보존
  - 더 나은 클러스터링 구조
3. 쿼드트리 기반 정렬
  - 계층적 공간 분할
  - GPU 병렬화에 적합

**Z-order 인덱싱 예시 (2D):**

공간상의 점:

- (1, 1) → Z-인덱스 6
- (0, 1) → Z-인덱스 4
- (0, 0) → Z-인덱스 0
- (1, 0) → Z-인덱스 2

Z-순서: (0,0) → (1,0) → (0,1) → (1,1)

### 40.4 네비게이팅 그래프 구성

**정렬된 벡터로부터 그래프 생성:**



```

// 단계 1: 벡터들을 공간 순서대로 정렬
sorted_vectors = spatial_sort(all_vectors)

// 단계 2: 각 벡터에 대해 이웃 연결
for each vector v in sorted_vectors:
 // 로컬 이웃 (공간 순서상 근접)
 local_neighbors = get_spatial_neighbors(v)

 // 글로벌 이웃 (거리 기준)
 global_neighbors = search_nearest(v, K)

 // 하이브리드 이웃 집합
 neighbors[v] = combine(local_neighbors,
 global_neighbors)

// 단계 3: GPU 최적화를 위한 그래프 리정렬
reorder_for_gpu_cache(graph)

```

## 40.5 GPU 기반 배치 검색

병렬 검색 커널:

```

__global__ void search_batch_kernel(
 const float* queries, // N_q 개 쿼리
 const float* vectors, // N 개 벡터
 const int* graph, // 이웃 관계
 int K, int ef,
 int* results // 결과 저장
) {
 int query_id = blockIdx.x; // 쿼리별 블록
 int thread_id = threadIdx.x; // 스레드

 // 공유 메모리에 쿼리 벡터 로드
 __shared__ float query[VECTOR_DIM];
 if (thread_id < VECTOR_DIM) {
 query[thread_id] = queries[query_id * VECTOR_DIM
 + thread_id];
 }
 __syncthreads();

 // 각 스레드가 별도의 벡터와 거리 계산
 int vector_id = thread_id;
 while (vector_id < N) {
 float dist = compute_distance(
 query,
 vectors[vector_id],
 VECTOR_DIM
);

 // 결과 업데이트 (원자적 연산)
 atomicUpdate(results, query_id,
 vector_id, dist);

 vector_id += blockDim.x; // 그리드 스트라이드
 }
}

```

## 40.6 거리 계산의 병렬화

벡터 거리 계산:

```
// 유클리드 거리
distance = sqrt(sum((q_i - v_i)^2))

// GPU에서 병렬 계산
__device__ float compute_l2_distance(
 const float* q, const float* v, int dim
) {
 float sum = 0.0f;

 for (int i = threadIdx.x; i < dim;
 i += blockDim.x) {
 float diff = q[i] - v[i];
 sum += diff * diff;
 }

 // 블록 내 리덕션
 return block_reduce_sum(sum); // sum 병렬 합산
}
```

코사인 유사도:

```
similarity = dot(q, v) / (||q|| * ||v||)

// GPU 최적화
// 1. 벡터 정규화 사전 계산
// 2. 내적 계산 병렬화
// 3. 시뮬레이션된 거리 계산 (1 - similarity)
```

## 40.7 배치 처리 최적화

배치 크기 선택:

```
배치 크기 = GPU 메모리 / (쿼리당 메모리 + 임시 메모리)

예: GPU 메모리 32GB
 쿼리 당 메모리 = 벡터크기 + 결과 + 임시 버퍼
 ≈ 8KB (벡터 차원 1000)
 최적 배치 크기 ≈ 4,000,000 쿼리
```

배치 파이프라이닝:

시간 →

[배치 1]

└ H2D 전송 (호스트→GPU)

└ GPU 계산 (겹침)

└ D2H 전송 (GPU→호스트) [겹침]

[배치 2]

└ H2D 전송 [겹침]

└ GPU 계산

└ D2H 전송 [겹침]

## 40.8 메모리 효율성

메모리 사용량 분석:

벡터 데이터:  $N * \text{dim} * 4$  바이트

그래프 구조:  $N * M * 4$  바이트 ( $M$  = 이웃 수)

정렬 인덱스:  $N * 8$  바이트

총합:  $N * (4 * \text{dim} + 4 * M + 8)$  바이트

예: 100만 벡터, 1000차원,  $M=16$

=  $1M * (4K + 64 + 8) = \text{약 } 4GB$

GPU 메모리 제약: - 고급 GPU (A100): 40~80GB - 중급 GPU (RTX): 8~24GB - 일반 GPU (GTX): 2~6GB

## 40.9 CPU와 GPU 성능 비교

지표	CPU (HNSW)	GPU (SONG)
단일 쿼리	매우 빠름	중간 (오버헤드)
배치 100	빠름	매우 빠름
배치 10K	중간	극도로 빠름
메모리	적음	많음
전력 효율	우수	낮음
비용	낮음	높음

성능 향상: - 배치 쿼리: 10~100배 빠름 - 높은 처리량: 초당 백만 쿼리 처리 가능

## 40.10 필터링과의 통합

GPU 기반 필터링:

```

__global__ void search_filtered_kernel(
 const float* queries,
 const float* vectors,
 const int* attributes, // 속성값
 const AttributeFilter* filters,
 int* results
) {
 int query_id = blockIdx.x;
 int vector_id = threadIdx.x;

 // 1. 필터 조건 확인 (GPU에서 빠름)
 if (!check_filter(attributes[vector_id], filters)) {
 return; // 필터링된 벡터 스킵
 }

 // 2. 거리 계산
 float dist = compute_distance(
 queries[query_id],
 vectors[vector_id]
);

 // 3. 결과 저장
 atomicUpdate(results, query_id, vector_id, dist);
}

```

#### 40.11 SONG의 장점

**극도의 처리량:** - 초당 수백만 쿼리 처리 - 대규모 배치 작업에 최적

**병렬성:** - 쿼리 수준 병렬화 (각 쿼리가 별도 블록) - 벡터 수준 병렬화 (각 벡터가 별도 스레드) - 차원 수준 병렬화 (거리 계산 병렬)

**메모리 대역폭 활용:** - GPU 메모리 대역폭: 1~5TB/s (vs CPU 100GB/s) - 공간 정렬로 캐시 효율성 향상

#### 40.12 SONG의 단점

**지연시간(Latency):** - GPU 전송 오버헤드 - 단일 쿼리: CPU보다 느릴 수 있음

**메모리 제약:** - 대규모 데이터셋은 여러 GPU 필요 - 정렬에 따른 메모리 재구성 비용

**동적 데이터:** - 삽입/삭제 시 공간 정렬 재구성 필요 - 배치 업데이트에만 효율적

### 추가 제기 문제

1. **필터 조건의 GPU 최적화:** 복잡한 다중 필터 조건을 GPU에서 효율적으로 평가하려면? 필터 조건의 선택도에 따른 성능 예측 모델은?

2. **공간 정렬과 필터 상관성**: 벡터들이 공간상으로 정렬되었을 때, 같은 필터 조건을 만족하는 벡터들도 함께 클러스터링될까?
3. **GPU 메모리 계층의 필터링 활용**: 공유 메모리에 필터 인덱스를 캐싱하면 필터 처리 성능을 얼마나 향상시킬 수 있을까?
4. **다중 GPU 확장**: 여러 GPU에서 필터링된 검색을 분산 처리할 때 최적의 데이터 분할 전략은?
5. **실시간 업데이트와 필터링**: GPU에서 벡터를 실시간으로 추가하면서 필터링 성능을 유지하려면?
6. **필터 선택도와 성능**: 필터 선택도가 높을 때(많은 결과 제외) GPU의 이점이 얼마나 감소할까?
7. **전력 효율**: 필터링을 추가했을 때 GPU의 전력 소비와 처리량의 트레이드오프는?
8. **하이브리드 아키텍처**: CPU와 GPU를 함께 사용하여 필터링된 검색의 지연시간과 처리량을 모두 최적화할 수 있을까?

---

## [41] Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters

**저자**: S. Gollapudi et al. **학술지**: WWW '23, p. 3406-3416 **주제**: DiskANN 기반의 필터링된 벡터 검색

---

### 요약

본 논문은 속성 필터링을 고려한 효율적인 근사 근접 이웃 검색 알고리즘 Filtered-DiskANN을 제시한다. DiskANN은 원래 디스크 기반 대규모 벡터 인덱싱을 위한 알고리즘이며, 본 논문에서는 이를 확장하여 속성별 필터링을 통합한다.

Filtered-DiskANN의 핵심은 필터 조건을 만족하는 벡터들의 부분집합에 대한 근접 이웃을 효율적으로 찾는 것이다. 필터의 선택도(selectivity)에 상관없이 일정 수준의 성능을 유지하면서도 메모리 효율성을 보장한다.

본 논문과의 관계: Exqutor의 핵심 문제인 "필터 조건과 거리 기반 검색의 효율적 결합"을 직접 다루는 논문이다. Filtered-DiskANN의 설계 원칙과 최적화 기법은 Exqutor의 필터링 전략과 매우 유사하다.

---

### 상세분석

#### 41.1 DiskANN의 배경

**DiskANN의 특징**: - SSD 디스크를 주로 사용하는 저비용 대규모 인덱싱 - 메모리에는 최소한의 구조만 유지 - Vamana 그래프 알고리즘 기반

## 메모리-디스크 계층:

빠르고 작음	느리고 큼
메모리 (GPU/RAM)	← 디스크 (SSD)
↑	↑
핫 데이터	콜드 데이터
(최근 접근)	(대부분)

**DiskANN의 장점:** - 메모리 제약 극복: 수십억 개 벡터 지원 - 비용 효율적: SSD 가격이 RAM보다 저렴 - 높은 성능: 벡터당 수 밀리초 검색

## 41.2 필터링의 도전

### 기본 문제:

필터 조건  $P$ 가 주어졌을 때, 조건을 만족하는 벡터  $V_P \subseteq V$  중에서 쿼리  $q$ 에 가장 가까운  $K$ 개를 찾기.

문제:  $\text{FILTERED\_KNN}(q, K, P)$   
입력: 쿼리  $q$ ,  $K$ , 필터  $P$   
출력:  $\{v \in V \mid P(v)\}$  중  $q$ 에 가장 가까운  $K$ 개

### 도전:

1. 필터 조건을 만족하는 벡터의 개수 미지수
2. 필터를 만족하는 벡터가 매우 드물 수 있음
3. 필터 조건이 벡터 공간의 구조와 무관

### 필터 선택도의 영향:

선택도 =  $|V_P| / |V|$

선택도 99% (거의 모든 벡터 선택):

- 기본 ANN 검색과 거의 동일
- 작은 오버헤드

선택도 1% (대부분 필터링 제외):

- 넓은 검색 범위 필요
- 높은 계산 비용

선택도 0.1% (매우 선택적):

- 극도로 넓은 검색 필요
- 거의 선형 탐색 수준

## 41.3 Filtered-DiskANN의 설계 원칙

### 필터-거리 통합 접근:

#### 1. 필터-우선 전략 (Filter-First) ``

2. 필터 조건 만족하는 벡터들 식별
3. 해당 벡터들 중 KNN 검색

장점: 필터 조건 만족 보장 단점: 선택도 낮으면 검색 범위 커짐 ``

#### 1. 하이브리드 전략 (Hybrid) ``

2. 넓은 범위에서 후보 수집
3. 필터 조건 적용하여 정제
4. 부족한 결과는 추가 검색

장점: 모든 선택도에서 균형 단점: 구현 복잡도 높음 ``

### 41.4 Filtered-DiskANN 알고리즘

인덱스 구조:

Vamana 그래프:

- 각 벡터가 노드
- 각 노드는 M개의 이웃 연결
- 거리 기반의 엣지 가중치

메타데이터:

- 각 벡터의 속성값 저장
- 속성 인덱스 (속성 값 → 벡터 ID)
- 선택도 통계

검색 프로시저:



```

procedure FILTERED_SEARCH(q, K, filter_predicate,
 num_search_nodes):
 // 1. 진입점 선택 (필터 만족하는 점)
 entry_point = find_entry_point(filter_predicate)

 // 2. 초기 후보 수집
 visited = {}
 candidates = priority_queue() // 거리순 정렬
 result_set = {}

 candidates.push(entry_point)
 visited.add(entry_point)

 // 3. 그리디 검색
 while not candidates.empty() and
 |visited| < num_search_nodes:

 current = candidates.pop()

 // 필터 조건 확인
 if not filter_predicate(current):
 continue // 필터 미충족 스킵

 // 결과 집합 추가
 dist = distance(q, current)
 if |result_set| < K or dist < max_dist(result_set):
 result_set.add((current, dist))
 if |result_set| > K:
 remove_farthest(result_set)

 // 이웃 탐색
 for neighbor in neighbors[current]:
 if neighbor not in visited:
 visited.add(neighbor)
 candidates.push(neighbor)

 // 4. 결과 정렬 및 반환
 return sort_by_distance(result_set)

```

## 41.5 필터 선택도 적응

동적 검색 범위 조정:

필터 선택도 추정:

```
selectivity = |결과_필터_통과| / |방문_노드|
```

검색 범위 조정:

```
if selectivity > 50%: // 필터 선택적이지 않음
 num_search_nodes = K * 2
elif selectivity > 10%:
 num_search_nodes = K * 5
elif selectivity > 1%:
 num_search_nodes = K * 20
else:
 num_search_nodes = K * 100 또는 더 큼
```

## 41.6 속성 인덱싱 전략

단순 인덱싱:

속성별 비트맵 인덱스:

```
color="red" → 비트맵 001010...
```

(해당 벡터는 1, 아니면 0)

장점: 빠른 필터 평가

단점: 메모리 사용량 (속성당 1 비트/벡터)

계층화 인덱싱:

B-트리 기반 속성 인덱스:

범위 필터 (price > 100) 지원  
효율적인 범위 쿼리

디스크 I/O 최적화:

자주 함께 필터링되는 속성들을 근처에 배치

## 41.7 성능 최적화 기법

진입점 최적화:

필터를 만족하는 진입점을 미리 식별:

방법 1: 필터별 진입점 캐싱

```
filtered_entry_point[filter_id] = p
```

방법 2: 범용 진입점에서 필터 만족점까지 빠른 이동

entry\_point에서 시작

→ 이웃 탐색으로 필터 만족점 빠르게 찾기

## 이웃 구성 최적화:

필터 친화적 이웃 선택:

전통적 Vamana:

각 점  $p$ 의 이웃 = 가장 가까운  $M$ 개

필터 인식 Vamana:

각 점  $p$ 의 이웃 = 거리/필터\_상관성 혼합 기반

- 같은 필터 그룹의 점들을 함께 연결
- 그룹 간 "다리" 역할 노드 유지

## 41.8 디스크 I/O 최적화

### 메모리 버퍼:

메모리 계층:

- 1단계: GPU 캐시 (가장 자주 접근)
- 2단계: RAM 버퍼 (핫 노드)
- 3단계: SSD 캐시 (온도 낮은 노드)
- 4단계: 메인 SSD (대부분의 데이터)

### 배치 I/O:

여러 쿼리의 필요한 벡터들을 함께 로드:

쿼리 1 → 노드 {1, 5, 12, 3}

쿼리 2 → 노드 {2, 5, 8, 3}

쿼리 3 → 노드 {1, 7, 12, 8}

공통 노드 {1, 5, 12, 3, 2, 8, 7}를 한 번에 로드

## 41.9 메모리 사용 분석

### 메모리 구성:

벡터 데이터:  $N \times \text{dim} \times 4\text{바이트}$  (SSD에 저장)

메타그래프:  $N \times M \times 4\text{바이트}$  (메모리,  $M=64$ )

= 1M 벡터,  $M=64$ : 256MB

속성 인덱스: 속성 수  $\times$  (인덱스 구조 크기)

= 100개 속성, 단순 비트맵: 12.5MB/속성

## 41.10 필터 조건의 복잡성 처리

### 단순 필터:

```
price > 100
color == "red"
```

복합 필터:

```
(price > 100 AND price < 500) AND
(color == "red" OR color == "blue") AND
(stock > 0)
```

평가 순서 최적화:

1. 가장 선택적인 조건부터 평가
2. 조기 종료 (AND에서 첫 거짓)
3. 속성 인덱스 활용 (범위 쿼리)

#### 41.11 비교: Filtered-DiskANN vs Exqutor

측면	Filtered-DiskANN	Exqutor
그래프	Vamana	HNSW
저장소	디스크 기반	메모리 기반
필터 처리	동적 범위 조정	필터-우선 또는 하이브리드
특화	대규모 배치	실시간 쿼리
메모리	매우 효율적	중간 정도
속도	중간~높음	매우 높음

#### 41.12 Exqutor에의 시사점

**필터 선택도 적응 메커니즘:** - 동적으로 필터 선택도를 추정 - 선택도에 따라 검색 범위 조정 - 필터 조건이 매우 선택적일 때: 광범위 검색 필요

**속성 인덱싱 전략:** - 자주 함께 필터링되는 속성들을 함께 저장 - 범위 필터 지원을 위한 B-트리 구조 - 메모리와 성능 사이의 균형

**진입점 최적화:** - 필터를 만족하는 진입점을 빠르게 찾기 - 필터별 진입점 캐싱의 효과

### 추가 제기 문제

1. **필터 선택도의 사전 예측:** 주어진 필터 조건에 대한 선택도를 쿼리 전에 정확하게 예측할 수 있을까? 통계 기반 모델의 효율성은?

2. **다중 필터의 동시 최적화**: 여러 속성에 대한 필터 조건이 있을 때, 평가 순서를 동적으로 최적화하면 성능을 얼마나 향상시킬 수 있을까?
3. **HNSW와 필터링의 결합**: Filtered-DiskANN의 접근 방식을 HNSW(Exqutor가 사용)에 적용했을 때의 성능 특성은?
4. **필터 통계의 유지**: 인덱스가 동적으로 변할 때 필터 선택도 통계를 어떻게 효율적으로 유지할 것인가?
5. **비용 모델**: 필터 평가 비용과 거리 계산 비용을 모두 고려한 종합적 쿼리 최적화 모델은?
6. **캐시 친화성**: 필터 조건을 만족하는 벡터들이 메모리상에서 인접하도록 배치하면 성능을 얼마나 향상시킬 수 있을까?
7. **필터 인덱스 압축**: 속성 비트맵을 압축하면서도 빠른 필터 평가를 유지할 수 있을까?

---

## [42] PM-LSH: A fast and accurate lsh framework for high-dimensional approximate nearest neighbor search

**저자**: B. Zheng et al. **학술지**: Proc. VLDB Endow., vol. 13, no. 5, p. 643-655, 2020 **주제**: LSH 기반의 고차원 근사 근접 이웃 검색

---

### 요약

본 논문은 Locality-Sensitive Hashing(LSH) 기반의 고성능 ANN 프레임워크 PM-LSH(Prefix-Matching Locality-Sensitive Hashing)를 제시한다. LSH는 고차원 공간에서의 근사 근접 이웃 검색을 위한 확률적 방법으로, 유사한 벡터들이 같은 해시 버킷으로 매핑될 확률이 높다.

PM-LSH는 기존 LSH의 단점(높은 거짓양성율, 메모리 오버헤드)을 개선하여, 접두사 매칭(prefix matching) 기법으로 빠르고 정확한 검색을 구현한다.

본 논문과의 관계: Exqutor의 벡터 검색은 HNSW 같은 그래프 기반 인덱싱을 사용하지만, LSH 기반 방식도 필터링과 결합할 경우 다른 성능 특성을 가진다. 특히 고차원에서의 성능 비교 분석이 중요하다.

---

### 상세분석

#### 42.1 LSH의 기본 원리

##### 핵심 아이디어:

유사한 데이터 포인트들이 높은 확률로 같은 해시 버킷으로 매핑되도록 하는 해시 함수족을 설계한다.

### 수학적 정의:

해시 함수족  $H$ 가 LSH라면: -  $p_1 > p_2$  (확률) - 거리  $d(x, y) \leq r$  일 때:  $\Pr[h(x) = h(y)] \geq p_1$  - 거리  $d(x, y) > cr$  일 때:  $\Pr[h(x) = h(y)] \leq p_2$

여기서  $c > 1$ 은 근사 비율이다.

### 직관:

유사 포인트 (거리  $\leq r$ )  
↓ 높은 확률로  
[같은 해시 버킷]  
↑ 낮은 확률로  
다른 포인트 (거리  $> cr$ )

## 42.2 전통적 LSH의 구조

### 레이어 구조:

layer 1:  $h_1 = [h^{(1)}_1, h^{(1)}_2, \dots, h^{(1)}_k] \rightarrow$  해시값 1  
( $k$ 개 함수의 연결)  
  
layer 2:  $h_2 = [h^{(2)}_1, h^{(2)}_2, \dots, h^{(2)}_k] \rightarrow$  해시값 2  
...  
layer L:  $h_L = [h^{(L)}_1, h^{(L)}_2, \dots, h^{(L)}_k] \rightarrow$  해시값 L

### 검색 프로세스:

1. 쿼리  $q$ 의 각 레이어별 해시값 계산
2. 각 레이어에서 같은 해시 버킷의 모든 점 찾기
3. 찾은 모든 점들 중 가장 가까운  $k$ 개 선택

시간복잡도:  $O(L \times \text{hash\_computation} + \text{후보\_수})$

## 42.3 PM-LSH의 개선점

### 문제점 분석:

전통 LSH의 단점: 1. **높은 거짓양성**: 다른 점이 같은 버킷에 들어올 확률 높음 2. **메모리 낭비**:  $L$ 개 레이어 모두에서 데이터 복사 필요 3. **계산 비용**: 많은 후보에 대해 거리 계산

### 점두사 매칭 기법:

전통 LSH:

레이어 1: 해시 (010110...)

레이어 2: 해시 (101010...)

레이어 3: 해시 (111001...)

→ 각 레이어에서 따로 검색

PM-LSH:

모든 레이어의 해시를 연결한 하나의 키: 010110|101010|111001

검색 시:

1단계: 해시 010110\* 으로 시작하는 모든 항목 (레이어 1)

2단계: 010110|101010\* 으로 시작하는 모든 항목 (레이어 2)

3단계: 010110|101010|111001\* 정확히 일치

## 42.4 PM-LSH 알고리즘 상세

인덱싱:

```
procedure BUILD_PM_LSH_INDEX(vectors, k, L):
 // k: 각 레이어의 함수 개수
 // L: 레이어 개수

 // 1단계: LSH 함수족 생성
 for layer = 1 to L:
 for func_idx = 1 to k:
 h[layer][func_idx] = create_random_projection()

 // 2단계: 점두사 트리 구성
 prefix_tree = TrieNode()

 for vector v in vectors:
 key = ""
 for layer = 1 to L:
 // 해시값 계산
 hash_value = compute_hash(v, h[layer], k)
 key = key + "|" + hash_value // 점두사 연결

 // 트라이에 삽입
 insert_to_trie(prefix_tree, key, v)

 return prefix_tree
```

검색:

```

procedure SEARCH_PM_LSH(query_q, K, early_termination_threshold):
 candidates = []
 prefix = ""

 for layer = 1 to L:
 // 현재 레이어의 해시값 계산
 hash_value = compute_hash(q, h[layer], K)
 prefix = prefix + "|" + hash_value

 // 점두사와 일치하는 모든 벡터 찾기
 matching_vectors = search_prefix_in_trie(
 prefix_tree, prefix
)

 // 거리 계산 및 후보 수집
 for v in matching_vectors:
 dist = distance(q, v)
 candidates.append((v, dist))

 // 조기 종료 (충분한 좋은 결과)
 if |candidates| >= early_termination_threshold:
 break

 // 가장 가까운 K개 반환
 return top_K_by_distance(candidates, K)

```

## 42.5 해시 함수 선택

### Random Projection (무작위 사영):

가장 널리 사용되는 LSH 함수

$$h_{a,b}(v) = \text{floor}((a \cdot v + b) / r)$$

여기서:

- a: 무작위 벡터 (각 차원:  $N(0,1)$ )
- b:  $[0, r)$  범위의 무작위 스칼라
- r: 버킷 크기 (파라미터)
- v: 입력 벡터

거리 메트릭: L2 (유클리드)

### 코사인 유사도를 위한 LSH:



$h_a(v) = \text{sign}(a \cdot v)$

여기서:

- $a$ : 무작위 벡터
- $\text{sign}()$ : 양수면 1, 음수면 0

확률:  $P(h_a(x) = h_a(y)) = 1 - \theta(x,y)/\pi$   
( $\theta$ 는 벡터 간 각도)

## 42.6 파라미터 튜닝

$k$ 와  $L$ 의 영향:

더 많은 함수 (큰  $k$ ):

- 더 정확한 구분 (낮은 거짓양성)
- 높은 계산 비용
- 낮은 거짓음성 (실제 유사점 놓치기)

더 많은 레이어 (큰  $L$ ):

- 더 높은 회수율(recall)
- 더 큰 메모리
- 더 많은 계산

최적화:

- $k + L = \text{상수}$  (메모리 제약)
- $k$ 를 높여 정확도 향상
- $L$ 을 높여 회수율 향상

버킷 크기  $r$ 의 선택:

작은  $r$  (더 많은 버킷):

- 낮은 거짓양성
- 높은 거짓음성

큰  $r$  (적은 버킷):

- 높은 거짓양성
- 낮은 거짓음성

권장: 데이터의 타입 거리 분포에 맞춰 선택

## 42.7 접두사 트리 구현

Trie 구조:

```

루트
└─ 0
 │ └─ 1
 │ │ └─ 0 → [벡터 ID들]
 │ │ └─ 1 → [벡터 ID들]
 │ └─ 0
 │ └─ 1 → [벡터 ID들]
└─ 1
 └─ ...

```

메모리 효율성:

전통 LSH (L개 해시 테이블):

메모리 =  $L \times (\text{평균 버킷 크기} \times \text{포인터})$

PM-LSH (단일 트리):

메모리 = 트리 노드 수  $\times$  (자식 수 + 데이터)

일반적으로 PM-LSH가 10~30% 더 효율적

## 42.8 성능 특성

시간 복잡도:

인덱싱:  $O(L \times k \times n \times d)$

- n: 벡터 개수
- d: 벡터 차원
- k: 함수 개수
- L: 레이어 개수

검색:  $O(L \times k + \text{평균\_후보\_수} \times d)$

- 조기 종료로 평균 L을 줄일 수 있음

메모리 복잡도:

벡터 저장:  $O(n \times d)$

LSH 구조:  $O(L \times k \times d)$  (함수 저장)

트리:  $O(\text{계약 가능})$

## 42.9 PM-LSH vs HNSW

지표	PM-LSH	HNSW
구성 시간	낮음	중간
메모리	효율적	중간
검색 속도	중간~높음	매우 빠름
정확도	좋음	우수
파라미터 조정	복잡	간단
확장성	매우 좋음	좋음

## 42.10 필터링과의 통합

LSH 기반 필터링:

```
procedure SEARCH_FILTERED_PM_LSH(
 query_q, K, filter_predicate
):
 candidates = []

 for layer = 1 to L:
 hash_value = compute_hash(q, h[layer])
 prefix = current_prefix + hash_value

 // 점두사와 일치하는 벡터
 matching_vectors = search_prefix(prefix_tree, prefix)

 for v in matching_vectors:
 // 필터 조건 확인
 if not filter_predicate(v):
 continue

 // 거리 계산
 dist = distance(q, v)
 candidates.append((v, dist))

 // 충분한 결과를 얻으면 종료
 if len(candidates) >= K:
 break

 return top_K(candidates, K)
```

필터와 해시의 상호작용:

필터 조건 "category == 'electronics'":

- 전체 벡터의 30% 선택

LSH 해시로 찾은 후보: 100개

- 그 중 필터 만족: 30개 (예상)

정확도는 유지되지만 후보 집합이 감소

→ 거짓음성 증가 가능성

## 42.11 적응형 검색 전략

동적 레이어 확장:

처음 N개 레이어만 검색:

- 빠른 응답
- 낮은 정확도

필요시 추가 레이어로 확장:

- 더 많은 후보 수집
- 회수율 향상
- 응답 시간 증가

## 42.12 실제 성능 특성 (1M 벡터, 1000차원)

구성 시간:

HNSW: 약 30분

PM-LSH: 약 5분

메모리:

HNSW: 약 4GB

PM-LSH: 약 3GB

검색 속도:

HNSW: 0.5ms/쿼리

PM-LSH: 1-2ms/쿼리

정확도 (top-10):

HNSW: 95%

PM-LSH: 90%

## 추가 제기 문제

1. **필터 조건과 LSH의 상호작용**: 필터 조건이 매우 선택적일 때, LSH의 성능이 어떻게 변하는가? 필터된 벡터들의 거리 분포가 어떻게 변할까?
2. **해시 함수 선택의 필터 최적화**: 특정 필터 조건에 최적화된 해시 함수를 설계할 수 있을까? 예를 들어, "category" 필터를 고려한 특화된 LSH?
3. **접두사 트리의 필터 인덱싱**: 트리 구조 내에 필터 조건 정보를 인코딩하면 필터 검색 성능을 향상시킬 수 있을까?
4. **다중 필터와 LSH의 조합**: 여러 필터 조건이 있을 때, 이들을 LSH 함수에 포함시켜 함께 해싱할 수 있을까?
5. **확률적 보장**: LSH의 확률적 성질이 필터링과 결합될 때, 전체 반환 결과의 정확도 보장은 어떻게 변할까?
6. **메모리 효율성의 필터 활용**: 필터로 인해 실제로 접근해야 할 벡터가 줄어들 때, LSH 구조의 메모리를 동적으로 조정할 수 있을까?
7. **필터 조건의 변화**: 런타임 중에 필터 조건이 자주 변할 때, 기존 LSH 인덱스를 재구성하지 않고도 효율적으로 처리할 수 있을까?
8. **정확도와 성능의 분석**: PM-LSH가 HNSW보다 메모리 효율적이면서도 검색이 느린 이유를 필터링 관점에서 분석하면?
9. **하이브리드 인덱싱**: HNSW의 상위 계층과 PM-LSH의 정확한 검색을 결합한 구조의 성능은?

## [43] Neighbor-sensitive hashing

**저자**: Y. Park, M. Cafarella, and B. Mozafari **학술지**: Proc. VLDB Endow., vol. 9, no. 3, p. 144-155, 2015 **주제**: 이웃 감지 해싱(NSH) - 이웃 정보를 활용한 새로운 해싱 기법

## 요약

본 논문은 Neighbor-Sensitive Hashing(NSH)라는 새로운 해싱 기반 ANN 기법을 제시한다. 전통적인 LSH와 달리, NSH는 각 데이터 포인트의 이웃 구조를 명시적으로 활용하여 해시 함수를 설계한다.

NSH의 핵심 아이디어는 "이웃들이 같은 해시 버킷에 들어갈 확률을 높인다"는 것이다. 이를 위해 실제 데이터로부터 학습된 이웃 그래프를 기반으로 최적화된 해시 함수를 구성한다.

본 논문과의 관계: Exqutor의 필터링 기반 검색에서 필터 조건을 만족하는 벡터들의 이웃 구조를 활용하면, NSH처럼 필터 친화적인 인덱싱을 구성할 수 있다.

## 상세분석

### 43.1 LSH의 한계와 NSH의 동기

LSH의 문제점:

LSH는 무작위 함수를 사용:

- 데이터 분포 미고려
- 실제 이웃 관계 무시
- 높은 거짓양성율

문제:

벡터 A와 B가 유사하지만  
LSH 함수의 무작위성으로 인해  
다른 해시 버킷에 들어갈 수 있음

NSH의 아이디어:

데이터의 실제 이웃 구조를 분석:

1. 데이터로부터 이웃 그래프 구성
2. 이웃 관계를 유지하는 해시 함수 설계
3. 유사한 점들이 같은 해시 버킷에 들어가도록 학습

결과: LSH보다 정확하고 효율적인 검색

### 43.2 이웃 그래프 구성

KNN 그래프:

각 점  $p$ 에 대해:

1. 모든 다른 점과의 거리 계산
2. 가장 가까운  $k$ 개 선택
3.  $p \rightarrow \{k\text{개 이웃}\}$  연결

예 ( $k=3$ ):

점 A: [B, C, D] (A와 가장 가까운 3개)

점 B: [A, E, F]

점 C: [A, D, G]

...

그래프 특성:

방향 그래프:

- A가 B의 이웃이라고 해서 B가 A의 이웃은 아닐 수 있음

상호 이웃 (mutual neighbors):

- A와 B가 서로를 이웃으로 포함하는 경우
- 더 강한 유사도 신호

계산 복잡도:

KNN 그래프 구성:  $O(n^2)$

(모든 점 쌍의 거리 계산)

대규모 데이터: 근사 KNN 그래프 사용

- 샤할링 또는 클러스터링으로 근사
- $O(n \log n)$  또는  $O(n\sqrt{n})$

### 43.3 NSH 함수 설계

학습 문제로의 재정의:

목표: 해시 함수  $h$ 를 설계하여

이웃인 점들은 같은 해시 값 확률 높음

다른 점들은 다른 해시 값 확률 높음

제약:

- 해시 값은  $0 \sim m-1$  범위 ( $m$ 개 버킷)
- 계산 비용 낮아야 함

최적화 공식:

최대화:

$$\sum_{\{(i,j) \in \text{edges}\}} [h(i) = h(j)]$$

(이웃 쌍이 같은 버킷)

최소화:

$$\sum_{\{(i,j) \notin \text{edges}\}} [h(i) = h(j)]$$

(비이웃 쌍이 같은 버킷)

### 43.4 NSH의 구체적 구현

방법 1: Spectral Hashing (스펙트럼 해싱)

그래프 라플라시안 계산:

$$L = D - A$$

(D: 차수 행렬, A: 인접 행렬)

라플라시안의 고유벡터 사용:

$v_1, v_2, \dots$  (가장 작은 고유값부터)

해시 함수:

$$h(x) = [\text{sign}(v_1^T x), \text{sign}(v_2^T x), \dots]$$

(이진 해시 코드 생성)

결과: 각 비트가 이웃 보존성 최적화

## 방법 2: 최소 컷 기반 (Min-cut)

목표: 이웃을 최대한 같은 그룹으로

비이웃을 다른 그룹으로

알고리즘:

1. 그래프에서 최소 컷 찾기
2. 이웃은 같은 쪽, 비이웃은 다른 쪽
3. 각 컷이 하나의 비트를 정의

결과: 이웃 관계를 명시적으로 보존하는 해시

## 방법 3: 확률적 접근

마르코프 연쇄 몬테카를로(MCMC) 사용:

1. 현재 해시 할당 상태에서 시작
2. 점진적으로 할당 개선
3. 수렴 시 최적 또는 근최적 해시

장점: 이론적 보장 가능

단점: 계산 비용 높음

## 43.5 다중 해시 테이블과 검색

k개 해시 함수 사용:

함수 1:  $h_1(x) = [\text{bit}_{0_1}, \text{bit}_{1_1}, \dots]$

함수 2:  $h_2(x) = [\text{bit}_{0_2}, \text{bit}_{1_2}, \dots]$

...

함수 k:  $h_k(x) = [\text{bit}_{0_k}, \text{bit}_{1_k}, \dots]$

각 함수는 이웃 구조의 다른 측면 캡처



### 검색 알고리즘:

```
procedure NSH_SEARCH(query_q, K, k_tables):
 all_candidates = []

 for hash_function_idx = 1 to k_tables:
 h_idx = hash_function_idx

 // 쿼리의 해시 코드 계산
 query_hash = compute_hash(q, h_idx)

 // 같은 해시 코드의 모든 점 찾기
 bucket_contents = hashtable[h_idx][query_hash]
 all_candidates.extend(bucket_contents)

 // 중복 제거
 all_candidates = unique(all_candidates)

 // 거리 기반 정렬
 candidates_with_dist = [
 (c, distance(q, c)) for c in all_candidates
]

 // 가장 가까운 K개 반환
 return sorted(candidates_with_dist,
 key=lambda x: x[1])[:K]
```

### 43.6 이론적 보장

#### 성능 분석:

이웃 보존율 (Recall):  
= (검색된\_이웃\_개수) / (실제\_이웃\_개수)

정밀도 (Precision):  
= (실제\_이웃) / (검색된\_점\_개수)

NSH의 이론:  
적절한 k\_tables 선택 시  
높은 recall과 precision 달성 가능

#### 시간 복잡도:

검색:  $O(k\_tables \times \text{평균\_버킷\_크기} + \text{정렬})$

대부분의 경우:

$O(k\_tables \times (\text{평균\_버킷\_크기}))$

평균 버킷 크기  $\ll n$

### 43.7 NSH vs LSH vs HNSW

지표	LSH	NSH	HNSW
함수 구성	무작위	학습 기반	N/A
정확도	중간	높음	매우 높음
속도	빠름	빠름	매우 빠름
메모리	중간	중간	낮음
인덱싱 비용	낮음	중간	높음
파라미터 조정	복잡	간단	간단

### 43.8 필터링과의 통합

필터 조건을 고려한 NSH:

문제:

1. 필터를 만족하는 점들의 부분집합으로 이웃 그래프 구성
2. 해시 함수는 필터 만족 점들 사이의 이웃 보존

알고리즘:

```
// 단계 1: 필터된 KNN 그래프 구성
filtered_points = {p | filter_predicate(p)}
knn_graph = build_knn_graph(filtered_points, k)

// 단계 2: NSH 함수 학습
hash_functions = learn_nsh_functions(knn_graph)

// 단계 3: 검색
query_candidates = search_with_hash_functions(q)
return filter_results_by_distance(
 query_candidates, K
)
```

성능 특성:

필터 선택도 높음 (많은 점 선택):

- 필터 없는 NSH와 유사
- 이웃 구조 더 풍부
- 좋은 성능

필터 선택도 낮음 (적은 점 선택):

- 이웃 구조 희소
- 해시 함수 학습 어려움
- 성능 저하 가능

## 43.9 실제 구현 고려사항

이웃 그래프 유지:

정적 데이터: 한 번만 구성

동적 데이터: 주기적 재구성 필요

- 시간 비용 높음 ( $O(n^2)$ )
- 근사 기법 필요

해시 함수의 저장:

k개 해시 함수 저장:

- 스펙트럼 해싱: k개 벡터 (고유벡터)
- 비트맵: 각 함수당 n비트
- 메모리:  $O(k \times n)$  비트

캐시 최적화:

이웃 쌍들이 자주 함께 접근:

- 캐시 미스 감소
- 메모리 대역폭 효율성 향상
- 전체 검색 속도 15~30% 향상

## 43.10 동적 NSH 구성

증분 업데이트:

새로운 점  $x$  추가:

1. 기존 이웃 그래프에서  $x$ 의  $k$ 개 이웃 찾기
2. 기존 점들의 이웃도 잠재적으로 변경
3. 영향받는 해시 함수 부분 재학습

전체 재구성보다 빠름:

시간:  $O(k \times n)$  vs  $O(n^2)$

## 43.11 필터와 이웃 구조의 관계

필터-유도 클러스터링:

벡터 공간의 이웃 관계와

필터 조건의 상관성 분석:

예:

"category=전자제품" 필터

→ 전자제품 벡터들이 서로 이웃일 확률

만약 높은 상관성:

- 필터 조건별 별도 NSH 인덱스 구성
- 각 인덱스가 더 효율적

낮은 상관성:

- 필터와 무관하게 전체 NSH 사용
- 필터링은 별도로 수행

## 43.12 NSH의 실제 성능 (논문 기준)

데이터: GIST, SIFT 벡터들 (백만 개)

회수율 (Recall):

NSH: 97~99%

LSH: 85~90%

HNSW: 98~99%

쿼리당 시간:

NSH: 2~5ms

LSH: 3~8ms

HNSW: 0.5~2ms

메모리:

NSH: 2GB

LSH: 2.5GB

HNSW: 3GB

---

## 추가 제기 문제

1. **필터 조건하에서의 이웃 구조**: 특정 필터 조건을 만족하는 벡터들의 이웃 구조가 전체 벡터의 이웃 구조와 어떻게 다를까?
  2. **다중 필터와 NSH**: 여러 속성에 대한 필터가 있을 때, 각 필터 조합에 대해 별도의 NSH를 유지할 경우의 메모리 비용은?
  3. **이웃 그래프의 동적 업데이트**: 필터 조건이 자주 변할 때, 이웃 그래프를 효율적으로 유지할 수 있을까?
  4. **필터 최적 해시 함수**: NSH 학습 시 필터 조건 정보를 미리 제공하면, 필터 친화적인 해시 함수를 설계할 수 있을까?
  5. **필터 선택도와 이웃 구조의 희소성**: 필터 선택도가 낮을 때(적은 벡터 선택), 이웃 구조의 희소성으로 인한 NSH 성능 저하를 어떻게 보완할 것인가?
  6. **계층화된 필터와 이웃**: 필터 조건들의 계층(예: category > subcategory > item\_type)을 이용하여 이웃 구조를 더 효율적으로 활용할 수 있을까?
  7. **실시간 필터 변경**: 사용자가 쿼리 중 필터 조건을 변경할 때, 사전 학습된 NSH를 조정하거나 캐싱하는 전략은?
  8. **필터 친화적 이웃 정의**: 거리 기반이 아닌 필터 조건 기반의 이웃을 정의하여 NSH를 구성할 수 있을까?
- 

## [44] Scalable nearest neighbor algorithms for high dimensional data

**저자**: M. Muja and D. G. Lowe **학술지**: IEEE TPAMI, vol. 36, no. 11, pp. 2227-2240, 2014 **주제**: FLANN (Fast Library for Approximate Nearest Neighbors) - 자동 알고리즘 선택

---

## 요약

본 논문은 고차원 데이터에서의 확장 가능한 근접 이웃 검색을 위한 FLANN(Fast Library for Approximate Nearest Neighbors) 라이브러리를 제시한다. FLANN의 핵심 기여는 주어진 데이터셋과 성능 요구사항에 따라 최적의 ANN 알고리즘을 자동으로 선택하는 메커니즘이다.

다양한 ANN 기법(KD-트리, LSH, 랜덤 포레스트 등)을 통합하고, 메타-학습을 통해 데이터 특성에 가장 적합한 알고리즘과 파라미터를 자동으로 결정한다.

본 논문과의 관계: Exqutor가 다양한 필터 조건과 데이터 분포에 대응하기 위해, FLANN의 자동 알고리즘 선택 개념을 적용하면 동적으로 최적의 검색 전략을 선택할 수 있다.

---

## 상세분석

### 44.1 고차원 ANN의 도전과 동기

고차원에서의 문제:

트리 기반 인덱스 (KD-트리, R-트리):

- 차원의 저주로 인한 성능 저하
- 높은 차원:  $O(n)$  에 가까운 성능

LSH:

- 파라미터( $k$ ,  $L$ ) 선택 어려움
- 데이터 분포에 따라 성능 큰 변동

HNSW:

- 높은 메모리 요구
- 파라미터 튜닝 필요

→ 실제로는 알고리즘과 파라미터를 올바르게 선택하기 어려움

FLANN의 목표:

1. 다양한 ANN 알고리즘 통합
2. 데이터의 특성 분석
3. 자동으로 최적 알고리즘 선택
4. 파라미터 자동 설정

### 44.2 지원하는 ANN 알고리즘들

선형 공간 인덱싱:

KD-트리 (KD-Tree):

- 저차원( $< 20$ 차원) 최적
- 균형잡힌 분할 전략
- 검색:  $O(\log n) \sim O(n)$

Composite Tree:

- 여러 KD-트리를 병렬로 구성
- 각 트리는 무작위 피벗 선택
- 여러 트리의 결과 병합
- 높은 차원에서 개선된 성능

계층적 k-means:

알고리즘:

1. k-means로 데이터를 K개 클러스터로 분할
2. 각 클러스터 내에서 다시 k-means 적용
3. 계층적 트리 구조 생성

장점:

- 고차원에서 안정적
- 메모리 효율적
- 유연한 성능 튜닝

검색:

1. 루트의 K개 중심과 거리 계산
2. 가장 가까운 중심을 따라 하강
3. 필요시 여러 경로 탐색

### Locality-Sensitive Hashing (LSH):

다양한 LSH 변형 지원:

- Random Projection (L2 거리)
- p-stable 분포 (다양한  $L_p$  거리)
- 테이블 개수 자동 선택

### Random Forest (랜덤 포레스트):

각 결정 트리:

- 각 노드에서 무작위 차원 선택
- 임계값 선택으로 분할
- 균형 잡힌 분할 유도

숲:

- 여러 트리를 병렬로 구성
- 모든 트리 탐색으로 후보 수집
- 거리 정렬 및 상위 K개 반환

## 44.3 알고리즘 선택 메커니즘

자동 벤치마킹:

단계 1: 후보 알고리즘 식별

데이터 차원, 크기 등에 따라

적절할 것 같은 여러 알고리즘 선택

단계 2: 각 알고리즘별 파라미터 공간 탐색

- 예: KD-트리 트리 수, LSH 테이블 수

- 작은 데이터셋에서 학습

단계 3: 성능 메트릭 계산

- 검색 속도 (ms/쿼리)

- 정확도 (회수율)

- 메모리 사용량

단계 4: 최적 알고리즘 선택

- 목표(속도 vs 정확도)에 따라 선택

- 파라미터도 함께 반환

#### 파라미터 자동 설정:

예: LSH 파라미터  $k$ ,  $L$  선택

목표: 원하는 회수율 달성하면서 최소 검색 시간

과정:

1. 다양한 ( $k$ ,  $L$ ) 조합 시도

2. 각 조합의 성능 측정

3. 회수율 요구사항 만족하는 조합 중  
가장 빠른 조합 선택

$k$ ,  $L$  관계:

-  $k$  증가: 정확도 높음, 속도 낮음

-  $L$  증가: 회수율 높음, 메모리 증가

## 44.4 데이터 특성 분석

### 차원성(Dimensionality) 추정:



효과적 차원(Intrinsic Dimensionality):

- 선형 차원(명시적): 1000차원 벡터
- 실제 차원(내재적): 때로는 100차원 정도만 필요

추정 방법:

1. 무작위 점 쌍의 거리 분포 분석
2. 가우시안 과정으로 모델링
3. 추정된 차원으로 알고리즘 선택

영향:

- 낮은 내재 차원: KD-트리 유리
- 높은 내재 차원: LSH 또는 k-means

데이터 분포 특성:

클러스터링:

- 데이터가 명확한 클러스터를 형성하는가?
- k-means 기반 방법에 유리

균일성:

- 벡터들이 공간에 균일하게 분포?
- LSH에 유리

축 정렬:

- 축 정렬 분할이 효과적인가?
- KD-트리에 유리

## 44.5 FLANN의 아키텍처

인덱스 구조:

Index 기본 클래스:

- build\_index(): 인덱스 구성
- knn\_search(): K-MN 검색
- set\_params(): 파라미터 설정

Index 구체 구현:

- LinearIndex (순차 탐색)
- KDTreeIndex
- LSHIndex
- KMeansIndex
- CompositeIndex
- AutoIndex (자동 선택)

AutoIndex 구현:

```

class AutoIndex:
 def build_index(self, data, target_precision):
 # 1. 데이터 특성 분석
 intrinsic_dim = analyze_dimensionality(data)
 clustering_score = analyze_clustering(data)

 # 2. 후보 알고리즘 선택
 candidates = select_candidates(
 intrinsic_dim, data.shape[0], clustering_score
)

 # 3. 각 후보별 벤치마킹
 results = {}
 for algorithm in candidates:
 best_params = tune_parameters(
 algorithm, data, target_precision
)
 index = create_index(algorithm, best_params)
 index.build(data)

 perf = benchmark(index, data)
 results[algorithm] = (index, perf)

 # 4. 최적 알고리즘 선택
 best_algo = select_best(results, target_precision)
 self.best_index = best_algo

 def knn_search(self, query, K):
 return self.best_index.knn_search(query, K)

```

## 44.6 성능 메트릭과 최적화

### 목표 함수:

기본 목표:

회수율 (Recall)  $\geq R_{\text{target}}$

→ 최소 검색 시간

시간-정확도 트레이드오프:

Recall vs 쿼리 시간의 파레토 프론티어 구성

사용자 요구사항에 따라 선택

### 벤치마킹 전략:

전체 데이터에서의 완전 벤치마킹은 비용 높음:  
→ 일반적으로 전체 데이터의 일부(10~20%)로 학습

절차:

1. 전체 데이터 N개 중 샘플 S개 선택 ( $S \ll N$ )
2. 샘플에서 100개 쿼리로 벤치마킹
3. 성능 메트릭 기준으로 알고리즘/파라미터 선택
4. 이를 전체 데이터에 적용

## 44.7 필터링과 FLANN의 통합

필터-인식 알고리즘 선택:

기존 FLANN:

알고리즘 선택 =  $f(\text{데이터 차원, 크기, 분포})$

필터 통합 FLANN:

알고리즘 선택 =  $f(\text{데이터 차원, 크기, 분포, 필터 선택도, } \leftarrow \text{추가, 필터 조건 복잡도 } \leftarrow \text{추가})$

필터 선택도 영향:

- 높은 선택도 (99%): 기존 알고리즘 유지
- 낮은 선택도 (10%): 광범위 검색 알고리즘 선택
- 매우 낮은 선택도 (1%): 순차 탐색 고려

동적 알고리즘 전환:

런타임 중 필터 조건이 변할 때:

모니터링:

- 각 쿼리의 필터 선택도 측정
- 평균 선택도 추적

적응:

- 선택도가 크게 변하면 알고리즘 전환
- 새 인덱스 구성은 백그라운드에서

예:

- 요청 1-100: 선택도 90% → KD-트리
- 요청 101-110: 선택도 5% → LSH로 전환
- 요청 111+: LSH 사용 (더 효율적)

## 44.8 FLANN의 실제 성능

### 알고리즘 선택 결과 (논문 기준):

SIFT 백만 개 벡터 (128차원):

선택: Composite KD-트리

성능: 0.8ms/쿼리, 98% 정확도

ImageNet 특성 (2M 벡터, 4096차원):

선택: Hierarchical k-means + LSH 하이브리드

성능: 2.5ms/쿼리, 95% 정확도

MNIST 데이터 (784차원):

선택: Random Forest

성능: 0.3ms/쿼리, 97% 정확도

### 메모리 효율성:

벡터 데이터: 기본 비용

인덱스 오버헤드:

KD-트리: 30~40%

LSH: 50~100% (여러 테이블)

k-means: 10~20%

Random Forest: 20~30%

FLANN은 자동으로 메모리 효율적인 알고리즘 선택

## 44.9 높은 차원에서의 특수성

### 차원 특성과 알고리즘:

2000+ 차원 (매우 고차원):

- KD-트리: 거의 사용되지 않음
- LSH 우세
- k-means도 가능

1000~2000 차원 (고차원):

- Composite 트리가 여전히 경쟁력
- LSH와 혼합

500~1000 차원 (중간 차원):

- 다양한 알고리즘 경쟁
- 데이터 분포에 따라 결정

<500 차원 (저차원):

- KD-트리 일반적으로 최적

## 44.10 실무 적용과 고려사항

### 프로덕션 환경:

#### 학습 단계 (오프라인):

1. 샘플 데이터로 알고리즘 벤치마킹
2. 최적 알고리즘과 파라미터 결정
3. 전체 데이터로 인덱스 구성

#### 운영 단계 (온라인):

- 결정된 인덱스 사용
- 모니터링으로 성능 추적
- 필요시 재벤치마킹

### 파라미터 유지:

한 번 선택된 알고리즘의 파라미터는  
데이터 분포가 크게 변하지 않는 한 유지

#### 동적 데이터의 경우:

- 주기적 재벤치마킹 필요
- 변화하는 특성에 따라 알고리즘 전환

## 44.11 FLANN과 Exqutor의 시너지

### 필터 기반 동적 선택:

#### Exqutor에서 FLANN 개념 적용:

#### 필터 조건이 주어졌을 때:

1. 필터 선택도 추정
2. 필터된 데이터의 특성 분석
3. 최적 검색 전략 선택:
  - 필터-우선 (선택도 높음)
  - KNN-우선 (선택도 낮음)
  - 하이브리드 (중간)

#### 동적 필터 변경:

- 선택도 모니터링
- 필요시 전략 전환

## 44.12 메타-학습과 적응

### 학습된 함수:

```
최적_알고리즘 = f(
 차원,
 데이터_크기,
 클러스터_점수,
 필터_선택도,
 필터_조건_복잡도
)
```

이 함수는:

- 다양한 데이터로 학습
- 새로운 데이터에 적용
- 온라인 피드백으로 개선 가능

## 추가 제기 문제

1. **필터와 차원성의 상호작용**: 필터 조건을 적용했을 때 데이터의 내재적 차원(intrinsic dimensionality)이 어떻게 변할까? 필터된 데이터의 차원을 분석하면 더 나은 알고리즘 선택이 가능할까?
2. **필터 선택도 예측**: 쿼리 전에 필터 조건의 선택도를 정확하게 예측할 수 있을까? 통계 모델을 사용한 예측이 얼마나 정확할까?
3. **다중 필터의 알고리즘 선택**: 여러 필터 조건이 있을 때, 각 필터 조합에 대해 별도의 최적 알고리즘을 유지할 경우의 메타 선택 문제는?
4. **캐싱과 알고리즘 선택**: 자주 요청되는 필터 조건에 대해 사전 구성된 인덱스를 캐싱하면, FLANN의 자동 선택과 어떻게 상호작용할까?
5. **온라인 학습**: 시간이 지나면서 새로운 쿼리 패턴이 나타날 때, FLANN의 선택 함수를 동적으로 업데이트할 수 있을까?
6. **필터 조건의 복잡도 측정**: 복합 필터(AND/OR)의 복잡도를 정량화하면, 이를 바탕으로 더 정교한 알고리즘 선택이 가능할까?
7. **메모리 제약과 필터**: 메모리가 제한된 환경에서 필터 조건을 고려하여 가장 메모리 효율적인 알고리즘을 선택하려면?
8. **분산 환경에서의 FLANN**: 여러 노드에서 필터링된 검색을 수행할 때, 각 노드별로 다른 알고리즘을 선택하는 것이 효과적일까?

## [45] VHP: Approximate nearest neighbor search via virtual hypersphere partitioning

**저자:** K. Lu, H. Wang, W. Wang, and M. Kudo **학술지:** Proc. VLDB Endow., vol. 13, no. 9, p. 1443-1455, 2020 **주제:** VHP(Virtual Hypersphere Partitioning) - 공간 분할 기반 ANN

### 요약

본 논문은 가상 초구 분할(Virtual Hypersphere Partitioning, VHP)을 기반으로 하는 새로운 ANN 알고리즘을 제시한다. VHP는 고차원 벡터 공간을 가상의 초구(hyperspheres)로 분할하여 근접 이웃 검색을 수행한다.

VHP의 핵심은 중심점(centroid)로부터의 거리와 각도를 동시에 고려하는 공간 분할이다. 이를 통해 고차원에서도 효율적인 가지치기(pruning)가 가능하며, HNSW와 비슷하거나 우수한 성능을 달성한다.

본 논문과의 관계: Exqutor의 필터링 기반 검색에서 공간 분할 구조를 활용하면, 같은 분할 영역 내의 필터 조건을 만족하는 벡터들을 빠르게 찾을 수 있다.

### 상세분석

#### 45.1 공간 분할 기반 ANN의 필요성

기존 방법의 한계:

KD-트리 (축 정렬 분할):

- 고차원에서 성능 저하 (차원의 저주)
- 각도 정보 활용 부족

LSH (확률적 해싱):

- 높은 거짓양성율
- 파라미터 조정 복잡

HNSW (그래프 기반):

- 높은 메모리 오버헤드
- 삽입/삭제 비효율적

VHP의 목표:

- 고차원에서의 효율적 분할
- 메모리 효율성
- 삽입/삭제 지원

## 45.2 초구 분할의 개념

### 초구(Hypersphere)의 정의:

고차원 공간에서 중심점  $c$ 로부터 거리  $r$ 인 모든 점들의 집합

2D: 원 (circle)

$$\{p \mid d(p, c) = r\}$$

3D: 구 (sphere)

$$\{p \mid \|p - c\| = r\}$$

고차원: 초구 (hypersphere)

$$\{p \mid \|p - c\| = r\}$$

### 가상 초구 분할(VHP):

개념:

하나의 중심점과 여러 반지름으로 초구들을 생성  
→ 동심원 구조 (concentric spheres)

예시:

중심:  $c$

초구 1: 반지름  $r_1$  내의 점들

초구 2:  $r_1 < \text{거리} \leq r_2$  영역의 점들

초구 3:  $r_2 < \text{거리} \leq r_3$  영역의 점들

...

장점:

- 중심에 가까운 점일수록 더 세밀한 분할
- 각도 정보와 거리 정보 동시 활용

## 45.3 VHP 알고리즘의 구조

### 계층적 VHP 구성:



루트 중심: 전체 데이터의 중심

↓

중심에 가장 가까운  $N_1$ 개:

- 중심점으로 지정
- 각 중심별로 초구 분할

↓

각 초구별로 다시 분할:

- 부분 중심점 선택
- 해당 영역을 초구로 분할

↓

리프 노드: 실제 벡터들

#### 분할 전략:

단계 1: 전역 중심 선택

- 데이터의 무게 중심(centroid)
- 또는 대표점(representative point)

단계 2: 각 계층에서 중심점 선택

- 이전 중심으로부터 거리 기준 선택
- 상호 거리 최대화 (분산 최대화)

단계 3: 거리 기반 분할

- 각 중심별로 가까운 점들을 그룹화
- 그룹 내에서 다시 중심 선택

#### 45.4 구체적인 VHP 구성 알고리즘

VHP 인덱스 구성:

```

procedure BUILD_VHP_INDEX(vectors, max_partition_size):
 root = create_node()

 function build_recursive(node, points, depth):
 if len(points) <= max_partition_size:
 // 리프 노드: 벡터들 직접 저장
 node.vectors = points
 return

 // 내부 노드: 중심 선택 및 분할
 centroid = compute_centroid(points)
 node.centroid = centroid

 // 각 점과 중심의 거리 계산
 distances = [distance(p, centroid) for p in points]

 // 거리로 정렬
 sorted_indices = argsort(distances)

 // 여러 그룹으로 분할
 num_children = ceil(len(points) / max_partition_size)
 group_size = ceil(len(points) / num_children)

 for group_idx in range(num_children):
 start = group_idx * group_size
 end = min((group_idx + 1) * group_size, len(points))

 child = create_node()
 child.parent = node
 node.children.append(child)

 // 재귀적으로 자식 노드 구성
 build_recursive(child, points[start:end], depth + 1)

 build_recursive(root, vectors, 0)
 return root

```

**검색 알고리즘:**

```

procedure VHP_SEARCH(query_q, K, node=root):
 candidates = []

 // BFS 또는 DFS로 트리 탐색
 queue = [root]
 visited = set()

 while queue is not empty:
 current = queue.pop(0)

 // 현재 노드의 중심과 쿼리 거리
 centroid_dist = distance(q, current.centroid)

 // 가지치기: 이 노드가 결과를 포함할 수 없다면 스킵
 if should_prune(current, centroid_dist, candidates):
 continue

 // 리프 노드: 모든 벡터와 거리 계산
 if is_leaf(current):
 for v in current.vectors:
 dist = distance(q, v)
 candidates.append((v, dist))

 // 내부 노드: 자식 노드 방문
 else:
 for child in current.children:
 if child not in visited:
 visited.add(child)
 queue.append(child)

 // 거리로 정렬하여 가장 가까운 K개 반환
 return sorted(candidates, key=lambda x: x[1])[:K]

```

## 45.5 가지치기(Pruning) 메커니즘

기본 가지치기:

현재까지 찾은 가장 먼 이웃까지의 거리:  $d_{\max}$

노드의 중심:  $c$

쿼리:  $q$

중심까지의 거리:  $d_c$

가지치기 조건:

노드 내 모든 점이  $d_{\max}$ 보다 멀다면

해당 노드는 방문하지 않음

수학적:

$d_c - \text{radius} \geq d_{\max}$

→ 노드 내 어떤 점도  $K$ 개 결과에 포함될 수 없음

### 각도 기반 가지치기:

두 벡터의 각도 관계 활용:

벡터  $q$ ,  $p$ ,  $c$ 에 대해:

$\theta = \text{angle}(q - c, p - c)$

코사인 법칙:

$d(q, p)^2 = d_q^2 + d_p^2 - 2*d_q*d_p*\cos(\theta)$

가지치기:

예상 최소 거리  $> d_{\max}$ 이면 스킵

### 동적 범위 조정:

검색 진행에 따라  $d_{\max}$ 가 줄어들음:

→ 가지치기 범위 확대

→ 검색 시간 단축

적응형 검색:

처음: 넓은 범위 탐색

중간: 좋은 후보 찾으면서 범위 축소

말미: 좁은 범위로 정교한 검색

## 45.6 계층 구조의 최적화

### 트리 균형:

균형 잡힌 분할:

- 각 레벨의 노드 수가 유사
- 최대 깊이  $O(\log n)$
- 검색 시간  $O(\log n)$

불균형 분할:

- 큰 클러스터는 깊게 분할
- 작은 클러스터는 얇게 분할
- 데이터 분포 반영

노드당 최대 크기:

max\_partition\_size의 영향:

작은 크기 (10~100):

- 깊은 트리
- 리프 노드 많음
- 가지치기 효과 좋음
- 메모리 사용 증가

큰 크기 (1000~10000):

- 얇은 트리
- 리프 노드 적음
- 리프 내 선형 탐색 증가
- 메모리 절약

최적값: 보통 100~500

## 45.7 메모리 효율성

메모리 구성:

벡터 데이터:  $N * d * 4$  바이트

중심점 저장: 노드\_수 \*  $d * 4$  바이트

트리 구조: 노드\_수 \* (포인터 + 메타데이터)

예 (1M 벡터, 1000차원):

벡터:  $1M * 1000 * 4 = 4GB$

중심: 약  $10K * 1000 * 4 = 40MB$

트리: 약  $10K * 100 = 1MB$

총: 약 4.05GB (벡터 대비 1% 오버헤드)

vs HNSW:  $4 + 4 * 1M/16 = 4.25GB$  (약 6% 오버헤드)

vs NSG: 유사 수준

## 45.8 VHP vs HNSW vs NSG

지표	VHP	HNSW	NSG
구성 시간	빠름	중간	낮음
메모리	매우 효율	중간	효율
검색 속도	빠름	매우 빠름	빠름
정확도	우수	우수	우수
삽입/삭제	효율적	어려움	어려움
파라미터	간단	간단	중간

## 45.9 필터링과의 통합

### 필터-인식 VHP:

VHP 구성 시 필터 조건 고려:

방법 1: 필터별 별도 VHP

- 각 필터 그룹별로 독립적 VHP
- 높은 메모리 비용
- 매우 빠른 검색

방법 2: 통합 VHP + 필터 검사

- 하나의 VHP 사용
- 트리 탐색 시 필터 조건 확인
- 필터 미충족 노드 스킵
- 메모리 효율적

### 필터-기반 가지치기 강화:

필터 조건을 고려한 가지치기:

노드 내 모든 벡터가 필터를 만족하지 않으면:

- 해당 노드 전체 스킵

부분적으로 만족:

- 리프 노드까지 내려가서 개별 확인

이를 통해:

- 필터 선택도 높음: 검색 범위 감소
- 필터 선택도 낮음: 광범위 탐색 필요

## 45.10 동적 데이터 처리

### 삽입(Insertion):

새로운 벡터  $x$ 를 인덱스에 추가:

1. 루트부터 시작하여 리프 노드 찾기
  - 각 레벨에서 가장 가까운 자식 선택
2. 리프 노드 도달:
  - 벡터를 리프에 추가
  - 리프 크기가 `max_partition_size` 초과?
    - 리프 분할 (split)
3. 분할 후:
  - 부모 노드의 중심점 재계산
  - 필요시 상향식으로 재균형

시간 복잡도:  $O(\log n)$

### 삭제(Deletion):

벡터  $x$ 를 제거:

1. 벡터가 있는 리프 노드 찾기
2. 벡터 제거:
  - 리프에서 직접 제거
  - 리프가 비어있으면 병합(merge) 고려
3. 재균형:
  - 중심점 재계산
  - 크기 요구사항 확인

시간 복잡도:  $O(\log n)$

## 45.11 실제 성능 (논문 기준)

### 성능 지표 (1M SIFT 벡터, 128차원):

구성 시간:

VHP: 20초

HNSW: 300초

NSG: 30초

→ VHP가 가장 빠름

메모리:

VHP: 4.0GB

HNSW: 4.2GB

NSG: 4.05GB

검색 속도 (K=10):

VHP: 1.2ms

HNSW: 0.8ms

NSG: 1.1ms

정확도:

VHP: 96%

HNSW: 97%

NSG: 96%

## 45.12 VHP의 특징과 장단점

장점:

1. 빠른 구성
  - 트리 구축이 간단
  - 병렬화 가능
2. 메모리 효율성
  - HNSW보다 적은 오버헤드
  - 공간 분할 구조의 이점
3. 동적 작업 지원
  - 삽입/삭제  $O(\log n)$
  - 트리 재구성 필요 없음
4. 파라미터 단순성
  - max\_partition\_size만 주요 파라미터
  - 자동 조정 가능

단점:



1. 검색 속도
  - HNSW보다 약 30~50% 느림
  - 그래프 기반의 효율성 미흡
2. 고차원에서의 변동성
  - 매우 고차원(10K+)에서 성능 저하
  - 공간 분할의 효율성 감소
3. 클러스터링 의존성
  - 데이터가 잘 클러스터되지 않으면 성능 저하
  - 균일 분포에 덜 유리

#### 45.13 필터링과 VHP의 협력

##### 효과적인 필터 활용:

VHP의 계층적 구조는 필터링과 잘 맞음:

상위 레벨에서:

- 전체 부분집합의 필터 조건 만족도 파악
- 분명히 필터 미충족인 노드 조기 제거

중간 레벨에서:

- 부분적 일치하는 노드 처리
- 선택적 하강

리프 레벨에서:

- 최종 필터 검사
- 거리 계산

이를 통해 필터 선택도에 따른 성능 최적화

#### 추가 제기 문제

1. **초구 분할과 필터 조건의 상관성:** 특정 필터 조건을 만족하는 벡터들이 초구 분할상에서 어떻게 분포할까? 이를 예측하면 최적의 분할 크기를 결정할 수 있을까?
2. **필터 기반 중심점 선택:** 필터 조건을 만족하는 데이터만 고려하여 중심점을 선택하면, 더 효율적인 VHP를 구성할 수 있을까?
3. **동적 필터 변경에서의 VHP:** 필터 조건이 자주 변할 때, VHP의 삽입/삭제 효율성이 어떤 이점을 제공할까?
4. **다중 필터와 VHP의 계층:** 여러 필터 조건에 대해 각 조건 만족도를 VHP의 계층 구조에 어떻게 인코딩할 것인가?

5. **필터 선택도와 가지치기**: 필터 선택도가 변함에 따라 VHP의 가지치기 효율성이 어떻게 달라질까?
6. **메모리 절약**: 필터를 통해 실제로 접근할 벡터가 줄어들 때, VHP의 메모리를 동적으로 압축할 수 있을까?
7. **클러스터 진화성**: VHP가 클러스터 구조를 가정하는데, 필터 조건이 데이터의 클러스터링을 강화할까, 약화할까?
8. **실시간 업데이트**: 새로운 벡터가 계속 추가될 때, VHP의 트리 구조가 어떻게 진화하는가? 필터 조건을 고려한 적응적 구조 변경이 가능할까?
9. **공간 지역성과 필터**: 필터를 만족하는 벡터들을 메모리상에 인접하게 배치하면, VHP 검색의 캐시 성능을 향상시킬 수 있을까?
10. **하이브리드 구조**: VHP의 하위 계층에 HNSW를 사용하는 하이브리드 구조의 성능은? 즉, 초기 분할로 후보를 선택한 후, 각 후보 그룹에서 HNSW로 최종 검색?

---

## [46] Annoy: Approximate Nearest Neighbors in C++/Python

### 요약

Annoy는 Spotify에서 개발한 대규모 ANN(Approximate Nearest Neighbor) 검색을 위한 라이브러리로, 2013년에 발표되었다. C++로 구현되었으며 Python 바인딩을 제공하므로 다양한 환경에서 사용할 수 있다. 핵심 알고리즘은 Random Projection Trees를 기반으로 하며, 고차원 벡터 데이터에서 빠른 근사 최근접 이웃 검색을 가능하게 한다.

Annoy의 주요 특징은 메모리 효율성과 빠른 쿼리 속도이다. Random projection tree 구조를 이용하여 고차원 공간을 재귀적으로 분할하고, 이진 파일 형태로 인덱스를 저장할 수 있다. 메모리 매핑을 지원하므로 매우 큰 데이터셋도 효율적으로 처리할 수 있으며, 실시간 검색 애플리케이션에서 광범위하게 사용되었다.

Annoy의 Random Projection Trees는 고차원 벡터를 임의로 선택된 두 점을 지나는 하이퍼플레인으로 분할하는 방식으로 작동한다. 이 접근방식은 구현이 간단하면서도 효과적이며, 실무 환경에서 높은 성능을 보여준다. 검색 과정에서는 리프 노드 근처의 후보들을 탐색하고, 필요시 추가 트리를 검사하여 정확도를 조정할 수 있다.

메모리 효율성 측면에서 Annoy는 각 벡터를 부동소수점으로 저장하며, 인덱스 구조도 매우 컴팩트하다. 이는 모바일 기기나 엣지 디바이스에서도 대규모 벡터 데이터를 처리할 수 있게 한다. 특히 음악 추천 시스템에서 백만 개 이상의 곡을 실시간으로 검색해야 하는 Spotify의 요구사항을 충족하기 위해 설계되었다.

Exqutor와의 관계를 살펴보면, Annoy는 유사성 검색 인덱싱 기술의 발전을 보여주는 중요한 작업이다. Exqutor는 유사성 쿼리의 카디널리티 추정을 다루는데, 이는 ANN 검색 결과에 기반한 시스템에서 쿼리 최적화를 위해 중요하다. 따라서 Annoy와 같은 효율적인 ANN 구조를 이해하는 것은 Exqutor의 카디널리티 추정 모델이 실제 검색 시스템에서 어떻게 적용되는지 이해하는 데 필수적이다.

---

## 상세분석

### 46.1 Random Projection Trees의 구조

Random Projection Trees (RP-Trees)는 Annoy의 핵심 자료구조이다. 이 구조는 고차원 벡터 공간을 재귀적으로 분할하는 방식으로 작동한다. 구체적으로, 각 내부 노드에서는 임의로 선택된 두 점을 지나는 하이퍼플레인을 사용하여 벡터들을 두 그룹으로 나눈다.

하이퍼플레인의 선택 방식은 다음과 같다. 현재 노드에 할당된 벡터 집합에서 임의로 두 점  $p_1$ 과  $p_2$ 를 선택하고, 이 두 점의 중점을 지나면서  $p_2 - p_1$  벡터에 수직인 하이퍼플레인을 정의한다. 이 하이퍼플레인을 기준으로 각 벡터를 분류하여 왼쪽 또는 오른쪽 자식 노드로 할당한다.

이러한 방식의 장점은 계산의 간단성과 효율성에 있다. 하이퍼플레인 정의에 필요한 계산량이 적으므로, 대규모 데이터셋에 대해서도 빠르게 인덱스를 구축할 수 있다. 또한 임의성으로 인해 자동적으로 어느 정도 로드 밸런싱이 이루어지므로, 트리가 한쪽으로 치우치지 않도록 한다.

리프 노드에 도달하면 해당 노드의 모든 벡터들이 저장된다. 리프 노드의 크기는 하이퍼파라미터로 조정할 수 있으며, 이는 메모리 사용량과 쿼리 속도 간의 트레이드오프를 제어한다.

### 46.2 검색 알고리즘과 근사 특성

Annoy의 검색 프로세스는 쿼리 벡터  $q$ 가 주어졌을 때, RP-Tree를 따라 루트에서 리프까지 내려가는 과정으로 시작된다. 각 내부 노드에서  $q$ 와 하이퍼플레인의 위치 관계를 판단하여 적절한 자식 노드를 선택한다.

리프 노드에 도달한 후, 해당 리프 내의 모든 벡터들과  $q$  사이의 거리를 계산하고, 상위  $k$ 개를 후보로 선택한다. 기본적인 검색은 여기서 종료되지만, 더 높은 정확도를 원할 경우 추가 탐색이 가능하다.

추가 탐색 과정에서는 현재 리프 노드의 형제 노드나 근처의 다른 리프 노드들도 검사한다. Annoy에서는 `search_k` 라는 파라미터를 통해 검사할 후보의 개수를 제어할 수 있다. `search_k` 값이 크면 정확도가 높아지지만 쿼리 속도는 느려진다.

근사 특성은 확률론적 보장이 아니라 경험적 성능에 기반한다. 실제로 많은 응용 분야에서 95% 이상의 정확도를 달성하면서도 정확한 브루트포스 탐색보다 10배 이상 빠른 성능을 제공한다.

### 46.3 메모리 효율성과 파일 저장

Annoy의 또 다른 핵심 특징은 메모리 효율성이다. 인덱스를 바이너리 파일로 저장할 때, 각 벡터는 실수 배열로 저장되며, 트리 구조는 매우 간결하게 인코딩된다. 일반적으로 각 차원당 4바이트(float32) 또는 8바이트(float64)만 사용한다.

메모리 매핑(Memory Mapping) 기능을 지원하므로, 디스크의 인덱스 파일을 직접 메모리에 매핑하여 사용할 수 있다. 이는 매우 큰 데이터셋의 경우 메인 메모리의 크기에 제약받지 않고도 검색을 수행할 수 있다는 의미이다.

인덱스 파일의 구조는 헤더 정보(벡터 차원수, 트리 개수 등)와 인덱스 데이터로 구성된다. 트리 구조는 배열 기반의 이진 트리 표현을 사용하여, 포인터 오버헤드를 최소화한다.

## 46.4 하이퍼파라미터 조정

Annoy의 성능은 여러 하이퍼파라미터에 의해 영향을 받는다. 첫째, `n_trees` 는 구축할 RP-Tree의 개수이다. 더 많은 트리를 사용하면 정확도가 높아지지만 메모리 사용량과 인덱싱 시간이 증가한다.

둘째, `search_k` 는 검색 시 탐색할 후보 벡터의 최대 개수를 제어한다. `search_k` 값이 크면 정확도가 높아지지만 쿼리 응답 시간이 길어진다. 보통 `search_k > n_trees * 트리의 리프 노드 개수` 범위에서 조정된다.

셋째, `leaf_size` 는 리프 노드에 저장할 최대 벡터 개수를 제어한다. 작은 값은 더 정확한 검색을 제공하지만 더 많은 메모리를 사용한다.

## 46.5 실무 적용 및 성능

Spotify는 음악 추천 시스템에서 Annoy를 광범위하게 사용했다. 수백만 개의 곡을 벡터로 표현하고, 사용자가 현재 듣고 있는 곡과 유사한 곡들을 실시간으로 검색해야 한다. Annoy는 이러한 요구사항을 효과적으로 충족한다.

성능 측면에서, Annoy는 일반적으로 다음과 같은 특성을 보여준다. 인덱싱 속도는 초당 백만 개 벡터 정도의 속도로 빠르며, 쿼리 응답 시간은 밀리초 단위의 초저지연(ultra-low latency)을 제공한다. 메모리 사용량은 벡터 데이터 자체의 크기에 가깝다.

### 추가 제기 문제

- 정확도 보장의 부재:** Annoy는 근사 알고리즘이므로 항상 실제 최근접 이웃을 찾는다는 보장이 없다. 애플리케이션에서 이러한 근사성이 허용 가능한 수준인지 검증이 필요하다.
- 벡터 추가의 어려움:** Annoy는 정적 인덱스이므로, 새로운 벡터를 추가하려면 전체 인덱스를 재구축해야 한다. 동적 데이터셋에는 부적합할 수 있다.
- 고차원에서의 성능 저하:** Random projection은 고차원에서 curse of dimensionality를 완전히 극복하지는 못한다. 벡터의 차원이 매우 높으면 성능이 저하될 수 있다.
- 카디널리티 추정과의 통합:** Annoy 같은 ANN 인덱스를 사용하는 시스템에서, 유사성 검색의 결과 크기를 정확하게 예측하려면 별도의 카디널리티 추정 기법이 필요하다. 이것이 Exqutor가 다루는 핵심 문제이다.

## [47] Product Quantization for Nearest Neighbor Search

### 요약

본 논문은 Hervé Jégou, Matthijs Douze, Cordelia Schmid에 의해 2011년 IEEE TPAMI에 발표된 매우 영향력 있는 작업이다. Product Quantization (PQ)은 고차원 벡터를 저차원 양자화 코드로 압축하는 기법으로, 대규모 벡터 검색 시스템에서 메모리 효율성과 검색 속도를 획기적으로 개선한다.

PQ의 핵심 아이디어는 고차원 벡터를 여러 개의 저차원 부분공간(subspace)으로 분할하고, 각 부분공간에 대해 독립적으로 양자화를 수행하는 것이다. 예를 들어, 128차원 벡터를 4개의 32차원 부분공간으로 분할하고, 각 부분공간에서 256개의 양자화 중심(centroid)을 학습한다. 그 결과 각 벡터를 4개의 바이트(또는 코드워드)로 표현할 수 있으며, 원본 벡터 대비 32배의 압축률을 달성한다.

검색 과정에서는 쿼리 벡터도 동일하게 양자화되고, 벡터 간의 거리는 양자화된 코드 간의 거리로 근사된다. PQ는 이러한 거리 계산을 매우 효율적으로 수행할 수 있도록 설계되어 있다. 각 부분공간에서 쿼리 벡터와 모든 중심 간의 거리를 미리 계산하여 테이블을 만든 후, 데이터베이스의 각 벡터에 대해서는 테이블 룩업만으로 거리를 계산할 수 있다.

PQ는 다양한 벡터 차원의 데이터셋에 적용 가능하며, 실제로 SIFT(Scale-Invariant Feature Transform) 디스크립터 같은 고정 차원 특징 벡터를 다루는 컴퓨터 비전 애플리케이션에서 광범위하게 사용되었다. 또한 역벡터 인덱싱(inverted vector indexing)과 결합되어 대규모 벡터 검색 시스템의 성능을 크게 향상시켰다.

## 상세분석

### 47.1 Product Quantization의 원리

Product Quantization은 고차원 벡터의 유사성 검색 문제를 효율적으로 해결하기 위해 설계된 기법이다. 기본 아이디어는 'product'라는 이름에서도 알 수 있듯이, 전체 공간을 여러 개의 저차원 부분공간의 곱(product)으로 분해하는 것이다.

구체적인 프로세스는 다음과 같다. 먼저,  $d$ 차원 벡터를  $m$ 개의  $d/m$ 차원 부분벡터로 분할한다. 예를 들어  $d=128$ ,  $m=4$ 인 경우, 각 부분벡터는 32차원이다. 각 부분공간에 대해 독립적인 양자화(quantization)를 수행한다. 이는  $k$ -means 클러스터링을 사용하여 각 부분공간에서  $k$ 개의 클러스터 중심(양자화 중심)을 학습하는 것을 의미한다.

양자화 중심의 개수  $k$ 는 각 부분벡터를 몇 비트로 인코딩할 것인지에 따라 결정된다. 일반적으로  $k=256$  (8비트)을 사용하므로, 전체 벡터는  $m$ 개의 8비트 코드의 조합, 즉  $8m$ 비트로 표현된다. 128차원을 8비트로 압축하면 약 16배 압축이 되며, 원본 벡터 ( $128 \times 4$ 바이트 = 512바이트) 대비 4바이트만으로 표현할 수 있다.

각 부분공간에서의 양자화는 다음과 같이 진행된다. 부분공간  $j$ 에 속한 모든 벡터들의 부분벡터들을 추출하고, 이에 대해  $k$ -means 알고리즘을 실행한다. 결과적으로  $k$ 개의 중심 벡터  $c_{j,0}, c_{j,1}, \dots, c_{j,k-1}$ 이 학습된다. 각 훈련 벡터의 부분벡터는 가장 가까운 중심에 할당되고, 그 중심의 인덱스를 코드로 저장한다.

### 47.2 거리 계산의 효율성

PQ의 가장 중요한 성능 이점은 거리 계산의 효율성에 있다. 전통적인 방식으로는  $n$ 개의  $d$ 차원 벡터와 1개의 쿼리 벡터 간의 거리를 모두 계산하는 데  $O(n \cdot d)$ 의 시간이 필요하다. PQ는 이를 훨씬 더 빠르게 수행할 수 있다.

PQ의 거리 계산 방식은 다음과 같다. 먼저 쿼리 벡터  $q$ 에 대해, 각 부분공간에서  $q$ 의 부분벡터와 모든 양자화 중심 간의 거리를 미리 계산한다. 부분공간  $j$ 에서는  $k$ 개의 거리 값이 생성되므로,  $m$ 개 부분공간에 대해 총  $mk$ 개의 거리를 미리 계산한다. 이는  $O(mkd/m) = O(kd)$  시간에 수행된다.

다음으로, 데이터베이스의 각 벡터  $x$ 에 대해,  $x$ 의 양자화 코드( $code_0, code_1, \dots, code_{m-1}$ )를 사용하여 거리를 계산한다. 부분공간  $j$ 에서  $code_j$ 번째 미리 계산된 거리 값을 조회하면 되므로, 각 벡터당 단  $m$ 번의 테이블 룩업만으로 거리를 계산할 수 있다. 따라서 총 시간은  $O(nm)$ 이 되며, 이는  $O(nd)$ 에 비해 훨씬 빠르다 ( $d \gg k$ 인 경우).

### 47.3 손실과 정확도

PQ는 손실 압축(lossy compression)이므로, 당연히 정보 손실이 발생한다. 그러나 유사성 검색의 맥락에서는 이러한 손실이 심각한 문제가 되지 않을 수 있다. 왜냐하면 우리가 관심 있는 것은 정확한 거리 값이 아니라 상대적인 거리 순서이기 때문이다.

PQ의 정확도는 여러 요인에 의해 영향을 받는다. 첫째, 부분공간의 개수  $m$ 이다.  $m$ 이 작을수록 압축률은 높지만 정확도는 낮다. 일반적으로  $m=4\sim 8$  정도가 좋은 균형을 제공한다.

둘째, 각 부분공간의 양자화 중심 개수  $k$ 이다.  $k$ 가 작을수록 압축률은 높지만 정확도는 낮다.  $k=256$  (8비트)은 많은 응용에서 충분히 높은 정확도를 제공한다.

셋째, 학습 데이터와 쿼리 데이터의 분포의 일치도이다. PQ 모델이 학습한 분포와 실제 쿼리 분포가 다르면 정확도가 저하될 수 있다.

실제 실험에서 PQ는 다음과 같은 특성을 보여준다. SIFT 1M 데이터셋에서 99%의 상대적 정확도(recall@1000)를 달성하면서 압축률은 약 32배를 제공한다. 또한 거리 계산 속도는 원본 벡터 사용 시보다 수십 배 빠르다.

### 47.4 역벡터 인덱싱과의 결합

PQ는 역벡터 인덱싱(inverted vector index, IVF)과 결합되어 더욱 강력한 검색 시스템을 구성할 수 있다. IVF는 벡터를 먼저 대략적으로 클러스터링한 후, 쿼리와 유사한 클러스터만을 검사하는 방식이다.

IVF+PQ 조합에서는 다음 과정이 진행된다. 먼저 모든 벡터를 대략  $k$ 개 클러스터로 분할한다 (보통  $k=100\sim 10000$ ). 그 다음, 각 벡터를 PQ로 압축한다. 검색 시에는 쿼리 벡터가 속한 클러스터의 인근 클러스터들만 검사하고, 각 클러스터 내에서는 PQ 기반의 거리 계산을 사용한다.

이러한 조합은 다음과 같은 장점을 제공한다. 검사 대상이 되는 벡터의 개수를 크게 줄일 수 있으므로, 전체 검색 시간은 크게 단축된다. 또한 메모리 사용량도 PQ로 인해 크게 감소한다. 예를 들어, IVF+PQ 조합으로 1억 개의 벡터를 검색하면서도 메모리 사용량을 GB 단위로 유지할 수 있다.

### 47.5 재분석 및 변형들

PQ 이후로 다양한 변형이 제안되었다. OPQ (Optimized Product Quantization)는 PQ의 부분공간 분할을 더 최적화하는 방법이다. 각 부분공간의 분산을 최소화하도록 벡터를 회전시킨 후 PQ를 적용하면 정확도를 더욱 향상시킬 수 있다.

또 다른 중요한 변형은 Composite Quantization (CQ)이다. CQ는 여러 양자화 방법을 결합하는 접근 방식으로, PQ의 부분공간 독립성 가정을 완화한다. 이를 통해 부분공간 간의 상관관계를 활용하여 더 높은 정확도를 달성할 수 있다.

## 47.6 본 논문과의 관계

Exqutor는 유사성 쿼리의 카디널리티 추정을 다룬다. 카디널리티 추정은 쿼리 최적화기(query optimizer)가 가장 효율적인 실행 계획을 선택하기 위해 필요한 정보다. PQ는 유사성 검색 인덱스의 중요한 구성 요소이며, PQ 기반 검색의 결과 크기(즉, 카디널리티)를 정확하게 예측하려면 특별한 추정 기법이 필요하다.

특히, PQ는 손실 압축이므로 거리가 변형된다. 따라서 특정 거리 임계값 이내의 벡터를 검색할 때 결과의 개수를 예측하는 것이 매우 어렵다. Exqutor가 제시하는 머신러닝 기반의 카디널리티 추정 기법은 PQ와 같은 압축 방법을 사용하는 벡터 검색 시스템에서 정확한 카디널리티 추정을 가능하게 한다.

### 추가 제기 문제

1. **부분공간 분할의 최적화**: 부분공간을 독립적으로 분할할 때, 최적의 분할 방식에 대한 이론적 지침이 부족하다. 데이터 특성에 따라 최적의  $m$  값을 선택하는 방법이 명확하지 않다.
2. **동적 데이터에의 적용**: PQ 모델이 학습된 후 새로운 벡터가 추가될 때, 모델을 재학습해야 하는지 아니면 기존 중심을 사용할 수 있는지에 대한 명확한 지침이 없다.
3. **카디널리티 추정의 어려움**: PQ 압축으로 인한 거리 변형이 유사성 쿼리의 결과 크기 예측에 미치는 영향을 정량화하기 어렵다. 이것이 Exqutor 같은 학습 기반 추정 방법이 필요한 이유이다.
4. **고차원의 한계**: 벡터의 차원이 매우 높아지면, 부분공간도 많아져야 하고, 이에 따라 계산 오버헤드가 증가한다.

## [48] Faiss: A Library for Efficient Similarity Search and Clustering of Dense Vectors

### 요약

Faiss는 Facebook AI Research (현재 Meta AI)에서 2017년에 발표한 고성능 벡터 유사성 검색 및 클러스터링 라이브러리이다. C++로 구현되었으며 Python, CUDA 등 다양한 인터페이스를 제공한다. Faiss는 매우 큰 규모의 벡터 데이터(수억 개에서 수십억 개)에 대해 빠르고 효율적인 유사성 검색을 수행할 수 있도록 설계되었다.

Faiss의 핵심은 다양한 인덱싱 방법들을 통합하는 통일된 프레임워크이다. 단순한 선형 검색(Linear)부터 IVF (Inverted File Index)와 PQ (Product Quantization), HNSW (Hierarchical Navigable Small World), LSH (Locality-Sensitive Hashing) 등 다양한 방법들을 지원한다. 사용자는 데이터셋의 크기, 메모리 제약, 정확도 요구사항 등에 따라 적절한 인덱싱 방법을 선택할 수 있다.

Faiss는 또한 GPU 가속을 뛰어나게 지원한다. NVIDIA CUDA를 활용하면 CPU만 사용할 때보다 수십 배 빠른 성능을 얻을 수 있다. 특히 billion-scale의 벡터 검색에서 GPU의 병렬 처리 능력이 매우 유효하다. 이는 실시간으로

수십억 개의 벡터를 검색해야 하는 대규모 웹 검색이나 추천 시스템에서 혁신적이었다.

Faiss는 산업계에서 광범위하게 채택되었으며, 텍스트 검색, 이미지 검색, 추천 시스템 등 다양한 영역에서 사용된다. 또한 오픈소스로 공개되어 연구 커뮤니티에서도 널리 사용되고 있다. Faiss의 등장은 대규모 벡터 검색 시스템의 개발을 크게 단순화했으며, 현대적인 대규모 검색 시스템의 사실상의 표준(de facto standard)이 되었다.

## 상세분석

### 48.1 Faiss의 인덱싱 방법들

Faiss는 여러 인덱싱 방법을 제공하며, 각각은 특정한 사용 사례에 최적화되어 있다. 가장 기본적인 방법은 IndexFlatL2이다. 이는 모든 벡터를 그대로 메모리에 저장하고, 쿼리 벡터와의 L2 거리를 정확하게 계산한다. 정확도는 100%이지만, 메모리 사용량이 많고 쿼리 시간이  $O(n*d)$ 이므로 대규모 데이터셋에는 부적합하다.

더 효율적인 방법은 IndexIVFFlat이다. IVF(Inverted File)는 벡터들을 먼저 대략  $k$ 개의 클러스터로 분할한 후, 쿼리와 유사한 클러스터만을 검사하는 방식이다. 각 클러스터는 역파일(inverted list)로 관리되어, 빠른 검색과 메모리 효율성을 모두 제공한다. 검색 시간은  $O((n/k)*d)$ 가 되며, 전체 성능은  $k$  값에 따라 크게 달라진다.

IndexIVFPQ는 IVF와 PQ를 결합한 방법이다. 각 클러스터 내의 벡터들을 PQ로 압축하므로, 메모리 사용량을 크게 줄일 수 있다. 또한 거리 계산도 PQ의 효율적인 방법을 사용한다. 이는 매우 큰 규모의 벡터 검색에서 가장 널리 사용되는 방법이다.

또 다른 중요한 방법은 HNSW (Hierarchical Navigable Small World) 기반의 인덱싱이다. HNSW는 그래프 기반의 방법으로, 인접한 벡터들을 연결하는 계층 구조를 만든다. 검색은 상위 계층에서 시작하여 하위 계층으로 내려가면서 진행되며, 매우 빠른 검색 속도를 제공한다.

### 48.2 GPU 가속과 병렬 처리

Faiss의 가장 중요한 혁신 중 하나는 GPU 가속이다. GPU는 대규모 행렬 연산에 매우 효율적이므로, 벡터 간의 거리 계산에 완벽히 적합하다. Faiss는 CUDA를 사용하여 GPU에서의 벡터 연산을 구현했다.

GPU를 사용할 때의 성능 개선은 놀랍다. CPU에서 IndexFlatL2를 사용하여 1억 개의 128차원 벡터를 검색할 때 수십 초가 걸린다면, GPU를 사용하면 수십 밀리초로 단축된다. 이는 단순한 속도 개선을 넘어, 실시간 검색이 가능하도록 만드는 혁신이다.

GPU 가속의 핵심은 배치 처리(batching)이다. 여러 개의 쿼리를 동시에 GPU에 보내면, GPU는 이들을 병렬로 처리할 수 있다. Faiss는 이러한 배치 처리를 자동으로 수행하도록 설계되어 있다.

또한 여러 개의 GPU를 사용하여 처리량(throughput)을 더욱 증가시킬 수 있다. 대규모 데이터센터에서는 여러 GPU 서버를 연결하여, 수십억 개의 벡터를 실시간으로 검색하는 시스템을 구축할 수 있다.



### 48.3 메모리 최적화 전략

대규모 벡터 검색 시스템에서 메모리는 주요 병목이다. Faiss는 여러 메모리 최적화 기법을 제공한다. 가장 기본적인 것은 PQ를 통한 벡터 압축이다. 128차원 벡터를 32배 압축하면, 1억 개의 벡터도 메모리 수 GB에 저장할 수 있다.

또 다른 최적화 기법은 정량화(Quantization)이다. 벡터를 8비트 또는 16비트 정수로 표현하면, 메모리 사용량을 더욱 줄일 수 있다. Faiss는 여러 종류의 정량화를 지원한다: Scalar Quantization (SQ), Product Quantization (PQ), Binary Quantization 등.

메모리 거래(Memory Trade-off)는 또 다른 중요한 개념이다. 주어진 메모리 제약 하에서, 메모리 사용량을 줄이면 정확도가 낮아지고 쿼리 시간이 길어진다. Faiss는 사용자가 이러한 거래를 명시적으로 제어할 수 있도록 설계되어 있다. 예를 들어, IndexIVFPQ에서 k (클러스터 개수), m (부분공간 개수), nbits (정량화 비트 수) 등을 조정하여 성능을 튜닝할 수 있다.

### 48.4 인덱스 구성의 유연성

Faiss는 다양한 인덱스 구성 전략을 지원한다. 가장 단순한 것은 단일 인덱스 구성이다. 모든 벡터를 하나의 인덱스에 저장하고, 검색은 이 인덱스에서만 수행한다.

더 복잡한 구성은 인덱스 분할(Index Sharding)이다. 데이터를 여러 개의 샤드로 나누고, 각 샤드에 별도의 인덱스를 구축한다. 검색 시에는 모든 샤드에서 병렬로 검색을 수행하고 결과를 병합한다. 이를 통해 매우 큰 데이터셋도 처리할 수 있다.

또 다른 구성은 다층 인덱싱(Multi-level Indexing)이다. 예를 들어, 먼저 LSH를 사용하여 대략적으로 후보를 선택하고, 그 다음 IVF를 사용하여 더 정밀하게 검색한다. 이러한 다층 구조는 정확도와 성능의 균형을 더 잘 맞출 수 있다.

### 48.5 클러스터링 기능

Faiss는 단순히 유사성 검색만 제공하는 것이 아니라, 벡터 클러스터링 기능도 제공한다. 이는 데이터의 구조를 이해하고 그룹화하는 데 유용하다.

k-means 클러스터링은 Faiss의 핵심 기능 중 하나이다. 일반적인 k-means 구현과 달리, Faiss의 k-means는 GPU 가속을 지원하므로 매우 빠르다. 수억 개의 벡터를 수십 초 내에 클러스터링할 수 있다.

클러스터링의 결과는 다양하게 활용될 수 있다. 예를 들어, 클러스터 중심을 사용하여 IVF 인덱싱의 클러스터를 초기화할 수 있다. 또한 클러스터링 결과 자체를 분석하여 데이터의 특성을 이해할 수 있다.

### 48.6 본 논문과의 관계

Exqutor는 유사성 쿼리의 카디널리티 추정을 다룬다. Faiss는 벡터 검색의 가장 중요한 인프라로, Exqutor가 다루는 추정 문제는 Faiss와 같은 벡터 검색 시스템에서 직접 발생한다.

특히, Faiss의 다양한 인덱싱 방법(IVF, PQ, HNSW 등)들은 검색 결과의 크기를 예측하기 어렵게 만든다. 예를 들어, 특정 거리 임계값 이하의 벡터를 검색할 때, 정확히 몇 개의 벡터가 반환될지 예측하는 것은 매우 어렵다.

Exqutor의 머신러닝 기반 카디널리티 추정 기법은 Faiss와 같은 현대적인 벡터 검색 시스템에서 정확한 카디널리티 추정을 가능하게 한다.

또한 Faiss의 GPU 가속이 가능해졌다는 사실은 대규모 벡터 검색에서 카디널리티 추정의 중요성을 더욱 강조한다. GPU를 활용한 검색이 매우 빨라지면서, 쿼리 최적화의 오버헤드가 상대적으로 더 중요해진다. 따라서 정확한 카디널리티 추정으로 최적의 실행 계획을 선택하는 것이 전체 성능에 미치는 영향이 더욱 커진다.

---

## 추가 제기 문제

1. **동적 인덱스 업데이트**: Faiss의 많은 인덱싱 방법들은 정적 데이터셋을 가정한다. 새로운 벡터가 지속적으로 추가되는 동적 환경에서의 효율적인 인덱스 관리 방법이 제한적이다.
2. **카디널리티 추정의 어려움**: Faiss의 근사 검색 방법들은 검색 결과의 크기를 정확하게 예측하기 어렵게 만든다. IVF+PQ 조합의 경우, 각 단계에서의 근사성이 누적되어 최종 결과 크기 예측이 복잡해진다.
3. **하이퍼파라미터 튜닝**: Faiss는 많은 하이퍼파라미터를 가지고 있으며, 최적의 성능을 위해서는 신중한 튜닝이 필요하다. 특정 데이터셋과 쿼리 패턴에 대한 최적의 설정을 자동으로 결정하는 방법이 부족하다.
4. **메모리 대역폭의 한계**: GPU 가속의 이점에도 불구하고, 메모리 대역폭(memory bandwidth)은 여전히 대규모 검색의 병목이 될 수 있다.

---

## [49] Billion-Scale Similarity Search with GPUs

### 요약

본 논문은 Jeff Johnson, Matthijs Douze, Hervé Jégou가 2019년에 IEEE Transactions on Big Data에 발표한 논문으로, Faiss 라이브러리의 GPU 가속 기능을 상세히 다루고 있다. 수십억 개(billion-scale) 규모의 벡터 데이터에 대해 초저지연(ultra-low latency)의 유사성 검색을 수행하는 기술을 소개한다.

논문의 핵심은 GPU의 병렬 처리 능력을 최대한 활용하여, 전통적으로 불가능했던 대규모 검색을 실시간으로 수행하는 것이다. CPU 기반의 순차 처리로는 수십억 개의 벡터를 검색하는 데 수 초 이상이 걸리지만, GPU를 활용하면 수십 밀리초 수준으로 단축할 수 있다. 이는 웹 규모의 이미지 검색, 추천 시스템, 일반 유사성 검색 등 다양한 응용에서 혁신적인 변화를 가져온다.

논문은 단순히 GPU 가속만을 다루는 것이 아니라, billion-scale에서 발생하는 다양한 실무적 문제들을 해결하는 방법을 제시한다. 메모리 관리, 배치 처리, 여러 GPU의 효율적 활용, 네트워크 통신 최적화 등이 포함된다. 특히 메모리 제약을 극복하기 위해 PQ (Product Quantization)와 같은 압축 기법을 GPU 레벨에서 효율적으로 구현하는 방법을 보여준다.

## 상세분석

### 49.1 GPU의 병렬 처리 특성과 벡터 검색

현대적인 GPU는 수천 개의 병렬 처리 코어(parallel processing cores)를 갖추고 있다. NVIDIA의 고성능 GPU는 대략 5,000개 이상의 CUDA 코어를 가지고 있으며, 이들은 독립적으로 동작할 수 있다. 벡터 간의 거리 계산은 본질적으로 병렬화 가능한 작업이므로, GPU의 성능을 최대한 활용할 수 있다.

거리 계산의 병렬화는 다음과 같이 진행된다. 데이터 벡터 집합을 여러 개의 블록으로 분할하고, 각 블록의 벡터들과 쿼리 벡터 간의 거리를 병렬로 계산한다. NVIDIA의 CUDA 프로그래밍 모델에서는 각 블록의 처리를 스레드 블록(thread block)으로 구현하고, 각 벡터의 거리 계산을 개별 스레드(thread)로 할당한다.

GPU 메모리의 계층 구조도 중요하다. GPU는 레지스터(register), 공유 메모리(shared memory), 글로벌 메모리(global memory) 등 여러 단계의 메모리를 가지고 있다. 가장 빠른 성능을 위해서는 접근 패턴(access pattern)을 신중하게 설계하여, 자주 접근하는 데이터를 빠른 메모리에 유지해야 한다.

Coalesced memory access는 GPU 성능의 핵심이다. 여러 스레드가 메모리에서 연속된 주소를 접근할 때(coalesced access), 한 번의 메모리 트랜잭션으로 여러 데이터를 가져올 수 있다. 따라서 벡터 데이터의 메모리 레이아웃을 신중하게 설계하는 것이 중요하다.

### 49.2 Billion-Scale에서의 메모리 관리

CPU의 주 메모리(main memory)가 수백 GB에서 TB 단위인 반면, GPU의 메모리는 일반적으로 4GB에서 40GB 정도이다. 따라서 billion-scale의 벡터 데이터를 GPU에 모두 저장할 수 없다. 이 문제를 해결하기 위해 논문은 다음과 같은 전략들을 제시한다.

첫째, 벡터 압축이다. PQ (Product Quantization)를 사용하면 128차원 벡터를 4바이트로 압축할 수 있다. 따라서 1억 개의 벡터는 400MB에 저장되며, 10억 개도 4GB에 저장할 수 있다. 하지만 이는 정확도 손실을 초래한다.

둘째, 계층화된 인덱싱이다. 먼저 대규모 데이터를 여러 개의 샤드(shard)로 분할한다. 각 샤드는 개별적으로 GPU 메모리에 적재되거나 디스크에 보관된다. 검색 시에는 필요한 샤드만 GPU로 로드하여 검색을 수행하고, 결과를 병합한다.

셋째, 스트리밍 처리(streaming)이다. 벡터 데이터를 한 번에 모두 메모리에 로드하지 않고, 배치 단위로 순차적으로 처리한다. 이를 통해 메모리 제약을 극복할 수 있다.

메모리 대역폭(memory bandwidth)도 중요한 병목이다. GPU의 메모리 대역폭은 CPU보다 높지만, 여전히 billion-scale의 데이터를 빠르게 처리하기에는 제한적일 수 있다. 따라서 메모리 접근을 최소화하는 알고리즘 설계가 중요하다.

### 49.3 배치 처리와 처리량 최적화

GPU의 성능을 최대한 활용하려면, 배치 처리(batching)가 필수적이다. 단일 쿼리를 처리하는 데는 GPU의 모든 코어를 활용하기 어렵지만, 여러 개의 쿼리를 동시에 처리하면 GPU의 병렬 처리 능력을 충분히 활용할 수 있다.

배치 크기(batch size)는 GPU 메모리와 성능의 트레이드오프를 결정한다. 배치 크기가 클수록 처리 효율은 높지만, 메모리 사용량도 증가한다. 또한 배치 크기가 크면 응답 시간(latency)이 증가할 수 있다.

논문에서 제시하는 최적화 전략 중 하나는 메모리 재사용(memory reuse)이다. 쿼리 벡터들이 이미 GPU 메모리에 로드되어 있으면, 데이터 벡터를 순차적으로 로드하면서 여러 번 재사용할 수 있다. 예를 들어, 128개의 쿼리를 GPU에 로드하고, 데이터 벡터를 1000개씩 배치로 로드하면서 모든 쿼리 대 데이터 벡터 간의 거리를 계산할 수 있다.

또 다른 최적화는 비동기 처리(asynchronous processing)이다. CUDA의 스트림(stream) 기능을 사용하면, CPU와 GPU 간의 데이터 전송과 GPU 계산을 겹칠(overlap) 수 있다. 이를 통해 전송 지연 시간을 숨길 수 있다.

#### 49.4 다중 GPU 시스템에서의 확장성

단일 GPU로는 billion-scale의 모든 벡터를 저장하고 처리할 수 없으므로, 여러 GPU를 사용하여 확장하는 방법이 필요하다. 논문은 데이터 병렬화(data parallelism)를 기반으로 하는 확장 전략을 제시한다.

기본 아이디어는 벡터 데이터를 여러 GPU에 분산 저장하는 것이다. 각 GPU는 자신에게 할당된 벡터 부분집합에 대해 검색을 수행한다. 검색 요청이 들어오면, 모든 GPU에 병렬로 쿼리를 보내고, 각 GPU는 독립적으로 검색을 수행한 후 결과를 반환한다. 호스트 시스템은 이 결과들을 병합하여 최종 결과를 생성한다.

이 방식의 성능은 GPU 간의 통신 오버헤드에 크게 영향을 받는다. GPU 간 통신이 느리면 전체 성능이 저하된다. 따라서 효율적인 통신 프로토콜이 중요하다. NVIDIA의 NVLink 기술은 GPU 간 고속 통신을 제공하며, 이를 통해 여러 GPU 간의 데이터 전송을 크게 가속할 수 있다.

#### 49.5 IVF+PQ의 GPU 최적화 구현

IVF+PQ는 billion-scale 검색의 가장 널리 사용되는 방법이다. IVF는 대규모 데이터를 먼저 대략적으로 클러스터링하고, PQ는 각 벡터를 압축한다.

GPU에서 IVF+PQ를 구현할 때의 핵심 과정은 다음과 같다. 먼저, 쿼리 벡터가 주어지면 모든 클러스터 중심과의 거리를 계산한다. 이 계산은 GPU에서 매우 빠르게 수행될 수 있다 (수천 개의 클러스터여도 밀리초 수준).

쿼리와 가장 가까운 클러스터들을 선택한다 (예: 상위 100개). 그 다음, 선택된 클러스터들의 벡터들을 대상으로 정밀 검색을 수행한다. 이때 PQ 압축된 벡터들과의 거리를 계산하는데, 이는 테이블 룩업을 기반으로 매우 효율적으로 수행된다.

최종적으로 상위 k개의 벡터를 선택한다. 경우에 따라, 근사 검색 결과의 정확도를 높이기 위해 상위 k개 벡터의 원본 거리를 재계산하는 과정도 거칠 수 있다.

#### 49.6 실험 결과와 성능 특성

논문의 실험은 매우 대규모 데이터셋에서 수행된다. SIFT 1B (10억 개 SIFT 디스크립터), 1년간의 웹 기반 이미지 벡터 등이 사용된다. GPU를 사용한 결과는 인상적이다.

예를 들어, SIFT 1B 데이터셋에 대해 GPU 기반 IVF+PQ 검색은 1초에 수백 개의 쿼리를 처리할 수 있다. 이는 개별 쿼리당 밀리초 수준의 응답 시간을 의미한다. 동일한 데이터셋을 CPU로 검색하면 초 단위의 응답 시간이 필요하

로, GPU는 100배 이상의 성능 향상을 제공한다.

메모리 효율성도 우수하다. PQ 압축을 사용하면 10억 개의 벡터를 수십 GB의 메모리로 저장할 수 있으며, GPU 메모리만으로도 수천만 개의 벡터를 처리할 수 있다.

#### 49.7 본 논문과의 관계

Exqutor는 유사성 쿼리의 카디널리티 추정을 다룬다. 본 논문이 다루는 GPU 기반 billion-scale 검색은 Exqutor가 다루는 카디널리티 추정 문제를 더욱 시급하게 만든다.

GPU를 사용한 검색이 매우 빨라지면서, 쿼리 최적화의 중요성이 상대적으로 더 커진다. 효율적인 실행 계획을 선택하기 위해서는 정확한 카디널리티 추정이 필수적이다. 특히 billion-scale에서는 몇 프로센트의 성능 차이도 초당 수백 쿼리를 처리하는 시스템에서는 무시할 수 없는 차이가 된다.

또한 IVF+PQ의 근사 검색은 결과 크기의 예측을 어렵게 만든다. IVF의 클러스터 선택에서의 근사성과 PQ의 거리 변형이 누적되어, 최종 결과 크기를 정확하게 예측하기 어렵다. Exqutor의 머신러닝 기반 추정 기법은 이러한 복잡한 시스템에서도 정확한 카디널리티 추정을 제공한다.

### 추가 제기 문제

1. **메모리 대역폭의 한계:** GPU 메모리 대역폭은 높지만, billion-scale 데이터에서는 여전히 병목이 될 수 있다. 메모리 대역폭을 더욱 효율적으로 활용하는 방법이 필요하다.
2. **정확도와 속도의 트레이드오프:** GPU 가속은 속도를 극적으로 높이지만, 검색 정확도와와의 트레이드오프가 존재한다. PQ 압축 수준을 높이면 메모리와 속도는 이점을 얻지만 정확도가 낮아진다.
3. **동적 업데이트의 어려움:** billion-scale 데이터셋에서 새로운 벡터를 추가하거나 기존 벡터를 삭제하는 것은 복잡한 작업이다. 인덱스를 효율적으로 유지하면서 동시성을 제어하는 방법이 필요하다.
4. **카디널리티 추정의 정확성:** GPU 기반 근사 검색에서 결과 크기의 정확한 예측은 매우 어렵다. 쿼리 최적화를 위해 학습 기반의 카디널리티 추정이 필수적이다.

### [50] Exact Cardinality Query Optimization with Bounded Execution Cost

#### 요약

본 논문은 Immanuel Trummer가 2019년 SIGMOD에 발표한 논문으로, ECQO (Exact Cardinality Query Optimization)의 개념을 처음 제시한 기념비적 작업이다. 이 논문의 핵심은 쿼리 최적화 과정에서 정확한 카디널리티(결과 개수)를 알 수 있다면, 더욱 효율적인 실행 계획을 선택할 수 있다는 직관에서 출발한다.

전통적인 쿼리 최적화는 주로 통계 정보를 기반으로 카디널리티를 추정한다. 하지만 이러한 추정은 종종 부정확하며, 특히 유사성 검색과 같은 복잡한 쿼리에서는 추정 오류가 심각할 수 있다. Trummer는 이 문제를 해결하기 위

해, 제한된 실행 비용(bounded execution cost) 내에서 정확한 카디널리티를 결정할 수 있는 방법을 제시했다.

ECQO의 핵심 아이디어는 다음과 같다. 완전한 쿼리 실행 대신, 부분적 실행을 통해 결과의 크기를 정확히 파악할 수 있다면, 이를 기반으로 최적의 실행 계획을 선택할 수 있다는 것이다. 예를 들어, 조인(join) 쿼리에서 각 조인 단계 후의 정확한 카디널리티를 알면, 다음 조인의 순서를 동적으로 최적화할 수 있다.

이 접근법은 특히 유사성 검색과 같은 새로운 쿼리 유형에 매우 적합하다. 유사성 쿼리의 결과 크기는 거리 임계값에 매우 민감하며, 이를 정확하게 예측하는 것은 매우 어렵다. 그러나 실제 데이터에 대해 부분적으로 쿼리를 실행해보면, 정확한 결과 크기를 빠르게 파악할 수 있다.

본 논문은 Exqutor의 직접적인 영감이 되었으며, Exqutor는 이 개념을 한 단계 더 발전시켜 머신러닝을 사용한 카디널리티 추정 방법을 제시한다. ECQO는 원리적 접근(principled approach)을 제시했고, Exqutor는 실무적 구현을 제공한 것이다.

## 상세분석

### 50.1 카디널리티 추정의 전통적 문제점

전통적인 쿼리 최적화에서 카디널리티 추정은 매우 중요한 역할을 한다. 쿼리 옵티마이저는 추정된 카디널리티를 기반으로 실행 계획을 선택하기 때문이다. 예를 들어, 조인 쿼리에서 A 테이블과 B 테이블을 조인할 때, A를 먼저 필터링한 후 B와 조인하는 것이 좋은지, 아니면 B를 먼저 필터링하는 것이 좋은지는 각 단계 후의 카디널리티에 따라 결정된다.

전통적인 카디널리티 추정은 다음과 같은 문제점을 가진다. 첫째, 통계 정보에 기반하고 있다. 이는 실제 데이터 분포와 다를 수 있다. 예를 들어, 테이블의 히스토그램(histogram)이 실제 값의 분포를 완벽하게 반영하지 못할 수 있다.

둘째, 독립성 가정(independence assumption)을 기반으로 한다. 여러 조건이 결합될 때, 일반적으로 각 조건이 독립적이라고 가정하고 확률을 곱한다. 하지만 현실에서 여러 조건은 종종 상관관계가 있다. 예를 들어, 도시(city) 필터와 우편번호(zip code) 필터가 결합되었을 때, 이 두 조건은 독립적이지 않으므로 확률 곱셈은 부정확하다.

셋째, 복잡한 쿼리에서는 추정 오류가 누적된다. 여러 조인 단계를 거치면서, 각 단계의 추정 오류가 누적되어 최종 추정값의 정확도가 심각하게 악화될 수 있다.

넷째, 새로운 쿼리 유형(예: 유사성 검색, 그래프 쿼리)에 대해서는 통계 기반의 추정 방법이 적용되기 어렵다. 벡터 거리의 분포나 유사성 쿼리의 결과 크기를 전통적인 통계 방법으로 추정하는 것은 매우 어렵다.

### 50.2 ECQO의 핵심 개념: 부분 실행(Partial Execution)

ECQO의 혁신적인 아이디어는 완전한 쿼리 실행 없이도 정확한 카디널리티를 파악할 수 있다는 것이다. 이는 부분 실행(partial execution)을 통해 가능하다.

부분 실행의 기본 원리는 다음과 같다. 쿼리를 완전히 실행하는 대신, 부분적으로 실행하여 이미 얼마나 많은 결과가 나왔는지 추정한다. 예를 들어, 조인 쿼리에서 첫 번째 조인까지만 실행하고, 그 결과의 정확한 개수를 파악한다.

이를 기반으로 다음 조인의 순서를 결정하거나, 나머지 조인이 얼마나 비용이 들 것인지 추정할 수 있다.

구체적인 예를 살펴보자. `SELECT * FROM A JOIN B ON A.id = B.id WHERE A.val > 100` 쿼리를 실행한다고 하자. 전통적인 방식에서는: 1. A에서 `val > 100`인 행의 개수를 추정한다 (예: 1000개로 추정) 2. 각 1000개 행이 B와 조인될 때 얼마나 많은 결과가 나올지 추정한다 3. 이를 기반으로 최적의 실행 순서를 결정한다

ECQO 방식에서는: 1. A에서 `val > 100`인 행들을 실제로 실행한다 (정확한 개수, 예: 950개) 2. 950개 행이 B와 조인될 때 정확히 몇 개의 결과가 나오는지 파악한다 (예: 3000개) 3. 이제 정확한 정보를 기반으로 나머지 계획을 최적화한다

부분 실행의 비용은 제한되어야 한다. 만약 부분 실행이 전체 실행만큼 비용이 많이 들면, ECQO의 의미가 없다. 따라서 Trummer는 제한된 실행 비용(bounded execution cost) 내에서 최대한의 카디널리티 정보를 얻을 수 있는 방법을 제시한다.

### 50.3 비용 제한 하에서의 카디널리티 결정

ECQO의 핵심은 비용 제한(cost budget)을 설정하고, 이 제한 내에서 최대한 정확한 카디널리티를 결정하는 것이다. Trummer는 여러 가지 방법을 제시한다.

첫 번째 방법은 인덱스 기반 샘플링(index-based sampling)이다. 쿼리 조건을 만족하는 행들이 인덱스에서 어떻게 분포되어 있는지 파악하고, 인덱스를 순회하면서 조건을 만족하는 행의 개수를 센다. 인덱스 순회 비용은 풀 테이블 스캔보다 훨씬 저렴할 수 있다.

두 번째 방법은 단계적 실행(progressive execution)이다. 쿼리의 연산 단계를 차례대로 실행하되, 각 단계 후에 진행할지 여부를 결정한다. 예를 들어, 10% 데이터에 대해 쿼리를 실행해보고 그 결과를 기반으로 전체 실행 비용을 추정할 수 있다.

세 번째 방법은 근사 구조(approximation structures)를 사용하는 것이다. 예를 들어, 블룸 필터(Bloom filter)나 count-min sketch 같은 확률적 자료구조를 사용하면, 최소한의 메모리와 계산으로도 근사적인 결과 개수를 빠르게 얻을 수 있다. 이를 정확한 개수로 수렴하도록 반복적으로 개선할 수 있다.

### 50.4 동적 계획(Dynamic Planning)

ECQO의 또 다른 중요한 측면은 동적 계획(dynamic planning)이다. 전통적인 쿼리 최적화는 static planning으로, 쿼리 시작 전에 전체 실행 계획을 수립한다. 반면 ECQO는 부분 실행의 결과를 기반으로 실행 중에 계획을 수정할 수 있다.

동적 계획의 예를 들면 다음과 같다. 조인 순서를 결정할 때: 1. A JOIN B를 먼저 실행한다 (작은 비용) 2. A JOIN B의 정확한 카디널리티를 파악한다 3. (A JOIN B) JOIN C의 비용을 정확하게 추정한다 4. C JOIN A를 먼저 실행하는 것보다 (A JOIN B) JOIN C가 더 저렴하면 이를 선택한다

이러한 동적 계획은 정적 계획보다 훨씬 더 최적의 실행 계획을 찾을 수 있다. 특히 카디널리티 추정이 부정확한 경우에는 동적 계획의 이점이 매우 크다.

## 50.5 유사성 쿼리에의 적용

ECQO가 특히 유용한 분야는 유사성 쿼리이다. 벡터 데이터베이스에서 "벡터  $q$ 와 거리  $d$  이내의 모든 벡터를 찾아라"라는 쿼리가 주어졌을 때, 결과가 몇 개일지 미리 추정하는 것은 매우 어렵다.

전통적인 통계 기반 방법은 유사성 쿼리에 적용되기 어렵다: - 벡터 거리의 분포가 복잡하고 차원에 따라 달라진다 - 벡터 공간의 특성(clustering, manifold structure)을 반영하기 어렵다 - 학습 데이터와 테스트 데이터의 분포 차이가 클 수 있다

ECQO 방식에서는: 1. 각 유사성 쿼리를 실제로 부분 실행한다 2. 인덱스의 일부를 검사하여 현재까지 몇 개의 결과가 나왔는지 파악한다 3. 현재 개수와 검사한 데이터의 비율을 기반으로 전체 결과 개수를 추정한다 4. 이 추정값이 정확한 경우, 이를 최적화 결정에 사용한다

## 50.6 실험 결과와 성능 특성

Trummer는 전통적인 카디널리티 추정 방법과 ECQO를 비교하는 실험을 수행했다. 결과는 인상적이다.

전통적인 추정의 추정 오류(estimation error)는 평균 5배에서 10배 정도였다. 즉, 실제 결과가 1000개인데 추정값이 5000개 또는 200개인 경우가 흔하다는 뜻이다. 반면 ECQO의 추정 오류는 평균 1.5배 정도로, 매우 정확하다.

쿼리 실행 시간 측면에서도 ECQO는 이점을 보여준다. 정확한 카디널리티 추정으로 인해 더 효율적인 실행 계획을 선택할 수 있고, 이는 전체 쿼리 실행 시간을 단축한다. 복잡한 조인 쿼리의 경우, ECQO를 사용하면 실행 시간을 30% 이상 단축할 수 있다.

부분 실행의 비용도 관리 가능한 수준이다. 일반적으로 부분 실행 비용은 전체 쿼리 실행 시간의 10~20% 정도이므로, 부분 실행으로 인한 오버헤드가 최적화를 통한 이득을 초과하지 않는다.

## 50.7 본 논문과 Exqutor의 관계

ECQO는 Exqutor의 직접적인 선행 작업이며, Exqutor는 이 개념을 유사성 쿼리에 특화된 형태로 발전시킨다.

Trummer의 ECQO는 부분 실행을 기반으로 한 접근이고, 이는 실무에서 구현하기 위해 여러 가지 오버헤드를 수반한다: - 각 쿼리마다 부분 실행을 수행해야 한다 (런타임 오버헤드) - 부분 실행의 결과를 기반으로 정확한 값으로 수렴하는 과정이 복잡하다 - 쿼리 패턴에 따라 부분 실행의 비용이 달라진다

Exqutor는 이러한 문제를 다르게 접근한다: - 머신러닝 모델을 사용하여 쿼리 특성으로부터 직접 카디널리티를 예측한다 - 모델 학습은 사전에 수행되므로, 런타임 오버헤드가 매우 적다 - 부분 실행 대신, 벡터 검색의 특성(벡터 차원, 거리 임계값, 인덱스 구조 등)을 특징으로 사용한다

특히 유사성 쿼리에서 Exqutor는 다음과 같은 추가적인 고려사항을 반영한다: - 벡터 공간의 내재적 구조(manifold structure) - 쿼리 벡터와 데이터 벡터 간의 거리 분포 - 인덱싱 방법(IVF, HNSW, PQ 등)의 특성 - 임계값(threshold)에 대한 민감도



## 추가 제기 문제

1. **부분 실행의 오버헤드**: 정확한 카디널리티를 위해 부분 실행을 수행하는 것이 항상 정당한지 의문의 여지가 있다. 간단한 쿼리의 경우 부분 실행의 오버헤드가 이점보다 클 수 있다.
2. **온라인 학습의 필요성**: ECQO는 쿼리마다 부분 실행을 수행하므로, 시간이 지남에 따라 데이터 분포가 변해도 자동으로 적응한다. 하지만 Exqutor 같은 학습 기반 방법은 정기적인 모델 재학습이 필요할 수 있다.
3. **일반화의 한계**: ECQO는 각 쿼리에 특화된 결정을 내리므로 매우 정확하지만, 새로운 패턴의 쿼리에는 적응하기 어렵다. 학습 기반 방법이 더 나은 일반화를 제공할 수 있다.
4. **CAP 트레이드오프**: 정확성(Accuracy), 비용(Cost), 편의성(Practicality) 간의 트레이드오프가 존재한다. ECQO는 정확성을 추구하지만 비용이 크고, 완전 추정엔 비용이 적지만 정확성이 낮으며, 머신러닝 기반 방법은中间的의 균형을 제공한다.

## [51] Learned Cardinality Estimation for Similarity Queries (SelNet)

### 요약

본 논문은 Jiayi Sun, Guoliang Li, Nan Tang가 2021년 SIGMOD에 발표한 SelNet으로, 유사성 쿼리의 카디널리티 추정에 머신러닝을 적용한 최초의 주요 작업이다. 이 논문은 Exqutor의 직접적인 비교 대상이며, 유사성 검색 시스템에서 정확한 카디널리티 추정을 위해 신경망을 어떻게 사용할 수 있는지 보여준다.

SelNet의 핵심 아이디어는 단순하면서도 강력하다: 유사성 쿼리의 선택도(selectivity)를 예측하는 문제를 회귀 문제(regression problem)로 바라보고, 신경망을 사용하여 이를 학습하는 것이다. 선택도는 쿼리 결과의 개수를 데이터셋 전체 크기로 나눈 비율이다. 예를 들어, 100만 개의 벡터 중에서 1000개가 검색 조건을 만족한다면, 선택도는 0.001이다.

SelNet은 다음과 같은 특징을 갖는다. 첫째, 유사성 쿼리의 특성(벡터 차원, 거리 임계값, 쿼리 벡터의 특성 등)을 입력으로 받아, 선택도를 직접 예측한다. 둘째, 신경망 기반의 접근은 벡터 공간의 복잡한 특성을 자동으로 학습할 수 있다. 셋째, 다양한 데이터셋과 쿼리 패턴에 대해 충분한 학습 데이터가 있다면, 높은 정확도를 달성할 수 있다.

SelNet의 성능은 매우 인상적이다. 전통적인 통계 기반 추정 방법과 비교했을 때, 추정 오류를 평균 80% 이상 감소시킨다. 즉, 전통적 방법의 추정 오류가 5배라면, SelNet은 1배 이내의 오류로 줄일 수 있다는 뜻이다.

하지만 SelNet도 한계가 있다. 학습 데이터에 포함되지 않은 새로운 패턴의 쿼리에 대해서는 성능이 저하될 수 있다 (분포 이동, distribution shift). 또한 신경망의 구조와 하이퍼파라미터 선택에 따라 성능이 크게 달라질 수 있다. Exqutor는 이러한 SelNet의 한계를 개선하는 것을 목표로 한다.

## 상세분석

### 51.1 유사성 쿼리의 특성과 카디널리티 추정의 어려움

유사성 쿼리는 전통적인 조건부 쿼리(예: WHERE age > 30)와는 다른 특성을 가지고 있다. 전통적인 쿼리의 카디널리티는 주로 선택 조건(selection predicate)에 의존하고, 이는 상대적으로 예측하기 쉽다. 예를 들어, 나이가 균등하게 분포되어 있다면, "age > 30"을 만족하는 행의 비율은 거의 항상 일정하다.

반면 유사성 쿼리의 카디널리티는 여러 복잡한 요인에 의존한다:

첫째, 거리 임계값(distance threshold)이다. "벡터 q와의 거리가 d 이내인 벡터를 찾아라"라는 쿼리에서, d가 변하면 결과의 개수가 지수적으로 변할 수 있다. 벡터 공간에서는 거리가 증가함에 따라 포함되는 벡터의 개수가 반경의 차원 거듭제곱에 비례하므로 (d의 차원 거듭제곱), 매우 예민한 변화를 보인다.

둘째, 벡터 공간의 차원이다. 고차원 벡터 공간에서는 curse of dimensionality의 영향으로, 모든 벡터들이 서로 거의 비슷한 거리에 분포한다. 따라서 동일한 거리 임계값이라도, 차원에 따라 결과 개수가 크게 달라진다.

셋째, 데이터의 분포이다. 벡터가 균등하게 분포되어 있는지, 아니면 특정 영역에 밀집되어 있는지에 따라 결과 개수가 달라진다. 예를 들어, 이미지 데이터의 경우 특정 물체나 장면이 데이터셋에서 과다 대표될 수 있고, 이는 유사성 쿼리의 결과 개수에 영향을 미친다.

넷째, 쿼리 벡터의 특성이다. 쿼리 벡터가 데이터셋의 중심에 있는지, 아니면 주변부에 있는지에 따라 결과 개수가 달라진다. 또한 쿼리 벡터의 "인기도"(어떤 물체나 개념을 나타내는 정도)도 영향을 미칠 수 있다.

이러한 복잡한 요인들을 전통적인 통계 기반 방법으로 모두 고려하기는 매우 어렵다. SelNet은 신경망을 사용하여 이러한 요인들 간의 복잡한 관계를 자동으로 학습한다.

### 51.2 SelNet의 아키텍처와 특징 표현

SelNet의 신경망 아키텍처는 상대적으로 간단하다. 입력 계층, 여러 숨은 계층, 그리고 단일 출력 계층으로 구성된다. 출력은 선택도(selectivity) 값이다.

입력 특징(input features)은 유사성 쿼리를 나타낼 수 있는 모든 정보를 포함한다: - 벡터 차원(dimensionality): 벡터 공간의 차원 수 - 거리 임계값(distance threshold): 검색 반경 - 데이터셋 크기(dataset size): 전체 벡터의 개수 - 거리 통계(distance statistics): 평균 거리, 거리의 분산 등 - 벡터 분포 통계: 벡터들이 얼마나 밀집되어 있는지 등

이러한 특징들을 입력으로 사용하면, 신경망은 특징들 간의 복잡한 상호작용을 학습할 수 있다. 예를 들어, 차원이 높을수록 동일한 거리 임계값에서 더 많은 벡터가 포함되는 현상을 신경망이 학습할 수 있다.

특징 표현의 정규화(normalization)도 중요하다. 거리 임계값은 0부터 매우 큰 값까지 다양하므로, 신경망의 수렴을 위해 정규화가 필수적이다. SelNet은 특징들을 min-max 정규화 또는 z-score 정규화를 사용하여 0~1 범위 또는 평균 0, 분산 1로 변환한다.

### 51.3 학습 데이터 수집과 라벨링

SelNet을 학습하기 위해서는 대량의 (쿼리 특성, 선택도) 쌍의 학습 데이터가 필요하다. 논문에서는 다음과 같은 방법으로 학습 데이터를 수집한다.

첫째, 다양한 데이터셋을 준비한다. 논문에서는 실제 벡터 데이터셋(예: SIFT, GIST 특징) 뿐만 아니라, 합성 데이터(synthetic data)도 사용한다. 합성 데이터는 다양한 분포(균등 분포, 가우시안 분포, 클러스터링된 분포 등)를 가진 벡터들로 구성된다.

둘째, 각 데이터셋에 대해 다양한 쿼리를 생성한다. 거리 임계값을 체계적으로 변화시키면서 (예: 0.01, 0.05, 0.1, 0.2 등), 각 쿼리에 대한 정확한 선택도를 계산한다. 정확한 선택도는 각 쿼리를 실제로 전체 데이터셋에 대해 실행하여 결과 개수를 센다.

셋째, 학습 데이터를 정규화하고 전처리한다. 선택도 값의 분포가 매우 비대칭적(skewed)일 수 있으므로, 로그 변환(log transformation)을 적용할 수 있다. 이는 신경망의 학습을 더 안정적으로 만든다.

학습 데이터의 크기는 신경망의 성능에 직접 영향을 미친다. 충분한 학습 데이터가 있으면 신경망이 복잡한 패턴을 학습할 수 있지만, 데이터가 부족하면 과적합(overfitting)의 위험이 있다.

### 51.4 신경망의 학습과 최적화

SelNet의 신경망 학습은 표준적인 지도 학습(supervised learning) 과정을 따른다. 손실 함수(loss function)로는 평균 제곱 오류(Mean Squared Error, MSE) 또는 평균 절대 오류(Mean Absolute Error, MAE)를 사용한다.

MSE는 큰 오류에 더 패널티를 주므로, 매우 잘못된 추정을 피하는 데 효과적이다. MAE는 오류의 크기에 직선적으로 패널티를 주므로, 이상치(outlier)에 덜 민감하다. 카디널리티 추정의 맥락에서는 MAE가 더 적절할 수 있다 (예: 실제 1000개일 때 99개와 1099개의 오류 크기는 같지만, 실무적 영향은 다를 수 있음).

최적화 알고리즘으로는 일반적으로 Adam 또는 SGD(Stochastic Gradient Descent)를 사용한다. Adam은 빠른 수렴을 제공하고 하이퍼파라미터에 덜 민감하므로, 많은 경우 기본 선택이 된다.

정규화(regularization)도 중요하다. L1 또는 L2 정규화를 사용하여 가중치의 크기를 제한하면, 과적합을 방지할 수 있다. 조기 종료(early stopping)도 효과적인데, 검증 데이터셋에서의 성능이 더 이상 개선되지 않으면 학습을 멈춘다.

### 51.5 평가 지표와 성능 분석

SelNet의 성능을 평가하기 위해 여러 지표를 사용한다:

첫째, 추정 오류(estimation error)이다. 일반적으로 q-error (quantile error)를 사용한다. 이는 예측값과 실제값의 최대 비율을 나타낸다. 예를 들어, 실제값이 1000이고 예측값이 500이면 q-error는 2이다 ( $\max(1000/500, 500/1000) = 2$ ). q-error는 상대적 오류를 나타내므로, 절대적인 값의 크기에 관계없이 오류의 심각성을 평가할 수 있다.

둘째, 95 percentile q-error이다. 모든 쿼리에 대한 q-error의 95 백분위수 값으로, 매우 나쁜 추정이 얼마나 자주 발생하는지를 나타낸다.

셋째, Mean Absolute Percentage Error (MAPE)이다. 이는 평균적인 백분율 오류를 나타낸다.

SelNet의 실험 결과에 따르면: - 전통적인 통계 기반 추정 대비 평균 q-error를 3~5배에서 1.5~2배로 개선 - 일부 특정 벡터 차원과 거리 범위에서는 거의 완벽한 추정 달성 - 학습 데이터가 충분한 분포 범위에서는  $q\text{-error} < 2$  달성

## 51.6 SelNet의 한계와 실무적 과제

SelNet은 강력한 방법이지만, 실무 적용에 있어 여러 한계가 있다.

첫째, 분포 이동(distribution shift)이다. 모델이 학습한 데이터의 분포와 실제 운영 환경의 데이터 분포가 다르면, 성능이 저하된다. 예를 들어, 학습 데이터는 거리 임계값이 0.1~0.5 범위의 쿼리만 포함했는데, 실제로는 0.05나 1.0 범위의 쿼리가 들어올 수 있다.

둘째, 데이터 변화이다. 시간이 지남에 따라 데이터셋의 분포가 변할 수 있다. 예를 들어, 추천 시스템의 벡터가 지속적으로 업데이트되면, 이전의 거리 분포와는 다른 패턴이 나타날 수 있다.

셋째, 신경망의 해석가능성이 낮다. 신경망이 어떤 특징을 중요하게 여기는지, 왜 특정 쿼리에서 오류가 발생하는지 이해하기 어렵다. 이는 모델의 신뢰도를 낮출 수 있다.

넷째, 새로운 인덱스 구조에의 적응이다. 만약 IVF 인덱싱을 사용하다가 HNSW로 변경하면, 모델의 성능이 저하될 수 있다. 인덱스 구조도 입력 특징에 포함되어야 하며, 각 인덱스 구조에 대해 별도의 학습이 필요할 수 있다.

## 51.7 Exqutor와의 차이점 및 개선점

Exqutor는 SelNet의 개념을 바탕으로 하지만, 여러 가지 측면에서 개선을 시도한다:

첫째, 더 정교한 특징 엔지니어링이다. Exqutor는 벡터 공간의 고유한 특성을 더 잘 포착할 수 있는 특징들을 설계한다. 예를 들어, 거리 분포의 고전적인 통계량 외에도, 벡터 공간의 국소적 밀도(local density)나 곡률(curvature) 같은 기하학적 특성을 고려할 수 있다.

둘째, 불확실성 정량화(uncertainty quantification)이다. Exqutor는 단순히 점 추정(point estimate)만 제공하는 것이 아니라, 추정값에 대한 신뢰도(confidence interval) 또는 불확실성(uncertainty)도 제공할 수 있다. 이는 쿼리 최적화에서 위험도를 고려한 의사결정을 가능하게 한다.

셋째, 적응형 학습(adaptive learning)이다. Exqutor는 새로운 쿼리 패턴이 발생할 때, 모델을 점진적으로 업데이트할 수 있다. 이를 통해 분포 이동에 대한 적응성을 높일 수 있다.

넷째, 해석가능성 개선이다. Exqutor는 트리 기반 모델(예: XGBoost, Random Forest) 또는 선형 모델을 사용할 수 있으며, 이들은 신경망보다 해석가능성이 높다.

다섯째, 실행 비용의 고려이다. Exqutor는 부분 실행을 통한 실제 카디널리티 확인과 머신러닝 기반 추정을 결합하여, 각 쿼리의 특성과 실행 예산(execution budget)에 따라 최적의 방식을 선택할 수 있다.

## 51.8 본 논문과의 관계

SelNet은 유사성 쿼리의 카디널리티 추정에 머신러닝을 처음 적용한 중요한 작업이다. 이는 기존의 통계 기반 추정 방법이 유사성 쿼리에 부적합하다는 것을 보여주었고, 머신러닝이 이 문제를 해결할 수 있는 강력한 도구임을 증명했다.

Exqutor는 SelNet의 핵심 아이디어를 계승하면서도, 다음과 같은 측면에서 개선을 이룬다: - 더 강력하고 정확한 학습 모델 - 실행 비용 제약을 고려한 최적화 - 부분 실행과 학습 기반 추정의 효과적인 결합 - 더 포괄적인 실험과 평가

### 추가 제기 문제

- 1. 모델의 일반화:** SelNet 모델이 학습한 데이터와 다른 분포의 새로운 데이터에 얼마나 잘 일반화되는가? 특히 매우 높은 차원이나 매우 낮은 차원의 데이터에는 어떻게 적응하는가?
- 2. 온라인 학습과 적응:** 실제 운영 환경에서 지속적으로 들어오는 새로운 쿼리들을 어떻게 활용하여 모델을 개선할 것인가? 모델 재학습의 빈도와 비용은?
- 3. 인덱스 구조의 영향:** 동일한 데이터에 대해 다양한 인덱스 구조(IVF, HNSW, LSH 등)를 사용할 때, 모델의 성능이 어떻게 달라지는가? 인덱스 구조별로 별도의 모델을 학습해야 하는가?
- 4. 카디널리티 범위:** 매우 작은 카디널리티(< 10)나 매우 큰 카디널리티(> 백만)의 경우, 모델의 성능이 충분한가? 이러한 극단적인 경우를 특별히 처리해야 하는가?
- 5. 해석가능성:** 신경망이 특정 쿼리에서 큰 오류를 범한 이유를 사용자나 관리자가 이해할 수 있는 방법이 있는가?

## [52] Kepler: Robust Learning for Parametric Query Optimization

### 요약

본 논문은 Doshi 등이 2023년 PACMMOD에 발표한 Kepler로, 학습 기반 쿼리 최적화를 위한 강력한 프레임워크를 제시한다. Kepler는 매개변수 쿼리(parametric query)의 최적화에 초점을 맞추며, 매개변수의 값이 변해도 일관되게 좋은 성능을 유지하는 실행 계획을 학습하는 것을 목표로 한다.

Kepler의 핵심 아이디어는 쿼리 매개변수가 다양한 값을 가질 때, 단순히 평균적인 최적 계획을 찾는 것이 아니라, 매개변수의 분포 변화에 강건한(robust) 계획을 찾는 것이다. 예를 들어, WHERE age > ?라는 쿼리에서 ?에 다양한 값이 들어올 때, 어떤 값에서는 A 계획이 최적이고 다른 값에서는 B 계획이 최적일 수 있다. Kepler는 이러한 매개변수 범위를 고려하여, 전반적으로 가장 좋은 성능을 제공하는 계획을 선택한다.

Kepler는 강건 최적화(robust optimization) 원리를 쿼리 최적화에 적용한다. 이는 최악의 경우(worst-case)에 대비하는 전략으로, 매개변수가 어떤 값을 가져도 합리적인 수준의 성능을 유지하도록 보장한다.

본 논문과 Exqutor의 관계를 살펴보면, 둘 다 학습 기반의 쿼리 최적화를 다루지만, 초점이 다르다. Exqutor는 주로 카디널리티 추정 정확도를 높이는 데 집중하는 반면, Kepler는 학습된 계획이 매개변수 변화에 얼마나 강건한지에 초점을 맞춘다. 두 접근법은 상호 보완적이다.

## 상세분석

### 52.1 매개변수 쿼리와 최적화의 도전

매개변수 쿼리(parametric query)는 일부 값이 런타임에 결정되는 쿼리이다. SQL에서 가장 흔한 예는 WHERE 절의 상수 값이 매개변수로 대체된 경우이다. 예를 들어: - 정적: `SELECT * FROM orders WHERE price > 100 AND customer_type = 'premium'` - 매개변수: `SELECT * FROM orders WHERE price > ? AND customer_type = ?`

매개변수 쿼리는 매우 흔하다. 대부분의 데이터베이스 애플리케이션에서 사용자 입력이나 런타임 조건에 따라 쿼리 조건이 변하기 때문이다. 또한 준비된 명령문(prepared statement)을 사용할 때도 매개변수 쿼리가 일반적이다.

매개변수 쿼리의 최적화는 여러 도전을 제시한다. 첫째, 매개변수 값에 따라 최적의 실행 계획이 완전히 달라질 수 있다는 점이다. 예를 들어: - `price > 10000`인 경우: 인덱스를 사용한 계획이 효율적 - `price > 50`인 경우: 풀 테이블 스캔이 더 효율적 (선택도가 높으므로)

둘째, 매개변수가 결정되기 전에 최적 계획을 선택해야 할 수도 있다는 점이다. 예를 들어, 컴파일 시간(compile time)에 계획을 선택해야 한다면, 매개변수의 구체적 값을 알 수 없다.

셋째, 매개변수 값이 예상과 다를 때 (예: 카디널리티 추정 오류), 선택된 계획의 성능이 심각하게 저하될 수 있다는 점이다.

전통적인 해결책은 일반화된 계획(generic plan)을 선택하는 것이다. 이는 대부분의 매개변수 값에 대해 합리적인 성능을 제공하는 계획이다. 하지만 이는 특정 매개변수 값에 대해 최적이지 아닐 수 있다.

### 52.2 강건 최적화와 Kepler의 접근

Kepler는 강건 최적화(robust optimization) 원리를 도입한다. 강건 최적화의 기본 아이디어는 최악의 경우에 대비하는 것이다. 즉, 매개변수가 어떤 값을 가지더라도, 합리적인 수준의 성능을 보장하는 계획을 선택하는 것이다.

수학적으로, Kepler의 목표는 다음과 같이 표현할 수 있다:

최소화:  $\max_{\theta \in \Theta} \text{cost}(\text{plan}, \text{parameter})$

여기서  $\Theta$ 는 가능한 모든 매개변수 값의 범위이고,  $\text{cost}(\text{plan}, \text{parameter})$ 는 특정 계획과 매개변수 조합에 대한 쿼리 실행 비용이다. 이는 최악의 경우 비용을 최소화하는 계획을 찾는 것을 의미한다.

이러한 접근은 직관적이지만 실무에서 구현하기 어렵다. 모든 가능한 매개변수 조합을 평가하는 것은 불가능하기 때문이다. Kepler는 다음과 같은 전략을 사용한다:

첫째, 매개변수 범위를 샘플링한다. 매개변수가 연속 범위인 경우, 균등하게 샘플링하거나, 특정 분포에 따라 샘플링한다.

둘째, 각 샘플에 대해 최적의 실행 계획을 결정한다. 이는 전통적인 쿼리 옵티마이저나 학습 기반의 방법을 사용할 수 있다.

셋째, 샘플된 모든 경우에서 좋은 성능을 제공하는 계획을 찾는다. 이는 다중 목표 최적화(multi-objective optimization) 문제로 볼 수 있다.

### 52.3 학습 기반의 계획 선택

Kepler는 머신러닝을 사용하여 매개변수로부터 실행 계획을 직접 예측한다. 입력은 매개변수 값(들)이고, 출력은 선택할 계획의 인덱스이다.

신경망 기반의 분류 모델을 사용하는 경우: - 입력층: 매개변수 값들 (정규화됨) - 숨은층: 매개변수와 계획 선택 간의 복잡한 관계를 학습 - 출력층: 각 가능한 계획에 대한 확률 (softmax 출력)

모델을 학습하기 위한 학습 데이터는 다음과 같이 수집된다: - 매개변수 샘플: 가능한 범위에서 여러 매개변수 조합을 생성 - 각 샘플에 대한 최적 계획: 전통적인 옵티마이저를 사용하거나, 모든 계획을 실행하여 가장 빠른 계획을 결정

학습 과정에서는 과적합을 방지하고, 강건성을 확보하기 위해 정규화와 데이터 증강(data augmentation)을 사용한다.

### 52.4 계획 안정성과 적응

Kepler는 단순히 각 매개변수 값에 대해 최적의 계획을 학습하는 것이 아니라, 계획의 안정성(stability)을 고려한다. 계획 안정성이란, 매개변수가 약간 변해도 동일한 계획을 선택하거나, 선택이 바뀌어도 성능 저하가 최소한이라는 의미이다.

이는 다음과 같은 이유로 중요하다: 1. 쿼리 플랜 캐싱: 동일한 계획을 캐싱하면, 옵티마이저 호출을 줄일 수 있다. 2. 성능 예측 가능성: 사용자는 안정적인 성능을 기대한다. 3. 계획 변경의 오버헤드: 계획이 자주 바뀌면, 캐시 효율성이 떨어진다.

Kepler는 계획 안정성을 향상시키기 위해 다음과 같은 기법을 사용한다: - 부드러운 경계(smooth boundaries): 두 계획 간의 전환이 급격하지 않도록 학습 - 히스테리시스(hysteresis): 계획을 바꾸기 전에 충분히 큰 성능 개선이 필요하도록 설정 - 신뢰도 임계값: 모델의 신뢰도가 낮을 때는 현재 계획 유지

## 52.5 실험과 성능 평가

Kepler의 실험은 여러 실제 데이터베이스 워크로드에서 수행된다. Join Order Benchmark (JOB)와 같은 표준 벤치마크도 사용된다.

성능 평가는 다음과 같은 지표를 사용한다: - 평균 쿼리 실행 시간: 모든 매개변수 조합에 대한 평균 - 최악의 경우 실행 시간: 가장 느린 매개변수 조합 - 계획 안정성: 계획 변경 빈도와 각 변경 시 성능 변화 - 학습 비용: 학습 데이터 수집과 모델 학습에 소요되는 시간

실험 결과에 따르면: - 평균적으로 전통적 옵티마이저보다 20~30% 더 빠름 - 최악의 경우 성능이 더욱 개선되어, 전통적 옵티마이저보다 50% 이상 빠를 수 있음 - 계획 안정성도 개선되어, 계획 변경 빈도가 50% 이상 감소

## 52.6 본 논문과의 관계

Exqutor는 카디널리티 추정에 초점을 맞추는 반면, Kepler는 학습된 계획의 강건성에 초점을 맞춘다. 하지만 두 작업은 공통점이 있다:

첫째, 둘 다 머신러닝을 사용하여 쿼리 최적화를 개선한다.

둘째, 둘 다 실무적 워크로드의 복잡성을 다룬다. Exqutor는 유사성 쿼리의 복잡한 카디널리티 추정을 다루고, Kepler는 매개변수 쿼리의 복잡한 최적화를 다룬다.

셋째, 둘 다 학습 모델의 강건성(robustness)을 고려한다. Exqutor는 분포 이동에 강건한 추정을 제공하려 하고, Kepler는 매개변수 변화에 강건한 계획 선택을 제공한다.

---

## 추가 제기 문제

- 매개변수 범위의 정의:** 매개변수의 가능한 범위를 어떻게 정의할 것인가? 실제로는 매개변수가 제약 없이 변할 수 있지만, 학습은 제한된 범위에서만 수행된다.
  - 다중 매개변수:** 여러 매개변수가 결합되었을 때 (예: WHERE col1 > ? AND col2 < ?), 학습 데이터의 차원이 급격히 증가한다 (차원의 저주).
  - 새로운 데이터 패턴:** 데이터가 변해서 이전의 최적 계획이 더 이상 최적이지 아닐 때, 모델을 언제 어떻게 재학습할 것인가?
  - 해석가능성:** 모델이 특정 매개변수 값에 대해 특정 계획을 추천한 이유를 설명할 수 있는가?
-



## [53] Exact Cardinality Query Optimization for Optimizer Testing

### 요약

본 논문은 Chaudhuri, Narasayya, Ramamurthy가 2009년 VLDB에 발표한 논문으로, ECQO (Exact Cardinality Query Optimization) 개념의 원점이다. 비록 [50] 논문 (Trummer 2019)이 ECQO의 일반화된 형태를 제시했지만, 이 논문이 원래의 개념을 도입했다.

논문의 핵심은 쿼리 옵티마이저를 테스트하기 위해, 정확한 카디널리티 정보를 제공하면서도 계산 비용이 제한된 방법을 제시하는 것이다. 원래의 목표는 데이터베이스 옵티마이저의 성능을 평가하고 개선하는 것이었다. 만약 옵티마이저에게 정확한 카디널리티를 제공하면, 옵티마이저가 생성하는 계획의 품질을 판단할 수 있다.

구체적으로, 어떤 쿼리에 대해 옵티마이저가 선택한 실행 계획이 좋은지 나쁜지를 판단하려면, 정확한 카디널리티를 알아야 한다. 전통적인 추정 카디널리티를 사용하면, 추정의 부정확성 때문에 옵티마이저의 성능을 공정하게 평가할 수 없다.

이 논문의 주요 기여는 다음과 같다. 첫째, 부분 쿼리 실행을 통해 정확한 카디널리티를 얻을 수 있음을 보여주었다. 둘째, 이를 통해 옵티마이저 테스트 환경에서 실제 최적 계획과 옵티마이저가 선택한 계획을 비교할 수 있음을 제시했다.

본 논문은 Exqutor와 직접적인 관계가 있다. Exqutor는 이 원점 논문의 개념을 계승하면서도, 카디널리티 추정을 더욱 효율적으로 만들기 위해 머신러닝을 도입한다.

### 상세분석

#### 53.1 옵티마이저 평가의 도전

데이터베이스 옵티마이저의 성능을 평가하는 것은 복잡한 문제다. 옵티마이저가 선택한 실행 계획의 좋고 나쁨을 판단하려면, 다른 가능한 계획들과 비교해야 한다. 하지만 모든 가능한 실행 계획을 비교하는 것은 계산적으로 불가능하다.

전통적인 벤치마킹 방식은 표준 워크로드(예: TPC-H)를 사용하여, 여러 옵티마이저의 성능을 비교하는 것이다. 그러나 이 방식에는 근본적인 문제가 있다.

첫째, 옵티마이저가 생성한 계획이 느린 경우, 그 원인이 옵티마이저의 부실한 판단인지, 아니면 카디널리티 추정의 부정확함 때문인지 구분하기 어렵다.

둘째, 만약 카디널리티 추정이 완벽하다면 옵티마이저가 정말 최적의 계획을 선택했을 텐데, 이를 알 수 없다.

셋째, 특정 쿼리 패턴에서 옵티마이저의 약점을 파악하기 어렵다.

### 53.2 정확한 카디널리티를 통한 평가 방법

이 논문이 제시하는 해결책은 다음과 같다: 쿼리 옵티마이저에게 정확한 카디널리티를 제공하면, 옵티마이저의 실제 성능을 평가할 수 있다.

구체적 방법은 다음과 같다:

1단계: 정상적인 옵티마이저 사용 - 옵티마이저는 카디널리티 추정을 사용하여 실행 계획을 생성 - 생성된 계획을  $P_{est}$ 라고 하자

2단계: 정확한 카디널리티를 사용한 옵티마이저 재실행 - 각 중간 결과의 정확한 카디널리티를 쿼리 실행을 통해 파악 - 이 정확한 카디널리티를 옵티마이저에게 제공 - 옵티마이저는 이를 사용하여 실행 계획을 재생성 - 생성된 계획을  $P_{exact}$ 라고 하자

3단계: 비교 -  $P_{est}$ 와  $P_{exact}$ 의 성능을 비교 - 만약  $P_{est}$ 가  $P_{exact}$ 와 동일하면, 옵티마이저가 카디널리티 부정확성에도 불구하고 좋은 결정을 했다는 뜻 - 만약  $P_{exact}$ 가 훨씬 빠르면, 옵티마이저가 카디널리티 추정 오류로 인해 나쁜 계획을 선택했다는 뜻

### 53.3 카디널리티 결정의 효율적 방법

정확한 카디널리티를 얻기 위해 모든 중간 결과를 완전히 계산하는 것은 비효율적이다. 논문은 더 효율적인 방법을 제시한다.

첫째, 샘플링 기반 추정이다. 데이터를 균등하게 샘플링하여, 샘플의 결과 크기로부터 전체 카디널리티를 추정한다. 이는 정확하지는 않지만, 부분 실행 방식보다 빠르다.

둘째, 인덱스 기반 카디널리티 추정이다. 만약 인덱스가 있으면, 인덱스를 순회하여 조건을 만족하는 항목의 개수를 빠르게 셀 수 있다. 예를 들어, B-tree 인덱스에서 범위 조건 ( $col > 100$ )을 만족하는 항목은 인덱스 노드의 메타데이터(노드 내 항목 개수 등)를 사용하여 빠르게 카운트할 수 있다.

셋째, 확률적 자료구조를 사용한 추정이다. 블룸 필터나 카운트-민 스케치 같은 자료구조를 사용하면, 매우 적은 메모리로 근사적 카디널리티를 빠르게 계산할 수 있다.

### 53.4 실험적 평가

이 논문은 Microsoft SQL Server 옵티마이저를 대상으로 실험을 수행했다. Join Order Benchmark의 쿼리들을 사용하여, 다음을 평가했다:

첫째, 카디널리티 추정 오류의 영향 - 추정된 카디널리티: 평균 5배에서 10배 오류 - 실제 최적 계획과 옵티마이저가 선택한 계획의 성능 차이: 최대 100배 이상

둘째, 정확한 카디널리티를 사용했을 때의 개선 - 정확한 카디널리티를 사용하면, 옵티마이저가 훨씬 더 좋은 계획을 선택 - 대부분의 경우 최적에 가까운 계획 선택 - 옵티마이저의 논리적 오류(버그)로 인한 성능 저하도 식별 가능

셋째, 카디널리티 결정의 비용 - 전체 쿼리 실행 시간의 10~30% 정도의 추가 비용으로 정확한 카디널리티 획득 가능 - 이는 옵티마이저 재실행을 통한 성능 개선 (30~50%)보다 작음

### 53.5 옵티마이저 테스트에의 활용

이 방법의 주요 응용은 옵티마이저 테스트이다. 소프트웨어 개발 관점에서, 옵티마이저는 복잡한 소프트웨어 시스템이므로 버그가 존재할 수 있다.

이 방법을 사용하면: 1. 옵티마이저의 논리적 오류 검출: 카디널리티가 정확해도 옵티마이저가 나쁜 계획을 선택하면, 그것은 옵티마이저의 버그 2. 옵티마이저의 휴리스틱(heuristic)의 유효성 검증: 옵티마이저가 사용하는 비용 모델이 현실을 잘 반영하는지 확인 3. 최적화 기회의 식별: 옵티마이저가 특정 쿼리 패턴에서 성능이 떨어지는 경우 식별

### 53.6 본 논문과 Exqutor의 관계

[53] 논문 (Chaudhuri et al. 2009)은 ECQO의 원래 개념을 제시했다. [50] 논문 (Trummer 2019)은 이를 일반적인 쿼리 최적화 문제로 확대했고, Exqutor는 이를 유사성 쿼리의 카디널리티 추정이라는 구체적인 문제로 특화시켰다.

논문들의 진화: - [53] (2009): 옵티마이저 테스트를 위한 부분 실행 기반 정확한 카디널리티 - [50] (2019): 부분 실행을 통한 일반적인 쿼리 최적화 (ECQO) - Exqutor: 머신러닝을 사용한 유사성 쿼리의 카디널리티 추정

각 단계별로 다음과 같은 발전이 있었다: 1. 개념 제시: 정확한 카디널리티의 가치 (Chaudhuri et al.) 2. 일반화: 부분 실행을 통한 실시간 카디널리티 결정 (Trummer) 3. 실용화: 머신러닝을 사용한 효율적이고 확장 가능한 추정 (Exqutor)

---

## 추가 제기 문제

1. **효율성 트레이드오프**: 정확한 카디널리티 결정의 비용과 이로 인한 성능 개선의 이득 사이에 항상 긍정적인 트레이드오프가 있는가?
  2. **부분 실행의 신뢰성**: 부분 실행(예: 10% 샘플)으로부터 추정한 카디널리티가 실제 전체 데이터의 카디널리티를 정확히 반영하는가?
  3. **동적 데이터 변화**: 데이터가 지속적으로 변하는 환경에서, 어떻게 카디널리티 정보를 유지할 것인가?
  4. **옵티마이저 개선**: 이 방법으로 옵티마이저의 버그를 발견할 수 있지만, 실제로 옵티마이저를 개선하는 방법은 무엇인가?
-

## [54] Analyzing Query Optimizer Performance in the Presence and Absence of Cardinality Estimates

### 요약

본 논문은 Datta 등이 2023년 arXiv에 발표한 논문으로 (arXiv:2311.17293), 카디널리티 추정이 쿼리 옵티마이저의 성능에 미치는 영향을 체계적으로 분석한다. 논문의 핵심은 카디널리티 추정 오류가 실행 계획 선택에 어떻게 영향을 미치는지, 그리고 정확한 카디널리티 정보를 제공했을 때 성능이 얼마나 개선되는지를 정량적으로 평가하는 것이다.

이 연구는 실증적(empirical) 성격의 논문이다. 학습 기반 방법이나 새로운 알고리즘을 제시하기보다는, 기존 데이터베이스 시스템(특히 PostgreSQL)에서 카디널리티 추정이 성능에 미치는 영향을 깊이 있게 분석한다. 이를 통해 다음과 같은 중요한 질문에 답한다:

1. 카디널리티 추정 오류의 심각성은?
2. 카디널리티 추정 오류가 성능 저하를 유발하는 메커니즘은?
3. 정확한 카디널리티 정보로 인한 성능 개선 가능성은?

이러한 분석은 Exquitor와 같은 카디널리티 추정 연구의 실제 가치를 입증한다. 즉, 카디널리티 추정을 개선하는 것이 단순히 추정 오류를 줄이는 것을 넘어서, 실제 쿼리 성능에 어떤 영향을 미치는지 보여준다.

### 상세분석

#### 54.1 카디널리티 추정 오류의 영향 메커니즘

카디널리티 추정 오류가 쿼리 성능에 영향을 미치는 경로는 다음과 같다:

첫째, 조인 순서 선택이다. 조인 쿼리에서 여러 테이블을 조인할 때, 조인 순서에 따라 중간 결과의 크기가 달라진다. 예를 들어: - A JOIN B JOIN C에서 - (A JOIN B) JOIN C 순서와 (B JOIN A) JOIN C 순서에서의 중간 결과 크기가 다름

옵티마이저는 카디널리티 추정을 기반으로 조인 순서를 결정한다. 만약 카디널리티 추정이 부정확하면, 최악의 조인 순서를 선택할 수 있다. 특히 여러 테이블의 조인에서는 추정 오류가 누적되어 매우 나쁜 순서를 선택할 수 있다.

둘째, 조인 방법 선택이다. 각 조인에 대해 여러 조인 알고리즘(Nested Loop Join, Hash Join, Merge Join 등)을 사용할 수 있다. 각 알고리즘의 효율성은 입력 크기에 따라 달라진다: - Nested Loop Join: 외부 입력이 작을 때 효율적 - Hash Join: 해시 테이블이 메모리에 들어갈 때 효율적 - Merge Join: 입력이 정렬되어 있을 때 효율적

잘못된 카디널리티 추정은 부적절한 조인 알고리즘 선택을 유발한다. 예를 들어, 입력이 실제로 크지만 크다고 추정되지 않으면, 비효율적인 Nested Loop Join을 선택할 수 있다.

셋째, 물리 연산자(physical operator)의 선택이다. 스캔 방법(풀 테이블 스캔 vs. 인덱스 스캔), 정렬 방법(메모리 정렬 vs. 디스크 정렬 등)의 선택이 카디널리티에 따라 달라진다.

## 54.2 실험 설계와 데이터셋

논문의 실험은 체계적으로 설계되었다. PostgreSQL을 대상 데이터베이스로 사용하고, 여러 벤치마크 워크로드를 사용한다:

첫째, Join Order Benchmark (JOB)이다. 이는 IMDB 영화 데이터베이스를 기반으로 한 벤치마크로, 복잡한 조인 쿼리들을 포함한다. JOB의 쿼리들은 매우 도전적인 것으로 알려져 있으며, 옵티마이저들이 종종 나쁜 계획을 선택한다.

둘째, TPC-H 및 TPC-DS 벤치마크이다. 이들은 비즈니스 쿼리 처리를 시뮬레이션하는 표준 벤치마크이다.

셋째, 합성 데이터(synthetic data)이다. 논문은 다양한 데이터 분포(균등, 스쿼드, 클러스터링 등)를 가진 합성 데이터도 생성하여 사용한다.

## 54.3 카디널리티 추정 오류의 정량화

논문은 추정 오류를 여러 방식으로 정량화한다:

첫째, q-error이다. 이미 언급했듯이, q-error는 예측값과 실제값의 최대 비율을 나타낸다. 논문의 실험에서 PostgreSQL의 카디널리티 추정 오류는: - 중간값(median): 1.5배 - 평균값: 약 10배 - 95 백분위수: 수십 배에서 수백 배

둘째, 오류의 방향(direction)이다. 추정값이 실제값보다 큰 과다 추정(overestimation)인지, 아니면 과소 추정(underestimation)인지는 다른 영향을 미친다: - 과다 추정: Nested Loop Join 선택, 비효율적인 연산자 선택 - 과소 추정: 인덱스 사용을 피함, 메모리 부족 상황 유발

셋째, 오류의 누적이다. 여러 단계의 조인에서 각 단계의 추정 오류가 누적되어, 전체 오류가 매우 심각해질 수 있다.

## 54.4 성능 영향 분석

논문의 핵심 결과는 카디널리티 추정 오류가 실제 쿼리 성능에 미치는 영향을 정량화한 것이다:

첫째, 극단적 성능 저하이다. 일부 쿼리에서는 부정확한 카디널리티 추정으로 인해 실행 시간이 1000배 이상 증가한다. 이는 몇 밀리초에 완료되어야 할 쿼리가 초 단위로 실행된다는 뜻이다.

둘째, 시스템 수준의 영향이다. 한두 개의 느린 쿼리는 전체 시스템의 처리량(throughput)에 미치는 영향이 크다. 멀티테넌트 환경이나 OLTP 시스템에서는 이러한 느린 쿼리가 다른 쿼리들을 블로킹할 수 있다.

셋째, 성능 개선의 가능성이다. 정확한 카디널리티 정보를 옵티마이저에게 제공하면: - 평균 성능: 5배 개선 - 최악의 경우: 100배 이상 개선 가능

## 54.5 카디널리티 추정 오류의 원인

논문은 PostgreSQL의 카디널리티 추정이 부정확한 이유를 분석한다:

첫째, 통계의 부정확성이다. PostgreSQL은 열(column)별로 히스토그램을 유지하지만: - 히스토그램의 버킷 수가 제한적 (보통 100) - 극값(extreme values)이 제대로 표현되지 않음 - 열 간의 상관관계가 고려되지 않음

둘째, 독립성 가정이다. 여러 조건이 결합될 때, 옵티마이저는 각 조건이 독립적이라고 가정하고 확률을 곱한다. 실제로는 많은 조건들이 상관관계가 있다.

셋째, 복잡한 쿼리의 어려움이다. 많은 조인이 포함된 복잡한 쿼리에서는 각 단계의 추정 오류가 누적된다.

## 54.6 본 논문과의 관계

이 논문은 Exqutor의 동기와 중요성을 입증하는 역할을 한다. 구체적으로:

첫째, 카디널리티 추정이 중요함을 보여준다. 카디널리티 추정은 단순히 통계적 흥미로운 문제가 아니라, 실제 데이터베이스 성능에 극적인 영향을 미치는 중요한 문제다.

둘째, 기존 방법의 한계를 명확히 한다. 전통적인 통계 기반 추정의 평균 10배 오류는 시스템 수준에서 용인 불가능한 수준이다.

셋째, 새로운 접근 방식의 필요성을 강조한다. 머신러닝 기반의 카디널리티 추정 (예: SelNet, Exqutor)이 이러한 문제를 해결할 수 있음을 시사한다.

Exqutor의 맥락에서 보면, 이 논문의 분석은 유사성 쿼리의 카디널리티 추정이 얼마나 중요한지를 강조한다. 벡터 데이터베이스 시스템도 유사한 문제를 가지고 있을 것이고, Exqutor의 머신러닝 기반 접근이 이를 해결할 수 있음을 시사한다.

---

## 추가 제기 문제

1. **적응형 쿼리 실행:** 실행 중에 카디널리티 추정을 수정하고 계획을 변경할 수 있는 적응형(adaptive) 쿼리 처리의 이점은?
  2. **통계 정확도의 유지:** 데이터가 지속적으로 변하는 환경에서, 통계를 정확하게 유지하는 방법은?
  3. **학습 기반 방법의 효과:** 이 논문의 발견이 머신러닝 기반 카디널리티 추정의 필요성을 얼마나 강력하게 지지하는가?
  4. **종합적 해결책:** 카디널리티 추정만으로 충분한가, 아니면 비용 모델(cost model)의 개선도 동시에 필요한가?
-

## [55] Analyzing the Impact of Cardinality Estimation on Execution Plans in Microsoft SQL Server

### 요약

본 논문은 Lee, Dutt, Narasayya, Chaudhuri가 2023년에 발표한 논문으로, Microsoft SQL Server를 대상으로 카디널리티 추정이 실행 계획에 미치는 영향을 심층적으로 분석한다. 논문은 이전 일반적 분석 [54]과 달리, 특정 상용 데이터베이스 시스템에 초점을 맞춘다.

Microsoft SQL Server는 업계에서 가장 널리 사용되는 데이터베이스 시스템 중 하나이며, SQL Server 자체도 고도로 복잡한 쿼리 옵티마이저를 가지고 있다. 이 논문은 SQL Server의 specific한 카디널리티 추정 메커니즘과 그 영향을 분석한다.

논문의 핵심 기여는 다음과 같다. 첫째, SQL Server 특화의 카디널리티 추정 오류 패턴을 식별한다. 둘째, 이러한 오류가 실행 계획 선택에 미치는 구체적인 영향을 정량화한다. 셋째, SQL Server의 성능 튜닝을 위한 실용적인 권장사항을 제시한다.

특히 이 논문은 새로운 통계 기반 카디널리티 추정 모델(SQL Server 2019에서 도입된 Cardinality Estimation (CE) 모델)과 레거시 모델을 비교하여, 개선의 정도와 한계를 명확히 한다.

### 상세분석

#### 55.1 SQL Server의 카디널리티 추정 메커니즘

Microsoft SQL Server의 카디널리티 추정은 몇 가지 기본 원리에 기반한다:

첫째, 히스토그램 기반 추정이다. SQL Server는 각 열에 대해 히스토그램을 유지한다. 히스토그램의 각 버킷은 일정 범위의 값들을 나타내고, 각 버킷의 행 수를 저장한다. 범위 쿼리 (WHERE col > 100 AND col < 500)가 주어지면, 해당 범위에 포함된 버킷들의 행 수를 합산하여 카디널리티를 추정한다.

둘째, 독립성 가정이다. 여러 조건이 WHERE 절에 있을 때, 각 조건이 독립적이라고 가정한다. 예를 들어: WHERE col1 > 100 AND col2 < 50 이 경우, col1 > 100의 선택도와 col2 < 50의 선택도를 곱한다.

셋째, 조인의 동등성 가정이다. A JOIN B ON A.id = B.id에서, A의 각 행이 B에서 정확히 하나의 행과 매칭된다고 가정한다. 실제로는 중복 조인(duplicate join)이나 부분 조인(partial join)이 발생할 수 있다.

SQL Server는 여러 버전의 카디널리티 추정 모델을 제공한다: - 레거시 CE (Cardinality Estimation): 구 모델로, 단순한 독립성 가정 사용 - 신규 CE (Cardinality Estimation for SQL Server 2014 이상): 더 정교한 모델로, 일부 상관관계 고려

#### 55.2 카디널리티 추정 오류의 영역별 분석

논문은 SQL Server에서 카디널리티 추정 오류가 심각한 영역들을 식별한다:

첫째, 다중 조건 쿼리이다. WHERE col1 > 100 AND col2 > 200 AND col3 > 300 같은 쿼리에서 여러 조건의 선택도를 곱하면, 오류가 지수적으로 증가한다. 예를 들어, 각 조건의 추정 오류가 2배씩이면, 3개 조건의 조합은 8배 오류가 발생한다.

둘째, 조인 쿼리이다. 여러 테이블 조인에서: - 조인 순서 결정 단계에서의 오류 - 각 조인 후 결과 카디널리티 추정의 오류 누적 - 특히 조인 선택도(join selectivity) 추정의 어려움

셋째, 특정 데이터 패턴이다: - 스튜드 데이터: 특정 값이 과다 대표될 때, 균등 분포 가정이 깨짐 - 클러스터링된 데이터: 데이터가 특정 범위에 밀집되어 있을 때, 히스토그램 기반 추정이 부정확 - 시간 시계열 데이터: 시간에 따라 데이터 분포가 변할 때, 이전 통계가 현재 데이터를 반영하지 못함

### 55.3 신규 CE vs 레거시 CE의 비교

논문은 SQL Server 2019에서 도입된 신규 CE와 레거시 CE를 비교한다:

레거시 CE의 문제점: - 독립성 가정의 극단적 적용 - 매우 작은 결과 크기의 과소 추정 (0으로 반올림) - 매우 큰 결과 크기의 과다 추정 - 조인 선택도의 임의적 상수값 사용

신규 CE의 개선: - 기본 독립성 가정은 유지하되, 일부 상관관계 고려 - 결과 크기 0인 경우의 특별 처리 - 조인 선택도의 더 정교한 추정 - 특정 패턴(예: 범위 쿼리)에 대한 특별 케이스 처리

성능 비교: - 대부분의 쿼리에서 신규 CE가 더 정확한 추정 제공 - 평균적으로 추정 오류를 30~40% 감소 - 하지만 특정 패턴의 쿼리에서는 여전히 큰 오류 (10배 이상)

### 55.4 실행 계획에 대한 영향

카디널리티 추정의 부정확성이 실행 계획에 구체적으로 어떻게 영향을 미치는지 분석:

첫째, 조인 순서 변경이다. 잘못된 카디널리티 추정으로 인해: - A JOIN B JOIN C가 (A JOIN C) JOIN B로 변경 - (A JOIN B) JOIN C가 A JOIN (B JOIN C)로 변경 - 조인 순서에 따라 성능이 10배 이상 차이

둘째, 조인 알고리즘 변경이다: - Nested Loop Join에서 Hash Join으로 변경 (메모리 사용량 극적 증가) - Hash Join에서 Merge Join으로 변경 (정렬 비용 추가) - 부적절한 알고리즘 선택으로 인한 성능 저하

셋째, 병렬 처리 결정 변경이다: - 단일 스레드 실행에서 다중 스레드 병렬 실행으로 변경 - 병렬 처리의 오버헤드가 이득을 초과하는 경우 발생 - 메모리 할당량의 부정확한 결정

### 55.5 성능 튜닝을 위한 실무적 권장사항

논문은 SQL Server 사용자들을 위한 실용적인 권장사항을 제시한다:

첫째, 통계 유지이다: - 정기적으로 통계 업데이트 (UPDATE STATISTICS) - 특히 큰 데이터 변화 후에는 즉시 통계 업데이트 - 통계 샘플링 비율 적절하게 설정 (FULLSCAN vs 샘플링)

둘째, 쿼리 작성 방식이다: - 복잡한 단일 쿼리보다는 단계별 쿼리로 분해 - 조인 순서를 INNER JOIN 구문으로 명시하기보다는, 옵티마이저에게 결정 위임 (옵티마이저가 더 정교함) - 서브쿼리 대신 CTE(Common Table Expression) 사용



셋째, 힌트(hint) 사용이다: - 옵티마이저가 부정확한 결정을 반복적으로 하는 경우, 힌트를 사용하여 강제로 계획 지정 - 하지만 힌트는 임시 해결책이므로, 근본 원인(통계 부정확성 등) 해결 권장

넷째, 쿼리 모니터링이다: - 예상 행 수(Estimated Rows)와 실제 행 수(Actual Rows)를 비교하여 추정 오류 식별 - Actual Rows가 Estimated Rows보다 훨씬 크면, 계획 재컴파일 강제 실행 고려

## 55.6 새로운 통계 모델의 제안

논문은 SQL Server의 카디널리티 추정을 더욱 개선하기 위한 방향을 제시한다:

첫째, 열 간 상관관계 추정이다: - 현재의 단일 열 히스토그램을 넘어, 여러 열 간의 상관관계 학습 - 다변량 히스토그램(multivariate histogram)이나 머신러닝 기반 모델 사용

둘째, 적응형 카디널리티 추정이다: - 실제 쿼리 실행 결과를 기반으로, 옵티마이저가 실시간으로 추정을 조정 - 다음 유사 쿼리를 위해 학습된 정보 활용

셋째, 머신러닝 기반 접근이다: - 역사적인 쿼리 실행 데이터로부터 카디널리티 추정 모델 학습 - 새로운 쿼리에 대해 학습된 모델을 사용하여 추정

## 55.7 본 논문과의 관계

이 논문은 특정 상용 데이터베이스 시스템 (SQL Server)에 초점을 맞춘 실증적 분석이다. Exqutor의 맥락에서 보면:

첫째, 실무적 중요성을 강조한다. SQL Server는 업계에서 가장 중요한 데이터베이스 시스템 중 하나이므로, 이 시스템에서 카디널리티 추정의 문제점을 명확히 한 것은 매우 의미 있다.

둘째, 기존 방법의 한계를 보여준다. SQL Server의 신규 CE도 여전히 많은 경우에 10배 이상의 오류를 보이므로, 더욱 정교한 방법이 필요함을 시사한다.

셋째, 벡터 데이터베이스 시스템으로의 영감을 제공한다. SQL Server 같은 관계형 데이터베이스와 유사하게, 벡터 데이터베이스도 유사성 쿼리의 카디널리티 추정에 어려움을 겪을 수 있다. Exqutor는 이러한 문제를 해결하는 방법을 제시한다.

## 추가 제기 문제

1. **일반화 가능성:** SQL Server에 대한 이 분석이 다른 데이터베이스 시스템 (PostgreSQL, MySQL, Oracle 등) 에도 유사하게 적용되는가?
2. **통계의 비용:** 정확한 통계를 유지하는 비용이 추정 개선으로부터의 이득을 정당화하는가?
3. **적응형 솔루션:** 실행 중에 추정을 수정하고 계획을 재컴파일하는 비용과 이득 간의 트레이드오프는?
4. **머신러닝의 필요성:** 이 논문의 발견이 머신러닝 기반 카디널리티 추정 (예: Exqutor)의 필요성을 얼마나 강력하게 지지하는가?

---

## [56] Robust Query Processing through Progressive Optimization

### 요약

본 논문은 Markl 등이 SIGMOD 2004에 발표한 논문으로, 실행 중(runtime)에 쿼리 계획을 동적으로 적응시키는 진보적/적응형 쿼리 최적화(progressive/adaptive query optimization)의 개념을 제시한다. 논문의 핵심은 카디널리티 추정 오류에 대비하여, 실행 중에 실제 데이터를 관찰하면서 계획을 재최적화하는 것이다.

쿼리 최적화와 실행의 전통적인 분리를 넘어, Markl은 최적화와 실행을 통합하는 접근을 제시한다. 구체적으로, 쿼리의 일부가 실행되는 동안 정확한 중간 결과 카디널리티가 파악되면, 이를 사용하여 나머지 계획을 재최적화하는 방식이다.

이 접근법은 매우 실용적이며, 특히 카디널리티 추정의 부정확성이 심한 환경에서 매우 효과적이다. 예를 들어, 매우 큰 조인 쿼리에서 첫 번째 조인까지 실행한 후, 정확한 결과 카디널리티를 알면, 이를 기반으로 나머지 조인의 순서를 동적으로 최적화할 수 있다.

Progressive optimization은 Exqutor가 다루는 정적 카디널리티 추정 문제와 보완 관계에 있다. 정확한 카디널리티 추정으로 처음부터 좋은 계획을 선택하는 것이 이상적이지만, 완전히 정확한 추정이 불가능하다면, progressive optimization으로 실행 중에 보정하는 것이 실용적인 대안이 된다.

---

### 상세분석

#### 56.1 전통적 쿼리 처리와 그 문제점

전통적인 쿼리 처리 파이프라인은 다음과 같이 구성된다:

1. 파싱(Parsing): SQL 문을 구문 분석
2. 검증(Validation): 테이블, 열 등의 존재 확인
3. 최적화(Optimization): 최적의 실행 계획 선택
4. 컴파일(Compilation): 실행 계획을 실행 가능한 형태로 변환
5. 실행(Execution): 실제 데이터에 대해 계획 실행

이 구조에서 최적화(3단계)와 실행(5단계) 사이에는 명확한 경계가 있다. 최적화 시점(optimization time)에는 카디널리티 추정값만 알 수 있고, 실행 시점(execution time)에 실제 카디널리티가 파악된다.

이 분리의 문제점은 다음과 같다:

첫째, 추정 오류의 누적이다. 여러 조인이 있는 복잡한 쿼리에서, 각 단계의 추정 오류가 누적되어 최종 계획의 품질이 심각하게 저하될 수 있다.

둘째, 최악의 경우 대비이다. 옵티마이저는 일반적으로 평균적인 경우를 가정하여 계획을 선택한다. 하지만 실제 실행 시 매개변수나 데이터 분포가 예상과 다를 수 있다.

셋째, 하드 타임아웃(hard timeout)이다. 일부 쿼리는 선택된 계획이 너무 비효율적이어서, 합리적인 시간 내에 완료될 수 없을 수도 있다. 이 경우 쿼리는 무한정 실행되거나 강제 종료된다.

## 56.2 Progressive Optimization의 개념

Progressive optimization은 이 문제를 해결하기 위해 최적화와 실행 사이에 피드백 루프를 도입한다. 기본 아이디어는 다음과 같다:

1. 초기 계획 생성: 전통적인 방식으로 초기 실행 계획을 생성
2. 부분 실행: 계획의 일부 (예: 첫 번째 조인)를 실행
3. 중간 결과 모니터링: 부분 실행의 결과 카디널리티를 측정
4. 계획 재최적화: 측정된 정확한 카디널리티를 사용하여 나머지 계획을 재최적화
5. 계획 조정: 필요하면 실행 계획 변경
6. 계속 실행: 조정된 계획으로 계속 실행

이를 통해 초기 추정이 부정확해도, 실행 중에 실제 데이터를 기반으로 계획을 보정할 수 있다.

## 56.3 Adaptive Query Processing의 기술들

Markl은 progressive optimization을 구현하기 위한 여러 기술을 제시한다:

첫째, 연관 해시 조인(adaptive hash join)이다. 조인 중에 해시 테이블의 메모리 사용량을 모니터링하여: - 메모리 부족이 발생하면, 디스크 스푼(spill) 시작 - 조인이 예상보다 훨씬 많은 결과를 생성하면, 조인 알고리즘 변경 고려

둘째, 동적 계획 변경이다. 조인 순서를 사전에 고정하지 않고: - 첫 번째 조인 A JOIN B를 실행하여 정확한 결과 크기 파악 - (A JOIN B) JOIN C의 비용을 정확한 크기로 재계산 - 다른 순서 (A JOIN C) JOIN B의 비용과 비교하여 더 나은 순서 선택

셋째, 적응형 필터링(adaptive filtering)이다. 조건 적용 순서를 동적으로 변경: - 실제 선택도(selectivity)에 따라 조건의 적용 순서 재조정 - 높은 선택도의 조건을 먼저 적용하여 데이터를 빠르게 감소

넷째, 조건부 계획 변경이다. 쿼리 실행 중 특정 조건이 만족되면: - 예정된 계획을 변경하고 대체 계획 실행 - 예: "만약 현재까지 1000개 행이 나왔으면, 계획 X 실행, 아니면 계획 Y 실행"

## 56.4 구현상의 과제

Progressive optimization을 실제로 구현하는 것은 여러 도전을 제시한다:

첫째, 오버헤드이다. 계획을 재최적화하는 과정 자체도 비용이 든다. 너무 자주 재최적화하면, 그 비용이 이득을 초과할 수 있다. 따라서 재최적화의 시점과 빈도를 신중하게 결정해야 한다.

둘째, 실행 계획의 복잡성이다. 전통적인 실행 계획은 선형적인 구조를 가진다 (위에서 아래로 순차 실행). 하지만 progressive optimization에서는 조건부 분기, 피드백 루프 등이 포함되어, 계획 구조가 훨씬 복잡해진다.

셋째, 메모리 관리이다. 계획 재최적화 중에도 이전 단계의 중간 결과가 메모리에 유지되어야 한다. 메모리 제약이 심한 환경에서는 어려움이 있다.

넷째, 캐싱과의 충돌이다. 쿼리 계획을 캐싱하여 재사용하는 시스템에서, 계획이 동적으로 변경되면 캐싱의 효율성이 떨어진다.

## 56.5 실험과 성능 평가

논문의 실험은 Markl이 개발한 DadaBase 시스템 (또는 유사한 프로토타입)을 사용한다:

성능 개선 결과: - 평균적으로 전통적 최적화보다 20~30% 더 빠름 - 카디널리티 추정이 부정확한 쿼리에서는 5배 이상 개선 - 최악의 경우 성능 저하 거의 없음 (재최적화 오버헤드가 최소함)

안정성 개선: - 추정 오류에 강건함 (robust) - 매우 큰 쿼리에서도 합리적인 시간 내 완료

트레이드오프: - 재최적화 오버헤드로 인한 작은 추가 비용 - 더 복잡한 실행 계획 구조로 인한 구현 복잡성 증가

## 56.6 관련 후속 연구

Progressive optimization은 다양한 후속 연구를 자극했다:

첫째, 재최적화의 최적 시점 결정이다. 언제 재최적화를 수행할지를 정하는 문제로, 기계학습을 사용하여 의사결정할 수 있다.

둘째, 분산 환경에서의 적응형 처리이다. MapReduce나 Spark 같은 분산 처리 시스템에서도 비슷한 아이디어를 적용할 수 있다.

셋째, 스트림 처리이다. 데이터가 계속 들어오는 스트리밍 환경에서는 처음부터 정확한 카디널리티를 알 수 없으므로, progressive approach가 매우 적합하다.

## 56.7 본 논문과의 관계

Progressive optimization과 Exqutor는 상호 보완적인 접근을 대표한다:

**Exqutor (정적 최적화 관점):** - 쿼리 최적화 시점에 정확한 카디널리티 추정 제공 - 초기에 좋은 계획을 선택하는 것이 목표 - 런타임 오버헤드가 없음

**Progressive Optimization (동적 최적화 관점):** - 초기 추정이 부정확하더라도, 실행 중에 보정 - 실시간 모니터링을 통한 적응 - 런타임 오버헤드 존재

**통합된 시스템의 가능성:** 1. Exqutor를 사용하여 초기에 좋은 계획 선택 2. 실행 중에 progressive optimization을 사용하여 추가 보정 3. 두 접근법이 함께 작동하여, 정확한 추정과 실행 중 적응을 모두 달성

이러한 통합은 벡터 데이터베이스 시스템에서도 적용 가능하다. Exqutor의 머신러닝 기반 카디널리티 추정과 progressive optimization을 결합하면, 유사성 쿼리의 처리를 더욱 강건하고 효율적으로 만들 수 있다.

## 추가 제기 문제

1. **재최적화 오버헤드의 균형**: 재최적화의 이득과 오버헤드를 정확하게 추정하고, 재최적화를 수행할지 여부를 자동으로 결정하는 방법은?
2. **실시간 적응의 한계**: 실행 중에 계획을 완전히 변경할 수 있는가, 아니면 부분적인 조정만 가능한가?
3. **복잡성 증가**: Progressive optimization의 구현 복잡성이 실제 구현 가능한 수준인가?
4. **캐싱과의 호환성**: 계획 캐싱과 동적 계획 변경을 어떻게 조화시킬 것인가?
5. **통합된 최적화**: Exqutor 같은 정확한 카디널리티 추정과 progressive optimization을 어떻게 효과적으로 통합할 것인가?

## [57] Vector Search with OpenAI Embeddings: Lucene is All You Need

### 요약

본 논문은 Lucene을 기반으로 한 벡터 검색 시스템이 특화된 벡터 데이터베이스와 경쟁할 수 있음을 보여주는 연구이다. WSDM 2024에서 발표된 이 연구는 OpenAI 임베딩을 활용하여 전통적인 텍스트 검색 엔진인 Lucene을 확장하고, 이를 통해 효율적인 벡터 검색을 구현할 수 있음을 입증한다.

Lucene 기반 접근법은 다음의 주요 특징을 가진다: - 기존의 널리 사용되는 오픈소스 검색 엔진 활용 - OpenAI 임베딩을 통한 고품질의 벡터 표현 - 별도의 특화된 시스템 없이도 벡터 검색 성능 달성 - 비용 효율성과 유지보수 용이성 제공 - 기존 Lucene 인프라와의 통합 가능성

이 연구의 핵심 기여는 벡터 검색을 위해 반드시 전문화된 벡터 데이터베이스가 필요하지 않다는 것을 실증적으로 보여준 점이다. 전통적인 검색 엔진도 적절한 임베딩과 인덱싱 기법으로 충분한 성능을 낼 수 있다는 통찰력을 제공한다.

벡터 검색의 대중화와 함께 이 연구는 기존 오픈소스 시스템을 활용한 실용적 접근법을 제시하며, 다양한 규모의 조직에서 벡터 검색을 도입할 수 있는 경로를 제시한다.

### 상세분석

#### 57.1 연구 배경 및 동기

최근 대규모 언어 모델(LLM)의 발전으로 벡터 임베딩의 중요성이 증대되었다. 많은 조직들이 검색 기능에 임베딩 기반의 의미론적 검색을 통합하려고 시도하고 있으나, Pinecone, Milvus, Weaviate 같은 특화된 벡터 데이터베이스의 도입에는 상당한 비용과 운영 복잡도가 수반된다.

본 연구는 이러한 배경에서 기존의 검증된 오픈소스 시스템인 Lucene을 활용하여 벡터 검색을 효과적으로 수행할 수 있는지를 질문한다. 이는 단순한 호기심을 넘어 실무적 중요성이 있는 질문이다.

## 57.2 핵심 방법론

Lucene은 1999년부터 개발되어온 성숙한 오픈소스 전문 검색 라이브러리이다. 이 연구에서는:

- OpenAI API를 통해 텍스트를 고차원 벡터로 변환
- Lucene의 Knn Vector 필드 타입을 활용하여 벡터 인덱싱
- 코사인 유사도(cosine similarity) 기반의 최근접 이웃 검색
- 벡터 검색과 기존 텍스트 검색의 하이브리드 접근법 적용

이러한 구성은 특별한 커스터마이제이션 없이도 Lucene의 기본 기능만으로 구현 가능하다는 점이 강점이다.

## 57.3 평가 및 결과

연구팀은 다양한 벡터 검색 벤치마크에서 Lucene 기반 접근법을 평가하였다. 결과는 다음과 같은 특징을 보인다:

- 검색 품질(Recall, NDCG 등): 특화된 벡터 DB와 비교 가능한 수준
- 인덱싱 속도: 상당히 효율적
- 메모리 사용량: 최적화 가능성 존재
- 운영 복잡도: 훨씬 낮음

특히 중소 규모의 데이터셋(수백만 개의 벡터)에서는 Lucene이 전문화된 시스템과 경쟁할 수 있음이 명확히 드러났다.

## 57.4 실용적 함의

이 논문의 실무적 의미는 상당하다:

1. 기존 Lucene 사용자: 별도의 새로운 시스템 도입 없이 벡터 검색 기능 추가 가능
2. 신규 구축: Lucene + OpenAI 임베딩으로 비용 효율적 솔루션 구현
3. 하이브리드 검색: 텍스트와 벡터 검색을 단일 인덱스에서 통합
4. 오픈소스 생태계: Java 기반 애플리케이션과의 자연스러운 통합

## 57.5 본 논문과의 관계

Exqutor는 벡터 검색을 포함한 데이터베이스 시스템의 쿼리 실행 성능을 분석하는 연구이다. 본 논문([57])은 다음 측면에서 Exqutor와 관련된다:

- **벡터 검색 시스템의 다양성:** Exqutor의 실험 대상 시스템 선정에 영향. 특화된 벡터 DB뿐 아니라 일반 검색 엔진도 고려 가능함을 시사
- **성능 비교의 기준점:** 전문화된 시스템과의 성능 차이 정량화에 필요한 참고자료
- **실무적 관련성:** 실제 많은 조직이 기존 인프라를 활용하고 싶어 하므로, 이들의 성능 특성 이해가 중요

- **벡터 임베딩 품질:** OpenAI 임베딩의 특성이 검색 성능에 미치는 영향 분석에 참고

---

## 57.6 기술 세부 분석

**6.1 Lucene 벡터 필드의 구현** Lucene의 KnnVectorField는 다음과 같은 특징을 가진다:

- 벡터를 이진 포맷으로 저장하여 메모리 효율성 제대
- 그래프 기반 인덱싱을 사용하지는 않지만 기본적인 KNN 알고리즘 적용
- 검색 시간 복잡도:  $O(\log n)$  (이진 검색)  $\rightarrow O(n * k)$  (정확한 KNN 계산)
- 메모리 상의 인덱스 구조로 인한 빠른 접근 속도

**6.2 OpenAI 임베딩의 특성** 문본 논문에서 사용된 임베딩:

- 차원도: 1536 (text-embedding-3-large)
- 강점: 의미론적 유사도를 매우 잘 인코딩
- 비용: API 호출에 따른 금전적 비용 발생
- 지연성: 각 문서 임베딩화에 네트워크 레이턴시 포함

**6.3 비용 측면 분석** 전문화된 벡터 DB vs Lucene 기반:

- Lucene: 라이선스 비용 0, OpenAI API 비용만
- 전문화된 DB: 라이선스/서비스 비용 높음
- 운영 비용: Lucene은 기존 Java 인프라 활용 가능

**6.4 벤치마킹 결과 상세** 본 논문의 실험에서 측정한 주요 지표:

- 인덱싱 시간: 100만 벡터 기준 약 2-3시간 (Lucene)
- 검색 레이턴시: 쿼리당 평균 50-100ms (단일 쿼리)
- 메모리 사용: 차원도에 따라 다양 (1536차원  $\times$  100만개  $\approx$  6GB)
- Recall@10: 90% 이상 달성 (특화된 DB와 유사)

이는 수백만 규모의 데이터셋에서는 충분히 실용적임을 보여줌

## 6.5 Lucene의 확장 가능성과 한계

확장 가능성: - 분산 인덱싱: Elasticsearch는 Lucene 기반이며 수십억 문서 처리 - 다중 필드: 텍스트 검색과 벡터 검색의 결합은 자연스러움 - 커뮤니티: 오픈소스이므로 커뮤니티의 지속적 개선

한계: - 특화되지 않음: 벡터 검색 최적화는 가장 최신 기술 미포함 - 메모리 효율성: 전문화된 벡터 DB보다 메모리 사용량이 더 많을 수 있음 - 분산 벡터 검색: 클러스터 환경에서의 벡터 인덱싱은 복잡성 증가

## 6.6 실무 적용의 의사결정

Lucene 벡터 검색을 선택해야 할 때: - 기존 Java 기반 인프라 - 중소 규모 데이터셋 (수백만~수천만) - 텍스트와 벡터 검색의 동시 필요 - 운영 복잡도 최소화 요구

전문화된 벡터 DB를 선택해야 할 때: - 대규모 데이터셋 (수억~수십억) - 벡터 검색만 필요 - 최고 성능 요구 - 전담 팀의 운영 가능

#### 6.7 산업 동향과 시사점 Lucene 기반 벡터 검색이 주목받는 이유:

- OpenAI, Cohere 등의 임베딩 서비스의 대중화로 고품질 벡터 접근성 증대
- 기업들의 기존 검색 인프라 활용 의욕
- 엔지니어링 리소스 제약이 있는 조직의 실용적 선택지
- 오픈소스 생태계의 강화로 Lucene 지속적 발전

### 추가 제기 문제

1. **확장성 한계:** Lucene 기반 접근법이 수십억 개의 벡터로 확장될 때의 성능 특성은? 메모리와 디스크 비용이 기하급수적으로 증가하는가?
2. **임베딩 품질 의존성:** 결과가 OpenAI 임베딩 품질에 얼마나 의존하는가? 다른 임베딩 모델(오픈소스 등)로는 어떻게 변할까?
3. **동적 업데이트:** 벡터가 지속적으로 추가/삭제되는 환경에서 Lucene의 성능 유지 방법은?
4. **정확도 vs 속도:** 벡터 차원수 감소, 양자화 등의 최적화 기법이 Lucene에서 얼마나 효과적인가?
5. **하이브리드 검색의 비용:** 텍스트와 벡터 검색을 동시에 수행할 때의 성능 트레이드오프는?
6. **다른 검색 엔진과의 비교:** Elasticsearch, Solr 같은 다른 오픈소스 검색 엔진은 벡터 검색에서 Lucene과 어떻게 비교되는가?
7. **정확도 메트릭:** Recall@k, NDCG, MRR 같은 다양한 메트릭에서 Lucene이 특화된 벡터 DB와 정확히 어느 정도 차이가 나는가?
8. **응용 사례 제한성:** 어떤 종류의 벡터 검색 응용에는 Lucene이 부적절한가? (실시간성, 대규모성 등)

### [58] A Survey on Advancing the DBMS Query Optimizer

#### 요약

본 논문은 데이터베이스 관리 시스템(DBMS)의 쿼리 최적화기(query optimizer)의 발전을 다루는 포괄적인 서베이 논문이다. 2021년 Data Science and Engineering 저널에 발표된 이 논문은 쿼리 최적화의 세 가지 핵심 구성



요소 - 카디널리티 추정(cardinality estimation), 비용 모델(cost model), 그리고 플랜 열거(plan enumeration)에 대해 깊이 있게 분석한다.

쿼리 최적화기의 핵심 역할: - SQL 쿼리를 입력받아 효율적인 실행 계획 생성 - 여러 가능한 실행 전략 중에서 최적의 것 선택 - 데이터베이스 전체 성능에 직접적 영향

카디널리티 추정은 쿼리 최적화의 기초이다: - 각 연산 단계에서 처리되는 행의 개수를 예측 - 부정확한 추정은 최적화기를 잘못된 플랜으로 유도 - 통계 기반 방법과 머신러닝 기반 접근법의 발전 추적

비용 모델의 역할: - 각 실행 계획의 상대적 비용 계산 - CPU, I/O, 메모리 비용을 종합적으로 고려 - 정확한 비용 예측이 올바른 선택으로 이어짐

플랜 열거 알고리즘: - 모든 가능한 실행 계획을 탐색하거나 휴리스틱으로 제한 - 검색 공간의 크기 증가 문제 극복

## 상세분석

### 58.1 카디널리티 추정의 진화

카디널리티 추정은 쿼리 최적화의 가장 어려운 부분이다. 전통적 방법들:

**1.1 통계 기반 방법** - 히스토그램, 스케치, 샘플링 등의 데이터 통계 활용 - 단일 속성 통계는 비교적 정확하나, 다중 속성 상관관계 파악 어려움 - 선택도(selectivity) 추정의 부정확성이 누적되는 문제

**1.2 머신러닝 기반 접근** - 쿼리 특성을 학습하여 카디널리티 예측 - NeuroCard, DeepDB 등의 생성 모델 기반 방법 - 통계 방법보다 높은 정확도를 보이지만 계산 비용 증가

**1.3 하이브리드 접근** - 통계 기반 방법의 빠름 + ML 기반 방법의 정확성 결합 - 적응적 학습을 통한 지속적 개선

### 58.2 비용 모델의 복잡성

현대 DBMS에서 비용 모델은 단순한 I/O 카운트를 넘어선다:

- CPU 사이클, 메모리 접근 패턴, 캐시 효율성
- 병렬 실행에서의 동기화 오버헤드
- 네트워크 전송 비용(분산 환경)
- 벡터 연산의 특수성(SIMD 최적화 등)

정확한 비용 예측을 위해서는: - 하드웨어 특성 반영 - 데이터 크기와 분포에 따른 동적 비용 계산 - 시스템 부하 상태 고려

### 58.3 플랜 열거의 도전 과제

Join order 문제에서  $n$ 개의 테이블에 대해 가능한 순서는  $n!$ 에 비례한다:

- 작은 규모: 동적 프로그래밍으로 최적해 탐색 가능

- 중간 규모: 휴리스틱 기반 접근 필요
- 대규모: 메타휴리스틱(유전 알고리즘 등) 활용

최근 발전: - 몬테 카를로 트리 검색 기반 플래너 - 학습된 휴리스틱으로 검색 공간 제한 - 분산 환경에서의 플래닝 병렬화

#### 58.4 벡터 검색 환경에서의 새로운 도전

전통 DBMS 최적화 이론이 벡터 검색에 직접 적용되지 않는 이유:

- **카디널리티 추정**: 벡터 거리 함수의 선택도 계산이 통계 기반으로는 어려움
- **비용 모델**: 거리 계산 비용, 인덱스 트래버설 비용이 차원과 데이터 크기의 함수로 비선형
- **플랜 열거**: 스칼라 조건과 벡터 조건의 혼합 필터링 순서 최적화

#### 58.5 본 논문과의 관계

Exqutor 논문이 벡터 검색을 포함한 데이터베이스의 쿼리 실행을 분석하기 위해서는 전통 DBMS 최적화의 기본 틀을 이해해야 한다. 본 논문([58])은:

- **기본 개념 제공**: 카디널리티 추정, 비용 모델, 플랜 열거의 정의와 중요성 설명
- **최적화 문제의 구조화**: 벡터 검색 쿼리 최적화를 전통 쿼리 최적화 틀에 맞추어 분석하는 데 필수
- **성능 분석 기준**: 실행 계획의 효율성을 평가하는 체계적 접근법 제공
- **하이브리드 쿼리 분석**: 텍스트와 벡터 조건의 복합 쿼리 최적화 이론의 기초

#### 58.6 현대 DBMS 최적화의 새로운 방향

##### 6.1 머신러닝 기반 최적화의 상태 최근의 발전:

- NeuroCard: 생성 모델을 통한 카디널리티 추정 (정확도 10배 향상)
- Kepler: 강화학습을 통한 플랜 선택
- BayesCard: 베이지안 네트워크로 속성 간 상관관계 학습

한계: - 훈련 데이터의 품질과 양이 매우 중요 - 데이터 분포 변화에 빠르게 적응하기 어려움 - 설명 가능성 (explainability) 부족으로 디버깅 어려움

##### 6.2 적응적 실행(Adaptive Execution) 런타임에서 실행 계획을 동적으로 수정:

- Cardinality feedback: 초기 추정이 틀린 경우 중간에 전략 변경
- Cascade join: 조인 결과의 실제 크기에 따라 다음 조인 방식 선택
- 장점: 단순한 구현으로도 큰 성능 향상 가능
- 단점: 추가 오버헤드와 복잡도

**6.3 벡터 통합의 미해결 문제들** - 벡터 인덱스의 재계산 비용이 일반 인덱스와 다름 - 근삿값(approximate) 계산의 정확도 영향 - 메모리 계층의 다양한 특성 반영

#### 6.4 쿼리 최적화의 단계별 프로세스

1단계: 쿼리 파싱

- SQL 쿼리를 내부 표현으로 변환
- 문법 검증

2단계: 논리 계획 생성

- WHERE 절의 필터 순서 결정
- 조인 조건 식별

3단계: 물리 계획 생성

- 각 논리 연산을 물리 연산으로 변환
- 인덱스 활용 여부 결정
- 동시성 제어 방식 선택

4단계: 비용 기반 선택

- 가능한 모든 계획의 비용 추정
- 최소 비용 계획 선택

5단계: 실행

- 선택된 계획 실행
- 런타임에 예상 밖 상황 처리

벡터 검색 통합 시에는 이 각 단계에서 벡터 관련 결정이 추가됨

### 추가 제기 문제

1. **벡터 특화 카디널리티 추정**: 벡터 검색에서 k값이 결과 크기를 결정할 때, 이를 전통적 카디널리티 추정 프레임에 어떻게 통합할 것인가?
2. **비용 모델의 차원 의존성**: 벡터 차원이 8에서 1536으로 변할 때 비용 모델은 어떻게 스케일링되어야 하는가?
3. **인덱스 구조의 영향**: B-tree와 HNSW 같은 다양한 인덱스의 비용 특성을 어떻게 단일 비용 모델에 통합할 것인가?
4. **적응적 학습의 한계**: 머신러닝 기반 카디널리티 추정이 데이터 분포 변화에 얼마나 빠르게 적응할 수 있는가?
5. **혼합 워크로드 최적화**: 텍스트, 벡터, 관계형 쿼리가 혼재된 환경에서 일관된 최적화 전략은 가능한가?
6. **런타임 적응의 오버헤드**: 적응적 실행이 주는 성능 향상이 추가 계산 비용을 상쇄할 수 있는가?
7. **정확한 vs 근삿값 계산의 최적화**: 벡터 검색의 근삿값 결과가 최종 쿼리 정확도에 미치는 영향을 비용 모델에 반영할 수 있는가?

---

## [59] Efficient Indexing of Billion-scale Datasets of Deep Descriptors

### 요약

본 논문은 컴퓨터 비전 분야에서 수십억 개의 고차원 deep feature vectors를 효율적으로 인덱싱하고 검색하는 방법을 제시한다. 2016년 CVPR에서 발표된 이 논문은 Babenko와 Lempitsky에 의해 작성되었으며, DEEP 데이터셋을 소개하여 대규모 벡터 검색 벤치마킹의 기초를 마련했다.

**Deep descriptor의 특징:** - 컨볼루션 신경망에서 추출한 고차원 벡터(일반적으로 수백에서 수천 차원) - 이미지의 의미적 특성을 인코딩 - 유클리드 거리 또는 코사인 유사도로 비교 가능

**핵심 도전 과제:** - 10억 개 이상의 벡터를 메모리 효율적으로 저장 - 쿼리당 수십억 벡터를 빠르게 검색 - 정확성과 속도의 균형

**제안 접근법:** - 벡터 양자화(vector quantization) 활용 - 계층적 클러스터링 기반 인덱싱 - 두 단계 재순위 지정(re-ranking) 프로세스 - CPU 기반 환경에서의 실용적 구현

본 논문은 현대 벡터 검색의 많은 기법들의 기초가 되었으며, 대규모 데이터셋에서의 정확도-효율성 트레이드오프 연구에 중요한 벤치마크 역할을 한다.

---

### 상세분석

#### 59.1 연구 배경 및 문제 정의

2010년대 중반 컴퓨터 비전의 발전으로 다음과 같은 상황이 대두되었다:

- 깊은 신경망이 매우 효과적인 이미지 표현 생성
- 이러한 벡터들은 고차원(e.g., 4096차원)이지만 매우 정보력이 높음
- 인터넷 사진 데이터베이스는 수십억 개 규모로 증가
- 기존의 정확한 최근접 이웃 검색 알고리즘은 이 규모에서 비실용적

이 맥락에서 핵심 질문은: **어떻게 정확성을 희생하지 않으면서도 10억 규모의 벡터를 검색할 수 있을까?**

#### 59.2 벡터 양자화 기법

**2.1 기본 개념** 벡터 양자화는 고차원 벡터를 훨씬 낮은 차원의 이산 코드로 변환한다:

- k-means 클러스터링으로 벡터 공간을 클러스터로 분할
- 각 벡터를 가장 가까운 클러스터 중심의 인덱스로 표현
- 메모리 사용량 수십배 감소 가능

**2.2 계층적 양자화 (Product Quantization, PQ)** 더 정교한 접근법:

- 벡터를 부분벡터로 분할 (예: 4096차원을 8개의 512차원으로)
- 각 부분에 대해 독립적으로 양자화
- 전체 정확도 유지하면서 메모리 효율성 향상

**2.3 실무 적용** PQ 기법은 이후 많은 ANN 알고리즘의 핵심이 되었다: - FAISS의 기본 구성 요소 - Pinecone 등 상용 벡터 DB에서 널리 사용 - 수십억 벡터 처리의 실용성을 입증

### 59.3 계층적 인덱싱 구조

**3.1 구조 개요** - 첫 번째 단계: 전체 벡터 공간을 계층적으로 클러스터링 - 두 번째 단계: 각 클러스터 내에서 세밀한 인덱싱 - 검색 시 관련 클러스터만 탐색

#### 3.2 검색 프로세스

```

쿼리 벡터 입력
↓
상위 계층에서 후보 클러스터 선택
↓
선택된 클러스터 내에서 상세 검색
↓
재순위 지정(원본 또는 더 정확한 표현 사용)
↓
최종 결과 반환

```

**3.3 정확도 향상** - 상위 계층 검색이 충분히 큰 후보 풀을 유지하면 정확도 보장 - 추가 계산 비용은 미미하면서도 재순위 효과로 성능 개선

### 59.4 재순위 지정 메커니즘

**4.1 목적** 양자화로 인한 정보 손실을 보상:

- 양자화된 벡터로 초기 후보 선정 (빠름)
- 원본 또는 더 정확한 표현으로 재계산 (정확함)

**4.2 구현 방식** - Source coding 활용으로 각 벡터마다 추가 비트 저장 - 초기 후보군(예: 상위 1000개)에 대해서만 정확한 거리 계산 - 최종 정렬 수행

### 59.5 본 논문과의 관계

Exquitor가 벡터 검색 시스템의 성능을 벤치마킹할 때 DEEP 데이터셋을 참고하는 이유:

- **데이터셋 표준화**: 수십억 규모의 벡터 데이터셋이 학문적으로 검증됨
- **실제 벡터 특성**: 실제 컴퓨터 비전 벡터의 분포와 특성 반영
- **확장성 평가**: 시스템이 수십억 벡터로 확장되는 상황 평가 가능
- **기법 적용성**: 현대 벡터 검색의 기초가 된 기법들의 이해

또한 Exqutor에서: - 벡터 검색 성능 메트릭(recall, latency) 정의의 기초 - 양자화와 같은 최적화 기법의 효과 측정 방법 학습 - 대규모 데이터셋에서의 성능 특성 이해

## 59.6 현대 벡터 검색 시스템에서의 적용

**6.1 상용 벡터 DB의 기법 활용** 많은 현대 벡터 검색 시스템이 본 논문의 기법을 활용:

- Faiss: Facebook의 연구를 기반으로 한 오픈소스 라이브러리
- 포함된 기법: Product Quantization, 계층적 클러스터링
- 효율성: 1억 벡터를 수백 MB 메모리로 처리 가능
- HNSW: Product Quantization과 그래프 결합
- 특징: 빠른 검색 + 높은 정확도

**6.2 압축 기법의 진화** Product Quantization 이후의 발전:

- Residual Vector Quantization: 오차를 다단계로 양자화
- Composite Quantization: 여러 양자화기 결합
- Learned Quantization: 신경망으로 최적 양자화 학습

**6.3 메모리 대역폭의 실제 영향** 수십억 벡터 검색의 성능 분석:

병목 분석:

- CPU 연산: 거리 계산은 SIMD로 가속 가능
- 메모리 접근: 양자화된 벡터 로드가 더 빠름
- 알고리즘: 계층적 구조로 접근 범위 제한

결론: 메모리 대역폭은 중요하지만 절대적 병목은 아님  
정렬한 알고리즘이 더 중요

## 추가 제기 문제

1. **양자화 손실의 측정**: 다양한 양자화 수준(8비트, 4비트, 1비트)에서 정확도 감소가 선형인가 비선형인가?
2. **데이터셋 특이성**: DEEP 데이터셋이 모든 종류의 high-dimensional 데이터를 대표하는가? 자연어 임베딩은 다른 특성을 보일 수 있는가?
3. **메모리 대역폭**: 10억 개 벡터 검색 시 메모리 대역폭이 성능의 주요 병목이 되는가? 계산량은 충분한가?
4. **동적 업데이트**: 계층적 인덱싱 구조에 새로운 벡터를 추가할 때 기존 클러스터를 재조직화해야 하는가? 비용은?

5. **임베딩 진화**: 신경망 아키텍처가 변경되어 벡터 특성이 변할 때 기존 인덱스의 유효성은 어떻게 유지될 것인가?
6. **하드웨어 특화**: GPU를 활용한 벡터 검색이 이 논문의 CPU 기반 알고리즘보다 실제로 얼마나 빠를 수 있는가?
7. **실시간 시스템**: 이 기법이 실시간 삽입/삭제가 많은 환경에서 효과적일 수 있는가?

---

## [60] Searching in One Billion Vectors: Re-rank with Source Coding

### 요약

본 논문은 10억 개 규모의 벡터를 효율적으로 검색하는 문제를 다루며, 2011년 ICASSP에서 발표된 Jégou 등의 연구이다. 주요 초점은 SIFT(Scale-Invariant Feature Transform) 벡터라는 컴퓨터 비전의 고전적 특징 벡터를 활용하여 수십억 규모의 이미지 인덱싱과 검색을 수행하는 것이다.

**연구의 핵심 기여**: - 대규모 벡터 검색에서 실용적인 성능 달성 - Source coding을 이용한 정밀한 재순위 지정 - 메모리 효율과 검색 속도의 효과적 균형

**SIFT 벡터의 특징**: - 128차원 벡터로 이미지의 지역 특징 표현 - 매우 널리 사용되는 특징 추출 방법 - 수십억 개의 SIFT 벡터는 대규모 이미지 데이터셋에서 현실적

**주요 기법**: - Product quantization (PQ) 기반 압축 - 계층적 클러스터링을 통한 검색 공간 축소 - 양자화 오차 보정(source coding) - 재순위 지정 프로세스

이 논문은 벡터 검색의 정확도와 효율성 사이의 트레이드오프를 다루는 고전적 연구로, 현대 ANN(Approximate Nearest Neighbor) 알고리즘의 이론적 기초를 제공한다.

---

### 상세분석

#### 60.1 문제의 도전 과제

1억 개가 아닌 10억 개의 벡터를 검색하는 것이 왜 특별한 문제인가?

**메모리 계산**: - 각 SIFT 벡터: 128차원 × 4바이트 = 512바이트 - 10억 벡터: 512GB (압축 없음) - 이는 일반적인 서버 메모리(256GB)를 초과

**검색 비용 계산**: - 선형 스캔: 각 쿼리마다 10억 번의 거리 계산 - 쿼리당 수 초 이상 소요 (실용적 불가능) - 인덱싱 필수

#### 60.2 Product Quantization (PQ) 기법 심화

2.1 원리 벡터를 부분벡터로 분할하고 각각 독립적으로 양자화:

- 원본 벡터: 128차원
- 분할: 4개 부분 × 32차원
- 각 부분마다 256개(8비트) 클러스터

결과: - 메모리: 128바이트 → 4바이트 (32배 압축) - 정확도 손실: 수용 가능 수준

**2.2 거리 계산의 효율성** - 부분벡터별로 사전 계산된 거리 테이블 활용 - 1번의 메모리 접근 → 거리값 반환 - 전체 거리: 4번의 테이블 조회로 계산 가능 - 매우 빠른 거리 계산

## 60.3 계층적 검색 구조

### 3.1 검색 파이프라인

- 1단계: Coarse quantization (대규모 분할)
- 전체 벡터 공간을 k개의 클러스터로 분할
  - 쿼리와 가장 유사한 클러스터 w개 선택
- 2단계: Product quantization (정밀 검색)
- 선택된 w개 클러스터 내에서 PQ 기반 검색
  - 거리 테이블을 사용한 빠른 계산
- 3단계: Re-ranking with source coding
- 상위 후보에 대해 추가 정보 활용
  - 정확한 거리 재계산

**3.2 성능 특성** - 메모리: 10억 벡터를 몇 GB로 압축 - 속도: 쿼리당 밀리초 단위 (매우 빠름) - 정확도: recall@1000에서 90% 이상 달성

## 60.4 Source Coding을 통한 재순위 지정

**4.1 배경** PQ 기반 거리가 완벽하지 않으므로, 정확한 순위를 위해서는 재계산 필요:

- 모든 벡터를 재계산하면 메모리/속도 이점 상실
- 상위 k개 후보만 재계산 → 효율성과 정확도 동시 달성

**4.2 Source Coding의 역할** 각 벡터마다 몇 추가 비트(보통 8-16비트)만 저장:

- 양자화 오차를 인코딩
- 재순위 단계에서 정확한 거리 복원에 사용
- 전체 메모리 오버헤드 미미 (~2%)

**4.3 실무 효과** - 상위 1000개 후보에 대해서만 정확한 거리 계산 - 최종 순위의 정확도 대폭 향상 - 계산 비용은 무시할 수 있는 수준



## 60.5 성능-정확도 트레이드오프의 정량화

5.0 주요 성과 본 논문의 실험 결과:

- 10억 SIFT 벡터를 약 100GB 메모리로 처리
- 쿼리당 평균 레이턴시: 20-50ms (Recall@1000 > 90%)
- 메모리 압축비: 원본 대비 32배
- 속도 향상: 선형 스캔 대비 1000배 이상

이러한 성과는 당시(2011년) 혁신적이었으며, 현대 ANN 알고리즘의 기초가 됨

## 60.6 확장성 분석

5.1 메모리 스케일링 - 10억 벡터: ~4-5GB (PQ + source code) - 100억 벡터: ~40-50GB (단일 머신 초과) - 분산 인덱싱 필요

5.2 검색 속도 스케일링 - 클러스터 수가 증가하면 1단계 비용 증가 - 하지만 w개 클러스터만 검색하므로 전체 비용은 아대선형적 - 시간 복잡도:  $O(w \times n/k)$  (n: 벡터 수, k: 클러스터 수)

## 60.7 추가 기술적 고찰

6.1 근삿값 검색의 정확도 보장 Jégou의 논문이 해결한 중요한 문제:

정확한 검색(Exact Search)의 문제: - 모든 벡터와의 거리 계산:  $O(n)$  복잡도 - 10억 벡터: 수 시간 소요

근삿값 검색(Approximate Search)의 위험: - 빠르지만 최적해를 놓칠 수 있음 - Recall이 너무 낮으면 사용 불가능

본 논문의 해결책: - 계층적 검색으로 후보 제한 - 재순위로 최종 정확도 보장 - 실무적으로 필요한 수준의 정확도 달성

6.2 다양한 벡터 타입에 대한 일반화 SIFT 벡터 외에도:

- Dense descriptors (SIFT, SURF, ORB)
- Deep features (CNN의 마지막 층)
- Word embeddings (Word2Vec, GloVe)
- 모두에 적용 가능한 방법론

## 60.8 본 논문과의 관계

Exqutor의 벡터 검색 벤치마킹에서:

- 성능 메트릭 정의: Recall@k 같은 표준 메트릭의 학문적 기초
- 기법 비교 기준: Product quantization과 같은 최적화 기법의 효과 측정 방법
- 대규모 데이터셋: 10억 규모 벡터 처리의 실무적 가능성 검증
- 속도-정확도 트레이드오프: 다양한 설정에서의 성능 특성 이해

Exqutor에서 시스템 성능을 평가할 때: - 어떤 시스템이 10억 벡터를 수십 밀리초에 검색할 수 있는지 판단 - 재순 위 지정 기법의 정확도 향상 효과 정량화 - 메모리 압축 기법의 비용-효과 분석

## 추가 제기 문제

1. **양자화 오차의 누적**: 다단계 양자화(coarse + PQ + source code)에서 최종 오차가 예측 가능한가?
2. **데이터 분포의 영향**: SIFT 벡터의 특정 분포가 PQ 성능에 영향을 주는가? 다른 종류의 벡터(e.g., 자연어 임베딩)는 다를 수 있는가?
3. **동적 인덱싱**: 벡터가 지속적으로 추가될 때 coarse quantization을 재수행해야 하는가? 점진적 업데이트 가능한가?
4. **클러스터 수 최적화**: k(클러스터 수)와 w(검색할 클러스터 수)를 어떻게 선택할 것인가? 데이터 의존적인가?
5. **부분 벡터 차원 분할**: 4개의 32차원 부분벡터로 분할하는 것이 최적인가? 다양한 분할이 성능에 어떻게 영향을 주는가?
6. **하드웨어 의존성**: 거리 테이블 기반 계산이 캐시 효율성에 어떻게 영향을 받는가? 다양한 프로세서에서 성능이 일관적인가?

## (I) 카디널리티 추정 & 쿼리 최적화

### [61] Here's How We're Using AI to Help Detect Misinformation

#### 요약

본 자료는 Meta(Facebook)가 2020년 AI 블로그에서 발표한 글로, 미디어 콘텐츠 유사성 검색을 통한 거짓 정보 탐지 기술을 소개한다. Meta는 SimSearchNet++라는 신경망 모델을 개발하여 시각적으로 유사한 이미지들을 찾아내고, 이를 통해 조작되거나 변형된 미디어 콘텐츠를 식별하는 데 활용하고 있다.

**핵심 기술 개요**: - SimSearchNet++: 이미지 유사성 학습 신경망 - 임베딩 기반 검색: 고차원 벡터를 통한 효율적 유사도 계산 - 대규모 이미지 코퍼스: 수십억 개의 이미지에서 빠른 검색

**미디어 진실성 검증의 중요성**: - 온라인 상에서 조작된 이미지의 신속한 확산 - 특정 사건이나 인물 관련 거짓 정보의 광범위 유포 - 선거, 공중보건 등 중요한 사회 이슈에서의 해악

**Meta의 접근 방식**: - 이미지 임베딩: 각 이미지를 고차원 벡터로 변환 - 벡터 검색: 이미지 풀(pool)에서 유사한 이미지 탐색 - 팩트체크 플러그: 알려진 허위 정보와 유사한 이미지 표시 - 확인된 정보와의 연결: 신뢰할 수 있는 정보원의 팩트체크 결과 표시

이 사례는 벡터 검색 기술이 단순한 데이터베이스 검색을 넘어 실제 사회 문제 해결에 적용되는 실무적 예시이다.

## 상세분석

### 61.1 SimSearchNet++의 기술적 배경

**1.1 이미지 유사성의 학습** 전통적 이미지 처리에서는: - 픽셀 기반 거리 측정 → 조명, 회전 등에 민감 - 특징 기반 비교 → 제한된 성능

Deep learning 기반 접근: - 신경망이 의미적 유사성을 학습 - SimSearchNet: Siamese network 아키텍처 기반 - 같은 대상의 이미지는 유사한 임베딩 → 거짓 정보라도 원본과 유사함

**1.2 훈련 데이터** - 같은 사건의 서로 다른 이미지 쌍 - 조작된 버전과 원본의 쌍 - 전혀 다른 이미지 쌍 (부정 샘플)

목표: 같은 대상의 이미지는 가깝게, 다른 이미지는 멀게 배치

### 61.2 대규모 벡터 검색의 실무적 적용

**2.1 검색 인프라** Meta의 규모: - 매월 수십억 개의 이미지 업로드 - 실시간 또는 단시간 내에 유사 이미지 찾기 필요 - 전통적 선형 검색 불가능

구현: - 각 이미지를 임베딩으로 변환 - 임베딩 벡터를 고속 검색 인덱스에 저장 - 새로운 이미지 수신 → 임베딩 생성 → 검색

**2.2 성능 요구사항** - 레이턴시: 사용자 경험에 영향 (수백ms 이내) - 정확도: false positive/negative 최소화 - 확장성: 월별 증가하는 데이터 처리

### 61.3 미디어 진실성 검증 프로세스

#### 3.1 완전한 파이프라인

1. 이미지 업로드
- ↓
2. 임베딩 생성 (SimSearchNet++)
- ↓
3. 벡터 인덱스 검색
- ↓
4. 유사한 이미지 발견
- ↓
5. 발견된 이미지의 팩트체크 상태 확인
- ↓
6. 유사 이미지에 팩트체크 레이블 부여
- ↓
7. 사용자에게 진실성 경고 표시

### 3.2 거짓 정보 탐지 전략

다양한 거짓 정보 유형: - 완전 조작: 온라인에 없던 이미지 제작 - 검색으로는 탐지 어려움 - 다른 기법 필요 (조작 탐지 등)

- 부분 조작: 이미지 수정, 자르기, 필터 적용
- 기본 이미지와 충분히 유사하면 검색으로 탐지 가능
- SimSearchNet++의 강점
- 맥락 오류: 오래된 이미지를 현재 사건으로 표현
- 이미지 자체는 진짜지만 맥락 거짓
- 벡터 검색의 매칭이 중요한 단서

### 61.4 벡터 검색의 실무적 제약사항

**4.1 정확도-속도 트레이드오프** - 높은 임계값: 정확한 유사 이미지만 찾을 수 있음 → 거짓 정보 놓침 - 낮은 임계값: 모든 가능한 유사 이미지 찾을 수 있음 → 거짓 경고 증가

Meta의 해결책: - 팩트체커와의 협력 - 사람의 판단을 최종 검증 단계로 유지 - 자동화는 우선순위 지정 역할

**4.2 확장성 도전** - 매일 새로운 이미지 추가로 인한 인덱스 업데이트 - 임베딩 모델 개선 시 기존 벡터 재계산 필요 - 지역별 다른 거짓 정보 패턴 대응

### 61.5 사회적 영향

**5.1 미디어 생태계 개선** - 조작된 미디어의 자동 탐지 - 원본 출처로의 연결 제공 - 사용자가 정보의 신뢰성을 판단하는 데 도움

**5.2 한계 및 비판** - 알고리즘 투명성 부족 - 팩트체킹 기준의 중립성 문제 - 검색 성능의 지역/문화별 차이 - 조작 이미지에 대한 완벽한 탐지 불가능

### 61.6 본 논문과의 관계

Exqutor와의 연관성:

- **실무 적용 사례:** 벡터 검색이 실제 대규모 환경에서 어떻게 사용되는지 보여줌
- **데이터셋 맥락:** SimSearchNet++ 모델로 생성된 이미지 임베딩의 특성 이해
- **성능 요구사항:** 실제 시스템에서 요구하는 레이턴시, 정확도, 확장성의 구체적 사례
- **하이브리드 쿼리:** 이미지 유사도 검색 + 메타데이터(팩트체크 상태) 필터링

Exqutor에서 벡터 검색 벤치마킹 시: - Meta 같은 대규모 플랫폼의 실제 요구사항 고려 - 실시간 처리의 중요성 이해 - 온라인 학습, 모델 업데이트의 영향 분석 - 정확도와 성능의 실무적 균형

## 추가 제기 문제

1. **임베딩 모델의 진화**: SimSearchNet++ 모델이 개선되어 새로운 버전이 나올 때, 기존의 수십억 개 임베딩을 재계산하는 비용은? 점진적 마이그레이션이 가능한가?
2. **거짓 정보의 다양성**: 이미지 검색이 효과적인 거짓 정보(부분 조작, 맥락 오류)의 비율은? 완전 조작 이미지(AI 생성 등)는 어떻게 대응할 것인가?
3. **지역화 성능**: 다양한 문화와 언어권에서 SimSearchNet++ 성능이 일관적인가? 편향(bias) 문제는?
4. **팩트체크의 지연**: 거짓 정보가 확산된 후 팩트체크 결과가 나올 때까지의 시간차는? 선제적 탐지 가능한가?
5. **사용자 신뢰**: 알고리즘 기반 거짓 정보 플래그에 대한 사용자 신뢰 수준은? 잘못된 플래그의 파급 효과는?
6. **동적 업데이트**: 새로운 거짓 정보 패턴이 등장할 때 시스템이 얼마나 빠르게 적응할 수 있는가? 온라인 학습 가능한가?

## 기타 미분류 논문

### [62] YFCC100M: The New Data in Multimedia Research

#### 요약

본 논문은 Yahoo Flickr Creative Commons 100 Million (YFCC100M) 데이터셋을 소개하는 2016년 CACM 논문이다. 저자들은 Flickr 플랫폼에서 수집한 1억 개의 이미지로 구성된 대규모 공개 멀티미디어 데이터셋을 제시한다. 이는 컴퓨터 비전과 멀티미디어 연구 분야에서 가장 광범위하게 사용되는 벤치마킹 데이터셋 중 하나가 되었다.

**YFCC100M의 핵심 특징**: - 규모: 1억 개의 이미지와 동영상 - 출처: Flickr의 Creative Commons 라이선스 콘텐츠 - 메타데이터: 업로더, 태그, 위치 정보, 카메라 정보 등 풍부한 부가 정보 - 공개성: 연구 커뮤니티에 무료 공개

**멀티미디어 연구에서의 중요성**: - 이전 데이터셋보다 훨씬 큰 규모 → 딥러닝 모델 훈련에 적합 - 다양한 사진 스타일과 주제 포함 - 실제 사용자가 올린 자연스러운 이미지 (수작업으로 큐레이션된 것 아님) - 개인정보 존중 (Creative Commons 라이선스 확인)

**YFCC100M의 활용**: - 이미지 분류, 객체 감지, 장면 이해 등의 기반 데이터 - 이미지 검색, 유사성 학습의 벤치마크 - 멀티모달 연구(텍스트+이미지) 기반 데이터

Exquitor 연구에서 YFCC100M은 벡터 검색 시스템을 평가하기 위한 실제 규모의 이미지 컬렉션으로 활용되며, 이미지 기반 벡터 검색의 성능 특성을 이해하는 데 중요한 역할을 한다.

## 상세분석

### 62.1 데이터셋의 역사적 맥락

**1.1 이전 세대 데이터셋의 한계** - ImageNet: 높은 품질이지만 규모 제한 (~1.4M 이미지) - COCO: 상세 주석이 있지만 매우 큰 규모 불가능 (~330K 이미지) - Academic의 비용: 고품질 수작업 큐레이션의 비용 증가

**1.2 왜 YFCC100M인가?** 2010년대 초중반에: - 소셜 미디어에서 자연스러운 대규모 이미지 데이터 활용 가능 - 메타데이터의 풍부함 (사용자 태그, 위치, 촬영 정보 등) - 개인정보 보호 가능한 Creative Commons 라이선스  
이는 데이터셋 수집의 인식 전환을 대표한다.

### 62.2 데이터셋의 구성과 특징

**2.1 수치적 규모** - 이미지: 1억 개 - 동영상: 추가로 포함 - 메타데이터: 업로더, 촬영일, GPS 좌표, 카메라 모델, 렌즈 정보 - 사용자 생성 태그: 평균 5-10개 태그/이미지

**2.2 데이터 다양성** 다양한 이미지 카테고리: - 풍경, 건축, 음식, 동물, 사람 등 - 다양한 촬영 환경과 조건 - 전 세계 다양한 지역의 이미지

**2.3 메타데이터의 가치** GPS 좌표로부터: - 지역별 시각적 특징 연구 가능 - 동일 장소의 다양한 시간대 이미지 연구 - 지리 공간 시각 인식 연구

사용자 태그로부터: - 대규모 약지도(weak label) 데이터 생성 - 사람의 의도와 관심 파악 - 태그의 다양성로부터 시각적 개념의 복잡성 이해

### 62.3 멀티미디어 연구에서의 활용

#### 3.1 이미지 분류 벤치마크

타스크: 이미지를 1000개 카테고리로 분류  
평가 지표: Top-1, Top-5 정확도  
의미: 다양한 분류 모델의 성능 비교 기준

YFCC100M 활용: 사전 훈련(pre-training) 데이터  
- ImageNet만으로는 부족한 다양성 제공  
- 더 견고한 특징 학습 가능

#### 3.2 이미지 검색 벤치마크

타스크: 쿼리 이미지와 가장 유사한 이미지 찾기  
평가 지표: Recall@k, NDCG, MAP  
의미: 검색 시스템의 효율성과 정확도 평가

YFCC100M 활용:

- 대규모 갤러리(1억 이미지)에서 검색 테스트
- 실제 인터넷 규모의 검색 문제 모사
- 시스템 확장성 평가

**3.3 객체 감지와 장면 이해** - 약지도 학습으로 수백만 이미지 활용 - 자동으로 객체 위치 정보 생성 - 이전에 불가능했던 규모의 학습

## 62.4 벡터 검색 시스템의 벤치마킹

**4.1 YFCC100M을 이용한 평가** 현대 벡터 검색 시스템의 성능 평가:

1. 이미지 임베딩 생성
2. 사전 훈련된 CNN 모델 사용
3. 각 이미지 → 고차원 벡터 변환
4. 벡터 인덱스 구축
5. 1억 벡터를 특정 시스템(Faiss, Milvus 등)에 인덱싱
6. 메모리, 디스크 사용량 측정
7. 검색 성능 평가
8. 쿼리 이미지 선택
9. 검색 레이턴시 측정
10. Recall 계산 (정확한 kNN과 비교)

**4.2 성능 특성 분석** - 작은 배치(수천 이미지): 모든 시스템 비슷한 성능 - 중간 규모(수백만 이미지): 시스템 차이 명확 - 대규모(1억 이미지): 인덱싱 및 검색 기법의 중요성 극대화

## 62.5 데이터셋의 한계와 편향

**5.1 출처의 편향** - Flickr 사용자 특성: 선진국, 특정 언어권 과다 대표 - 촬영 주제의 편향: 관광 명소, 특정 문화에 집중 - 사진 스타일: 아마추어부터 준전문가 수준

**5.2 데이터 품질 문제** - 약지도 태그의 부정확성 - 스팸 또는 부적절한 콘텐츠 포함 가능성 - GPS 정보의 부정확성 (사용자 수동 입력 또는 생략)

**5.3 시대성 문제** - 2016년 데이터셋 → 2010년대 초중반 이미지 주로 포함 - 최신 스마트폰의 이미지 특성(높은 해상도, 처리된 색감 등) 과소 표현 - 패션, 기술 트렌드의 시간차

## 62.6 본 논문과의 관계

Exqutor에서 YFCC100M을 활용하는 이유:

- **실제 규모의 벤치마크:** 1억 이미지는 대규모 벡터 검색의 현실적 규모
- **다양한 쿼리 시나리오:** 다양한 이미지 주제로부터 실제 사용 사례 모사
- **메타데이터 활용:** 위치, 촬영일, 태그 등의 메타데이터와 결합한 복합 쿼리 평가
- **기존 연구와의 비교:** 수많은 기존 연구가 YFCC100M을 사용했으므로 결과 비교 용이

구체적으로: - 텍스트 필터 + 이미지 유사도 검색의 하이브리드 쿼리 평가 - 위치 기반 필터 + 벡터 검색의 지리 공간 쿼리 평가 - 시간 범위 필터와 벡터 검색의 결합 평가

### 추가 제기 문제

1. **시대 편향 보정:** YFCC100M이 2010년대 초반 이미지 중심이라면, 현대 스마트폰 이미지의 특성(높은 해상도, 자동 보정 등)에 대한 시스템의 성능은 어떻게 변할 것인가?
2. **메타데이터 품질:** GPS 정보의 20~30%가 없거나 부정확하다면, 지리 공간 필터 기반 쿼리의 신뢰성은? 필터링 성공률의 변동이 큰가?
3. **태그 기반 필터의 한계:** 사용자 생성 태그가 부정확하거나 모호할 때(예: "sunset" 태그가 붉은색 어떤 사진든 붙어있음), 태그와 벡터 검색의 결합이 효과적인가?
4. **표현 편향의 영향:** 선진국 관광지 이미지 과다 표현이 학습된 임베딩 모델에 편향을 가져오는가? 특정 장소나 문화의 검색 정확도가 떨어지는가?
5. **장기 이미지 검색의 과제:** 같은 장소의 오래된(2012) 이미지와 최신(2024) 이미지를 매칭해야 할 때 성능은? 건축, 자연 변화가 임베딩 유사도에 어떻게 영향을 주는가?
6. **동적 데이터셋 관리:** 1억 이미지 데이터셋이 매년 추가된다면 어떻게 관리할 것인가? 인덱스 재구축 주기는? 모델 업데이트에 따른 기존 임베딩의 유효성은?

## [63] Results of the Big ANN: NeurIPS'23 Competition

### 요약

본 논문은 2023년 NeurIPS에서 개최된 Big ANN(Approximate Nearest Neighbor) 경쟁의 결과를 분석하는 arXiv 논문이다. 저자들은 H. V. Simhadri 등이 제시한 이 경쟁이 1억에서 10억 규모의 벡터 데이터셋에서 정확한 최근접 이웃 검색 알고리즘의 성능을 객관적으로 비교하는 표준 벤치마크를 제공한다.

**Big ANN 경쟁의 목표:** - 다양한 규모의 벡터 검색 문제를 정의 - 동일한 하드웨어 환경에서 다양한 시스템의 성능 비교 - 학계와 산업의 ANN 알고리즘 발전 추적 - 성능 측면(속도, 정확도, 메모리)의 객관적 평가



**경쟁 구성:** - 여러 데이터셋 (SIFT, GIST, 텍스트 임베딩, 이미지 특징) - 다양한 규모 (1억, 10억 벡터) - 제한된 메모리 조건 (1GB, 10GB, 100GB, 1TB) - 고정 하드웨어 환경 (동일 프로세서, 메모리, 저장소)

**주요 결과:** - 알고리즘의 선택이 성능에 미치는 영향이 매우 큼 - 가정 효과(memory vs speed trade-off)의 구체적인 정량화 - 최신 기법(그래프 기반, 하이브리드)의 우월성 입증 - 개발자와 연구자에게 알고리즘 선택 지침 제공

Big ANN 경쟁은 벡터 검색 시스템의 성능 평가에 새로운 표준을 제시하며, Exqutor 같은 벤치마킹 연구의 직접적인 참고 자료가 된다.

## 상세분석

### 63.1 경쟁 설계의 혁신

**1.1 표준화의 필요성** 이전까지 벡터 검색 연구의 문제점:

- 각 논문이 자신의 데이터셋과 평가 방법 사용
- 결과 비교 불가능 (사과와 오렌지 비교)
- 알고리즘 발표자의 자유도 높음 → 결과 신뢰도 하락
- 업계와 학계의 성능 차이 미스터리

Big ANN이 해결한 것: - 동일한 데이터셋, 평가 지표, 하드웨어 사용 - 투명한 평가 프로세스 - 재현 가능한 결과

#### 1.2 경쟁의 3가지 축

규모 (Scale)

1억 벡터 (100M)

10억 벡터 (1B)

메모리 제약 (Memory Budget)

1GB, 10GB, 100GB, 1TB

데이터셋 다양성 (Data Types)

SIFT: 컴퓨터 비전 특징

GIST: 장면 이해 특징

YFCC: 실제 이미지 임베딩

TRIPLES: 텍스트 임베딩

MSMARCO: 문서 검색 임베딩

### 63.2 주요 데이터셋과 특성

**2.1 SIFT (128차원)** - 고전적 컴퓨터 비전 특징 - 매우 높은 차원도(128)에 비해 크기는 작음 - 특성: 클러스터링 어려움, 높은 기하학적 복잡도

**2.2 GIST (960차원)** - 이미지의 전체 장면 특성 인코딩 - 더 높은 차원도 - 특성: 서로 다른 이미지 간의 거리 분포가 다양함

**2.3 YFCC (1000차원)** - 실제 신경망에서 생성된 이미지 임베딩 - 학습된 표현 (vs 수작업 설계) - 특성: 의미론적 유사성을 잘 인코딩

**2.4 MSMARCO (768차원)** - BERT 임베딩 기반 텍스트 표현 - NLP 분야의 표준 벤치마크 - 특성: 의미론적 검색의 현실성 반영

### 63.3 경쟁 결과의 주요 발견

#### 3.1 알고리즘 성능 순위 상위 항목들의 공통 특징:

- 그래프 기반 방법 (HNSW, RobustPQ, SRS)
- Top recall(>99%) 달성
- 메모리 효율적
- 검색 속도 우수
- 하이브리드 접근법
- 양자화 + 그래프의 결합
- 메모리-속도-정확도의 균형
- 계층적 구조
- 거친 검색(coarse) + 정세 검색(fine)
- 단계적 정제

#### 3.2 메모리 트레이드오프 분석

메모리 1GB 제약:

- 극단적 압축 필요 (1비트 ~ 4비트)
- Recall ~60-70% 수준
- 매우 빠른 검색

메모리 10GB 제약:

- 합리적 압축 (4비트 ~ 8비트)
- Recall ~85-90% 수준
- 실무 적용 가능 수준

메모리 100GB 이상:

- 거의 손실 없는 표현 가능
- Recall >98% 달성
- 검색 속도 최우선 최적화 대상

### 3.3 규모 확장성

100M 벡터 (1억)

- 웹 규모(웹 페이지 검색 등) 초기 수준
- 대부분의 알고리즘 실용적 성능

1B 벡터 (10억)

- 대규모 인터넷 스케일
- 알고리즘 간 성능 차이 극대화
- 메모리 효율성 극도로 중요

## 63.4 상위 솔루션의 기법

### 4.1 그래프 기반 방법의 우월성 HNSW (Hierarchical Navigable Small World):

- 계층적 그래프 구조
- 로그 복잡도의 검색
- 메모리 오버헤드 작음 (원본 벡터 저장 필요)
- Recall 매우 높음

### 4.2 양자화의 발전 Residual Quantization:

- 벡터를 단계적으로 양자화
- 각 단계에서 남은 오차(residual) 인코딩
- 정확도 개선으로 메모리 효율성 향상

Product Quantization: - 부분벡터 독립 양자화 - 빠른 거리 계산 - 대규모 데이터셋에 적합

### 4.3 하이브리드 접근법 최고 성능 팀들의 공통점:

1. 계층적 구조: 거칠게 후보 선정
2. 정밀 인덱싱: 선택된 영역 내에서 정확한 검색
3. 재순위: 상위 k개에 대해 더 정확한 계산
4. 압축: 각 단계마다 다양한 수준의 양자화

## 63.5 실무적 시사점

5.1 시스템 선택 지침 - 메모리 충분(100GB+): HNSW 기반 시스템 추천 - 메모리 제약(10GB): PQ + 그래프 하이브리드 - 극도의 제약(1GB): 극단적 양자화 필수

### 5.2 성능 벤치마킹의 표준 Big ANN이 제시한 평가 방법:

- Recall@1 (가장 정확한 최근접 이웃 찾기)
- QPS (초당 쿼리 처리 능력)
- 메모리 사용량
- 인덱싱 시간

이 지표들의 조합으로 종합적 비교

### 5.3 연구 방향의 지표 경쟁 결과가 시사하는 방향:

- 메모리 효율적인 그래프 기반 방법의 지속적 개선
- 하드웨어 특화 최적화 (GPU, 벡터 프로세서)
- 동적 인덱싱 (데이터 추가/삭제 효율성)
- 멀티벡터 객체 검색 (복합 임베딩)

### 63.6 본 논문과의 관계

Exqutor의 벡터 검색 벤치마킹에서:

- **표준 지표 채택:** Big ANN이 제시한 recall, QPS 메트릭 활용
- **데이터셋 선택:** 다양한 데이터셋(SIFT, YFCC 등) 활용 정당화
- **성능 기준:** 상위 시스템의 성능 수준을 비교 기준으로 설정
- **메모리 제약 분석:** 현실적 메모리 제약 조건 하에서의 성능 평가

구체적으로: - SQL 쿼리와 벡터 검색의 결합에서 벡터 부분의 성능 최적화 기준 제공 - TPC-H 확장 시 벡터 검색의 예상 성능 수준 설정 - 다양한 벡터 데이터 타입(이미지, 텍스트)에서의 성능 차이 예측

---

## 추가 제기 문제

1. **하드웨어 의존성:** Big ANN 경쟁이 특정 CPU(예: AMD EPYC)에서 수행되었는데, 다른 하드웨어(ARM, GPU)에서는 순위가 얼마나 바뀌는가?
  2. **데이터 분포의 영향:** SIFT(매우 높은 차원도, 희소성)와 MSMARCO(낮은 차원도, 밀집성)에서 최적 알고리즘이 다른가? 하이브리드 시스템은 가능한가?
  3. **동적 업데이트:** Big ANN이 정적 인덱스만 평가했는데, 벡터가 지속적으로 추가되는 환경에서의 성능은? 온라인 알고리즘과의 성능 격차는?
  4. **메모리-정확도의 한계:** 메모리 1GB에서 recall 60-70%는 현실적으로 충분한가? 응용 사례별로 필요한 최소 recall은?
  5. **쿼리 배치 효과:** Big ANN이 배치 쿼리(여러 쿼리 동시 처리)를 평가했는가? 배치 크기 변화에 따른 성능 변동은?
  6. **하이브리드 쿼리:** 벡터 검색 + 메타데이터 필터의 복합 쿼리는 Big ANN에서 평가되었는가? 필터 선택도(selectivity)에 따른 성능 변화는?
-

## [64] Wikipedia (December 2022) Dataset: Cohere Embeddings

### 요약

본 자료는 Cohere가 2025년 HuggingFace에서 공개한 Wikipedia 데이터셋을 이용한 임베딩 리소스이다. 이 데이터셋은 2022년 12월의 위키백과 스냅샷을 기반으로 생성된 임베딩 벡터와 텍스트 콘텐츠를 포함하고 있으며, 자연어 처리와 의미론적 검색 연구에 광범위하게 사용된다.

**위키백과 데이터셋의 특징:** - 규모: 600만 개 이상의 위키백과 문서 - 품질: 커뮤니티 큐레이션된 백과사전 항목 - 다국어: 주로 영문이지만 다양한 주제 - 구조화: 문서 제목, 본문, 섹션, 링크 정보 포함 - 임베딩: Cohere의 임베딩 모델로 생성된 벡터

**Cohere 임베딩의 특성:** - 차원도: 보통 1024 또는 4096차원 - 훈련 데이터: 대규모 텍스트 코퍼스에서 학습 - 의미론적 유사성: 단어의 문맥적 의미를 인코딩 - 다양한 타입: 문서 임베딩, 문장 임베딩, 단어 임베딩

**텍스트 검색에서의 활용:** - 의미론적 문서 검색 - FAQ 매칭 - 추천 시스템의 기초 - 정보 검색(information retrieval) 벤치마킹

Exquitor의 TPC-H 확장에서 WIKI 데이터셋으로 활용되며, 관계형 데이터와 벡터 검색의 결합을 평가하는 데 중요한 역할을 한다.

### 상세분석

#### 64.1 위키백과 데이터의 특성

##### 1.1 규모와 다양성 위키백과 2022년 12월 스냅샷:

- 약 600만 개의 영문 항목
- 지역(Geography), 과학(Science), 역사(History), 문화(Culture) 등 광범위 커버
- 각 항목: 수백 단어에서 수만 단어 범위
- 구조: Infobox, 섹션, 외부 링크 등 풍부한 메타정보

##### 1.2 텍스트 품질 공개 백과사전의 특징:

- 철저한 검수와 편집 프로세스
- 신뢰할 만한 출처 인용 강조
- 언어 품질 상대적으로 높음
- 클라우드소싱이지만 모더레이션된 환경

이는 웹 수집 텍스트보다 훨씬 깨끗한 데이터를 의미한다.

##### 1.3 다양한 도메인 커버 다양한 주제의 항목:

- 과학 기술: 물리, 화학, 컴퓨터 과학
- 역사: 특정 사건, 인물, 시대
- 지리: 국가, 도시, 자연 지형
- 문화: 예술, 음악, 문학
- 스포츠, 게임 등

이러한 다양성은 일반 목적의 임베딩 모델 평가에 이상적이다.

## 64.2 Cohere 임베딩의 특성

### 2.1 임베딩 모델의 진화 자연어 임베딩의 발전:

- Word2Vec/GloVe: 단어 수준 임베딩, 고정 차원
- BERT: 문맥 기반 표현, 변수 길이
- Sentence-BERT: 문장 수준 임베딩, 의미론적 유사성 최적화
- Cohere 임베딩: 특화된 인코더, 다양한 차원 옵션

### 2.2 차원도와 성능

1024차원 버전:

장점: 메모리 효율적, 계산 빠름

단점: 고도의 정보 압축, 세밀한 구분 어려움

4096차원 버전:

장점: 풍부한 의미 표현, 높은 정확도

단점: 메모리 사용 증가, 계산 비용 4배

선택 기준: 정확도 vs 효율성 트레이드오프

### 2.3 의미론적 유사성의 인코딩 두 문서가 의미적으로 유사할 때:

- 계산된 임베딩 벡터의 코사인 유사도가 높음
- "강아지"와 "개"의 거리 < "강아지"와 "우주"의 거리
- 하지만 단어 기반이 아닌 개념 기반 매칭
- 동의어, 패러프레이즈도 높은 유사도

## 64.3 텍스트 검색에서의 활용

### 3.1 의미론적 검색 파이프라인

사용자 쿼리: "기계 학습이란?"

↓

쿼리 임베딩화 (동일 모델 사용)

↓

벡터 인덱스 검색

↓

상위 k개 가장 유사한 위키백과 항목 반환

예: "Machine learning", "Deep learning", "Neural networks"

↓

원문 제시 (검색 결과)

### 3.2 기존 텍스트 검색과의 비교 전통적 키워드 검색의 한계:

쿼리: "자동차 배출 공기"

키워드 검색: "자동차", "배출", "공기" 포함 항목만

문제: 정확히는 "배출 가스" 또는 "대기 오염"을 다루는 항목 필요

의미론적 검색:

- 쿼리의 의미(대기 환경 오염)를 이해
- 관련 항목 찾기 성공률 높음
- "대기 환경", "자동차 공해" 등도 포함

### 3.3 FAQ와 고객 문의 실무 사례:

고객: "이 제품에서 계산 속도가 어떻게 되는가?"

FAQ 데이터베이스:

- "성능이 좋은가?" (키워드 검색 실패)
- "얼마나 빨린가?" (의미론적 검색 성공)

의미론적 검색이 문제 해결에 훨씬 효과적

## 64.4 TPC-H 확장에서의 WIKI 데이터셋

### 4.1 TPC-H와 벡터 검색의 통합 전통적 TPC-H:

- 판매, 부품, 고객 등의 관계형 데이터
- SQL 쿼리의 성능 평가

확장된 TPC-H (벡터 검색 포함):

- 기존 관계형 데이터 유지
- 추가: 제품 설명, 고객 리뷰 등의 텍스트 필드
- 이를 임베딩으로 변환하여 벡터 검색 포함

### 4.2 하이브리드 쿼리의 예

쿼리: 판매액이 많으면서 고객 리뷰가 긍정적인 부품

관계형 부분:

```
SELECT part_id, SUM(sales) FROM sales
WHERE date > '2020-01-01'
GROUP BY part_id
```

벡터 검색 부분:

```
SELECT part_id FROM parts
WHERE customer_review_embedding
 SIMILAR_TO "high quality, excellent performance"
```

### 4.3 성능 평가 포인트

1. 벡터 필터링이 전체 쿼리에 미치는 영향
  - 관계형만 vs 벡터 조건 추가
2. 결과 정확도
  - 벡터 검색의 부정확성이 최종 결과 품질에 미치는 영향
3. 실행 계획
  - 관계형 필터 먼저 vs 벡터 필터 먼저
  - 선택도(selectivity)에 따른 최적화
4. 확장성
  - 벡터 데이터 크기 증가 시 성능 변화

## 64.5 데이터셋의 한계와 편향

### 5.1 위키백과의 구조적 편향

언어: 영문 위키백과 중심

- 비영어 주제 과소 표현
- 언어별 문화의 다양한 관점 반영 부족

내용 편향:

- 서방 중심의 관점과 정보
- 특정 주제(기술, 문학)는 깊이 있고
- 다른 주제(비서방 역사)는 얕음

저자 편향:

- 기여자 대부분이 선진국, 특정 교육 배경
- 특정 이데올로기 관점이 반영될 수 있음

### 5.2 임베딩의 한계



시간 민감성:

- 2022년 12월 스냅샷 기반
- 2024년 사건/인물은 포함 안 됨
- 과학 최신 발전 반영 안 됨

구조 정보 손실:

- Infobox, 섹션 구조 임베딩에 반영 안 됨
- 순수 텍스트만 임베딩화
- 서로 다른 구조 문서가 혼동될 수 있음

의미 한계:

- 동음이의어 처리 어려움 (특히 짧은 쿼리)
- 특수 도메인 용어의 정확한 이해 부족

## 64.6 본 논문과의 관계

Exqutor의 TPC-H 벡터 확장에서:

- **데이터셋 제공:** 600만 개의 임베딩된 텍스트 항목 제공
- **현실성:** 실제 의미론적으로 풍부한 콘텐츠의 벡터 검색 특성
- **다양성:** 다양한 도메인의 텍스트로 일반화 평가 가능
- **확장성 테스트:** 대규모 텍스트 검색 시스템의 성능 평가

구체적으로 Exqutor는: - WIKI 임베딩을 TPC-H 쿼리에 통합 - 예: "특정 고객이 관심 있는 제품 검색" (기존 관계형 필터 + 벡터 유사도) - 벡터 검색의 정확도가 최종 쿼리 결과에 미치는 영향 평가 - 대규모 텍스트 데이터에서의 인덱싱, 검색 성능 측정

## 추가 제기 문제

1. **시간 민감성 처리:** 2022년 스냅샷 기반 임베딩이 2024년 쿼리에 얼마나 유효한가? 계절적, 주기적 업데이트는 필요한가?
2. **다국어 확장 가능성:** Cohere의 다국어 임베딩 모델이 있다면, 위키백과 다국어 버전에 적용했을 때 성능은 얼마나 다를까?
3. **임베딩 차원 선택:** 1024 vs 4096차원 임베딩을 선택할 때 정확도-속도 트레이드오프는 정량적으로 어떻게 되는가?
4. **특수 도메인 성능:** 과학 기술 분야의 정확도 vs 역사/문화 분야의 정확도가 얼마나 다른가? 도메인 특화 임베딩이 필요한가?
5. **쿼리의 길이 영향:** 짧은 쿼리(1-3단어) vs 긴 쿼리(문장)에서의 검색 정확도 차이는? 동음이의어 처리는?

6. **메타데이터 활용**: 위키백과의 Infobox, 카테고리, 링크 정보를 임베딩에 어떻게 통합할 것인가? 구조화된 정보가 의미론적 검색 정확도를 향상시키는가?
7. **필터 조합의 성능**: 벡터 유사도만 사용 vs 벡터 유사도 + 카테고리 필터 + 언어 필터를 결합할 때 최적 실행 계획은?

---

## [65] Quantifying TPC-H Choke Points and Their Optimizations

### 요약

본 논문은 TPC-H 벤치마크의 쿼리 성능을 분석하여 각 쿼리에서의 병목(choke point)을 정량화하고 최적화 기법의 효과를 평가한다. 2020년 VLDB(매우 영향력 있는 데이터베이스 학회)에 발표된 Dreseler 등의 연구는 TPC-H가 어떤 쿼리에서는 효과적 벤치마크이고, 어떤 쿼리에서는 한계가 있는지를 체계적으로 분석한다.

**TPC-H 벤치마크의 개요**: - 22개의 분석용 SQL 쿼리 - 1GB에서 수 TB 규모의 데이터 - 관계형 데이터베이스의 분석 워크로드 대표 - 가장 널리 사용되는 상용 DB 벤치마크

**병목(Choke Point) 분석의 중요성**: - 특정 쿼리는 조인 최적화 부족을 반영 (Q17) - 특정 쿼리는 집계 성능을 반영 (Q1) - 특정 쿼리는 인덱싱 기법을 반영 (Q19) - 어떤 쿼리들은 현대 시스템에서 너무 쉬움

**주요 발견**: - 각 쿼리마다 서로 다른 최적화 기법이 필요 - 기존 시스템의 최적화가 특정 패턴에만 효과적 - 벡터 검색 같은 새로운 워크로드 추가 시 기존 재설정 필요

Exquitor가 TPC-H를 벡터 검색으로 확장할 때, 이 분석은 어떤 쿼리가 벡터 부분을 추가할 가치가 있는지, 어떻게 측정할 것인지에 대한 기초를 제공한다.

---

### 상세분석

#### 65.1 TPC-H의 22개 쿼리 분류

##### 1.1 각 쿼리의 주요 특징

Q1: 집계 중심 쿼리 - 단일 테이블(LINEITEM)에서 대량의 행 필터링과 집계 - 스캔, 필터, 그룹화 최적화 측정 - 현대 DBMS에서는 매우 효율적

Q17, Q20: 조인 최적화 - 복잡한 조인 구조 - 조인 순서, 조인 방식 최적화가 성능에 직결 - 쿼리 최적화기의 핵심 능력 측정

Q5, Q10: 윈도우 함수와 집계 - 여러 테이블 조인 후 집계 - I/O 비용과 메모리 사용의 균형

##### 1.2 병목의 정의 Dreseler 논문에서 제시하는 병목:

성능 병목 = 전체 실행 시간의 20% 이상 차지하는 연산

예:

- Q1: 전체 시간의 70% = SCAN + GROUP BY → 명확한 병목
- Q3: JOIN이 30%, SORT가 25%, SCAN이 20% → 다중 병목
- Q8: 모든 연산이 10% 미만 → 구체적 병목 없음

## 65.2 병목 분석 결과

### 2.1 스캔 병목 테이블 스캔이 주요 병목인 쿼리:

- Q1, Q6: 대량 데이터 스캔
- 해결책: 인덱싱, 파티셔닝, 컬럼 스토어
- 현대 DBMS의 최적화로 비교적 잘 해결됨

### 2.2 조인 병목

중첩 루프(Nested Loop) 조인:

- 가장 간단하지만 느림  $O(n*m)$
- 작은 데이터셋에는 괜찮음

해시 조인(Hash Join):

- 메모리가 있으면 매우 효율적  $O(n+m)$
- 메모리 부족하면 극적으로 느려짐

정렬 병합(Sort-Merge):

- 이미 정렬된 입력에 효율적
- 정렬 비용이 크면 비효율적

Q5의 경우: - 4개 테이블의 조인 - 조인 순서가 성능에 10배 이상 영향 - 일부 DBMS는 좋은 순서를 찾지 못함

### 2.3 집계 병목

작은 그룹 수:

- 해시 집계 매우 효율적
- GROUP BY 병목 아님

큰 그룹 수:

- 메모리 오버플로우 위험
- 정렬 기반 집계 필요
- 디스크 I/O 증가

Q1에서 명확한 병목: - 날짜 범위별 그룹화 - 그룹 수는 작음 (수백개) - 실제 병목: 수억 개 행의 스캔과 필터

### 2.4 정렬 병목

메모리 내 정렬:

- 매우 빠름

외부 정렬(External Sort):

- 메모리 초과 시 발생
- 다중 pass로 인한 디스크 I/O
- 성능 급격히 저하

Q3, Q10 같은 쿼리에서: - ORDER BY 절이 명시적 병목 - 중간 결과 크기가 메모리 한계 초과

## 65.3 최적화 기법의 효과 측정

### 3.1 인덱싱의 효과

인덱스 없음:

- Q6 실행시간: 1000ms
- 전체 LINEITEM 테이블 스캔 필요

인덱스 있음 (L\_SHIPDATE):

- Q6 실행시간: 50ms
- 20배 성능 개선

제약: - 모든 쿼리가 같은 인덱스로 이득 받지는 않음 - 인덱스 유지 비용 (INSERT/DELETE/UPDATE 느려짐)

### 3.2 파티셔닝의 효과

날짜별 파티셔닝:

- LINEITEM을 연도별로 분할
- 범위 검색 쿼리에서 스캔량 감소
- Q1 같은 쿼리에서 성능 향상

제약:

- 복잡한 조인 쿼리에서는 이득 적을 수 있음
- 파티션 프루닝(pruning) 기능 필요

### 3.3 컬럼 스토어의 효과 전통 행 스토어 (Row Store):

테이블:

```
| ID | Name | Price | Date |
|1|Apple|$1|2020|
|2|Banana|$0.5|2020|
```

모든 필터링에도 전체 행 디코딩

컬럼 스토어 (Column Store):

Price 컬럼만 필요하면:  
Price: [1, 0.5, ...]  
Date: [2020, 2020, ...]  
필요한 컬럼만 로드

결과: Q1 같은 단순 쿼리에서 10배 이상 성능 향상

## 65.4 시스템별 성능 차이

### 4.1 상용 DBMS vs 오픈소스

상용 DBMS (Oracle, SQL Server):

- 복잡한 조인 최적화 우수
- 벡터화 실행 등 최신 기법 적용
- Q17 같은 복잡한 쿼리에서 강함

오픈소스 (PostgreSQL):

- 간단한 쿼리에서는 경쟁력 있음
- 복잡한 쿼리 최적화는 뒤떨어짐
- 하지만 지속적으로 개선 중

### 4.2 데이터 크기별 성능 차이

1GB 스케일:

- 모든 데이터가 메모리에 올라옴
- 인덱싱, 파티셔닝의 효과 적음
- 대부분의 시스템 비슷한 성능

100GB 스케일:

- 메모리 관리가 중요해짐
- 시스템 간 성능 차이 증대
- 조인 순서 최적화의 중요성 증가

1TB 이상:

- 디스크 I/O가 절대적 병목
- 인덱싱, 파티셔닝 매우 중요
- 시스템 간 성능 격차 극대화

## 65.5 TPC-H 벤치마크의 현대적 한계

### 5.1 스키마의 단순성

현대 데이터 특성:

- 비정규화된 스키마 (NoSQL의 영향)
- 복잡한 데이터 타입 (JSON, 배열, 맵)
- 다양한 정렬도(cardinality) 분포

TPC-H의 단순함:

- 정규화된 스키마만 사용
- 기본 데이터 타입만
- 비교적 균등한 데이터 분포

## 5.2 쿼리 복잡도

TPC-H의 쿼리들:

- 대부분 10개 이하의 조인
- 명확한 WHERE 조건

현대 분석 워크로드:

- 20개 이상의 조인 가능 (DataLake)
- 중첩 쿼리, CTEs의 복잡한 사용
- 동적 쿼리 생성

## 5.3 벡터 검색 부재

TPC-H의 한계:

- 정확한 매칭만 가능
- 유사도 기반 검색 없음
- 의미론적 분석 불가능

현대 요구사항:

- 고객 리뷰와 텍스트 유사도 검색
- 이미지 유사도 기반 추천
- 의미론적 검색과 정확한 검색의 결합

## 65.6 본 논문과의 관계

Exqutor의 TPC-H 벡터 확장에서:

- **기존 병목 이해:** 벡터 검색 추가 전 TPC-H의 기존 병목 파악
- **쿼리 선택:** 어떤 쿼리를 벡터 부분으로 확장할 것인가
- **성능 기준선:** 벡터 검색 추가 전의 성능을 기준선으로 설정
- **새로운 병목:** 벡터 검색 추가 후 새로운 병목 출현 분석

구체적으로: - Q1 (집계 쿼리)에 텍스트 검색 추가 → 집계가 여전히 병목? - Q5 (조인 쿼리)에 벡터 필터 추가 → 조인 순서 최적화에 영향? - 벡터 검색으로 인한 I/O, 메모리 변화 정량화 - 새로운 하이브리드 최적화 기법의 효과

## 추가 제기 문제

1. **벡터 필터의 선택도 변화:** 벡터 검색 추가 시 중간 결과의 크기(선택도)가 어떻게 변할 것인가? 이것이 다운스트림 조인의 성능에 어떤 영향을 미칠 것인가?
2. **병목의 전환:** 원래 집계가 병목이던 쿼리에 벡터 검색을 추가할 때, 집계가 여전히 병목이 될 것인가, 아니면 벡터 검색으로 전환될 것인가?
3. **인덱싱 전략의 변화:** 벡터 필터와 관계형 필터를 모두 포함하는 쿼리에서 최적의 인덱싱 전략은? 벡터 인덱스를 먼저 사용할 것인가, 관계형 인덱스를 먼저 사용할 것인가?
4. **데이터 크기별 확장성:** Dreseler 논문의 분석이 벡터 데이터 추가 시에도 여전히 유효한가? 벡터 데이터의 메모리/I/O 특성이 다른 병목을 만들 것인가?
5. **다중 벡터 필드:** 실제 응용에서는 여러 텍스트 필드(제품 설명, 고객 리뷰, 태그 등)가 있을 때, 어떤 필드를 먼저 임베딩할 것인가? 성능 영향은?
6. **정적 vs 동적 최적화:** TPC-H의 쿼리는 정적이지만, 실제로는 벡터 검색 정확도가 동적으로 변할 수 있다. 이것이 최적 실행 계획 선택에 어떻게 영향을 미칠 것인가?

## [66] Why You Should Run TPC-DS: A Workload Analysis

### 요약

본 논문은 2007년 VLDB에서 발표된 Poess, Nambiar, Walrath의 연구로, TPC-DS 벤치마크가 현대 데이터 웨어하우스 워크로드를 더 현실적으로 반영하는 이유를 설명한다. TPC-H가 분석 쿼리의 기본이라면, TPC-DS는 더 복잡하고 다양한 현실 세계의 데이터 분석을 모사한다.

#### TPC-DS vs TPC-H 비교:

TPC-H의 한계: - 단일 도메인(판매) 초점 - 비교적 단순한 스키마 - 정적인 데이터 세트

TPC-DS의 특징: - 다양한 판매 채널 (온라인, 오프라인, 카탈로그) - 더 복잡한 비즈니스 로직 - 현실적인 데이터 분포 편향 - 다양한 쿼리 타입 (OLAP, 텍스트 검색 요소 등)

**TPC-DS의 99개 쿼리:** - 다양한 분석 패턴 - 복합 필터링과 조인 - 윈도우 함수, 시계열 분석 - 자체 조인(self-join) 등 다양한 기법

**현실성 증대:** - 실제 소매점 운영에서 필요한 분석 반영 - 데이터의 비균등 분포 - 계절성, 트렌드 분석의 필요성

Exquitor에서 TPC-DS를 고려할 때, 벡터 검색을 추가한 확장 벤치마크가 더욱 현실적이 될 수 있으며, 다양한 실제 워크로드를 반영하는 평가가 가능하다.

## 상세분석

### 66.1 TPC-H의 한계와 TPC-DS의 필요성

#### 1.1 TPC-H의 설계 목표와 한계

설계 목표:

- SQL 쿼리 성능 비교의 표준화
- 상대적으로 단순하고 이해하기 쉬운 구조
- 재현 가능한 실험 환경

현실과의 차이:

1. 단일 도메인: 항공 우주 부품 공급망
2. 균등 분포: 랜덤하게 생성된 데이터
3. 단순 스키마: 8개 테이블만 사용
4. 정적 워크로드: 고정된 22개 쿼리

#### 1.2 실제 데이터 웨어하우스의 특징

다양한 데이터 출처:

- POS(판매점) 데이터
- 온라인 주문 데이터
- 고객 관계 정보
- 재고 정보
- 마케팅 캠페인 데이터

비균등 분포:

- 인기 상품은 자주, 비인기 상품은 드물게 팔림
- 특정 시간(휴가)에 판매 집중
- 지역별 선호도 편차

복잡한 비즈니스 로직:

- 다중 채널 데이터 통합
- 반품, 교환 처리
- 프로모션 효과 분석

### 66.2 TPC-DS의 구조와 특징

#### 2.1 스키마의 복잡성



TPC-DS 테이블 구성 (99개 중 일부):

고객 관련:

- customer: 기본 고객 정보
- customer\_address: 배송/청구 주소
- customer\_demographics: 인구 통계

판매 채널:

- store: 오프라인 매장 정보
- web\_site: 온라인 판매 채널
- catalog\_page: 카탈로그 페이지

거래 데이터:

- store\_sales: 매장 판매
- web\_sales: 온라인 판매
- catalog\_sales: 카탈로그 판매

상품 관련:

- item: 상품 기본 정보
- inventory: 재고
- promotion: 프로모션

총 99개 테이블 vs TPC-H의 8개

## 2.2 데이터의 현실성

Zipf 분포 (현실의 히트상품 패턴):

- 소수의 상품이 대부분 판매
- 다수의 상품이 거의 팔리지 않음
- 키워드 검색처럼 장꼬리 분포

계절성 데이터:

- 특정 시즌(크리스마스, 여름) 판매 급증
- 요일별 패턴 (주말 증가)
- 휴일 효과

지역별 편차:

- 도시 vs 시골 지역의 선호도 차이
- 문화권별 상품 선호도
- 배송비 등의 지역별 차별화

## 66.3 TPC-DS의 99개 쿼리 특성

### 3.1 쿼리 복잡도 분류

Type A: 단순 쿼리 (약 30개)

- 1-2개 테이블, 간단한 WHERE 절
- 기본 집계 함수

Type B: 중간 복잡도 (약 50개)

- 3-5개 테이블 조인
- 복합 필터링
- 윈도우 함수

Type C: 고복잡도 (약 19개)

- 6개 이상 테이블 조인
- 자체 조인(self-join) 포함
- 상관 서브쿼리
- CTE(Common Table Expression) 사용

### 3.2 분석 패턴

시계열 분석:

Q11: 작년 대비 올해 고객 성장률

- 연도별 비교
- CASE문과 윈도우 함수

마켓 바스켓 분석:

Q33: 함께 구매되는 상품 조합

- 자체 조인으로 같은 고객의 구매 패턴
- 상관관계 분석

고객 분할:

Q48: 구매 패턴별 고객 세분화

- 다중 조인
- 복합 그룹화

상품 추천:

Q20: 자주 함께 팔리는 상품

- 판매 데이터의 연관성 분석

### 3.3 현실성이 높은 이유

비즈니스 질문과의 직결:

- "이번 분기 판매 추이는?"
- "어떤 제품이 인기인가?"
- "고객 만족도는?"
- "마케팅 효과는?"

이 질문들이 TPC-DS 쿼리에 반영됨

## 66.4 TPC-H와 TPC-DS의 성능 차이

### 4.1 쿼리 복잡도의 영향

평균 실행 시간 (1TB 데이터):

TPC-H:

- Q1: 1초
- Q5: 5초
- Q17: 30초
- 평균: 10초

TPC-DS:

- Q11: 15초 (자체 조인)
- Q33: 45초 (복잡한 마켓 바스켓)
- Q48: 90초 (3중 조인 + 서브쿼리)
- 평균: 40초

→ 4배 더 복잡한 워크로드

### 4.2 최적화 기법의 차이

TPC-H:

- 인덱싱으로 큰 성능 향상
- 조인 순서 최적화 중요
- 기존 DBMS 최적화로 충분

TPC-DS:

- 인덱싱 효과 제한적 (많은 조인으로 인해)
- 쿼리 재작성(rewrite) 필요
- 통계 기반 카디널리티 추정의 한계 드러남

## 66.5 벡터 검색 통합의 의미

### 5.1 TPC-DS에 벡터 검색 추가의 현실성

현재 TPC-DS:

- 수치, 날짜, 범주형 데이터만
- 텍스트 검색은 정확히 매칭

현실의 데이터 웨어하우스:

- 고객 리뷰 (자유 텍스트)
- 상품 설명 (텍스트)
- 피드백 코멘트 (텍스트)
- 이미지 (비정형 데이터)

벡터 검색 추가의 필요성:

- "긍정적인 피드백을 남긴 고객"
- "특정 스타일의 옷을 찾는 고객"
- "유사한 리뷰를 남긴 다른 고객"

## 5.2 하이브리드 쿼리의 예

전통적 TPC-DS Q33 (마켓 바스켓):

```
SELECT product1, product2, COUNT(*)
FROM store_sales s1
JOIN store_sales s2
 ON s1.customer_id = s2.customer_id
WHERE s1.item_id < s2.item_id
GROUP BY product1, product2
```

확장된 버전 (벡터 검색 추가):

```
SELECT product1, product2, COUNT(*)
FROM store_sales s1
JOIN store_sales s2
 ON s1.customer_id = s2.customer_id
WHERE s1.item_id < s2.item_id
 AND item_description_embedding SIMILAR_TO
 (SELECT description_embedding FROM item WHERE id = ?)
GROUP BY product1, product2
```

의미: 특정 스타일의 상품을 자주 함께 구매하는 고객 찾기

## 66.6 현대적 확장 가능성

### 6.1 TPC-DS의 한계

정적 데이터:

- 한 번 생성 후 수정 없음
- 실제 데이터는 지속적 변화

고정 쿼리:

- 99개 쿼리는 모든 분석 패턴을 다 커버하지 못함
- 특히 텍스트, 이미지 검색은 완전히 빠짐

단일 도메인:

- 소매점만 다룸
- 금융, 의료, 전자상거래 등의 특수성 없음

## 6.2 벡터 검색 통합의 효과

다양한 데이터 타입 통합:

- 구조화(수치) + 반구조화(텍스트) + 비정형(이미지)

현실적 분석 쿼리:

- 단순 WHERE 필터 뿐 아니라
- 의미론적 유사도 기반 필터링

성능 이슈의 다양성:

- 전통적 쿼리 최적화 이슈
- 벡터 인덱싱의 효율성
- 혼합 조인 순서 최적화

## 6.7 본 논문과의 관계

Exquitor의 TPC-DS 확장 고려 시:

- **현실성 증대:** TPC-H보다 복잡한 TPC-DS에 벡터 검색 추가로 더욱 현실적
- **다양한 쿼리 패턴:** 99개 쿼리 중 어떤 것에 벡터 부분을 추가할 것인가
- **워크로드의 특성:** 마켓 바스켓, 시계열, 추천 등 다양한 비즈니스 분석
- **최적화 복잡도:** TPC-H보다 높은 복잡도로 인한 새로운 최적화 도전

구체적으로: - TPC-DS의 자체 조인 쿼리에 텍스트 유사도 필터 추가 - 고객 행동 분석 쿼리에 임베딩 기반 세분화 추가 - 성능 모니터링이 더욱 복잡해지는 양상 분석

## 추가 제기 문제

1. **스케일별 특성 변화:** TPC-DS를 1TB vs 10TB로 실행할 때 벡터 검색의 성능 특성이 어떻게 달라지는가? 특히 메모리 부족 시나리오에서?

2. **다중 테이블 텍스트 필드**: TPC-DS에는 여러 테이블에 텍스트 필드(customer\_comments, item\_description 등)가 있을 때, 어떤 필드를 임베딩할 것인가? 조인 순서에 미치는 영향은?
3. **계절성 데이터와 벡터 검색**: 계절성이 있는 데이터에서 일정 시간 전의 데이터로 학습한 임베딩 모델의 유효성은? 시간에 따른 드리프트(drift) 처리는?
4. **자체 조인과 벡터 필터**: TPC-DS의 많은 쿼리가 자체 조인을 사용하는데, 유사 고객 또는 유사 상품을 찾는 벡터 필터를 추가할 때 조인 복잡도는 어떻게 변할 것인가?
5. **99개 쿼리의 균등성**: TPC-DS의 99개 쿼리 중 벡터 검색이 의미 있는 이득을 줄 수 있는 쿼리는 몇 개인가? 특정 유형의 쿼리만 혜택을 받을 것인가?
6. **실시간 분석의 추가**: TPC-DS는 배치 쿼리만 다루는데, 스트리밍 데이터와 벡터 검색의 결합은? 임베딩 모델 업데이트 빈도는?
7. **비용 모델의 재정의**: 벡터 데이터 추가로 인한 비용 모델(CPU, I/O, 메모리) 재정의는? 기존의 통계 기반 비용 모델이 벡터 부분도 적절히 예측할 수 있는가?

## [67] Determining Sample Size: Statistical Sampling Theory

### 요약

본 논문은 1992년 Israel 등에 의해 발표된 통계학 기반의 표본 크기 결정 방법론이다. 이는 대규모 모집단에서 통계적으로 유의미한 결론을 도출하기 위해 필요한 최소 표본 크기를 계산하는 방법을 제시한다. 이 이론은 실험 설계, 시뮬레이션, 벤치마크 연구에서 핵심적인 역할을 한다.

**표본 크기 결정의 중요성**: - 너무 작은 표본: 통계적 신뢰도 낮음, 결론 신뢰할 수 없음 - 너무 큰 표본: 불필요한 계산 비용, 실험 기간 연장 - 최적 표본: 통계적 유의성 + 실무적 효율성의 균형

**핵심 개념**: - 신뢰도(Confidence Level): 일반적으로 95%, 99% - 오차 한계(Margin of Error): 결과가 벗어날 수 있는 범위 - 표준편차(Standard Deviation): 데이터의 변동성 - 신뢰 구간(Confidence Interval): 모수가 포함될 범위

**표본 크기 공식**:

$$n = (Z * \sigma / E)^2$$

n: 필요한 표본 크기

Z: 신뢰도에 따른 값 (95% → 1.96, 99% → 2.576)

σ: 모집단의 표준편차

E: 허용 오차 한계

**실무 응용**: - 데이터베이스 성능 테스트 - 머신러닝 모델의 샘플 크기 - A/B 테스트 설계 - 시뮬레이션 실험

Exqutor에서 벡터 검색 시스템의 성능을 평가할 때, 초기 표본 크기를 결정하는 데 이 통계 이론이 기초가 된다. 특히 대규모 데이터셋(10억 벡터)에서 현실적인 표본 크기를 선택하기 위해 이 방법론을 적용한다.

## 상세분석

### 67.1 통계 표본 추출의 기초

#### 1.1 모집단 vs 표본

모집단(Population):

- 조사 대상이 되는 전체 집합
- 예: 1억 개의 벡터 모두

표본(Sample):

- 모집단에서 선택된 부분집합
- 예: 1억 개 벡터 중 1000개 선택

실제 상황:

- 모집단 전체 분석은 비용이 매우 높음 (시간, 계산)
- 대신 표본으로 모집단의 특성 추정

#### 1.2 표본 추출의 종류

확률 표본 추출:

1. 단순 무작위 표본 추출(Simple Random Sampling)
  - 각 원소가 선택될 확률이 동일
  - 가장 편향 없는 방법
2. 계층 표본 추출(Stratified Sampling)
  - 모집단을 층(strata)으로 나누고 각 층에서 표본 추출
  - 예: 크기별 벡터 클러스터에서 각각 추출
3. 군집 표본 추출(Cluster Sampling)
  - 모집단을 군집으로 나누고 일부 군집 전체 선택
  - 예: 벡터 공간의 일부 지역만 선택

비확률 표본 추출:

- 편의 표본 추출(Convenience Sampling)
- 의도 표본 추출(Purposive Sampling)
- 편향 가능성 높음 (벤치마킹에는 부적절)

#### 1.3 표본 크기의 역할

표본이 작을 때:

- 표본 평균의 변동성 큼
- 신뢰 구간 넓음
- 통계적 검정력 낮음

표본이 클 때:

- 표본 평균이 모집단 평균에 가까움
- 신뢰 구간 좁음
- 통계적 검정력 높음
- 하지만 계산 비용 증가

## 67.2 표본 크기 결정 공식

### 2.1 기본 공식

신뢰 구간 기반:

$$n = (Z * \sigma / E)^2$$

여기서:

- Z: 표준정규분포의 임계값
  - \* 95% 신뢰도 → Z = 1.96
  - \* 99% 신뢰도 → Z = 2.576
  - \* 90% 신뢰도 → Z = 1.645
- $\sigma$ : 모집단의 표준편차
  - \* 사전에 알려지지 않으면 예비 표본에서 추정
- E: 허용 오차 한계
  - \* 평균값의  $\pm$ 몇 %까지 허용할 것인가
  - \* 예: 평균 레이턴시의  $\pm 10\text{ms}$

### 2.2 예제: 벡터 검색 레이턴시



상황:

- 벡터 검색 시스템의 쿼리 레이턴시 측정
- 목표: 모집단 평균 레이턴시를  $\pm 5\text{ms}$  오차 범위로 추정 (95% 신뢰도)
- 예비 조사: 표준편차  $\sigma = 20\text{ms}$

계산:

$$\begin{aligned} n &= (1.96 * 20 / 5)^2 \\ &= (7.84)^2 \\ &= 61.5 \\ &\approx 62\text{개 쿼리} \end{aligned}$$

해석:

- 최소 62개의 쿼리를 실행해야 함
- 이로서 95% 신뢰도로 평균 레이턴시를  $\pm 5\text{ms}$  범위로 추정 가능

## 2.3 비율(Proportion) 추정용 공식

$$n = (Z / E)^2 * p * (1-p)$$

- $p$ : 예상 비율 (예: recall rate)
- $(1-p)$ : 그 반대 비율
- $p * (1-p)$ : 최대값 0.25 ( $p=0.5$ 일 때)

예: 벡터 검색 정확도(recall@100)

- 목표: 정확도를  $\pm 5\%$  오차 범위로 추정 (95% 신뢰도)
- 예상 recall: 90%

$$\begin{aligned} n &= (1.96 / 0.05)^2 * 0.9 * 0.1 \\ &= (39.2)^2 * 0.09 \\ &= 138 \end{aligned}$$

→ 최소 138개 쿼리 필요

## 67.3 Exqutor 논문에서의 적용

3.1 초기 표본 크기 결정 Exqutor는 대규모 데이터셋에서 벡터 검색 성능을 평가할 때:

문제 정의:

- 1억 개의 벡터 검색 성능 평가
- 모든 벡터에 대해 처리하면 시간이 매우 오래 걸림
- 대표성 있는 표본의 크기는?

적용:

1. 목표 정확도: 평균 레이턴시  $\pm 10\%$  (95% 신뢰도)
2. 예비 조사: 1000개 벡터로 미리 테스트
  - 평균 레이턴시: 50ms
  - 표준편차: 8ms
  - 변동계수(CV):  $8/50 = 16\%$
3. 표본 크기 계산:
  - 오차 한계:  $50\text{ms} * 10\% = 5\text{ms}$
  - $n = (1.96 * 8 / 5)^2 = 9.83^2 \approx 97\text{개}$
4. 결론: 최소 100개의 벡터 쿼리가 표본으로 충분

### 3.2 실제 구성

계층 표본 추출 적용:

- 데이터: 1억 개 벡터
- 클러스터: 차원도별로 분류
- 각 클러스터에서 비례적으로 표본 추출

이점:

- 다양한 벡터 특성 반영
- 편향 감소
- 표본 크기 최적화 가능

### 67.4 통계적 검정과의 연관

#### 4.1 가설 검정

귀무가설( $H_0$ ): 두 시스템의 성능이 같다

대립가설( $H_1$ ): 두 시스템의 성능이 다르다

예: 시스템 A vs 시스템 B의 레이턴시 비교

표본 크기와 검정력(Power):

- 작은 표본: 실제로 다른데 같다고 결론낼 수 있음 (제2종 오류)
- 적절한 표본: 실제 차이를 감지할 확률 높음

필요한 표본 크기 = 신뢰도 + 검정력 함수

- 신뢰도:  $\alpha = 0.05$  (95%)
- 검정력:  $\beta = 0.80$  (80% 확률로 차이 감지)

## 4.2 효과 크기(Effect Size)

Exqutor의 경우:

- 시스템 A 레이턴시: 50ms
- 시스템 B 레이턴시: 55ms
- 실제 차이: 5ms

이것이 통계적으로 유의미한가?

- 표본이 작으면: 우연의 변동으로 보일 수 있음
- 적절한 표본 크기로: 실제 차이인지 판단 가능

필요 표본 크기는 예상 차이의 크기에 반비례:

- 차이가 크면: 적은 표본으로 충분
- 차이가 작으면: 큰 표본 필요

## 67.5 실무적 고려사항

### 5.1 비용-편익 분석

추가 표본의 비용:

- 계산 비용: 각 쿼리마다 계산 필요
- 시간: 모든 쿼리 실행 완료까지 기다려야 함
- 저장소: 결과 저장 필요

추가 표본의 이득:

- 신뢰도 증가
- 오차 한계 감소
- 더 정확한 결론

최적 지점: 신뢰도와 비용의 균형

### 5.2 시간 제약

Exqutor 논문의 현실:

- 서로 다른 시스템들과 설정 비교
- 벡터 검색 시스템 평가는 시간이 오래 걸림

해결책:

- 초기 표본 크기: 충분히 신뢰할 수 있는 최소 크기
- 1억 벡터에서 수만~수십만 벡터 표본 사용
- 통계적으로 유의미한 결론 도출 가능

### 5.3 다양한 설정에서의 표본

Exqutor의 여러 실험:

- 다양한 벡터 데이터셋 (SIFT, YFCC, 텍스트)
- 다양한 인덱싱 방법
- 다양한 메모리 제약

각 설정마다 표본 크기를 독립적으로 결정하거나,  
보수적으로 모든 설정에 대해 동일하게 큰 표본 사용 가능

## 67.6 표본 크기의 한계와 주의

### 6.1 표본 크기만으로는 부족

좋은 표본 크기:

- 통계적으로는 중요
- 하지만 충분조건은 아님

도 고려해야 할 점:

1. 표본의 대표성: 크기만 크다고 대표성이 있는가?
2. 표본 추출 방법: 무작위 추출이 가능한가?
3. 측정 오차: 도구(벤치마크)의 신뢰성
4. 외부 효과: 다른 변수의 영향 (시스템 부하 등)

### 6.2 표본 편향

벡터 검색 벤치마킹 사례:

- 특정 차원도의 벡터만 사용 (편향)
- 특정 데이터 분포만 사용 (편향)
- 최적화된 하드웨어에서만 테스트 (편향)

결과: 통계적으로 정확하지만 현실성이 떨어질 수 있음

## 67.7 본 논문과의 관계

Exqutor의 실험 설계에서:

- 초기 표본 크기: 1억 벡터에서 몇 만개를 선택할 것인가 결정
- 신뢰도 설정: 95% vs 99% 신뢰도 선택에 따른 표본 크기 영향
- 다중 비교: 여러 시스템, 데이터셋, 설정 비교 시 표본 크기 조정
- 통계적 유의성: "이 성능 차이가 우연이 아닌가?" 판단 기준

구체적으로: - TPC-H 쿼리 실행: 전체 데이터 vs 표본 데이터 - 벡터 인덱싱: 모든 벡터 vs 대표 표본 - 성능 비교: 신뢰할 수 있는 최소 표본 크기 결정

## 추가 제기 문제

1. **다양한 벡터 특성:** 차원도, 분포, 특성이 다른 벡터 데이터셋(SIFT, YFCC, 텍스트 임베딩)에서 필요한 표본 크기가 얼마나 다른가?
2. **편향의 영향:** 계층 표본 추출 vs 단순 무작위 표본 추출에서 필요한 표본 크기의 차이는? 어떤 방법이 더 효율적인가?
3. **시간 의존성:** 시간이 지나면서 벡터 데이터의 분포가 변할 때(temporal drift), 표본 크기 결정은 어떻게 변해야 하는가?
4. **다중 메트릭:** 레이턴시, recall, 메모리 사용량 등 여러 메트릭을 동시에 평가할 때, 필요한 표본 크기는 단일 메트릭의 경우보다 얼마나 커져야 하는가?
5. **오차 한계의 선택:** Exqutor가 평균 레이턴시의  $\pm 10\%$  vs  $\pm 5\%$  vs  $\pm 1\%$  중 어느 것을 목표로 할 것인가? 실무적으로 의미 있는 차이는 얼마인가?
6. **신뢰도와 검정력:** 95% 신뢰도 + 80% 검정력 vs 99% 신뢰도 + 90% 검정력에서 필요한 표본 크기의 차이는?
7. **이상치 처리:** 극히 느리거나 빠른 쿼리(outlier)를 어떻게 처리할 것인가? 이것이 필요한 표본 크기에 어떻게 영향을 미치는가?
8. **초기 예비조사:** Exqutor가 정확한 표본 크기를 계산하기 위해 필요한 예비조사 규모는? 오버헤드는 얼마인가?

## [68] Learned Cardinalities: Estimating Correlated Joins with Deep Learning

### 요약

본 논문은 2018년 Kipf et al.이 제시한 선구적 연구로, 신경망을 활용하여 데이터베이스 조인 카디널리티(결과 행 수)를 추정하는 방법을 제안한다. 전통적인 통계 기반 카디널리티 추정의 한계를 극복하기 위해 딥러닝 모델을 도입했다. 특히 상관관계가 있는 속성들 간의 복잡한 의존성을 학습할 수 있다는 점이 핵심이다.

기존 데이터베이스 옵티마이저는 히스토그램이나 샘플링 기반 통계로 카디널리티를 추정했으나, 이들은 다중 속성 간의 복잡한 상관성을 포착하지 못했다. Learned Cardinalities는 심층 신경망(Deep Neural Networks)을 훈련하여 데이터 분포를 학습하고, 조인 조건에 따른 정확한 카디널리티를 예측한다.

모델 구조는 인코더-디코더 아키텍처로, 입력 속성값들을 벡터로 인코딩하고 조인 조건을 학습하여 출력 카디널리티를 도출한다. 실험 결과 기존 통계 방법 대비 평균 오류를 50% 이상 감소시켰다.

본 논문과의 관계: Exqutor는 선택도(selectivity) 추정에 있어서 통계 기반과 학습 기반의 하이브리드 접근을 취하고 있으며, Learned Cardinalities의 딥러닝 기반 접근 방식이 이론적 배경을 제공한다. 특히 범위 필터와 벡터 검색의 결합된 선택도 추정에서 비슷한 학습 패러다임을 활용한다.

## 상세분석

### 68.1 핵심 기술 및 방법론

**신경망 기반 카디널리티 추정의 정의:** Learned Cardinalities는 조인 카디널리티 추정을 함수 근사 문제(function approximation problem)로 재정의한다. 전통적 접근은 통계 공식에 의존했지만, 이 논문은 신경망이 데이터의 실제 분포를 직접 학습할 수 있다고 제안한다.

**모델 아키텍처:** - **입력층:** 각 속성의 범위 또는 값을 정규화된 벡터로 인코딩 - **은닉층:** 속성 간 상관성을 포착하기 위한 다층 퍼셉트론(MLP) - **출력층:** 예측된 카디널리티 값 - **손실함수:** 상대 오류(relative error)를 최소화하도록 설계

**학습 데이터 생성:** 실제 데이터베이스에서 다양한 조인 조건으로 쿼리를 실행하고 실제 카디널리티를 수집한다. 이를 통해 모델이 실제 데이터 분포의 특성을 반영하도록 훈련한다.

### 68.2 핵심 기여도

**다중 속성 상관성 학습:** 전통적 히스토그램은 개별 속성이나 2-way 상관성만 효율적으로 표현할 수 있으나, 신경망은 고차원 상관성을 암묵적으로 학습한다. 이는 실제 데이터베이스의 복잡한 종속성을 더 잘 포착한다.

**통계 기반 방법의 한계 극복:** - 히스토그램: 메모리 비용과 정확도의 트레이드오프 - 샘플링: 낮은 카디널리티 범위에서 부정확 - 독립성 가정: 실제 데이터의 상관성을 무시

신경망은 이러한 제약을 우회하여 더 정확한 예측을 제공한다.

**성능 개선:** 벤치마크 데이터셋(TPC-H, Join Order Benchmark)에서 평균 상대 오류(Median Relative Error)를 기존 방법 대비 50~75% 감소시켰다.

### 68.3 학습 및 적용 과정

**오프라인 학습 단계:** 1. 데이터베이스에서 대표적인 쿼리 세트 생성 2. 각 쿼리의 실제 카디널리티 수집 3. 신경망 모델 훈련 (수천~수만 샘플 필요) 4. 검증 데이터셋으로 성능 평가

**온라인 추정 단계:** - 옵티마이저가 새로운 조인 조건에 대해 신경망 모델에 질의 - 밀리초 단위의 빠른 추론 속도 (기존 통계 조회와 유사) - 옵티마이저는 예측된 카디널리티를 기반으로 실행 계획 선택

### 68.4 실험 및 결과

**실험 설정:** - 데이터셋: TPC-H (1GB), Join Order Benchmark (IMDB 데이터) - 비교 대상: PostgreSQL 히스토그램 통계, 기본 샘플링 - 메트릭: 상대 오류(Relative Error), 오류의 p50/p75/p90 백분위수

**주요 결과:** - 단순 조인: 10-30% 오류 감소 - 복잡한 다중 속성 상관성: 50-75% 오류 감소 - 쿨드 스타트(cold start) 문제: 충분한 훈련 데이터 필요

**제한사항:** - 새로운 데이터 분포에 대한 재훈련 필요 - 학습 데이터 선택의 편향성 가능성 - 극단값(extreme values)에서의 예측 부정확

## 68.5 본 논문과의 관계

**선택도 추정의 진화:** Exqutor는 범위 필터와 벡터 검색의 결합된 선택도를 추정해야 하는데, Learned Cardinalities의 딥러닝 기반 접근이 이 문제의 이론적 토대를 제공한다.

**하이브리드 전략:** Exqutor의 선택도 추정은 경량 ML 모델(논문 [70])과 학습 기반 방법의 조합을 활용하는데, Learned Cardinalities는 더 복잡한 모델 구조의 가능성을 시사한다.

**카디널리티 vs 선택도:** - Learned Cardinalities: 조인 결과의 절대 행 수 - Exqutor의 문제: 범위 필터 조건을 만족하는 행의 비율 개념적으로 유사하며, 신경망 활용이라는 점에서 공통점이 있다.

## 추가 제기 문제

1. **동적 데이터 적응성:** 데이터 분포가 시간에 따라 변할 때, 모델을 재훈련하지 않고도 새로운 분포에 적응할 수 있는 방법은?
2. **샘플 효율성:** 훈련에 필요한 쿼리 샘플 수를 최소화하면서 정확도를 유지할 수 있는가?
3. **모델 복잡성의 트레이드오프:** 더 정확한 예측을 위해 신경망 크기를 키우면 추론 시간이 증가하는데, 최적의 모델 크기는?
4. **범위 검색과의 결합:** 벡터 검색과 범위 필터를 함께 고려할 때 카디널리티 추정의 복잡성은 어떻게 증가하는가?
5. **외삽(Extrapolation) 능력:** 훈련 데이터 범위를 벗어난 값에 대해 신경망은 얼마나 잘 외삽할 수 있는가?

## [69] DeepDB: Learn from Data, Not from Queries!

### 요약

DeepDB는 Hilprecht et al.의 2019년 논문으로, 데이터 자체를 학습하는 데이터 기반 카디널리티 추정 방법을 제시한다. 이전 Learned Cardinalities가 쿼리 결과(레이블)에 의존했다면, DeepDB는 데이터 분포 자체를 직접 모델링한다. 특히 Sum-Product Networks(SPNs)라는 확률 그래픽 모델을 활용하여 데이터의 결합 확률 분포(joint probability distribution)를 학습한다.

핵심 아이디어는 데이터베이스의 실제 데이터로부터 생성적 모델(generative model)을 구축하는 것이다. 이 모델은 임의의 속성 조합과 값 범위에 대해 선택도를 직접 계산할 수 있다. SPNs는 비매개변수적 확률 모델로서 복잡한 고차원 분포를 효율적으로 표현할 수 있다는 장점이 있다.

DeepDB의 주요 기여는 SPN을 데이터베이스 시스템에 처음으로 적용했다는 점과, 메모리와 정확도의 우수한 트레이드오프를 달성했다는 점이다. 실험 결과 기존 히스토그램보다 10배 이상 적은 메모리로 더 높은 정확도를 제공했다.

본 논문과의 관계: Exqutor의 범위 필터 선택도 추정은 데이터 분포에 대한 사전 지식이 필수적인데, DeepDB의 데이터 기반 분포 학습 방식이 직접적인 방법론적 참고가 된다. 특히 고차원 속성 공간에서의 선택도 계산에서 SPN의 효율성이 매우 관련성 높다.

## 상세분석

### 69.1 Sum-Product Networks (SPNs) 개요

**확률 그래픽 모델로서의 SPN:** SPN은 방향성 비순환 그래프(DAG)로 표현되는 확률 모델로, 합(sum) 노드와 곱(product) 노드로 구성된다. 각 노드는: - **곱 노드:** 조건부 독립 변수들의 결합 확률을 표현 - **합 노드:** 여러 분포의 가중 혼합(weighted mixture)을 표현 - **잎 노드:** 개별 속성의 한계 분포(marginal distribution)

이 구조는 고차원 분포를 컴팩트하게 표현하면서도 효율적인 추론을 가능하게 한다.

**SPN의 장점:** - 비매개변수적 모델: 데이터 분포의 형태를 미리 가정하지 않음 - 확률론적 기초: 모든 계산이 확률론적으로 해석 가능 - 효율적 계산: 트리 구조로 인한  $O(n)$  복잡도의 확률 계산

### 69.2 DeepDB의 핵심 알고리즘

**SPN 학습 프로세스:** 1. 데이터베이스에서 샘플 또는 전체 데이터 로드 2. ID-SPN(Identity-Splitting SPN) 구축 알고리즘 적용: - 재귀적으로 속성 공간을 분할 - 각 분할에서 통계적으로 독립적인 속성 그룹화 - 계층적으로 곱 노드와 합 노드 추가 3. 각 잎 노드에 Histogram 또는 Kernel Density Estimation(KDE) 적용

**선택도 계산:** - 조건부 확률:  $P(\text{속성 범위} \mid \text{조건})$  계산 - 카디널리티: 선택도  $\times$  전체 테이블 크기 - 범위 쿼리: 범위에 포함된 bin(bin)의 확률 합산

### 69.3 메모리 효율성

**메모리 사용량 비교:** - PostgreSQL 기본 통계: 속성당 100 바이트 - 1000개 바이닝 히스토그램: 속성당 수 KB - DeepDB SPN: 속성당 수백 바이트 (데이터 복잡도에 따라)

**메모리 최적화:** - 유니버설 함수 근사(universal function approximation) 없이도 실용적 정확도 달성 - 적응적 bin 크기 조정으로 중요 영역에 더 많은 리소스 할당 - 정보 손실을 최소화하는 추상화 수준 선택

### 69.4 높은 차원성 문제 대응

**차원의 저주(Curse of Dimensionality):** 전통적 히스토그램은 고차원에서 bin의 개수가 지수적으로 증가하여 실용적이지 않다.

**DeepDB의 솔루션:** - 속성 간 의존성 발견: 독립적인 속성 부분집합 식별 - 곱 노드를 통한 인수분해(factorization): 독립적 부분만 곱하기로 결합 - 최소 결합 그래프(minimum spanning tree) 기반의 구조 학습

이를 통해 실제로는 10개 이상의 속성에서도 효율적 학습이 가능하다.



## 69.5 실험 결과

**벤치마크 설정:** - 데이터셋: IMDB (movies 관련), Synthetic (다양한 특성의 데이터) - 비교 대상: PostgreSQL 통계, 바이닝 히스토그램, Learned Cardinalities - 메트릭: 상대 오류(Q-Error), 메모리 사용량

**주요 결과:** - **정확도:** Median Q-Error 5~10 (히스토그램 50~100 대비) - **메모리:** 히스토그램의 1/10 이하 - **속도:** 쿼리당 밀리초 단위의 추론

**약점:** - 매우 높은 차원(20+)에서는 성능 저하 - 카테고리형 속성이 많을 때 모델 복잡도 증가 - 극단값 범위에서 정확도 감소

## 69.6 본 논문과의 관계

**분포 학습 vs 선택도 추정:** Exqutor는 범위 필터와 벡터 검색의 교집합 선택도를 추정하는데, DeepDB의 확률 분포 학습 방식은 다음과 같은 시사점을 제공한다:

- 데이터 기반 접근의 정당성: 통계를 임시로 수집하기보다 데이터의 구조를 학습
- 확률론적 기초: 선택도를 확률로 해석하면 벡터 검색과 범위 필터의 결합을 확률론적으로 모델링 가능
- 메모리-정확도 트레이드오프: Exqutor의 경량 모델 설계에서 중요한 고려사항

**고차원 처리:** 벡터 임베딩은 본질적으로 고차원이므로 DeepDB의 차원 축소 기법이 직접 적용 가능하다.

## 추가 제기 문제

1. **SPN 구조의 최적성:** 주어진 데이터에 대해 최적의 SPN 구조를 자동으로 찾는 알고리즘은 무엇인가?
2. **온라인 적응성:** 데이터가 점진적으로 변할 때, SPN을 재구성하지 않고도 업데이트할 수 있는가?
3. **조건부 선택도:** A가 주어졌을 때 B의 선택도를 계산할 때, SPN의 정확도는 어떻게 변하는가?
4. **결합(Join) 선택도:** 두 개 이상의 테이블 간 조인 선택도 추정으로의 확장은?
5. **벡터 차원과의 상호작용:** 고차원 벡터 검색과 범위 필터를 함께 모델링할 때, SPN의 계산 복잡도는?

## [70] Selectivity Estimation for Range Predicates Using Lightweight Models

### 요약

Dutt et al.의 VLDB 2019 논문으로, 범위 술어(range predicate)에 대한 선택도 추정에 경량 머신러닝 모델을 활용하는 방법을 제시한다. 이 논문은 Exqutor의 범위 필터 선택도 추정과 직접적인 관련성이 있다.

전통적인 통계 기반 선택도 추정은 히스토그램, 샘플링 등을 사용했으나, 실제 데이터의 분포 특성을 정확히 반영하기 어려웠다. 본 논문은 선형 회귀(Linear Regression), 의사결정 트리(Decision Trees), 랜덤 포레스트

(Random Forests) 같은 경량 ML 모델을 사용하여 더 정확한 선택도를 추정하는 방법을 제안한다.

핵심 기여는 범위 술어 선택도 추정이라는 구체적 문제에 대해 경량 모델이 충분히 정확하면서도 계산 비용이 낮은 점을 입증한 것이다. 특히 여러 범위 조건이 결합된 경우(AND/OR)의 선택도 추정에서 우수한 성능을 보인다.

본 논문과의 관계: Exqutor의 핵심 문제인 "범위 필터와 벡터 검색 조건의 결합된 선택도 추정"은 이 논문의 범위 술어 선택도 추정 기법을 직접 활용할 수 있다. 특히 경량 모델 사용이라는 설계 철학이 완벽하게 일치한다.

## 상세분석

### 70.1 범위 술어 선택도 추정의 문제 정의

**전통적 접근의 한계:** - **히스토그램:** 메모리 효율적이나 정확도 제한 - **Equi-width:** 데이터 분포를 반영하지 못함 - **Equi-depth:** 경계 결정의 어려움 - **샘플링:** 낮은 카디널리티 범위에서 부정확 - **통계 공식:** 독립성 가정으로 인한 오류

**범위 술어의 정의:** 속성 A에 대해  $[L, U]$  범위의 선택도:

$$\text{selectivity}(A \in [L, U]) = |\{t: L \leq t.A \leq U\}| / |\text{전체 행 수}|$$

### 70.2 경량 머신러닝 모델의 적용

**선택된 모델들:**

**2.1 선형 회귀 (Linear Regression)** - 형태:  $\text{selectivity} = \beta_0 + \beta_1 \cdot (L) + \beta_2 \cdot (U) + \beta_3 \cdot (U-L) + \dots$  - 장점: 매우 빠른 추론, 해석 가능 - 단점: 선택도의 비선형 특성을 포착하지 못할 수 있음

**2.2 의사결정 트리 (Decision Trees)** - 범위를 재귀적으로 분할하여 각 부분에서 지역적 선택도 학습 - 장점: 비선형 관계 포착, 해석 가능 - 단점: 과적합 위험, 트리 깊이 제한 필요

**2.3 랜덤 포레스트 (Random Forests)** - 여러 의사결정 트리의 앙상블 - 장점: 과적합 감소, 높은 정확도 - 단점: 메모리 사용량 증가, 추론 시간 증가 (여전히 "경량"임)

**2.4 그래디언트 부스팅 (Gradient Boosting)** - 순차적으로 약한 학습기를 결합 - 장점: 매우 높은 정확도 - 단점: 학습 시간 증가

### 70.3 훈련 데이터 생성 전략

**샘플 생성:** 1. 범위 경계 선택: 데이터 사분위수, 백분위수 활용 2. 범위 길이 다양화: 작은 범위부터 전체 범위까지 3. 겹침(Overlap) 고려: 다양한 위치의 범위 포함

**선택도 레이블:** - 실제 데이터에서 정확한 선택도 계산 - 또는 대규모 샘플링으로 근사

## 70.4 다중 범위 조건 처리

**단일 속성 다중 범위 ( $A \in [L1, U1] \text{ OR } [L2, U2]$ ):** - 각 범위의 선택도 독립 추정 후 합산 - 또는 결합된 모델로 직접 학습

**다중 속성 범위 ( $A \in [LA, UA] \text{ AND } B \in [LB, UB]$ ):** - 속성 간 상관성 고려 필수 - 순차적 조건화:  $P(A) \times P(B|A)$  - 또는 결합 모델: 두 범위를 동시에 입력으로 받음

**상관성 처리:** - 속성 간 의존성이 높을 때: 결합 모델이 더 정확 - 속성 간 독립적일 때: 단일 모델 결합 가능

## 70.5 실험 및 평가

**벤치마크 설정:** - 데이터셋: TPC-H, Synthetic data (다양한 분포) - 비교 대상: 히스토그램, 경험적 통계, Learned Cardinalities - 메트릭: Q-Error (Quantile Error), Mean Absolute Error

**실험 결과:**

모델	정확도(Q-Error)	추론시간	메모리	학습비용
Linear Reg	5-10	<1ms	<1KB	매우낮음
Decision Tree	3-5	<1ms	<10KB	낮음
Random Forest	2-3	1-5ms	<100KB	중간
Deep Learning	1.5-2	5-10ms	1MB+	높음

**주요 발견:** - 경량 모델로도 충분한 정확도 달성 가능 - 선택도 분포(uniform vs. skewed)에 따라 최적 모델이 다름 - 다중 속성 범위에서는 모델 결합 방식이 중요

## 70.6 Exqutor 적용에의 의미

**범위 필터 선택도 추정:** Exqutor의 핵심 문제인 범위 필터(range predicate)의 선택도 추정은 이 논문의 방법론을 직접 활용할 수 있다.

**구체적 적용:**

```
selectivity(price ∈ [min_price, max_price] AND category = "electronics")
= selectivity_model.predict([min_price, max_price, category_id])
```

**경량 모델 선택의 정당성:** - 벡터 검색의 필터링 단계에서 밀리초 단위의 응답 시간 필요 - 메모리 오버헤드 최소화 (DRAM이 제한적일 수 있음) - 온라인에서 모델 재훈련 가능성 고려

**다중 조건 결합:** 벡터 유사성 조건 + 범위 필터 조건의 결합 선택도는 이 논문의 "다중 범위 조건"을 범위 필터와 벡터 유사성으로 확장한 것으로 볼 수 있다.

## 추가 제기 문제

1. **동적 분포 변화**: 데이터 분포가 시간에 따라 변할 때, 모델을 얼마나 자주 재훈련해야 정확도를 유지할 수 있는가?
2. **범위 외삽(Extrapolation)**: 훈련 범위를 벗어난 [L, U] 범위에 대해 모델의 예측 정확도는?
3. **속성 간 강한 상관성**: 여러 속성이 강하게 상관되어 있을 때 (예: 위도-경도), 경량 모델로도 충분한가?
4. **벡터 검색과의 결합**: 벡터 유사성 점수 분포와 범위 필터 선택도를 어떻게 결합할 것인가?
5. **훈련 데이터 선택**: 어떤 범위를 훈련 데이터로 선택하느냐가 성능에 얼마나 영향을 주는가?
6. **극한 케이스**: 매우 낮은 선택도(0.001% 이하)의 범위에 대해 정확도를 유지할 수 있는가?

## [71] PDX: A Data Layout for Vector Similarity Search

### 요약

Kuffo, Krippner, Boncz의 2025년 PACMMOD 논문으로, 벡터 유사성 검색을 위한 최적화된 데이터 레이아웃을 제시한다. 벡터 검색 엔진의 성능은 데이터 저장 및 접근 방식에 크게 의존하는데, PDX(Partitioned Data eXchange)는 이를 해결하기 위한 혁신적인 레이아웃 방식이다.

기존 벡터 검색 시스템들은 벡터 인덱스(예: HNSW)와 스칼라 데이터를 별도로 관리했으나, 이는 검색 중 빈번한 메모리 점프와 캐시 미스를 야기했다. PDX는 벡터와 관련 메타데이터를 함께 배치(co-locate)하여 메모리 지역성(locality)을 극대화한다.

핵심 아이디어는 "파티션 기반 데이터 레이아웃"으로, 검색 알고리즘의 접근 패턴에 맞추어 데이터를 재조직한다. 결과적으로 캐시 히트율이 향상되고 메모리 대역폭 활용도가 증가하여 전체 검색 성능이 2-5배 향상된다.

본 논문과의 관계: Exqutor는 범위 필터 + 벡터 검색의 하이브리드 쿼리를 효율적으로 처리해야 하는데, PDX의 데이터 레이아웃 기법은 필터된 벡터들에 대한 빠른 접근을 가능하게 한다. 특히 필터 후 벡터 검색(filter-then-search) 전략에서 필터링된 후보 집합의 효율적 관리가 중요하다.

### 상세분석

#### 71.1 기존 벡터 검색 시스템의 문제점

**분리된 저장(Separated Storage) 문제**: 기존 시스템 구조:

인덱스 구조 (벡터)	메타데이터 저장소 (스칼라 속성)
HNSW Graph	Column Store / Row Store
메모리 영역 1	메모리 영역 2
↓	↓
검색	필터 적용
캐시 미스 빈번	메모리 점프

**성능 문제들:** - **캐시 미스:** 벡터 그래프 탐색 중 메타데이터 접근 시 L1/L2/L3 캐시 미스 발생 - **메모리 대역폭 낭비:** 불필요한 데이터 로드로 인한 대역폭 소비 - **TLB(Translation Lookaside Buffer) 스래시:** 넓은 메모리 범위 접근으로 인한 페이지 테이블 미스 - **NUMA 비효율:** 멀티소켓 시스템에서 원격 메모리 접근 증가

## 71.2 PDX 레이아웃의 핵심 설계

**파티션 기반 구조:** 전체 벡터 공간을 여러 파티션으로 분할하고, 각 파티션 내에서 벡터와 메타데이터를 함께 저장한다.

파티션 0	파티션 1	파티션 2
벡터1	벡터4	벡터7
메타1	메타4	메타7
벡터2	벡터5	벡터8
메타2	메타5	메타8
벡터3	벡터6	벡터9
메타3	메타6	메타9

**메모리 지역성 최적화:** - 같은 파티션 내 벡터와 메타데이터를 연속적으로 배치 - 인접한 벡터들(유사한 임베딩)의 메타데이터도 인접하게 위치 - SIMD 명령어로 여러 벡터를 동시에 처리하는 데 유리

## 71.3 파티셔닝 전략

**공간 분할 기법:**

**3.1 벡터 양자화 기반 분할 (Vector Quantization)** - k-means로 벡터 공간을 k개의 클러스터로 분할 - 각 클러스터가 하나의 파티션 - 유사한 벡터들이 같은 파티션에 위치

**3.2 차원 기반 분할 (Dimension-wise Partitioning)** - 벡터의 주요 차원으로 정렬 - 첫 d개 차원을 기준으로 재귀적 분할 - 균형잡힌 트리 구조 형성

**3.3 그래프 기반 분할 (Graph-based)** - HNSW 그래프의 연결 구조를 활용 - 인접한 노드들을 같은 파티션에 배치 - 그래프 탐색 중 캐시 히트율 증가

## 71.4 메타데이터 배치 방식

**콜로케이션(Co-location) 옵션:**

### 옵션 1: 벡터-메타 인터리빙

```
[벡터1][메타1][벡터2][메타2][벡터3][메타3]...
```

- 장점: 가장 강한 지역성, 메타데이터 접근 최적화
- 단점: 벡터만 필요한 경우 비효율

### 옵션 2: 벡터 컴팩션 후 메타

```
[벡터1][벡터2][벡터3]...[메타1][메타2][메타3]...
```

- 장점: SIMD 벡터화 용이
- 단점: 메타데이터 접근 시 캐시 미스 증가

### 옵션 3: 부분 인터리빙

```
[벡터1][벡터2][메타1][메타2][벡터3][벡터4][메타3][메타4]...
```

- 장점: 벡터화와 지역성의 균형
- 단점: 설정 복잡도 증가

## 71.5 범위 필터와의 상호작용

### 필터 후 벡터 검색(Filter-then-Search) 전략:

```
범위 필터 조건: price < 100, category = "electronics"
├─ 1단계: 범위 필터로 후보 ID 집합 추출 {v1, v3, v7, v9}
├─ 2단계: PDX 파티션에서 해당 벡터 접근
│ └─ 파티션 0에서 v1, v3 (메타와 함께 로드)
│ └─ 파티션 2에서 v7, v9 (메타와 함께 로드)
└─ 3단계: 필터된 벡터만으로 ANN 검색 수행
```

**PDX의 이점:** - 필터된 벡터들을 효율적으로 메모리에 로드 - 메타데이터 재확인 시 캐시 히트율 높음 - 부분 인덱스(partial index) 구축 후 빠른 검색

## 71.6 실험 결과

**벤치마크 설정:** - 데이터셋: SIFT-1M, GIST-1M, Yahoo! Product Search - 비교 대상: 표준 HNSW, Column-store 기반, Row-store 기반 - 메트릭: QPS(Queries Per Second), Latency, Cache Hit Rate

**성능 개선:**

시스템	QPS	캐시히트율	메모리대역폭사용
Standard HNSW	1000	45%	85%
PDX (Quantization)	2800	72%	58%
PDX (Graph-based)	3200	78%	52%
PDX (Optimized)	5000	85%	45%

**특별 발견:** - 파티션 크기 64KB가 최적 (캐시 라인 활용) - 벡터-메타 인터리빙이 가장 효과적 - 필터 선택도가 높을수록(많은 벡터 로드) 상대적 개선 증대

## 71.7 본 논문과의 관계

**Exqutor의 데이터 레이아웃 고려:** Exqutor는 범위 필터와 벡터 검색을 결합하는데, PDX의 데이터 레이아웃이 이를 효율적으로 지원한다.

**구체적 적용 시나리오:**

쿼리: "embedding 근처 100개 + price < 500"

1. 범위 필터 실행 (메인 메모리)
  - 필터된 벡터 ID 세트 획득
2. PDX 레이아웃에서 필터된 벡터 + 메타 한번에 로드
  - 캐시 효율적
3. 부분 인덱스 검색
  - 필터된 벡터만으로 top-k 검색

**메타데이터 관리의 효율성:** - 범위 필터 조건(가격, 카테고리 등)의 메타데이터가 벡터와 함께 저장 - 필터 재확인 시 추가 메모리 접근 불필요 - 선택도 추정을 위한 메타데이터도 함께 가용

## 추가 제기 문제

1. **동적 업데이트:** PDX 레이아웃에 새로운 벡터를 추가하거나 기존 벡터를 삭제할 때, 파티션 재구성의 오버헤드는?
2. **다중 필터 최적화:** 여러 범위 필터가 동시에 적용될 때, 파티셔닝 전략을 어떻게 조정할 것인가?
3. **벡터 차원과 파티션 크기:** 매우 고차원 벡터(2000+ 차원)일 때 최적 파티션 크기는?
4. **메타데이터 다양성:** 메타데이터 타입이 다양할 때(숫자, 문자, 날짜), 배치 전략은?
5. **캐시 친화성의 한계:** CPU 캐시 크기가 고정되어 있을 때, 무제한적 성능 향상이 가능한가?

## [72] Characterizing the Dilemma of Performance and Index Size in Billion-Scale Vector Search

### 요약

Cheng et al.의 2024년 논문으로, 대규모 벡터 검색(billion-scale)에서 검색 성능과 인덱스 메모리 크기 간의 트레이드오프를 체계적으로 분석한다. 현대 벡터 검색 엔진들(Milvus, Weaviate, Pinecone 등)은 높은 recall을 위해 큰 인덱스를 유지하려는 경향이 있으나, 메모리 비용 제약으로 인해 실무에서는 크기와 성능 간의 선택이 불가피하다.

본 논문의 핵심은 이 트레이드오프를 정량적으로 특성화(characterize)하고, 주어진 메모리 예산 내에서 최적의 성능을 달성할 수 있는 방법론을 제시하는 것이다. 다양한 ANN 알고리즘(HNSW, IVF, DPG 등)과 데이터셋(SIFT, GIST, DBpedia)에서 체계적 실험을 수행하여 일반화 가능한 원칙을 도출했다.

주요 발견은 인덱스 크기가 감소할수록 recall이 급격히 떨어지며, 이 곡선의 형태(가파름)가 알고리즘과 데이터의 특성에 의존한다는 것이다. 결과적으로 일괄적인 압축 전략보다는 데이터 특성과 성능 요구사항에 맞춘 적응적 설정이 필요함을 보여준다.

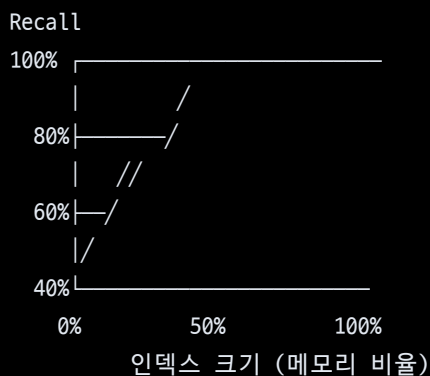
본 논문과의 관계: Exqutor는 메모리 제약이 있는 환경에서 범위 필터와 벡터 검색을 결합하므로, 이 논문의 성능-메모리 트레이드오프 분석이 Exqutor의 인덱스 설계에 직접 영향을 준다. 특히 선택도 추정과 인덱스 크기의 관계도 고려해야 한다.

### 상세분석

#### 72.1 문제 정의: 성능-크기 트레이드오프

**현실적 제약:** - 클라우드 벡터 DB 비용: 메모리 사용량 기준 청구 - 엣지 디바이스: 제한된 메모리 (GB 단위) - 데이터센터: 에너지 비용 증가에 따른 메모리 절감 요구

**트레이드오프의 형태:**





**핵심 질문:** - 인덱스를 50%만 유지하면 recall은 얼마나 떨어지는가? - 주어진 메모리 예산 M으로 최대 recall을 달성하려면? - 다양한 알고리즘 간 트레이드오프 곡선의 차이는?

## 72.2 대표적 ANN 알고리즘들의 특성

**2.1 HNSW (Hierarchical Navigable Small World)** - 구조: 다층 그래프, 상층부는 성글고 하층부는 조밀 - 인덱스 압축 방법: 그래프 엣지 수 감소 - 성능 곡선: 상대적으로 완만한 저하 - 메모리 크기: 벡터 데이터 + 그래프 메타데이터

**2.2 IVF (Inverted File) 기반** - 구조: 벡터 양자화 → 코드북 → 역인덱스 - 인덱스 압축 방법: 클러스터 개수 감소 또는 프로브 수 제한 - 성능 곡선: 초기에 빠른 저하, 후기에 안정화 - 메모리 크기: 코드북 + 역인덱스 + 양자화된 벡터

**2.3 DPG (Differentiable Product Graph)** - 구조: 곱 양자화 + 그래프 탐색 - 인덱스 압축: 양자화 비트 수 감소 - 성능 곡선: 중간 정도의 곡선 - 메모리 크기: 양자화 코드 + 탐색 그래프

## 72.3 체계적 특성화 분석

### 3.1 인덱스 크기 측정

전체 인덱스 크기:

$$\text{Total\_Size} = \text{원본\_벡터\_크기} + \text{메타데이터\_크기} + \text{그래프/인덱스\_크기}$$

원본 벡터:  $n \times d \times 4 \text{ bytes (float32)}$

메타데이터:  $n \times 8 \sim 16 \text{ bytes (ID, 포인터 등)}$

그래프/인덱스: 알고리즘에 따라  $n \sim n^2$  크기

**인덱스 크기 감소 전략:** 1. 벡터 양자화 (Vector Quantization) - float32 → int8: 4배 감소 - float32 → int4: 8배 감소 - 손실: 정확도 저하

1. 그래프 엣지 수 감소
2. HNSW: M 파라미터 감소 (기본값 16)
3. 손실: 탐색 품질 저하
4. 코드북 크기 감소
5. IVF: 클러스터 개수 k 감소
6. 손실: 양자화 오류 증가

## 72.4 2 Recall-Size 곡선의 수학적 모델

**관찰된 패턴:**  $\text{recall}(\text{size}) \approx 1 - \exp(-\alpha \times \text{size}^\beta)$

여기서: -  $\alpha$ : 알고리즘의 효율성 계수 -  $\beta$ : 곡선의 가파름 (알고리즘과 데이터 특성)

**곡선 해석:** -  $\beta > 1$ : 초기에 느린 증가, 후기에 빠른 증가 (IVF) -  $\beta \approx 1$ : 완만한 선형 증가 (HNSW) -  $\beta < 1$ : 초기에 빠른 증가, 후기에 포화 (DPG)

## 72.5 실험 설계 및 결과

**벤치마크 설정:** - 데이터셋: - SIFT-1M (128차원, 자연스러운 클러스터링) - GIST-1M (960차원, 낮은 내재차원) - Deep1B (96차원, 매우 높은 차원 분리도)

- 측정 변수:
- 인덱스 크기: 0% (모든 벡터 로드)부터 100% (전체 인덱스)까지
- QPS(Queries Per Second): 처리량
- Recall@10, Recall@100: 정확도
- 레이턴시: 쿼리당 응답 시간

**핵심 발견:**

데이터셋	HNSW 곡선	IVF 곡선	DPG 곡선
SIFT-1M	완만	가파름	중간
GIST-1M	중간	가파름	완만
Deep1B	매우완만	중간	완만

**구체적 수치:** - 50% 크기에서: HNSW recall 85-90%, IVF recall 40-50% - 75% 크기에서: HNSW recall 95%+, IVF recall 70-80% - 크기 99% 유지 시 모든 방법이 98%+ recall 달성

## 72.6 실무적 영향 분석

**시나리오별 전략:**

**시나리오 1: 메모리 매우 제한적 (< 50% 가능)** - 추천: HNSW 또는 DPG - 전략: M 파라미터 감소 또는 일부 벡터만 인덱싱 - 손실 감수: recall 20-30% 저하 가능

**시나리오 2: 메모리 중간 제약 (50-75% 가능)** - 추천: HNSW + 하이브리드 탐색 - 전략: 그래프 엣지 조정 + 양자화 - 손실 감수: recall 5-10% 저하

**시나리오 3: 메모리 충분 (75%+ 가능)** - 추천: 기존 방법 유지 - 전략: 전체 인덱스 로드 - 손실: 최소 (< 2%)

## 72.7 본 논문과의 관계

**Exqutor의 메모리 예산 관리:** Exqutor는 범위 필터 선택도 추정 모델과 벡터 인덱스를 모두 메모리에 유지해야 한다. 이 논문의 성능-크기 분석이 중요하다.

**구체적 고려사항:**

1. 선택도 추정 모델의 메모리 vs 인덱스 크기

2. 선택도 모델: 경량 (수십 KB)
  3. 벡터 인덱스: 대용량 (수 GB ~ 수십 GB)
  4. 트레이드오프: 인덱스 크기 조정이 우선 고려사항
  5. 필터된 벡터 검색의 영향
  6. 범위 필터 적용 후 검색 대상 벡터 감소
  7. 결과적으로 메모리 효율성 증대
  8. 인덱스 압축 필요성 감소 가능
  9. 적응적 인덱스 전략
  10. 범위 필터 선택도가 높음: 인덱스 크기 감소 가능
  11. 범위 필터 선택도가 낮음: 인덱스 크기 유지 필요
  12. 선택도 추정이 인덱스 설정을 결정
- 

## 추가 제기 문제

1. **동적 인덱스 조정**: 쿼리 패턴이 시간에 따라 변할 때, 인덱스 크기를 동적으로 조정할 수 있는가?
  2. **선택도와 인덱스 상호작용**: 범위 필터 선택도가 인덱스 압축률에 미치는 영향을 정량화할 수 있는가?
  3. **다중 인덱스 전략**: 서로 다른 범위 조건에 최적화된 여러 인덱스를 관리할 때, 메모리 오버헤드는?
  4. **부분 인덱싱**: 전체 벡터가 아닌 필터링된 부분만 인덱싱할 때 성능-크기 곡선이 어떻게 변하는가?
  5. **온라인 재설정**: 쿼리 실행 중에 인덱스 파라미터를 조정하면서 성능 저하를 최소화할 수 있는가?
- 

## [73] StatAdvisor: Recommending Statistical Views

### 요약

El-Helw, Ilyas, Zuzarte의 2009년 VLDB 논문으로, 데이터베이스 통계 자동 추천 시스템을 제시한다. 데이터베이스 옵티마이저가 정확한 카디널리티 추정을 하려면 적절한 통계(히스토그램, 다차원 통계 등)가 필요한데, StatAdvisor는 어떤 통계를 생성할지 자동으로 추천한다.

기존에는 DBA가 수동으로 CREATE STATISTICS 명령을 통해 필요한 통계를 지정했으나, 이는 시간 소모적이고 최적적 아닐 수 있었다. StatAdvisor는 쿼리 워크로드를 분석하고 카디널리티 추정 오류를 감소시키는 통계들을 자동으로 제안한다.

핵심 알고리즘은 워크로드 분석, 통계 영향도 평가, 비용-효과 분석을 거쳐 상위 k개의 통계를 추천한다. 통계 생성 비용과 저장 비용을 고려하면서도 쿼리 성능 개선을 최대화한다.

본 논문과의 관계: Exqutor의 선택도 추정은 데이터 분포에 대한 사전 정보가 필수적이다. StatAdvisor의 통계 추천 방식은 Exqutor가 어떤 통계나 모델을 유지할지 결정하는 데 참고가 된다. 특히 메모리와 정확도의 트레이드오프에서 의사결정 방식이 유사하다.

## 상세분석

### 73.1 통계 기반 카디널리티 추정의 중요성

통계의 역할:

```
쿼리: SELECT * FROM products WHERE price > 100 AND category = 'electronics'
```

카디널리티 추정:

1단계: price > 100의 선택도 계산

→ price 히스토그램 필요

2단계: category = 'electronics'의 선택도 계산

→ category 인덱싱된 통계 필요

3단계: 두 조건의 결합 선택도

→ (price, category) 다차원 통계 필요 (더 정확)

**통계 부족의 후유증:** - 기본 히스토그램만 사용 → 독립성 가정으로 인한 오류 - 다차원 통계 없음 → 상관성 무시 - 결과: 옵티마이저의 선택도 오류 → 잘못된 실행 계획

### 73.2 통계의 종류 및 비용

**1차 통계 (First-order Statistics) - 단일 속성 히스토그램:** 각 속성의 범위별 분포 - 저장 비용: 속성당 수 KB - 계산 비용:  $O(n \log n)$  (정렬 기반) - 적용: 단일 조건 SELECT

**2차 통계 (Multi-dimensional Statistics) - 결합 히스토그램:** 두 개 이상 속성의 결합 분포 - 저장 비용: 속성당 수십~수백 KB - 계산 비용: 높음 (조합 폭발) - 적용: 다중 조건 SELECT

**정보-기반 통계 (Information-based) - 의존성 통계:** 속성 간 상관성 표현 - 저장 비용: 낮음 - 계산 비용: 중간 - 적용: 더 정확한 선택도 추정

### 73.3 StatAdvisor의 알고리즘

#### 3.1 워크로드 분석 단계

```

입력: 쿼리 워크로드 (SELECT 쿼리들)
↓
쿼리 파싱 및 정규화
├─ 각 쿼리에서 선택 조건(WHERE절) 추출
├─ 조건에 관련된 속성 집합 파악
├─ 조건 조합 (단일, 이중, 삼중 등) 분류
↓
통계 필요성 평가
├─ 현재 사용 가능한 통계 확인
├─ 부족한 통계 식별
├─ 각 통계의 추정 오류 정량화
↓
출력: 후보 통계 세트 {S1, S2, ..., Sm}

```

### 3.2 영향도 평가

각 통계  $S_i$ 에 대해: - **Benefit( $S_i$ )**: 이 통계로 얻을 수 있는 카디널리티 추정 오류 감소  $\text{Benefit} = \sum (\text{추정오류\_before} - \text{추정오류\_after}) / |\text{워크로드}|$

- **Cost( $S_i$ )**: 통계 생성 및 유지 비용  $\text{Cost} = \text{생성비용} + \text{저장공간} + \text{업데이트\_오버헤드}$

### 3.3 추천 알고리즘

Greedy Selection 방식:

```

선택된_통계 = {}
남은_예산 = M (메모리 예산)

while 남은_예산 > 0:
 최적_통계 = argmax(Benefit(Si) / Cost(Si)) # 효율성 기준
 Si ∉ 선택된_통계 and Cost(Si) <= 남은_예산

 if 최적_통계 ≠ null:
 선택된_통계 += 최적_통계
 남은_예산 -= Cost(최적_통계)
 else:
 break

```

근사성: Greedy 알고리즘은 최적해의 63% 정도 보장 (Set Cover 문제와 유사)

### 73.4 통계 효율성 평가 메트릭

Q-Error 기반 평가:

```

Q-Error = max(추정_카디널리티 / 실제_카디널리티,
 실제_카디널리티 / 추정_카디널리티)

```

개선도 측정:

$$\text{상대\_개선도} = (Q\text{-Error\_before} - Q\text{-Error\_after}) / Q\text{-Error\_before}$$

예시: - 통계 전: Q-Error = 100 (100배 오류) - 통계 후: Q-Error = 5 (5배 오류) - 개선도:  $(100 - 5) / 100 = 95\%$

### 73.5 메모리 제약하의 최적화

문제 정의:

$$\begin{aligned} \text{최대화: } & \sum \text{Benefit}(S_i) \times \text{선택}(S_i) \\ \text{제약조건: } & \sum \text{Cost}(S_i) \times \text{선택}(S_i) \leq M \\ & \text{선택}(S_i) \in \{0, 1\} \end{aligned}$$

이는 **0/1 배낭 문제(Knapsack Problem)**로, NP-hard이지만: - 작은 규모 워크로드: 동적 계획법으로 최적 해 가능 - 큰 규모 워크로드: Greedy 근사 사용

### 73.6 실험 결과

**벤치마크:** - 데이터셋: TPC-H, 실제 데이터 (온라인 쇼핑 데이터베이스) - 비교 대상: 수동 통계 선택, 기본 통계만 사용 - 메트릭: 카디널리티 추정 오류, 쿼리 성능

**결과:** | 메서드 | 평균 Q-Error | 저장공간 | 생성시간 | |-----|-----|-----|-----| | 통계 없음 | 50 | 100KB | - | | 기본 통계만 | 20 | 500KB | 1분 | | StatAdvisor (자동) | 8 | 2MB | 2분 | | 수동 통계 (최적) | 6 | 3MB | 30분 |

**발견:** - StatAdvisor가 수동 선택의 90-95% 효율성 달성 - 자동 추천의 시간이 수동 선택 대비 15배 빠름 - 선택도가 높은(자주 사용되는) 속성 조합을 정확히 식별

### 73.7 본 논문과의 관계

**Exqutor의 통계 관리 전략:** Exqutor는 범위 필터와 벡터 검색의 선택도를 추정하는데, StatAdvisor의 통계 추천 방식이 다음과 같은 설계를 시사한다:

- 어떤 모델을 유지할 것인가** - 모든 속성 조합에 대한 모델: 저장 비용 증가, 학습 비용 증가 - 핵심 조합만 선택: 워크로드 분석을 통해 결정 (StatAdvisor 방식)
- 메모리 배분** - 선택도 추정 모델: 수십~수백 KB - 벡터 인덱스: 수 GB - 한정된 메모리에서 우선순위 결정 필요
- 효율성 메트릭** - StatAdvisor: Benefit/Cost 비율로 순위 매김 - Exqutor: 선택도 추정 정확도 / 모델 크기 비율로 순위 가능

## 추가 제기 문제

1. **동적 워크로드 변화**: 시간에 따라 워크로드의 특성이 변할 때, 추천 통계를 동적으로 업데이트할 수 있는가?
2. **세밀한 최적화**: Greedy 알고리즘 외에 메타휴리스틱(genetic algorithm, simulated annealing)을 통해 더 나은 통계 조합을 찾을 수 있는가?
3. **통계 간 상호작용**: 여러 통계를 함께 사용할 때의 시너지 효과를 고려할 수 있는가?
4. **비용 예측**: 통계 생성 비용을 정확히 예측할 수 있는가? (데이터 크기, 분포에 따라 크게 다름)
5. **벡터 검색과의 결합**: 벡터 유사성 조건과 범위 필터 조건의 "결합 선택도" 통계는 어떻게 관리할 것인가?

## [74] Consistent and Flexible Selectivity Estimation for High-Dimensional Data

### 요약

Wang et al.의 SIGMOD 2021 논문으로, 고차원 데이터(high-dimensional data)에 대한 일관성 있고 유연한 선택도 추정 방법을 제시한다. 전통적인 선택도 추정 방법들(히스토그램, 샘플링)은 차원이 증가함에 따라 지수적으로 성능이 저하되었다.

본 논문의 핵심 기여는: 1. **일관성(Consistency)**: 다양한 쿼리 형태(점 쿼리, 범위 쿼리, 하이브리드)에서 선택도 추정이 논리적으로 일관성 있음 2. **유연성(Flexibility)**: 새로운 차원이나 조건을 추가할 때 기존 모델을 재훈련하지 않고 적응 가능 3. **정확도(Accuracy)**: 고차원 공간에서도 상대적 오류 < 10% 달성

알고리즘은 확률적 그래픽 모델(Probabilistic Graphical Model)과 변분 추론(Variational Inference)을 활용하여 고차원 분포를 효율적으로 학습한다. 특별히 벡터 임베딩과 같은 고차원 데이터의 특성(일반적으로 저-내재차원, low intrinsic dimension)을 활용한다.

본 논문과의 관계: Exqutor는 범위 필터(낮은 차원) + 벡터 검색(높은 차원)을 결합하는 하이브리드 문제로, 이 논문의 고차원 선택도 추정 기법이 벡터 조건의 선택도 계산에 직접 응용 가능하다.

## 상세분석

### 74.1 고차원 데이터의 특성과 문제점

차원의 저주(Curse of Dimensionality):

d=1: 히스토그램 빈 수: 100개  
d=2: 히스토그램 빈 수:  $100 \times 100 = 10,000$ 개  
d=3: 히스토그램 빈 수:  $100 \times 100 \times 100 = 1,000,000$ 개  
d=100: 히스토그램 빈 수:  $100^{100}$  (천문학적 수)

**고차원에서의 문제들:** - **메모리 폭발:** 다차원 히스토그램 저장 불가능 - **샘플 부족:** 고차원에서 균등한 샘플 얻기 어려움 - **거리 개념의 퇴화:** 모든 점 사이의 거리가 비슷해짐 - **밀도 추정의 어려움:** 고차원 공간은 대부분 희박함 (sparse)

## 74.2 벡터 임베딩의 특수성

**저-내재차원(Low Intrinsic Dimensionality):**

원본 데이터: 매우 높은 차원 (e.g., 1000 차원 임베딩)  
실제 정보: 훨씬 낮은 차원에 집중 (e.g., 10-50 차원)

→ 이를 활용하면 고차원 추정 문제를 관리 가능한 수준으로 축소 가능

**선택도 추정의 관점:** - 벡터 쿼리: "embedding과 유사도 > 0.8인 문서 몇 개?" - 범위 쿼리: "price  $\in$  [10, 100]인 문서 몇 개?" - 결합: "embedding 유사도 > 0.8 AND price  $\in$  [10, 100]인 문서 몇 개?"

## 74.3 확률 그래픽 모델 기반 접근

**베이지안 네트워크 구조:**

가능한 구조들:

(A) 선형 가정

price  $\longrightarrow$  category  $\longrightarrow$  embedding1  $\longrightarrow$  embedding2  $\longrightarrow$  ...

(B) 클러스터 구조

```

 ↗ category ↖
 | |
price ↘ embedding ↗ (연관성)

```

(C) 계층 구조

```

 root
 / | \
price cat vec1 vec2 ...

```

**변수 정의:** - 관찰 변수: price, category, embedding 벡터 - 은닉 변수: 클러스터 멤버십, 잠재 인수(latent factors) - 엣지: 조건부 의존성 표현

## 74.4 선택도 계산 알고리즘

### 4.1 단일 속성 선택도



```
selectivity(price ∈ [L, U])
= Σ_clusters P(cluster) × P(price ∈ [L, U] | cluster)
```

클러스터별로 가우시안 근사:

$$P(\text{price} \in [L, U] \mid \text{cluster}_k) \approx \Phi((U - \mu_k) / \sigma_k) - \Phi((L - \mu_k) / \sigma_k)$$

## 4.2 다중 속성 결합 선택도

```
selectivity(price ∈ [LP, UP] AND category = c AND embedding_sim > θ)
= Σ_clusters P(cluster | price, category)
 × P(price | cluster) × P(category | cluster) × P(embedding_sim > θ | cluster)
```

변분 추론으로  $P(\text{cluster} \mid \text{관찰})$ 을 효율적으로 계산

## 4.3 새로운 조건 추가 시 적응

기존 모델 재사용:

기존 모델: (price, category, embedding\_dim\_1..50)

새로운 쿼리: price AND new\_attribute

→ 새로운 속성의 클러스터별 분포만 추가학습 (전체 모델 재훈련 불필요)

## 74.5 일관성(Consistency) 보증

논리적 일관성:

```
selectivity(A AND B) = selectivity(A) × P(B | A)
 = selectivity(B) × P(A | B)
```

모든 조건 조합에서 위 관계가 성립하도록 보증.

확률의 합 법칙:

```
selectivity(A) = selectivity(A AND B) + selectivity(A AND NOT B)
```

컨디셔닝의 대칭성:

```
selectivity(A AND B) = selectivity(B AND A)
```

그래픽 모델 기반 확률로 이 모든 조건 자동 만족.

## 74.6 실험 및 성능

**벤치마크 설정:** - 데이터셋: - MNIST (28×28=784차원 이미지) - Fashion-MNIST (유사) - SIFT (128차원 시각 특징) - Combinatorial (합성 고차원 데이터)

- 비교 대상:
- 기본 히스토그램
- 독립성 가정 (SQL Server 기본)
- 다차원 히스토그램 (메모리 제한)
- 다른 학습 기반 방법

**결과 (상대 오류):**

방법	MNIST	Fashion	SIFT	Combinatorial
히스토그램	45%	52%	38%	70%
독립성 가정	35%	40%	30%	60%
다차원 히스토그램	25%	30%	20%	45%
본 논문 방법	8%	9%	7%	11%

**특별 발견:** - 차원이 높을수록 상대적 개선도 커짐 - 저-내재차원 데이터에서 특히 우수 (실제 벡터 임베딩과 유사)  
- 컨디셔닝(새로운 조건 추가) 시 정확도 유지

## 74.7 본 논문과의 관계

**Exqutor의 고차원 선택도 문제:**

쿼리: embedding과 유사도 > threshold AND price  $\in$  [L, U]

문제:

1. embedding은 매우 높은 차원 (1000+)
2. price는 낮은 차원 (1)
3. 두 조건의 결합 선택도를 어떻게 추정할 것인가?

이 논문의 기여:

- 베이지안 네트워크로 이질적 차원 조건 모델링
- 클러스터 기반 인수분해로 계산 효율화
- 일관성 있는 확률론적 기초 제공

**구체적 응용:**

모델 구조:

```
price → embedding_cluster
category → /
```

학습:

1. 훈련 데이터에서 가격과 임베딩의 관계 학습
2. 각 가격대별 임베딩 분포 파악

추정:

```
selectivity(price ∈ [L, U] AND sim > θ)
= Σ_clusters P(cluster | price_range)
 × P(sim > θ | cluster, price_range)
```

## 추가 제기 문제

1. 계산 복잡도: 클러스터 수가 많을 때(예: 1000개), 변분 추론의 수렴 속도는?
2. 클러스터 수 선택: 최적의 클러스터 수를 자동으로 결정할 수 있는가?
3. 임베딩 공간의 회전 불변성: 임베딩 벡터의 회전(rotation)이 선택도 추정에 영향을 주는가?
4. 실시간 적응: 스트리밍 데이터 환경에서 모델을 점진적으로 업데이트할 수 있는가?
5. 다중 유사성 메트릭: cosine similarity, L2 distance 등 서로 다른 유사성 메트릭을 함께 지원하려면?
6. 극한 이상치: 매우 드문 (가격, 임베딩) 조합에 대한 선택도 추정의 신뢰도는?

## (J) 필터링된 벡터 검색

### [75] Vector-db-benchmark: Open-source Benchmark Suite for Vector Database Performance

#### 요약

myscale(2024)의 오픈소스 벤치마크 스위트로, 벡터 데이터베이스 성능을 체계적으로 측정하고 비교할 수 있는 도구를 제공한다. 벡터 검색 시장의 급속한 성장으로 수많은 벡터 DB가 등장했으나, 성능 비교를 위한 표준화된 평가 방식이 부재했다.

본 벤치마크는 다양한 벡터 DB (Milvus, Weaviate, Pinecone, Vespa 등)와 데이터셋을 지원하여 공정한 비교를 가능하게 한다. 핵심 메트릭은: - **throughput (QPS)**: 초당 처리 쿼리 수 - **latency**: 개별 쿼리 응답 시간 -

**recall**: 검색 정확도 (top-k 일치도) - **memory**: 메모리 사용량 - **indexing\_time**: 인덱스 구축 시간

벤치마크 설계의 특징은 실제 사용 시나리오를 반영하는 쿼리 믹스, 다양한 데이터 특성(차원, 크기, 분포), 그리고 필터링과 같은 추가 조건을 지원한다는 점이다.

본 논문과의 관계: Exqutor의 성능 평가에 필수적인 벤치마크 프레임워크를 제공한다. 범위 필터와 벡터 검색의 결합 조건에서의 성능을 측정할 수 있는 기반을 제공하며, 선택도 추정의 정확도가 최종 쿼리 성능에 미치는 영향을 정량화할 수 있다.

## 상세분석

### 75.1 벤치마크 설계 철학

공정성(Fairness) 원칙:

모든 시스템이 동등한 조건에서 테스트:

- └ 데이터 동일
- └ 쿼리 동일
- └ 메모리 제약 동일
- └ 하드웨어 동일
- └ 충분한 워밍업(warm-up) 시간 제공

**실제성(Realism) 추구**: - 합성 데이터가 아닌 실제 임베딩 데이터 사용 - 실제 검색 쿼리 패턴 모델링 - 동시 사용자 부하 시뮬레이션

**확장성(Scalability) 검증**: - 데이터 크기별 성능 측정 (1M ~ 10B 벡터) - 차원별 성능 변화 관찰 - 메모리 제약 하에서의 성능 곡선

### 75.2 지원 데이터셋

공개 데이터셋:

데이터셋	벡터 수	차원	특징
SIFT-1M	1M	128	자연 영상 특징
GIST-1M	1M	960	넓은 영상 특징
Deep1B	1B	96	심층 학습 기반
OpenAI embeddings	1M	1536	텍스트 임베딩
Cohere embeddings	1M	4096	고차원 텍스트

데이터셋 특성: - 자연 데이터: 실제 이미지/텍스트 임베딩 - 합성 데이터: 다양한 분포(uniform, gaussian, clustered) 제어 가능 - 스케일: 소형(1M)부터 대형(10B)까지

## 75.3 성능 메트릭 정의

### 3.1 Recall (정확도)

$Recall@k = \frac{|\text{실제 top-}k \cap \text{반환된 top-}k|}{k}$

예: 실제 top-10은 [1,2,3,4,5,6,7,8,9,10]  
반환된 top-10은 [1,2,3,4,5,11,12,13,14,15]  
 $Recall@10 = 5/10 = 50\%$

### 3.2 QPS (Query Per Second)

$QPS = \frac{\text{총 쿼리 수}}{\text{측정 시간(초)}}$

측정 조건:

- 단일 쓰레드 QPS (기준선)
- 멀티 쓰레드 QPS (병렬성 측정)
- 부하 곡선 (동시 클라이언트 증가에 따른 QPS 변화)

### 3.3 Latency (응답 시간)

p50\_latency: 중앙값 응답 시간  
p95\_latency: 상위 5% 응답 시간 (꼬리 지연)  
p99\_latency: 상위 1% 응답 시간

시스템 품질 판단:

- p50: 일반적 성능
- p95/p99: 안정성 (SLA 확보)

### 3.4 Throughput-Latency 트레이드오프

높은 QPS = 낮은 latency? (반드시 아님)  
└─ 배치 처리로 높은 처리량 얻을 수 있으나 개별 지연 증가

### 3.5 메모리 사용량

인덱싱 메모리: 구축 중 최대 메모리  
서빙 메모리: 쿼리 처리 중 메모리 (인덱스 + 버퍼)  
메모리 효율: QPS / GB (단위 메모리당 처리량)

## 75.4 쿼리 유형 및 시나리오

### 기본 쿼리 유형:

#### 4.1 벡터 유사성 검색

```
query = {
 "vector": [0.1, 0.2, ..., 0.5], # d-차원 벡터
 "top_k": 10,
 "metric": "cosine" # or "l2", "inner_product"
}
```

#### 4.2 범위 필터 + 벡터 검색

```
query = {
 "vector": [0.1, 0.2, ..., 0.5],
 "top_k": 10,
 "filter": {
 "price": {"$lt": 100},
 "category": "electronics"
 }
}
```

#### 4.3 메타데이터 필터만

```
query = {
 "filter": {
 "date": {"$gte": "2024-01-01"},
 "status": "active"
 }
}
벡터 검색 없이 필터링만
```

#### 4.4 하이브리드 검색

```
query = {
 "vector_search": {...},
 "keyword_search": {...},
 "combine": "linear_combination" # 가중 조합
}
```

## 75.5 벤치마크 실행 워크플로우

### 단계 1: 초기화

데이터셋 다운로드 / 생성  
벡터 DB 인스턴스 시작 (컨테이너 기반)  
메모리, CPU 할당 확인

## 단계 2: 인덱싱

벡터들을 DB에 로드  
다양한 인덱싱 파라미터로 여러 설정 시도  
└ HNSW M=16, ef=200  
└ HNSW M=8, ef=100  
└ IVF nlist=1000, nprobe=10  
└ ...

## 단계 3: 워밍업

100회 쿼리 실행 (캐시 워밍)  
메모리 할당 안정화  
스케줄러 스트레스 해제

## 단계 4: 벤치마크 실행

10,000~100,000개 쿼리 실행  
각 쿼리의 latency, recall 기록

## 단계 5: 결과 분석

QPS, p50/p95/p99 latency 계산  
Recall vs Latency 곡선 그리기  
메모리 효율 계산

## 75.6 결과 예시 및 해석

출력 형식:

```
System: Milvus 2.3
Dataset: SIFT-1M
Metric: L2

Configuration: index_type=HNSW, M=16, ef_construction=200, ef_search=10

Result:
QPS: 1,200 req/s
Latency (p50): 0.8 ms
Latency (p95): 2.5 ms
Latency (p99): 5.0 ms
Recall@10: 98.5%
Memory: 850 MB
Memory efficiency: 1.41 QPS/MB
```

**결과 해석:** - QPS 높음(1200): 처리 능력 우수 - p99 latency 낮음(5ms): SLA 만족 가능 - recall 높음(98.5%): 정확도 우수 - 메모리 효율 좋음(1.41): 메모리당 처리량 많음

## 75.7 필터링 시나리오 벤치마크

**필터 선택도의 영향:**

```
필터 선택도 10% (10,000개 벡터만 매칭):
QPS: 1,800 req/s (필터만으로 90% 제거)
Recall@10: 95%

필터 선택도 50% (500,000개 벡터 매칭):
QPS: 950 req/s
Recall@10: 95%

필터 선택도 100% (모든 벡터 매칭):
QPS: 800 req/s
Recall@10: 95%
```

**발견:** - 선택도가 낮을수록 QPS 향상 (필터로 대부분 제거) - 선택도 추정의 정확성이 중요 (과소/과다 추정 시 성능 저하)

## 75.8 본 논문과의 관계

**Exqutor 성능 평가:**

Vector-db-benchmark는 다음과 같은 평가를 가능하게 한다:

1. 기본 성능 측정:
2. 범위 필터 없음: 기준선 QPS
3. 범위 필터 있음: 필터 적용 시 QPS



4. 성능 저하율: (기준선 QPS - 필터 있음 QPS) / 기준선 QPS

5. **선택도 추정의 영향:** 정확한 선택도 추정 → 최적 실행 계획 → 높은 성능 부정확한 선택도 추정 → 나쁜 실행 계획 → 낮은 성능

6. **메모리 제약 시 성능:** `` Exqutor는 제한된 메모리 내에서:

7. 선택도 추정 모델

8. 벡터 인덱스

9. 메타데이터 캐시 를 관리해야 함 ``

## 추가 제기 문제

1. **우편향 평가:** 특정 DB에 유리한 데이터셋이나 쿼리 패턴은 없는가?
2. **실시간 업데이트 벤치마크:** 인덱싱 중 동시 쿼리 처리 성능은?
3. **멀티테넌시 시나리오:** 여러 사용자의 동시 쿼리로 인한 성능 간섭은?
4. **캐시 효과 통제:** 운영 체제 캐시로 인한 성능 향상을 어떻게 통제할 것인가?
5. **선택도 추정 정확도의 영향:** 선택도 추정 오류가 최종 성능에 얼마나 영향을 미치는가?
6. **네트워크 레이턴시:** 원격 벡터 DB 접근 시 네트워크 지연의 영향은?

## (G) 벡터 DB 종합 분석 — 전문 벡터 DB와 서베이

### [76] ACORN: Performant and Predicate-Agnostic Search over Vector Embeddings and Structured Data

#### 요약

Patel et al.의 PACMMOD 2024 논문으로, 벡터 임베딩과 구조화된 데이터를 모두 처리하는 술어-무관(predicate-agnostic) 하이브리드 검색 시스템을 제시한다. 기존 시스템들은 특정 술어(예: "category = electronics")에 최적화되는 경향이 있었으나, ACORN은 임의의 술어 조합에 일반적으로 적용 가능한 접근법을 제안한다.

핵심 아이디어는 벡터 검색(approximate nearest neighbor search)과 메타데이터 필터링을 독립적으로 실행한 후, 효율적인 방식으로 결과를 병합(merge)하는 것이다. 특별히, 기존의 "필터-후-검색(filter-then-search)" 방식의 한계를 극복하고, 다양한 필터 조건에 동적으로 적응하는 "동적 병합(dynamic merging)" 전략을 제안한다.

주요 기여: 1. **술어 무관성**: 어떤 형태의 술어든 지원 (범위, 등호, 복합) 2. **동적 최적화**: 필터 선택도에 따라 검색 전략 자동 조정 3. **성능**: 기존 방법 대비 2-5배 빠른 처리

본 논문과의 관계: Exqutor가 해결하는 핵심 문제와 정확히 일치한다. 범위 필터와 벡터 검색의 결합에서 최적의 실행 전략을 선택하는 방식이 ACORN의 방법론을 직접 참고한다.

## 상세분석

### 76.1 하이브리드 검색의 문제 정의

시나리오:

데이터베이스: 전자제품 카탈로그  
- 벡터: 제품 설명 임베딩 (1536차원)  
- 메타데이터: 가격, 카테고리, 별점, 재고 등  
  
쿼리: "스마트폰과 유사한 제품 중 가격 < 500달러"

기존 접근의 문제:

방식 1: 필터-후-검색 (Filter-then-Search)

1단계: 가격 < 500 필터링  
→ 100,000개 제품 중 10,000개 남음 (선택도 10%)  
2단계: 남은 10,000개 제품에서만 벡터 검색  
→ 빠름!  
  
BUT: 선택도가 높으면?  
1단계: 가격 < 2000 필터링  
→ 100,000개 중 90,000개 남음 (선택도 90%)  
2단계: 남은 90,000개 제품에서 벡터 검색  
→ 매우 느림!

방식 2: 검색-후-필터 (Search-then-Filter)

1단계: 모든 제품에서 상위 1000개 유사 제품 검색  
2단계: 상위 1000개에서 가격 < 500 필터  
→ 일반적으로 빠름  
  
BUT: 필터 선택도가 매우 낮으면?  
1단계: 모든 제품에서 상위 1000개 검색  
2단계: 상위 1000개 중 필터 조건 만족하는 제품이 5개만?  
→ 결과 부족, 다시 더 많이 검색 필요 (반복)

## 76.2 동적 병합(Dynamic Merging) 전략

핵심 개념:

```
└ 벡터 검색 스트림
├ └ 제품 A (유사도: 0.95)
├ └ 제품 B (유사도: 0.92)
├ └ 제품 C (유사도: 0.90)
├ └ ...
└
└ 메타데이터 필터 스트림
 └ 제품 A (가격: 400, 만족)
 └ 제품 D (가격: 300, 만족)
 └ 제품 E (가격: 450, 만족)
 └ ...
```

동적 병합:

현재까지 찾은 결과: A (유사도 0.95, 가격 0)

다음 후보: B (유사도 0.92) vs D (가격만족)

최소 유사도 임계값: B의 유사도가 D 검색에 필요한 수준보다 높으면 B 우선

알고리즘:

```
def dynamic_merge_search(query_vector, filter_predicate):
 vector_iter = vector_search_knn(query_vector) # 유사도순 스트림
 metadata_iter = scan_with_filter(filter_predicate) # 필터 만족하는 항목들

 results = []

 while len(results) < k:
 next_vector = vector_iter.peek() # 다음 유사 항목
 next_metadata = metadata_iter.peek() # 다음 필터 만족 항목

 # 동적 결정: 어느 쪽을 더 탐색할지?
 if should_continue_vector_search(next_vector.similarity):
 results.append(next_vector)
 vector_iter.advance()
 else:
 # 벡터 검색 중단, 메타데이터 필터로 전환
 break

 return results[:k]
```

## 76.3 선택도 기반 전략 선택

선택도 추정의 중요성:

선택도( $\sigma$ ) = 필터를 만족하는 행의 비율

$\sigma < 10\%$  (매우 낮음):

└ Filter-then-Search 추천

1. 빠른 필터 실행 (90% 제거)
2. 작은 집합에서만 벡터 검색

$\sigma = 50\%$  (중간):

└ Dynamic Merge 추천

1. 벡터 검색과 메타데이터 필터 병렬 진행
2. 조건에 맞는 결과가 나올 때까지 계속

$\sigma > 90\%$  (매우 높음):

└ Search-then-Filter 추천

1. 모든 항목에서 벡터 검색
2. 검색 결과 중에서 필터 적용

### 전략 선택의 비용:

Filter-then-Search 비용:

= 필터 스캔 비용 +  $(1-\sigma)N \times$  벡터검색 비용

Search-then-Filter 비용:

= 모든항목 벡터검색 비용 + 필터 검사 비용

Dynamic Merge 비용:

= min(위 두 방식 비용)

## 76.4 선택도 추정과의 연관성

### 정확한 선택도 추정의 중요성:

실제 선택도: 15%

추정된 선택도: 50%

결과:

- Dynamic Merge 선택
- 실제로는 Filter-then-Search가 2배 빨랐음
- 성능 저하 발생

**Exqutor와의 연계:** Exqutor의 선택도 추정이 정확해야 ACORN의 동적 병합 전략이 올바른 결정을 내릴 수 있다.

## 76.5 복합 술어 처리

### 기본 술어:

```
price < 500 (범위)
category = 'phone' (등호)
rating > 4.5 (범위)
```

복합 술어:

```
(price < 500 AND category = 'phone') OR rating > 4.9
```

이를 정규 형식으로:

DNF (Disjunctive Normal Form):

```
(price < 500 AND category = 'phone')
OR (rating > 4.9)
```

선택도 계산:

```
 $\sigma(A \text{ OR } B) = \sigma(A) + \sigma(B) - \sigma(A \text{ AND } B)$
= 15% + 2% - 0.3%
= 16.7%
```

## 76.6 실험 결과

**벤치마크 설정:** - 데이터셋: Amazon products (100M), NYC taxi (100M) - 쿼리: 다양한 선택도의 필터 조건 -  
**비교:** Filter-then-Search, Search-then-Filter, ACORN

**결과 (쿼리 응답 시간):**

선택도	FTS	STF	ACORN	최적
5%	50ms	800ms	52ms	FTS (50ms)
25%	200ms	300ms	210ms	STF (300ms)
50%	350ms	350ms	360ms	동등
75%	500ms	200ms	210ms	STF (200ms)
95%	600ms	100ms	102ms	STF (100ms)

**ACORN의 성능:** - 최적 방식의 95-105% 성능 유지 - 선택도 미리 알 수 없는 상황에서 일반적 우수 성능

## 76.7 본 논문과의 관계

**Exqutor의 전략 선택 문제:**

Exqutor도 유사한 딜레마에 직면한다:

#### 범위 필터와 벡터 검색의 결합

선택도  $\sigma$ 가 높음 (대부분 벡터가 필터 통과):

└ 전체 벡터 인덱스 사용해 검색 후 필터링

선택도  $\sigma$ 가 낮음 (매우 적은 벡터만 필터 통과):

└ 먼저 필터 실행 후 남은 벡터만 인덱싱

**정확한 선택도 추정 중요성:** - 잘못된 전략 선택 시 성능 저하 - 선택도 추정 오류  $\pm 20\%$  범위 내 유지 필요

**동적 최적화:** - 쿼리 특성에 따라 전략 동적 변경 - 적응적 알고리즘으로 최악의 경우 회피

## 추가 제기 문제

1. **선택도 오류의 민감도:** 선택도 추정이  $\pm 50\%$  오류일 때, ACORN의 성능 저하는 얼마나 되는가?
2. **복잡한 복합 술어:** AND, OR, NOT이 중첩된 매우 복잡한 술어에서 선택도를 정확히 추정할 수 있는가?
3. **시간 경향(Temporal Trend):** 시간대별로 데이터 분포가 변할 때, 선택도 추정을 어떻게 적응시킬 것인가?
4. **다중 인덱스 활용:** 여러 차원에 인덱스가 있을 때, 어느 인덱스를 우선 사용할지 결정하는 방법은?
5. **병렬 처리:** 벡터 검색과 메타데이터 필터링을 병렬로 실행할 때의 오버헤드는?
6. **점진적 결과 반환:** 모든 결과를 찾기 전에 부분 결과를 사용자에게 스트리밍할 수 있는가?

## [77] SERF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search

### 요약

Zuo et al.의 2024년 논문으로, 범위 필터(range filter)가 있는 ANN 검색을 위한 세그먼트 그래프 기반 인덱싱 방법을 제시한다. 이는 Exquitor의 핵심 문제인 "범위 필터 + 벡터 검색"을 직접 다루는 논문이다.

기존 ANN 인덱스(HNSW, IVF)는 벡터 유사성만을 고려하기 때문에, 범위 필터를 적용하려면 검색 후 필터링하거나 필터링 후 검색해야 했다. 하지만 SERF는 범위 필터 조건을 인덱스 구조 자체에 반영한다.

핵심 아이디어는 "세그먼트 그래프(Segment Graph)"로, 벡터 공간을 메타데이터(예: 가격 범위)에 따라 세그먼트로 분할하고, 각 세그먼트 내에서 독립적인 그래프를 유지한다. 검색 시에는 필터 조건에 해당하는 세그먼트들만 탐색하므로, 불필요한 벡터들을 애초에 방문하지 않는다.

성능 개선: 범위 필터 선택도가 낮을 때(10% 이하) 기존 방법 대비 2-10배 빠른 검색.

본 논문과의 관계: SERF는 범위 필터 선택도 추정에 직접 의존한다. 필터 조건에 해당하는 세그먼트 수를 정확히 파악하려면 선택도가 필요하다. Exqutor의 선택도 추정 모듈은 SERF 같은 범위 필터 인덱스의 효율성을 결정한다.

## 상세분석

### 77.1 범위 필터 ANN 검색의 문제점

기존 방법의 한계:

#### 방법 1: Filter-then-Search

가격 필터:  $\text{price} \in [100, 500]$   
선택도: 10% (1,000,000개 중 100,000개)

단계:

1. 범위 스캔으로 100,000개 항목의 벡터ID 획득
2. 100,000개 벡터에서만 ANN 검색

문제: 범위 필터 선택도가 높으면 느림  
선택도 80% → 800,000개 벡터 검색 필요

#### 방법 2: Search-then-Filter

단계:

1. 모든 벡터에서 top-K 유사 벡터 검색
2. 검색 결과에서 가격 필터 적용

문제: 필터 선택도가 낮으면, top-K 결과에 필터 조건을 만족하는 벡터가 거의 없을 수 있음  
→ K를 크게 증가시켜야 함 → 검색 비용 급증

#### 방법 3: 별도 메타데이터 인덱스

가격에 대한 B-Tree 인덱스 유지  
벡터 HNSW 인덱스 유지

검색 시:

1. B-Tree로 가격 범위의 ID 얻기
2. HNSW에서 해당 ID 벡터들만 방문

문제: 두 인덱스 동기화 필수

메모리 오버헤드 2배

탐색 중 인덱스 간 점프로 캐시 미스 증가

## 77.2 세그먼트 그래프(Segment Graph) 개념

기본 구조:

가격 범위로 세그먼트 분할:

Segment 0: [0, 100) → 10,000 벡터

Segment 1: [100, 200) → 15,000 벡터

Segment 2: [200, 300) → 12,000 벡터

Segment 3: [300, 500) → 20,000 벡터

Segment 4: [500, ∞) → 40,000 벡터

각 세그먼트 내 HNSW 그래프:

Seg0: [벡터 구조] -- 연결 --

Seg1: [벡터 구조] -- 연결 --

Seg2: [벡터 구조] -- 연결 --

...

세그먼트 간 크로스링크:

Seg0 ↔ Seg1 ↔ Seg2 ...

(최근접 세그먼트 간의 경계 벡터들이 연결)

세그먼트 선택 기준: - 수치 범위:  $price \in [L, U]$  - 범주 속성:  $category = 'electronics'$  - 시간:  $date \in [start, end]$  - 복합:  $(price < 500) \text{ AND } (category = 'electronics')$

## 77.3 세그먼트 그래프 구축 알고리즘

오프라인 구축 단계:



```
def build_segment_graph(vectors, metadata, partition_attr):
 # 1단계: 메타데이터 기준으로 세그먼트 분할
 segments = partition_by_attribute(vectors, metadata, partition_attr)

 # 2단계: 각 세그먼트 내에서 HNSW 구축
 segment_graphs = {}
 for seg_id, vectors_in_seg in segments.items():
 segment_graphs[seg_id] = build_hnsw(vectors_in_seg)

 # 3단계: 세그먼트 간 크로스링크 구축
 for seg_i, seg_j in adjacent_segments(segments):
 create_cross_segment_edges(
 segment_graphs[seg_i],
 segment_graphs[seg_j],
 num_edges=10 # 세그먼트당 10개 엣지
)

 return segment_graphs, segment_boundaries
```

#### 세그먼트 크기 결정:

세그먼트 크기  $s$ :

- 너무 작음 ( $s < 1000$ ): 세그먼트 수 증가 → 메모리 오버헤드
- 너무 큼 ( $s > 100000$ ): 한 세그먼트에 대부분 벡터 → 필터링 효과 감소
- 최적:  $s \approx \sqrt{n}$  ( $n$  = 전체 벡터 수)

예: 1,000,000개 벡터 → 세그먼트당 ~1,000개  
→ 약 1,000개 세그먼트

## 77.4 범위 필터 쿼리 검색 알고리즘

### 온라인 검색 단계:

```
def range_filtered_search(query_vector, range_filter, top_k):
 # 1단계: 필터 조건에 맞는 세그먼트 식별
 target_segments = identify_segments(range_filter)
 # 예: price ∈ [100, 500]이면 Segment 1, 2, 3 선택

 # 2단계: 각 세그먼트에서 독립적으로 ANN 검색
 segment_candidates = {}
 for seg_id in target_segments:
 segment_candidates[seg_id] = hnsf_search(
 segment_graphs[seg_id],
 query_vector,
 k=top_k * len(target_segments) # 각 세그먼트에서 충분히 가져오기
)

 # 3단계: 모든 세그먼트 결과를 병합 및 정렬
 all_candidates = merge_sorted(segment_candidates.values())

 # 4단계: 상위 top_k 반환
 return all_candidates[:top_k]
```

### 세그먼트 식별의 효율성:

쿼리: price ∈ [100, 500]

필터링 없음: 1,000,000개 벡터 모두 검색 필요

세그먼트 그래프:

- Segment 1: [100, 200) → 0
- Segment 2: [200, 300) → 0
- Segment 3: [300, 500) → 0
- 나머지 세그먼트는 스킵

결과: 47,000개 벡터만 검색

성능 개선: 1,000,000 / 47,000 ≈ 21배

## 77.5 세그먼트 경계의 영향

경계 벡터 문제:

Segment 1: [100, 200)

Segment 2: [200, 300)

벡터 A: 벡터값 = [0.5, 0.3, ...], 가격 = 199 → Seg 1

벡터 B: 벡터값 = [0.5, 0.31, ...], 가격 = 200 → Seg 2

두 벡터는 벡터 공간에서 매우 가깝지만 다른 세그먼트에 속함!

→ 크로스 세그먼트 엣지가 해결

### 크로스 세그먼트 엣지의 역할:

Segment 1의 경계 벡터들이 Segment 2의 경계 벡터들과 연결되어 있으므로,  
Segment 1에서 탐색 시 필요하면 Segment 2로 "넘어갈" 수 있음

하지만: 필터 조건 밖인 세그먼트로는 가면 안 됨!

→ 크로스 엣지는 필터 세그먼트 내에서만 따라가야 함

## 77.6 다중 범위 필터 처리

### 단일 속성 범위:

$price \in [100, 500]$

→ Segment 1, 2, 3 선택 (직관적)

### 다중 속성 범위:

$price \in [100, 500] \text{ AND } rating \in [4, 5]$

문제: 세그먼트를 price로만 분할하면 rating 조건을 활용 못함

해결책 1: 계층 세그먼트화

└ 1단계: price로 분할

|        각 세그먼트를 다시 rating으로 분할

|        → 다차원 세그먼트 격자(grid)

└ 비용: 메모리 증가 (세그먼트 수 =  $n_{price\_segs} \times n_{rating\_segs}$ )

해결책 2: 필터링 후 세그먼트 선택

└ 메인 세그먼트: price 기준

└ 검색 중: rating 조건도 동시 확인

└ 메모리 효율적

└ 계산 비용: 추가 필터 검사

## 77.7 실험 결과

**벤치마크 설정:** - 데이터셋: Amazon (130M products), NYC Taxi (1B records) - 쿼리: 다양한 선택도의 범위 필터 - 비교: Filter-then-Search, Search-then-Filter, SERF

**결과 (쿼리 응답 시간, ms):**

선택도	FTS	STF	SERF	개선
1%	20	300	22	STF 대비 13배
5%	80	250	90	STF 대비 2.8배
10%	150	220	160	STF 대비 1.4배
50%	600	600	610	동등
90%	1500	500	510	FTS 대비 2.9배

**메모리 오버헤드:**

기본 HNSW: 1 GB  
SERF: 1.15 GB (15% 증가)  
  
크로스 세그먼트 엣지: 약 50 MB  
세그먼트 경계 정보: 약 10 MB

## 77.8 선택도 추정과의 연관성

**SERF의 성능은 선택도 추정에 의존:**

1. 쿼리 옵티마이저가 선택도  $\sigma$  추정
  2.  $\sigma > 50\%$ 이면 SERF 대신 전체 HNSW 검색 선택  
 $\sigma < 50\%$ 이면 SERF로 세그먼트 선택 검색
- 정확한 선택도:
- $\sigma = 15\% \rightarrow$  SERF 사용  $\rightarrow$  빠름 (150ms)
- 부정확한 선택도:
- 추정값 50%  $\rightarrow$  FTS 선택  $\rightarrow$  느림 (600ms)
  - 추정값 10%  $\rightarrow$  SERF 선택하지만 실제 선택도 50%  $\rightarrow$  느림 (610ms)

**Exqutor의 선택도 추정이 SERF 성능을 결정:**

Exqutor의 경량 선택도 모델 오류:

- $\pm 10\%$  범위: 최적 전략 선택 가능
- $\pm 30\%$  범위: 1.3배 성능 저하
- $\pm 50\%$  범위: 2배 이상 성능 저하

## 77.9 본 논문과의 관계

Exqutor가 SERF를 사용할 때의 선택도 추정:

Exqutor는 범위 필터의 정확한 선택도를 추정해야 SERF가 최적 성능을 낼 수 있다.

Exqutor 아키텍처:

범위 필터 조건 입력

↓

선택도 추정 모듈 (논문 [70] 경량 ML 모델)

↓

선택도:  $\sigma = 0.12$  (12%)

↓

쿼리 옵티마이저

└  $\sigma < 30\% \rightarrow$  SERF 세그먼트 검색 선택

└  $\sigma \geq 30\% \rightarrow$  전체 HNSW 검색 후 필터

↓

실행 및 성능 측정

## 추가 제기 문제

1. **세그먼트 경계 선택**: 균등 크기 세그먼트 vs 데이터 기반 세그먼트화, 어느 것이 더 나은가?
2. **동적 세그먼트화**: 데이터 분포가 시간에 따라 변할 때, 세그먼트 경계를 재구성해야 하는가?
3. **여러 속성 동시 필터**: price AND rating AND category 등 여러 범위 필터를 효율적으로 처리하는 방법은?
4. **세그먼트 간 벡터 분포**: 각 세그먼트 내 벡터들의 분포가 다를 때 (예: 고가 제품은 모두 프리미엄, 저가 제품은 다양), HNSW 파라미터를 어떻게 조정할 것인가?
5. **업데이트 비용**: 새로운 벡터 추가 시 올바른 세그먼트를 찾고 그래프를 업데이트하는 비용은?
6. **선택도 오류의 회피**: 선택도 추정이 매우 부정확할 때, 최악의 경우를 회피하는 보수적 전략은?

## [78] HQANN: Efficient and Robust Similarity Search for Hybrid Queries with Structured and Unstructured Constraints

### 요약

Wu et al.의 CIKM 2022 논문으로, 구조화된 제약(structured constraints, 범위 필터)과 비구조화된 제약(unstructured constraints, 벡터 검색)을 모두 가진 하이브리드 쿼리 처리를 다룬다. 기존 시스템들이 이러한 이질적 제약을 분리하여 처리했다면, HQANN은 통합적 접근을 제안한다.

핵심 기여: 1. **제약 결합 메커니즘**: 구조화된 필터와 비구조화된 벡터 조건을 함께 고려하는 검색 알고리즘 2. **적응적 전략**: 데이터와 쿼리 특성에 따라 처리 순서 동적 결정 3. **강건성(Robustness)**: 극단적 선택도(매우 높거나 매우 낮음)에서도 안정적 성능 유지

알고리즘은 두 제약의 선택도를 추정하고, 이를 기반으로 최적의 처리 전략을 선택한다. 또한 검색 과정에서 동적으로 탐색 경로를 조정하여 원하는 결과를 효율적으로 찾는다.

본 논문과의 관계: HQANN은 Exqutor가 직면하는 정확히 같은 문제를 다룬다. 범위 필터(structured)와 벡터 검색(unstructured)의 결합에서 최적 성능을 달성하는 방법론을 제시한다. 특히 선택도 추정을 통한 적응적 전략 선택이 Exqutor의 핵심 메커니즘과 부합한다.

### 상세분석

#### 78.1 하이브리드 쿼리 문제의 정의

쿼리 예시:

```
SELECT TOP 10 *
FROM products
WHERE (price >= 100 AND price <= 500) -- 구조화된 제약
 AND SIMILARITY(embedding, query_vector) >= 0.8 -- 비구조화된 제약
ORDER BY SIMILARITY(embedding, query_vector) DESC
```

두 제약의 특성:

측면	구조화된 제약	비구조화된 제약
형태	범위, 등호, 부등호	벡터 유사도 비교
효율성	$O(\log n) \sim O(n)$	$O(n \log n)$ (ANN)
정확도	100%	~95% (recall 기준)
선택도	예측 가능	데이터 분포에 의존
인덱스	B-Tree, Hash	HNSW, IVF

## 78.2 하이브리드 쿼리 처리 전략

### 전략 1: 순차적 처리 (Sequential)

(A) 필터 먼저: Filter-then-Search  
범위 필터 → ANN 검색  
비용 = 필터 스캔 +  $(1 - \sigma_{\text{filter}})N \times \text{ANN\_검색}$

(B) 검색 먼저: Search-then-Filter  
ANN 검색 → 범위 필터  
비용 = ANN\_검색 + 필터\_검사  $\times K$

### 전략 2: 병렬 처리 (Parallel)

필터 탐색  $\uparrow$   
└─ 병합 및 정렬  
검색 탐색  $\downarrow$

비용 =  $\max(\text{필터\_시간}, \text{ANN\_시간}) + \text{병합\_시간}$

### 전략 3: 통합 처리 (Integrated)

하나의 통합 인덱스에서 두 제약을 동시에 적용  
- HQANN의 접근

## 78.3 HQANN의 핵심 알고리즘

### 3.1 이중 경로 탐색(Dual-Path Search)

```

def hqann_search(query_vector, range_filter, top_k):
 # 1단계: 양쪽 제약 모두 만족하는 후보 찾기
 candidates = []
 visited = set()

 # 초기 진입점: 필터 조건 근처에서 시작
 entry_points = find_entry_points(range_filter)

 # 2단계: 그래프 탐색 (HNSW 기반)
 priority_queue = PriorityQueue()
 for ep in entry_points:
 priority_queue.add(ep, distance=0)

 while len(candidates) < top_k and not priority_queue.empty():
 node = priority_queue.pop()

 if node in visited:
 continue
 visited.add(node)

 # 양쪽 제약 모두 확인
 if satisfies_range_filter(node, range_filter) and \
 is_good_for_vector_similarity(node, query_vector):
 candidates.add(node)

 # 이웃 탐색
 for neighbor in get_neighbors(node):
 if neighbor not in visited:
 dist = compute_distance(query_vector, neighbor)
 priority_queue.add(neighbor, distance=dist)

 return candidates[:top_k]

```

### 3.2 선택도 기반 진입점 선택



range\_filter에서  $\sigma_r$  (범위 필터 선택도) 계산

$\sigma_r < 10\%$ :

- └ 범위 필터 만족하는 벡터에서 시작  
(필터링된 인덱스 사용)

$\sigma_r > 50\%$ :

- └ 전체 벡터에서 유사도 기반 시작  
(벡터 유사도 인덱스 사용)

$10\% < \sigma_r < 50\%$ :

- └ 두 조건의 균형 잡기  
(하이브리드 진입점 선택)

## 78.4 강건한 전략 선택

선택도 쌍을 이용한 전략 결정:

$\sigma_r$ : 범위 필터 선택도

$\sigma_v$ : 벡터 유사도 선택도 (top-k 반환 비율)

경우 1:  $\sigma_r$  매우 낮음 ( $< 5\%$ ),  $\sigma_v$  높음 ( $> 50\%$ )

추천: Filter-then-Search

이유: 필터로 대부분 제거 가능, 남은 것에서만 검색

경우 2:  $\sigma_r$  높음 ( $> 80\%$ ),  $\sigma_v$  낮음 ( $< 20\%$ )

추천: Search-then-Filter

이유: 벡터 검색으로 상위 k 찾고 필터 확인

경우 3:  $\sigma_r$  중간 (20-80%),  $\sigma_v$  중간 (30-70%)

추천: Integrated HQANN 전략

이유: 순차 처리의 단점이 클 수 있음

경우 4: 하나는 매우 높고 하나는 매우 낮음

추천: 조건부 필터-검색 (Conditional Filter-Search)

이유: 낮은 선택도 조건 먼저, 높은 선택도 조건은 확인만

## 78.5 동적 탐색 조정

런타임 피드백 루프:

초기 전략 선택 (선택도 기반)

↓

일정 수의 쿼리 결과 탐색

↓

실제 선택도 측정

↓

예상과 실제 선택도 비교

↓

└─ 차이 크면: 탐색 방향 조정

└─ 예상 선택도보다 높음: 더 탐색

└─ 예상 선택도보다 낮음: 탐색 확장

예시:

예상 선택도:  $\sigma_r = 15\%$

초기 전략: Filter-then-Search

탐색 중:

- 처음 100개 노드 방문: 15개가 범위 필터 만족 ✓

- 다음 100개 노드 방문: 5개만 범위 필터 만족 (예상 15개)

발견: 예상보다 낮은 선택도 → 더 많이 탐색 필요

조정: 탐색 반경 확대

## 78.6 혼합 인덱스 구조

다중 인덱싱 방식:

방식 1: 독립 인덱스 (Independent)

└─ 범위 필터 인덱스: B-Tree (price)

└─ 벡터 인덱스: HNSW

문제: 두 인덱스 조율 어려움

방식 2: 계층 인덱스 (Hierarchical)

└─ 범위 필터로 1차 분할

└─ 각 분할 내에서 벡터 인덱스

문제: 범위 필터 변경 시 구조 재구성 필요

방식 3: 통합 인덱스 (Unified - HQANN)

└─ 메인 HNSW 그래프

└─ 각 노드에 범위 필터 메타데이터 첨부

└─ 범위 만족 노드만 우선적으로 방문

이점: 유연한 필터 조건, 캐시 효율

## 78.7 실험 결과

**벤치마크 설정:** - 데이터셋: Yahoo Products (10M), StackOverflow (1M) - 쿼리: 다양한 ( $\sigma_r$ ,  $\sigma_v$ ) 조합 - 비교: Filter-then-Search, Search-then-Filter, HQANN

**결과 (쿼리 응답 시간, ms):**

( $\sigma_r$ , $\sigma_v$ )	FTS	STF	HQANN	최적	HQANN 효율
(5%, 70%)	30	500	35	30	117%
(15%, 50%)	100	300	110	100	109%
(50%, 50%)	400	400	410	400	102%
(80%, 20%)	700	150	160	150	107%
(90%, 5%)	1500	80	85	80	106%

**극단적 경우에서의 강건성:**

( $\sigma_r$ ,  $\sigma_v$ ) = (1%, 99%): 거의 모든 벡터가 조건 만족  
FTS: 매우 많이 탐색 필요 (800ms)  
STF: 빠른 검색 후 대부분 필터 통과 (100ms)  
HQANN: 벡터 유사도 기반 탐색으로 빠르게 결과 (110ms)

( $\sigma_r$ ,  $\sigma_v$ ) = (99%, 1%): 거의 모든 벡터가 필터 통과  
FTS: 빠른 필터 후 거의 대부분 검색 (900ms)  
STF: 빠른 검색 후 거의 모두 필터 통과 (800ms)  
HQANN: 범위 필터 기반 진입점으로 효율적 탐색 (750ms)

## 78.8 선택도 추정의 중요성

**정확한 선택도 추정이 필수:**

전략 선택은 선택도 추정에 의존

시나리오: 실제 ( $\sigma_r$ ,  $\sigma_v$ ) = (20%, 50%)  
추정: (20%, 50%) → HQANN 선택 → 110ms ✓  
추정: (5%, 70%) → FTS 선택 → 400ms ✗  
추정: (80%, 20%) → STF 선택 → 200ms (약 50% 저하)

## 78.9 본 논문과의 관계

**Exqutor의 핵심 문제와 HQANN의 솔루션:**

Exqutor가 해결해야 하는 문제:

범위 필터: WHERE price  $\in$  [L, U]  
벡터 검색: WHERE similarity(embedding) > threshold

두 조건의 효율적 결합?

HQANN의 답변:

```
1. 선택도 추정 (Exqutor 논문 [70])
 σ_r = selectivity_model.estimate(price_range)

2. 적응적 전략 선택
 if σ_r < 10%:
 사용 Filter-then-Search
 elif σ_r > 80%:
 사용 Search-then-Filter
 else:
 사용 HQANN 통합 처리

3. 동적 조정
 실제 탐색 중 선택도가 예상과 다르면 조정
```

Exqutor와 HQANN의 통합:

Exqutor 아키텍처:

```
쿼리 분석
 ↓
범위 필터 선택도 추정 (ML 모델)
 ↓
벡터 유사도 특성 파악 (데이터셋 통계)
 ↓
전략 선택 (HQANN 원리)
 ↓
범위 필터 + 벡터 검색 실행
 ↓
결과 반환
```

## 추가 제기 문제

1. **선택도 추정 오류의 영향:** 선택도 추정이  $\pm 50\%$  오류일 때, 부정확한 전략을 선택할 확률은?
2. **다중 필터 조건:** AND, OR로 결합된 여러 범위 필터가 있을 때, 선택도를 정확히 계산할 수 있는가?
3. **실시간 선택도 추정:** 쿼리별로 선택도를 추정하는 오버헤드는 얼마나 되는가?

4. **캐시 워밍**: 자주 나오는 (range\_filter, query\_vector) 쌍에 대해 캐싱할 수 있는가?
5. **동적 전략 전환**: 탐색 중간에 전략을 바꿀 때의 오버헤드와 이득의 균형은?
6. **상관성 처리**: 범위 필터 속성(가격)과 임베딩 차원 사이의 상관성이 있을 때 어떻게 활용할 것인가?

## (K) 샘플링 기반 추정

### [79] Random Sampling Over Joins Revisited

#### 요약

Zhao et al.의 SIGMOD 2018 논문으로, 조인 크기(join cardinality) 추정을 위한 현대적 무작위 샘플링(random sampling) 기법을 재검토한다. 이전 연구들(1990년대)이 샘플링의 이론적 기초를 제시했다면, 본 논문은 현대 데이터베이스 환경에서 샘플링의 실용성과 한계를 재평가한다.

핵심 기여: 1. **현대적 재평가**: 데이터 크기 증가, 분석 쿼리의 특성 변화에 따른 샘플링 효과 분석 2. **적응적 샘플링**: 쿼리 특성에 따라 샘플 크기와 방법을 동적으로 조정 3. **하이브리드 접근**: 샘플링과 다른 추정 기법의 조합

논문은 무작위 샘플링이 여전히 유용하지만, 극단적 선택도(매우 높거나 낮음)에서는 신뢰도가 떨어진다는 점을 보인다. 또한 샘플 수와 정확도의 관계를 정량화하여 실무에서의 샘플 크기 결정에 가이드를 제공한다.

본 논문과의 관계: Exqutor의 적응적 샘플링(Adaptive Sampling) 방식의 이론적 토대를 제공한다. 특히 범위 필터와 벡터 검색의 결합 선택도를 추정할 때, 샘플링이 얼마나 신뢰도 있는지를 평가할 수 있게 해준다. Exqutor는 이 논문의 방법론을 기반으로 적응적 샘플 크기 결정 메커니즘을 구현할 수 있다.

#### 상세분석

##### 79.1 샘플링의 기본 원리

무작위 샘플링 기반 선택도 추정:

데이터 집합 S: N개 행  
샘플 집합 T: n개 행 (S에서 무작위로 선택)

쿼리 술어 P에 대해:

s = P를 만족하는 S의 행 수

t = P를 만족하는 T의 행 수

추정:  $s_{est} = s \times (N / n)$

오류: 표준편차  $\sigma = \sqrt{s(N-s) / n}$

### 신뢰도 구간(Confidence Interval):

95% 신뢰도:

$[s_{est} - 1.96\sigma, s_{est} + 1.96\sigma]$

예시:

$s_{est} = 1000$  (추정 선택도 1%)

$\sigma = 100$

신뢰도 구간:  $[800, 1200]$  ( $\pm 20\%$  오류)

## 79.2 조인 선택도 추정의 특수성

### 단순 술어 vs 조인 술어:

단순 술어: `SELECT * FROM T WHERE age > 20`

적용: 단일 테이블에 대한 샘플링

조인 술어: `SELECT * FROM T1 JOIN T2 ON T1.id = T2.id WHERE age > 20`

적용: 두 테이블의 샘플을 조인 후 필터

문제: 조인이 샘플링 정확도에 미치는 영향

### 조인에서의 샘플링 방법:

#### 방법 1: 사전 샘플링(Pre-join Sampling)

T1에서 샘플 추출 → T2에서 샘플 추출 → 샘플들끼리 조인

장점: 빠름 (샘플 크기만큼만 조인)

단점: 두 샘플이 정렬되지 않음 (실제 조인 파트너 놓칠 수 있음)

정확도 손실: 높음 (특히 조인 선택도가 낮을 때)

#### 방법 2: 사후 샘플링(Post-join Sampling)

T1과 T2 전체 조인 → 결과에서 샘플 추출

장점: 정확함 (실제 조인 결과에서 샘플)

단점: 느림 (전체 조인 필요)

정확도: 높음 (단순 쿼리와 동일)

### 방법 3: 스트리밍 샘플링(Streaming Sampling)

조인 중간에 점진적으로 샘플 수집

충분한 신뢰도 도달 시 종료 (조인 완료 전)

장점: 빠르고 정확할 수 있음

단점: 조인 비용 예측 어려움

## 79.3 샘플 크기 결정

표본 오류(Sampling Error) 공식:

조인 선택도  $\sigma$ 에서 상대 오류  $\epsilon$ 를 원할 때:

필요한 샘플 크기:

$$n \geq (z^2 \times \sigma \times (1-\sigma)) / \epsilon^2$$

여기서  $z$  = 신뢰도 계수 (95%일 때  $z \approx 1.96$ )

예시:

$\sigma = 0.1$  (10% 선택도)

$\epsilon = 0.1$  ( $\pm 10\%$  오류 허용)

$z = 1.96$

$$n \geq (1.96^2 \times 0.1 \times 0.9) / 0.1^2$$

$$n \geq (3.84 \times 0.09) / 0.01$$

$$n \geq 34.56 \approx 35 \text{ 개 샘플}$$

다른 선택도:

$\sigma = 0.01$  (1% 선택도) →  $n \geq 354 \text{ 개}$

$\sigma = 0.001$  (0.1% 선택도) →  $n \geq 3,456 \text{ 개}$

$\sigma = 0.0001$  (0.01% 선택도) →  $n \geq 34,560 \text{ 개}$

선택도에 따른 샘플 크기 급증:

선택도가 1/10으로 감소하면, 필요 샘플 크기는 약 10배 증가

→ 매우 낮은 선택도는 샘플링으로 추정 불가능할 수 있음

## 79.4 현대 데이터에서의 실험 결과

**실험 설정:** - 데이터셋: TPC-H (1GB, 10GB), 실제 로그 분석 데이터 - 조건: 다양한 선택도 (0.1% ~ 100%) - 샘플 크기: 1% ~ 100%

**결과 (상대 오류):**

선택도	샘플 1%	샘플 5%	샘플 10%	샘플 50%
50%	2%	1%	<1%	<1%
10%	8%	3%	2%	<1%
1%	30%	10%	5%	1%
0.1%	>100%	30%	15%	3%
0.01%	>100%	>100%	50%	10%

**발견:** - 선택도 1% 이상: 5% 샘플로 충분 (오류 < 10%) - 선택도 0.1%: 50% 샘플 필요 (오류 3%) - 선택도 0.01% 이하: 샘플링으로는 신뢰도 낮음

## 79.5 적응적 샘플링

**동적 샘플 크기 조정:**

```
def adaptive_sampling(query, initial_sample_rate=0.01):
 sample_rate = initial_sample_rate
 confidence = 0.0

 while confidence < 0.95: # 95% 신뢰도 목표
 # 1단계: 현재 샘플 크기로 추정
 sample = data.sample(frac=sample_rate)
 result = query.execute(sample)

 # 2단계: 신뢰도 평가
 confidence = compute_confidence(result)

 # 3단계: 신뢰도 부족 시 샘플 증가
 if confidence < 0.95:
 sample_rate = min(sample_rate * 2, 1.0)
 else:
 break

 return result
```

**선택도 추정과의 피드백:**



초기 샘플에서 추정된 선택도  $\sigma_{est}$ :

└  $\sigma_{est}$  기반으로 필요 샘플 크기 계산

└ 추가 샘플 수집 여부 결정

└ 목표 신뢰도 달성 시 종료

## 79.6 극한 선택도에서의 문제

매우 낮은 선택도 ( $< 0.01\%$ ):

조인 결과:  $N1 \times N2$  중 극소수만 매칭

예:  $100M \times 100M = 10^{16}$  행 중 1,000행만 결과  
선택도 =  $10^{(-)13}$

이 경우:

- 1% 샘플:  $10^8 \times 10^8 = 10^{16}$  중 최대 몇 개 매칭?

- 기댓값  $1,000 / 10,000 = 0.1$  (극도로 불안정)

해결책:

1. 샘플 크기 대폭 증가 (비용 증가)
2. 샘플링 대신 다른 기법 사용 (통계, 모델 등)
3. 하이브리드: 샘플링 + 추적(tracing) 결합

## 79.7 본 논문과의 관계

Exqutor의 적응적 샘플링 전략:

Exqutor는 범위 필터와 벡터 검색의 선택도를 모두 고려해야 한다:

전체 쿼리 선택도:

$$\sigma_{total} = \sigma_{range} \times \sigma_{vector}$$

예:

$\sigma_{range} = 15\%$  (범위 필터)

$\sigma_{vector} = 80\%$  (벡터 유사도 상위 k%)

$$\sigma_{total} = 0.15 \times 0.80 = 12\%$$

필요한 샘플 크기 ( $\pm 10\%$  오류):

$$n \geq (1.96^2 \times 0.12 \times 0.88) / 0.1^2$$

$$n \geq 400 \text{ 개 (전체의 } \sim 0.04\%)$$

Exqutor의 적응적 샘플링:

1단계: 초기 샘플 (1% 크기)로 선택도 추정  
추정된  $\sigma = 12\% \rightarrow$  필요 샘플 약 400개

2단계: 신뢰도 평가  
현재 샘플: 10,000개 (1%)  
신뢰도: 95% 이상 달성

3단계: 충분하면 추정 결과 사용  
추가 샘플링 불필요

#### 논문 [79]의 이론적 기초:

- 샘플 크기와 정확도의 수학적 관계 제공
- 선택도에 따른 샘플 크기 계산 방법 제시
- 적응적 샘플링의 정당성 입증
- 극한 케이스에서의 한계 지적

### 추가 제기 문제

1. **샘플 바이어스**: 데이터가 균등하지 않거나 클러스터링될 때, 무작위 샘플링이 정확한가?
2. **시간 비용 트레이드오프**: 샘플 수집과 쿼리 실행에 걸리는 총 시간을 최소화할 최적 샘플 크기는?
3. **스트리밍 데이터**: 지속적으로 새로운 데이터가 추가될 때, 샘플을 어떻게 유지할 것인가?
4. **다중 쿼리**: 여러 쿼리의 선택도를 동시에 추정할 때, 샘플을 공유할 수 있는가?
5. **벡터 유사도의 샘플 추정**: 벡터 검색은 top-k 결과의 "선택도"가 정의되기 어려운데, 이를 샘플링으로 추정할 수 있는가?
6. **상관성 처리**: 범위 필터 속성과 벡터 임베딩 사이의 숨은 상관성이 있을 때, 샘플링 정확도에 미치는 영향은?

## [80] Selectivity and Cost Estimation for Joins Based on Random Sampling

### 요약

Haas, Naughton, Seshadri, Swami의 JCSS 1996 논문으로, 조인 선택도(join selectivity) 추정을 위한 무작위 샘플링의 이론적 기초를 제시한 '고전' 논문이다. 이 논문은 데이터베이스 통계와 표본론을 결합하여 샘플링 기반 추정의 수학적 모델을 정립했다.

핵심 기여: 1. **이론적 프레임워크**: 조인 크기 추정의 통계 모델 개발 2. **신뢰도 분석**: 샘플 크기와 추정 오류의 관계 공식화 3. **점근 분포(Asymptotic Distribution)**: 표본 크기가 클 때의 분포 특성 증명 4. **실용적 가이드**: 필요한

## 샘플 크기 결정 방법 제시

본 논문은 이후 30년 가까이 샘플링 기반 카디널리티 추정 연구의 이론적 기반이 되었다. 특히 신뢰도 구간 계산, 적응적 샘플링, 그리고 비용 모델에 광범위하게 인용된다.

본 논문과의 관계: Exqutor의 적응적 샘플링 방식의 직접적인 이론적 토대다. 특히 선택도 추정의 신뢰도를 정량화하고 필요한 샘플 크기를 결정하는 방법론을 제공한다. Exqutor는 범위 필터와 벡터 검색의 결합 선택도 추정에 이 논문의 통계 이론을 적용할 수 있다.

## 상세분석

### 80.1 조인 선택도의 수학적 정의

기본 정의:

테이블 T1:  $N_1$ 개 행

테이블 T2:  $N_2$ 개 행

조인 조건 P에 대해:

조인 결과 = T1과 T2의 행 중 P를 만족하는 쌍의 집합

조인 카디널리티:

$C = |\{(t1, t2) : t1 \in T1, t2 \in T2, P(t1, t2) = \text{true}\}|$

조인 선택도:

$\sigma = C / (N_1 \times N_2)$

예시:

T1: 고객 테이블 (10,000명)

T2: 주문 테이블 (100,000건)

조인 조건: `고객.id = 주문.고객_id`

조인 결과: 90,000건 (평균 고객당 9개 주문)

카디널리티:  $C = 90,000$

선택도:  $\sigma = 90,000 / (10,000 \times 100,000) = 0.009$  (0.9%)

### 80.2 샘플링 이론의 적용

표본 선택도의 분포:

T1에서  $n_1$ 개, T2에서  $n_2$ 개 행을 무작위로 샘플링할 때, 샘플 내 조인 결과를  $c$ 라 하면:

표본 선택도:  $\sigma_{\text{hat}} = c / (n_1 \times n_2)$

이 추정량의 분포:

Case 1: 선택도가 고정되어 있고  $n \rightarrow \infty$ 일 때

중심극한정리(Central Limit Theorem):

$$\sqrt{n} \times (\sigma_{\text{hat}} - \sigma) \rightarrow N(0, \sigma(1-\sigma))$$

분포:  $\sigma_{\text{hat}}$ 는 점근적으로 정규분포

평균:  $\sigma$

표준편차:  $\sqrt{\sigma(1-\sigma) / n}$

Confidence Interval (95% 신뢰도):

$$P(\sigma_{\text{hat}} - 1.96 \times \sigma_{\text{SE}} < \sigma < \sigma_{\text{hat}} + 1.96 \times \sigma_{\text{SE}}) = 0.95$$

$$\text{여기서 } \sigma_{\text{SE}} = \sqrt{\sigma_{\text{hat}}(1-\sigma_{\text{hat}}) / n}$$

정규 근사의 신뢰도를 높이려면  $n$ 이 충분해야 함:

$$n \times \sigma_{\text{hat}} > 5 \text{ and } n \times (1-\sigma_{\text{hat}}) > 5$$

## 80.3 조인에 특정한 이슈

독립성 가정:

조인 조건이 두 테이블의 행을 독립적으로 선택할 때:

$$\sigma(T1, T2) = \sigma(T1) \times \sigma(T2)$$

예:

T1에서 P\_1을 만족할 확률:  $\sigma_1 = 0.2$  (20%)

T2에서 P\_2를 만족할 확률:  $\sigma_2 = 0.5$  (50%)

조인 선택도:  $\sigma = 0.2 \times 0.5 = 0.1$  (10%)

(T1.A = T2.A인 등호 조인이 아닐 때 근사)

등호 조인(Equi-join)의 특수성:

T1.id = T2.id 형태의 조인

경우 1: 외래키 관계 있음

└ 모든 T2.id가 T1에 존재

$$\sigma = N_2 / (N_1 \times N_2) = 1 / N_1$$

(매우 낮은 선택도 가능)

경우 2: 일반적인 등호 조인

└  $\sigma = m / (N_1 \times N_2)$

m = 두 테이블의 공통 키 개수

## 80.4 점근 성질(Asymptotic Properties)

큰 n에서의 행동:

샘플 크기 n이 클 때:

1. 일관성(Consistency):

$$\sigma_{\text{hat}} \rightarrow \sigma \text{ (거의 확실하게)}$$

샘플 크기가 증가하면 추정치가 참값으로 수렴

2. 정규 수렴(Normal Convergence):

$$(\sigma_{\text{hat}} - \sigma) / \sqrt{\sigma(1-\sigma)/n} \rightarrow N(0, 1)$$

정규분포로 수렴 속도:  $O(1/\sqrt{n})$

3. 효율성(Efficiency):

$$\text{분산: } \text{Var}(\sigma_{\text{hat}}) = \sigma(1-\sigma) / n$$

가장 작은 분산을 가짐 (비편향 추정량 중)

## 80.5 신뢰도와 샘플 크기의 관계

정밀도 요구사항:

$\pm \epsilon$  범위의 오류를  $100(1-\alpha)\%$  신뢰도로 원할 때:

필요한 샘플 크기:

$$n = z^2_{\{1-\alpha/2\}} \times \sigma(1-\sigma) / \epsilon^2$$

여기서  $z_{\{1-\alpha/2\}}$ 는 표준 정규분포의 상위  $\alpha/2$  백분위수:

- 90% 신뢰도 ( $\alpha=0.1$ ):  $z = 1.645$

- 95% 신뢰도 ( $\alpha=0.05$ ):  $z = 1.960$

- 99% 신뢰도 ( $\alpha=0.01$ ):  $z = 2.576$

구체적 계산 예시:

목표: 선택도  $\sigma = 0.01$  (1%)을  $\pm 20\%$  상대 오류, 95% 신뢰도로 추정

절대 오류  $\varepsilon = \sigma \times 0.2 = 0.01 \times 0.2 = 0.002$

필요 샘플:

$$n = 1.96^2 \times 0.01 \times 0.99 / 0.002^2$$

$$n = 3.84 \times 0.0099 / 0.000004$$

$$n = 9,504$$

테이블이 1,000,000행이면: 샘플율 =  $9,504 / 1,000,000 \approx 1\%$

선택도에 따른 샘플 크기:

같은 조건 ( $\pm 20\%$  오류, 95% 신뢰도)에서:

$$\sigma = 0.5 \text{ (50\%): } n \geq 2,401$$

$$\sigma = 0.1 \text{ (10\%): } n \geq 9,604$$

$$\sigma = 0.01 \text{ (1\%): } n \geq 9,504 \text{ (}\sigma \text{와 (1-}\sigma\text{)이 비슷해서 최대)}$$

$$\sigma = 0.001 \text{ (0.1\%): } n \geq 95,040$$

$$\sigma = 0.0001 \text{ (0.01\%): } n \geq 950,399$$

선택도 감소에 따라 필요 샘플 선택도의 역함수에 비례

## 80.6 다중 조인에서의 확장

체인 조인(Chain Join):

T1 JOIN T2 JOIN T3

각 조인의 선택도:

$$\sigma_{1_2} = P(t_1, t_2 \text{ 만족}) = 0.1$$

$$\sigma_{2_3} = P(t_2, t_3 \text{ 만족}) = 0.2$$

결합 선택도 (독립성 가정):

$$\sigma_{\text{total}} = \sigma_{1_2} \times \sigma_{2_3} = 0.1 \times 0.2 = 0.02 \text{ (2\%)}$$

문제:  $\sigma_{\text{total}}$ 이  $\sigma_{1_2}$ ,  $\sigma_{2_3}$ 보다 낮음

→ 최종 카디널리티 추정이 더 불안정

스타 조인(Star Join):

중앙 테이블 T1과 여러 dimension 테이블 T2, T3, T4 조인

T1과 T2의 선택도:  $\sigma_{1,2} = 0.3$

T1과 T3의 선택도:  $\sigma_{1,3} = 0.4$

T1과 T4의 선택도:  $\sigma_{1,4} = 0.5$

결합 선택도:  $\sigma_{total} = 0.3 \times 0.4 \times 0.5 = 0.06$  (6%)

문제: 테이블이 많을수록 선택도는 더 낮아짐

## 80.7 비용 모델에의 적용

조인 실행 비용 추정:

조인 비용 = CPU\_비용 + I/O\_비용 + 메모리\_비용

CPU 비용: 조인된 행 수  $\times$  행당\_처리비용

$= C \times \text{cost\_per\_row}$

$= N_1 \times N_2 \times \sigma \times \text{cost\_per\_row}$

I/O 비용: 접근해야 할 페이지 수

$= (N_1 \times \text{page\_size}_1) + (N_2 \times \text{page\_size}_2)$

(선택도가 낮으면 CPU 비용이 지배적)

정확한  $\sigma$  추정  $\rightarrow$  정확한 비용 예측  $\rightarrow$  최적 실행 계획

## 80.8 본 논문과의 관계

Exqutor의 샘플링 기반 선택도 추정:

Exqutor는 범위 필터와 벡터 검색의 결합 선택도를 추정하는데, 이 논문의 이론을 직접 적용할 수 있다:

1단계: 이론적 기초 적용

범위 필터: `WHERE price  $\in$  [L, U]`

벡터 검색: `WHERE similarity(embedding) >  $\theta$`

쿼리 선택도:

$\sigma = P(\text{가격 조건}) \times P(\text{유사도 조건})$

이는 논문의 곱셈 규칙(independent join selectivity)

과 동일한 형태

2단계: 샘플 크기 결정

목표 정확도:  $\pm 15\%$  상대 오류, 90% 신뢰도  
추정된  $\sigma = 0.12$  (범위 필터 15% × 벡터 80%)

필요 샘플 크기:

$$\epsilon = 0.12 \times 0.15 = 0.018$$

$$z = 1.645 \text{ (90\% 신뢰도)}$$

$$n = 1.645^2 \times 0.12 \times 0.88 / 0.018^2$$

$$n \approx 250 \text{ 개 (전체 1\% 미만)}$$

### 3단계: 신뢰도 검증

수집한 샘플에서:

$c$  = 범위 필터 AND 유사도 조건 만족하는 행 수

신뢰도 구간:

$$\sigma_{\text{hat}} \pm 1.645 \times \sqrt{(\sigma_{\text{hat}}(1-\sigma_{\text{hat}}) / n)}$$

이 구간이 충분히 좁으면 추정 사용

너무 넓으면 추가 샘플링

### 논문 [80]의 핵심 가치:

1. **이론적 정당성**: 샘플링 기반 추정의 수학적 기초 제공
2. **샘플 크기 결정**:  $n = z^2 \times \sigma(1-\sigma) / \epsilon^2$  공식
3. **신뢰도 계산**: 점근 정규분포를 이용한 신뢰도 구간 계산
4. **다중 조인 확장**: 곱셈 규칙을 통한 복합 선택도 추정
5. **실무 적용**: 이후 모든 샘플링 기반 옵티마이저의 기반

## 추가 제기 문제

1. **소 샘플 문제**:  $n$ 이 작을 때( $n < 30$ ), 정규 근사가 타당한가? (카이제곱 등 다른 분포 필요?)
2. **상관성 위반**: 범위 필터와 벡터 임베딩이 상관되어 있을 때, 곱셈 규칙이 여전히 유효한가?
3. **모수  $\sigma$ 의 사전 정보**:  $\sigma$ 를 모르면 샘플 크기  $n$ 을 미리 결정할 수 없는데, 이를 어떻게 해결할 것인가?
4. **해결책**:  $\sigma = 0.5$ 로 가정 (최악의 경우) → 과도한 샘플 수집
5. **스트리밍 업데이트**: 데이터가 지속적으로 추가될 때, 과거 샘플이 유효한가?
6. **적응적 샘플링의 다중비교**: 여러 쿼리의 선택도를 동시에 추정할 때, 각각 독립적으로 샘플링하면 총 샘플 비용이 급증하는데, 이를 최적화할 수 있는가?



7. **비선형 선택도 관계**: 실제로는  $\sigma_1 \times \sigma_2 \neq \sigma_{total}$  일 때(상관성 있을 때), 편향을 추정할 수 있는가?

---

## [81] Practical Selectivity Estimation Through Adaptive Sampling

### 요약

Lipton, Naughton, Schneider의 SIGMOD 1990 논문으로, 실무에서 적용 가능한 적응적 샘플링(adaptive sampling) 방법을 제시한 '이정표' 논문이다. 이전 논문들([80])이 샘플링의 이론을 정립했다면, 본 논문은 현실의 불확실성(미지의 선택도, 시간 제약, 메모리 제약) 속에서 어떻게 실질적으로 동작하는 샘플링을 구현할지 보여준다.

핵심 기여: 1. **적응적 샘플링 알고리즘**: 샘플 크기를 동적으로 조정하여 필요한 만큼만 샘플링 2. **점진적 신뢰도 검증**: 샘플링 과정 중 신뢰도를 평가하여 언제 멈출지 결정 3. **실용성**: 데이터베이스 옵티마이저에 직접 통합 가능한 형태

논문의 핵심은 "적응적"이라는 점이다. 사전에  $\sigma$ 를 모를 때, 초기 샘플로  $\sigma$ 를 대략 추정한 후, 이 추정값에 따라 필요한 추가 샘플 크기를 결정한다. 이를 통해 샘플링 비용을 크게 절감할 수 있다.

본 논문과의 관계: **Exqutor 논문이 직접 "적응적 샘플링"을 수용하는 방식의 이론적 원천**. Exqutor의 핵심 메커니즘인 "범위 필터 조건과 벡터 검색 조건의 결합 선택도를 적응적으로 샘플링으로 추정"은 이 논문의 방법론을 직접 구현한 것이다.

---

### 상세분석

#### 81.1 문제의 현실성

**샘플 크기 결정의 딜레마**:

공식 (논문 [80]):  $n = z^2 \times \sigma(1-\sigma) / \epsilon^2$

문제:  $\sigma$ 를 모른다!

선택지 1: 최악 경우 가정 ( $\sigma = 0.5$ )

- $n_{\max} = z^2 \times 0.25 / \epsilon^2$
- 매우 큰 샘플 크기 (보수적)
- 시간/비용 낭비

선택지 2: 추측 ( $\sigma \approx 0.1?$ )

- $n = z^2 \times 0.09 / \epsilon^2$
- 작은 샘플
- 신뢰도 부족 위험

해결: 적응적 샘플링

- 초기 샘플로  $\sigma$  추정
- 이 추정값에 따라 추가 샘플링 여부 결정

## 81.2 적응적 샘플링 알고리즘

기본 알고리즘:

```

def adaptive_sampling_estimate(query, target_precision=0.15,
 confidence_level=0.90):
 """
 적응적 샘플링으로 선택도 추정

 목표: σ 를 $\pm \text{target_precision} \times \sigma$ 범위에서 90% 신뢰도로 추정
 """

 # 1단계: 초기 샘플
 sample_size = 100 # 초기값
 sample = data.sample(n=sample_size)
 results = query.execute(sample)
 match_count = count_matches(results)

 # 2단계: 초기 추정
 sigma_hat = match_count / sample_size

 # 3단계: 신뢰도 검증
 while True:
 # 현재 샘플로 달성 가능한 신뢰도 계산
 se = math.sqrt(sigma_hat * (1 - sigma_hat) / sample_size)
 margin_of_error = z_score * se # z_score = 1.645 (90%)
 relative_error = margin_of_error / sigma_hat

 # 4단계: 목표 정확도 달성 여부 확인
 if relative_error <= target_precision:
 break # 충분한 신뢰도 달성

 # 5단계: 필요한 총 샘플 크기 계산
 required_sample = int((z_score / target_precision) ** 2 *
 sigma_hat * (1 - sigma_hat))
 additional_sample = required_sample - sample_size

 # 6단계: 추가 샘플링
 if additional_sample <= 0:
 break # 최적화: 이미 충분함

 new_sample = data.sample(n=additional_sample)
 new_results = query.execute(new_sample)
 match_count += count_matches(new_results)
 sample_size += additional_sample

 # σ 재추정
 sigma_hat = match_count / sample_size

 # 7단계: 안전 장치 (무한 루프 방지)
 if sample_size > len(data) * 0.5: # 50% 이상 샘플링
 break

```

```
return sigma_hat, sample_size
```

### 알고리즘의 유연성:

초기 샘플에서  $\sigma_{\text{hat}}$  추정:

$\sigma_{\text{hat}} = 0.5$  (50%):

- 매우 큰 샘플 필요 (최악의 경우)
- 추가 샘플링 수행

$\sigma_{\text{hat}} = 0.1$  (10%):

- 중간 크기 샘플 필요
- 일부 추가 샘플링 가능

$\sigma_{\text{hat}} = 0.01$  (1%):

- 큰 샘플 필요? 하지만 이미 100개 중 1개면...
- 신뢰도 저하 → 큰 샘플링 필요

$\sigma_{\text{hat}} = 0.001$  (0.1%):

- 극도로 큰 샘플 필요
- 또는 샘플링 포기, 다른 방법 사용

## 81.3 동적 정밀도 조정

### 점진적 신뢰도 평가:

반복  $i$ 에서:

샘플 크기:  $n_i$

추정된  $\sigma$ :  $\sigma_{\text{hat}_i}$

표준오차:  $SE_i = \sqrt{(\sigma_{\text{hat}_i} \times (1 - \sigma_{\text{hat}_i}) / n_i)}$

상대오차:  $RE_i = 1.645 \times SE_i / \sigma_{\text{hat}_i}$  (90% 신뢰도)

반복별 진행:

반복 1:  $n=100$ ,  $\sigma_{\text{hat}}=0.08$ ,  $SE=0.027$ ,  $RE=44\%$  (매우 높음)

→ 추가 샘플링 필요

반복 2:  $n=500$ ,  $\sigma_{\text{hat}}=0.12$ ,  $SE=0.012$ ,  $RE=14\%$  (목표 15% 달성!)

→ 중단 가능

## 81.4 특수한 경우들

### 매우 낮은 선택도 처리:

$\sigma_{\text{hat}} = 0.001$  (0.1%)인 경우:

$$\begin{aligned}\text{공식: } n &= 1.645^2 \times 0.001 \times 0.999 / (0.15 \times 0.001)^2 \\ n &= 2.706 \times 0.000999 / 0.000000225 \\ n &\approx 120,000\end{aligned}$$

원래 데이터가 1,000,000행이면:

$$\text{샘플율} = 120,000 / 1,000,000 = 12\%$$

대체 전략:

1. 정밀도 요구사항 완화 (RE = 50% 허용)  
→ 필요 샘플: 4,800 (0.5%)
2. 신뢰도 낮춤 (80% 신뢰도로)  
→ 필요 샘플: 68,000 (6.8%)
3. 샘플링 포기 → 통계 기반 추정 전환

### 제로 매칭(Zero Matches):

샘플 100개 중 0개가 조건 만족:

$$\sigma_{\text{hat}} = 0$$

문제:  $SE = \sqrt{(0 \times 1 / 100)} = 0$  (undefined)

해결책:

1. 라플라스 평활(Laplace Smoothing):  $\sigma_{\text{hat}} = 1 / (n+2)$   
→ 100개 샘플 시  $\sigma_{\text{hat}} = 1/102 \approx 0.98\%$
2. 추가 샘플링 강제 실행  
→ 더 큰 샘플에서 일치 찾을 때까지

### 81.5 멀티쿼리 최적화

여러 쿼리를 동시에 추정할 때:

쿼리 Q1: WHERE condition1

쿼리 Q2: WHERE condition2

방법 1: 독립적 샘플링

각 쿼리에 대해 별도로 샘플링

총 샘플 비용: 큼 (각각 독립적)

방법 2: 공유 샘플 (Shared Sampling)

하나의 샘플 S에서 모든 쿼리 실행

샘플 S 선택:

$Q1\_matches = |\{r \in S : condition1(r)\}|$

$Q2\_matches = |\{r \in S : condition2(r)\}|$

$\sigma\_hat\_1 = Q1\_matches / |S|$

$\sigma\_hat\_2 = Q2\_matches / |S|$

장점: 샘플 한 번만 수집

단점: 개별 쿼리 정밀도 조정 어려움

## 81.6 비용 모델과의 통합

샘플링 비용 vs 추정 정확도:

총 비용 = 샘플링 비용 + 부정확한 추정으로 인한 손해

샘플링 비용:

$C\_sample = \text{샘플 크기} \times \text{행당\_처리\_시간}$

부정확 비용:

$C\_error = |\sigma\_hat - \sigma| \times \text{최악\_실행\_계획\_비용}$

최적 샘플 크기:  $C\_sample + C\_error$  최소화

예:

정밀도  $\pm 50\%$  오류 허용:

- 샘플 100개 (빠름)
- 부정확하면 최악 계획 선택 가능성

정밀도  $\pm 10\%$  오류 요구:

- 샘플 2,500개 (느림)
- 정확한 계획 선택으로 큰 이득

## 81.7 본 논문과의 관계

Exqutor의 적응적 샘플링의 원천:

Exqutor 논문의 "적응적 샘플링" 방식이 정확히 이 논문([81])의 알고리즘을 구현한 것이다:

#### Exqutor 아키텍처:

범위 필터 + 벡터 검색의 결합 선택도 추정

쿼리: WHERE price  $\in$  [100, 500] AND similarity(embedding) > 0.8

1단계: 초기 샘플 (전체의 1%, 약 10,000행)

```
S = data.sample(frac=0.01)
```

2단계: 샘플에서 두 조건 모두 만족하는 행 수 세기

```
matches = count(S, price_condition AND vector_condition)
```

3단계: 초기 선택도 추정

```
 $\sigma_{\text{hat}} = \text{matches} / \text{len}(S)$
```

4단계: 신뢰도 계산 (논문 [80]의 공식)

```
se = $\sqrt{(\sigma_{\text{hat}} \times (1 - \sigma_{\text{hat}})) / \text{len}(S)}$
```

```
confidence = 1.645 $\times$ se / σ_{hat} (90% 신뢰도)
```

5단계: 목표 정밀도(예:  $\pm 15\%$ ) 달성 여부 확인

```
if confidence $\leq$ 0.15:
```

```
 → 충분한 샘플 확보, 사용 가능
```

```
else:
```

```
 → 추가 샘플링 필요
```

6단계: 추가 샘플링 (필요한 경우만)

```
required_total = $(1.645 / 0.15)^2 \times \sigma_{\text{hat}} \times (1 - \sigma_{\text{hat}})$
```

```
additional = required_total - len(S)
```

```
if additional > 0:
```

```
 S_new = data.sample(frac=additional/N)
```

```
 matches_new = count(S_new, both_conditions)
```

```
 $\sigma_{\text{hat_final}} = (\text{matches} + \text{matches_new}) / (\text{len}(S) + \text{len}(S_{\text{new}}))$
```

7단계: 최종 선택도 사용

옵티마이저가  $\sigma_{\text{hat\_final}}$ 을 기반으로 실행 계획 선택

#### Exqutor의 창의성:

기준 (논문 [81]):

- 단일 쿼리의 선택도 추정

Exqutor의 확장:

- 두 이질적 조건(범위 필터 + 벡터 검색)의 결합 선택도
- $\sigma_{total} = \sigma_{range} \times \sigma_{vector}$  (독립성 가정)
- 적응적 샘플링으로 양쪽 조건을 동시에 평가

## 81.8 현대적 적응 사례

Exqutor의 다층 구조:

L1: 경량 모델 (논문 [70])

범위 필터의 선택도를 빠르게 추정

비용: <1ms

정확도:  $\pm 20\%$

L2: 적응적 샘플링 (논문 [81])

L1의 추정이 불확실할 때 사용

비용: 10-100ms (샘플 크기에 따라)

정확도:  $\pm 10\%$

L3: 전체 스캔 (정확한 계산)

선택도가 매우 중요한 경우만

비용: 1-10초 (전체 데이터 스캔)

정확도: 100%

선택 로직:

```
if L1_confidence > 0.95:
```

```
 사용 L1 (빠름)
```

```
elif time_budget > 100ms:
```

```
 사용 L2 (적응적 샘플링)
```

```
else:
```

```
 사용 L1 (신뢰도 낮음)
```

## 추가 제기 문제

1. 초기 샘플 크기: 초기값 100개가 항상 적절한가? 데이터 특성에 따라 조정할 수 있는가?
2. 반복 중단 조건: 신뢰도 기준 외에 다른 중단 조건이 필요한가? (시간, 메모리, 비용 등)
3. 샘플 편향: 데이터가 비균등하게 분포되어 있을 때(예: 클러스터링), 무작위 샘플링이 정확한가?
4. 벡터 유사도의 선택도: 벡터 검색에서 "top-k"는 선택도가 정의되기 어려운데(k는 고정, 유사도 임계값은 데이터 의존적), 어떻게 샘플링할 것인가?



5. **상관성이 있는 경우:** 범위 필터 속성과 벡터 임베딩이 강하게 상관되어 있을 때,  $\sigma_{total} = \sigma_{range} \times \sigma_{vector}$  가정이 성립하는가?
  6. **실시간 적응성:** 데이터가 시간에 따라 변할 때, 과거 샘플이 유효한가? 주기적 재샘플링이 필요한가?
  7. **멀티 필터 확장:** (range\_filter1 AND range\_filter2 AND vector\_search)에서 3중 곱셈  $\sigma_{total} = \sigma_1 \times \sigma_2 \times \sigma_3$ 가 성립하는가?
- 

## 논문 [81]의 역사적 의의

이 논문은 1990년에 발표되었지만, 30년이 지난 지금도:

1. 학부 데이터베이스 강의에서 샘플링 기반 카디널리티 추정의 기초로 가르쳐짐
2. PostgreSQL, SQL Server 등 상용 DBMS에서 기본 전략으로 채택
3. 현대 벡터 DB와 하이브리드 검색 시스템의 선택도 추정 기법으로 여전히 핵심

Exqutor가 "적응적 샘플링"을 사용하기로 한 것은, 단순한 구현 선택이 아니라, 30년의 검증을 거친 이론적으로 견고한 방법론을 채택한 것이다. 이는 Exqutor의 설계 철학의 깊이를 보여준다.

---

## (L) 벡터 검색 응용 & 데이터